



计算物理学（2026 春）笔记

作者：雨落潇潇

组织：北京大学物理学院

时间：June 10, 2026

版本：0.1

目录

第一章 计算物理引言与基础	1
1.1 课程内容、教学目标与学科定位	1
1.1.1 课程内容与学习目标	1
1.1.2 计算物理的学科定位	1
1.2 复杂性、多尺度与自由度层级	2
1.2.1 为什么现代物理离不开计算	2
1.2.2 多尺度 (Multi-scale) 视角	2
1.2.3 多体问题与自由度 (Degrees of Freedom)	3
1.2.4 交叉学科与 SciDAC 理念	3
1.3 超级计算、计算基础设施与学科前沿	3
1.3.1 超级计算的重要性	3
1.3.2 新型计算基础设施：任务化与异步并行	4
1.3.3 计算机发展与物理发展的关系	4
1.3.4 计算物理的新前沿	4
1.4 计算基础、浮点数与数值表示	5
1.4.1 数值计算的基本特点	5
1.4.2 单精度、双精度与内存估算	5
1.4.3 IEEE 754 浮点表示	5
1.5 程序结构、编程语言与系统基础	6
1.5.1 程序结构与伪代码	6
1.5.2 科学计算中的主要编程语言	6
1.5.3 操作系统、编译与库链接	7
1.6 计算性能、存储模型与内存访问	7
1.6.1 计算中要平衡的三件事	7
1.6.2 行优先、列优先与缓存命中	8
1.6.3 栈、堆与动态内存	9
1.7 计算误差、稳定性与算法观	9
1.7.1 误差的来源与度量	9
1.7.2 误差放大与算法稳定性	9
1.7.3 相近数相减的经典例子	9
1.7.4 减少误差的基本原则	10
1.7.5 20 世纪十大算法与 21 世纪算法趋势	10
1.8 课程作业与考核安排	11
1.8.1 作业主题与考核方式	11
1.8.2 报告格式要求	11
第二章 线性代数问题 (Linear Algebra Problems)	12
2.1 引言与基础知识	12
2.2 Gauss 消元法与主元法 (Gaussian Elimination & Pivoting)	12
2.2.1 标准 Gauss 消元法	12
2.2.2 列主元消元法 (Partial Pivoting)	13
2.3 LU 分解 (LU Decomposition)	13

2.3.1 基本原理与 Doolittle 算法	13
2.3.2 高斯消元与 LU 分解的关系	14
2.3.3 列主元 LU 分解 ($PA = LU$)	14
2.3.4 利用 LU 分解求逆矩阵与行列式	15
2.3.5 特殊矩阵分解	15
2.3.5.1 LDL^T 分解	15
2.3.5.2 Cholesky 分解	16
2.3.5.3 三对角矩阵与 Thomas 算法	16
2.4 迭代求解法 (Iterative Methods)	17
2.4.1 从直接迭代到矩阵分裂	18
2.4.2 Jacobi 迭代 (Jacobi Iteration)	19
2.4.2.1 由分量方程出发的推导	19
2.4.2.2 矩阵形式	19
2.4.2.3 误差传播与收敛条件	20
2.4.2.4 终止准则	20
2.4.3 Gauss-Seidel 迭代 (Gauss-Seidel Iteration)	21
2.4.3.1 基本思想: 立即使用新值	21
2.4.3.2 矩阵形式	21
2.4.3.3 收敛性质	21
2.4.4 SOR 迭代 (Successive Over-Relaxation)	22
2.4.4.1 引入松弛参数的动机	22
2.4.4.2 参数 ω 的含义	22
2.4.4.3 矩阵形式	22
2.4.4.4 收敛性与参数选择	22
2.4.5 三种方法的统一比较	23
2.4.5.1 Jacobi	23
2.4.5.2 Gauss-Seidel	23
2.4.5.3 SOR	23
2.4.6 一个简单例子	23
2.4.6.1 Jacobi 迭代	24
2.4.6.2 Gauss-Seidel 迭代	24
2.4.7 实现时的几个注意事项	24
2.5 共轭梯度法 (Conjugate Gradient Method)	25
2.5.1 从线性方程组到极小化问题	25
2.5.2 沿给定方向的一维极小化	26
2.5.3 残差向量与最速下降法	26
2.5.4 为什么需要“共轭方向”	27
2.5.5 共轭梯度法的基本递推关系	28
2.5.5.1 方向递推的想法	28
2.5.5.2 步长公式	28
2.5.5.3 正交性简化	29
2.5.6 共轭梯度算法的标准形式	29
2.5.7 几何解释与 Krylov 子空间	30
2.5.8 收敛性与条件数的影响	31
2.5.9 一个 2×2 的具体例子	31

2.5.10 与前面几种迭代法的比较	32
2.5.11 预条件共轭梯度法 (Preconditioned Conjugate Gradient, PCG)	33
2.5.12 CG 的适用范围与推广	34
2.6 QR 分解 (QR Decomposition)	34
2.6.1 基本定义与几何意义	34
2.6.2 为什么 QR 分解有用	34
2.6.3 QR 分解与 LU 分解的比较	35
2.6.4 QR 分解的列向量形式	35
2.6.5 用 Gram-Schmidt 过程构造 QR 分解	36
2.6.5.1 第一步	36
2.6.5.2 第二步	36
2.6.5.3 一般步骤	36
2.6.6 由 Gram-Schmidt 得到 Q 与 R	37
2.6.7 投影 (Projection) 的理解	37
2.6.8 例子: 对给定矩阵做 QR 分解	38
2.6.9 正交性检验	40
2.6.10 QR 分解如何用来解方程组	40
2.6.11 QR 分解与最小二乘问题	41
2.6.12 Householder 变换的补充说明	41
2.7 $M \times N$ 矩阵、SVD 与线性最小二乘	41
2.7.1 一般 $M \times N$ 线性系统的三种基本情形	42
2.7.2 奇异问题与病态问题	42
2.7.3 奇异值分解 (SVD) 的定义	43
2.7.4 奇异值、秩与退化方向	44
2.7.5 SVD 与广义逆 (pseudo-inverse)	44
2.7.6 超定方程组 $M > N$: 线性最小二乘问题	45
2.7.6.1 正规方程 (normal equations)	45
2.7.6.2 用 SVD 求最小二乘解	45
2.7.7 欠定方程组 $M < N$: 无穷多解与最小范数解	46
2.7.8 秩、零空间与奇异值之间的关系	46
2.7.9 截断 SVD 与压缩 (compression)	47
2.7.10 线性最小二乘拟合中的 SVD	47
2.7.11 为什么 SVD 比普通消元更稳健	48
2.8 范数、条件数与稀疏线性方程	48
2.8.1 向量范数 (Vector Norm)	49
2.8.1.1 1-范数	49
2.8.1.2 2-范数 (Euclidean norm)	49
2.8.1.3 ∞ -范数 (maximum norm)	49
2.8.2 Cauchy-Schwarz 不等式与三角不等式	50
2.8.3 矩阵范数 (Matrix Norm)	50
2.8.3.1 ∞ -矩阵范数	50
2.8.3.2 Frobenius 范数	51
2.8.4 条件数 (Condition Number)	51
2.8.5 残差、误差与条件数的关系	51
2.8.6 一个病态矩阵的例子	52

2.8.7 2-范数条件数与奇异值	52
2.8.8 预条件化 (Preconditioning)	53
2.9 稀疏线性方程 (Sparse Linear Equations)	53
2.9.1 直接法与特殊结构	54
2.9.2 大型稀疏系统中的迭代法	54
2.9.3 Sherman–Morrison 公式: 秩一修正 (Rank-One Update)	54
2.9.3.1 用辅助变量重写公式	55
2.9.4 用 Sherman–Morrison 公式解修正后的线性系统	55
2.9.5 Woodbury 矩阵恒等式 (Woodbury Matrix Identity)	56
2.9.6 Sherman–Morrison 与 Woodbury 公式的应用	56
2.10 软件与程序库 (LAPACK / BLAS)	56
2.11 全章总结	56
第三章 数值微分、插值与积分方法	58
3.1 本章导读	58
3.2 拉格朗日插值 (Lagrange Interpolation)	58
3.2.1 基本思想	58
3.2.2 两点插值与三点插值	59
3.2.2.1 两点直线插值	59
3.2.2.2 三点二次插值	59
3.2.3 插值基函数的图像理解	60
3.2.4 插值余项 (remainder term) 与误差估计	60
3.2.4.1 一个误差估计例子	61
3.3 Hermite 插值 (Hermite Interpolation)	62
3.3.1 为什么需要 Hermite 插值	62
3.3.2 Hermite 插值公式	62
3.3.3 Hermite 插值的余项	62
3.4 高阶插值的不稳定性与龙格现象 (Runge Phenomenon)	63
3.4.1 龙格现象	63
3.4.2 为什么会出现这种现象	63
3.5 三次样条插值 (Cubic Spline Interpolation)	64
3.5.1 基本思想	64
3.5.2 插值条件与光滑连接条件	64
3.5.2.1 插值条件	64
3.5.2.2 光滑连接条件	64
3.5.3 常见边界条件	64
3.5.3.1 自然边界条件 (natural boundary conditions)	64
3.5.3.2 一阶导数边界条件 (clamped boundary conditions)	65
3.5.4 三点样条例子	65
3.5.5 一般系数关系的推导	65
3.5.6 消去 b_j 后的三对角方程组	66
3.5.7 自然样条的矩阵形式	66
3.5.8 给定端点一阶导数时的方程组	67
3.5.9 若给定的是二阶导数边界条件	67
3.5.10 非均匀格点 (nonuniform grids)	67

3.6 Akima 插值 (Akima Interpolation)	68
3.6.1 基本思想	68
3.6.2 区间上的三次表示	68
3.6.3 特殊情况与边界处理	68
3.6.4 由端点函数值与导数确定区间三次多项式	69
3.7 多维插值 (Multidimensional Interpolation)	69
3.7.1 双线性插值 (bilinear interpolation)	69
3.7.2 高维情形的递推思想	70
3.7.3 其他多维插值方法	70
3.8 分式插值与函数逼近 (Rational Interpolation and Approximation)	70
3.8.1 分式多项式插值 (Rational Function Interpolation)	70
3.8.2 Padé 近似 (Padé approximation)	70
3.8.3 函数逼近的几种常见方式	71
3.9 数值微分 (Numerical Differentiation)	71
3.9.1 最基本的前向差分 (Forward Difference)	71
3.9.2 从 Lagrange 插值出发的一般 n 点微分公式	72
3.9.3 三点微分公式 (Three-Point Formula)	72
3.9.4 五点微分公式 (Five-Point Formula)	73
3.9.5 二阶导数公式 (Second Derivative Formula)	74
3.9.6 精度、误差与格式选择	75
3.9.7 其他数值微分方法	75
3.10 数值积分 (Numerical Integration)	76
3.10.1 由插值公式导出一般求积公式	76
3.10.2 Simpson 公式的推导	77
3.10.3 牛顿-柯特斯公式 (Newton-Cotes Formula)	77
3.10.3.1 一般形式	77
3.10.3.2 柯特斯系数的积分表示	78
3.10.3.3 柯特斯系数的基本性质	78
3.10.3.4 前几阶柯特斯系数表	79
3.10.4 常用低阶牛顿-柯特斯公式	79
3.10.4.1 梯形公式 (Trapezoidal Rule)	79
3.10.4.2 Simpson 公式	79
3.10.4.3 五点柯特斯公式	80
3.10.5 为什么积分通常比微分更精确	80
3.10.6 复合积分公式 (Composite Quadrature Formula)	80
3.10.6.1 复合 Simpson 公式	81
3.11 余项外推 (Richardson Extrapolation) 与 Romberg 积分	81
3.11.1 Richardson 外推的基本思想	82
3.11.2 只含偶数阶误差时的 Richardson 外推	82
3.11.3 统一形式	83
3.11.4 Richardson 外推在数值微分中的应用	83
3.11.5 余项外推在数值积分中的应用	84
3.11.6 Romberg 表的结构	84
3.11.7 为什么 Romberg 积分有效	85
3.12 变步长积分与龙贝格算法 (Adaptive Integration and Romberg Acceleration)	85

3.12.1 为什么要做变步长积分	85
3.12.2 以复化梯形公式为例	86
3.12.3 误差估计与判敛条件	86
3.12.4 龙贝格算法：对复化梯形法做加速	86
3.12.5 对复化 Simpson 法的加速	86
3.12.6 对复化柯特斯法的加速	87
3.13 高斯积分 (Gaussian Quadrature)	87
3.13.1 高斯积分的基本定义	87
3.13.2 高斯点的充要条件	88
3.13.3 Gauss–Legendre 积分公式	88
3.13.3.1 勒让德多项式的定义与性质	88
3.13.3.2 为什么勒让德多项式的零点是高斯点	89
3.13.4 N 点 Gauss–Legendre 公式具有 $2N - 1$ 次精度	89
3.13.5 高斯权重的确定	90
3.13.6 二点 Gauss–Legendre 公式的例子	90
3.13.7 积分区间变换	91
3.13.8 前几阶 Gauss–Legendre 节点与权重	91
3.13.9 带权高斯积分的一般形式	92
3.13.10 几类常用的带权高斯积分	93
3.13.10.1 Gauss–Legendre 公式	93
3.13.10.2 Gauss–Laguerre 公式	93
3.13.10.3 Gauss–Hermite 公式	93
3.13.10.4 Gauss–Chebyshev 公式	94
3.13.11 无穷区间与区间变换	94
3.13.12 高斯积分的优势与局限	94
3.14 WKB 数值计算示例：量子穿透几率与寿命	95
3.14.1 问题背景	95
3.14.2 公式	95
3.14.3 数值实现思路	96
3.14.4 附表： $V(q)$ 与 $M(q)$ 的离散数据	96
3.14.4.1 势能表 $V(q)$	96
3.14.4.2 质量参数表 $M(q)$	97
3.15 本章总结 (Summary)	97

第四章 优化与非线性方程 98

4.1 非线性方程求根	98
4.1.1 从物理问题到求根问题	98
4.1.2 二分法 (Bisection Method)	99
4.1.3 不动点法 (Fixed-Point Iteration)	100
4.1.4 不动点法的具体例子	100
4.1.5 牛顿法 (Newton Method)	101
4.1.6 割线法 (Secant Method)	102
4.1.7 多重根问题与修正牛顿法 (Modified Newton Method)	102
4.1.8 Aitken 的 Δ^2 加速与 Steffensen 方法	103
4.1.9 代数多项式与 Horner (秦九韶) 算法	104

4.1.10 Brent 方法 (Van Wijngaarden–Dekker–Brent Method)	105
4.2 非线性方程组	105
4.2.1 不动点迭代法	106
4.2.1.1 基本思想	106
4.2.1.2 一个三元非线性方程组示例	106
4.2.1.3 加速的不动点迭代	107
4.2.2 牛顿法	107
4.2.2.1 由单变量牛顿法到多变量牛顿法	107
4.2.2.2 牛顿法示例	108
4.2.3 准牛顿法	109
4.2.3.1 基本想法	109
4.2.3.2 一般迭代格式	110
4.2.3.3 Sherman–Morrison 公式与逆矩阵更新	110
4.2.4 Broyden 方法	111
4.2.4.1 基本公式	111
4.2.4.2 Broyden 方法示例	111
4.2.5 改进的 Broyden 方法	113
4.2.5.1 从自洽迭代出发	113
4.2.5.2 标准 Broyden 形式	113
4.2.5.3 改进的 Broyden 方法	114
4.3 优化问题	115
4.3.1 基本概念与 Hessian 判据	115
4.3.1.1 梯度与 Hessian 矩阵	115
4.3.1.2 二阶 Taylor 展开与驻点判别	116
4.3.2 全局优化方法概览	117
4.3.3 梯度法优化	118
4.3.3.1 基本下降思想	118
4.3.3.2 Fletcher–Reeves 非线性共轭梯度法	118
4.3.3.3 线性共轭梯度法回顾	118
4.3.3.4 另一套等价记号	119
4.3.4 拟牛顿法优化	120
4.3.4.1 一维牛顿法与极值问题	120
4.3.4.2 多维牛顿法与 Hessian 线性系统	120
4.3.4.3 拟牛顿法的基本思想	120
4.3.4.4 割线条件 (secant equation)	121
4.3.5 拟牛顿法的迭代过程与常见更新公式	121
4.3.5.1 一般框架	121
4.3.5.2 DFP 方法	122
4.3.5.3 BFGS 方法	122
4.3.5.4 Broyden 更新	122
4.3.5.5 Broyden family	123
4.3.5.6 SR1 更新	123
4.3.5.7 几种拟牛顿法的比较	123
4.3.6 BFGS 算法	123
4.3.6.1 基本设定	124

4.3.6.2 BFGS 更新公式的推导	124
4.3.6.3 逆矩阵形式	125
4.3.6.4 BFGS 的标准算法步骤	125
4.3.6.5 二次函数上的 BFGS 例子	126
4.3.7 Broyden 优化	127
4.3.7.1 从非线性方程到非线性优化	127
4.3.7.2 Broyden 型 Hessian 更新	128
4.3.7.3 与 BFGS 的比较	128
4.3.8 线性规划与单纯形法	129
4.3.8.1 线性规划的标准形式	129
4.3.8.2 单纯形法的基本思想	129
4.3.8.3 例：用单纯形法求线性规划极大值	129
4.3.9 Nelder-Mead 方法（下山单纯形法）	132
4.3.9.1 几何上的 simplex	132
4.3.9.2 算法基本思想	132
4.3.9.3 反射操作	132
4.3.9.4 膨胀操作	133
4.3.9.5 收缩操作	133
4.3.9.6 缩小操作	133
4.3.9.7 Nelder-Mead 的完整流程	133
4.3.9.8 二维情形的几何图像	134
4.3.10 模拟退火法	135
4.3.10.1 方法背景与基本思想	135
4.3.10.2 热力学类比	135
4.3.11 Metropolis 过程与基本步骤	136
4.3.11.1 基本原理	136
4.3.11.2 接受概率的性质	136
4.3.11.3 一次完整退火过程的结构	137
4.3.12 模拟退火法的伪代码结构	137
4.3.13 关键参数与调控原则	138
4.3.13.1 初始温度的控制	138
4.3.13.2 降温方法的控制	138
4.3.13.3 每一温度下的迭代长度控制	138
4.3.13.4 终止准则	139
4.3.14 模拟退火法解决旅行者问题	139
4.3.14.1 问题背景	139
4.3.14.2 状态、目标函数与邻域	139
4.3.14.3 旅行商问题上的退火流程	140
4.3.14.4 图像演化的物理含义	140
4.3.15 模拟退火法的算法特征与适用场景	141
4.3.15.1 优点	141
4.3.15.2 缺点	141
4.3.15.3 与其他优化方法的比较	141
4.3.16 遗传算法	142
4.3.16.1 基本思想	142

4.3.16.2 优化问题与生物学过程的对应	142
4.3.17 遗传算法的基本组成	143
4.3.17.1 编码 (encoding)	143
4.3.17.2 种群 (population)	143
4.3.17.3 适应度函数 (fitness function)	143
4.3.17.4 选择 (selection)	144
4.3.17.5 交叉 (crossover)	144
4.3.17.6 突变 (mutation)	144
4.3.18 遗传算法的完整流程	145
4.3.18.1 初始种群	145
4.3.18.2 适应度评估	145
4.3.18.3 选择、交叉、突变	145
4.3.18.4 终止判断	145
4.3.18.5 算法伪代码	145
4.3.19 遗传算法的典型行为	146
4.3.20 遗传算法的优点	146
4.3.20.1 较强的全局搜索能力	146
4.3.20.2 适合处理复杂问题	146
4.3.20.3 并行性好	147
4.3.21 遗传算法的局限性	147
4.3.21.1 对具体设计非常敏感	147
4.3.21.2 可能出现过早趋同	147
4.3.22 几个重要实现细节	147
4.3.22.1 最大化与最小化的统一处理	147
4.3.22.2 精英保留	148
4.3.22.3 参数选择	148
第五章 本征值问题 (Eigenvalue Problems)	150
5.1 本征值问题的物理背景	150
5.1.1 本征值、可观测量与 Hilbert 空间	150
5.1.2 从独立粒子图像到 Fock 空间展开	150
5.1.3 为什么量子多体问题会变成巨型对角化问题	151
5.1.4 特征多项式视角与数值求解的现实困难	151
5.2 相似变换、谱不变量与 Jacobi 方法	152
5.2.1 相似变换有相同的本征值	152
5.2.2 实对称矩阵的正交对角化	152
5.2.3 Jacobi 旋转的单步思想	153
5.2.4 Jacobi 方法的完整迭代	154
5.2.5 Jacobi 方法的并行化思路	155
5.3 Householder 变换与矩阵约化	155
5.3.1 Householder 反射的定义与性质	155
5.3.2 把向量反射到坐标轴方向	156
5.3.3 Householder 相似变换如何约化矩阵	156
5.3.4 从一般矩阵到上 Hessenberg, 从对称矩阵到三对角	157
5.3.5 例: 用 Householder 变换将对称矩阵化为三对角矩阵。	158

5.3.6 三对角矩阵后的后续处理	160
5.4 QR 算法、位移与 Schur 分解	160
5.4.1 QR 算法的基本形式	160
5.4.2 对 Hessenberg / 三对角矩阵做 Givens 型 QR 分解	161
5.4.3 例: QR 算法的一次迭代。	161
5.4.4 用 Householder 反射做 QR 分解	163
5.4.5 例: 用 Householder 变换对矩阵作 QR 分解。	164
5.4.6 位移 QR (Shifted QR)	166
5.4.7 Schur 分解	167
5.5 变换后的本征值问题	167
5.5.1 广义本征值问题的标准化	167
5.5.2 Hermitian 问题化为实对称问题	168
5.6 奇异值分解 (Singular Value Decomposition)	169
5.6.1 SVD 的定义与基本性质	169
5.6.2 SVD 与 $A^T A$ 的关系	169
5.6.3 如何由特征分解构造 SVD	170
5.6.4 例: 由特征分解构造 SVD。	170
5.6.5 SVD 的几何意义与图像压缩	172
5.7 幂法、反幂法与 Wielandt 缩小法	173
5.7.1 幂法 (Power Method)	173
5.7.2 例: 用幂法求主特征值与对应特征向量。	173
5.7.3 对称矩阵情形与收敛直观	175
5.7.4 例: 对称矩阵的幂法。	175
5.7.5 反幂法 (Inverse Power Method) 与位移幂法	177
5.7.6 Wielandt 缩小法 (deflation)	177
5.7.7 例: 已知一个本征对后的降阶求解。	178
5.8 Krylov 子空间、Lanczos 与 Arnoldi	180
5.8.1 Krylov 子空间的定义	180
5.8.2 Lanczos 迭代: 对称矩阵的三项递推	180
5.8.3 Lanczos 为什么适合大规模稀疏问题	181
5.8.4 Lanczos 两步例子	181
5.8.5 Arnoldi 迭代: 非对称矩阵的 Krylov 正交化	182
5.8.6 例: Arnoldi 迭代的数值计算。	182
5.9 直接法与迭代法的比较	186
5.9.1 直接法: 适合中等规模稠密问题	186
5.9.2 迭代法: 适合超大规模稀疏问题	186
5.10 重整化思想: NRG 与 DMRG	186
5.10.1 为什么精确对角化会失效	186
5.10.2 Wilson 的数值重整化群 (Numerical Renormalization Group, NRG)	186
5.10.3 密度矩阵重整化群 (DMRG) 的基本思想	187
5.10.4 DMRG 中的 warm-up 与 sweeping	187
5.10.5 DMRG 的应用范围	188
5.11 本章总结	188

6.1 Monte Carlo 方法的基本思想	189
6.1.1 从统计抽样到数值计算	189
6.1.2 历史背景与典型应用	189
6.2 概率统计基础	190
6.2.1 PDF、CDF 与期望值	190
6.2.2 中心极限定理	190
6.3 随机数与概率分布采样	191
6.3.1 伪随机数生成器	191
6.3.2 正态分布与 Box-Muller 方法	191
6.4 Monte Carlo 积分	192
6.4.1 空间随机投点法	192
6.4.2 求期望值法	192
6.4.3 任意区间与多维积分	193
6.4.4 量子多体体系中的积分	193
6.5 重要性采样与 Bootstrap	193
6.5.1 为什么需要改进采样	193
6.5.2 重要性采样公式	193
6.5.3 变量变换实现非均匀采样	194
6.5.4 具体积分例子	194
6.5.5 Bootstrap 误差估计	195
6.6 随机行走与 Markov 过程	196
6.6.1 一维随机行走	196
6.6.2 Markov 随机过程	196
6.6.3 转移矩阵与二项式表达	197
6.6.4 从随机行走走到扩散方程	197
6.6.5 有限转移矩阵例子	198
6.7 Metropolis、Metropolis-Hastings 与 MCMC	198
6.7.1 Metropolis 行走的动机	198
6.7.2 接受率与 Master 方程	199
6.7.3 Boltzmann 分布与 Metropolis 接受率	200
6.7.4 Metropolis-Hastings 算法	200
6.7.5 MCMC 积分	201
6.8 Hamiltonian Monte Carlo	201
6.8.1 引入动量变量	201
6.8.2 Hamilton 方程与接受率	201
6.9 统计物理中的 Monte Carlo: 二维 Ising 模型	202
6.9.1 二维 Ising 模型	202
6.9.2 单自旋翻转的能量差	202
6.9.3 二维 Ising 的 Metropolis 算法	203
6.9.4 相变与有限尺寸效应	203
6.9.5 Kosterlitz-Thouless 转变	203
6.10 量子 Monte Carlo 方法	204
6.10.1 变分量子 Monte Carlo 的基本流程	204
6.10.2 用 Metropolis 抽样 $ \Psi_T ^2$	204
6.10.3 一维谐振子的 VMC 例子	205

6.10.4 VMC 计算中的误差	206
6.11 Diffusion Monte Carlo	206
6.11.1 虚时间演化与 Green 函数	206
6.11.2 转移概率与 sign problem	206
6.12 本章总结	207
第七章 微分方程 (Differential Equations)	208
7.1 初值问题与 Euler 法	208
7.1.1 一阶常微分方程初值问题	208
7.1.2 高阶 Taylor 展开法	209
7.2 Runge-Kutta 方法	210
7.2.1 从 Taylor 法到二阶 Runge-Kutta 法	210
7.2.2 Modified Euler 方法与 Heun 方法	211
7.2.3 四阶 Runge-Kutta 方法	211
7.2.4 Runge-Kutta-Fehlberg 与 Verner 方法	212
7.3 Predictor-Corrector 方法	212
7.3.1 多步法的基本思想	212
7.3.2 预测-校正流程	213
7.4 微分方程组与高阶微分方程	213
7.4.1 一阶微分方程组	213
7.4.2 方程组的四阶 Runge-Kutta 方法	214
7.4.3 高阶微分方程化为一阶方程组	214
7.5 刚性微分方程与稳定性	217
7.5.1 刚性方程的现象	217
7.5.2 隐式梯形法与 Gear 方法	218
7.5.3 显式与隐式格式的稳定性比较	218
7.6 边值问题与打靶法	220
7.6.1 边值问题的基本形式	220
7.6.2 线性打靶法	220
7.6.3 非线性打靶法	220
7.6.4 灵敏度方程与 Newton 打靶	221
7.7 Numerov 方法	222
7.7.1 基本推导	222
7.7.2 非齐次 Numerov 公式	222
7.8 打靶法求解球对称势 Schrödinger 方程	223
7.8.1 径向方程与方势阱	223
7.8.2 束缚态边界条件与能量匹配	223
7.8.3 散射态与共振态	224
7.9 Green 函数与 Wronskian	224
7.9.1 一个 Dirichlet 边值问题	224
7.9.2 Green 函数形式	225
7.9.3 Wronskian 与一般二阶方程	225
7.10 有限差分法求解常微分方程	226
7.10.1 中心差分离散	226
7.10.2 边界条件与线性方程组	226

7.11 Tolman–Oppenheimer–Volkoff 方程	227
7.11.1 中子星结构方程	227
7.11.2 数值积分流程	228
7.12 本章总结	228
第八章 偏微分方程	230
8.1 二维有限差分法	230
8.1.1 二维 Poisson 方程与五点格式	230
8.1.2 二维网格到一维未知量的投影	231
8.1.3 边界条件	232
8.1.4 一个 Poisson 方程算例	233
8.1.5 高阶差分算符与对称性	233
8.2 Fourier 微分算符与 FFT	234
8.2.1 均匀周期网格上的 Fourier 表示	234
8.2.2 Fourier 微分矩阵	234
8.2.3 FFT 的物理用途	235
8.3 基矢展开法	235
8.3.1 从微分方程到矩阵本征值问题	235
8.3.2 变形谐振子基	237
8.4 离散变量表示 DVR	237
8.4.1 DVR 的基本思想	237
8.4.2 由正交多项式构造 DVR	238
8.4.3 sinc DVR	239
8.5 B-样条展开法	239
8.5.1 B-样条基函数	239
8.5.2 边界条件与微分算符矩阵	241
8.5.3 轴对称方程中的 B-样条/DVR	241
8.5.4 自旋轨道耦合与轴对称边界	242
8.6 小波与多小波分析	245
8.6.1 分层局域基	245
8.6.2 多小波分解与过滤	245
8.6.3 微分方程到积分方程	246
8.7 有限元方法	247
8.7.1 有限元思想与 Delaunay 三角剖分	247
8.7.2 Poisson 方程的弱形式	247
8.7.3 自然坐标与线性三角元	248
8.7.4 局域刚度矩阵与组装	248
8.8 边界条件与 Coulomb 势	249
8.8.1 Coulomb 势边界条件	249
8.8.2 球谐函数与多极矩	249
8.8.3 轴对称 Coulomb 势的解析检验	250
8.9 Jacobi 迭代法	250
8.10 周期、吸收与 Twisted 边界条件	251
8.10.1 周期边界与 Fourier 微分	251
8.10.2 Twisted 边界条件	251

8.11 本章总结	252
第九章 动力学方程	253
9.1 时间演化问题与量子动力学背景	253
9.1.1 时间相关方程作为初值问题	253
9.2 扩散方程的时间离散	253
9.2.1 显式格式、隐式格式与中点思想	253
9.2.2 Crank–Nicolson 格式	255
9.3 波动方程的时间推进	255
9.3.1 一维波动方程	255
9.3.2 二维波动方程与自然边界处理	256
9.4 海啸模型：变系数波动方程	257
9.4.1 浅水波模型	257
9.4.2 变系数项的离散	257
9.5 时间演化算符	258
9.5.1 传播子与酉性	258
9.5.2 Crank–Nicolson 作为 Cayley 传播子	258
9.5.3 一维 TDSE 的 CN 离散	259
9.5.4 二阶传播格式	259
9.6 Split-operator method	260
9.6.1 算符分裂与 BCH 关系	260
9.6.2 FFT 实现步骤	260
9.7 Krylov 子空间传播	261
9.7.1 Krylov 子空间与 Lanczos 递推	261
9.7.2 小三对角 Hamiltonian 的指数	261
9.8 多项式传播子与 Chebyshev Propagator	262
9.8.1 Polynomial Propagators	262
9.8.2 Chebyshev 多项式	262
9.8.3 Chebyshev Propagator 的归一化能谱	263
9.8.4 Magnus 展开	264
9.9 时间相关薛定谔方程传播方法小结	264
9.10 虚时间演化与 Damped 梯度法	265
9.10.1 虚时间演化求基态	265
9.10.2 Damped 梯度法	265
9.11 分子动力学	266
9.11.1 经典分子动力学	266
9.11.2 量子分子动力学	267
9.12 非平衡统计动力学	267
9.12.1 Brown 运动与 Langevin 方程	267
9.12.2 输运理论与 BUU 方程	267
9.12.3 流体力学与 Navier–Stokes 方程	267
9.13 本章总结	268
第十章 神经网络与机器学习	269
10.1 机器学习、统计与物理图像	269

10.1.1 从统计物理到神经网络	269
10.1.2 统计与神经网络问题	270
10.2 经典统计	271
10.2.1 均值、方差、标准差与相关系数	271
10.2.2 信息熵、相对熵与条件熵	272
10.2.3 协方差矩阵与误差传播	273
10.3 Bayes 统计	274
10.3.1 基本概率公式与 Bayes 定理	274
10.3.2 一个医学检测例子	275
10.3.3 Bayesian 推断、预测与模型比较	275
10.4 线性回归、高斯过程与自助法	276
10.4.1 线性回归与最小二乘	276
10.4.2 多元线性回归、Bayesian linear regression 与 SVD	277
10.4.3 Gaussian Process Regression	277
10.4.4 自助法 bootstrapping	280
10.5 AI for Science 与 Machine Learning for Physics	280
10.5.1 从大数据到科学信息	280
10.6 人工神经网络	281
10.6.1 基本思想与学习类型	281
10.6.2 生物神经元与人工神经元	282
10.6.3 机器学习作为非线性优化	282
10.6.4 BP 神经网络与误差反传	282
10.7 Bayesian 神经网络	284
10.7.1 权重作为概率分布	284
10.7.2 Bayesian 预测与 MCMC	285
10.7.3 共轭先验与 Bayesian learning	285
10.7.4 Stochastic Gradient Langevin Dynamics	286
10.7.5 正则化的 Bayesian 解释	286
10.7.6 Occam's Razor 与模型证据	287
10.7.7 物理应用中的 BNN、GPR 与置信区间	287
10.8 机器学习的应用	288
10.8.1 数据融合、模型混合与神经量子态	288
10.8.2 Bayesian model mixing for nuclear mass	289
10.8.3 核能、核取证与物理信息机器学习	290
10.9 其他机器学习方法	291
10.9.1 CNN、深度学习、RNN 与 KAN	291
10.9.2 SVM、Decision Tree、Random Forest 与 Boosting	292
10.9.3 生成式模型、GPT 与 Transformer	292
10.9.4 AI4S、黑箱问题与物理引导	292
10.9.5 Bayesian Optimization	294
10.10 本章总结	294
第十一章 超级计算与量子计算	295
11.1 计算机发展与物理的关系	295
11.1.1 超级计算机的发展趋势	295

11.1.2 TOP500 与性能演化	296
11.1.3 从向量机到异构超级计算	297
11.1.4 集群、互连与云计算	297
11.2 MPI 与 OpenMP 并行编程	301
11.2.1 MPI 的基本思想	301
11.2.2 MPI 通信、阻塞与非阻塞	303
11.2.3 Master-Worker 模式	303
11.2.4 OpenMP 多线程并行	304
11.2.5 MPI+OpenMP 混合并行	305
11.3 GPU 与协处理器	305
11.3.1 GPU 加速的基本思想	305
11.3.2 GPU 线性代数库与 MAGMA	307
11.3.3 并行效率与通信瓶颈	307
11.4 量子计算原理	308
11.4.1 从经典比特到量子比特	308
11.4.2 量子计算的优势、算法与路线图	308
11.4.3 量子逻辑门	309
11.4.4 CNOT、纠缠与测量	311
11.4.5 量子程序与真实量子芯片	312
11.5 量子多体问题的云量子计算	314
11.5.1 配对 Hamiltonian 的量子比特映射	314
11.5.2 VQE、UCC 与 EPA	315
11.5.3 量子测量与 Pauli 基变换	315
11.5.4 配对模型结果、误差缓解与激发态	316
11.5.5 有限量子体系与热激发	317
11.5.6 氦核束缚能的云量子计算	318
11.6 量子傅里叶变换与量子相位估计	319
11.6.1 计算基与整数编码	319
11.6.2 从 DFT 到 QFT	320
11.6.3 QFT 的乘积形式	321
11.6.4 QFT 线路实现	321
11.6.5 量子相位估计	322
11.6.6 SBSR 与粒子数恢复	324
11.7 计算挑战与第一性原理核结构量子计算	324
11.7.1 量子多体计算挑战	324
11.7.2 第一性原理核结构的量子计算	325
11.8 本章总结	325

第一章 计算物理引言与基础

内容提要

- 课程内容、教学目标与计算物理的学科定位
- 复杂体系、多尺度问题、多体问题与自由度层级
- 超级计算、新型计算基础设施与机器学习、量子计算前沿
- 浮点数表示、程序结构、编程语言与操作系统基础
- 存储模型、内存访问、计算误差、稳定性与经典算法
- 课程作业、考核方式与报告规范

1.1 课程内容、教学目标与学科定位

1.1.1 课程内容与学习目标

计算物理学 (Computational Physics) 训练的是一套从物理问题出发的数值化思维链条:

物理建模 $\rightarrow$ 数值方程 $\rightarrow$ 算法设计 $\rightarrow$ 程序实现 $\rightarrow$ 结果分析 $\rightarrow$ 科学报告.

本课程后续的主要内容包括:

- 引言;
- 线性方程组问题;
- 插值、微分与积分方法;
- 优化与非线性方程;
- 本征值问题;
- 蒙特卡罗方法;
- 常微分方程;
- 偏微分方程;
- 动力学方程;
- 高性能计算;
- 机器学习;
- 量子计算。

 **笔记** 课程目标可以概括为三层:

- 方法层: 掌握线性代数、离散化、优化、蒙特卡罗、微分方程等经典数值方法;
- 实践层: 把算法落成可运行、可验证、可复现的程序, 并完成结果分析与报告撰写;
- 前沿层: 理解超级计算 (Supercomputing)、机器学习 (Machine Learning)、量子计算 (Quantum Computing) 等新工具如何进入现代物理研究。

课程推荐的经典参考书是 *Numerical Recipes in C++* (W. H. Press, S. A. Teukolsky, W. T. Vetterling, B. P. Flannery)。

1.1.2 计算物理的学科定位

计算物理常被称为“**第三种物理学**” (Physics of the third kind)。传统上, 物理学有两条经典路线:

- 理论 (Theory)**: 依靠解析推导建立规律;
- 实验 (Experiment)**: 依靠观测与测量验证理论。

而面对强关联 (strongly correlated)、非微扰 (non-perturbative)、高维、多过程耦合、强非线性等复杂问题时，仅靠解析方法往往已经不足，于是出现了第三条路线：

Theory Experiment Computation.

现代物理世界常常本质上是**相互作用的量子多体系统 (interacting quantum many-particle systems)**。这些问题即使“方程已经写下来”，也可能因为维数过高、耦合过强、不可解析而无法直接求解。计算物理的任务，正是把这种“写得出但解不动”的问题转化为可计算的问题。

注 因此，计算物理不是简单的“程序设计课”，也不是单纯的“数值分析课”，而是物理、数学与计算机科学三者交汇处的一门学科。

1.2 复杂性、多尺度与自由度层级

1.2.1 为什么现代物理离不开计算

计算物理之所以成为现代物理研究的核心方法之一，根本原因在于研究对象的复杂性不断增长。问题复杂性的升级链可以概括为：

- 从单体问题到多体问题；
- 从线性问题到非线性问题；
- 从低维体系到高维体系；
- 从标量模型到矢量、张量系统；
- 从常微分方程 (ODE) 到偏微分方程 (PDE)；
- 从低阶微扰到高阶微扰，乃至非微扰；
- 从静态结构到动态演化；
- 从理想模型到复杂实际模型；
- 从单一学科到交叉学科；
- 从少量数据到大数据、异构数据与缺陷数据。

这意味着：**问题规模、变量维数和耦合强度几乎总在上升，而解析工具的适用范围却在缩小**。因而，计算已经不是理论和实验之后的“附属技能”，而是现代物理的必要支柱。

1.2.2 多尺度 (Multi-scale) 视角

可以把物理对象按尺度排成一架“量子阶梯”：从超弦 (superstrings)、夸克与轻子 (quarks and leptons)、重子与介子 (baryons and mesons)、原子核 (atomic nuclei)、原子 (atoms)、分子 (molecules)，一直到材料、细胞、行星、恒星、星系和星系团。它说明了一个核心事实：**现代物理问题往往同时跨越极大的长度、能量和时间尺度**。

定义 1.1

多尺度问题 (**multi-scale problem**) 是指系统的重要物理过程同时出现在多个相差悬殊的尺度上，而且这些尺度之间存在耦合，不能简单分开处理。

例如：

- 材料模拟中，电子尺度决定化学键，而微米尺度决定器件性能；
- 天体物理中，微观核反应截面会反馈到恒星演化和可观测量；
- 流体与等离子体中，局部微观涨落可能影响整体输运与稳定性。

多尺度问题最困难的地方在于：**既不能只看最小尺度，也不能只看最大尺度**。一个好的算法必须决定：

1. 哪些尺度需要精确求解；
2. 哪些尺度可以有效近似；

3. 不同尺度之间怎样交换信息。

1.2.3 多体问题与自由度 (Degrees of Freedom)

从少体到多体，最直接的困难是**自由度数量爆炸 (explosion of degrees of freedom)**。以核物理为例，不同分辨率 (resolution) 下自由度会发生明显变化：

- 在强子层次，研究对象是质子、中子等强子；
- 在核子层次，研究对象是原子核内部的核子结构与反应；
- 在更高分辨率下，需要考虑夸克、胶子以及量子色动力学 (QCD)；
- 在高温高密条件下，还可能进入夸克胶子等离子体 (quark-gluon plasma, QGP) 等新相。

这说明“系统是什么”并不是固定不变的，而取决于观测分辨率。当分辨率提高时，原本被看成“整体”的对象会被分解成更多内部自由度；当分辨率降低时，大量微观变量又会被压缩成少数有效变量。

例题 1.1 核物理中的层级建模 核物理中常见的层级包括：

1. **集体模型 (collective model)**：用形变、转动等少数宏观自由度描述原子核；
2. **密度泛函理论 (Density Functional Theory, DFT)**：保留更细致的平均场与密度分布；
3. **ab initio 方法**：尽量直接从核子间相互作用出发；
4. **LQCD / QCD**：进一步把核子结构追溯到夸克与胶子。

随着层级向下，理论更基础，但计算复杂度也急剧上升。

1.2.4 交叉学科与 SciDAC 理念

从气候科学 (climate science)、聚变研究 (fusion research)、核物理 (nuclear physics)、高能物理 (high energy physics)、天体物理 (astrophysics) 到材料、化学、生物、环境科学等多个方向，都能看到计算物理的身影。这说明它并不局限于某一个狭窄分支，而是一种普适的方法论。

SciDAC (Scientific Discovery through Advanced Computing) 所强调的核心理念，就是把

$$\text{Physics} \cap \text{Math} \cap \text{Computer Science}$$

真正组织成研究共同体。物理给出问题与模型，数学给出算法与分析，计算机科学提供硬件、软件与并行环境。

注 这也是为什么学习计算物理时不能只关心公式推导。一个成熟的计算方案必须同时回答三个问题：

1. 模型是否合理；
2. 算法是否稳定；
3. 程序在真实硬件上是否高效。

1.3 超级计算、计算基础设施与学科前沿

1.3.1 超级计算的重要性

没有超级计算 (supercomputing)，很多从头计算 (ab initio) 与大规模密度泛函理论 (large-scale DFT) 几乎不可能实现。

现代复杂物理问题之所以需要超算，通常是因为它们同时具备以下特征：

- 状态空间维数极高；
- 稀疏矩阵规模巨大；
- 需要长时间迭代或参数扫描；
- 需要大规模数据存储、交换与后处理。

现代高性能计算中特别需要关注的新硬件结构包括：

- 多核 CPU (multi-core CPU)；

- GPU 加速 (graphics processing unit acceleration);
- 更一般的异构计算 (heterogeneous computing)。

其背后的关键认识是：**移动数据 (moving data)** 往往比做一次浮点运算更昂贵。因而现代高性能算法不只关心“乘加次数”，更关心数据布局、缓存命中和通信代价。

1.3.2 新型计算基础设施：任务化与异步并行

当问题本身具有多分辨率自适应 (multi-resolution adaptive) 结构时，程序设计也必须从传统的顺序流程转向**任务化 (task-oriented)** 与**异步化 (asynchronous)**。

典型组成包括：

- **MPI**：进程间分布式并行；
- **Global Arrays**：统一访问分布式数组；
- **Multi-threading**：单节点内多线程并行；
- **Futures**：用依赖关系管理异步任务；
- **Load balancing**：让计算负载在不同核之间尽量均衡。

这一思路特别适合格点空间上求解微分方程。某些局部区域需要高分辨率，另一些区域只需粗网格，于是程序需要做到：

1. 先把大问题切成很多小任务；
2. 用依赖图管理“哪些任务必须先做、哪些可以并行做”；
3. 在 MPI 进程与线程池之间动态调度；
4. 用自适应网格细化 (adaptive mesh refinement, AMR) 把算力集中到真正困难的局部。

例题 1.2 任务依赖的直观理解 若任务 B 和 D 都依赖于任务 A ，而任务 C 独立，那么合理的调度方式不是严格按

$$A \rightarrow B \rightarrow C \rightarrow D$$

顺序执行，而应在完成 A 后尽快并发推进 B 、 D ，同时让 C 插空执行。这就是并行程序设计中“依赖优先于顺序”的基本思想。

1.3.3 计算机发展与物理发展的关系

物理理论的进展始终与计算能力紧密耦合。早期许多模型之所以能成功，是因为它们在当时的计算条件下刚好可处理；而今天从唯象理论走向更微观、更高精度的理论时，大规模数值计算已经成为不可绕开的环节。

 **笔记** 计算物理的重要源头之一来自核武器研发与相关大规模模拟需求。这说明计算物理从诞生之初就与“复杂、多参数、强耦合、不能依靠实验直接反复试错”的问题密切相关。

1.3.4 计算物理的新前沿

当前两个最活跃的新方向是：

- **机器学习 (Machine Learning)**；
- **量子计算 (Quantum Computing)**。

机器学习 机器学习推动了 **AI for Science (AI4S)** 范式。它尤其适合处理以下问题：

1. 很难写出精确闭式方程，但存在大量数据；
2. 需要在高维参数空间中寻找复杂关联；
3. 需要构造代理模型 (surrogate model) 或快速近似器。

ChatGPT、AlphaFold 等系统展示了数据驱动方法在语言、结构预测和科学建模中的潜力。

量子计算 量子计算依赖叠加态 (superposition) 与量子纠缠 (entanglement) 实现信息处理。它最自然的优势领域包括量子多体问题、某些组合优化问题以及某些经典计算极难承受的高维 Hilbert 空间模拟。它常被视为突破经典计算在能耗和速度上瓶颈的重要候选技术。

1.4 计算基础、浮点数与数值表示

1.4.1 数值计算的基本特点

计算机中的数值并不是连续实数，而是**离散表示 (discrete representation)**。这意味着：

1. 无理数和大多数有理数只能近似存储；
2. 连续变量在数值方法中往往要离散化；
3. 精度受机器字长 (word length) 限制；
4. 溢出 (overflow)、下溢 (underflow) 和无效数 (NaN) 都是实际存在的数值现象。

最常见的数据类型包括定点整数 (integer, int) 和浮点数 (floating-point number, float / double)，其中

$$8 \text{ bits} = 1 \text{ byte}.$$

1.4.2 单精度、双精度与内存估算

- 单精度 (single precision / float): 32 位, 4 Bytes, 通常约 7 位十进制有效数字；
- 双精度 (double precision / double): 64 位, 8 Bytes, 通常约 16 位十进制有效数字。

例题 1.3 双精度复数矩阵的内存占用 一个 5000×5000 的双精度复数矩阵，元素总数为

$$5000 \times 5000 = 2.5 \times 10^7.$$

一个双精度复数由实部和虚部组成，各占 8 Bytes，因此每个元素占

$$16 \text{ Bytes}.$$

总占用约为

$$5000 \times 5000 \times 16 = 4.0 \times 10^8 \text{ Bytes}.$$

若按 $1 \text{ MiB} = 1024^2 \text{ Bytes}$ 换算，则约为

$$\frac{4.0 \times 10^8}{1024^2} \approx 381 \text{ MB}.$$

这说明：**即使问题还没有开始计算，单是存储就可能先成为瓶颈。**

1.4.3 IEEE 754 浮点表示

以单精度为例，一个浮点数可写作

$$x = (-1)^S \times 1.M \times 2^e,$$

其中：

- S 是符号位 (sign bit)；
- E 是阶码 (exponent field)；
- M 是尾数小数部分 (mantissa / significand fraction)；
- 实际指数 $e = E - 127$ (对单精度而言)。

对单精度，正常数的大致范围是

$$2^{-126} \sim 2^{127} \approx 10^{-38} \sim 10^{38};$$

对双精度，正常数范围大致可到

$$4.9 \times 10^{-324} \sim 1.7 \times 10^{308}.$$

定义 1.2 (IEEE 754 双精度结构)

双精度数由 64 位组成：

- 1 位符号位；
- 11 位阶码，偏置为 1023；
- 52 位尾数小数部分，隐藏最高位 1。

因此双精度数的标准化形式为

$$x = (-1)^S \times 1.M \times 2^{E-1023}.$$



例题 1.4 十进制数 11.375 的二进制表示 先把整数与小数部分分别转成二进制：

$$11 = (1011)_2, \quad 0.375 = 2^{-2} + 2^{-3} = (0.011)_2.$$

因此

$$11.375 = (1011.011)_2.$$

把它标准化为

$$1.011011 \times 2^3.$$

于是：

- 符号位 $S = 0$ ；
- 实际指数 $e = 3$ ；
- 单精度阶码 $E = 127 + 3 = 130 = (10000010)_2$ ；
- 尾数字段写成 $011011000 \dots 0$ 。

这就是十进制数“11.375”的典型二进制浮点编码示例。

1.5 程序结构、编程语言与系统基础

1.5.1 程序结构与伪代码

数值程序最常见的的基本结构包括：

- 循环 (for, do while)；
- 分支 (if / else if / else)；
- 子程序与函数调用 (subroutine / function call)；
- 参数传递 (argument passing)；
- 全局变量与局部变量 (global / local variables)。

在真正写程序之前，先用**伪代码 (pseudocode)** 或**流程图 (control flow diagram)** 把逻辑整理清楚，往往能显著减少错误。实践中尤其要注意：

- 尽量避免过于复杂的流程结构；
- 养成写程序注释 (comment) 的习惯；
- 让程序既可靠又简洁。

这是因为数值程序除了语法错误之外，还常有更隐蔽的错误来源，如溢出、数据被覆盖、参数传递不一致以及误差累积。

1.5.2 科学计算中的主要编程语言

Fortran 全称为 *FORmula TRANslation*，于 1957 年由 John Backus 领导的团队推出。它长期是科学计算和超级计算的主力语言，优势包括：

- 数组和矩阵运算语义清晰；

- 复数支持自然；
- 编译器针对数值计算优化成熟；
- 历史程序库极其丰富。

C/C++ C 语言与 Fortran 一样接近底层，适合高性能实现；**C++** 则在此基础上加入类、继承和面向对象机制，适合大型工程。它们在系统开发、算法库和混合架构程序中非常常见。

解释型语言 Python、Matlab、JavaScript 等解释型语言开发速度快、生态丰富，但其高层部分往往较慢；实际高性能计算时，经常由底层 C/Fortran 库承担核心数值工作。今天大量 Python 科学计算代码，本质上也是“用 Python 调度 C/Fortran 内核”。

汇编语言 汇编语言（assembly language）直接操作机器指令和寄存器，性能潜力高，但开发成本极大，通常只在编译器、内核库或极少数性能关键模块中出现。

1.5.3 操作系统、编译与库链接

科学计算程序的基本生命周期包括

写代码 → 编译 → 链接 → 运行.

在 Linux 环境中，经常要接触：

- 编译器：g++, gfortran, ifort 等；
- 并行环境变量：例如 OMP_NUM_THREADS 控制 OpenMP 线程数；
- 作业调度命令：用于集群提交并行任务；
- 头文件与模块接口：保证函数声明、变量类型和外部库调用一致。

定义 1.3

静态库（static library）是在链接时直接并入可执行文件的库，Linux 中常见后缀为 .a；动态库（shared library）是在运行时装载的库，Linux 中常见后缀为 .so。

例题 1.5 库链接的基本理解 若编译命令中写

`-llapack -lblas,`

其含义是：在链接阶段把 LAPACK 和 BLAS 库加入程序。数值程序能否成功运行，除了算法本身外，还取决于：

1. 头文件是否正确；
2. 库路径是否正确；
3. 运行时环境变量是否设置一致。

1.6 计算性能、存储模型与内存访问

1.6.1 计算中要平衡的三件事

计算中的核心考量可概括为三点：

- 精度（accuracy）；
- 速度（speed）；
- 可靠性（reliability）。

三者往往不能同时做到极致，必须根据问题目标权衡。

影响速度的主要因素

- 浮点运算能力；
- 内存访问速度；
- 进程间通信速度；

- 磁盘读写性能;
- 算法复杂度、收敛速度与硬件利用率。

影响精度的主要因素

- 机器误差 (machine error);
- 舍入误差 (round-off error);
- 截断误差 (truncation error);
- 离散误差 (discretization error);
- 统计误差 (statistical error)。

影响可靠性的主要因素

- 程序实现是否规范;
- 是否存在误差累计和数值溢出;
- 迭代法是否收敛;
- 算法是否稳定。

此外, 并行计算还必须考虑:

- **存储限制:** 节点内存是否足够, 堆栈是否会溢出, 内存分配/回收是否合理;
- **通信限制:** 本地计算与远程通信如何平衡, 消息大小与缓存如何优化。

1.6.2 行优先、列优先与缓存命中

多维数组在内存中最终都要按一维方式连续存储, 但不同语言的约定不同:

- **C / C++:** 行优先 (row-major order);
- **Fortran / Matlab:** 列优先 (column-major order)。

若矩阵为

$$A = (a_{ij})_{m \times n},$$

则在 C 中更接近连续存储顺序的是

$$a_{11}, a_{12}, \dots, a_{1n}, a_{21}, a_{22}, \dots, a_{2n}, \dots, a_{mn},$$

而在 Fortran 中更接近

$$a_{11}, a_{21}, \dots, a_{m1}, a_{12}, a_{22}, \dots, a_{m2}, \dots, a_{mn}.$$

性质 为了提高缓存 (cache) 命中率, 双重循环的内层循环应该沿着内存连续方向推进:

- 对 C/C++, 内层最好扫列;
- 对 Fortran/Matlab, 内层最好扫行。

例题 1.6 同一程序, 不同访存顺序导致显著速度差异 对 C 语言二维数组, 下面的写法不符合行优先存储:

```
for (j = 0; j < 5000; j++) {
    for (i = 0; i < 1000; i++) {
        a[i][j] = a[i][j]*a[i][j] + 2.0*a[i][j] + 10.0;
    }
}
```

一次典型测试中, 这种写法的运行时间约为 0.063 s。

而按行优先方式改写为

```
for (i = 0; i < 1000; i++) {
    for (j = 0; j < 5000; j++) {
        a[i][j] = a[i][j]*a[i][j] + 2.0*a[i][j] + 10.0;
    }
}
```



```
}

```

运行时间约降到 0.046 s。

两段代码的浮点运算量完全一样，差别主要来自缓存与访存局部性（memory locality）。

1.6.3 栈、堆与动态内存

- **栈 (stack)**: 先进后出，空间较小，但分配释放极快，适合生命周期明确的局部变量；
- **堆 (heap)**: 适合大对象和动态申请内存，但管理成本更高。

在 C 中常见的动态分配是 `malloc`，在 Fortran 中常见的是 `allocate`。科学计算中的大数组通常不能都放在栈上，否则很容易触发栈溢出。

1.7 计算误差、稳定性与算法观

1.7.1 误差的来源与度量

计算误差主要有两大来源：

1. **舍入误差 (round-off error)**: 由于机器有效位有限，无法表示无限精确的小数；
2. **截断误差 (truncation error)**: 由于把无限过程截断为有限近似，例如泰勒展开只保留前几项、微分方程用有限差分代替连续导数。

若某个量的真值为 x ，近似值为 $\tilde{x}$ ，则

$$\text{绝对误差 (absolute error)} = |\tilde{x} - x|,$$

$$\text{相对误差 (relative error)} = \frac{|\tilde{x} - x|}{|x|} \quad (x \neq 0).$$

1.7.2 误差放大与算法稳定性

所谓**算法稳定性 (algorithmic stability)**，核心是在计算过程中避免误差被指数式放大。一个稳定算法不一定零误差，但它不会把输入中的微小扰动放大成完全错误的输出。

数值计算中的一个基本原则是：**避免误差的指数传播**。这意味着在每一步设计中都要问：

1. 这一步是否会造成灾难性消去 (catastrophic cancellation)？
2. 这一步是否会把很小的误差除以很小的量？
3. 这一步是否会把很多小误差长期积累起来？

1.7.3 相近数相减的经典例子

考虑

$$f(x) = \frac{1 - \cos x}{\sin x},$$

利用恒等变形，也可写成

$$f(x) = \frac{\sin x}{1 + \cos x}.$$

当

$$x = 0.006$$

时，取近似值

$$\sin(0.006) \approx 0.59999 \times 10^{-2}, \quad \cos(0.006) \approx 0.99998.$$

直接代入第一式

$$f(x) = \frac{1 - 0.99998}{0.59999 \times 10^{-2}} = \frac{0.2 \times 10^{-4}}{0.59999 \times 10^{-2}} = 0.33334 \times 10^{-2}.$$

改用第二式

$$f(x) = \frac{0.59999 \times 10^{-2}}{1 + 0.99998} = \frac{0.59999 \times 10^{-2}}{1.99998} = 0.30000 \times 10^{-2}.$$

后者明显更接近真实值。这是因为第一式中的

$$1 - \cos x$$

是两个非常接近的数相减，有效数字大量丢失；而第二式把问题转化成了稳定得多的加法与除法。

结论 当两个量彼此接近时，相减会导致前面许多有效数字互相抵消，只剩下误差主导的尾数。这种现象称为灾难性消去（catastrophic cancellation）。

1.7.4 减少误差的基本原则

1. 避免相近的两数相减；
2. 避免很小的数作除数；
3. 防止大数吃掉小数；
4. 尽量减少无意义的中间步骤与误差累积；
5. 优先使用稳定算法；
6. 区分系统误差、统计误差与推断误差。

例题 1.7 防止大数吃掉小数 若用双精度把

$$10^{16} + 1 - 10^{16}$$

按从左到右顺序计算，则中间的

$$10^{16} + 1$$

可能仍被存成 10^{16} ，最后结果变成 0 而不是 1。这说明当大数和小数直接相加时，小量可能完全丢失。因此求和时常采用先加小量后加大量，或使用补偿求和（compensated summation）等更稳定方法。

1.7.5 20 世纪十大算法与 21 世纪算法趋势

20 世纪最具代表性的一类算法榜单中，常会包括：

1. Metropolis 算法及 Gibbs sampler；
2. 整数关系发现（integer relation detection）；
3. 线性规划单纯形法（simplex method）与内点法（interior-point methods）；
4. 线性方程与最小二乘中的矩阵分解算法：LU、Cholesky、Householder；
5. 快速排序（quicksort）；
6. 共轭梯度法与 Krylov 子空间方法；
7. 求本征值的 QR 算法与 SVD；
8. 快速傅里叶变换（FFT）；
9. 快速多极子方法（fast multipole method）；
10. 不同版本的榜单中还常会加入有限元法（FEM）、多重网格法（multigrid method）、有限差分法、拟牛顿法等重要方法。

注 这类榜单并不是要背诵名次，而是要抓住一个事实：真正改变科学计算面貌的，往往不是单个公式，而是把大规模问题从“不可算”变成“可算”的算法思想。

进入 21 世纪后，尤其值得关注的新方向是：

- 量子计算；

- 大数据；
- 机器学习，包括 Transformer、深度卷积网络（Deep CNN）等。

1.8 课程作业与考核安排

虽然这一部分不属于算法理论，但它决定了如何把课堂知识真正转化为独立研究训练，因此也值得在笔记中保留下来。

1.8.1 作业主题与考核方式

一种常见的课程作业安排可采用“11 选 3”的方式，题目包括：

1. 求解一维薛定谔方程；
2. 求解椭圆 Poisson 方程；
3. 用 QR 或 Lanczos 方法求本征值；
4. 求解非线性方程组；
5. 全局多参数优化；
6. 用蒙特卡罗方法求解 Ising 模型；
7. 用 WKB 方法计算核裂变寿命；
8. 求解中子星 TOV 方程；
9. 实现并行计算；
10. 实现神经网络；
11. 实现量子计算示例。

此外也允许选择一个科研题目作为替代。考核比例为

考试40%， 平时60%。

若按一个典型学期安排，可把时间节点记为：

- 2026 年 4 月 15 日：第 1 次作业；
- 2026 年 5 月 15 日：第 2 次作业；
- 2026 年 6 月 15 日：第 3 次作业；
- 2026 年 6 月 10 日：课程结束；
- 2026 年 6 月 22 日：考试。

1.8.2 报告格式要求

每次作业报告应至少包含以下四部分：

1. 问题描述；
2. 解题思路与算法分析；
3. 结果展示与讨论；
4. 源代码。

提交格式为 PDF 报告，并附源代码。实现语言可选 C、Fortran 或 Python。讨论思路是允许的，但结果和代码必须独立完成。

第二章 线性代数问题 (Linear Algebra Problems)

内容提要

- 线性代数问题的物理背景 (如 Poisson 方程离散化)
- 直接求解法: Gauss 消元法、LU 分解法、QR 分解
- 迭代求解法: Jacobi、Gauss-Seidel、SOR
- 高阶优化算法: 共轭梯度法 (CG)
- 奇异问题求解: SVD 分解与最小二乘拟合
- 矩阵范数、条件数与稀疏矩阵处理

2.1 引言与基础知识

线性代数在计算物理中有极其广泛的应用, 如数据拟合、物理问题离散化 (微分方程数值解)。TOP500 超级计算机的 Linpack benchmark 测试核心就是求解大规模线性方程组 $A \cdot x = b$ 。

经典物理实例: Poisson 方程 (拉普拉斯方程)

$$\nabla^2 \varphi = f \quad (\Delta \varphi = 0)$$

通过差分离散化后, 会转化为一个大型稀疏矩阵求解问题。

定义 2.1 (常见特殊矩阵定义)

在物理计算中, 不同对称性的矩阵有特定的快速算法, 需要熟记以下基本关系:

- 对称矩阵 (Symmetric): $A = A^T$, 元素 $a_{ij} = a_{ji}$
- 实正交矩阵 (Real Orthogonal): $A = (A^T)^{-1}$, 满足 $\sum_k a_{ik} a_{jk} = \delta_{ij}$
- 厄米矩阵 (Hermitian): $A = A^\dagger$ (共轭转置), 元素 $a_{ij} = a_{ji}^*$
- 酉矩阵 (Unitary): $A = (A^\dagger)^{-1}$, 满足 $\sum_k a_{ik} a_{jk}^* = \delta_{ij}$

2.2 Gauss 消元法与主元法 (Gaussian Elimination & Pivoting)

2.2.1 标准 Gauss 消元法

求解线性方程组的基础方法是通过初等行变换, 将原方程组化为上三角矩阵, 然后通过回代 (Back Substitution) 求解。

设有方程组 $Ax = b$, 我们可以构造增广矩阵 (Augmented Matrix) $\tilde{A} = [A, b]$:

$$\tilde{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} & \vdots & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & \vdots & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & \vdots & b_n \end{bmatrix}$$

逐列消元过程 (Forward Elimination): 第 k 步消元时, 使用主元 $a_{kk}^{(k)}$ 消去其下方的所有元素。对于 $j = k+1, \dots, n$:

$$m_{jk} = \frac{a_{jk}^{(k)}}{a_{kk}^{(k)}}$$

$$(E_j) \leftarrow (E_j) - m_{jk}(E_k)$$

回代过程 (Back Substitution): 得到上三角矩阵后, 从最后一行开始反向求解:

$$x_n = \frac{a_{n,n+1}}{a_{nn}}$$

$$x_i = \frac{a_{i,n+1} - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}}, \quad i = n-1, n-2, \dots, 1$$

 **笔记** 利用 Gauss-Jordan 消元法可以将矩阵化为对角矩阵, 此种方法可以用来求解矩阵的逆。具体方法如下: 1. 构造增广矩阵 $\tilde{A} = [A, I]$, 其中 I 是单位矩阵。2. 通过初等行变换将左侧的 A 化为单位矩阵 I , 此时右侧的 I 就会变成 A^{-1} 。3. 最终得到 $\tilde{A} = [I, A^{-1}]$ 。

2.2.2 列主元消元法 (Partial Pivoting)

 **笔记** 如果在消元过程中, 主元 $a_{kk} \approx 0$ 或者相对于其他元素非常小, 会导致除法产生巨大的舍入误差。为了解决这个问题, 需要引入列主元法。

策略: 在第 k 步消元前, 在第 k 列及其下方的元素中寻找绝对值最大的元素, 将其所在行与第 k 行交换, 即找到 $p \geq k$ 使得 $|a_{pk}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}|$, 然后执行行交换 $(E_k) \leftrightarrow (E_p)$ 。这不改变方程的解, 但极大提高了数值稳定性。

例题 2.1 为什么需要列主元? 考虑方程组:

$$\begin{cases} 0.003000x_1 + 59.14x_2 = 59.17 \\ 5.291x_1 - 6.130x_2 = 46.78 \end{cases}$$

常规求解 (不换行): 第一步乘数为 $5.291/0.003 \approx 1763$ 。由于浮点数精度截断, 会导致 $x_2 \approx 1.001$, 进而得出 $x_1 \approx -10.00$ (严重错误)。

列主元求解: 第一列中 $|5.291| > |0.003000|$, 交换两行后进行计算, 最终得到极高精度的解 $x_1 = 10.00, x_2 = 1.00$ 。

2.3 LU 分解 (LU Decomposition)

2.3.1 基本原理与 Doolittle 算法

LU 分解是将矩阵 A 分解为一个下三角矩阵 L (对角线为 1) 和一个上三角矩阵 U 的乘积: $A = L \cdot U$ 。

解方程 $Ax = b$ 转变为两步:

1. 解 $Ly = b$ (向前替代求 y)
2. 解 $Ux = y$ (向后替代求 x)

Doolittle 递推算法公式: 通过矩阵乘法对应元素相等 $a_{ij} = \sum_{t=1}^{\min(i,j)} l_{it}u_{tj}$, 可以交替求出 U 的行和 L 的列:

$$\text{求 } U \text{ 的第 } i \text{ 行 } (j \geq i): \quad u_{ij} = a_{ij} - \sum_{t=1}^{i-1} l_{it}u_{tj}$$

$$\text{求 } L \text{ 的第 } i \text{ 列 } (j > i): \quad l_{ji} = \frac{a_{ji} - \sum_{t=1}^{i-1} l_{jt}u_{ti}}{u_{ii}}$$

例题 2.2 LU 分解计算示例 对矩阵 $A = \begin{bmatrix} 1 & 2 & 4 \\ 3 & 8 & 14 \\ 2 & 6 & 13 \end{bmatrix}$ 进行 LU 分解。

第一步: 求 U 的第一行和 L 的第一列:

$$u_{11} = 1, \quad u_{12} = 2, \quad u_{13} = 4$$

$$l_{21} = 3/1 = 3, \quad l_{31} = 2/1 = 2$$

第二步：求 U 的第二行和 L 的第二列：

$$u_{22} = a_{22} - l_{21}u_{12} = 8 - 3 \times 2 = 2$$

$$u_{23} = a_{23} - l_{21}u_{13} = 14 - 3 \times 4 = 2$$

$$l_{32} = (a_{32} - l_{31}u_{12})/u_{22} = (6 - 2 \times 2)/2 = 1$$

第三步：求 U 的第三行：

$$u_{33} = a_{33} - l_{31}u_{13} - l_{32}u_{23} = 13 - 2 \times 4 - 1 \times 2 = 3$$

最终得到：

$$A = LU = \begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 2 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & 2 & 2 \\ 0 & 0 & 3 \end{bmatrix}$$

2.3.2 高斯消元与 LU 分解的关系

高斯消元的过程本质上就是左乘一系列下三角变换矩阵 $M^{(k)}$ 。

$$M^{(n-1)} \dots M^{(2)} M^{(1)} A = U$$

令 $L = (M^{(n-1)} \dots M^{(1)})^{-1} = (M^{(1)})^{-1} \dots (M^{(n-1)})^{-1}$ ，即可得到唯一的 $A = LU$ 分解。这证明了无主元的高斯消元法等价于 LU 分解。

2.3.3 列主元 LU 分解 ($PA = LU$)

引入置换矩阵 (Permutation Matrix) P 记录行交换信息后，分解形式变为

$$PA = LU.$$

这里的关键点在于：列主元法中并不是只在开始换一次行，而是每一步消元前都可能继续换行。

设第 k 步的行交换由置换矩阵 P_k 表示，消元矩阵为 M_k ，则整个过程可以写成

$$M_{n-1}P_{n-1} \dots M_2P_2M_1P_1A = U.$$

把所有行交换合并成

$$P = P_{n-1} \dots P_2P_1,$$

再把所有消元乘数整理到下三角矩阵中，就得到

$$PA = LU.$$

因此， P 不是“某一次交换”，而是全部交换历史的累计结果。



笔记 在真正实现列主元 LU 时，若第 k 步发生换行，不仅要交换当前矩阵中的两行，也要同步交换 L 中前 $k-1$ 列已生成的乘数信息。正因为 bookkeeping 比无主元情形复杂，所以实际程序通常专门保存一个 pivot 数组（例如 LAPACK 中的 IPIV）来记录置换信息。

解方程的步骤变为：

1. 先对右端做相同置换：

$$PAx = Pb \implies LUx = Pb;$$

2. 解前代方程

$$Ly = Pb;$$

3. 解回代方程

$$Ux = y.$$

2.3.4 利用 LU 分解求逆矩阵与行列式

若已经有

$$PA = LU,$$

则求逆矩阵不必重新做高斯消元，而可以把

$$A^{-1} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_n]$$

的每一列视为线性方程组

$$A\mathbf{x}_j = \mathbf{e}_j, \quad j = 1, 2, \dots, n$$

的解，其中 $\mathbf{e}_j$ 是第 j 个标准基向量。于是只需逐列求解

$$PA\mathbf{x}_j = P\mathbf{e}_j \implies LU\mathbf{x}_j = P\mathbf{e}_j.$$

也就是说，对每个 j ：

1. 先解

$$L\mathbf{y}_j = P\mathbf{e}_j;$$

2. 再解

$$U\mathbf{x}_j = \mathbf{y}_j.$$

把全部 $\mathbf{x}_j$ 拼起来，就是 A^{-1} 。

注 因此，求一次 LU 分解就等于为很多右端 $\mathbf{b}$ 预处理好了求解器。这也是 LU 分解在“同一个系数矩阵、多个不同右端”的问题中尤其高效的原因。

对行列式而言，由于

$$\det(PA) = \det(L) \det(U),$$

而 $\det(L) = 1$ (Doolittle 形式下 L 对角元全为 1)，故

$$\det(A) = \det(P)^{-1} \prod_{i=1}^n u_{ii}.$$

若总共交换了 s 次行，则

$$\det(P) = (-1)^s, \quad \det(A) = (-1)^s \prod_{i=1}^n u_{ii}.$$

当某一步出现 $u_{ii} = 0$ 时，便说明矩阵奇异 (singular)，此时 $\det(A) = 0$ 。

2.3.5 特殊矩阵分解

2.3.5.1 LDL^T 分解

若 A 是对称正定矩阵 (symmetric positive definite matrix, SPD)，则它不仅可以做一般的 LU 分解，还能进一步写成

$$A = LDL^T,$$

其中 L 是单位下三角矩阵， D 是对角元为正数的对角矩阵。这种分解把“缩放”与“几何方向”分离开来，在理论分析和实际编程中都很方便。

把

$$a_{ij} = \sum_{k=1}^{\min(i,j)} l_{ik} d_k l_{jk}$$

按列递推, 可得

$$d_i = a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2 d_k,$$

$$l_{ji} = \frac{1}{d_i} \left(a_{ji} - \sum_{k=1}^{i-1} l_{jk} l_{ik} d_k \right), \quad j > i.$$

 **笔记** LDL^T 分解不需要开平方运算, 因此在某些硬件环境或符号推导中更方便; 而若进一步把 D 的平方根吸收到 L 中, 就得到更常见的 Cholesky 分解。

2.3.5.2 Cholesky 分解

对 SPD 矩阵, Cholesky 分解写成

$$A = LL^T,$$

其中 L 为下三角矩阵, 且对角元取正。把

$$A = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

逐项相乘, 可得

$$a_{ii} = \sum_{k=1}^i l_{ik}^2, \quad a_{ji} = \sum_{k=1}^i l_{jk} l_{ik} \quad (j > i).$$

于是递推公式为

$$l_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2},$$

$$l_{ji} = \frac{a_{ji} - \sum_{k=1}^{i-1} l_{jk} l_{ik}}{l_{ii}}, \quad j > i.$$

结论 与一般 LU 分解相比, Cholesky 只需要存储和计算一半矩阵信息, 因此在 SPD 问题上更省时、更省内存。这也是它在最小二乘正规方程、协方差矩阵、有限元离散系统中广泛使用的原因。

2.3.5.3 三对角矩阵与 Thomas 算法

三对角矩阵大量出现于一维边值问题、样条插值和许多差分离散中。设

$$A = \begin{bmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \cdots & 0 \\ 0 & a_3 & b_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & c_{n-1} \\ 0 & 0 & 0 & a_n & b_n \end{bmatrix}.$$

它的 LU 分解可写成

$$A = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ \ell_2 & 1 & 0 & \cdots & 0 \\ 0 & \ell_3 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & 0 \\ 0 & 0 & 0 & \ell_n & 1 \end{bmatrix} \begin{bmatrix} \beta_1 & c_1 & 0 & \cdots & 0 \\ 0 & \beta_2 & c_2 & \cdots & 0 \\ 0 & 0 & \beta_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & c_{n-1} \\ 0 & 0 & 0 & 0 & \beta_n \end{bmatrix},$$

其中递推关系为

$$\beta_1 = b_1, \quad \ell_i = \frac{a_i}{\beta_{i-1}}, \quad \beta_i = b_i - \ell_i c_{i-1} \quad (i = 2, \dots, n).$$

若要求解

$$Ax = d,$$

则先做前代:

$$y_1 = d_1, \quad y_i = d_i - \ell_i y_{i-1} \quad (i = 2, \dots, n),$$

再做回代:

$$x_n = \frac{y_n}{\beta_n}, \quad x_i = \frac{y_i - c_i x_{i+1}}{\beta_i} \quad (i = n-1, \dots, 1).$$

注 这一算法通常称为 **Thomas 算法**。由于每一步只做常数次运算，它的总复杂度是

$$O(n),$$

远小于一般稠密矩阵消元的 $O(n^3)$ 。

例题 2.3 一维 poisson 方程

$$-v''(x) = f(x)$$

二阶微分:

$$-\frac{v_{i+1} - 2v_i + v_{i-1}}{h^2} = f_i, \quad i = 1, 2, \dots, n$$

假设边界条件: $v_0 = v_{n+1} = 0$ 可以得到一个三对角矩阵的线性方程组:

$$\begin{bmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ 0 & -1 & 2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & -1 \\ 0 & 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_n \end{bmatrix} = h^2 \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ \vdots \\ f_n \end{bmatrix}$$

此时 $a_i = c_i = -1$, $b_i = 2$,

所以 Thomas 算法中的递推简化为

$$\beta_1 = 2, \quad \beta_i = 2 - \frac{1}{\beta_{i-1}} \quad (i \geq 2),$$

而前代、回代也都只涉及相邻两个网格点。正因为这种结构极度稀疏，离散 Poisson 方程才可以在线性复杂度下快速求解。

2.4 迭代求解法 (Iterative Methods)

在上一节讨论直接法 (direct methods) 时，我们通常通过高斯消元、LU 分解等方式在有限步内求得线性方程组

$$Ax = b$$

的精确解 (忽略舍入误差时)。但当线性方程组维数很大、矩阵稀疏 (sparse)、或者只需要中等精度近似解时，直接法往往面临两个问题:

1. 存储量大，尤其对于高维稀疏矩阵，消元过程中会产生大量填充 (fill-in);
2. 计算代价高，而很多实际问题只需要“逐步逼近”的数值解即可。

因此，自然会想到将求解问题改写成一个**逐次逼近 (successive approximation)** 的问题: 先给定一个初值 $x^{(0)}$ ，再不断构造

$$x^{(1)}, x^{(2)}, \dots, x^{(k)}, \dots$$

使得当 $k \rightarrow \infty$ 时,

$$\mathbf{x}^{(k)} \rightarrow \mathbf{x},$$

其中 $\mathbf{x}$ 为原方程组的精确解。

定义 2.2

若线性方程组 $A\mathbf{x} = \mathbf{b}$ 被改写为

$$\mathbf{x} = B\mathbf{x} + \mathbf{f},$$

并由此构造迭代序列

$$\mathbf{x}^{(k)} = B\mathbf{x}^{(k-1)} + \mathbf{f}, \quad k = 1, 2, \dots,$$

则称该方法为定常迭代法 (stationary iterative method), 其中 B 称为迭代矩阵 (iteration matrix)。

2.4.1 从直接迭代到矩阵分裂

最朴素的想法是把

$$A\mathbf{x} = \mathbf{b}$$

直接改写为

$$\mathbf{x} = (I - A)\mathbf{x} + \mathbf{b}.$$

于是得到最简单的“直接迭代”

$$\mathbf{x}^{(k)} = (I - A)\mathbf{x}^{(k-1)} + \mathbf{b}.$$

它的形式虽然很简单, 但通常收敛性并不好, 因为它要求

$$\rho(I - A) < 1,$$

这里 $\rho(\cdot)$ 表示谱半径 (spectral radius), 这个条件对一般矩阵并不容易满足。

因此, 实际中更常见的做法不是直接使用 $I - A$, 而是对矩阵 A 做更合理的矩阵分裂 (matrix splitting)。

定义 2.3

若将矩阵 A 写成

$$A = M - N,$$

其中 M 可逆, 则原方程组

$$A\mathbf{x} = \mathbf{b}$$

可改写为

$$M\mathbf{x} = N\mathbf{x} + \mathbf{b},$$

进而得到迭代格式

$$\mathbf{x}^{(k)} = M^{-1}N\mathbf{x}^{(k-1)} + M^{-1}\mathbf{b}.$$

此时迭代矩阵为

$$B = M^{-1}N.$$

因此, 构造一个好的迭代法, 本质上就是: 把 A 分裂成一个容易求逆的部分 M 与其余部分 N , 使得求逆代价小, 同时 $\rho(M^{-1}N)$ 尽量小, 从而保证快速收敛。

 **笔记** 后面会看到, Jacobi、Gauss-Seidel、SOR 三种方法都属于这种“矩阵分裂 + 不动点迭代”的统一框架。

2.4.2 Jacobi 迭代 (Jacobi Iteration)

2.4.2.1 由分量方程出发的推导

考虑线性方程组

$$A\mathbf{x} = \mathbf{b}, \quad A = (a_{ij})_{n \times n}, \quad \mathbf{x} = (x_1, \dots, x_n)^T, \quad \mathbf{b} = (b_1, \dots, b_n)^T.$$

其第 i 个分量方程为

$$a_{i1}x_1 + a_{i2}x_2 + \dots + a_{ii}x_i + \dots + a_{in}x_n = b_i.$$

若 $a_{ii} \neq 0$, 可将其解出 x_i :

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j \right).$$

这就是 Jacobi 迭代的起点。若在第 k 次迭代中, 右端全部使用上一步 $\mathbf{x}^{(k-1)}$ 的分量, 则得到

$$x_i^{(k)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j^{(k-1)} \right), \quad i = 1, 2, \dots, n.$$

也可等价写成

$$x_i^{(k)} = \frac{1}{a_{ii}} \left[\sum_{\substack{j=1 \\ j \neq i}}^n (-a_{ij}x_j^{(k-1)}) + b_i \right].$$

注 Jacobi 方法的核心特征是: 第 k 步中所有新分量 $x_i^{(k)}$ 都只依赖于旧向量 $\mathbf{x}^{(k-1)}$ 。因此各分量可以并行计算 (parallelizable), 这是它的重要优点。

2.4.2.2 矩阵形式

我们把矩阵 A 分解为

$$A = D - L - U,$$

其中

$$D = \begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{pmatrix},$$

而 L 与 U 分别表示严格下三角与严格上三角部分, 这里把它们的非零元写成 $-a_{ij}$ 的形式。于是

$$A\mathbf{x} = \mathbf{b}$$

等价于

$$(D - L - U)\mathbf{x} = \mathbf{b},$$

即

$$D\mathbf{x} = (L + U)\mathbf{x} + \mathbf{b}.$$

两边左乘 D^{-1} , 得到

$$\mathbf{x} = D^{-1}(L + U)\mathbf{x} + D^{-1}\mathbf{b}.$$

因此 Jacobi 迭代可写成

$$\mathbf{x}^{(k)} = D^{-1}(L + U)\mathbf{x}^{(k-1)} + D^{-1}\mathbf{b}, \quad k = 1, 2, \dots$$

所以 Jacobi 迭代矩阵为

$$B_J = D^{-1}(L + U).$$

2.4.2.3 误差传播与收敛条件

设精确解为 $\mathbf{x}$, 定义误差

$$\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}.$$

由于精确解也满足

$$\mathbf{x} = B_J\mathbf{x} + D^{-1}\mathbf{b},$$

两式相减可得

$$\mathbf{e}^{(k)} = B_J\mathbf{e}^{(k-1)}.$$

继续递推,

$$\mathbf{e}^{(k)} = B_J^k\mathbf{e}^{(0)}.$$

因此 Jacobi 方法收敛的充分必要条件是

$$\rho(B_J) < 1.$$

定理 2.1 (定常迭代法的收敛判据)

对于定常迭代

$$\mathbf{x}^{(k)} = B\mathbf{x}^{(k-1)} + \mathbf{f},$$

它对任意初值 $\mathbf{x}^{(0)}$ 收敛到唯一不动点, 当且仅当

$$\rho(B) < 1.$$

在实际中, 直接计算谱半径并不总是方便, 因此常用一些充分条件。对于 Jacobi 方法, 一个常见的充分条件是矩阵 A **严格对角占优 (strictly diagonally dominant)**, 即对每一行都有

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|.$$

此时 Jacobi 迭代一定收敛。



笔记 从数值直觉看, 严格对角占优意味着每个未知量 x_i 在第 i 个方程中的“自耦合”最强, 其他未知量对它的干扰相对较弱, 因此用旧值来近似右端时不会引入过大的误差。

2.4.2.4 终止准则

$$\frac{\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|_{\infty}}{\|\mathbf{x}^{(k)}\|_{\infty}} < 10^{-3},$$

其中

$$\|\mathbf{y}\|_{\infty} = \max_i |y_i|$$

称为无穷范数 (infinity norm)。

这个准则衡量的是相邻两次迭代结果的相对变化已经足够小。它简单直观, 但要注意: “相邻两次结果变化

小”并不总等价于“已经接近精确解”。因此在实际编程中，除了这种差分准则外，也常监控残差 (residual)

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)},$$

例如要求

$$\frac{\|\mathbf{r}^{(k)}\|_{\infty}}{\|\mathbf{b}\|_{\infty}} < \varepsilon.$$

2.4.3 Gauss–Seidel 迭代 (Gauss–Seidel Iteration)

2.4.3.1 基本思想：立即使用新值

Jacobi 方法在计算 $x_i^{(k)}$ 时，右端全部使用旧值 $\mathbf{x}^{(k-1)}$ 。但若我们按 $i = 1, 2, \dots, n$ 的顺序逐个更新，就会发现：当计算到第 i 个分量时，前面的

$$x_1^{(k)}, x_2^{(k)}, \dots, x_{i-1}^{(k)}$$

已经算出来了，完全可以直接代入使用，而不必继续使用旧值。

于是得到 Gauss–Seidel 迭代公式：

$$x_i^{(k)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} \right], \quad i = 1, 2, \dots, n.$$

这就是 Gauss–Seidel 的逐分量更新公式。

注 Gauss–Seidel 与 Jacobi 的本质区别就在于：

- Jacobi：旧解代入，所有分量同时更新；
- Gauss–Seidel：新解立即代入，按顺序逐个更新。

这使得 Gauss–Seidel 往往比 Jacobi 收敛更快，但并行性较差。

2.4.3.2 矩阵形式

仍使用分裂

$$A = D - L - U.$$

则

$$(D - L - U)\mathbf{x} = \mathbf{b}$$

可整理为

$$(D - L)\mathbf{x} = U\mathbf{x} + \mathbf{b}.$$

因此 Gauss–Seidel 迭代写作

$$(D - L)\mathbf{x}^{(k)} = U\mathbf{x}^{(k-1)} + \mathbf{b},$$

即

$$\mathbf{x}^{(k)} = (D - L)^{-1}U\mathbf{x}^{(k-1)} + (D - L)^{-1}\mathbf{b}.$$

故其迭代矩阵为

$$B_{GS} = (D - L)^{-1}U.$$

2.4.3.3 收敛性质

同样地，Gauss–Seidel 的误差满足

$$\mathbf{e}^{(k)} = B_{GS}\mathbf{e}^{(k-1)},$$

故收敛条件是

$$\rho(B_{GS}) < 1.$$

在实际理论中, Gauss–Seidel 的收敛性通常比 Jacobi 更好。常见结论包括:

1. 若 A 严格对角占优, 则 Jacobi 与 Gauss–Seidel 都收敛;
2. 若 A 是对称正定矩阵 (symmetric positive definite, SPD), 则 Gauss–Seidel 收敛。

结论 在多数常见情形下, Gauss–Seidel 收敛速度通常优于 Jacobi, 因为它在每一步中更充分地利用了最新信息。

2.4.4 SOR 迭代 (Successive Over-Relaxation)

2.4.4.1 引入松弛参数的动机

既然 Gauss–Seidel 的“新值立即代入”能够加快收敛, 那么一个自然问题是: 我们能否在更新时再“迈大一点步”, 以更快逼近精确解?

这就引出了**超松弛法 (Successive Over-Relaxation, SOR)**。它在 Gauss–Seidel 新值的基础上再加入一个参数 ω , 令

$$x_i^{(k)} = (1 - \omega)x_i^{(k-1)} + \frac{\omega}{a_{ii}} \left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^n a_{ij}x_j^{(k-1)} \right].$$

这就是 SOR 的标准逐分量更新公式。

2.4.4.2 参数 ω 的含义

- 当 $\omega = 1$ 时, SOR 退化为 Gauss–Seidel;
- 当 $0 < \omega < 1$ 时, 称为**欠松弛 (under-relaxation)**, 步子更保守;
- 当 $1 < \omega < 2$ 时, 称为**超松弛 (over-relaxation)**, 常常可能更快;
- 若 ω 取值不合适, 也可能导致发散。

因此, SOR 的思想可以理解为: 先用 Gauss–Seidel 给出一个“自然更新方向”, 再用参数 ω 控制沿这个方向前进的幅度。

2.4.4.3 矩阵形式

SOR 的矩阵形式可写为

$$(D - \omega L)\mathbf{x}^{(k)} = [(1 - \omega)D + \omega U]\mathbf{x}^{(k-1)} + \omega \mathbf{b},$$

即

$$\mathbf{x}^{(k)} = (D - \omega L)^{-1}[(1 - \omega)D + \omega U]\mathbf{x}^{(k-1)} + \omega(D - \omega L)^{-1}\mathbf{b}.$$

故其迭代矩阵为

$$B_\omega = (D - \omega L)^{-1}[(1 - \omega)D + \omega U].$$

2.4.4.4 收敛性与参数选择

SOR 的收敛条件同样是

$$\rho(B_\omega) < 1.$$

在理论上, 如果 A 是对称正定矩阵, 则当

$$0 < \omega < 2$$

时, SOR 迭代收敛。但“最优” ω 并没有统一公式, 它依赖于矩阵本身的谱性质。

 **笔记** 实际计算中, ω 常通过经验、实验扫描, 或结合问题结构来选取。合适的 ω 能显著加快收敛; 不合适的 ω 则可能比 Gauss-Seidel 还慢, 甚至直接发散。

2.4.5 三种方法的统一比较

将三种方法放在一起看, 可以清楚地看到它们之间的联系:

2.4.5.1 Jacobi

$$\mathbf{x}^{(k)} = D^{-1}(L + U)\mathbf{x}^{(k-1)} + D^{-1}\mathbf{b}.$$

特点:

- 全部使用旧值;
- 容易并行实现;
- 单步计算简单;
- 通常收敛较慢。

2.4.5.2 Gauss-Seidel

$$\mathbf{x}^{(k)} = (D - L)^{-1}U\mathbf{x}^{(k-1)} + (D - L)^{-1}\mathbf{b}.$$

特点:

- 立即使用新值;
- 通常比 Jacobi 更快;
- 顺序性更强, 不如 Jacobi 易并行。

2.4.5.3 SOR

$$\mathbf{x}^{(k)} = (D - \omega L)^{-1}[(1 - \omega)D + \omega U]\mathbf{x}^{(k-1)} + \omega(D - \omega L)^{-1}\mathbf{b}.$$

特点:

- 是 Gauss-Seidel 的加速版本;
- 依赖参数 ω ;
- 参数合适时可明显加快收敛。

方法	是否用新值	是否易并行	一般收敛速度
Jacobi	否	是	较慢
Gauss-Seidel	是	较差	中等
SOR	是 (并加权)	较差	可能更快

2.4.6 一个简单例子

考虑线性方程组

$$\begin{cases} 2x_1 + x_2 = 5, \\ x_1 + 3x_2 = 6. \end{cases}$$

其精确解为

$$x_1 = \frac{9}{5}, \quad x_2 = \frac{7}{5}.$$

2.4.6.1 Jacobi 迭代

把方程组改写为

$$x_1 = \frac{5 - x_2}{2}, \quad x_2 = \frac{6 - x_1}{3}.$$

于是 Jacobi 迭代为

$$x_1^{(k)} = \frac{5 - x_2^{(k-1)}}{2}, \quad x_2^{(k)} = \frac{6 - x_1^{(k-1)}}{3}.$$

若取初值

$$\mathbf{x}^{(0)} = (0, 0)^T,$$

则

$$\mathbf{x}^{(1)} = \left(\frac{5}{2}, 2\right)^T,$$

$$\mathbf{x}^{(2)} = \left(\frac{3}{2}, \frac{7}{6}\right)^T,$$

$$\mathbf{x}^{(3)} = \left(\frac{23}{12}, \frac{3}{2}\right)^T,$$

可见它在解附近来回摆动并逐渐收敛。

2.4.6.2 Gauss-Seidel 迭代

若使用 Gauss-Seidel, 则

$$x_1^{(k)} = \frac{5 - x_2^{(k-1)}}{2}, \quad x_2^{(k)} = \frac{6 - x_1^{(k)}}{3}.$$

取同样初值 $\mathbf{x}^{(0)} = (0, 0)^T$, 得到

$$x_1^{(1)} = \frac{5}{2}, \quad x_2^{(1)} = \frac{7}{6},$$

所以

$$\mathbf{x}^{(1)} = \left(\frac{5}{2}, \frac{7}{6}\right)^T.$$

继续迭代,

$$x_1^{(2)} = \frac{5 - \frac{7}{6}}{2} = \frac{23}{12}, \quad x_2^{(2)} = \frac{6 - \frac{23}{12}}{3} = \frac{49}{36}.$$

可以看出, Gauss-Seidel 在同样初值下通常更快逼近精确解。

例题 2.4 这个例子非常适合帮助理解两种方法的区别:

Jacobi 在每一步中“所有分量一起更新”, 所以新值只来自旧值; 而 Gauss-Seidel 是“边算边用”, 后面的分量能立刻利用前面刚算出的新信息, 因此往往更高效。

2.4.7 实现时的几个注意事项

1. **对角元不能为零**: Jacobi、Gauss-Seidel、SOR 都需要除以 a_{ii} , 因此要求 $a_{ii} \neq 0$ 。若某个对角元为零, 通常需要先交换方程顺序或变量顺序。
2. **初值会影响收敛过程**: 虽然若方法收敛则对任意初值都会趋于同一解, 但不同初值可能使前期收敛快慢差别很大。

3. 应同时关注残差与相邻步差：只看

$$\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|$$

有时会误判，因此最好也检查

$$\|\mathbf{b} - A\mathbf{x}^{(k)}\|.$$

4. 矩阵重排可能改善收敛性：适当调整方程顺序，使对角元更大、更接近对角占优，常常有助于迭代稳定。
5. 稀疏矩阵中特别有优势：因为每一步只涉及矩阵与向量的局部运算，而不需要像直接法那样引入大量填充项。

2.5 共轭梯度法 (Conjugate Gradient Method)

上一节介绍的 Jacobi、Gauss-Seidel 与 SOR 都属于定常迭代法 (stationary iterative methods)。它们实现简单，但对于大规模线性方程组，尤其是当矩阵为稀疏对称正定矩阵 (symmetric positive-definite matrix, SPD) 时，还可以构造更高效的迭代方法，这就是共轭梯度法 (conjugate gradient method, CG)。

共轭梯度法的核心思想并不是“逐点修正一个分量”，而是：

1. 先把线性方程组

$$A\mathbf{x} = \mathbf{b}$$

转化为一个二次函数的极小化问题；

2. 再在一系列特殊选择的搜索方向上做一维最优化；
3. 这些方向彼此并非普通意义下的正交，而是关于矩阵 A 的共轭 (conjugate)；
4. 由于方向选择得足够“聪明”，在精确算术下， n 维问题至多 n 步即可得到精确解。

 **笔记** 共轭梯度法最经典、最标准的适用场景是：

$$A \in \mathbb{R}^{n \times n} \text{ 对称正定.}$$

对这一类矩阵，它既有清晰的几何解释，也有很强的计算效率，因此在科学计算、偏微分方程离散、机器学习与优化问题中都极为重要。

2.5.1 从线性方程组到极小化问题

考虑线性方程组

$$A\mathbf{x} = \mathbf{b},$$

其中 A 为 $n \times n$ 的对称正定矩阵。定义二次函数

$$g(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}.$$

由于 A 对称，容易求得梯度

$$\nabla g(\mathbf{x}) = A\mathbf{x} - \mathbf{b}.$$

因此

$$\nabla g(\mathbf{x}) = 0 \iff A\mathbf{x} = \mathbf{b}.$$

定理 2.2

若 A 是对称正定矩阵，则函数

$$g(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x}$$

是严格凸函数 (strictly convex function)，因此它存在唯一极小点，而这个极小点恰好就是线性方程组

$$A\mathbf{x} = \mathbf{b}$$

的唯一解。



证明 因为 A 对称正定，所以对任意非零向量 $\mathbf{y}$ 都有

$$\mathbf{y}^T A \mathbf{y} > 0.$$

这意味着 $g(\mathbf{x})$ 的 Hessian 矩阵就是 A ，且正定，因此 g 严格凸。严格凸函数的驻点必为唯一全局极小点。而驻点条件

$$\nabla g(\mathbf{x}) = A\mathbf{x} - \mathbf{b} = 0$$

正是方程组 $A\mathbf{x} = \mathbf{b}$ 。

注 这一视角非常重要。它告诉我们：求解 $A\mathbf{x} = \mathbf{b}$ 并不一定要从代数消元出发，也可以从优化问题出发。共轭梯度法本质上就是利用这一优化结构来设计迭代步骤。

2.5.2 沿给定方向的一维极小化

设当前已有一个近似解 $\mathbf{x}$ ，并选定一个非零方向 $\mathbf{v} \neq \mathbf{0}$ 。考虑沿直线

$$\mathbf{x} + t\mathbf{v}$$

上的函数值变化，其中 $t \in \mathbb{R}$ 为实参数。定义

$$\phi(t) = g(\mathbf{x} + t\mathbf{v}).$$

代入 g 的定义得

$$\phi(t) = \frac{1}{2}(\mathbf{x} + t\mathbf{v})^T A(\mathbf{x} + t\mathbf{v}) - \mathbf{b}^T(\mathbf{x} + t\mathbf{v}).$$

展开可得

$$\phi(t) = g(\mathbf{x}) + t\mathbf{v}^T(A\mathbf{x} - \mathbf{b}) + \frac{1}{2}t^2\mathbf{v}^T A \mathbf{v}.$$

由于 A 正定，故

$$\mathbf{v}^T A \mathbf{v} > 0 \quad (\mathbf{v} \neq \mathbf{0}),$$

因此 $\phi(t)$ 是关于 t 的严格凸二次函数，故存在唯一极小值点。令

$$\phi'(t) = 0,$$

得到最优步长

$$\hat{t} = \frac{\mathbf{v}^T(\mathbf{b} - A\mathbf{x})}{\mathbf{v}^T A \mathbf{v}}.$$

命题 2.1

对于任意当前点 $\mathbf{x}$ 与任意非零方向 $\mathbf{v}$ ，沿 $\mathbf{v}$ 的最佳一步更新为

$$\mathbf{x}_{\text{new}} = \mathbf{x} + \frac{\mathbf{v}^T(\mathbf{b} - A\mathbf{x})}{\mathbf{v}^T A \mathbf{v}} \mathbf{v}.$$

若 $\mathbf{v}^T(\mathbf{b} - A\mathbf{x}) \neq 0$ ，则该更新必使目标函数值严格下降。



注 这一步就是精确线搜索 (exact line search)。共轭梯度法、最速下降法 (steepest descent) 都以此为核心，只是它们对“方向 $\mathbf{v}$ 如何选取”的策略不同。

2.5.3 残差向量与最速下降法

定义当前近似解 $\mathbf{x}$ 的残差向量 (residual vector)

$$\mathbf{r} = \mathbf{b} - A\mathbf{x}.$$

注意到

$$\nabla g(\mathbf{x}) = A\mathbf{x} - \mathbf{b} = -\mathbf{r},$$

所以

$$-\nabla g(\mathbf{x}) = \mathbf{r}.$$

也就是说，残差正好就是函数 g 的**最速下降方向** (steepest descent direction)。

因此，若在每一步都取搜索方向为当前残差，

$$\mathbf{v}^{(k+1)} = \mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)},$$

再配合上一小节的一维极小化步长，就得到**最速下降法** (method of steepest descent)。

设当前迭代点为 $\mathbf{x}^{(k-1)}$ ，方向为 $\mathbf{v}^{(k)}$ ，则步长取为

$$t_k = \frac{\langle \mathbf{v}^{(k)}, \mathbf{b} - A\mathbf{x}^{(k-1)} \rangle}{\langle \mathbf{v}^{(k)}, A\mathbf{v}^{(k)} \rangle},$$

更新为

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + t_k \mathbf{v}^{(k)}.$$

若选

$$\mathbf{v}^{(k)} = \mathbf{r}^{(k-1)},$$

便得到最速下降法。



笔记 最速下降法每一步都沿局部最陡方向前进，听起来似乎应该非常高效，但实际上往往并不理想。特别是当二次函数的等高线非常狭长时，迭代路径会出现明显的“之”字形震荡 (zig-zag)，收敛速度并不快。

2.5.4 为什么需要“共轭方向”

为了说明 CG 的思想，先提出一个更强的问题：是否能找到一组非零方向

$$\{\mathbf{v}^{(1)}, \mathbf{v}^{(2)}, \dots, \mathbf{v}^{(n)}\},$$

使得沿这些方向依次做精确线搜索后，最多 n 步就能到达精确解？

答案是可以的，但这些方向必须满足一种特殊的“正交性”，即关于 A 的共轭性。

定义 2.4

若非零向量 $\mathbf{u}, \mathbf{v}$ 满足

$$\mathbf{u}^T A \mathbf{v} = 0,$$

则称 $\mathbf{u}$ 与 $\mathbf{v}$ 关于 A 共轭 (A-conjugate)，或称 A -正交。

定义 2.5

若一组非零向量

$$\mathbf{v}^{(1)}, \mathbf{v}^{(2)}, \dots, \mathbf{v}^{(n)}$$

满足

$$(\mathbf{v}^{(i)})^T A \mathbf{v}^{(j)} = 0, \quad i \neq j,$$

则称它们为一组共轭方向 (conjugate directions)。

定理 2.3

若 A 对称正定, 且

$$\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(n)}$$

是一组两两 A -共轭的非零向量, 则它们线性无关, 从而构成 $\mathbb{R}^n$ 的一组基。若沿这些方向依次做精确线搜索, 则在精确算术下至多 n 步可得到方程组的精确解。



证明 [证明思路] 因为 A 正定, 故

$$(\mathbf{v}^{(i)})^T A \mathbf{v}^{(i)} > 0.$$

若存在非平凡线性组合

$$\sum_{i=1}^n c_i \mathbf{v}^{(i)} = \mathbf{0},$$

两边左乘 $(\mathbf{v}^{(j)})^T A$, 利用共轭性得

$$c_j (\mathbf{v}^{(j)})^T A \mathbf{v}^{(j)} = 0,$$

从而 $c_j = 0$ 。因此各向量线性无关, 构成一组基。

进一步, 由于解误差 $\mathbf{e}^{(0)} = \mathbf{x}_* - \mathbf{x}^{(0)}$ 可在这组基下唯一展开, 逐次做精确线搜索相当于一步步消去误差在各个共轭方向上的分量, 因此至多 n 步达到精确解。

注 这一定理极其关键: 它告诉我们, 真正的难点并不在“每一步沿哪个方向做极小化”, 而在“如何动态构造一组良好的共轭方向”。共轭梯度法正是用残差递推地构造出这组方向, 而不需要预先知道整个基。

2.5.5 共轭梯度法的基本递推关系

2.5.5.1 方向递推的想法

设第 k 步的搜索方向为 $\mathbf{v}^{(k)}$ 。共轭梯度法希望:

1. 当前新方向应尽量朝着当前残差方向前进, 因为残差仍然代表下降信息;
2. 但新方向又不能简单等于残差, 否则就会退化成最速下降法;
3. 因此最自然的做法是: 让新方向由**当前残差**与**上一方向**线性组合得到。

于是设

$$\mathbf{v}^{(k)} = \mathbf{r}^{(k-1)} + s_{k-1} \mathbf{v}^{(k-1)}.$$

其中 s_{k-1} 由共轭条件确定。要求

$$(\mathbf{v}^{(k-1)})^T A \mathbf{v}^{(k)} = 0.$$

代入上式得

$$(\mathbf{v}^{(k-1)})^T A \mathbf{r}^{(k-1)} + s_{k-1} (\mathbf{v}^{(k-1)})^T A \mathbf{v}^{(k-1)} = 0.$$

故

$$s_{k-1} = -\frac{(\mathbf{v}^{(k-1)})^T A \mathbf{r}^{(k-1)}}{(\mathbf{v}^{(k-1)})^T A \mathbf{v}^{(k-1)}}.$$

2.5.5.2 步长公式

对于给定方向 $\mathbf{v}^{(k)}$, 第 k 步的最优步长仍由一维极小化得到:

$$t_k = \frac{(\mathbf{v}^{(k)})^T \mathbf{r}^{(k-1)}}{(\mathbf{v}^{(k)})^T A \mathbf{v}^{(k)}}.$$

更新点为

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + t_k \mathbf{v}^{(k)}.$$

于是残差递推为

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)} = \mathbf{b} - A(\mathbf{x}^{(k-1)} + t_k \mathbf{v}^{(k)}) = \mathbf{r}^{(k-1)} - t_k A\mathbf{v}^{(k)}.$$

2.5.5.3 正交性简化

CG 中有一个非常重要的性质：不同步的残差彼此正交，即

$$(\mathbf{r}^{(i)})^T \mathbf{r}^{(j)} = 0, \quad i \neq j.$$

此外，

$$(\mathbf{r}^{(k)})^T \mathbf{v}^{(j)} = 0, \quad j = 1, 2, \dots, k.$$

利用这些关系，可以把上面的步长与方向系数进一步化简。

首先，

$$\mathbf{v}^{(k)} = \mathbf{r}^{(k-1)} + s_{k-1} \mathbf{v}^{(k-1)},$$

两边与 $\mathbf{r}^{(k-1)}$ 做内积，利用

$$(\mathbf{v}^{(k-1)})^T \mathbf{r}^{(k-1)} = 0,$$

得到

$$(\mathbf{v}^{(k)})^T \mathbf{r}^{(k-1)} = (\mathbf{r}^{(k-1)})^T \mathbf{r}^{(k-1)}.$$

因此

$$t_k = \frac{(\mathbf{r}^{(k-1)})^T \mathbf{r}^{(k-1)}}{(\mathbf{v}^{(k)})^T A\mathbf{v}^{(k)}}.$$

再进一步，通过残差与共轭条件的递推关系，可得

$$s_k = \frac{(\mathbf{r}^{(k)})^T \mathbf{r}^{(k)}}{(\mathbf{r}^{(k-1)})^T \mathbf{r}^{(k-1)}}.$$

这就是最常用的 Fletcher-Reeves 型线性 CG 递推公式。

性质 共轭梯度法中，以下三类性质同时成立：

1. 残差正交性

$$(\mathbf{r}^{(i)})^T \mathbf{r}^{(j)} = 0, \quad i \neq j;$$

2. 方向共轭性

$$(\mathbf{v}^{(i)})^T A\mathbf{v}^{(j)} = 0, \quad i \neq j;$$

3. 残差对旧方向正交

$$(\mathbf{r}^{(k)})^T \mathbf{v}^{(j)} = 0, \quad j = 1, 2, \dots, k.$$

2.5.6 共轭梯度算法的标准形式

把上面的结论汇总，可得到最常用的 CG 算法。

定义 2.6 (共轭梯度法标准迭代)

给定初值 $\mathbf{x}^{(0)}$ ，先定义

$$\mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)}, \quad \mathbf{v}^{(1)} = \mathbf{r}^{(0)}.$$

然后对 $k = 1, 2, \dots$ 迭代:

$$t_k = \frac{(\mathbf{r}^{(k-1)})^T \mathbf{r}^{(k-1)}}{(\mathbf{v}^{(k)})^T A \mathbf{v}^{(k)}},$$

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + t_k \mathbf{v}^{(k)},$$

$$\mathbf{r}^{(k)} = \mathbf{r}^{(k-1)} - t_k A \mathbf{v}^{(k)},$$

若满足终止准则则停止; 否则令

$$s_k = \frac{(\mathbf{r}^{(k)})^T \mathbf{r}^{(k)}}{(\mathbf{r}^{(k-1)})^T \mathbf{r}^{(k-1)}},$$

并更新方向

$$\mathbf{v}^{(k+1)} = \mathbf{r}^{(k)} + s_k \mathbf{v}^{(k)}.$$

 **笔记** 这套公式看起来与最速下降法非常相似, 但最本质的差别是:

$$\mathbf{v}^{(k+1)} \neq \mathbf{r}^{(k)},$$

而是

$$\mathbf{v}^{(k+1)} = \mathbf{r}^{(k)} + s_k \mathbf{v}^{(k)}.$$

也就是说, 新方向不仅包含当前的残差信息, 还继承了上一轮已经建立好的共轭结构。因此 CG 比最速下降法显著更快。

2.5.7 几何解释与 Krylov 子空间

共轭梯度法中的搜索方向 $\mathbf{v}_i$ 与残差 $\mathbf{r}_i$ 张成同一个 Krylov 子空间 (Krylov subspace)。

定义 2.7

给定矩阵 A 与向量 $\mathbf{r}_0$, 第 k 阶 Krylov 子空间定义为

$$\mathcal{K}_k(A, \mathbf{r}_0) = \text{span}\{\mathbf{r}_0, A\mathbf{r}_0, A^2\mathbf{r}_0, \dots, A^{k-1}\mathbf{r}_0\}.$$

CG 的核心结构可以概括为:

1. $\mathbf{r}_0, \mathbf{r}_1, \dots$ 是关于普通内积的正交基方向;
2. $\mathbf{v}_1, \mathbf{v}_2, \dots$ 是关于 A -内积

$$\langle \mathbf{u}, \mathbf{v} \rangle_A = \mathbf{u}^T A \mathbf{v}$$

的正交基方向;

3. 它们张成同一个 Krylov 子空间;
4. 第 k 步近似解 $\mathbf{x}^{(k)}$ 可以看成真解在仿射空间

$$\mathbf{x}^{(0)} + \mathcal{K}_k(A, \mathbf{r}^{(0)})$$

上的最佳投影。

更准确地说, $\mathbf{x}^{(k)}$ 是如下极小化问题的解:

$$\mathbf{x}^{(k)} = \arg \min_{\mathbf{x} \in \mathbf{x}^{(0)} + \mathcal{K}_k(A, \mathbf{r}^{(0)})} g(\mathbf{x}).$$

注 这说明 CG 并不是“瞎迭代”, 而是在逐步扩大的 Krylov 子空间中做最优逼近。每一步都利用了比前一步更多的矩阵信息, 因此效率远优于只看局部梯度的最速下降法。

2.5.8 收敛性与条件数的影响

在精确算术下, 若 A 是 $n \times n$ 的 SPD 矩阵, 则 CG 至多 n 步精确收敛。但在浮点运算中, 由于舍入误差存在, 通常不会真的“刚好 n 步结束”, 因此更常用的是误差估计与残差判据。

设 A -范数定义为

$$\|\mathbf{e}\|_A = \sqrt{\mathbf{e}^T A \mathbf{e}}.$$

则 CG 有经典估计

$$\|\mathbf{x}_* - \mathbf{x}^{(k)}\|_A \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k \|\mathbf{x}_* - \mathbf{x}^{(0)}\|_A,$$

其中

$$\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}$$

是矩阵 A 的条件数 (condition number)。

结论 CG 的收敛速度与条件数密切相关: $\kappa(A)$ 越小, 收敛越快; $\kappa(A)$ 越大, 收敛越慢。这正是后面引入预条件 (preconditioning) 思想的根本动机。

2.5.9 一个 2×2 的具体例子

为了看清算法的实际运算过程, 考虑

$$A = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \mathbf{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

矩阵 A 显然对称, 且

$$4 > 0, \quad \det(A) = 12 - 1 = 11 > 0,$$

故 A 对称正定, 可用共轭梯度法。

精确解可由直接法求得:

$$\begin{cases} 4x_1 + x_2 = 1, \\ x_1 + 3x_2 = 2, \end{cases}$$

解为

$$\mathbf{x}_* = \begin{pmatrix} \frac{1}{11} \\ \frac{7}{11} \end{pmatrix}.$$

例题 2.5 用 CG 法求解该方程组

解 由初值

$$\mathbf{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

出发, 先算初始残差

$$\mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

于是初始方向

$$\mathbf{v}^{(1)} = \mathbf{r}^{(0)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

第 1 步先计算

$$A\mathbf{v}^{(1)} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 6 \\ 7 \end{pmatrix}.$$

步长

$$t_1 = \frac{(\mathbf{r}^{(0)})^T \mathbf{r}^{(0)}}{(\mathbf{v}^{(1)})^T A \mathbf{v}^{(1)}} = \frac{1^2 + 2^2}{(1, 2) \cdot (6, 7)} = \frac{5}{20} = \frac{1}{4}.$$

更新解

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + t_1 \mathbf{v}^{(1)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \frac{1}{4} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} \frac{1}{4} \\ \frac{1}{2} \end{pmatrix}.$$

更新残差

$$\mathbf{r}^{(1)} = \mathbf{r}^{(0)} - t_1 A \mathbf{v}^{(1)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} - \frac{1}{4} \begin{pmatrix} 6 \\ 7 \end{pmatrix} = \begin{pmatrix} -\frac{1}{4} \\ \frac{1}{4} \end{pmatrix}.$$

再计算

$$s_1 = \frac{(\mathbf{r}^{(1)})^T \mathbf{r}^{(1)}}{(\mathbf{r}^{(0)})^T \mathbf{r}^{(0)}} = \frac{\frac{1}{4} + \frac{1}{16}}{5} = \frac{5/16}{5} = \frac{1}{16}.$$

因此新的搜索方向为

$$\mathbf{v}^{(2)} = \mathbf{r}^{(1)} + s_1 \mathbf{v}^{(1)} = \begin{pmatrix} -\frac{1}{4} \\ \frac{1}{4} \end{pmatrix} + \frac{1}{16} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} -\frac{7}{16} \\ \frac{3}{8} \end{pmatrix}.$$

第 2 步先算

$$A \mathbf{v}^{(2)} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} -\frac{7}{16} \\ \frac{3}{8} \end{pmatrix} = \begin{pmatrix} -\frac{11}{8} \\ \frac{11}{16} \end{pmatrix}.$$

步长

$$t_2 = \frac{(\mathbf{r}^{(1)})^T \mathbf{r}^{(1)}}{(\mathbf{v}^{(2)})^T A \mathbf{v}^{(2)}} = \frac{5/16}{55/64} = \frac{4}{11}.$$

更新解

$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + t_2 \mathbf{v}^{(2)} = \begin{pmatrix} \frac{1}{4} \\ \frac{1}{2} \end{pmatrix} + \frac{4}{11} \begin{pmatrix} -\frac{7}{16} \\ \frac{3}{8} \end{pmatrix} = \begin{pmatrix} \frac{1}{11} \\ \frac{7}{11} \end{pmatrix}.$$

这恰好就是精确解。

结论 在这个 2 维问题中，共轭梯度法只用了 2 步就得到精确解，这与“至多 n 步收敛”的理论完全一致。

2.5.10 与前面几种迭代法的比较

给出如下线性方程组

$$A \mathbf{x} = \mathbf{b},$$

其中

$$A = \begin{pmatrix} 0.2 & 0.1 & 1 & 1 & 0 \\ 0.1 & 4 & -1 & 1 & -1 \\ 1 & -1 & 60 & 0 & -2 \\ 1 & 1 & 0 & 8 & 4 \\ 0 & -1 & -2 & 4 & 700 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}.$$

在相同精度标准下，得到如下结果：

- **Jacobi**: 49 次迭代，

$$\mathbf{x}^{(k)} \approx (7.86277141, 0.42320802, -0.07348669, -0.53975964, 0.01062847)^T,$$

$$\|\mathbf{x}^* - \mathbf{x}^{(k)}\|_{\infty} \approx 0.00305834.$$

- **Gauss–Seidel**: 15 次迭代,

$$\mathbf{x}^{(k)} \approx (7.83525748, 0.42257868, -0.07319124, -0.53753055, 0.01060903)^T,$$

$$\|\mathbf{x}^* - \mathbf{x}^{(k)}\|_\infty \approx 0.02445559.$$

- **SOR** ($\omega = 1.25$): 7 次迭代,

$$\mathbf{x}^{(k)} \approx (7.85152706, 0.42277371, -0.07348303, -0.53978369, 0.01062286)^T,$$

$$\|\mathbf{x}^* - \mathbf{x}^{(k)}\|_\infty \approx 0.00818607.$$

- **Conjugate Gradient**: 5 次迭代,

$$\mathbf{x}^{(k)} \approx (7.85341523, 0.42298677, -0.07347963, -0.53987920, 0.008628916)^T,$$

$$\|\mathbf{x}^* - \mathbf{x}^{(k)}\|_\infty \approx 0.00629785.$$

- **Preconditioned Conjugate Gradient**: 4 次迭代,

$$\mathbf{x}^{(k)} \approx (7.85968827, 0.42288329, -0.07359878, -0.54063200, 0.01064344)^T,$$

$$\|\mathbf{x}^* - \mathbf{x}^{(k)}\|_\infty \approx 0.00009312.$$

注 这组数据非常清楚地体现了几个事实:

1. Jacobi 最慢;
2. Gauss–Seidel 与 SOR 由于利用了新信息, 因此明显更快;
3. CG 对对称正定问题非常高效;
4. 引入预条件后, PCG 往往还能进一步减少迭代次数, 并显著改善精度。

2.5.11 预条件共轭梯度法 (Preconditioned Conjugate Gradient, PCG)

前面已经指出, CG 的收敛速度受条件数 $\kappa(A)$ 强烈影响。若 A 条件数较大, 就希望先做一个“等价变形”, 把问题变得更好求。设 M 是一个容易求解的对称正定矩阵, 且它尽量逼近 A , 则称 M 为**预条件子** (preconditioner)。

基本想法是把原问题

$$A\mathbf{x} = \mathbf{b}$$

变换成一个更适合迭代的等价系统, 例如

$$M^{-1}A\mathbf{x} = M^{-1}\mathbf{b},$$

或者更对称地写成

$$M^{-1/2}AM^{-1/2}\mathbf{y} = M^{-1/2}\mathbf{b}, \quad \mathbf{x} = M^{-1/2}\mathbf{y}.$$

若 M 选得好, 则新系统的有效条件数会大幅下降, 从而使 CG 收敛更快。

PCG 的标准递推只是在 CG 中额外引入

$$M\mathbf{z}^{(k)} = \mathbf{r}^{(k)},$$

即

$$\mathbf{z}^{(k)} = M^{-1}\mathbf{r}^{(k)}.$$

随后把方向递推中的残差 $\mathbf{r}^{(k)}$ 换成 $\mathbf{z}^{(k)}$ 来构造。

 **笔记** 预条件的本质是: 用一个容易求解、又尽量接近 A 的矩阵 M , 来改善问题的谱性质。这也是现代大型线性代数计算的核心思想之一。

2.5.12 CG 的适用范围与推广

虽然本节讨论的是线性方程组中的标准 CG，但需要注意：**共轭梯度思想并不局限于线性 SPD 问题**。它还可以推广到：

1. 非线性方程组；
2. 非线性优化 (nonlinear optimization)；
3. 机器学习中的大规模优化问题。

在这些场景中，虽然公式不再与线性 CG 完全一致，但“利用梯度信息 + 构造共轭方向 + 避免重复搜索”的思想是相通的。

2.6 QR 分解 (QR Decomposition)

2.6.1 基本定义与几何意义

在线性代数中，**QR 分解**是指把一个矩阵分解为一个正交矩阵与一个上三角矩阵的乘积。对于一个列满秩矩阵

$$A \in \mathbb{R}^{m \times n} \quad (m \geq n),$$

若存在矩阵

$$Q \in \mathbb{R}^{m \times n}, \quad R \in \mathbb{R}^{n \times n},$$

使得

$$A = QR,$$

并满足

1. Q 的列向量两两正交且都已单位化，即

$$Q^T Q = I_n;$$

2. R 是上三角矩阵 (upper triangular matrix),

则称 $A = QR$ 为矩阵 A 的 **QR 分解**。

当 A 是 $n \times n$ 的可逆方阵时， Q 还是一个正交矩阵 (orthogonal matrix)，满足

$$Q^T Q = Q Q^T = I.$$

定义 2.8

若矩阵 Q 满足

$$Q^T Q = I,$$

则称 Q 为正交矩阵 (**orthogonal matrix**)。其列向量构成一组标准正交基 (orthonormal basis)。



注 从几何上看，QR 分解做了两件事：

1. 用正交化过程把 A 的列向量变成一组相互正交的单位向量，组成矩阵 Q ；
2. 再用一个上三角矩阵 R 记录“原来的列向量是如何由这组正交基线性组合出来的”。

所以 QR 分解的本质就是：**把一般列向量组分解为“方向正交化”与“坐标展开”两部分**。

2.6.2 为什么 QR 分解有用

QR 分解可用于：

- 求解线性方程组 (linear equations)；
- 最小二乘问题 (least squares problems)；

- 约束最小二乘问题 (constrained least squares problems)。

其原因在于：正交矩阵的数值性质非常好。因为

$$Q^{-1} = Q^T,$$

所以对正交矩阵的求逆既简单又稳定；而上三角矩阵 R 则可以通过回代 (back substitution) 快速求解。

例如，对方阵线性方程组

$$Ax = b,$$

若 $A = QR$ ，则

$$QRx = b.$$

左乘 Q^T 得

$$Rx = Q^T b.$$

由于 R 是上三角矩阵，所以 x 可由回代法直接求出。

结论 QR 分解把“求解一般线性方程组”的问题转化成了：

1. 一次正交变换 $Q^T b$ ；
2. 一个上三角方程组 $Rx = Q^T b$ 的求解。

这就是它在线性方程组与最小二乘问题中非常重要的原因。

2.6.3 QR 分解与 LU 分解的比较

QR 分解所需运算量大约是 LU 分解的两倍。

这意味着若仅仅为了求解一个普通方阵线性方程组，QR 分解通常不如 LU 分解经济；但 QR 分解有两个显著优点：

1. 数值稳定性更好，尤其适合最小二乘问题；
2. 正交变换不放大向量范数，因此舍入误差传播通常更容易控制。

因此在实践中：

- 若目标是解一个普通可逆方阵方程组，LU 往往更高效；
- 若目标是最小二乘、正交投影或需要高稳定性的计算，QR 往往更合适。

2.6.4 QR 分解的列向量形式

设矩阵 A 的列向量写成

$$A = [a_1 \ a_2 \ \cdots \ a_n].$$

若已经找到一组标准正交向量

$$e_1, e_2, \dots, e_n,$$

记

$$Q = [e_1 \ e_2 \ \cdots \ e_n],$$

则每一列 a_k 都可以写成

$$a_k = \sum_{j=1}^k r_{jk} e_j,$$

其中

$$r_{jk} = e_j^T a_k.$$

于是

$$A = QR,$$

且

$$R = \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & r_{nn} \end{pmatrix}.$$

为什么 R 是上三角的? 因为第 k 列向量 $\mathbf{a}_k$ 只需要前 k 个正交基向量来展开, 所以

$$r_{jk} = 0, \quad j > k.$$

注 因此 QR 分解也可以理解为: 把 A 的列向量在一组正交基下重新表示。矩阵 Q 给出基, 矩阵 R 给出坐标。

2.6.5 用 Gram-Schmidt 过程构造 QR 分解

设

$$A = [\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_n].$$

我们希望从原列向量 $\mathbf{a}_1, \dots, \mathbf{a}_n$ 出发, 依次构造标准正交向量

$$\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n.$$

2.6.5.1 第一步

令

$$\mathbf{u}_1 = \mathbf{a}_1, \quad \mathbf{e}_1 = \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|}.$$

也就是说, 第一列先不做投影修正, 直接归一化。

2.6.5.2 第二步

第二列 $\mathbf{a}_2$ 一般不与 $\mathbf{e}_1$ 正交, 因此要减去它在 $\mathbf{e}_1$ 上的投影 (projection):

$$\mathbf{u}_2 = \mathbf{a}_2 - (\mathbf{a}_2 \cdot \mathbf{e}_1)\mathbf{e}_1.$$

这样得到的 $\mathbf{u}_2$ 与 $\mathbf{e}_1$ 正交。再归一化:

$$\mathbf{e}_2 = \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|}.$$

2.6.5.3 一般步骤

已构造出

$$\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_k,$$

则对下一列 $\mathbf{a}_{k+1}$, 先减去它在前 k 个正交方向上的全部投影:

$$\mathbf{u}_{k+1} = \mathbf{a}_{k+1} - \sum_{j=1}^k (\mathbf{a}_{k+1} \cdot \mathbf{e}_j)\mathbf{e}_j.$$

然后归一化:

$$\mathbf{e}_{k+1} = \frac{\mathbf{u}_{k+1}}{\|\mathbf{u}_{k+1}\|}.$$

定义 2.9

上述逐列“减去投影并归一化”的过程，称为 **Gram-Schmidt** 正交化过程。它把线性无关向量组变成一组张成同一子空间的标准正交向量组。

**2.6.6 由 Gram-Schmidt 得到 Q 与 R**

构造完

$$\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n$$

之后，定义

$$Q = [\mathbf{e}_1 \mathbf{e}_2 \cdots \mathbf{e}_n].$$

由于这些列向量标准正交，所以

$$Q^T Q = I.$$

另一方面，由于每个 $\mathbf{a}_k$ 可表示成

$$\mathbf{a}_k = \sum_{j=1}^k (\mathbf{e}_j, \mathbf{a}_k) \mathbf{e}_j,$$

因此

$$R = \begin{pmatrix} (\mathbf{e}_1, \mathbf{a}_1) & (\mathbf{e}_1, \mathbf{a}_2) & \cdots & (\mathbf{e}_1, \mathbf{a}_n) \\ 0 & (\mathbf{e}_2, \mathbf{a}_2) & \cdots & (\mathbf{e}_2, \mathbf{a}_n) \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & (\mathbf{e}_n, \mathbf{a}_n) \end{pmatrix}.$$

于是

$$A = QR.$$

性质 若要求 R 的对角元全为正数，则 QR 分解是唯一的。

证明 [说明] 若 $A = Q_1 R_1 = Q_2 R_2$ ，则

$$Q_2^T Q_1 = R_2 R_1^{-1}.$$

左边是正交矩阵，右边是上三角矩阵，因此它必须是一个对角线上元素为 ± 1 的对角矩阵。若进一步要求 R 对角元均为正，则这些符号都被固定为 $+1$ ，于是分解唯一。

2.6.7 投影 (Projection) 的理解

我们特别强调 “Projection out the un-orthogonal components”，这正是 Gram-Schmidt 的本质。

对于一个向量 $\mathbf{a}$ 与单位向量 $\mathbf{e}$ ， $\mathbf{a}$ 在 $\mathbf{e}$ 上的投影为

$$\text{proj}_{\mathbf{e}}(\mathbf{a}) = (\mathbf{a} \cdot \mathbf{e})\mathbf{e}.$$

因此

$$\mathbf{a} - \text{proj}_{\mathbf{e}}(\mathbf{a})$$

就是从 $\mathbf{a}$ 中“剔除掉沿 $\mathbf{e}$ 方向的分量”后剩下的部分，它必然与 $\mathbf{e}$ 正交。

Gram-Schmidt 所做的事情，就是不断重复这一过程：每次把新列向量中沿旧正交方向的分量全部减掉，只保留真正“新的、与前面都正交”的那一部分。

2.6.8 例子：对给定矩阵做 QR 分解

设

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}.$$

把它写成列向量形式：

$$\mathbf{a}_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{a}_2 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \quad \mathbf{a}_3 = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}.$$

例题 2.6 用 Gram-Schmidt 方法求该矩阵的 QR 分解。

解 第一步：构造 $\mathbf{e}_1$

令

$$\mathbf{u}_1 = \mathbf{a}_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}.$$

其范数为

$$\|\mathbf{u}_1\| = \sqrt{1^2 + 1^2} = \sqrt{2}.$$

因此

$$\mathbf{e}_1 = \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{pmatrix}.$$

第二步：构造 $\mathbf{e}_2$

先减去 $\mathbf{a}_2$ 在 $\mathbf{e}_1$ 上的投影：

$$\mathbf{a}_2 \cdot \mathbf{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}}.$$

所以

$$\mathbf{u}_2 = \mathbf{a}_2 - (\mathbf{a}_2 \cdot \mathbf{e}_1)\mathbf{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} - \frac{1}{\sqrt{2}} \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ 1 \end{pmatrix}.$$

其范数为

$$\|\mathbf{u}_2\| = \sqrt{\left(\frac{1}{2}\right)^2 + \left(-\frac{1}{2}\right)^2 + 1^2} = \sqrt{\frac{1}{4} + \frac{1}{4} + 1} = \sqrt{\frac{3}{2}}.$$

因此

$$\mathbf{e}_2 = \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|} = \frac{1}{\sqrt{3/2}} \begin{pmatrix} \frac{1}{2} \\ -\frac{1}{2} \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{6}} \end{pmatrix}.$$

第三步：构造 $\mathbf{e}_3$

先计算两个投影系数：

$$\mathbf{a}_3 \cdot \mathbf{e}_1 = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}},$$

$$\mathbf{a}_3 \cdot \mathbf{e}_2 = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{6}} \end{pmatrix} = \frac{1}{\sqrt{6}}.$$

因此

$$\mathbf{u}_3 = \mathbf{a}_3 - (\mathbf{a}_3 \cdot \mathbf{e}_1)\mathbf{e}_1 - (\mathbf{a}_3 \cdot \mathbf{e}_2)\mathbf{e}_2.$$

代入得

$$\mathbf{u}_3 = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} - \frac{1}{\sqrt{2}} \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{pmatrix} - \frac{1}{\sqrt{6}} \begin{pmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{6}} \end{pmatrix}.$$

即

$$\mathbf{u}_3 = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} - \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \\ 0 \end{pmatrix} - \begin{pmatrix} \frac{1}{6} \\ -\frac{1}{6} \\ \frac{1}{3} \end{pmatrix} = \begin{pmatrix} -\frac{2}{3} \\ \frac{2}{3} \\ \frac{2}{3} \end{pmatrix}.$$

其范数为

$$\|\mathbf{u}_3\| = \sqrt{3 \cdot \left(\frac{2}{3}\right)^2} = \frac{2}{\sqrt{3}}.$$

因此

$$\mathbf{e}_3 = \frac{\mathbf{u}_3}{\|\mathbf{u}_3\|} = \begin{pmatrix} -\frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \end{pmatrix}.$$

第四步：写出 Q

由

$$Q = [\mathbf{e}_1 \ \mathbf{e}_2 \ \mathbf{e}_3],$$

可得

$$Q = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{3}} \end{pmatrix}.$$

第五步：写出 R

由

$$r_{jk} = \mathbf{e}_j^T \mathbf{a}_k,$$

得到

$$r_{11} = \mathbf{e}_1^T \mathbf{a}_1 = \sqrt{2}, \quad r_{12} = \mathbf{e}_1^T \mathbf{a}_2 = \frac{1}{\sqrt{2}}, \quad r_{13} = \mathbf{e}_1^T \mathbf{a}_3 = \frac{1}{\sqrt{2}},$$

$$r_{22} = \mathbf{e}_2^T \mathbf{a}_2 = \frac{3}{\sqrt{6}}, \quad r_{23} = \mathbf{e}_2^T \mathbf{a}_3 = \frac{1}{\sqrt{6}},$$

$$r_{33} = \mathbf{e}_3^T \mathbf{a}_3 = \frac{2}{\sqrt{3}}.$$

因此

$$R = \begin{pmatrix} \sqrt{2} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \frac{3}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ 0 & 0 & \frac{2}{\sqrt{3}} \end{pmatrix}.$$

结论 于是矩阵 A 的 QR 分解为

$$A = QR,$$

其中

$$Q = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{3}} \\ 0 & \frac{2}{\sqrt{6}} & \frac{1}{\sqrt{3}} \end{pmatrix}, \quad R = \begin{pmatrix} \sqrt{2} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & \frac{3}{\sqrt{6}} & \frac{1}{\sqrt{6}} \\ 0 & 0 & \frac{2}{\sqrt{3}} \end{pmatrix}.$$

2.6.9 正交性检验

这一步非常重要，因为 QR 分解中最核心的要求就是 Q 必须正交。

我们只需验证

$$Q^T Q = I.$$

在上例中，三列分别是

$$\mathbf{e}_1 = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \end{pmatrix}, \quad \mathbf{e}_2 = \begin{pmatrix} \frac{1}{\sqrt{6}} \\ -\frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{6}} \end{pmatrix}, \quad \mathbf{e}_3 = \begin{pmatrix} -\frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \\ \frac{1}{\sqrt{3}} \end{pmatrix}.$$

容易检查：

$$\mathbf{e}_1 \cdot \mathbf{e}_1 = 1, \quad \mathbf{e}_2 \cdot \mathbf{e}_2 = 1, \quad \mathbf{e}_3 \cdot \mathbf{e}_3 = 1,$$

并且

$$\mathbf{e}_1 \cdot \mathbf{e}_2 = 0, \quad \mathbf{e}_1 \cdot \mathbf{e}_3 = 0, \quad \mathbf{e}_2 \cdot \mathbf{e}_3 = 0.$$

因此

$$Q^T Q = I.$$

注 在手算时，检验 $Q^T Q = I$ 一方面是检查结果是否正确，另一方面也能帮助理解 QR 分解真正依赖的是“列向量的标准正交性”，而不只是公式表面上的代入计算。

2.6.10 QR 分解如何用来解方程组

若

$$A = QR,$$

则对线性方程组

$$A\mathbf{x} = \mathbf{b}$$

有

$$QR\mathbf{x} = \mathbf{b}.$$

左乘 Q^T 得

$$R\mathbf{x} = Q^T\mathbf{b}.$$

因为 R 是上三角矩阵，所以可以通过回代法逐步解出：

$$r_{nn}x_n = (Q^T\mathbf{b})_n,$$

$$r_{n-1,n-1}x_{n-1} + r_{n-1,n}x_n = (Q^T\mathbf{b})_{n-1},$$

一直回代到 x_1 。

例题 2.7 如果 A 已经分解为 $A = QR$ ，那么求解 $A\mathbf{x} = \mathbf{b}$ 的主要工作就只剩下：

1. 计算 $\mathbf{c} = Q^T\mathbf{b}$;
2. 解上三角系统 $R\mathbf{x} = \mathbf{c}$ 。

这与 LU 分解中“先前代、再回代”类似，只不过这里利用的是正交矩阵。

2.6.11 QR 分解与最小二乘问题

QR 分解最重要的应用之一是线性最小二乘问题。设

$$A\mathbf{x} \approx \mathbf{b}, \quad A \in \mathbb{R}^{m \times n}, \quad m > n,$$

我们希望求

$$\min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|_2.$$

若 $A = QR$ ，其中 Q 的列正交， R 为上三角，则

$$\|A\mathbf{x} - \mathbf{b}\|_2 = \|QR\mathbf{x} - \mathbf{b}\|_2.$$

利用正交变换保持二范数不变，可把问题转化为一个上三角系统，从而稳定地求出最小二乘解。

 **笔记** 这正是为什么在最小二乘问题中，人们通常更偏好 QR 分解，而不是先构造正规方程

$$A^T A \mathbf{x} = A^T \mathbf{b}.$$

后者虽然形式简单，但会放大条件数，数值稳定性较差。

2.6.12 Householder 变换的补充说明

QR 分解“也可以通过一系列 Householder 变换实现”。这句话非常重要。因为经典 Gram-Schmidt 过程在浮点计算中可能会有数值不稳定问题，尤其当列向量接近线性相关时，正交性会受到舍入误差影响。为改进这一点，实际数值线性代数中更常用：

1. 改进 Gram-Schmidt (Modified Gram-Schmidt);
2. Householder 反射 (Householder reflections);
3. Givens 旋转 (Givens rotations)。

其中 Householder 方法尤其常见，因为它数值稳定性更好，也适合程序实现。

注 因此，从理论教学角度看，Gram-Schmidt 最容易帮助我们理解 QR 分解的本质；但从实际算法角度看，Householder 方法通常更值得优先采用。

2.7 $M \times N$ 矩阵、SVD 与线性最小二乘

在前面讨论线性方程组时，我们大多默认系数矩阵是方阵，即

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{N \times N}.$$

但在实际计算中，更一般也更常见的情形是

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{M \times N},$$

其中 M 与 N 未必相等。此时问题的结构会发生根本变化：有时方程太多，有时未知数太多，有时矩阵退化 (singular) 或接近退化 (ill-conditioned)，从而使直接的高斯消元或 LU 分解不再可靠，甚至根本不适用。

这一节的核心内容是：

1. 先理解一般 $M \times N$ 线性系统的三种基本情形；
2. 再引入奇异值分解 (Singular Value Decomposition, SVD) 作为统一工具；
3. 用 SVD 处理超定方程组 (overdetermined system)、欠定方程组 (underdetermined system)、退化方程组 (singular system) 以及线性最小二乘拟合 (linear least-squares fitting)。

2.7.1 一般 $M \times N$ 线性系统的三种基本情形

考虑

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{M \times N}.$$

根据方程个数 M 与未知数个数 N 的关系，可分为三类：

情形一： $M = N$

这时 A 是方阵。若 A 非奇异 (nonsingular)，则问题退化为通常的方阵求解问题，可以使用高斯消元、LU 分解、QR 分解、迭代等方法求解。

但若 A 退化，即

$$\det(A) = 0,$$

或者虽然 $\det(A) \neq 0$ 但矩阵病态 (ill-conditioned)，那么直接求逆会非常不稳定，这时更适合使用 SVD。

情形二： $M > N$

这时方程个数多于未知数个数，称为**超定系统 (overdetermined system)**。一般来说，方程彼此并不完全相容，因此通常不存在精确解。此时求解目标变为寻找“最接近满足全部方程”的解，即**最小二乘解 (least-squares solution)**。

情形三： $M < N$

这时未知数个数多于方程个数，称为**欠定系统 (underdetermined system)**。若系统相容，则一般会有无穷多个解；若系统不相容，则仍需要做某种广义意义下的逼近。SVD 能够清楚地区分出：

1. 哪些方向是受方程约束的；
2. 哪些方向属于零空间 (null space)，可以自由变化。

 **笔记** 一旦矩阵的行或列出现线性相关 (linear dependence)，或者接近线性相关，数值计算中的误差就会被显著放大。矩阵维数很大时，这种现象会更加严重。因此，对一般 $M \times N$ 矩阵，特别是退化或近退化问题，必须使用比普通消元更稳健的工具。

2.7.2 奇异问题与病态问题

在数值计算中，两个概念需要严格区分：

定义 2.10

若矩阵 A 的秩小于其最大可能秩，即

$$\text{rank}(A) < \min(M, N),$$

则称 A 是奇异 (**singular**) 或秩亏 (**rank-deficient**) 的。

定义 2.11

若矩阵虽然理论上满秩，但某些方向上的伸缩非常小，从而导致求解过程对舍入误差极为敏感，则称矩阵是病态 (**ill-conditioned**) 的。

奇异或病态的来源通常包括：

1. 某些行之间线性相关或近似线性相关；
2. 某些列之间线性相关或近似线性相关；
3. 模型中包含冗余变量；
4. 采样点分布不佳，导致设计矩阵几乎退化；
5. 矩阵尺度过大、数值范围差异悬殊。

注 从几何上看，病态矩阵会把某些方向压缩得极其厉害。因此，当我们试图“反过来”恢复这些方向上的信息时，就会把极小的噪声或舍入误差成倍放大。这也是为什么普通求逆在病态问题中往往毫无意义。

2.7.3 奇异值分解 (SVD) 的定义

对任意实矩阵

$$A \in \mathbb{R}^{M \times N},$$

都存在分解

$$A = U \Sigma V^T,$$

这称为**奇异值分解 (Singular Value Decomposition, SVD)**。

其中：

1. $U \in \mathbb{R}^{M \times M}$ 是正交矩阵 (orthogonal matrix)，满足

$$U^T U = U U^T = I_M;$$

2. $V \in \mathbb{R}^{N \times N}$ 是正交矩阵，满足

$$V^T V = V V^T = I_N;$$

3. $\Sigma \in \mathbb{R}^{M \times N}$ 是“对角形”矩阵 (rectangular diagonal matrix)，其非零元素只可能出现在主对角线上：

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_r \\ \vdots & \vdots & & \vdots \end{pmatrix},$$

其中

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0, \quad \sigma_{r+1} = \cdots = 0.$$

这些 σ_i 称为矩阵 A 的**奇异值 (singular values)**。

定义 2.12

矩阵 A 的非零奇异值个数等于其秩：

$$\text{rank}(A) = \#\{\sigma_i : \sigma_i > 0\}.$$



性质 SVD 将矩阵 A 分解为三部分：

1. V^T ：在输入空间中进行正交旋转；
2. Σ ：沿若干彼此正交的方向进行伸缩；
3. U ：在输出空间中再进行正交旋转。

注 因此，SVD 给出了矩阵作用的最本质几何图像：任意线性变换都等价于“旋转 $\rightarrow$ 伸缩 $\rightarrow$ 再旋转”。其中真正决定问题难度的，就是中间那组奇异值 σ_i 的大小与分布。

2.7.4 奇异值、秩与退化方向

设

$$A = U\Sigma V^T.$$

记

$$U = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_M], \quad V = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N].$$

则

$$A\mathbf{v}_i = \sigma_i \mathbf{u}_i, \quad i = 1, 2, \dots, r.$$

这说明：

1. $\mathbf{v}_i$ 是输入空间中的主方向；
2. $\mathbf{u}_i$ 是输出空间中的对应方向；
3. σ_i 表示矩阵在该方向上的伸缩倍数。

若某个 $\sigma_i = 0$ ，则

$$A\mathbf{v}_i = 0,$$

说明 $\mathbf{v}_i$ 落在 A 的零空间中。也就是说，沿这一方向的输入完全无法从输出中恢复。这正是退化问题的根源。

结论 奇异值越小，对应方向越“不稳定”；奇异值为零，对应方向的信息完全丢失。因此：

- 非零奇异值决定了矩阵真正有效的维数；
- 很小的奇异值对应近退化方向；
- 零奇异值对应严格退化方向。

2.7.5 SVD 与广义逆 (pseudo-inverse)

当 A 不是可逆方阵时，普通逆矩阵 A^{-1} 并不存在。但借助 SVD，可以定义 **Moore–Penrose 广义逆 (pseudo-inverse)**。

若

$$A = U\Sigma V^T,$$

则定义

$$A^+ = V\Sigma^+ U^T,$$

其中 Σ^+ 由 Σ 的非零奇异值取倒数得到：

$$\Sigma^+ = \text{diag}\left(\frac{1}{\sigma_1}, \frac{1}{\sigma_2}, \dots, \frac{1}{\sigma_r}, 0, \dots, 0\right).$$

于是一个自然的广义解写成

$$\mathbf{x} = A^+ \mathbf{b} = V \Sigma^+ U^T \mathbf{b}.$$

若矩阵病态，为避免极小奇异值导致误差爆炸，常采用**截断伪逆** (truncated pseudo-inverse)：

$$\Sigma_t^+ = \begin{cases} 1/\sigma_i, & \sigma_i > t, \\ 0, & \sigma_i \leq t, \end{cases}$$

其中 $t > 0$ 为阈值。于是

$$A_t^+ = V \Sigma_t^+ U^T.$$

 **笔记** 这一步的物理含义非常清楚：当某个奇异值太小，说明该方向几乎不可靠。与其把噪声放大 $1/\sigma_i$ 倍，不如干脆把它视作无效信息，直接截断为零。这实际上是一种正则化 (regularization) 思想。

2.7.6 超定方程组 $M > N$ ：线性最小二乘问题

现在考虑

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{M \times N}, \quad M > N.$$

这时方程个数多于未知数个数，一般不存在精确解。于是我们转而求解

$$\min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|_2,$$

这就是**线性最小二乘问题** (linear least-squares problem)。

2.7.6.1 正规方程 (normal equations)

若用经典方法推导，最小二乘解满足

$$A^T A \mathbf{x} = A^T \mathbf{b}.$$

这称为**正规方程** (normal equation)。

但是，直接求解正规方程有两个明显问题：

1. $A^T A$ 的条件数大约是 A 条件数的平方，数值稳定性变差；
2. 若 A 接近退化，则 $A^T A$ 更接近奇异。

因此，在病态问题中，更可靠的方法是直接使用 SVD。

2.7.6.2 用 SVD 求最小二乘解

设

$$A = U \Sigma V^T.$$

则

$$A\mathbf{x} - \mathbf{b} = U \Sigma V^T \mathbf{x} - \mathbf{b}.$$

由于正交变换保持 2-范数不变，

$$\|A\mathbf{x} - \mathbf{b}\|_2 = \|\Sigma V^T \mathbf{x} - U^T \mathbf{b}\|_2.$$

令

$$\mathbf{y} = V^T \mathbf{x}, \quad \mathbf{c} = U^T \mathbf{b},$$

则问题等价于

$$\min_{\mathbf{y}} \|\Sigma \mathbf{y} - \mathbf{c}\|_2.$$

因为 Σ 是对角形矩阵，这个问题按分量完全解耦。对于非零奇异值方向，最优解满足

$$y_i = \frac{c_i}{\sigma_i}, \quad i = 1, \dots, r.$$

于是

$$\mathbf{x} = V\Sigma^+U^T\mathbf{b}.$$

这就是最小二乘意义下的标准解。

性质 对于超定系统， $A^+\mathbf{b}$ 给出的解是：

1. 残差范数 $\|\mathbf{Ax} - \mathbf{b}\|_2$ 最小的解；
2. 在所有最小残差解中，欧氏范数 $\|\mathbf{x}\|_2$ 最小的解。

2.7.7 欠定方程组 $M < N$ ：无穷多解与最小范数解

现在考虑

$$\mathbf{Ax} = \mathbf{b}, \quad A \in \mathbb{R}^{M \times N}, \quad M < N.$$

这时未知数比方程多，若系统相容，通常会有无穷多个解。

设

$$\text{rank}(A) = r.$$

则解空间的自由维数为

$$N - r.$$

若系统相容，则一般解可写为

$$\mathbf{x} = \mathbf{x}_p + \mathbf{x}_n,$$

其中 $\mathbf{x}_p$ 是某个特解， $\mathbf{x}_n \in N(A)$ 属于零空间。

在 SVD 语言下，零空间正是由那些对应零奇异值的右奇异向量 $\mathbf{v}_i$ 张成的。也就是说，若

$$\sigma_i = 0,$$

则

$$A\mathbf{v}_i = 0,$$

这些 $\mathbf{v}_i$ 的线性组合可以任意加入到某个特解上，仍然得到新的解。

定理 2.4

对于欠定系统，在所有满足 $\mathbf{Ax} = \mathbf{b}$ 的解中，SVD 给出的

$$\mathbf{x} = A^+\mathbf{b} = V\Sigma^+U^T\mathbf{b}$$

是欧氏范数最小的解，即

$$\|\mathbf{x}\|_2 = \min.$$



注 因此，SVD 不仅能告诉我们“有没有解”，还能进一步告诉我们：

1. 解是否唯一；
2. 非唯一性来自哪些方向；
3. 哪个解是最自然、最稳定的最小范数解。

2.7.8 秩、零空间与奇异值之间的关系

从 SVD 出发，矩阵的几个核心概念会被统一起来。

性质 设

$$A = U\Sigma V^T, \quad \text{rank}(A) = r,$$

则有：

1. A 的秩等于非零奇异值个数；
2. A 的列空间 (column space) 由 $\mathbf{u}_1, \dots, \mathbf{u}_r$ 张成；
3. A 的行空间 (row space) 由 $\mathbf{v}_1, \dots, \mathbf{v}_r$ 张成；
4. A 的零空间由 $\mathbf{v}_{r+1}, \dots, \mathbf{v}_N$ 张成；
5. A^T 的零空间由 $\mathbf{u}_{r+1}, \dots, \mathbf{u}_M$ 张成。

结论 SVD 的最大优势之一在于：它不仅给出“如何算”，还直接揭示了矩阵的内部结构——秩、零空间、有效维数、病态方向，全部都能从奇异值与奇异向量中读出来。

2.7.9 截断 SVD 与压缩 (compression)

设

$$A = U\Sigma V^T,$$

且奇异值按降序排列

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0.$$

若只保留前 k 个最大的奇异值及对应奇异向量，就得到秩为 k 的近似矩阵

$$A_k = U_k \Sigma_k V_k^T,$$

其中：

- $U_k \in \mathbb{R}^{M \times k}$ 由前 k 个左奇异向量组成；
- $\Sigma_k \in \mathbb{R}^{k \times k}$ 由前 k 个奇异值组成；
- $V_k \in \mathbb{R}^{N \times k}$ 由前 k 个右奇异向量组成。

这种近似有两层重要意义：

1. **降秩近似 (low-rank approximation)**：用更少参数表示原矩阵；
2. **噪声抑制 (denoising)**：把小奇异值对应的噪声方向截断掉。

定理 2.5 (Eckart–Young 定理)

在所有秩不超过 k 的矩阵中， A_k 是最接近 A 的一个，且在 2-范数和 Frobenius 范数意义下都最优。



注 这说明截断 SVD 不仅是一种启发式压缩方法，而且在严格数学意义下是“最佳低秩逼近”。这也是它在图像压缩、降维、噪声过滤中的理论基础。

2.7.10 线性最小二乘拟合中的 SVD

SVD 在线性最小二乘拟合中尤其有用。考虑用三次多项式去拟合数据点

$$(y_1, b_1), (y_2, b_2), \dots, (y_m, b_m).$$

设拟合函数为

$$p(y) = x_1 + x_2 y + x_3 y^2 + x_4 y^3.$$

则对每个数据点有残差

$$r_i = p(y_i) - b_i.$$

把全部残差写成向量形式：

$$\mathbf{r} = \mathbf{A}\mathbf{x} - \mathbf{b},$$

其中

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix},$$

而设计矩阵 (design matrix) 为

$$A = \begin{pmatrix} 1 & y_1 & y_1^2 & y_1^3 \\ 1 & y_2 & y_2^2 & y_2^3 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & y_m & y_m^2 & y_m^3 \end{pmatrix}.$$

这就是一个典型的 **Vandermonde 矩阵 (Vandermonde matrix)**。

于是拟合问题转化为

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2.$$

若

$$A = U\Sigma V^T,$$

则拟合系数由

$$\mathbf{x} = A^+\mathbf{b} = V\Sigma^+U^T\mathbf{b}$$

给出。

例题 2.8 若数据量为 m ，拟合三次多项式，则未知系数有 4 个，而方程个数有 m 个。当

$$m > 4$$

时，这是一个超定系统。SVD 方法会自动给出残差平方和最小的最佳拟合系数。

 **笔记** Vandermonde 矩阵往往容易病态，尤其当多项式次数较高或采样点分布不理想时更是如此。因此在多项式拟合中，SVD 比直接解正规方程稳定得多。

2.7.11 为什么 SVD 比普通消元更稳健

SVD 的稳健性主要来自以下几点：

1. 正交矩阵 U, V 不放大 2-范数；
2. 奇异值把问题分解到彼此独立的方向上；
3. 病态方向可以通过阈值截断；
4. 对秩亏问题、近秩亏问题、最小二乘问题、最小范数问题都能统一处理。

相比之下，普通高斯消元或 LU 分解更适合非奇异方阵；一旦矩阵接近退化，求逆过程中的误差放大会非常明显。

注 从算法复杂度看，SVD 确实比 LU、QR 更复杂，运算代价也更高；但从适用范围与稳定性看，它几乎是处理一般线性系统最强有力的通用工具之一。

2.8 范数、条件数与稀疏线性方程

在数值线性代数中，**范数 (norm)** 用来刻画向量或矩阵的“大小”，**条件数 (condition number)** 用来刻画一个问题对扰动的敏感程度，而**稀疏线性方程 (sparse linear equations)** 则是大规模科学计算中的核心对象。三者彼此密切相关：

1. 范数提供了“误差有多大”的定量标准；

2. 条件数说明“输入中很小的误差会被放大多少”；
3. 稀疏问题则要求我们在保证精度的同时尽可能降低存储与计算成本。

本节先介绍常用向量范数与矩阵范数，再讨论条件数及其与误差的关系，最后介绍稀疏线性系统中的若干基本思想，以及 Sherman–Morrison 公式和 Woodbury 恒等式。

2.8.1 向量范数 (Vector Norm)

定义 2.13

在 $\mathbb{R}^n$ 上，一个函数

$$\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$$

若满足以下三条性质，则称其为一个范数：

1. 正定性 (positivity)：

$$\|\mathbf{x}\| \geq 0, \quad \|\mathbf{x}\| = 0 \iff \mathbf{x} = \mathbf{0};$$

2. 齐次性 (homogeneity)：

$$\|\alpha \mathbf{x}\| = |\alpha| \|\mathbf{x}\|, \quad \forall \alpha \in \mathbb{R};$$

3. 三角不等式 (triangle inequality)：

$$\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|.$$

最常见的一类范数是 p -范数 (p -norm)：

$$\|\mathbf{x}\|_p = (|x_1|^p + |x_2|^p + \cdots + |x_n|^p)^{1/p}, \quad p \geq 1.$$

2.8.1.1 1-范数

$$\|\mathbf{x}\|_1 = |x_1| + |x_2| + \cdots + |x_n|.$$

它衡量的是各分量绝对值之和，因此对“分量总量”的刻画非常直接。

2.8.1.2 2-范数 (Euclidean norm)

$$\|\mathbf{x}\|_2 = \left(|x_1|^2 + |x_2|^2 + \cdots + |x_n|^2 \right)^{1/2} = (\mathbf{x}^T \mathbf{x})^{1/2}.$$

它就是通常几何意义下的欧几里得长度 (Euclidean length)。

2.8.1.3 ∞ -范数 (maximum norm)

$$\|\mathbf{x}\|_\infty = \max_i |x_i|.$$

它只关注分量中绝对值最大的那一个，因此特别适用于控制“最大分量误差”。

注 从几何上看：

- $\|\mathbf{x}\|_1$ 强调“总和”；
- $\|\mathbf{x}\|_2$ 强调“几何长度”；
- $\|\mathbf{x}\|_\infty$ 强调“最大偏差”。

不同范数对应不同的误差衡量标准，因此后面定义条件数时，范数的选择会直接影响结果。

2.8.2 Cauchy–Schwarz 不等式与三角不等式

对 2-范数而言，一个最基本的不等式是 **Cauchy–Schwarz 不等式**：

$$|\mathbf{x}^T \mathbf{y}| \leq \|\mathbf{x}\|_2 \|\mathbf{y}\|_2.$$

它是欧氏空间中内积与长度之间的基本关系。

由 Cauchy–Schwarz 不等式可以进一步推出 2-范数的三角不等式：

$$\|\mathbf{x} + \mathbf{y}\|_2 \leq \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2.$$

证明 由展开得

$$\|\mathbf{x} + \mathbf{y}\|_2^2 = (\mathbf{x} + \mathbf{y})^T (\mathbf{x} + \mathbf{y}) = \|\mathbf{x}\|_2^2 + 2\mathbf{x}^T \mathbf{y} + \|\mathbf{y}\|_2^2.$$

再用 Cauchy–Schwarz 不等式，

$$2\mathbf{x}^T \mathbf{y} \leq 2\|\mathbf{x}\|_2 \|\mathbf{y}\|_2,$$

于是

$$\|\mathbf{x} + \mathbf{y}\|_2^2 \leq (\|\mathbf{x}\|_2 + \|\mathbf{y}\|_2)^2,$$

两边开方即得结论。

2.8.3 矩阵范数 (Matrix Norm)

与向量范数类似，矩阵也需要“大小”的定义。最自然的一类矩阵范数来自**诱导范数 (induced norm / subordinate norm)**。

定义 2.14

给定向量范数 $\|\cdot\|$ ，定义矩阵 A 的诱导范数为

$$\|A\| = \max_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|.$$

它表示矩阵 A 对单位向量可能产生的最大放大倍数。

由定义可立刻得到一个极重要性质：

$$\|A\mathbf{z}\| \leq \|A\| \|\mathbf{z}\|,$$

即线性变换不会把向量放大超过 $\|A\|$ 倍。

2.8.3.1 ∞ -矩阵范数

对 ∞ -范数诱导得到的矩阵范数定义为

$$\|A\|_\infty = \max_{\|\mathbf{x}\|_\infty=1} \|A\mathbf{x}\|_\infty.$$

对矩阵

$$A = (a_{ij})_{m \times n},$$

有显式公式

$$\|A\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|.$$

也就是说， ∞ -矩阵范数等于**最大行绝对值和 (maximum absolute row sum)**。

证明 对任意 $\|\mathbf{x}\|_\infty = 1$, 有 $|x_j| \leq 1$, 于是

$$|(\mathbf{Ax})_i| = \left| \sum_{j=1}^n a_{ij}x_j \right| \leq \sum_{j=1}^n |a_{ij}||x_j| \leq \sum_{j=1}^n |a_{ij}|.$$

再对所有 i 取最大, 得

$$\|\mathbf{Ax}\|_\infty \leq \max_i \sum_j |a_{ij}|,$$

从而

$$\|A\|_\infty \leq \max_i \sum_j |a_{ij}|.$$

另一方面, 只要适当选取使某一行达到同号的向量 $\mathbf{x}$, 就可以达到这个上界, 因此等号成立。

2.8.3.2 Frobenius 范数

另一类很常用但不是诱导范数的是 **Frobenius 范数**:

$$\|A\|_F = \left(\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2 \right)^{1/2}.$$

它可以看成把矩阵所有元素按一个长向量来计算 2-范数。

注 Frobenius 范数计算方便、形式对称, 因此在误差分析、低秩近似、SVD 与最优化中非常常见; 而诱导范数则更适合描述“矩阵作为算子时的放大能力”。

2.8.4 条件数 (Condition Number)

当我们求解线性方程组

$$\mathbf{Ax} = \mathbf{b}$$

时, 真正关心的不只是“能不能解”, 还关心“这个问题对扰动是否敏感”。这就引出了条件数。

定义 2.15

对非奇异矩阵 A , 相对于某个矩阵范数 $\|\cdot\|$ 的条件数定义为

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

它衡量的是矩阵 A 与其逆 A^{-1} 联合起来对误差的放大能力。

注 若 $\kappa(A) \approx 1$, 则问题对扰动不太敏感, 称 A 为**良态 (well-conditioned)**; 若 $\kappa(A) \gg 1$, 则问题非常敏感, 称 A 为**病态 (ill-conditioned)**。

2.8.5 残差、误差与条件数的关系

设 $\mathbf{x}$ 是精确解, $\tilde{\mathbf{x}}$ 是数值近似解。定义残差 (residual)

$$\mathbf{r} = \mathbf{b} - A\tilde{\mathbf{x}}.$$

注意这里残差并不是“真误差”

$$\mathbf{x} - \tilde{\mathbf{x}},$$

而只是把近似解代回方程后所剩下的“不平衡量”。

由

$$\mathbf{Ax} = \mathbf{b}, \quad A\tilde{\mathbf{x}} = \mathbf{b} - \mathbf{r},$$

两式相减得

$$A(\mathbf{x} - \tilde{\mathbf{x}}) = \mathbf{r},$$

于是

$$\mathbf{x} - \tilde{\mathbf{x}} = A^{-1}\mathbf{r}.$$

取范数可得

$$\|\mathbf{x} - \tilde{\mathbf{x}}\| \leq \|A^{-1}\| \|\mathbf{r}\|.$$

这说明：真误差由残差与逆矩阵的大小共同控制。

进一步，因为

$$\mathbf{b} = A\mathbf{x},$$

所以

$$\|\mathbf{b}\| = \|A\mathbf{x}\| \leq \|A\| \|\mathbf{x}\|, \quad \Rightarrow \quad \frac{1}{\|\mathbf{x}\|} \leq \frac{\|A\|}{\|\mathbf{b}\|}.$$

将其与上式结合，可得相对误差估计：

$$\frac{\|\mathbf{x} - \tilde{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \|A\| \|A^{-1}\| \frac{\|\mathbf{r}\|}{\|\mathbf{b}\|} = \kappa(A) \frac{\|\mathbf{r}\|}{\|\mathbf{b}\|}.$$

结论 相对误差不仅取决于残差的大小，还取决于条件数。也就是说：

- 小残差不一定意味着小误差；
- 若 $\kappa(A)$ 很大，即使 $\|\mathbf{r}\|$ 很小， $\|\mathbf{x} - \tilde{\mathbf{x}}\|$ 仍可能不小。

2.8.6 一个病态矩阵的例子

考虑矩阵

$$A = \begin{pmatrix} 1 & 2 \\ 1.0001 & 2 \end{pmatrix}.$$

它的逆为

$$A^{-1} = \begin{pmatrix} -10000 & 10000 \\ 5000.5 & -5000 \end{pmatrix}.$$

先计算 ∞ -范数：

$$\|A\|_{\infty} = \max\{|1| + |2|, |1.0001| + |2|\} = 3.0001,$$

$$\|A^{-1}\|_{\infty} = \max\{10000 + 10000, 5000.5 + 5000\} = 20000.$$

因此

$$\kappa_{\infty}(A) = \|A\|_{\infty} \|A^{-1}\|_{\infty} = (3.0001)(20000) = 60002.$$

注 这个条件数极大，说明矩阵虽然可逆，但非常病态。其原因在于两行

$$(1, 2), \quad (1.0001, 2)$$

非常接近，几乎线性相关。这样的矩阵在求解方程时会把微小扰动显著放大。

2.8.7 2-范数条件数与奇异值

条件数依赖于范数的选择。对 2-范数，有一个特别重要的公式：

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)},$$

其中 $\sigma_{\max}(A)$ 与 $\sigma_{\min}(A)$ 分别是矩阵 A 的最大和最小奇异值 (singular values)。

性质 对非奇异矩阵 A , 2-范数条件数满足

$$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}.$$

注 这个公式非常有意义。它表明 2-范数条件数本质上是在比较矩阵对不同方向的最大伸缩与最小伸缩：

- 若二者接近，则矩阵几乎“各向同性”，问题稳定；
- 若最小奇异值很小，则某个方向几乎被压扁，反问题极不稳定。

2.8.8 预条件化 (Preconditioning)

既然大的条件数会显著拖慢迭代收敛并放大误差，一个自然想法就是：先把原问题等价地变形成一个条件数更小的问题，这称为**预条件化 (preconditioning)**。

对称情形中，常考虑

$$\tilde{A} = C^{-1}A(C^{-1})^T,$$

再定义

$$\tilde{\mathbf{x}} = C^T \mathbf{x}, \quad \tilde{\mathbf{b}} = C^{-1} \mathbf{b}.$$

于是原方程

$$A\mathbf{x} = \mathbf{b}$$

可改写为

$$\tilde{A}\tilde{\mathbf{x}} = \tilde{\mathbf{b}}.$$

因为

$$\tilde{A}\tilde{\mathbf{x}} = (C^{-1}A(C^{-1})^T)(C^T \mathbf{x}) = C^{-1}A\mathbf{x} = C^{-1}\mathbf{b} = \tilde{\mathbf{b}}.$$

结论 预条件化的目标不是改变问题的解，而是改变问题的数值形状，使新的系数矩阵 $\tilde{A}$ 条件数更小，从而：

1. 提高迭代法的收敛速度；
2. 降低数值误差的放大程度。

在共轭梯度法中，预条件共轭梯度法 (preconditioned conjugate gradient, PCG) 正是建立在这一思想之上。预条件选得好时，迭代步数会明显减少。

2.9 稀疏线性方程 (Sparse Linear Equations)

定义 2.16

若矩阵中绝大多数元素都为零，只含少量非零元，则称该矩阵为稀疏矩阵 (sparse matrix)。

稀疏矩阵大量出现在：

- 偏微分方程离散；
- 图与网络问题；
- 压缩感知 (compressed sensing)；
- 机器学习与统计学习；
- 图像与信号处理；
- 人脸识别等模式识别问题。

2.9.1 直接法与特殊结构

对某些特殊结构的稀疏矩阵，直接法仍然很有效，例如：

1. 三对角矩阵 (tridiagonal matrix);
2. 带状矩阵 (band matrix);
3. 块对角矩阵 (block diagonal matrix)。

这些矩阵可以利用其特殊结构显著减少存储与运算量。

更一般地，稀疏矩阵常通过置换矩阵做重排，得到形如

$$A = P_1 L U P_2$$

的分解，以减少消元中的填充 (fill-in)。

2.9.2 大型稀疏系统中的迭代法

当矩阵规模很大时，即使它是稀疏的，直接分解往往仍然会带来较大的存储负担和填充效应。这时迭代法通常更加高效，例如：

- Jacobi;
- Gauss-Seidel;
- SOR;
- 共轭梯度法 (CG);
- 预条件共轭梯度法 (PCG);
- 更一般的 Krylov 子空间方法。

注 对于大型稀疏矩阵，真正宝贵的往往不是“某一步算得多快”，而是：

1. 是否能避免显式存储整个逆矩阵；
2. 是否能只利用矩阵与向量乘法；
3. 是否能保持稀疏结构不被破坏。

这正是迭代法在大规模稀疏问题中占优势的根本原因。

2.9.3 Sherman-Morrison 公式：秩一修正 (Rank-One Update)

设 A 为可逆矩阵，现对它做一个很小的修正：

$$A \longrightarrow A + \mathbf{u}\mathbf{v}^T.$$

其中 $\mathbf{u}, \mathbf{v}$ 为列向量， $\mathbf{u}\mathbf{v}^T$ 是一个秩为 1 的矩阵，称为秩一修正 (rank-one update)。

若每次矩阵只被这样小范围修改一次，那么完全重新求逆会非常浪费。Sherman-Morrison 公式正是专门处理这种情况的。

定理 2.6 (Sherman-Morrison Formula)

若 A 可逆，且

$$1 + \mathbf{v}^T A^{-1} \mathbf{u} \neq 0,$$

则

$$(A + \mathbf{u}\mathbf{v}^T)^{-1} = A^{-1} - \frac{A^{-1} \mathbf{u} \mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1} \mathbf{u}}.$$

证明 记

$$B = A^{-1} - \frac{A^{-1} \mathbf{u} \mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1} \mathbf{u}}.$$

只需验证

$$(A + \mathbf{u}\mathbf{v}^T)B = I.$$

展开:

$$(A + \mathbf{u}\mathbf{v}^T)B = AA^{-1} - A \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}} + \mathbf{u}\mathbf{v}^T A^{-1} - \mathbf{u}\mathbf{v}^T \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}.$$

即

$$I - \frac{\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}} + \mathbf{u}\mathbf{v}^T A^{-1} - \frac{\mathbf{u}(\mathbf{v}^T A^{-1}\mathbf{u})\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}}.$$

把后面三项合并, 恰好抵消, 得到 I 。因此公式成立。

2.9.3.1 用辅助变量重写公式

令

$$\mathbf{z} = A^{-1}\mathbf{u}, \quad \mathbf{w} = (A^{-1})^T \mathbf{v}, \quad \lambda = \mathbf{v}^T \mathbf{z} = \mathbf{v}^T A^{-1}\mathbf{u},$$

则 Sherman–Morrison 公式可写成更紧凑的形式:

$$(A + \mathbf{u}\mathbf{v}^T)^{-1} = A^{-1} - \frac{\mathbf{z}\mathbf{w}^T}{1 + \lambda}.$$

注 这个公式特别适合如下场景: A^{-1} 或“解关于 A 的线性方程”已经有现成结果, 而矩阵只发生了一个秩一变化。此时无需重新求逆, 只要多做几次矩阵向量求解即可。

2.9.4 用 Sherman–Morrison 公式解修正后的线性系统

除了直接更新逆矩阵, 更常见的用途是快速求解

$$(A + \mathbf{u}\mathbf{v}^T)\mathbf{x} = \mathbf{b}.$$

设

$$\mathbf{y} = A^{-1}\mathbf{b}, \quad \mathbf{z} = A^{-1}\mathbf{u}.$$

则由 Sherman–Morrison 公式立刻得到

$$\mathbf{x} = \mathbf{y} - \frac{\mathbf{v}^T \mathbf{y}}{1 + \mathbf{v}^T \mathbf{z}} \mathbf{z}.$$

证明 由

$$\mathbf{x} = (A + \mathbf{u}\mathbf{v}^T)^{-1} \mathbf{b}$$

和 Sherman–Morrison 公式,

$$\mathbf{x} = \left(A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{v}^T A^{-1}}{1 + \mathbf{v}^T A^{-1}\mathbf{u}} \right) \mathbf{b}.$$

记 $\mathbf{y} = A^{-1}\mathbf{b}$, $\mathbf{z} = A^{-1}\mathbf{u}$, 即得

$$\mathbf{x} = \mathbf{y} - \frac{\mathbf{v}^T \mathbf{y}}{1 + \mathbf{v}^T \mathbf{z}} \mathbf{z}.$$

例题 2.9 这说明: 若原来已经会解

$$A\mathbf{y} = \mathbf{b}, \quad A\mathbf{z} = \mathbf{u},$$

那么对修正后的系统

$$(A + \mathbf{u}\mathbf{v}^T)\mathbf{x} = \mathbf{b}$$

并不需要重新做一次完整分解, 只需用这两个辅助解 $\mathbf{y}, \mathbf{z}$ 就能直接写出新解。

2.9.5 Woodbury 矩阵恒等式 (Woodbury Matrix Identity)

Sherman–Morrison 公式处理的是秩一修正。若修正项是低秩矩阵

$$UV,$$

其中

$$U \in \mathbb{R}^{n \times k}, \quad V \in \mathbb{R}^{k \times n},$$

则其推广形式就是 **Woodbury 恒等式**。

定理 2.7 (Woodbury Matrix Identity)

若 A 可逆, 且 $I_k + VA^{-1}U$ 可逆, 则

$$(A + UV)^{-1} = A^{-1} - A^{-1}U(I_k + VA^{-1}U)^{-1}VA^{-1}.$$



注 Woodbury 公式把一个 $n \times n$ 矩阵的逆更新问题, 转化成一个 $k \times k$ 小矩阵

$$I_k + VA^{-1}U$$

的求逆问题。当 $k \ll n$ 时, 这种化简极其有价值。

2.9.6 Sherman–Morrison 与 Woodbury 公式的应用

这两类公式在数值计算中用途非常广泛, 例如:

1. 拟牛顿法 (quasi-Newton methods) 中的 BFGS 更新;
2. 动态修改协方差矩阵或 Hessian 近似;
3. 统计学习中的在线更新 (online update);
4. Green 函数方法中的低秩扰动修正;
5. 卡尔曼滤波及贝叶斯计算中的增量更新。

结论 Sherman–Morrison 与 Woodbury 公式的本质思想是: 当矩阵的变化是低秩的, 就不必把整个问题从头重算, 而应利用旧结构做增量更新。

2.10 软件与程序库 (LAPACK / BLAS)

在实际的高性能计算中, 手写直接求解器效率很低。计算物理学中广泛调用成熟的库:

- **BLAS (Basic Linear Algebra Subprograms)**: 针对底层硬件优化的向量、矩阵-向量、矩阵-矩阵乘法基本操作。
- **LAPACK (Linear Algebra PACKage)**: 基于 BLAS 构建的高级线性代数求解器。
 - 调用 DGESV: 求解普通线性代数方程组 $Ax = b$ (利用 $PA = LU$ 分解)。
 - 调用 DGESVD: 进行奇异值分解。
- 其他扩展: **Scalapack** (MPI 分布式内存), **MAGMA** (LAPACK + GPU), **ARPACK** (大规模稀疏矩阵)。

2.11 全章总结

- **Gauss/LU 分解**是解一般非奇异方阵的基础直接方法, 列主元法保障了数值稳定性。
- **迭代法 (Jacobi, Gauss-Seidel, SOR)** 占用内存极小, 极其适合超大型、稀疏矩阵求解。
- **共轭梯度法 (CG)** 是对称正定矩阵的最强迭代武器。
- **SVD** 算法虽然最复杂耗时, 但却是对付病态问题、奇异退化问题、超定/欠定最小二乘拟合问题的万能钥匙。

- 结合使用 **条件数**和误差分析评估结果可靠性，在工程中善用 LAPACK 等现成优质程序库。

第三章 数值微分、插值与积分方法

内容提要

- 拉格朗日插值的基本构造、插值基函数及误差余项。
- Hermite 插值、高阶插值的龙格现象，以及分段插值的必要性。
- 三次样条插值与 Akima 插值的基本思想、构造方法及适用特点。
- 多维插值、分式插值与 Padé 近似的基本概念。
- 由插值导出的数值微分公式，包括三点、五点及二阶微分公式。
- Newton-Cotes 求积公式、复合积分公式及其误差分析。
- Richardson 外推、Romberg 积分与变步长积分方法。
- 高斯积分、Gauss-Legendre 求积及带权高斯积分的基本原理。
- 利用插值与积分方法处理 WKB 穿透概率和寿命估计问题。

3.1 本章导读

本章围绕三个彼此紧密相关的数值问题展开：

- 已知离散函数值，如何在格点之间重建函数，即**插值 (interpolation)**；
- 已知离散函数值，如何近似计算导数，即**数值微分 (numerical differentiation)**；
- 已知函数值或离散采样点，如何近似计算定积分，即**数值积分 (numerical integration / quadrature)**。

这三类问题并不是彼此孤立的。一个重要思想是：**先做插值，再对插值多项式进行微分或积分**。因此，拉格朗日插值、样条插值等工具既能服务于函数重建，也能自然导出微分公式和求积公式。

3.2 拉格朗日插值 (Lagrange Interpolation)

3.2.1 基本思想

设有 $N + 1$ 个互异节点

$$x_0, x_1, \dots, x_N,$$

并且已知函数在这些点上的取值

$$y_i = f(x_i), \quad i = 0, 1, \dots, N.$$

我们希望构造一个不超过 N 次的多项式 $P_N(x)$ ，使得

$$P_N(x_i) = f(x_i) = y_i, \quad i = 0, 1, \dots, N.$$

这样的多项式称为**插值多项式 (interpolating polynomial)**。

Lagrange 的做法是先构造一组插值基函数 (Lagrange basis functions)

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^N \frac{x - x_j}{x_i - x_j}, \quad i = 0, 1, \dots, N.$$

它们满足最关键的插值性质

$$L_i(x_j) = \delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

于是插值多项式自然写成

$$P_N(x) = \sum_{i=0}^N f(x_i) L_i(x).$$

定义 3.1

称

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^N \frac{x - x_j}{x_i - x_j}$$

为第 i 个 **Lagrange** 插值基函数。它的作用是：在节点 x_i 处取值为 1，在其他所有插值节点处都取值为 0。♣

从结构上看， $P_N(x)$ 是对每个采样值 $f(x_i)$ 乘以一个“局部贡献权重” $L_i(x)$ 再求和。虽然这是一个全局多项式，但每一项都保留了“第 i 个节点贡献了多少”的清晰解释。

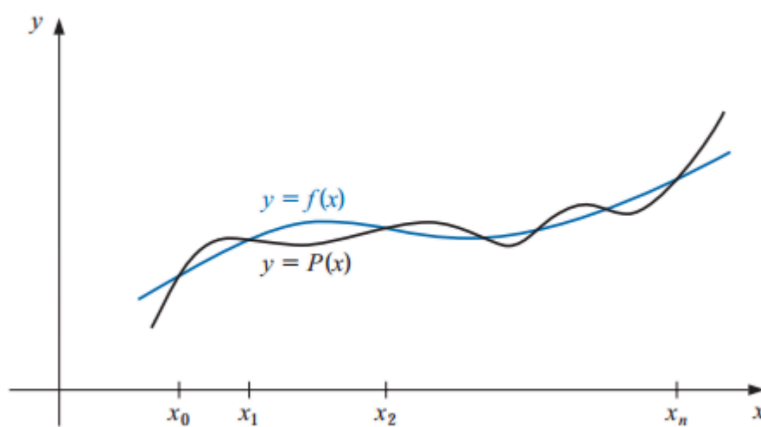


图 3.1: 插值效果图

3.2.2 两点插值与三点插值

3.2.2.1 两点直线插值

当只有两个点 x_0, x_1 时，插值多项式是一次多项式：

$$P_1(x) = f(x_0)L_0(x) + f(x_1)L_1(x),$$

其中

$$L_0(x) = \frac{x - x_1}{x_0 - x_1}, \quad L_1(x) = \frac{x - x_0}{x_1 - x_0}.$$

因此

$$P_1(x) = \frac{x - x_1}{x_0 - x_1} y_0 + \frac{x - x_0}{x_1 - x_0} y_1.$$

容易验证

$$P_1(x_0) = y_0, \quad P_1(x_1) = y_1.$$

这就是最熟悉的**线性插值** (linear interpolation)。

3.2.2.2 三点二次插值

当取三个节点 x_0, x_1, x_2 时，插值多项式为二次多项式：

$$P_2(x) = y_0 L_0(x) + y_1 L_1(x) + y_2 L_2(x),$$

其中

$$L_0(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)},$$

$$L_1(x) = \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)},$$

$$L_2(x) = \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}.$$

即

$$P_2(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}y_0 + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}y_1 + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}y_2.$$

例题 3.1 若采样点足够接近，二次插值通常比线性插值更平滑，也更能捕捉局部弯曲趋势；但如果节点取得过多、区间又很大，则高次多项式反而可能振荡得更厉害。

3.2.3 插值基函数的图像理解

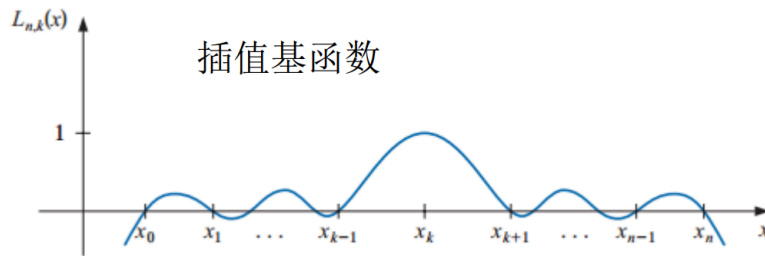


图 3.2: 插值基函数

Lagrange 基函数 $L_i(x)$ 具有如下图像特征：

1. 在 x_i 处达到 1；
2. 在其余插值节点 x_j ($j \neq i$) 处都为 0；
3. 它是一个 N 次多项式，因此在整个区间上是全局起作用的。

因此， $P_N(x) = \sum f(x_i)L_i(x)$ 的几何意义是：每个节点值 $f(x_i)$ 都通过一个“在本节点为 1、在其他节点为 0”的基函数向全区间传播。

注 Lagrange 插值的代数表达非常对称，也很适合理论推导；但若直接用于程序实现，在节点较多时重复计算乘积会比较低效。实际编程时往往更偏好 Newton 形式或重心形式 (barycentric form)，但本章的很多理论公式都从 Lagrange 形式最容易推出。

3.2.4 插值余项 (remainder term) 与误差估计

Lagrange 插值并不是一般函数的精确表示，它与原函数的差称为**插值余项**：

$$R_N(x) = f(x) - P_N(x).$$

若 $f \in C^{N+1}[x_0, x_N]$ ，则存在某个 $\xi(x) \in (x_0, x_N)$ ，使得

$$f(x) = P_N(x) + \frac{f^{(N+1)}(\xi(x))}{(N+1)!} \prod_{i=0}^N (x - x_i).$$

也即

$$R_N(x) = \frac{f^{(N+1)}(\xi(x))}{(N+1)!} \prod_{i=0}^N (x - x_i).$$

定理 3.1

若 $f \in C^{N+1}[x_0, x_N]$, 则 N 次 Lagrange 插值的误差由

$$R_N(x) = \frac{f^{(N+1)}(\xi(x))}{(N+1)!} \omega_{N+1}(x), \quad \omega_{N+1}(x) = \prod_{i=0}^N (x - x_i)$$

给出, 其中 $\xi(x) \in (x_0, x_N)$ 。



这个公式说明误差由两部分共同决定:

1. 函数本身的高阶导数 $f^{(N+1)}$, 它刻画函数本身有多“弯”;
2. 节点多项式 $\omega_{N+1}(x)$, 它刻画节点布局对误差的影响。

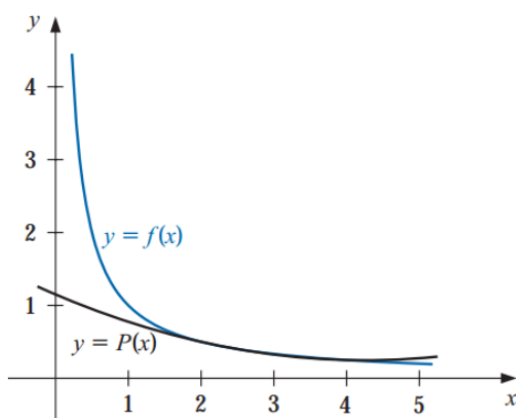
3.2.4.1 一个误差估计例子

图 3.3: 插值函数和原函数的比较

考虑

$$f(x) = x^{-1}, \quad x_0 = 2, \quad x_1 = 2.75, \quad x_2 = 4.$$

由三点插值得到一个二次多项式

$$P_2(x) = \frac{1}{22}x^2 - \frac{35}{88}x + \frac{49}{44}.$$

而

$$f'(x) = -x^{-2}, \quad f''(x) = 2x^{-3}, \quad f'''(x) = -6x^{-4}.$$

故余项为

$$R_2(x) = \frac{f'''(\xi(x))}{3!} (x-2)(x-2.75)(x-4), \quad \xi(x) \in (2, 4).$$

由于在区间 $[2, 4]$ 上有

$$|f'''(x)| = \frac{6}{x^4} \leq \frac{6}{2^4} = \frac{3}{8},$$

再结合三次因子的上界, 可得一个较小的误差估计。结果量级约为

$$|R_2(x)| \lesssim \frac{3}{512} \approx 0.00586.$$



笔记 误差公式最重要的意义并不在于每次都精确估计出常数, 而在于告诉我们:

1. 节点越合理, $\prod (x - x_i)$ 越小;
2. 函数越平滑, 高阶导数越小;
3. 盲目提高插值次数并不一定会使误差变小。

3.3 Hermite 插值 (Hermite Interpolation)

3.3.1 为什么需要 Hermite 插值

Lagrange 插值只利用了节点处的函数值 $f(x_i)$ 。但在很多物理问题中，除了函数值，我们还知道节点处的导数值 $f'(x_i)$ 。这时只要求

$$H(x_i) = f(x_i)$$

还不够，我们还希望

$$H'(x_i) = f'(x_i).$$

这种既匹配函数值又匹配导数值的插值称为 **Hermite 插值**。

若有 $n+1$ 个节点，并同时给定每个节点的一阶导数，则总共有 $2(n+1)$ 个条件，因此 Hermite 插值多项式一般取到 $2n+1$ 次。

3.3.2 Hermite 插值公式

设

$$L_{n,j}(x) = \prod_{\substack{k=0 \\ k \neq j}}^n \frac{x - x_k}{x_j - x_k}$$

是对应的 Lagrange 基函数，则 Hermite 插值多项式可写为

$$H_{2n+1}(x) = \sum_{j=0}^n f(x_j) H_{n,j}(x) + \sum_{j=0}^n f'(x_j) \hat{H}_{n,j}(x),$$

其中

$$H_{n,j}(x) = [1 - 2(x - x_j)L'_{n,j}(x_j)]L_{n,j}^2(x),$$

$$\hat{H}_{n,j}(x) = (x - x_j)L_{n,j}^2(x).$$

性质 Hermite 基函数满足

$$H_{n,j}(x_i) = \delta_{ij}, \quad \hat{H}_{n,j}(x_i) = 0,$$

以及

$$H'_{n,j}(x_i) = 0, \quad \hat{H}'_{n,j}(x_i) = \delta_{ij}.$$

因此 $H_{2n+1}(x)$ 自动满足

$$H_{2n+1}(x_i) = f(x_i), \quad H'_{2n+1}(x_i) = f'(x_i).$$

3.3.3 Hermite 插值的余项

若 $f \in C^{2n+2}$ ，则 Hermite 插值的余项为

$$f(x) = H_{2n+1}(x) + \frac{(x - x_0)^2(x - x_1)^2 \cdots (x - x_n)^2}{(2n+2)!} f^{(2n+2)}(\xi(x)).$$

与 Lagrange 余项相比，这里每个节点因子都出现了平方，反映了“函数值与导数值都被拟合”的更高匹配程度。

注 Hermite 插值的本质思想是：若已知更多局部信息，就应把这些信息直接纳入插值条件中，而不是浪费掉。它比单纯的 Lagrange 插值有更高的信息利用率，但代价是公式更复杂、稳定性分析也更细致。

3.4 高阶插值的不稳定性与龙格现象 (Runge Phenomenon)

3.4.1 龙格现象

高阶多项式插值最典型的问题是**龙格现象**：随着插值次数升高，插值多项式在区间边缘处可能发生剧烈振荡，误差甚至不降反升。

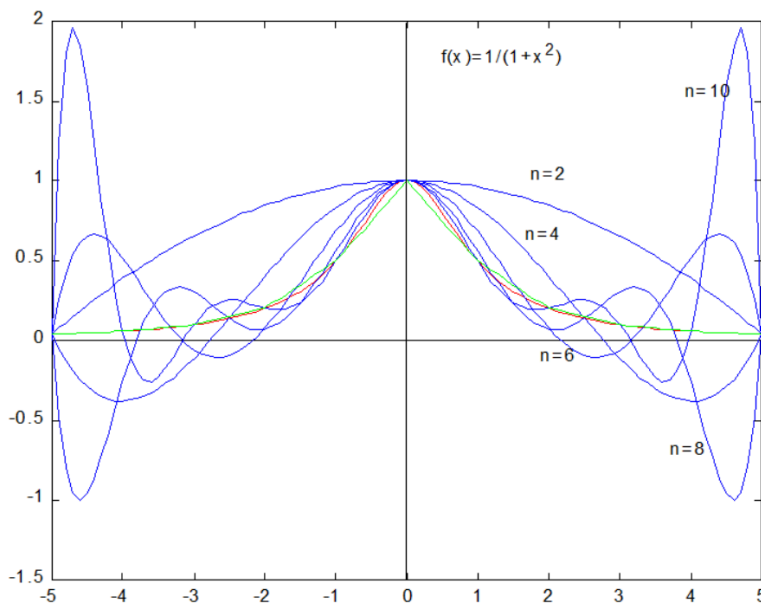


图 3.4: 龙格现象

对于许多被插函数，随着多项式阶数增加，**边缘点误差可能趋于无穷**。这不是个别例外，而是全局高阶插值的普遍风险。

定义 3.2

所谓龙格现象 (Runge phenomenon)，是指在等距节点上对一个足够光滑的函数做高次多项式插值时，插值多项式在区间端点附近出现越来越强的振荡，从而导致误差放大。

3.4.2 为什么会出现这种现象

从误差公式

$$R_N(x) = \frac{f^{(N+1)}(\xi(x))}{(N+1)!} \prod_{i=0}^N (x - x_i)$$

看，问题并不只在于高阶导数，也在节点多项式 $\omega_{N+1}(x)$ 。当节点均匀分布而次数又很高时， $\omega_{N+1}(x)$ 在端点附近往往会变得很大，导致误差放大。

笔记 这也是后面引入三次样条、Akima 插值等分段插值 (piecewise interpolation) 方法的直接动机：既要保留插值的精度，又要避免高阶多项式的全局振荡。

提醒：对高阶插值要保持谨慎 (be cautious about high-order interpolation)，尤其在数据可能较“硬” (stiff) 或曲率变化很强时更是如此。

3.5 三次样条插值 (Cubic Spline Interpolation)

3.5.1 基本思想

为了避免高阶全局多项式的振荡，我们把整体区间

$$x_0 < x_1 < \cdots < x_n$$

分成 n 个小区间 $[x_j, x_{j+1}]$ ，并在每个小区间上用一个三次多项式

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3, \quad j = 0, 1, \dots, n-1$$

去逼近函数。

这样总共有 $4n$ 个待定系数。为了唯一确定这些系数，需要加入插值条件、光滑连接条件以及边界条件。

3.5.2 插值条件与光滑连接条件

3.5.2.1 插值条件

每一段样条必须通过本段两端节点：

$$S_j(x_j) = f(x_j), \quad S_j(x_{j+1}) = f(x_{j+1}).$$

其中

$$S_j(x_j) = a_j = f(x_j).$$

3.5.2.2 光滑连接条件

在内部节点 x_{j+1} 处，相邻两段样条不仅函数值连续，而且一阶、二阶导数都连续：

$$S_{j+1}(x_{j+1}) = S_j(x_{j+1}),$$

$$S'_{j+1}(x_{j+1}) = S'_j(x_{j+1}),$$

$$S''_{j+1}(x_{j+1}) = S''_j(x_{j+1}).$$

因此三次样条属于 C^2 分段多项式。

定义 3.3

若分段三次多项式 $S(x)$ 满足

1. 在每个小区间 $[x_j, x_{j+1}]$ 上， $S(x)$ 是三次多项式；
2. $S(x_i) = f(x_i)$ ；
3. S, S', S'' 在整个区间上连续；

则称 $S(x)$ 为通过节点 $\{x_i\}$ 的三次样条插值函数 (cubic spline interpolant)。



3.5.3 常见边界条件

给出两类常见边界条件：

3.5.3.1 自然边界条件 (natural boundary conditions)

$$S''(x_0) = 0, \quad S''(x_n) = 0.$$

这相当于把样条曲线在两端看作“无弯矩”自由端。

3.5.3.2 一阶导数边界条件 (clamped boundary conditions)

$$S'(x_0) = f'(x_0), \quad S'(x_n) = f'(x_n).$$

也称**夹持边界条件 (clamped spline)**，它要求样条在端点的切线斜率已知。

3.5.4 三点样条例子

给出节点

$$(1, 2), \quad (2, 3), \quad (3, 5).$$

我们在两个区间上分别设

$$S_0(x) = a_0 + b_0(x-1) + c_0(x-1)^2 + d_0(x-1)^3, \quad x \in [1, 2],$$

$$S_1(x) = a_1 + b_1(x-2) + c_1(x-2)^2 + d_1(x-2)^3, \quad x \in [2, 3].$$

由插值条件得

$$a_0 = 2, \quad a_1 = 3,$$

$$a_0 + b_0 + c_0 + d_0 = 3, \quad a_1 + b_1 + c_1 + d_1 = 5.$$

在 $x = 2$ 处的一阶、二阶连续条件为

$$b_0 + 2c_0 + 3d_0 = b_1,$$

$$2c_0 + 6d_0 = 2c_1.$$

若取自然边界条件

$$S''_0(1) = 2c_0 = 0, \quad S''_1(3) = 2c_1 + 6d_1 = 0,$$

最终可解得结果

$$S(x) = \begin{cases} 2 + \frac{3}{4}(x-1) + \frac{1}{4}(x-1)^3, & x \in [1, 2], \\ 3 + \frac{3}{2}(x-2) + \frac{3}{4}(x-2)^2 - \frac{1}{4}(x-2)^3, & x \in [2, 3]. \end{cases}$$

 **笔记** 这个例子很好地说明了样条方法的特点：虽然每一段只用三次多项式，但通过连续条件把各段粘接起来后，整体曲线既平滑，又避免了高次全局多项式的强振荡。

3.5.5 一般系数关系的推导

记区间长度

$$h_j = x_{j+1} - x_j, \quad j = 0, 1, \dots, n-1.$$

由

$$a_j = S_j(x_j) = f(x_j)$$

可知 a_j 直接由数据给定。

先写出在 x_{j+1} 处的函数值连续条件：

$$a_{j+1} = S_j(x_{j+1}) = a_j + b_j h_j + c_j h_j^2 + d_j h_j^3.$$

再写一阶导数与二阶导数：

$$S'_j(x) = b_j + 2c_j(x - x_j) + 3d_j(x - x_j)^2,$$

$$S_j''(x) = 2c_j + 6d_j(x - x_j).$$

于是

$$b_{j+1} = b_j + 2c_j h_j + 3d_j h_j^2,$$

$$c_{j+1} = c_j + 3d_j h_j.$$

由最后一个式子消去 d_j :

$$d_j = \frac{c_{j+1} - c_j}{3h_j}.$$

代回前式可得

$$a_{j+1} = a_j + b_j h_j + \frac{h_j^2}{3}(2c_j + c_{j+1}),$$

$$b_{j+1} = b_j + h_j(c_j + c_{j+1}).$$

3.5.6 消去 b_j 后的三对角方程组

由

$$a_{j+1} = a_j + b_j h_j + \frac{h_j^2}{3}(2c_j + c_{j+1})$$

得到

$$b_j = \frac{a_{j+1} - a_j}{h_j} - \frac{h_j}{3}(2c_j + c_{j+1}).$$

同理

$$b_{j-1} = \frac{a_j - a_{j-1}}{h_{j-1}} - \frac{h_{j-1}}{3}(2c_{j-1} + c_j).$$

再利用

$$b_j = b_{j-1} + h_{j-1}(c_{j-1} + c_j),$$

可得内部节点满足

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_j c_{j+1} = 3\frac{a_{j+1} - a_j}{h_j} - 3\frac{a_j - a_{j-1}}{h_{j-1}},$$

其中 $j = 1, 2, \dots, n-1$ 。

这就是三次样条求解的核心：全部未知量最终化成了 c_j 的三对角线性方程组。

3.5.7 自然样条的矩阵形式

对自然样条有

$$S''(x_0) = 2c_0 = 0, \quad S''(x_n) = 2c_n = 0,$$

即

$$c_0 = 0, \quad c_n = 0.$$

于是未知向量

$$\mathbf{x} = (c_0, c_1, \dots, c_n)^\top$$

满足线性方程组

$$A\mathbf{x} = \mathbf{b},$$

其中

$$A = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 & 0 \\ h_0 & 2(h_0 + h_1) & h_1 & \cdots & 0 & 0 \\ 0 & h_1 & 2(h_1 + h_2) & \ddots & 0 & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \ddots & 2(h_{n-2} + h_{n-1}) & h_{n-1} \\ 0 & 0 & 0 & \cdots & 0 & 1 \end{pmatrix},$$

$$\mathbf{b} = \begin{pmatrix} 0 \\ 3 \frac{a_2 - a_1}{h_1} - 3 \frac{a_1 - a_0}{h_0} \\ 3 \frac{a_3 - a_2}{h_2} - 3 \frac{a_2 - a_1}{h_1} \\ \vdots \\ 3 \frac{a_n - a_{n-1}}{h_{n-1}} - 3 \frac{a_{n-1} - a_{n-2}}{h_{n-2}} \\ 0 \end{pmatrix}.$$

注 由于 A 是三对角矩阵, 求解成本很低, 可以用 Thomas 算法在线性时间 $O(n)$ 内完成。这也是三次样条在实际计算中非常高效的重要原因。

3.5.8 给定端点一阶导数时的方程组

若边界条件改为

$$S'(x_0) = f'(a), \quad S'(x_n) = f'(b),$$

则端点方程变为

$$2h_0c_0 + h_0c_1 = 3 \frac{a_1 - a_0}{h_0} - 3f'(a),$$

$$h_{n-1}c_{n-1} + 2h_{n-1}c_n = 3f'(b) - 3 \frac{a_n - a_{n-1}}{h_{n-1}}.$$

中间的内部方程保持不变, 因此依然得到一个三对角系统。

3.5.9 若给定的是二阶导数边界条件

因为

$$S''(x_0) = 2c_0, \quad S''(x_n) = 2c_n,$$

若给定

$$S''(x_0) = \alpha, \quad S''(x_n) = \beta,$$

则只需令

$$c_0 = \frac{\alpha}{2}, \quad c_n = \frac{\beta}{2},$$

并保持内部方程不变即可。这说明三次样条对不同边界条件具有统一的处理框架。

3.5.10 非均匀格点 (nonuniform grids)

我们自然地提出问题: 样条插值是否可以推广到非均匀格点?

答案是：不仅可以，而且上面的推导本身已经是非均匀格点版本，因为我们始终保留了

$$h_j = x_{j+1} - x_j$$

而没有假设所有 h_j 相等。均匀网格只是特殊情形

$$h_0 = h_1 = \cdots = h_{n-1} = h.$$

结论 三次样条的核心优势在于：

1. 分段处理，避免全局高阶振荡；
2. C^2 光滑，适合后续求导；
3. 最终只需求解一个三对角线性方程组，计算非常高效；
4. 对非均匀格点天然适用。

3.6 Akima 插值 (Akima Interpolation)

3.6.1 基本思想

Akima 插值是一种**分段三次插值**。它和三次样条一样能得到光滑的分段三次曲线，但不需要解全局线性方程组，因此具有更强的局部性 (locality)。

其优点是：

1. computationally very efficient and very precise;
2. 避免振荡；
3. 减少误差传播；
4. 具有良好的 locality。

这使它在实验数据处理、图形平滑、局部突变附近的插值中非常常用。

3.6.2 区间上的三次表示

在区间 $[x_i, x_{i+1}]$ 上取

$$s(x) = a_0 + a_1(x - x_i) + a_2(x - x_i)^2 + a_3(x - x_i)^3.$$

Akima 方法的关键不在于三次形式本身，而在于如何确定每个节点处的一阶导数 s'_i 。

先定义相邻节点的割线斜率

$$d_j = \frac{s_{j+1} - s_j}{x_{j+1} - x_j}.$$

再用斜率差构造权重

$$w_{i-1} = |d_{i+1} - d_i|, \quad w_i = |d_{i-1} - d_i|.$$

于是第 i 个节点处的一阶导数取为

$$s'_i = \frac{w_{i-1}d_{i-1} + w_id_i}{w_{i-1} + w_i}.$$

 **笔记** Akima 的直觉非常漂亮：若某一侧相邻斜率变化更剧烈，则说明该侧数据更“不稳定”，于是给它更小的影响；若某侧更平稳，则在确定导数时让它占更大权重。

3.6.3 特殊情况与边界处理

当分母 $w_{i-1} + w_i = 0$ 时，说明附近斜率变化过于特殊，需要改用特判。常见做法是用附近割线斜率的平均来替代，例如

$$s'_i = \frac{d_{i-1} + d_i}{2}$$

或使用其他指定的分段条件。

由于内点导数公式需要用到更外层的斜率，还要补充边界外推。例如常见的边界补充方式是

$$\begin{aligned} d_0 &= 2d_1 - d_2, & d_{-1} &= 2d_0 - d_1, \\ d_k &= 2d_{k-1} - d_{k-2}, & d_{k+1} &= 2d_k - d_{k-1}. \end{aligned}$$

3.6.4 由端点函数值与导数确定区间三次多项式

若在区间 $[x_i, x_{i+1}]$ 已知

$$y_i, \quad y_{i+1}, \quad t_i = s'_i, \quad t_{i+1} = s'_{i+1},$$

则可写成

$$y = p_0 + p_1(x - x_i) + p_2(x - x_i)^2 + p_3(x - x_i)^3,$$

其中

$$\begin{aligned} p_0 &= y_i, & p_1 &= t_i, \\ p_2 &= \frac{3(y_{i+1} - y_i)}{(x_{i+1} - x_i)^2} - \frac{2t_i + t_{i+1}}{x_{i+1} - x_i}, \\ p_3 &= \frac{t_i + t_{i+1} - 2\frac{y_{i+1} - y_i}{x_{i+1} - x_i}}{(x_{i+1} - x_i)^2}. \end{aligned}$$

注 Akima 插值通常只有一阶导数连续，而二阶导数可以不连续 (a discontinuous second derivative)。这一点与 C^2 的三次样条不同。代价是光滑性稍弱，优点是局部性更强、对异常点更稳健。

3.7 多维插值 (Multidimensional Interpolation)

3.7.1 双线性插值 (bilinear interpolation)

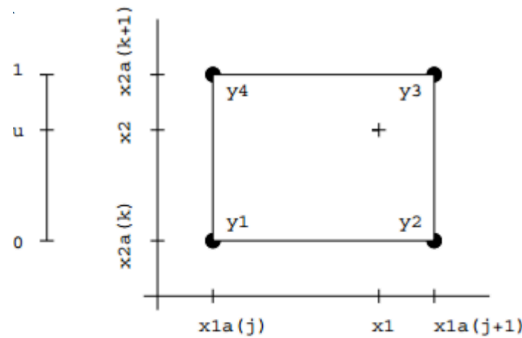


图 3.5: 双线性插值

对于二维规则网格上的一个小矩形单元，设四个角点函数值为

$$y_1, y_2, y_3, y_4.$$

把局部坐标归一化到

$$t = \frac{x_1 - x_1^{(j)}}{x_1^{(j+1)} - x_1^{(j)}}, \quad u = \frac{x_2 - x_2^{(k)}}{x_2^{(k+1)} - x_2^{(k)}},$$

则双线性插值公式为

$$y(x_1, x_2) = (1-t)(1-u)y_1 + t(1-u)y_2 + t uy_3 + (1-t)uy_4.$$

注 这个公式也可以理解成**面积权重 (area weights)**：点越靠近某个顶点，该顶点的函数值所占权重就越大。

3.7.2 高维情形的递推思想

以二维为例，思路是：

1. 先固定第二个变量，在第一个方向上做一维插值，得到若干中间值 $V_i(x)$ ；
2. 再把这些中间值作为关于第二个变量的数据，继续插值。

三维乃至更高维的情形可以依此类推，本质上是**张量积插值 (tensor-product interpolation)** 的思想。

3.7.3 其他多维插值方法

1. 回归分析 (regression analysis)；
2. Kriging 方法 (克里金法)；
3. Radial Basis Function, RBF (径向基函数)。

这些方法通常用于数据不规则分布、含噪数据逼近或地学统计等更复杂场景。

3.8 分式插值与函数逼近 (Rational Interpolation and Approximation)

3.8.1 分式多项式插值 (Rational Function Interpolation)

除了用多项式做插值，还可以用分式函数

$$R(x) = \frac{P_\mu(x)}{Q_\nu(x)} = \frac{p_0 + p_1x + \cdots + p_\mu x^\mu}{q_0 + q_1x + \cdots + q_\nu x^\nu}$$

进行插值。若作归一化 $q_0 = 1$ ，则未知系数总数为

$$(\mu + 1) + \nu = \mu + \nu + 1.$$

因此若有 $m + 1$ 个插值点，通常要求

$$m + 1 = \mu + \nu + 1.$$

注 分式插值的突出优点是：**可以描述极点 (poles) 和零点 (zeros of denominators)**。因此对于含有奇异行为的函数，分式逼近往往远优于普通多项式逼近。

3.8.2 Padé 近似 (Padé approximation)

Padé 近似是分式逼近中最重要的一类。若

$$f(x) = \sum_{k=0}^{\infty} c_k x^k,$$

则构造

$$R_{M,N}(x) = \frac{\sum_{k=0}^M a_k x^k}{1 + \sum_{k=1}^N b_k x^k},$$

要求它在 $x = 0$ 附近与 $f(x)$ 的 Taylor 展开尽可能一致，即

$$\left(1 + \sum_{k=1}^N b_k x^k\right) f(x) - \sum_{k=0}^M a_k x^k = O(x^{M+N+1}).$$

这等价于让左右两边在前 $M + N$ 阶系数完全匹配，从而得到一个线性方程组求出 b_k ，随后再反推出 a_k 。

定义 3.4

Padé 近似的本质是：用一个分式函数去匹配函数在某点附近的幂级数展开，并让尽可能多的低阶系数相同。

3.8.3 函数逼近的几种常见方式

我们由插值问题自然过渡到更一般的**函数逼近 (approximation)** 问题，列出三类典型方法：

1. Padé approximation;
2. Taylor 展开;
3. Chebyshev 展开。

例如

$$\sin x \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots + (-1)^n \frac{x^{2n+1}}{(2n+1)!}$$

是 Taylor 逼近；而

$$f(x) = \sin \frac{\pi}{2}x = \frac{1}{2}C_0 + \sum_{k=1}^{\infty} C_k T_k(x)$$

则是 Chebyshev 多项式展开的形式。

 **笔记** 在逼近理论里，Taylor 展开强调局部，Chebyshev 展开强调全区间最优性质，Padé 则善于逼近具有极点或有理结构的函数。三者各有适用场景。

3.9 数值微分 (Numerical Differentiation)

数值微分的基本任务是：在只知道函数离散取值

$$f(x_0), f(x_1), \dots, f(x_n)$$

的情况下，近似计算某点处的一阶或二阶导数。与数值积分相比，数值微分通常对误差更敏感，因为微分本质上会放大局部扰动与高频噪声，因此构造高精度、稳定的微分公式尤为重要。

本节的核心思想有两条：

1. 直接利用 Taylor 展开构造差分公式；
2. 更系统地，从插值多项式，尤其是 Lagrange 插值多项式出发，推导 n 点微分公式。

3.9.1 最基本的前向差分 (Forward Difference)

由 Taylor 展开，

$$f(x_0 + h) = f(x_0) + hf'(x_0) + \frac{h^2}{2}f''(\xi), \quad \xi \in (x_0, x_0 + h),$$

整理得

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2}f''(\xi).$$

因此，前向差分近似写成

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h},$$

其截断误差为

$$R = O(h).$$

注 前向差分只用到 x_0 与 $x_0 + h$ 两个点，因此实现最简单，但精度只有一阶。类似地，也可以得到后向差分

$$f'(x_0) \approx \frac{f(x_0) - f(x_0 - h)}{h},$$

误差同样是 $O(h)$ 。

3.9.2 从 Lagrange 插值出发的一般 n 点微分公式

设节点

$$x_0, x_1, \dots, x_n$$

互不相同, 函数在这些点上的值已知。令 $L_k(x)$ 为 Lagrange 基函数:

$$L_k(x) = \frac{(x-x_0) \cdots (x-x_{k-1})(x-x_{k+1}) \cdots (x-x_n)}{(x_k-x_0) \cdots (x_k-x_{k-1})(x_k-x_{k+1}) \cdots (x_k-x_n)}.$$

则插值多项式为

$$P_n(x) = \sum_{k=0}^n f(x_k) L_k(x).$$

带余项的插值公式写成

$$f(x) = \sum_{k=0}^n f(x_k) L_k(x) + \frac{(x-x_0)(x-x_1) \cdots (x-x_n)}{(n+1)!} f^{(n+1)}(\xi(x)).$$

对 x 求导, 得到

$$f'(x) = \sum_{k=0}^n f(x_k) L'_k(x) + \frac{d}{dx} \left[\frac{(x-x_0)(x-x_1) \cdots (x-x_n)}{(n+1)!} \right] f^{(n+1)}(\xi(x)) + \frac{(x-x_0) \cdots (x-x_n)}{(n+1)!} \frac{d}{dx} [f^{(n+1)}(\xi(x))].$$

在实际使用时, 通常把后两项整体视为误差项, 于是得到一般形式

$$f'(x) \approx \sum_{k=0}^n f(x_k) L'_k(x).$$

这说明: 数值微分公式本质上就是对插值多项式求导。

 **笔记** 这种方法的优点是系统、统一, 而且很容易推广到三点、五点, 乃至更多点的微分公式。后面看到的各种差分系数, 本质上都来自 $L'_k(x)$ 在特定节点处的取值。

3.9.3 三点微分公式 (Three-Point Formula)

取三个节点

$$x_0, \quad x_1, \quad x_2,$$

利用二次插值多项式求导, 可得一般三点公式

$$f'(x_j) = f(x_0) \frac{2x_j - x_1 - x_2}{(x_0 - x_1)(x_0 - x_2)} + f(x_1) \frac{2x_j - x_0 - x_2}{(x_1 - x_0)(x_1 - x_2)} + f(x_2) \frac{2x_j - x_0 - x_1}{(x_2 - x_0)(x_2 - x_1)} + \frac{1}{6} f^{(3)}(\xi_j) \prod_{\substack{k=0 \\ k \neq j}}^2 (x_j - x_k).$$

为了得到常用的等距格式, 令

$$x_1 = x_0 + h, \quad x_2 = x_0 + 2h.$$

则在三个节点上分别有:

在左端点 x_0 处

$$f'(x_0) = \frac{1}{h} \left[-\frac{3}{2} f(x_0) + 2f(x_0 + h) - \frac{1}{2} f(x_0 + 2h) \right] + \frac{h^2}{3} f^{(3)}(\xi_0).$$

整理成更常见的形式:

$$f'(x_0) = \frac{-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)}{2h} + \frac{h^2}{3} f^{(3)}(\xi_0).$$

这称为端点三点公式 (endpoint three-point formula)。

在中点 $x_0 + h$ 处

$$f'(x_0 + h) = \frac{1}{h} \left[-\frac{1}{2}f(x_0) + \frac{1}{2}f(x_0 + 2h) \right] - \frac{h^2}{6}f^{(3)}(\xi_1),$$

即

$$f'(x_0 + h) = \frac{f(x_0 + 2h) - f(x_0)}{2h} - \frac{h^2}{6}f^{(3)}(\xi_1).$$

若把 $x_0 + h$ 记作一般点 x_0 , 便得到标准的中点三点公式 (central three-point formula)

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} - \frac{h^2}{6}f^{(3)}(\xi_1).$$

在右端点 $x_0 + 2h$ 处

$$f'(x_0 + 2h) = \frac{1}{h} \left[\frac{1}{2}f(x_0) - 2f(x_0 + h) + \frac{3}{2}f(x_0 + 2h) \right] + \frac{h^2}{3}f^{(3)}(\xi_2),$$

即

$$f'(x_0 + 2h) = \frac{f(x_0) - 4f(x_0 + h) + 3f(x_0 + 2h)}{2h} + \frac{h^2}{3}f^{(3)}(\xi_2).$$

平移记号后可写成另一种端点三点公式

$$f'(x_0) = \frac{f(x_0 - 2h) - 4f(x_0 - h) + 3f(x_0)}{2h} + \frac{h^2}{3}f^{(3)}(\xi_2).$$

结论 三点公式中最重要的三条结论是:

$$f'(x_0) = \frac{-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)}{2h} + O(h^2),$$

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} + O(h^2),$$

$$f'(x_0) = \frac{f(x_0 - 2h) - 4f(x_0 - h) + 3f(x_0)}{2h} + O(h^2).$$

其中中点三点公式对称性最好, 是最常用的一阶导数中心差分格式。

3.9.4 五点微分公式 (Five-Point Formula)

为了进一步提高精度, 可以使用更多节点。最经典的是五点中心差分公式:

$$f'(x_0) = \frac{1}{12h} \left[f(x_0 - 2h) - 8f(x_0 - h) + 8f(x_0 + h) - f(x_0 + 2h) \right] + \frac{h^4}{30}f^{(5)}(\xi).$$

因此

$$f'(x_0) \approx \frac{f(x_0 - 2h) - 8f(x_0 - h) + 8f(x_0 + h) - f(x_0 + 2h)}{12h},$$

其截断误差为

$$O(h^4).$$

这说明五点中心公式比三点中心公式更精确: 前者为四阶精度, 后者为二阶精度。

五点端点公式

在边界处无法使用对称点, 因此需要单边高阶公式。一个常用的五点端点公式是

$$f'(x_0) = \frac{1}{12h} \left[-25f(x_0) + 48f(x_0 + h) - 36f(x_0 + 2h) + 16f(x_0 + 3h) - 3f(x_0 + 4h) \right] + \frac{h^4}{5} f^{(5)}(\xi).$$

因此

$$f'(x_0) \approx \frac{-25f(x_0) + 48f(x_0 + h) - 36f(x_0 + 2h) + 16f(x_0 + 3h) - 3f(x_0 + 4h)}{12h},$$

同样具有四阶精度。

注 五点端点公式非常有用, 因为在区间边界附近不能使用中心差分, 而数值算法又往往需要在边界处估算导数, 例如三次样条插值中的边界条件构造就可以用到这类高精度端点微分公式。

五点公式的统一观点

五点微分公式不仅能给出中点和端点处的导数, 还可以在五个连续节点

$$x_0, x_1, x_2, x_3, x_4$$

上分别构造一阶导数公式, 其系数可统一写成

$$\begin{cases} f'(x_0) = \frac{1}{12h} (-25y_0 + 48y_1 - 36y_2 + 16y_3 - 3y_4), \\ f'(x_1) = \frac{1}{12h} (-3y_0 - 10y_1 + 18y_2 - 6y_3 + y_4), \\ f'(x_2) = \frac{1}{12h} (y_0 - 8y_1 + 8y_3 - y_4), \\ f'(x_3) = \frac{1}{12h} (-y_0 + 6y_1 - 18y_2 + 10y_3 + 3y_4), \\ f'(x_4) = \frac{1}{12h} (3y_0 - 16y_1 + 36y_2 - 48y_3 + 25y_4), \end{cases}$$

其中

$$y_j = f(x_j).$$

对应的误差项分别为四阶量级 $O(h^4)$ 。



笔记 从这一统一表达可以看出: 中心点附近的差分系数更对称, 端点处的系数则明显偏向单边。这正反映了边界点可利用的信息不如内部点丰富。

3.9.5 二阶导数公式 (Second Derivative Formula)

除了求一阶导数, 数值计算中也经常需要二阶导数。由 Taylor 展开:

$$f(x_0 + h) = f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + \frac{1}{6}f^{(3)}(x_0)h^3 + \frac{1}{24}f^{(4)}(\xi_1)h^4,$$

$$f(x_0 - h) = f(x_0) - f'(x_0)h + \frac{1}{2}f''(x_0)h^2 - \frac{1}{6}f^{(3)}(x_0)h^3 + \frac{1}{24}f^{(4)}(\xi_{-1})h^4.$$

两式相加, 奇数阶导数项抵消, 得到

$$f(x_0 + h) + f(x_0 - h) = 2f(x_0) + f''(x_0)h^2 + \frac{1}{24}[f^{(4)}(\xi_1) + f^{(4)}(\xi_{-1})]h^4.$$

因此

$$f''(x_0) = \frac{f(x_0 - h) - 2f(x_0) + f(x_0 + h)}{h^2} - \frac{h^2}{24}[f^{(4)}(\xi_1) + f^{(4)}(\xi_{-1})].$$

于是二阶导数的三点中心公式为

$$f''(x_0) \approx \frac{f(x_0 - h) - 2f(x_0) + f(x_0 + h)}{h^2},$$

其误差为

$$O(h^2).$$

注 二阶导数的这一公式在离散偏微分方程中极其重要（需要记忆）。例如一维 Poisson 方程、热传导方程、波动方程等，几乎都要用到

$$\frac{f(x_0 - h) - 2f(x_0) + f(x_0 + h)}{h^2}$$

这样的离散 Laplace 算子。

3.9.6 精度、误差与格式选择

数值微分公式的误差主要由两个部分构成：

1. **截断误差 (truncation error)**：来自 Taylor 展开或插值余项的截断；
2. **舍入误差 (round-off error)**：来自有限精度浮点运算。

一般来说，使用更多节点、构造更高阶差分格式，可以减小截断误差；但同时也会使公式更复杂，并可能放大数据噪声。因此在实际计算中并不是阶数越高越好，而应综合考虑：

- 数据是否足够平滑；
- 网格是否均匀；
- 是否靠近边界；
- 是否存在测量噪声；
- 算法所要求的精度与代价。

3.9.7 其他数值微分方法

除了差分公式外，还有两类重要的数值微分思路。

1. 先插值，再求导

若先用插值函数逼近原函数，例如多项式插值、样条插值 (spline interpolation)，则可以直接对插值函数求导，从而得到一阶或二阶导数。这种方法的优点是：

1. 可以同时得到函数值、导数值、曲率等多种信息；
2. 对离散数据更平滑，适合含噪数据处理；
3. 特别适合构造连续导数，如三次样条可自然给出一阶、二阶导数。

2. 基矢展开法 (basis expansion methods)

还可以把函数展开在某组基底上，再对展开式求导，例如：

- Fourier 变换 (傅里叶方法)；
- B-spline 展开；
- 小波分析 (wavelet analysis)。

这类方法特别适合具有频域结构、周期性结构或多尺度结构的问题。

注 对于高精度数值计算，尤其是光滑函数与规则网格上的问题，谱方法 (spectral methods) 常常能获得远高于普通差分法的精度；但它们也更依赖问题本身的结构。

3.10 数值积分 (Numerical Integration)

数值积分的目标是：在函数 $f(x)$ 的解析积分难以直接求出、或者只知道若干离散函数值时，近似计算定积分

$$I = \int_a^b f(x) dx.$$

与数值微分相比，数值积分通常更稳定，也更容易达到较高精度。这一方面是因为积分本身具有“平均化”效应，另一方面也是因为积分公式往往来自对插值多项式的积分，而积分会在一定程度上抵消插值误差的振荡部分。

本节主要介绍三条主线：

1. 从 Lagrange 插值出发构造一般插值求积公式；
2. 在等距节点上得到牛顿-柯特斯 (Newton-Cotes) 公式，重点讨论梯形公式、Simpson 公式和五点柯特斯公式；
3. 为减小大区间上的误差，引入复合求积公式，重点讨论复合 Simpson 公式。

3.10.1 由插值公式导出一般求积公式

设

$$x_0, x_1, \dots, x_n \in [a, b]$$

是互异节点，对函数 f 作 n 次 Lagrange 插值，多项式记为

$$P_n(x) = \sum_{i=0}^n f(x_i) L_i(x),$$

其中 $L_i(x)$ 为 Lagrange 基函数。带余项的插值公式为

$$f(x) = P_n(x) + \frac{1}{(n+1)!} \omega_{n+1}(x) f^{(n+1)}(\xi(x)),$$

其中

$$\omega_{n+1}(x) = \prod_{i=0}^n (x - x_i), \quad \xi(x) \in (a, b).$$

对上式在 $[a, b]$ 上积分，得到

$$\int_a^b f(x) dx = \int_a^b \sum_{i=0}^n f(x_i) L_i(x) dx + \frac{1}{(n+1)!} \int_a^b \omega_{n+1}(x) f^{(n+1)}(\xi(x)) dx.$$

于是可写成

$$\int_a^b f(x) dx = \sum_{i=0}^n A_i f(x_i) + R_n(f),$$

其中

$$A_i = \int_a^b L_i(x) dx,$$

而余项为

$$R_n(f) = \frac{1}{(n+1)!} \int_a^b \omega_{n+1}(x) f^{(n+1)}(\xi(x)) dx.$$

定义 3.5

形如

$$\int_a^b f(x) dx \approx \sum_{i=0}^n A_i f(x_i)$$

的公式称为插值型求积公式 (interpolatory quadrature formula)。其中 A_i 称为求积权重 (weights)。



注 这一公式的本质非常清楚：先用插值多项式近似函数，再把多项式精确积分。于是，数值积分问题被转化为了“求若干节点值的加权和”的问题。

3.10.2 Simpson 公式的推导

取三个等距节点

$$x_0 = a, \quad x_1 = \frac{a+b}{2}, \quad x_2 = b.$$

用二次插值多项式 $L_2(x)$ 近似 $f(x)$ ，则

$$\int_a^b f(x) dx \approx \int_a^b L_2(x) dx = A_0 f(x_0) + A_1 f(x_1) + A_2 f(x_2).$$

直接计算三个权重：

$$A_0 = \int_a^b l_0(x) dx = \frac{1}{6}(b-a),$$

$$A_1 = \int_a^b l_1(x) dx = \frac{4}{6}(b-a),$$

$$A_2 = \int_a^b l_2(x) dx = \frac{1}{6}(b-a).$$

因此得到

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right].$$

定义 3.6

公式

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

称为 **Simpson 公式**，也称抛物线公式。

若记

$$h = \frac{b-a}{2},$$

则 Simpson 公式也可写成

$$\int_a^b f(x) dx \approx \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)].$$

它的误差项为

$$R_S = -\frac{h^5}{90} f^{(4)}(\xi) = -\frac{(b-a)^5}{2880} f^{(4)}(\xi), \quad \xi \in (a, b).$$

注 Simpson 公式用二次多项式逼近函数，但它对三次多项式实际上也是精确的，因此具有比表面看上去更高的代数精度。这是对称性带来的一个重要好处。

3.10.3 牛顿-柯特斯公式 (Newton-Cotes Formula)

3.10.3.1 一般形式

若把积分区间 $[a, b]$ 等分为 n 份，步长为

$$h = \frac{b-a}{n},$$

节点取为

$$x_k = a + kh, \quad k = 0, 1, \dots, n,$$

则由一般插值求积公式可得

$$\int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k),$$

其中

$$A_k = \int_a^b l_k(x) dx.$$

定义

$$c_k = \frac{1}{b-a} A_k,$$

则公式可写成

$$\int_a^b f(x) dx \approx (b-a) \sum_{k=0}^n c_k f(x_k) = (b-a) \sum_{k=0}^n c_k f(a+kh).$$

这里的 c_k 称为**柯特斯系数 (C 系数 (Cotes coefficients))**，对应的求积公式称为**闭型牛顿-柯特斯公式 (closed Newton-Cotes formula)**。

3.10.3.2 柯特斯系数的积分表示

对 c_k 作变量代换

$$x = a + th, \quad dx = h dt, \quad t \in [0, n],$$

并利用

$$x_i = a + ih,$$

可得

$$c_k = \frac{1}{b-a} \int_a^b \prod_{\substack{i=0 \\ i \neq k}}^n \frac{x - x_i}{x_k - x_i} dx = \frac{1}{n} \int_0^n \prod_{\substack{i=0 \\ i \neq k}}^n \frac{t - i}{k - i} dt.$$

再利用

$$\prod_{\substack{i=0 \\ i \neq k}}^n (k - i) = (-1)^{n-k} k! (n - k)!,$$

得到

$$c_k = \frac{(-1)^{n-k}}{n(n-k)!k!} \int_0^n \prod_{\substack{i=0 \\ i \neq k}}^n (t - i) dt.$$

3.10.3.3 柯特斯系数的基本性质

柯特斯系数满足几个重要性质：

1. 系数和为 1

$$\sum_{k=0}^n c_k = 1.$$

这是因为公式对常数函数 $f(x) \equiv 1$ 必须精确成立。

2. 对称性

$$c_k = c_{n-k}.$$

这来自等距节点关于区间中点的对称性。

3. 高阶时可能出现负系数。例如 $n = 8$ 时，部分柯特斯系数已经为负，这说明高阶牛顿-柯特斯公式会出现明显的不稳定性。

 **笔记** 正是由于高阶插值的振荡与高阶牛顿-柯特斯公式的不稳定性, 实际计算中通常更偏向于使用低阶公式并做复化, 而不是盲目提高单区间公式的阶数。

3.10.3.4 前几阶柯特斯系数表

常见低阶柯特斯系数如下:

n	c_0	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8
1	$\frac{1}{2}$	$\frac{1}{2}$							
2	$\frac{1}{6}$	$\frac{4}{6}$	$\frac{1}{6}$						
3	$\frac{1}{8}$	$\frac{3}{8}$	$\frac{3}{8}$	$\frac{1}{8}$					
4	$\frac{7}{90}$	$\frac{16}{45}$	$\frac{2}{15}$	$\frac{16}{45}$	$\frac{7}{90}$				
5	$\frac{19}{288}$	$\frac{25}{96}$	$\frac{25}{144}$	$\frac{25}{144}$	$\frac{25}{96}$	$\frac{19}{288}$			
6	$\frac{41}{840}$	$\frac{9}{35}$	$\frac{9}{280}$	$\frac{34}{105}$	$\frac{9}{280}$	$\frac{9}{35}$	$\frac{41}{840}$		
7	$\frac{751}{17280}$	$\frac{3577}{17280}$	$\frac{1323}{17280}$	$\frac{2989}{17280}$	$\frac{2989}{17280}$	$\frac{1323}{17280}$	$\frac{3577}{17280}$	$\frac{751}{17280}$	
8	$\frac{989}{28350}$	$\frac{5888}{28350}$	$-\frac{928}{28350}$	$\frac{10496}{28350}$	$-\frac{4540}{28350}$	$\frac{10496}{28350}$	$-\frac{928}{28350}$	$\frac{5888}{28350}$	$\frac{989}{28350}$

3.10.4 常用低阶牛顿-柯特斯公式

由于高阶公式常常不稳定, 最常用的是几种低阶公式。

3.10.4.1 梯形公式 (Trapezoidal Rule)

当 $n = 1$ 时,

$$c_0 = c_1 = \frac{1}{2},$$

于是

$$\int_a^b f(x) dx \approx \frac{b-a}{2} [f(a) + f(b)].$$

若记

$$h = b - a,$$

则误差为

$$R_T = -\frac{h^3}{12} f''(\xi), \quad \xi \in (a, b).$$

3.10.4.2 Simpson 公式

当 $n = 2$ 时,

$$c_0 = \frac{1}{6}, \quad c_1 = \frac{4}{6}, \quad c_2 = \frac{1}{6},$$

从而得到

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right].$$

若 $h = \frac{b-a}{2}$, 其误差为

$$R_S = -\frac{h^5}{90} f^{(4)}(\xi).$$

3.10.4.3 五点柯特斯公式

当 $n = 4$ 时,

$$c_0 = c_4 = \frac{7}{90}, \quad c_1 = c_3 = \frac{16}{45}, \quad c_2 = \frac{2}{15}.$$

于是

$$C = (b-a) \left[\frac{7}{90} f(x_0) + \frac{16}{45} f(x_1) + \frac{2}{15} f(x_2) + \frac{16}{45} f(x_3) + \frac{7}{90} f(x_4) \right].$$

若

$$h = \frac{b-a}{4},$$

则其误差为

$$R_C = -\frac{8h^7}{945} f^{(6)}(\xi).$$

定义 3.7

若一个求积公式对所有次数不超过 m 的多项式都严格精确, 则称它具有 m 次代数精度 (degree of precision)。

五点柯特斯公式具有 5 次代数精度, 即它对所有不超过 5 次的多项式积分都是精确的。

注 低阶公式之所以常用, 并不是因为它们“简单但粗糙”, 而是因为它们在稳定性与精度之间给出了更好的平衡。实际中通常把低阶公式用于很多小区间, 而不是在大区间上直接使用高阶公式。

3.10.5 为什么积分通常比微分更精确

数值积分往往比数值微分更稳定、更高精度, 其根本原因在于二者对误差的作用方式不同。

1. **微分会放大局部误差。** 因为导数本质上是差商, 任何微小的函数值扰动都会被除以 h 放大。
2. **积分会平均和抵消误差。** 插值误差在区间内通常呈振荡变化, 积分时正负部分往往会相互抵消。

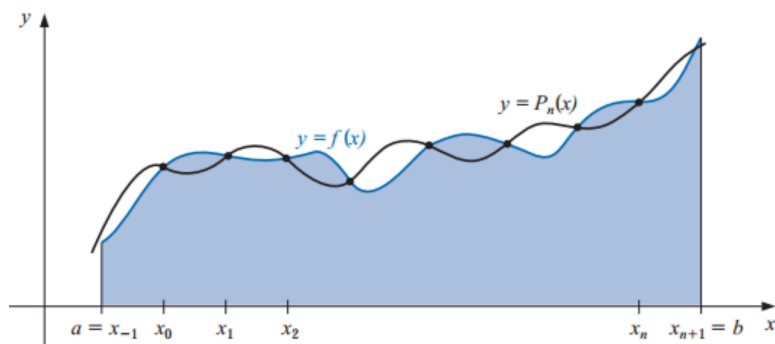


图 3.6: 积分误差的抵消现象

从几何图像上看, 插值多项式 $P_n(x)$ 与原函数 $f(x)$ 的差值有时在曲线上方, 有时在下方, 求面积时这些误差部分会发生明显抵消; 而求导时则恰恰会对局部偏差更加敏感。

结论 这也是为什么在相同步长 h 下, 数值积分公式常常可以达到比数值微分公式更高的实际精度。

3.10.6 复合积分公式 (Composite Quadrature Formula)

若区间 $[a, b]$ 较大, 直接在整个区间上使用一次低阶公式, 误差可能仍然过大。一个自然的改进方法是:

1. 把 $[a, b]$ 等分成许多小子区间;
2. 在每个子区间上应用低阶求积公式;
3. 再利用积分的可加性, 把各子区间结果相加。

这就得到复合求积公式 (composite quadrature formula)。

3.10.6.1 复合 Simpson 公式

设

$$x_j = a + jh, \quad j = 0, 1, \dots, n,$$

其中 n 为偶数, 且

$$h = \frac{b-a}{n}.$$

把区间分成 $\frac{n}{2}$ 个长度为 $2h$ 的小区间:

$$[x_0, x_2], [x_2, x_4], \dots, [x_{n-2}, x_n].$$

在每个小区间上应用 Simpson 公式, 有

$$\int_{x_{2j-2}}^{x_{2j}} f(x) dx = \frac{h}{3} [f(x_{2j-2}) + 4f(x_{2j-1}) + f(x_{2j})] - \frac{h^5}{90} f^{(4)}(\xi_j),$$

其中 $\xi_j \in (x_{2j-2}, x_{2j})$ 。

对 $j = 1, 2, \dots, \frac{n}{2}$ 求和, 得

$$\int_a^b f(x) dx = \sum_{j=1}^{n/2} \int_{x_{2j-2}}^{x_{2j}} f(x) dx.$$

代入上式并合并重复项, 得到

$$\int_a^b f(x) dx = \frac{h}{3} \left[f(a) + 2 \sum_{j=1}^{n/2-1} f(x_{2j}) + 4 \sum_{j=1}^{n/2} f(x_{2j-1}) + f(b) \right] - \sum_{j=1}^{n/2} \frac{h^5}{90} f^{(4)}(\xi_j).$$

由积分中值定理, 可将误差和写成

$$- \sum_{j=1}^{n/2} \frac{h^5}{90} f^{(4)}(\xi_j) = -\frac{b-a}{180} h^4 f^{(4)}(\mu), \quad \mu \in (a, b).$$

因此复合 Simpson 公式为

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(a) + 2 \sum_{j=1}^{n/2-1} f(x_{2j}) + 4 \sum_{j=1}^{n/2} f(x_{2j-1}) + f(b) \right],$$

误差为

$$R_{CS} = -\frac{b-a}{180} h^4 f^{(4)}(\mu).$$

注 复合 Simpson 公式的总误差是 $O(h^4)$ 。虽然每个小区间上的局部误差是 $O(h^5)$, 但共有大约 $O(1/h)$ 个小区间, 因此整体误差阶降为 $O(h^4)$ 。

3.11 余项外推 (Richardson Extrapolation) 与 Romberg 积分

在许多数值计算问题中, 我们往往已经拥有一个基本近似公式

$$N_1(h),$$

其中 h 表示网格步长、采样间距或离散尺度。通常当 $h \rightarrow 0$ 时, $N_1(h)$ 会逼近目标量 M , 但同时伴随一个可以按 h 的幂次展开的截断误差。余项外推 (extrapolation) 的基本思想就是: 利用同一公式在不同步长下的结果, 消去误差展开中的主导项, 从而构造出更高精度的近似。

这一思想最常见的两个应用是:

1. 数值微分中的 Richardson 外推;

2. 数值积分中的 Romberg 积分。

3.11.1 Richardson 外推的基本思想

设 M 是我们真正要求的量, 而 $N_1(h)$ 是一个用步长 h 得到的数值近似。若其误差展开为

$$M = N_1(h) + K_1 h + K_2 h^2 + K_3 h^3 + \cdots,$$

则当步长减半时,

$$M = N_1\left(\frac{h}{2}\right) + K_1 \frac{h}{2} + K_2 \frac{h^2}{4} + K_3 \frac{h^3}{8} + \cdots.$$

将第二式减去第一式, 可得

$$N_1\left(\frac{h}{2}\right) - N_1(h) = \frac{K_1 h}{2} - \frac{3K_2 h^2}{4} - \frac{7K_3 h^3}{8} - \cdots.$$

于是

$$N_2(h) = N_1\left(\frac{h}{2}\right) + \left[N_1\left(\frac{h}{2}\right) - N_1(h)\right]$$

满足

$$M = N_2(h) - \frac{K_2}{2} h^2 - \frac{3K_3}{4} h^3 - \cdots.$$

因此, $N_2(h)$ 已经消去了原来的一阶误差项 $K_1 h$, 从一阶精度提高到了至少二阶精度。

注 这一情形适用于那些主导误差为 $O(h)$ 的方法, 例如前向差分、后向差分等。此时步长加密一次并做简单线性组合, 就能把一阶误差项消去。

3.11.2 只含偶数阶误差时的 Richardson 外推

在许多重要问题中, 误差展开只含偶数次幂, 例如中心差分公式、复合梯形公式等。此时若

$$M = N_1(h) + K_1 h^2 + K_2 h^4 + K_3 h^6 + \cdots,$$

则

$$M = N_1\left(\frac{h}{2}\right) + K_1 \frac{h^2}{4} + K_2 \frac{h^4}{16} + K_3 \frac{h^6}{64} + \cdots.$$

将第二式乘以 4 后减去第一式, 可得

$$3M = \left[4N_1\left(\frac{h}{2}\right) - N_1(h)\right] + K_2 \left(\frac{h^4}{4} - h^4\right) + K_3 \left(\frac{h^6}{16} - h^6\right) + \cdots.$$

因此定义

$$N_2(h) = \frac{1}{3} \left[4N_1\left(\frac{h}{2}\right) - N_1(h)\right] = N_1\left(\frac{h}{2}\right) + \frac{1}{3} \left[N_1\left(\frac{h}{2}\right) - N_1(h)\right],$$

便有

$$M = N_2(h) - \frac{K_2}{4} h^4 - \frac{5K_3}{16} h^6 - \cdots.$$

也就是说, $N_2(h)$ 把原来的 $O(h^2)$ 误差提升为 $O(h^4)$ 。

继续对 $N_2(h)$ 使用同样的思想。由

$$M = N_2(h) + \tilde{K}_2 h^4 + \tilde{K}_3 h^6 + \cdots$$

与

$$M = N_2\left(\frac{h}{2}\right) + \tilde{K}_2 \frac{h^4}{16} + \tilde{K}_3 \frac{h^6}{64} + \cdots$$

可得

$$15M = \left[16N_2\left(\frac{h}{2}\right) - N_2(h)\right] + \frac{15}{64} \tilde{K}_3 h^6 + \cdots.$$

因此再定义

$$N_3(h) = \frac{1}{15} \left[16N_2\left(\frac{h}{2}\right) - N_2(h) \right],$$

则 $N_3(h)$ 的误差已提高到 $O(h^6)$ 。

结论 当误差展开只含偶数次幂时, Richardson 外推的前两级递推分别是

$$\begin{aligned} N_2(h) &= \frac{4N_1(h/2) - N_1(h)}{3}, \\ N_3(h) &= \frac{16N_2(h/2) - N_2(h)}{15}. \end{aligned}$$

每做一次外推, 误差阶通常提高两阶。

3.11.3 统一形式

从上面的推导可以看出, 余项外推本质上是在消去误差展开中的首项。更一般地, 若

$$M = N(h) + Ch^p + \text{更高阶项},$$

则有

$$M = N\left(\frac{h}{2}\right) + C\frac{h^p}{2^p} + \text{更高阶项}.$$

因此可构造

$$N^{[1]}(h) = \frac{2^p N(h/2) - N(h)}{2^p - 1},$$

从而消去 Ch^p 这一主导误差项。

 **笔记** 这一定理说明: Richardson 外推并不依赖某个特定公式, 它依赖的是你对误差主导阶 p 的了解。只要知道误差主项的阶数, 就可以系统地消去它。

3.11.4 Richardson 外推在数值微分中的应用

考虑函数

$$f(x) = xe^x.$$

其中心差分一阶导数公式为

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} - \frac{h^2}{6}f^{(3)}(x_0) - \frac{h^4}{120}f^{(5)}(x_0) - \dots.$$

因此若记

$$N_1(h) = \frac{f(x_0 + h) - f(x_0 - h)}{2h},$$

则有

$$f'(x_0) = N_1(h) + K_1h^2 + K_2h^4 + \dots,$$

这正属于“只有偶数阶误差项”的情形。

取 $x_0 = 2$, 则

$$N_1(0.2) = \frac{1}{0.4} [f(2.2) - f(1.8)] = 22.414160,$$

$$N_1(0.1) = \frac{1}{0.2} [f(2.1) - f(1.9)] = 22.228786.$$

应用 Richardson 外推, 得

$$N_2(0.2) = N_1(0.1) + \frac{1}{3} [N_1(0.1) - N_1(0.2)] = 22.166995.$$

若继续取

$$N_1(0.05) = 22.182564,$$

则

$$N_2(0.1) = N_1(0.05) + \frac{1}{3}[N_1(0.05) - N_1(0.1)] = 22.167156666 \dots$$

进一步再做一级外推:

$$N_3(0.2) = \frac{16N_2(0.1) - N_2(0.2)}{15} = 22.167167444 \dots$$

另一方面,

$$f'(x) = e^x(x+1),$$

故

$$f'(2) = 3e^2.$$

可见 $N_2(0.2)$ 已经明显优于原始的 $N_1(0.2)$, 而 $N_3(0.2)$ 又比 $N_2(0.2)$ 更接近真值。

注 这一例子非常典型地说明了 Richardson 外推的优势: 与其单纯不断减小步长, 不如把不同步长下的结果组合起来, 直接消去主导误差项。这样往往能以很少的额外代价换取显著的精度提升。

3.11.5 余项外推在数值积分中的应用

在数值积分中, 最经典的外推对象是复合梯形公式。设

$$T(h) = \frac{h}{2} \left[f(a) + 2 \sum_{j=1}^{n-1} f(x_j) + f(b) \right], \quad h = \frac{b-a}{n}.$$

对足够光滑的函数, 复合梯形公式具有误差展开

$$\int_a^b f(x) dx = T(h) + K_1 h^2 + K_2 h^4 + K_3 h^6 + \dots$$

这是一个只含偶数阶项的展开, 因此非常适合做 Richardson 外推。

对 $T(h)$ 施行一级外推, 得

$$R_{k,2} = R_{k,1} + \frac{1}{3}(R_{k,1} - R_{k-1,1}),$$

其中 $R_{k,1}$ 表示第 k 层网格上的梯形公式值, 通常取

$$R_{k,1} = T(h_k), \quad h_k = \frac{b-a}{2^{k-1}}.$$

继续对 $R_{k,2}$ 外推, 得

$$R_{k,3} = R_{k,2} + \frac{1}{15}(R_{k,2} - R_{k-1,2}).$$

更一般地,

$$R_{k,j} = R_{k,j-1} + \frac{1}{4^{j-1}-1}(R_{k,j-1} - R_{k-1,j-1}), \quad k = j, j+1, \dots$$

定义 3.8

以上由复合梯形公式开始、不断做 Richardson 外推所得到的积分算法, 称为 **Romberg 积分 (Romberg integration)**。

这里:

- k 表示网格加密层数;
- j 表示外推层次, 也可理解为消去误差项后的“精度等级”。

3.11.6 Romberg 表的结构

Romberg 积分通常整理成一个递推表。第一列是不同网格尺度下的复合梯形公式:

$$R_{1,1}, R_{2,1}, R_{3,1}, \dots$$

第二列是对第一列做一次 Richardson 外推得到的结果：

$$R_{2,2}, R_{3,2}, R_{4,2}, \dots$$

第三列再对第二列做一次外推：

$$R_{3,3}, R_{4,3}, R_{5,3}, \dots$$

依此类推，得到一个下三角形表格。一般来说，越往右的列精度越高；而在固定列内，越往下则表示网格越细。一个典型的 Romberg 表形如

$$\begin{array}{cccc} R_{1,1} & & & \\ R_{2,1} & R_{2,2} & & \\ R_{3,1} & R_{3,2} & R_{3,3} & \\ R_{4,1} & R_{4,2} & R_{4,3} & R_{4,4} \\ \vdots & \vdots & \vdots & \vdots \end{array}$$

注 Romberg 方法的高效之处在于：它并不是每次都重新设计新的积分公式，而是充分利用已有的梯形公式结果及其误差展开结构，逐层提高精度。因此它既保留了复合梯形公式实现简单的优点，又通过外推显著提高了精度。

3.11.7 为什么 Romberg 积分有效

Romberg 积分之所以特别成功，有两个原因：

1. 复合梯形公式具有非常规则的偶次幂误差展开

$$T(h) = I + K_1 h^2 + K_2 h^4 + \dots;$$

2. 每次将 h 减半时，误差项缩放规律极其清楚，因此可以精确构造线性组合来消去低阶误差项。

例如：

- $R_{k,2}$ 消去了 h^2 项；
- $R_{k,3}$ 再进一步消去了 h^4 项；
- $R_{k,4}$ 还可消去 h^6 项；

于是每向右推进一列，积分精度便进一步提高。

结论 Romberg 积分可以理解为：

复合梯形公式 + 系统的 Richardson 外推

的结合体。它把一个低阶、稳定、易实现的基本公式，逐步提升为高精度积分算法。

3.12 变步长积分与龙贝格算法 (Adaptive Integration and Romberg Acceleration)

3.12.1 为什么要做变步长积分

复化求积公式虽然能提高精度，但如果步长固定，就会面临两难：

1. 步长太大，精度不足；
2. 步长太小，计算量过大。

因此实际中常采用**变步长 (adaptive step size)**：逐次把步长二分，并重复利用旧结果，直到达到给定精度要求。

3.12.2 以复化梯形公式为例

把区间 $[a, b]$ 等分为 n 个子区间, 步长

$$h = \frac{b-a}{n}, \quad x_k = a + kh.$$

复化梯形公式为

$$T_n = \sum_{k=0}^{n-1} \frac{h}{2} [f(x_k) + f(x_{k+1})].$$

若把每个子区间再二分, 新增中点

$$x_{k+\frac{1}{2}} = a + \left(k + \frac{1}{2}\right)h,$$

则

$$T_{2n} = \frac{1}{2}T_n + \frac{h}{2} \sum_{k=0}^{n-1} f\left(x_{k+\frac{1}{2}}\right).$$

这个递推公式非常重要, 因为它允许我们在已有 T_n 的基础上高效得到 T_{2n} , 而不必完全重算。

3.12.3 误差估计与判敛条件

复化梯形公式的截断误差满足

$$R_n = I - T_n = \frac{b-a}{12} \left(\frac{b-a}{n}\right)^2 f''(\xi_n).$$

二分步长后有

$$R_{2n} = I - T_{2n} = \frac{b-a}{12} \left(\frac{b-a}{2n}\right)^2 f''(\xi_{2n}).$$

若假设二阶导数变化不大, 则近似有

$$\frac{R_{2n}}{R_n} \approx \frac{1}{4}.$$

从而

$$I - T_{2n} \approx \frac{1}{3}(T_{2n} - T_n).$$

因此只要相邻两次结果非常接近, 就能保证当前结果误差很小。判敛条件是

$$|T_{2n} - T_n| < \varepsilon.$$

3.12.4 龙贝格算法: 对复化梯形法做加速

由上式

$$I - T_n = O(h^2), \quad I - T_{2n} = O\left(\frac{h^2}{4}\right),$$

可以用 Richardson 外推得到更高精度近似

$$I \approx T_{2n} + \frac{1}{3}(T_{2n} - T_n).$$

这正是所谓的**龙贝格算法 (Romberg acceleration)**的基本思想。

3.12.5 对复化 Simpson 法的加速

若 S_n 表示复化 Simpson 公式结果, 则给出

$$I - S_n = -\frac{b-a}{2880} h^4 f^{(4)}(\xi_n),$$

$$I - S_{2n} = -\frac{b-a}{2880} \left(\frac{h}{2}\right)^4 f^{(4)}(\xi_{4n}).$$

若四阶导数变化不大, 则

$$\frac{I - S_n}{I - S_{2n}} \approx 16,$$

故

$$I \approx S_{2n} + \frac{1}{15}(S_{2n} - S_n).$$

记这个 5 次代数精度公式为

$$C_n = \frac{16S_{2n} - S_n}{15}.$$

3.12.6 对复化柯特斯法的加速

若 C_n 表示五点柯特斯公式对应的复化结果, 则写出

$$I - C_n = -\frac{2(b-a)}{945} \left(\frac{h}{4}\right)^6 f^{(6)}(\xi_n),$$

$$I - C_{2n} = -\frac{2(b-a)}{945} \left(\frac{h}{8}\right)^6 f^{(6)}(\xi_{6n}).$$

若六阶导数变化不大, 则

$$I \approx C_{2n} + \frac{1}{63}(C_{2n} - C_n).$$

于是得到 7 次代数精度的龙贝格公式

$$R_n = \frac{64C_{2n} - C_n}{63}.$$

结论 同一种基本求积公式, 只要其误差主项结构已知, 就几乎总能用 Richardson 思想做外推加速。Romberg 方法的本质并不是某一个特定公式, 而是对误差级数做系统消元。

3.13 高斯积分 (Gaussian Quadrature)

在前面讨论的 Newton-Cotes 求积公式中, 节点通常被预先固定为等距点; 这样做虽然构造方便, 但节点选择不“最优”, 因此在给定节点数时, 代数精度有限。高斯积分 (Gaussian quadrature) 的核心思想正好相反: 不预先固定节点位置, 而是把节点也当作待优化对象来选择, 从而在相同节点数下获得尽可能高的代数精度。

3.13.1 高斯积分的基本定义

考虑求积公式

$$\int_a^b f(x) dx \approx \sum_{k=0}^n A_k f(x_k),$$

其中 x_k 为节点, A_k 为权重。这里共有 $n+1$ 个节点、 $n+1$ 个权重, 因此一共有 $2n+2$ 个自由参数。

定义 3.9

若通过适当选择节点 x_k 与权重 A_k , 使得上式对所有次数不超过 $2n+1$ 的多项式都精确成立, 则称这些节点 x_k 为高斯点 (Gaussian nodes), 相应的求积公式称为高斯型求积公式 (Gaussian quadrature formula)。

注 由于有 $2n+2$ 个自由参数, 而要求公式对 $1, x, x^2, \dots, x^{2n+1}$ 共 $2n+2$ 个基多项式全部精确, 因此从参数个数上看, 达到 $2n+1$ 次代数精度是可能的; 而这也正是插值型求积公式在 $n+1$ 个节点下所能达到的最高代数精度。

如果用 N 表示节点个数, 则 $N = n+1$, 于是高斯积分的精度可写成:

结论 N 点高斯求积公式具有 $2N-1$ 次代数精度。

3.13.2 高斯点的充要条件

设

$$\omega_{n+1}(x) = \prod_{k=0}^n (x - x_k)$$

是以这些节点为零点的多项式。

定理 3.2

节点 $x_0, x_1, \dots, x_n$ 是高斯点的充要条件是：多项式 $\omega_{n+1}(x)$ 与任意次数不超过 n 的多项式 $P_n(x)$ 在区间 $[a, b]$ 上正交，即

$$\int_a^b P_n(x) \omega_{n+1}(x) dx = 0, \quad \forall P_n \in \mathbb{P}_n.$$

证明 [思路] 若节点 x_k 构成 $n+1$ 点插值求积公式，则对任意次数不超过 $2n+1$ 的多项式 $Q(x)$ ，应有公式精确成立。任取 $Q(x) = P_n(x)\omega_{n+1}(x)$ ，其次数不超过 $2n+1$ 。由于 $\omega_{n+1}(x_k) = 0$ ，所以

$$\sum_{k=0}^n A_k Q(x_k) = 0.$$

若求积公式对 Q 精确，则

$$\int_a^b P_n(x) \omega_{n+1}(x) dx = 0.$$

反过来，若 ω_{n+1} 与所有 $\mathbb{P}_n$ 中多项式正交，再利用多项式除法把任意 $2n+1$ 次以下多项式分解为

$$Q(x) = \omega_{n+1}(x)P_n(x) + R_n(x),$$

其中 $\deg R_n \leq n$ ，即可推出求积公式对 Q 精确，从而得到高斯精度。

注 这个定理说明：高斯点的本质不是“某种巧妙选点技巧”，而是**正交多项式的零点**。因此高斯求积与正交多项式理论天然联系在一起。

3.13.3 Gauss–Legendre 积分公式

在最常见的情形中，我们考虑标准区间

$$[-1, 1]$$

上的积分

$$\int_{-1}^1 f(x) dx.$$

在该区间上，权函数为

$$\rho(x) = 1,$$

相应的正交多项式是**勒让德多项式 (Legendre polynomials)**。

3.13.3.1 勒让德多项式的定义与性质

勒让德多项式 $L_n(x)$ (也常记作 $P_n(x)$) 可以由 Rodrigues 公式给出：

$$L_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n, \quad n = 0, 1, 2, \dots$$

它们满足正交关系

$$\int_{-1}^1 L_i(x) L_j(x) dx = \frac{2}{2i+1} \delta_{ij},$$

其中 δ_{ij} 为 Kronecker 符号。

它们还满足三项递推关系

$$(j+1)L_{j+1}(x) + jL_{j-1}(x) - (2j+1)xL_j(x) = 0,$$

也可写成

$$L_{j+1}(x) = \frac{2j+1}{j+1}xL_j(x) - \frac{j}{j+1}L_{j-1}(x).$$

归一化条件通常取为

$$L_n(1) = 1.$$

前几个勒让德多项式为

$$L_0(x) = 1,$$

$$L_1(x) = x,$$

$$L_2(x) = \frac{1}{2}(3x^2 - 1),$$

$$L_3(x) = \frac{1}{2}(5x^3 - 3x),$$

$$L_4(x) = \frac{1}{8}(35x^4 - 30x^2 + 3).$$

注 勒让德多项式不仅在数值积分中重要，它们还出现在球坐标分离变量、球谐函数、势论与量子力学等问题中，因此既有数值分析意义，也有物理背景。

3.13.3.2 为什么勒让德多项式的零点是高斯点

由于勒让德多项式在 $[-1, 1]$ 上构成关于权函数 $\rho(x) = 1$ 的正交系，因此 $L_{n+1}(x)$ 与任意次数不超过 n 的多项式正交。由前面的充要条件立刻可知：

结论 $L_{n+1}(x)$ 的 $n+1$ 个零点就是 $[-1, 1]$ 上 $n+1$ 点高斯求积的高斯点。

因此，若取 $x_0, \dots, x_n$ 为 $L_{n+1}(x)$ 的零点，则可构造高斯-勒让德求积公式

$$\int_{-1}^1 f(x) dx \approx \sum_{k=0}^n A_k f(x_k).$$

3.13.4 N 点 Gauss-Legendre 公式具有 $2N-1$ 次精度

现在用 N 表示节点个数，记 $L_N(x)$ 为 N 阶勒让德多项式，设其零点为

$$x_0, x_1, \dots, x_{N-1}.$$

构造相应的 N 点高斯-勒让德公式。

为了说明它具有 $2N-1$ 次代数精度，任取次数不超过 $2N-1$ 的多项式 $P_{2N-1}(x)$ 。根据多项式除法，可以把它写成

$$P_{2N-1}(x) = L_N(x)Q_{N-1}(x) + R_{N-1}(x),$$

其中

$$\deg Q_{N-1} \leq N-1, \quad \deg R_{N-1} \leq N-1.$$

由于 $L_N(x)$ 与所有次数不超过 $N-1$ 的多项式正交，因此

$$\int_{-1}^1 P_{2N-1}(x) dx = \int_{-1}^1 R_{N-1}(x) dx.$$

另一方面，在高斯点 x_k 上，由于

$$L_N(x_k) = 0,$$

所以

$$P_{2N-1}(x_k) = R_{N-1}(x_k), \quad k = 0, 1, \dots, N-1.$$

而 $R_{N-1}(x)$ 次数不超过 $N-1$, 插值求积公式对它必然精确, 因此

$$\int_{-1}^1 R_{N-1}(x) dx = \sum_{k=0}^{N-1} A_k R_{N-1}(x_k).$$

于是

$$\int_{-1}^1 P_{2N-1}(x) dx = \sum_{k=0}^{N-1} A_k P_{2N-1}(x_k).$$

定理 3.3

N 点 Gauss-Legendre 求积公式对所有次数不超过 $2N-1$ 的多项式都精确成立。



3.13.5 高斯权重的确定

已知节点是勒让德多项式的零点后, 还需要确定权重 A_k 。

一种一般的思路是: 对任意次数不超过 $N-1$ 的多项式 $Q_{N-1}(x)$, 把它在勒让德基底下展开:

$$Q_{N-1}(x) = \sum_{k=0}^{N-1} \alpha_k L_k(x).$$

于是

$$\int_{-1}^1 Q_{N-1}(x) dx = \sum_{k=0}^{N-1} \alpha_k \int_{-1}^1 L_k(x) dx.$$

由于

$$\int_{-1}^1 L_0(x) dx = 2, \quad \int_{-1}^1 L_k(x) dx = 0 \quad (k \geq 1),$$

所以

$$\int_{-1}^1 Q_{N-1}(x) dx = 2\alpha_0.$$

另一方面, 把 $Q_{N-1}(x)$ 在高斯点 x_i 处取值, 可得一个线性方程组, 从而求出 α_0 , 于是就能把积分写成节点值的线性组合

$$\int_{-1}^1 Q_{N-1}(x) dx = \sum_{i=0}^{N-1} \omega_i Q_{N-1}(x_i).$$

因此高斯权重 ω_i 也就确定了。

对于 Gauss-Legendre 公式, 权重有一个标准闭式公式:

$$\omega_i = \frac{2}{(1-x_i^2) [L'_N(x_i)]^2}, \quad i = 0, 1, \dots, N-1,$$

其中 x_i 是 $L_N(x)$ 的零点。

注 这个公式在实际计算中非常重要: 高斯节点由正交多项式的零点确定, 高斯权重由零点与导数值确定。这样一来, 整个求积公式完全由相应正交多项式决定。

3.13.6 二点 Gauss-Legendre 公式的例子

取 $N=2$ 。此时需要用到二阶勒让德多项式

$$L_2(x) = \frac{1}{2}(3x^2 - 1).$$

它的两个零点为

$$x_0 = -\frac{1}{\sqrt{3}}, \quad x_1 = \frac{1}{\sqrt{3}}.$$

用权重公式可得

$$\omega_0 = \omega_1 = 1.$$

因此二点 Gauss-Legendre 公式为

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right).$$

例题 3.2 取

$$f(x) = x^2.$$

则精确值为

$$\int_{-1}^1 x^2 dx = \frac{2}{3}.$$

而二点高斯公式给出

$$f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right) = \frac{1}{3} + \frac{1}{3} = \frac{2}{3},$$

完全精确。

相比之下，若用两端点梯形公式，则

$$\int_{-1}^1 x^2 dx \approx \frac{1 - (-1)}{2} [f(-1) + f(1)] = 1 \cdot \frac{1+1}{2} = 1,$$

误差明显较大。

结论 这说明：在相同“函数值采样次数”下，高斯积分往往远比等距 Newton-Cotes 公式更高效。

3.13.7 积分区间变换

Gauss-Legendre 公式天然适用于区间 $[-1, 1]$ 。若原积分定义在一般区间 $[a, b]$ 上，需要先作线性变换

$$t = \frac{b-a}{2}x + \frac{b+a}{2},$$

则

$$dt = \frac{b-a}{2} dx.$$

于是

$$\int_a^b f(t) dt = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{b-a}{2}x + \frac{b+a}{2}\right) dx.$$

因此，只要先把 $[a, b]$ 映射到 $[-1, 1]$ ，就可以直接使用标准 Gauss-Legendre 节点和权重。

结论 对一般区间 $[a, b]$ ， N 点高斯-勒让德公式写成

$$\int_a^b f(t) dt \approx \frac{b-a}{2} \sum_{i=0}^{N-1} \omega_i f\left(\frac{b-a}{2}x_i + \frac{b+a}{2}\right),$$

其中 x_i, ω_i 是 $[-1, 1]$ 上的标准高斯点和权重。

3.13.8 前几阶 Gauss-Legendre 节点与权重

前几阶公式如下：

二点公式

$$x_{1,2} = \pm \frac{1}{\sqrt{3}}, \quad \omega_{1,2} = 1.$$

三点公式

$$\begin{aligned} x_1 &= 0, & x_{2,3} &= \pm \sqrt{\frac{3}{5}}, \\ \omega_1 &= \frac{8}{9}, & \omega_{2,3} &= \frac{5}{9}. \end{aligned}$$

四点公式

$$\begin{aligned} x_{1,2} &= \pm \frac{1}{35} \sqrt{525 - 70\sqrt{30}}, & x_{3,4} &= \pm \frac{1}{35} \sqrt{525 + 70\sqrt{30}}, \\ \omega_{1,2} &= \frac{1}{36} (18 + \sqrt{30}), & \omega_{3,4} &= \frac{1}{36} (18 - \sqrt{30}). \end{aligned}$$

五点公式

$$\begin{aligned} x_1 &= 0, & x_{2,3} &= \pm \frac{1}{21} \sqrt{245 - 14\sqrt{70}}, & x_{4,5} &= \pm \frac{1}{21} \sqrt{245 + 14\sqrt{70}}, \\ \omega_1 &= \frac{128}{225}, & \omega_{2,3} &= \frac{322 + 13\sqrt{70}}{900}, & \omega_{4,5} &= \frac{322 - 13\sqrt{70}}{900}. \end{aligned}$$

注 这些节点和权重关于 0 对称，权重也成对相等。这是因为勒让德多项式具有奇偶性，而积分区间 $[-1, 1]$ 本身也关于原点对称。

3.13.9 带权高斯积分的一般形式

许多积分并不是

$$\int_a^b f(x) dx,$$

而是

$$\int_a^b \rho(x) f(x) dx,$$

其中 $\rho(x)$ 称为**权函数 (weight function)**。这时对应的高斯型求积公式写成

$$\int_a^b \rho(x) f(x) dx \approx \sum_{k=0}^n A_k f(x_k).$$

定义 3.10

若该求积公式用 $n+1$ 个节点达到 $2n+1$ 次代数精度，则称它为关于权函数 $\rho(x)$ 的带权高斯型求积公式。

这时高斯点满足的重要条件是：多项式

$$\omega_{n+1}(x) = \prod_{k=0}^n (x - x_k)$$

必须与所有次数不超过 n 的多项式关于权函数 $\rho(x)$ 正交, 即

$$\int_a^b \rho(x) \omega_{n+1}(x) P(x) dx = 0, \quad \forall P \in \mathbb{P}_n.$$

因此, 带权高斯积分的节点由与该权函数对应的正交多项式的零点给出。

3.13.10 几类常用的带权高斯积分

3.13.10.1 Gauss–Legendre 公式

对应权函数

$$\rho(x) = 1, \quad x \in [-1, 1].$$

正交多项式是勒让德多项式。

3.13.10.2 Gauss–Laguerre 公式

对应积分

$$\int_0^\infty e^{-x} f(x) dx.$$

权函数为

$$\rho(x) = e^{-x}, \quad x \in [0, \infty).$$

对应的正交多项式是拉盖尔多项式 (Laguerre polynomials)。前几个为

$$L_0(x) = 1,$$

$$L_1(x) = 1 - x,$$

$$L_2(x) = 2 - 4x + x^2,$$

$$L_3(x) = 6 - 18x + 9x^2 - x^3,$$

$$L_4(x) = x^4 - 16x^3 + 72x^2 - 96x + 24.$$

其递推关系为

$$(n+1)L_{n+1}(x) = (2n+1-x)L_n(x) - nL_{n-1}(x).$$

3.13.10.3 Gauss–Hermite 公式

对应积分

$$\int_{-\infty}^\infty e^{-x^2} f(x) dx.$$

权函数为

$$\rho(x) = e^{-x^2}, \quad x \in (-\infty, \infty).$$

对应的正交多项式是厄米多项式 (Hermite polynomials)。前几个为

$$H_0(x) = 1,$$

$$H_1(x) = 2x,$$

$$H_2(x) = 4x^2 - 2,$$

$$H_3(x) = 8x^3 - 12x,$$

$$H_4(x) = 16x^4 - 48x^2 + 12.$$

其递推关系为

$$H_{n+1}(x) = 2xH_n(x) - 2nH_{n-1}(x).$$

正交关系为

$$\int_{-\infty}^{\infty} e^{-x^2} H_n(x)^2 dx = 2^n n! \sqrt{\pi}.$$

3.13.10.4 Gauss–Chebyshev 公式

一种常见形式是

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx.$$

此时权函数为

$$\rho(x) = \frac{1}{\sqrt{1-x^2}}, \quad x \in [-1, 1].$$

对应正交多项式是切比雪夫多项式。对于第一类切比雪夫多项式 $T_n(x)$ ，节点和权重都有显式表达：

$$x_k = \cos \frac{(2k+1)\pi}{2N}, \quad k = 0, 1, \dots, N-1,$$

$$\omega_k = \frac{\pi}{N}.$$

注 Gauss–Chebyshev 公式的一个重要优点是节点与权重都可显式写出，而不像 Gauss–Legendre 那样通常需要数值求根。

3.13.11 无穷区间与区间变换

若积分区间不是 $[-1, 1]$ ，除了简单线性变换外，还可以把无穷区间映射到有限区间。例如

$$\int_0^{\infty} f(t) dt$$

可通过适当变量代换，转化到 $[-1, 1]$ 或 $[0, 1]$ 上再应用相应高斯公式。对于某些特殊权函数，更自然的做法则是直接使用对应的带权高斯积分，例如：

- 若积分本身含 e^{-x} ，优先考虑 Gauss–Laguerre；
- 若积分本身含 e^{-x^2} ，优先考虑 Gauss–Hermite；
- 若积分本身含 $(1-x^2)^{-1/2}$ ，优先考虑 Gauss–Chebyshev。

3.13.12 高斯积分的优势与局限

结论 高斯积分的最大优势在于：在给定节点数下，它能达到插值型求积公式所能达到的最高代数精度。

与 Newton–Cotes 公式相比，高斯积分有以下优点：

1. 节点数相同时，精度通常显著更高；
2. 节点不必等距，因此避免了高次等距插值的不稳定性；
3. 对多项式及光滑函数尤其高效；
4. 能自然推广到带权积分、无穷区间积分和多维积分。

但它也有一些特点需要注意：

1. 节点与权重的构造较 Newton–Cotes 更复杂；
2. 若被积函数不够光滑、存在奇点或剧烈振荡，则需要专门处理；
3. 对一般区间与一般权函数，需要选对对应的正交多项式族。

3.14 WKB 数值计算示例：量子穿透几率与寿命

3.14.1 问题背景

本章最后给出一个综合应用：已知形变变量 q 下的势能 $V(q)$ 与质量参数 $M(q)$ 的离散数据，要求用 WKB 近似估计体系的衰变寿命 τ 。

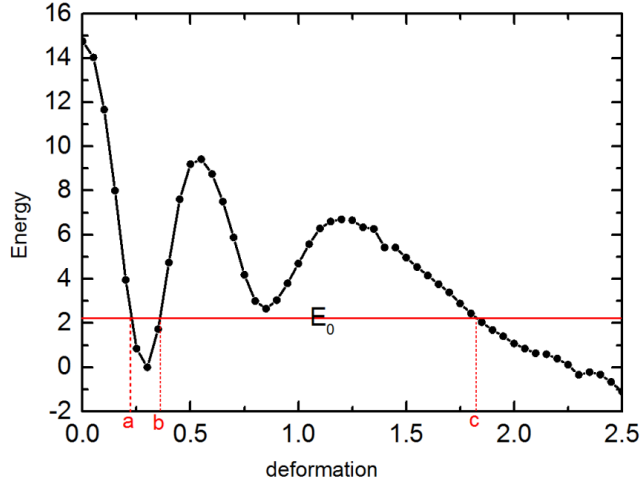


图 3.7: 示意图

设某个能量为 E_0 ，且对应三个转折点（turning points） a, b, c 满足图示关系，则：

- $[a, b]$ 为经典允许区（classically allowed region）；
- $[b, c]$ 为经典禁阻区（classically forbidden region）；
- 穿透几率由禁阻区指数衰减积分决定；
- 振动周期由允许区速度积分决定。

3.14.2 公式

$$W = \exp \left(-2 \int_b^c dq \sqrt{M(q)(V(q) - E_0)} \right),$$

$$T = \hbar \int_a^b dq \left(\sqrt{\frac{E_0 - V(q)}{M(q)}} \right)^{-1},$$

$$\tau = \frac{T}{W}.$$

其中：

1. $V(q)$ 单位为 MeV；
2. 形变 q 为无量纲；
3. $M(q)$ 单位为 MeV^{-1} ；
4. τ 最终单位为秒，故 $\hbar$ 需取 $\text{MeV} \cdot \text{s}$ 单位。

要求分别计算

$$E_0 = 0.9 \text{ MeV}, \quad E_0 = 4.8 \text{ MeV}$$

时的寿命大小。

3.14.3 数值实现思路

这是一个非常典型的“插值 + 求根 + 数值积分”综合问题。完整的计算流程可写成：

1. 由离散数据构造 $V(q)$ 与 $M(q)$ 的连续逼近，常用三次样条或 Akima 插值；
2. 对方程

$$V(q) - E_0 = 0$$

做数值求根，找到转折点 a, b, c ；

3. 在 $[a, b]$ 与 $[b, c]$ 上分别做数值积分，可用复化 Simpson、Gauss 积分或 Romberg；
4. 代入 W, T, τ 三式得到寿命。

 **笔记** 当 $E_0 = 4.8 \text{ MeV}$ 时，势能曲线可能出现多个交点，因此必须先结合图像和物理背景，判断究竟选取哪一个局域势阱及其对应的三个转折点。数值上“找到所有根”并不够，还必须选对有物理意义的那组三个根。

3.14.4 附表： $V(q)$ 与 $M(q)$ 的离散数据

3.14.4.1 势能表 $V(q)$

q	$V(q)$	q	$V(q)$
0.00	14.757	1.30	6.337
0.05	14.022	1.35	6.263
0.10	11.665	1.40	5.425
0.15	7.996	1.45	5.421
0.20	3.953	1.50	4.961
0.25	0.848	1.55	4.541
0.30	0.000	1.60	4.153
0.35	1.727	1.65	3.760
0.40	4.741	1.70	3.393
0.45	7.607	1.75	2.885
0.50	9.199	1.80	2.441
0.55	9.421	1.85	2.042
0.60	8.751	1.90	1.690
0.65	7.503	1.95	1.405
0.70	5.879	2.00	1.079
0.75	4.185	2.05	0.844
0.80	3.000	2.10	0.635
0.85	2.655	2.15	0.590
0.90	3.036	2.20	0.393
0.95	3.799	2.25	0.112
1.00	4.693	2.30	-0.339
1.05	5.570	2.35	-0.218
1.10	6.281	2.40	-0.329
1.15	6.602	2.45	-0.667
1.20	6.697	2.50	-1.097
1.25	6.654		

3.14.4.2 质量参数表 $M(q)$

q	$M(q)$	q	$M(q)$
0.00	271.7416	1.30	157.1452
0.05	272.5246	1.35	201.3586
0.10	289.6114	1.40	214.6870
0.15	334.4802	1.45	218.3236
0.20	424.6238	1.50	230.8458
0.25	510.0520	1.55	228.4040
0.30	493.5916	1.60	227.9806
0.35	462.7472	1.65	222.4010
0.40	364.1124	1.70	220.5856
0.45	246.5000	1.75	221.2410
0.50	275.4710	1.80	203.5452
0.55	327.9146	1.85	239.7198
0.60	317.6718	1.90	261.3016
0.65	315.2416	1.95	279.6180
0.70	332.7344	2.00	184.3646
0.75	323.2862	2.05	183.4018
0.80	328.7846	2.10	175.2644
0.85	308.0322	2.15	173.0314
0.90	270.7788	2.20	170.9782
0.95	272.8552	2.25	163.2932
1.00	210.8880	2.30	160.1090
1.05	186.1278	2.35	159.8074
1.10	193.7258	2.40	157.5744
1.15	201.3760	2.45	305.4860
1.20	186.2902	2.50	296.9339
1.25	189.1960		

3.15 本章总结 (Summary)

- Lagrange 插值法是本章的起点，它不仅给出函数重建公式，还能自然导出微分公式与积分公式；
- Hermite 插值把导数信息也纳入拟合条件，提高了信息利用率；
- 高次全局多项式会带来龙格现象，因此分段插值更实用；
- 三次样条插值兼顾平滑性与稳定性，是最重要的分段插值方法之一；
- Akima 插值具有良好的局部性，能有效避免振荡与误差传播；
- 中心差分、五点公式等数值微分方法本质上都可由插值推导出来；
- Newton–Cotes 公式、复化求积与变步长算法构成了经典数值积分的主线；
- Richardson 外推与 Romberg 算法通过消去误差主项来显著提高精度；
- 高斯积分通过最优选点，把 N 个节点的求积精度提升到 $2N - 1$ 次；
- 带权高斯积分、高斯–Laguerre、高斯–Hermite 等方法把这一思想推广到更一般的权函数情形；
- WKB 例子说明：插值、求根、积分三类技术往往在真实物理问题中同时出现。

第四章 优化与非线性方程

内容提要

- 非线性方程求根的物理背景、误差判据与收敛思想
- 二分法、不动点迭代、Newton 法、割线法与多重根修正
- Aitken 加速、Steffensen 方法、Horner 算法与 Brent 方法
- 非线性方程组的不动点迭代、Newton 法、准 Newton 法与 Broyden 方法
- 优化问题的 Hessian 判据、梯度法、拟 Newton 法、BFGS 与 Broyden 优化
- 线性规划、单纯形法、Nelder-Mead 方法、模拟退火与遗传算法

4.1 非线性方程求根

4.1.1 从物理问题到求根问题

在计算物理中，很多方程无法直接解析求解，因而必须转化为数值问题。一个典型例子是一维束缚态问题。考虑一维薛定谔方程

$$-\frac{\hbar^2}{2m} \frac{d^2 u}{dx^2} + V(x)u(x) = Eu(x),$$

其中方势阱取为

$$V(x) = \begin{cases} -V_0, & 0 \leq x < a, \\ 0, & x > a. \end{cases}$$

对于束缚态，能量满足 $E < 0$ 。将方程分区域写开，可得：

势阱内 $0 \leq x < a$

$$\frac{d^2 u}{dx^2} + \frac{2m}{\hbar^2} (V_0 + E)u(x) = 0 = \frac{d^2 u}{dx^2} + \frac{2m}{\hbar^2} (V_0 - |E|)u(x) = 0.$$

因此可写成振荡解

$$u_{\text{in}}(x) = A \sin(kx), \quad k = \frac{\sqrt{2m(V_0 - |E|)}}{\hbar}.$$

势阱外 $x > a$

$$\frac{d^2 u}{dx^2} + \frac{2m}{\hbar^2} Eu(x) = 0 = \frac{d^2 u}{dx^2} - \frac{2m}{\hbar^2} |E|u(x) = 0,$$

因此取满足无穷远衰减的解

$$u_{\text{out}}(x) = B \exp(-\kappa x), \quad \kappa = \frac{\sqrt{2m|E|}}{\hbar}.$$

在边界 $x = a$ 处要求波函数及其导数连续，即

$$u_{\text{in}}(a) = u_{\text{out}}(a), \quad u'_{\text{in}}(a) = u'_{\text{out}}(a).$$

代入得

$$A \sin(ka) = B e^{-\kappa a}, \quad Ak \cos(ka) = -\kappa B e^{-\kappa a}.$$

两式相除得到特征方程

$$k \cot(ka) = -\kappa,$$

即

$$\sqrt{2m(V_0 - |E|)} \cot\left(\frac{\sqrt{2ma^2(V_0 - |E|)}}{\hbar}\right) = -\sqrt{2m|E|}.$$

于是问题被转化为一个一维非线性方程求根问题:

$$f(E) = \sqrt{2m(V_0 - |E|)} \cot\left(\frac{\sqrt{2ma^2(V_0 - |E|)}}{\hbar}\right) + \sqrt{2m|E|} = 0.$$

例如取

$$V_0 = 20 \text{ MeV}, \quad a = 2 \text{ fm}, \quad m = 938,$$

则数值求解可得

$$|E| \approx 2.2 \text{ MeV},$$

这正对应氘核束缚能量的量级。

4.1.2 二分法 (Bisection Method)

设 f 在区间 $[a, b]$ 上连续, 且

$$f(a)f(b) < 0.$$

由介值定理, 至少存在一点 $p \in (a, b)$ 使得

$$f(p) = 0.$$

二分法的思想很简单: 不断取中点, 并用符号变化来缩小根所在区间。

令

$$a_1 = a, \quad b_1 = b, \quad p_1 = \frac{a_1 + b_1}{2}.$$

若 $f(p_1)$ 与 $f(a_1)$ 同号, 则根必在 (p_1, b_1) 中, 于是令

$$a_2 = p_1, \quad b_2 = b_1.$$

若 $f(p_1)$ 与 $f(a_1)$ 异号, 则根必在 (a_1, p_1) 中, 于是令

$$a_2 = a_1, \quad b_2 = p_1.$$

继续迭代,

$$p_n = \frac{a_n + b_n}{2},$$

每一步都把区间长度减半。

误差估计 由于

$$b_n - a_n = \frac{b - a}{2^{n-1}},$$

而根始终位于 $[a_n, b_n]$ 中, 所以

$$|p_n - p| \leq \frac{b_n - a_n}{2} = \frac{b - a}{2^n}.$$

这说明二分法收敛是线性的, 但具有严格的全局收敛性。

方法特点

- 优点: 稳定、可靠、有全局收敛性。
- 缺点: 收敛速度较慢。
- 对振荡函数、局部不光滑函数, 二分法反而常常比高阶方法更稳健。

4.1.3 不动点法 (Fixed-Point Iteration)

求解

$$f(x) = 0$$

的另一种思路,是把它改写为

$$x = g(x).$$

若某点 p 满足

$$g(p) = p,$$

则称 p 为函数 g 的一个**不动点 (fixed point)**。于是求根问题可转化为求不动点问题。

最自然的迭代格式为

$$p_n = g(p_{n-1}), \quad n \geq 1.$$

若极限存在, 则

$$p = \lim_{n \rightarrow \infty} p_n = \lim_{n \rightarrow \infty} g(p_{n-1}) = g\left(\lim_{n \rightarrow \infty} p_{n-1}\right) = g(p),$$

故极限点必为不动点。

收敛定理 若对区间 $[a, b]$ 上任意 x 都有

$$g(x) \in [a, b],$$

并且 $g'(x)$ 在 (a, b) 上存在, 且存在常数 $0 < k < 1$ 使

$$|g'(x)| \leq k, \quad \forall x \in (a, b),$$

则对任意初值 $p_0 \in [a, b]$, 迭代序列

$$p_n = g(p_{n-1})$$

都收敛到区间内唯一的不动点。

这个定理的本质是: g 必须是压缩映射 (contraction mapping)。

4.1.4 不动点法的具体例子

考虑方程

$$x^3 + 4x^2 - 10 = 0, \quad x \in [1, 2].$$

它可以改写成许多不同的不动点形式, 例如:

$$(a) \quad x = g_1(x) = x - x^3 - 4x^2 + 10,$$

$$(b) \quad x = g_2(x) = \left(\frac{10}{x} - 4x\right)^{1/2},$$

$$(c) \quad x = g_3(x) = \frac{1}{2}(10 - x^3)^{1/2},$$

$$(d) \quad x = g_4(x) = \left(\frac{10}{4+x}\right)^{1/2},$$

$$(e) \quad x = g_5(x) = x - \frac{x^3 + 4x^2 - 10}{3x^2 + 8x}.$$

其中 (e) 实际上已经是牛顿法。

对于 (d), 有

$$g_4'(x) = -\frac{5}{\sqrt{10}(4+x)^{3/2}},$$

因此

$$|g'_4(x)| \leq \frac{5}{\sqrt{10}(5)^{3/2}} < 0.15 < 1,$$

故它在区间 $[1, 2]$ 上满足压缩映射条件，因此收敛很快。

取初值

$$p_0 = 1.5,$$

迭代结果如下：

n	(a)	(b)	(c)	(d)	(e)
0	1.5	1.5	1.5	1.5	1.5
1	-0.875	0.8165	1.286953768	1.348399725	1.373333333
2	6.732	2.9969	1.402540804	1.367376372	1.36526015
3	-469.7	$\sqrt{-8.65}$	1.345458374	1.364957015	1.365230014
4	1.03×10^8	—	1.375170253	1.365264748	1.365230013
5	—	—	1.360094193	1.365225594	—
6	—	—	1.367846968	1.365230576	—
7	—	—	1.363887004	1.365229942	—
8	—	—	1.365916734	1.365230022	—
9	—	—	1.364878217	1.365230012	—
10	—	—	1.365410062	1.365230014	—
15	—	—	1.365223680	1.365230013	—
20	—	—	1.365230236	—	—
25	—	—	1.365230006	—	—
30	—	—	1.365230013	—	—

从这个表中可以看到：

- (a) 迅速发散；
- (b) 很快跑出定义域；
- (c) 收敛，但较慢；
- (d) 收敛很快，因为导数绝对值很小；
- (e) 即牛顿法，收敛最快。

这说明：不动点迭代的成败，并不只取决于原方程本身，而强烈依赖于你如何构造 $g(x)$ 。

4.1.5 牛顿法 (Newton Method)

牛顿法的出发点是：在当前点用切线近似函数。设 p 是方程

$$f(x) = 0$$

的根，取近似值 p_0 ，对 f 在 p_0 处作泰勒展开：

$$f(p) = f(p_0) + (p - p_0)f'(p_0) + \frac{(p - p_0)^2}{2}f''(\xi(p)).$$

由于 $f(p) = 0$ ，忽略二阶项可得

$$0 \approx f(p_0) + (p - p_0)f'(p_0),$$

所以

$$p \approx p_0 - \frac{f(p_0)}{f'(p_0)} \equiv p_1.$$

于是得到迭代格式

$$p_n = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})}, \quad n \geq 1.$$

几何解释在点 p_{n-1} 处作曲线 $y = f(x)$ 的切线，切线与 x 轴的交点作为新的近似值 p_n 。因此牛顿法本质上是切线迭代法。

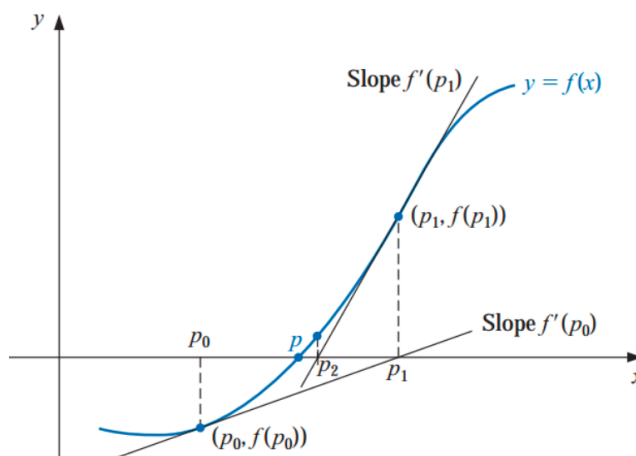


图 4.1: 牛顿法

优点与局限

- 对单根且初值足够好时，牛顿法通常具有二次收敛；
- 需要计算一阶导数；
- 初值选得不好时可能发散，或者收敛到错误的根；
- 对重根情形，普通牛顿法会显著降速。

4.1.6 割线法 (Secant Method)

牛顿法的主要代价在于要显式求导。若导数难以计算，可以用差商近似：

$$f'(p_{n-1}) = \lim_{x \rightarrow p_{n-1}} \frac{f(x) - f(p_{n-1})}{x - p_{n-1}} \approx \frac{f(p_{n-1}) - f(p_{n-2})}{p_{n-1} - p_{n-2}}.$$

代回牛顿迭代式，得到割线法：

$$p_n = p_{n-1} - \frac{f(p_{n-1})(p_{n-1} - p_{n-2})}{f(p_{n-1}) - f(p_{n-2})}.$$

解释它实际上用经过两点

$$(p_{n-2}, f(p_{n-2})), \quad (p_{n-1}, f(p_{n-1}))$$

的割线来代替切线，因此称为**割线法 (Secant Method)**。

特点

- 不需要显式求导；
- 每一步只需一次新的函数值；
- 局部收敛速度通常快于线性，但慢于牛顿法；
- 后面很多多变量算法都会借用“用差分代替导数”的基本思想。

4.1.7 多重根问题与修正牛顿法 (Modified Newton Method)

若 p 是 f 的 m 重根，可写成

$$f(x) = (x - p)^m q(x), \quad q(p) \neq 0.$$

这时普通牛顿法会退化为线性收敛。为恢复更快的收敛速度，引入

$$\mu(x) = \frac{f(x)}{f'(x)}.$$

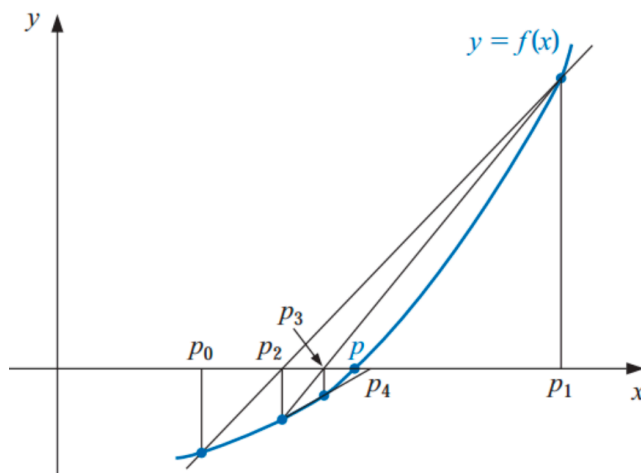


图 4.2: 割线法

代入分解式可得

$$\mu(x) = \frac{(x-p)^m q(x)}{m(x-p)^{m-1} q(x) + (x-p)^m q'(x)} = (x-p) \frac{q(x)}{mq(x) + (x-p)q'(x)}.$$

因此 $\mu(x)$ 在 $x=p$ 处有简单零点, 于是可以转而求解

$$\mu(x) = 0.$$

对 μ 应用牛顿法:

$$g(x) = x - \frac{\mu(x)}{\mu'(x)}.$$

经过化简可得

$$g(x) = x - \frac{f(x)f'(x)}{[f'(x)]^2 - f(x)f''(x)}.$$

于是修正牛顿迭代为

$$p_{n+1} = p_n - \frac{f(p_n)f'(p_n)}{[f'(p_n)]^2 - f(p_n)f''(p_n)}.$$

笔记

- 普通牛顿法把“求根”视为简单零点问题;
- 修正牛顿法先把多重根问题转化为简单根问题, 再做牛顿迭代;
- 代价是需要用到二阶导数, 但对于多重根, 其收敛速度明显改善。

4.1.8 Aitken 的 Δ^2 加速与 Steffensen 方法

若一个序列 $\{p_n\}$ 线性收敛到 p , 则近似有

$$\frac{p_{n+1} - p}{p_n - p} \approx \frac{p_{n+2} - p}{p_{n+1} - p},$$

即

$$(p_{n+1} - p)^2 \approx (p_{n+2} - p)(p_n - p).$$

解出 p , 得

$$p \approx \frac{p_{n+2}p_n - p_{n+1}^2}{p_{n+2} - 2p_{n+1} + p_n} = p_n - \frac{(p_{n+1} - p_n)^2}{p_{n+2} - 2p_{n+1} + p_n}.$$

定义

$$\Delta p_n = p_{n+1} - p_n, \quad \Delta^2 p_n = \Delta p_{n+1} - \Delta p_n = p_{n+2} - 2p_{n+1} + p_n,$$

则 Aitken 的 Δ^2 公式可写成

$$\hat{p}_n = p_n - \frac{(\Delta p_n)^2}{\Delta^2 p_n}.$$

Steffensen 方法若原不动点迭代为

$$p_{n+1} = g(p_n),$$

则可以用 Aitken 加速构造不需要导数的快速迭代:

$$p_0^{(0)}, \quad p_1^{(0)} = g(p_0^{(0)}), \quad p_2^{(0)} = g(p_1^{(0)}),$$

先形成

$$p_0^{(1)} = \{\Delta^2\}(p_0^{(0)}),$$

再令

$$p_1^{(1)} = g(p_0^{(1)}), \quad \dots$$

这就是 **Steffensen 方法**。它常被看成“不显式求导的牛顿法”。

4.1.9 代数多项式与 Horner (秦九韶) 算法

设多项式写成因式形式

$$P(x) = a_n(x - x_1)^{m_1}(x - x_2)^{m_2} \cdots (x - x_k)^{m_k}.$$

一个基本事实是: 次数不超过 n 的两个多项式, 若在 $n+1$ 个不同点取值相同, 则它们恒等相同。

现在把多项式写成标准形式

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0.$$

为了高效计算 $P(x_0)$, 引入递推

$$b_n = a_n, \quad b_k = a_k + b_{k+1}x_0, \quad k = n-1, n-2, \dots, 1, 0.$$

于是

$$b_0 = P(x_0).$$

同时定义

$$Q(x) = b_n x^{n-1} + b_{n-1} x^{n-2} + \cdots + b_2 x + b_1,$$

则有

$$P(x) = (x - x_0)Q(x) + b_0.$$

对其求导得

$$P'(x) = Q(x) + (x - x_0)Q'(x),$$

从而

$$P'(x_0) = Q(x_0).$$

与牛顿法结合若想求多项式的一个根, 可取初值 x_0 并作牛顿迭代:

$$x_1 = x_0 - \frac{P(x_0)}{P'(x_0)} = x_0 - \frac{P(x_0)}{Q(x_0)},$$

进而

$$x_n = x_{n-1} - \frac{P(x_{n-1})}{P'(x_{n-1})}.$$

若某一步出现 $b_0 = 0$, 就说明 x_0 本身已经是根, 此时

$$P(x) = (x - x_0)Q(x),$$

通过对 Q 继续求根，可以逐步得到其他根。这就是 **Horner 算法**，在中国数学史中也称**秦九韶算法**。

方法意义

- 它把多项式求值从大量幂运算转化为线性递推；
- 计算代价低，数值实现高效；
- 在多项式求根中，与牛顿法配合尤其常见。

4.1.10 Brent 方法 (Van Wijngaarden–Dekker–Brent Method)

Brent 方法是现代一维求根中极其重要的混合法。它把三种思想结合起来：

1. 根区间保持 (root bracketing)；
2. 二分法 (bisection)；
3. 逆二次插值 (inverse quadratic interpolation)。

设已有三点

$$[a, f(a)], \quad [b, f(b)], \quad [c, f(c)].$$

对函数的反函数关系 $x = x(y)$ 做二次插值，可写成

$$x = \frac{[y - f(a)][y - f(b)]c}{[f(c) - f(a)][f(c) - f(b)]} + \frac{[y - f(b)][y - f(c)]a}{[f(a) - f(b)][f(a) - f(c)]} + \frac{[y - f(c)][y - f(a)]b}{[f(b) - f(c)][f(b) - f(a)]}.$$

求根时取 $y = 0$ ，于是得到新的根近似。

Brent 方法常把这一步写成

$$x = b + \frac{P}{Q},$$

其中

$$R = \frac{f(b)}{f(c)}, \quad S = \frac{f(b)}{f(a)}, \quad T = \frac{f(a)}{f(c)},$$

并且

$$P = S[T(R - T)(c - b) - (1 - R)(b - a)],$$

$$Q = (T - 1)(R - 1)(S - 1).$$

核心思想如果插值修正量 P/Q 导致新点落在当前根区间之外，或者虽然仍在区间内，但区间缩小得不够快，那么算法就自动退回到二分法。这保证了：

- 既保持二分法的稳健性；
- 又尽可能利用高阶插值带来的快速收敛。

因此，Brent 方法通常兼具**高可靠性**与**高效率**，在工程与科学计算软件中被广泛采用。

4.2 非线性方程组

在许多数值计算问题中，我们需要求解的是一组非线性代数方程

$$\begin{cases} f_1(x_1, x_2, \dots, x_n) = 0, \\ f_2(x_1, x_2, \dots, x_n) = 0, \\ \vdots \\ f_n(x_1, x_2, \dots, x_n) = 0. \end{cases}$$

若记

$$\mathbf{x} = (x_1, x_2, \dots, x_n)^T, \quad \mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_n(\mathbf{x}))^T,$$

则问题可以紧凑地写成

$$\mathbf{F}(\mathbf{x}) = \mathbf{0}.$$

这一类问题通常没有统一的显式解析解，因此数值迭代法（iterative method）是最主要的求解手段。本节依次介绍不动点迭代（fixed-point iteration）、牛顿法（Newton method）、准牛顿法（quasi-Newton method）、Broyden 方法以及改进的 Broyden 方法（modified Broyden method）。

4.2.1 不动点迭代法

4.2.1.1 基本思想

若能把方程组

$$\mathbf{F}(\mathbf{x}) = \mathbf{0}$$

改写为

$$\mathbf{x} = \mathbf{G}(\mathbf{x}),$$

则原问题就转化为寻找映射 $\mathbf{G}$ 的不动点（fixed point）。于是可构造迭代格式

$$\mathbf{x}^{(k)} = \mathbf{G}(\mathbf{x}^{(k-1)}), \quad k = 1, 2, \dots$$

其分量形式为

$$x_i^{(k)} = g_i(x_1^{(k-1)}, x_2^{(k-1)}, \dots, x_n^{(k-1)}), \quad i = 1, 2, \dots, n.$$

不动点迭代的优点是形式简单、实现方便；缺点是收敛速度通常较慢，而且是否收敛高度依赖于改写方式。若在某邻域内 $\mathbf{G}$ 是压缩映射（contraction mapping），则迭代可收敛到唯一不动点。在线性化意义下，若在解 $\mathbf{x}^*$ 附近有

$$\rho(\mathbf{J}_{\mathbf{G}}(\mathbf{x}^*)) < 1,$$

其中 $\rho(\cdot)$ 表示谱半径（spectral radius），则局部收敛通常可以保证。

4.2.1.2 一个三元非线性方程组示例

考虑方程组

$$\begin{cases} f_1(x_1, x_2, x_3) = 3x_1 - \cos(x_2x_3) - \frac{1}{2} = 0, \\ f_2(x_1, x_2, x_3) = x_1^2 - 81(x_2 + 0.1)^2 + \sin x_3 + 1.06 = 0, \\ f_3(x_1, x_2, x_3) = e^{-x_1x_2} + 20x_3 + \frac{10\pi - 3}{3} = 0. \end{cases}$$

将其改写为

$$\begin{cases} x_1 = \frac{1}{3} \cos(x_2x_3) + \frac{1}{6}, \\ x_2 = \frac{1}{9} \sqrt{x_1^2 + \sin x_3 + 1.06} - 0.1, \\ x_3 = -\frac{1}{20} e^{-x_1x_2} - \frac{10\pi - 3}{60}. \end{cases}$$

于是得到最直接的 Jacobi 型不动点迭代：

$$\begin{cases} x_1^{(k)} = \frac{1}{3} \cos(x_2^{(k-1)}x_3^{(k-1)}) + \frac{1}{6}, \\ x_2^{(k)} = \frac{1}{9} \sqrt{(x_1^{(k-1)})^2 + \sin x_3^{(k-1)} + 1.06} - 0.1, \\ x_3^{(k)} = -\frac{1}{20} e^{-x_1^{(k-1)}x_2^{(k-1)}} - \frac{10\pi - 3}{60}. \end{cases}$$

这里第二个式子在开方时取了正根，这意味着我们选择的是与目标解相匹配的一条分支。事实上，可以直接验证该方程组有一个很自然的精确解

$$(x_1, x_2, x_3) = \left(\frac{1}{2}, 0, -\frac{\pi}{6}\right).$$

因为当 $x_2 = 0$ 时，

$$3x_1 - \cos(0) - \frac{1}{2} = 0 \implies 3x_1 - 1 - \frac{1}{2} = 0 \implies x_1 = \frac{1}{2},$$

又

$$e^{-x_1 x_2} = e^0 = 1, \quad 1 + 20x_3 + \frac{10\pi - 3}{3} = 0 \implies x_3 = -\frac{\pi}{6},$$

并且

$$x_1^2 - 81(x_2 + 0.1)^2 + \sin x_3 + 1.06 = \frac{1}{4} - 81 \cdot 0.01 - \frac{1}{2} + 1.06 = 0.$$

4.2.1.3 加速的不动点迭代

若在同一步中尽可能使用最新计算出的分量，则可得到类似 Gauss-Seidel 思想的加速格式：

$$\begin{cases} x_1^{(k)} = \frac{1}{3} \cos(x_2^{(k-1)} x_3^{(k-1)}) + \frac{1}{6}, \\ x_2^{(k)} = \frac{1}{9} \sqrt{(x_1^{(k)})^2 + \sin x_3^{(k-1)} + 1.06} - 0.1, \\ x_3^{(k)} = -\frac{1}{20} e^{-x_1^{(k)} x_2^{(k)}} - \frac{10\pi - 3}{60}. \end{cases}$$

与前一种格式相比，这里计算 $x_2^{(k)}$ 时已经用到了新的 $x_1^{(k)}$ ，计算 $x_3^{(k)}$ 时又进一步用到了新的 $x_1^{(k)}$ 与 $x_2^{(k)}$ 。这类“就地更新”通常会改善收敛速度，但其效果仍然取决于具体问题的结构。

4.2.2 牛顿法

4.2.2.1 由单变量牛顿法到多变量牛顿法

单变量方程

$$f(x) = 0$$

的牛顿迭代为

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

它的本质是：在点 x_n 处用切线近似函数，再用该切线与 x 轴的交点作为下一步迭代值。

对非线性方程组

$$\mathbf{F}(\mathbf{x}) = \mathbf{0},$$

需要引入 Jacobian 矩阵 (Jacobian matrix)

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1}(\mathbf{x}) & \frac{\partial f_1}{\partial x_2}(\mathbf{x}) & \cdots & \frac{\partial f_1}{\partial x_n}(\mathbf{x}) \\ \frac{\partial f_2}{\partial x_1}(\mathbf{x}) & \frac{\partial f_2}{\partial x_2}(\mathbf{x}) & \cdots & \frac{\partial f_2}{\partial x_n}(\mathbf{x}) \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1}(\mathbf{x}) & \frac{\partial f_n}{\partial x_2}(\mathbf{x}) & \cdots & \frac{\partial f_n}{\partial x_n}(\mathbf{x}) \end{bmatrix}.$$

若在当前迭代点 $\mathbf{x}^{(k-1)}$ 附近作一阶 Taylor 展开：

$$\mathbf{F}(\mathbf{x}^{(k-1)} + \mathbf{y}) \approx \mathbf{F}(\mathbf{x}^{(k-1)}) + \mathbf{J}(\mathbf{x}^{(k-1)}) \mathbf{y},$$

为了让下一步尽量满足 $\mathbf{F} = \mathbf{0}$ ，令右端近似为零，即

$$\mathbf{J}(\mathbf{x}^{(k-1)})\mathbf{y} = -\mathbf{F}(\mathbf{x}^{(k-1)}),$$

再令

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \mathbf{y}.$$

这就得到多变量牛顿法：

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} - \mathbf{J}(\mathbf{x}^{(k-1)})^{-1} \mathbf{F}(\mathbf{x}^{(k-1)}).$$

若定义

$$\mathbf{G}(\mathbf{x}) = \mathbf{x} - \mathbf{J}(\mathbf{x})^{-1} \mathbf{F}(\mathbf{x}),$$

则也可写为

$$\mathbf{x}^{(k)} = \mathbf{G}(\mathbf{x}^{(k-1)}).$$

实际实现中的关键点在数值计算中通常不直接求 $\mathbf{J}^{-1}$ ，而是求解线性方程组

$$\mathbf{J}(\mathbf{x}^{(k-1)})\mathbf{y} = -\mathbf{F}(\mathbf{x}^{(k-1)}),$$

然后更新

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \mathbf{y}.$$

这样做既更稳定，也更高效。

4.2.2.2 牛顿法示例

仍考虑前面的三元非线性方程组。其 Jacobian 矩阵为

$$\mathbf{J}(x_1, x_2, x_3) = \begin{bmatrix} 3 & x_3 \sin(x_2 x_3) & x_2 \sin(x_2 x_3) \\ 2x_1 & -162(x_2 + 0.1) & \cos x_3 \\ -x_2 e^{-x_1 x_2} & -x_1 e^{-x_1 x_2} & 20 \end{bmatrix}.$$

各项偏导的来源如下：

$$\begin{aligned} \frac{\partial}{\partial x_2} [-\cos(x_2 x_3)] &= \sin(x_2 x_3) x_3, & \frac{\partial}{\partial x_3} [-\cos(x_2 x_3)] &= \sin(x_2 x_3) x_2, \\ \frac{\partial}{\partial x_2} [-81(x_2 + 0.1)^2] &= -162(x_2 + 0.1), & \frac{\partial}{\partial x_3} (\sin x_3) &= \cos x_3, \\ \frac{\partial}{\partial x_1} (e^{-x_1 x_2}) &= -x_2 e^{-x_1 x_2}, & \frac{\partial}{\partial x_2} (e^{-x_1 x_2}) &= -x_1 e^{-x_1 x_2}. \end{aligned}$$

取初值

$$\mathbf{x}^{(0)} = (0.1, 0.1, -0.1)^\top.$$

代入可得

$$\mathbf{F}(\mathbf{x}^{(0)}) = \begin{bmatrix} -1.1999500004 \\ -2.2698334166 \\ 8.4620253457 \end{bmatrix}.$$

于是第一步需要求解

$$\mathbf{J}(\mathbf{x}^{(0)})\mathbf{y}^{(0)} = -\mathbf{F}(\mathbf{x}^{(0)}).$$

数值结果为

$$\mathbf{y}^{(0)} = \begin{bmatrix} 0.3998696729 \\ -0.0805331515 \\ -0.4215204719 \end{bmatrix},$$

从而

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \mathbf{y}^{(0)} = \begin{bmatrix} 0.4998696729 \\ 0.0194668485 \\ -0.5215204719 \end{bmatrix}.$$

继续迭代时，每一步都求解

$$\mathbf{J}(\mathbf{x}^{(k-1)})\mathbf{y}^{(k-1)} = -\mathbf{F}(\mathbf{x}^{(k-1)}), \quad \mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \mathbf{y}^{(k-1)}.$$

若初值足够接近真解且 Jacobian 在真解附近非奇异，则牛顿法通常具有二次收敛 (quadratic convergence) 的优良性质。

4.2.3 准牛顿法

4.2.3.1 基本想法

牛顿法的主要代价在于：每一步都要重新计算 Jacobian 矩阵，并求解新的线性方程组。当 Jacobian 很难显式写出、计算成本很高，或者问题维数很大时，希望用一个近似矩阵 $\mathbf{A}$ 来代替 Jacobian。

设已经有两次迭代点 $\mathbf{x}^{(0)}$ 与 $\mathbf{x}^{(1)}$ ，记

$$\mathbf{s}_0 = \mathbf{x}^{(1)} - \mathbf{x}^{(0)}, \quad \mathbf{y}_0 = \mathbf{F}(\mathbf{x}^{(1)}) - \mathbf{F}(\mathbf{x}^{(0)}).$$

一个合理的要求是让近似矩阵 $\mathbf{A}_1$ 满足割线条件 (secant condition)

$$\mathbf{A}_1 \mathbf{s}_0 = \mathbf{y}_0,$$

这正是多变量情形下

$$f'(x_1) \approx \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

的推广。

但仅有这一条件还不足以唯一确定 $\mathbf{A}_1$ 。为此再加上一条自然约束：对任意与 $\mathbf{s}_0$ 正交的向量 $\mathbf{z}$ ，更新不改变旧 Jacobian 在该方向上的作用，即

$$\mathbf{s}_0^T \mathbf{z} = 0 \implies \mathbf{A}_1 \mathbf{z} = \mathbf{J}(\mathbf{x}^{(0)}) \mathbf{z}.$$

这意味着从 $\mathbf{J}(\mathbf{x}^{(0)})$ 到 $\mathbf{A}_1$ 的修正只发生在 $\mathbf{s}_0$ 所张成的方向上，因此可以设它是秩一修正 (rank-one update)：

$$\mathbf{A}_1 = \mathbf{J}(\mathbf{x}^{(0)}) + \mathbf{u} \mathbf{s}_0^T.$$

将割线条件代入，得到

$$\mathbf{A}_1 \mathbf{s}_0 = \mathbf{J}(\mathbf{x}^{(0)}) \mathbf{s}_0 + \mathbf{u} \mathbf{s}_0^T \mathbf{s}_0 = \mathbf{y}_0,$$

故

$$\mathbf{u} = \frac{\mathbf{y}_0 - \mathbf{J}(\mathbf{x}^{(0)}) \mathbf{s}_0}{\|\mathbf{s}_0\|_2^2}.$$

于是

$$\boxed{\mathbf{A}_1 = \mathbf{J}(\mathbf{x}^{(0)}) + \frac{[\mathbf{F}(\mathbf{x}^{(1)}) - \mathbf{F}(\mathbf{x}^{(0)}) - \mathbf{J}(\mathbf{x}^{(0)})(\mathbf{x}^{(1)} - \mathbf{x}^{(0)})](\mathbf{x}^{(1)} - \mathbf{x}^{(0)})^T}{\|\mathbf{x}^{(1)} - \mathbf{x}^{(0)}\|_2^2}}.$$

这是最基本的准牛顿秩一更新思想。

4.2.3.2 一般迭代格式

记

$$s_i = \mathbf{x}^{(i)} - \mathbf{x}^{(i-1)}, \quad \mathbf{y}_i = \mathbf{F}(\mathbf{x}^{(i)}) - \mathbf{F}(\mathbf{x}^{(i-1)}).$$

则 Jacobian 近似矩阵按

$$\mathbf{A}_i = \mathbf{A}_{i-1} + \frac{(\mathbf{y}_i - \mathbf{A}_{i-1}\mathbf{s}_i)\mathbf{s}_i^\top}{\|\mathbf{s}_i\|_2^2}$$

更新，而新的迭代点由

$$\mathbf{x}^{(i+1)} = \mathbf{x}^{(i)} - \mathbf{A}_i^{-1}\mathbf{F}(\mathbf{x}^{(i)})$$

给出。

也可以把第二个式子写成线性方程组的形式。若记

$$\mathbf{s}_{i+1} = \mathbf{x}^{(i+1)} - \mathbf{x}^{(i)},$$

则有

$$\mathbf{A}_i\mathbf{s}_{i+1} = -\mathbf{F}(\mathbf{x}^{(i)}).$$

这里常有人担心它是否与割线条件

$$\mathbf{A}_i\mathbf{s}_i = \mathbf{y}_i$$

矛盾。实际上两者并不冲突：前者是**当前步**的牛顿型修正方程，后者是上**一段步长**所提供的割线信息。它们作用于不同的向量，因此完全可以同时成立。

4.2.3.3 Sherman–Morrison 公式与逆矩阵更新

若每次都重新求逆 $\mathbf{A}_i^{-1}$ ，开销仍较大。由于 $\mathbf{A}_i$ 是由 $\mathbf{A}_{i-1}$ 经秩一修正得到的，可以使用 Sherman–Morrison 公式：

$$(\mathbf{A} + \mathbf{x}\mathbf{y}^\top)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\mathbf{x}\mathbf{y}^\top\mathbf{A}^{-1}}{1 + \mathbf{y}^\top\mathbf{A}^{-1}\mathbf{x}}.$$

在这里取

$$\mathbf{A} = \mathbf{A}_{i-1}, \quad \mathbf{x} = \frac{\mathbf{y}_i - \mathbf{A}_{i-1}\mathbf{s}_i}{\|\mathbf{s}_i\|_2^2}, \quad \mathbf{y} = \mathbf{s}_i,$$

便有

$$\mathbf{A}_i^{-1} = \left(\mathbf{A}_{i-1} + \frac{(\mathbf{y}_i - \mathbf{A}_{i-1}\mathbf{s}_i)\mathbf{s}_i^\top}{\|\mathbf{s}_i\|_2^2} \right)^{-1}.$$

代入公式并整理：

$$\mathbf{A}_i^{-1} = \mathbf{A}_{i-1}^{-1} - \frac{\mathbf{A}_{i-1}^{-1} \left(\frac{\mathbf{y}_i - \mathbf{A}_{i-1}\mathbf{s}_i}{\|\mathbf{s}_i\|_2^2} \right) \mathbf{s}_i^\top \mathbf{A}_{i-1}^{-1}}{1 + \mathbf{s}_i^\top \mathbf{A}_{i-1}^{-1} \left(\frac{\mathbf{y}_i - \mathbf{A}_{i-1}\mathbf{s}_i}{\|\mathbf{s}_i\|_2^2} \right)}.$$

再利用

$$\mathbf{A}_{i-1}^{-1}\mathbf{A}_{i-1}\mathbf{s}_i = \mathbf{s}_i,$$

可化为

$$\mathbf{A}_i^{-1} = \mathbf{A}_{i-1}^{-1} - \frac{(\mathbf{A}_{i-1}^{-1}\mathbf{y}_i - \mathbf{s}_i)\mathbf{s}_i^\top \mathbf{A}_{i-1}^{-1}}{\mathbf{s}_i^\top \mathbf{A}_{i-1}^{-1}\mathbf{y}_i}.$$

等价地写成更常见的形式：

$$\mathbf{A}_i^{-1} = \mathbf{A}_{i-1}^{-1} + \frac{(\mathbf{s}_i - \mathbf{A}_{i-1}^{-1}\mathbf{y}_i)\mathbf{s}_i^{\top}\mathbf{A}_{i-1}^{-1}}{\mathbf{s}_i^{\top}\mathbf{A}_{i-1}^{-1}\mathbf{y}_i}.$$

因此，一旦已知上一轮的逆矩阵近似，就可以用秩一修正直接得到新一轮的逆矩阵近似，而不必重新做完整的矩阵求逆，效率会明显提高。

4.2.4 Broyden 方法

4.2.4.1 基本公式

Broyden 方法可以看成准牛顿法的一种标准实现。记

$$\mathbf{f}_n = \mathbf{f}(\mathbf{x}_n), \quad \Delta\mathbf{x}_n = \mathbf{x}_n - \mathbf{x}_{n-1}, \quad \Delta\mathbf{f}_n = \mathbf{f}_n - \mathbf{f}_{n-1}.$$

则割线关系写为

$$\mathbf{J}_n \Delta\mathbf{x}_n \simeq \Delta\mathbf{f}_n.$$

于是 Jacobian 的 Broyden 更新公式为

$$\mathbf{J}_n = \mathbf{J}_{n-1} + \frac{(\Delta\mathbf{f}_n - \mathbf{J}_{n-1}\Delta\mathbf{x}_n)\Delta\mathbf{x}_n^{\top}}{\|\Delta\mathbf{x}_n\|_2^2}.$$

随后用该近似 Jacobian 做牛顿型修正：

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \mathbf{J}_n^{-1}\mathbf{f}(\mathbf{x}_n).$$

若再结合 Sherman–Morrison 公式，则逆矩阵满足

$$\mathbf{J}_n^{-1} = \mathbf{J}_{n-1}^{-1} + \frac{(\Delta\mathbf{x}_n - \mathbf{J}_{n-1}^{-1}\Delta\mathbf{f}_n)\Delta\mathbf{x}_n^{\top}\mathbf{J}_{n-1}^{-1}}{\Delta\mathbf{x}_n^{\top}\mathbf{J}_{n-1}^{-1}\Delta\mathbf{f}_n}.$$

4.2.4.2 Broyden 方法示例

考虑求解方程组

$$\mathbf{g}(x, y, z) = \mathbf{0},$$

其中

$$\mathbf{g}(x, y, z) = \begin{bmatrix} x^2 + y^2 + z^2 - 3 \\ x^2 + y^2 - z - 1 \\ x + y + z - 3 \end{bmatrix}.$$

取初值

$$\mathbf{x}^{(0)} = (x^{(0)}, y^{(0)}, z^{(0)}) = (1, 0, 1).$$

其 Jacobian 为

$$\mathbf{J}_g(x, y, z) = \begin{bmatrix} 2x & 2y & 2z \\ 2x & 2y & -1 \\ 1 & 1 & 1 \end{bmatrix}.$$

于是初始矩阵

$$\mathbf{D}_0 = \mathbf{J}_g(\mathbf{x}^{(0)}) = \begin{bmatrix} 2 & 0 & 2 \\ 2 & 0 & -1 \\ 1 & 1 & 1 \end{bmatrix}.$$

又因为

$$\mathbf{g}(\mathbf{x}^{(0)}) = \begin{bmatrix} 1^2 + 0^2 + 1^2 - 3 \\ 1^2 + 0^2 - 1 - 1 \\ 1 + 0 + 1 - 3 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix},$$

第一步相当于解线性方程组

$$\mathbf{D}_0 \mathbf{d}^{(0)} = -\mathbf{g}(\mathbf{x}^{(0)}) = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix},$$

其中

$$\mathbf{d}^{(0)} = \begin{bmatrix} x^{(1)} - x^{(0)} \\ y^{(1)} - y^{(0)} \\ z^{(1)} - z^{(0)} \end{bmatrix}.$$

也就是

$$\begin{bmatrix} 2 & 0 & 2 \\ 2 & 0 & -1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x^{(1)} - 1 \\ y^{(1)} \\ z^{(1)} - 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

解得

$$\mathbf{d}^{(0)} = \left(\frac{1}{2}, \frac{1}{2}, 0\right)^{\top}, \quad \mathbf{x}^{(1)} = \left(\frac{3}{2}, \frac{1}{2}, 1\right)^{\top}.$$

下面更新 Jacobian 近似。先算

$$\mathbf{g}(\mathbf{x}^{(1)}) = \begin{bmatrix} \left(\frac{3}{2}\right)^2 + \left(\frac{1}{2}\right)^2 + 1^2 - 3 \\ \left(\frac{3}{2}\right)^2 + \left(\frac{1}{2}\right)^2 - 1 - 1 \\ \frac{3}{2} + \frac{1}{2} + 1 - 3 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \\ 0 \end{bmatrix}.$$

由于

$$\mathbf{D}_0 \mathbf{d}^{(0)} = -\mathbf{g}(\mathbf{x}^{(0)}),$$

所以

$$\Delta \mathbf{g}^{(1)} - \mathbf{D}_0 \mathbf{d}^{(0)} = (\mathbf{g}(\mathbf{x}^{(1)}) - \mathbf{g}(\mathbf{x}^{(0)})) - \mathbf{D}_0 \mathbf{d}^{(0)} = \mathbf{g}(\mathbf{x}^{(1)}).$$

于是更新公式可简化为

$$\mathbf{D}_1 = \mathbf{D}_0 + \frac{1}{\mathbf{d}^{(0)} \cdot \mathbf{d}^{(0)}} \mathbf{g}(\mathbf{x}^{(1)}) \otimes \mathbf{d}^{(0)},$$

其中 $\otimes$ 表示外积 (outer product), 即

$$\mathbf{u} \otimes \mathbf{v} = \mathbf{u} \mathbf{v}^{\top}.$$

因为

$$\mathbf{d}^{(0)} \cdot \mathbf{d}^{(0)} = \left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2 = \frac{1}{2},$$

且

$$\mathbf{g}(\mathbf{x}^{(1)}) \otimes \mathbf{d}^{(0)} = \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \\ 0 \end{bmatrix} \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \end{bmatrix} = \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & 0 \\ \frac{1}{4} & \frac{1}{4} & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

所以

$$\mathbf{D}_1 = \begin{bmatrix} 2 & 0 & 2 \\ 2 & 0 & -1 \\ 1 & 1 & 1 \end{bmatrix} + 2 \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & 0 \\ \frac{1}{4} & \frac{1}{4} & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} \frac{5}{2} & \frac{1}{2} & 2 \\ \frac{5}{2} & \frac{1}{2} & -1 \\ 1 & 1 & 1 \end{bmatrix}.$$

第二步需解

$$\mathbf{D}_1 \begin{bmatrix} x^{(2)} - \frac{3}{2} \\ y^{(2)} - \frac{1}{2} \\ z^{(2)} - 1 \end{bmatrix} = -\mathbf{g}(\mathbf{x}^{(1)}) = \begin{bmatrix} -\frac{1}{2} \\ -\frac{1}{2} \\ 0 \end{bmatrix}.$$

解得

$$\mathbf{d}^{(1)} = \begin{bmatrix} -\frac{1}{4} \\ \frac{1}{4} \\ 0 \end{bmatrix}, \quad \mathbf{x}^{(2)} = \left(\frac{5}{4}, \frac{3}{4}, 1 \right)^T.$$

继续迭代可逐步逼近解。事实上该方程组有明显解 $(1, 1, 1)$ ，而上述迭代序列正向该解靠近。

4.2.5 改进的 Broyden 方法

4.2.5.1 从自洽迭代出发

在很多大规模问题中，尤其是自洽场（self-consistent field, SCF）计算中，待求的并不是一个低维向量，而是一个高维对象 V 。此时常把问题写成

$$\mathbf{F}(V) = \mathbf{0}.$$

若 $V_{\text{in}}^{(m)}$ 表示第 m 步输入， $V_{\text{out}}^{(m)}$ 表示由“黑箱过程”输出的结果，则残差（residual）可定义为

$$\mathbf{F}^{(m)} = V_{\text{out}}^{(m)} - V_{\text{in}}^{(m)}.$$

最简单的线性混合（linear mixing）为

$$V_{\text{in}}^{(m+1)} = \alpha V_{\text{out}}^{(m)} + (1 - \alpha) V_{\text{in}}^{(m)} = V_{\text{in}}^{(m)} + \alpha \mathbf{F}^{(m)},$$

其中 α 称为混合参数（mixing parameter）。

线性混合实现简单、适用面广，但在复杂非线性问题中收敛往往很慢，甚至可能振荡。典型场景包括薛定谔方程、自洽 Hartree–Fock 方程，以及更一般的黑箱非线性映射。

4.2.5.2 标准 Broyden 形式

若把对 V 的更新视为非线性方程求根问题，则可写成

$$V_{\text{in}}^{(m+1)} = V_{\text{in}}^{(m)} - \mathbf{B}^{(m)} \mathbf{F}^{(m)},$$

其中

$$\mathbf{B}^{(m)} = (\mathbf{J}^{(m)})^{-1}, \quad J_{jk}^{(m)} = \frac{\partial F_j(V)}{\partial V_k}.$$

即 $\mathbf{B}^{(m)}$ 是 Jacobian 的逆近似。

标准 Broyden 逆矩阵更新公式可以写成

$$\mathbf{B}^{(m+1)} = \mathbf{B}^{(m)} + \frac{|\delta V_{\text{in}}\rangle - \mathbf{B}^{(m)} |\delta F\rangle}{\langle \delta V_{\text{in}} | \mathbf{B}^{(m)} | \delta F \rangle} \langle \delta V_{\text{in}} | \mathbf{B}^{(m)},$$

其中

$$|\delta V_{\text{in}}\rangle = V_{\text{in}}^{(m+1)} - V_{\text{in}}^{(m)}, \quad |\delta F\rangle = \mathbf{F}^{(m+1)} - \mathbf{F}^{(m)}.$$

这本质上仍然是秩一更新，只是写成了适合大规模问题的逆矩阵形式。

4.2.5.3 改进的 Broyden 方法

在真正的大规模计算中，问题的维数 N 往往非常大，直接维护一个 $N \times N$ 的 Jacobian 或其逆矩阵都不现实。改进的 Broyden 方法的核心思想是：**不显式存储完整的大矩阵，而只利用最近 M 次迭代中积累的信息。**

其更新格式可写为

$$V_{\text{in}}^{(m+1)} = V_{\text{in}}^{(m)} + \alpha F^{(m)} - \sum_{n=\tilde{m}}^{m-1} w_n \gamma_{mn} \mathbf{u}^{(n)}, \quad \tilde{m} = \max(1, m - M).$$

这表示：在简单线性混合

$$V_{\text{in}}^{(m)} + \alpha F^{(m)}$$

的基础上，再减去一个由过去 M 次历史信息构造出的修正项。

其中

$$\begin{aligned} \gamma_{mn} &= \sum_{k=\tilde{m}}^{m-1} c_k^{(m)} \beta_{kn}, & \beta_{kn} &= (\omega_0^2 \mathbf{I} + \mathbf{a})_{kn}^{-1}, \\ c_k^{(m)} &= w_k [\Delta \mathbf{F}^{(k)}]^\dagger \mathbf{F}^{(m)}, & a_{kn} &= w_k w_n (\Delta \mathbf{F}^{(n)})^\dagger \Delta \mathbf{F}^{(k)}, \\ \mathbf{u}^{(n)} &= \alpha \Delta \mathbf{F}^{(n)} + \Delta \mathbf{V}^{(n)}. \end{aligned}$$

这里

$$\Delta \mathbf{V}^{(n)} = \frac{V_{\text{in}}^{(n+1)} - V_{\text{in}}^{(n)}}{\|\mathbf{F}^{(n+1)} - \mathbf{F}^{(n)}\|}, \quad \Delta \mathbf{F}^{(n)} = \frac{\mathbf{F}^{(n+1)} - \mathbf{F}^{(n)}}{\|\mathbf{F}^{(n+1)} - \mathbf{F}^{(n)}\|}.$$

上式中 $\dagger$ 表示共轭转置；若变量都是实数，则它就是普通转置。

算法结构的意义 矩阵 $\mathbf{a} = (a_{kn})$ 是一个 $M \times M$ 小矩阵，而不是 $N \times N$ 大矩阵。由于通常 $M \ll N$ ，因此改进的 Broyden 方法把高维问题的关键代数运算压缩到了一个小历史窗口上，从而兼顾了：

- 利用历史信息加速收敛；
- 避免显式存储和更新大维 Jacobian；
- 在复杂黑箱问题中保持较高的实用性。

常用参数 实际计算中常取

$$\alpha = 0.7, \quad \omega_0 = 0.01, \quad w_n = 1 \quad (n = 1, 2, \dots, M).$$

其中 α 控制线性混合的强弱， ω_0 用于对小矩阵求逆时进行适度正则化， w_n 则反映对不同历史步的加权方式。

实现层面的说明：如何计算 β 与 γ 改进的 Broyden 方法真正需要处理的，不是原始的 $N \times N$ 大型 Jacobian，而是历史窗口中出现的小矩阵

$$\omega_0^2 \mathbf{I} + \mathbf{a} \in \mathbb{R}^{M \times M}.$$

因此

$$\beta = (\omega_0^2 \mathbf{I} + \mathbf{a})^{-1}$$

的计算，本质上只是一个小规模稠密线性代数问题。实际程序实现中，常把该矩阵看成实对称矩阵，调用 LAPACK 的对称矩阵例程完成分解与求逆；一种典型实现思路是先做 ‘DSYTRF’ 分解，再用 ‘DSYTRI’ 得到逆矩阵，随后再计算

$$\gamma_{mn} = \sum_{k=\tilde{m}}^{m-1} c_k^{(m)} \beta_{kn}.$$

从数值分析的角度看，这一步之所以高效，是因为我们只在一个 $M \times M$ 的历史子空间里做代数运算，而不是在原始 N 维变量空间里直接构造和更新完整 Jacobian。通常 M 很小，因此这一实现往往比把原问题直接交给通用的大规模迭代线性求解器更经济；在自洽场 (SCF) 混合问题中，它常能表现出比朴素共轭梯度迭代更快的整体收敛速度。

这类方法已经广泛应用于电子结构计算等领域，例如在 PWSCF 一类程序的混合（mixing）模块中就能见到 Broyden mixing routine 的典型实现。

小结

非线性方程组的数值求解可以看成是“如何不断构造更好的局部线性近似”的过程：

- 不动点法最简单，本质上是寻找映射的不动点；
- 牛顿法直接使用 Jacobian 的一阶线性化，局部收敛快；
- 准牛顿法与 Broyden 方法用低秩更新近似 Jacobian，减少了每步重新求导的代价；
- 改进的 Broyden 方法进一步面向大规模问题，只保留有限步历史信息，从而在高维自洽计算中表现出很强的实用价值。

从方法发展的脉络看，核心始终是两点：一是尽可能准确地近似真实 Jacobian，二是尽可能低成本地更新这种近似。这正是现代非线性迭代算法设计的基本思想。

4.3 优化问题

4.3.1 基本概念与 Hessian 判据

优化问题（optimization problem）的基本目标，是对目标函数

$$f(\mathbf{x}), \quad \mathbf{x} = (x_1, x_2, \dots, x_n)^T,$$

在给定约束条件下求其极小值或极大值。若不特别说明，本节主要讨论无约束优化（unconstrained optimization）问题。按研究目标不同，又可分为局部优化（local optimization）与全局优化（global optimization）两类。

全局优化在物理、经济、工程以及人工智能（artificial intelligence）中都有极其广泛的应用。例如：

- 物理中参数拟合（parameter fitting）；
- 最小二乘法（least squares）；
- 变分能量极小问题（variational energy minimization）；
- 路径积分（path integral）中的最优轨道或最优作用量问题；
- 工程设计中的结构参数优化；
- 机器学习与神经网络训练中的损失函数极小化。

4.3.1.1 梯度与 Hessian 矩阵

设 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 二阶连续可微，则其梯度（gradient）为

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix},$$

其 Hessian 矩阵（Hessian matrix）定义为

$$\mathbf{H}_f(\mathbf{x}) = \left(\frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} \right)_{n \times n}.$$

即

$$(\mathbf{H}_f)_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}.$$

若 $f \in C^2$, 由混合偏导交换定理可知

$$\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial^2 f}{\partial x_j \partial x_i},$$

因此 Hessian 是对称矩阵 (symmetric matrix)。

4.3.1.2 二阶 Taylor 展开与驻点判别

在点 $\mathbf{x}$ 附近, 对 $f(\mathbf{x} + \Delta\mathbf{x})$ 作二阶 Taylor 展开:

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \Delta\mathbf{x} + \frac{1}{2} \Delta\mathbf{x}^\top \mathbf{H}_f(\mathbf{x}) \Delta\mathbf{x}.$$

这一定式是多维优化理论最核心的局部近似工具。若 $\mathbf{x}^*$ 是一个局部极值点, 那么一阶必要条件为

$$\nabla f(\mathbf{x}^*) = \mathbf{0}.$$

这样的点称为驻点 (stationary point) 或临界点 (critical point)。

进一步地, 若在驻点 $\mathbf{x}^*$ 处考察二次型

$$Q(\Delta\mathbf{x}) = \Delta\mathbf{x}^\top \mathbf{H}_f(\mathbf{x}^*) \Delta\mathbf{x},$$

则有如下二阶判据:

- 若 $\mathbf{H}_f(\mathbf{x}^*)$ 正定 (positive definite), 则 $\mathbf{x}^*$ 为严格局部极小点;
- 若 $\mathbf{H}_f(\mathbf{x}^*)$ 负定 (negative definite), 则 $\mathbf{x}^*$ 为严格局部极大点;
- 若 $\mathbf{H}_f(\mathbf{x}^*)$ 不定 (indefinite), 则 $\mathbf{x}^*$ 为鞍点 (saddle point);
- 若 $\mathbf{H}_f(\mathbf{x}^*)$ 半正定或半负定, 则二阶判据不足以作出结论, 还需考察更高阶项。

在二维情形

$$f = f(x, y), \quad \mathbf{H}_f = \begin{bmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{bmatrix},$$

可用行列式 (determinant) 形式的判别法:

$$D = f_{xx}f_{yy} - f_{xy}^2.$$

若在驻点处有

$$D > 0, \quad f_{xx} > 0,$$

则为局部极小点; 若

$$D > 0, \quad f_{xx} < 0,$$

则为局部极大点; 若

$$D < 0,$$

则为鞍点; 若

$$D = 0,$$

则判别失败。

例题 4.1 设

$$f(x, y) = x^2 + y^2.$$

则

$$\nabla f = (2x, 2y)^\top, \quad \mathbf{H}_f = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

驻点只有 $(0, 0)$, 且 Hessian 正定, 因此 $(0, 0)$ 是严格局部极小点, 也是全局极小点。

再看

$$f(x, y) = x^2 - y^2.$$

则

$$\mathbf{H}_f = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix},$$

这是不定矩阵，因此 $(0, 0)$ 是鞍点而非极值点。

4.3.2 全局优化方法概览

从算法思想上看，优化方法大体可以分为两大类：

1. 利用目标函数局部导数信息的梯度型方法；
2. 不依赖梯度信息的无导数方法（gradient-free methods）。

对于一个多峰函数曲面，算法往往要同时处理两个问题：

- **坡度与曲率**：即利用梯度 $\partial J / \partial u$ 与 Hessian $\partial^2 J / \partial u^2$ 判断下降方向和局部曲率；
- **代价函数值**：直接比较 J 的大小，寻找更小的函数值。

典型策略包括：

- **Gradient search**：利用梯度信息寻找下降方向；
- **Gradient-free search**：不用梯度，仅根据函数值更新，例如随机搜索（random search）和下山单纯形法（down-hill simplex method，也称 Nelder–Mead 方法）；
- **Simulated annealing**：模拟退火，通过随机跃迁避免陷入局部极小；
- **Genetic algorithms**：遗传算法，利用选择、交叉、变异实现群体演化；
- **Particle swarm optimization**：粒子群优化，模拟群体协同搜索。

梯度法通常在局部收敛速度和高维问题处理中更有优势；而模拟退火、遗传算法、粒子群等方法更强调全局搜索能力，适合复杂、多峰、不可微或黑箱型目标函数。

在实际计算中，也有许多现成的全局优化程序库（software library）可供调用。例如，

https://www.mat.univie.ac.at/~neum/glopt/software_g.html

给出了一批经典全局优化程序的整理入口。

随机搜索（Random Search） 随机搜索是最朴素的无导数全局优化思想。它不利用梯度，也不显式利用曲率信息，而是在可行域中不断产生候选点 $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$ ，然后比较目标函数值

$$f(\mathbf{x}^{(1)}), f(\mathbf{x}^{(2)}), \dots$$

保留其中更优的点。它的优点是实现非常简单、对目标函数光滑性要求很低，甚至可用于含噪声、不可微或黑箱（black-box）问题；缺点则是没有充分利用局部结构信息，因此在高维空间中通常效率不高。尽管如此，随机搜索常作为许多复杂全局优化算法的出发点或基准方法（baseline method）。

粒子群优化（Particle Swarm Optimization, PSO） 粒子群优化把每个候选解看成一只“粒子”（particle）。第 i 个粒子在第 k 步具有位置 $\mathbf{x}_i^{(k)}$ 和速度 $\mathbf{v}_i^{(k)}$ ，并记它历史上到达过的最好位置为 $\mathbf{p}_i$ ，整个群体当前发现的最好位置为 $\mathbf{g}$ 。其典型更新形式为

$$\begin{aligned} \mathbf{v}_i^{(k+1)} &= \omega \mathbf{v}_i^{(k)} + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i^{(k)}) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i^{(k)}), \\ \mathbf{x}_i^{(k+1)} &= \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)}, \end{aligned}$$

其中 ω 是惯性权重（inertia weight）， c_1, c_2 是学习因子（learning coefficients）， $r_1, r_2 \in [0, 1]$ 是随机数。这个公式说明：粒子既保留原来的运动趋势，又受到“个体经验”和“群体经验”的共同吸引。PSO 不需要显式求导，适合多峰、复杂或黑箱优化问题；但它对参数选择较敏感，也可能出现早熟收敛（premature convergence），即过早聚集到某个局部极小附近。

4.3.3 梯度法优化

4.3.3.1 基本下降思想

优化的最自然想法是：若当前点为 $\mathbf{x}_k$ ，则沿某个下降方向 $\mathbf{p}_k$ 前进，即

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k,$$

其中 $\alpha_k > 0$ 为步长 (step size)，通常由线搜索 (line search) 确定。

最速下降法 (steepest descent) 的选择是

$$\mathbf{p}_k = -\nabla f(\mathbf{x}_k).$$

虽然这一方法简单，但在狭长谷底附近往往会出现明显的“之”字形振荡，收敛较慢。为改进这一点，引入共轭梯度法 (conjugate gradient method)。

4.3.3.2 Fletcher-Reeves 非线性共轭梯度法

Fletcher-Reeves 算法是非线性共轭梯度法 (nonlinear conjugate gradient) 的经典形式。其迭代步骤如下：

$$\mathbf{p}_0 = -\nabla f(\mathbf{x}_0), \quad k = 0.$$

当

$$\|\nabla f(\mathbf{x}_k)\| > \varepsilon$$

时，进行如下迭代：

1. 沿方向 $\mathbf{p}_k$ 对 f 做一维线搜索，求得步长 α_k ；
2. 更新

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k;$$

3. 计算

$$\beta_{k+1}^{\text{FR}} = \frac{\|\nabla f(\mathbf{x}_{k+1})\|^2}{\|\nabla f(\mathbf{x}_k)\|^2};$$

4. 构造新搜索方向

$$\mathbf{p}_{k+1} = -\nabla f(\mathbf{x}_{k+1}) + \beta_{k+1}^{\text{FR}} \mathbf{p}_k;$$

5. 令 $k \leftarrow k + 1$ ，继续迭代。

因此，Fletcher-Reeves 方法并不是完全丢弃旧方向信息，而是把新的负梯度方向与旧方向线性组合，从而形成“共轭”的下降方向。这样做的几何意义是：希望新的方向既能下降，又能避免重复沿旧方向做无效修正。

重启 (restart) 在实际计算中，常每经过 N 次搜索 (N 为变量维数) 后，将方向重新置为最速下降方向：

$$\mathbf{p}_k = -\nabla f(\mathbf{x}_k).$$

这样做的原因是：非线性情形下严格的共轭关系一般不能永久保持，周期性重启可以丢弃过时的方向信息，提高稳定性。

4.3.3.3 线性共轭梯度法回顾

为理解非线性共轭梯度法，先回顾线性共轭梯度法。设

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{b}^T \mathbf{x} + c,$$

其中 A 为对称正定矩阵 (symmetric positive definite matrix)。此时

$$\nabla f(\mathbf{x}) = A \mathbf{x} - \mathbf{b}.$$

极小化该二次函数等价于求解线性方程组

$$Ax = b.$$

一种常见记号是令残差

$$r_k = Ax_k - b = \nabla f(x_k), \quad p_0 = -r_0.$$

算法为：

1. 初始化

$$r_0 = Ax_0 - b, \quad p_0 = -r_0, \quad k = 0;$$

2. 当 $\|r_k\| > \varepsilon$ 时：

$$\alpha_k = -\frac{r_k^\top p_k}{p_k^\top A p_k},$$

$$x_{k+1} = x_k + \alpha_k p_k,$$

$$r_{k+1} = Ax_{k+1} - b = r_k + \alpha_k A p_k,$$

$$\beta_{k+1} = \frac{r_{k+1}^\top A p_k}{p_k^\top A p_k},$$

$$p_{k+1} = -r_{k+1} + \beta_{k+1} p_k, \quad k \leftarrow k + 1.$$

在线性二次问题上，这一方法具有非常漂亮的理论结构：不同搜索方向彼此 A -共轭（ A -conjugate），在精确算术下至多经过 N 步即可获得精确解。

4.3.3.4 另一套等价记号

也常把残差定义为

$$r^{(0)} = b - Ax^{(0)}, \quad v^{(1)} = r^{(0)}.$$

于是迭代可写为

$$x^{(k)} = x^{(k-1)} + t_k v^{(k)},$$

$$r^{(k)} = r^{(k-1)} - t_k A v^{(k)},$$

$$v^{(k+1)} = r^{(k)} + s_k v^{(k)}.$$

其中

$$t_k = \frac{\langle v^{(k)}, r^{(k-1)} \rangle}{\langle v^{(k)}, A v^{(k)} \rangle}.$$

由于共轭梯度法中的正交关系，可进一步化为

$$t_k = \frac{\langle r^{(k-1)}, r^{(k-1)} \rangle}{\langle v^{(k)}, A v^{(k)} \rangle}.$$

又有

$$s_k = -\frac{\langle v^{(k)}, A r^{(k)} \rangle}{\langle v^{(k)}, A v^{(k)} \rangle} = \frac{\langle r^{(k)}, r^{(k)} \rangle}{\langle r^{(k-1)}, r^{(k-1)} \rangle}.$$

这与 Fletcher-Reeves 公式在二次型上的形式完全一致。也就是说，非线性共轭梯度法可视为线性共轭梯度法在一般非线性极小化问题中的推广。

4.3.4 拟牛顿法优化

4.3.4.1 一维牛顿法与极值问题

在一维情形，若要求函数 $f(x)$ 的极值，首先要求驻点：

$$f'(x) = 0.$$

于是可以把极值问题转化为求解非线性方程

$$g(x) = f'(x) = 0.$$

对 $g(x)$ 应用牛顿法得

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

这就是一维优化中的牛顿法。

从几何上看，它是在当前点用二阶信息修正一阶导数的零点位置。若 x^* 是局部极小点，则希望其附近满足

$$f''(x^*) > 0.$$

若直接把这一公式用作“求最小值算法”，则要求迭代过程中二阶导数不至于改变符号，否则方向可能不是下降方向。因此它对初值较为敏感，需要较好的初始值才能保证收敛。

若要求极大值，通常不直接对原函数使用“最小化型牛顿法”，而是改为最小化 $-f(x)$ 。因此在实际优化语境中，牛顿法最自然的应用场景是局部极小问题。

4.3.4.2 多维牛顿法与 Hessian 线性系统

多维情形下，在当前点 $\mathbf{x}_k$ 附近作二阶近似：

$$f(\mathbf{x}_k + \Delta \mathbf{x}) \approx f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \Delta \mathbf{x} + \frac{1}{2} \Delta \mathbf{x}^\top \nabla^2 f(\mathbf{x}_k) \Delta \mathbf{x}.$$

若将右边视为关于 $\Delta \mathbf{x}$ 的二次函数，其极小点满足

$$\nabla^2 f(\mathbf{x}_k) \Delta \mathbf{x} = -\nabla f(\mathbf{x}_k).$$

设

$$\mathbf{d}^{(k)} = \Delta \mathbf{x}_k = \mathbf{x}_{k+1} - \mathbf{x}_k,$$

则牛顿步由线性方程组

$$\nabla^2 f(\mathbf{x}^{(k)}) \mathbf{d}^{(k)} = -\nabla f(\mathbf{x}^{(k)})$$

确定。

这说明：多维牛顿法的本质，是在每一步求解一个由 Hessian 给出的线性方程组。

然而，精确计算 Hessian 及其逆矩阵往往代价很高，尤其在高维问题中更是如此。因此引入 Hessian 的近似矩阵 B_k ，便得到拟牛顿法 (quasi-Newton method)。

4.3.4.3 拟牛顿法的基本思想

把

$$\nabla^2 f(\mathbf{x}_k)$$

用某个对称矩阵

$$B_k$$

近似，则二次模型写成

$$f(\mathbf{x}_k + \Delta \mathbf{x}) \approx f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top \Delta \mathbf{x} + \frac{1}{2} \Delta \mathbf{x}^\top B_k \Delta \mathbf{x}.$$

令该二次模型对 $\Delta \mathbf{x}$ 的梯度为零，得到

$$B_k \Delta \mathbf{x} = -\nabla f(\mathbf{x}_k),$$

从而

$$\Delta \mathbf{x} = -B_k^{-1} \nabla f(\mathbf{x}_k).$$

于是可做迭代

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}.$$

为增强算法稳定性，通常再加入步长因子 α_k ：

$$\Delta \mathbf{x}_k = -\alpha_k B_k^{-1} \nabla f(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_k.$$

这里 α_k 往往由单独的线搜索确定，而不必固定为 1。

4.3.4.4 割线条件 (secant equation)

为了更新 B_k ，考虑梯度的 Taylor 展开：

$$\nabla f(\mathbf{x}_k + \Delta \mathbf{x}_k) \approx \nabla f(\mathbf{x}_k) + B_{k+1} \Delta \mathbf{x}_k.$$

记

$$\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k), \quad \mathbf{s}_k = \Delta \mathbf{x}_k = \mathbf{x}_{k+1} - \mathbf{x}_k,$$

则上式可写成

$$\mathbf{y}_k \approx B_{k+1} \mathbf{s}_k.$$

在拟牛顿法中，把这一近似强制设为等式：

$$B_{k+1} \mathbf{s}_k = \mathbf{y}_k$$

这就是割线条件 (secant equation)。它是一维割线法

$$f'(x) \approx \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

在多维优化中的矩阵版本。

若定义

$$H_k = B_k^{-1},$$

则同样可在逆矩阵层面进行更新。

4.3.5 拟牛顿法的迭代过程与常见更新公式

4.3.5.1 一般框架

拟牛顿法的标准框架可写为：

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_k,$$

$$\Delta \mathbf{x}_k = -\alpha_k B_k^{-1} \nabla f(\mathbf{x}_k),$$

$$\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k),$$

$$H_{k+1} = B_{k+1}^{-1}.$$

关键问题便转化为：如何根据 $\mathbf{s}_k = \Delta \mathbf{x}_k$ 与 $\mathbf{y}_k$ ，从 B_k 或 H_k 构造出 B_{k+1} 或 H_{k+1} 。

4.3.5.2 DFP 方法

DFP 方法由 Davidon–Fletcher–Powell 提出，是最早成功的拟牛顿法之一。其逆 Hessian 更新公式最常见：

$$H_{k+1} = H_k + \frac{s_k s_k^T}{s_k^T y_k} - \frac{H_k y_k y_k^T H_k}{y_k^T H_k y_k}$$

与之对偶的 Hessian 近似更新可写为

$$B_{k+1} = \left(I - \frac{y_k s_k^T}{y_k^T s_k} \right) B_k \left(I - \frac{s_k y_k^T}{y_k^T s_k} \right) + \frac{y_k y_k^T}{y_k^T s_k}.$$

4.3.5.3 BFGS 方法

BFGS 方法由 Broyden, Fletcher, Goldfarb, Shanno 分别提出，是实际应用中最成功的拟牛顿法之一。其 Hessian 近似更新公式为

$$B_{k+1} = B_k + \frac{y_k y_k^T}{y_k^T s_k} - \frac{B_k s_k (B_k s_k)^T}{s_k^T B_k s_k}$$

而其逆矩阵更新公式为

$$H_{k+1} = \left(I - \frac{s_k y_k^T}{y_k^T s_k} \right) H_k \left(I - \frac{y_k s_k^T}{y_k^T s_k} \right) + \frac{s_k s_k^T}{y_k^T s_k}.$$

DFP 与 BFGS 的对偶性二者是彼此的对偶 (dual) 方法：若在 H_k 空间使用 DFP 更新，则对应的 B_k 更新正是 BFGS 形式；反之亦然。因此它们常被放在一起比较。

正定性保持若初始矩阵 B_0 对称正定，且满足曲率条件

$$y_k^T s_k > 0,$$

则 BFGS 与 DFP 都能保持矩阵的对称正定性。这一点非常重要，因为它保证了

$$\Delta x_k = -H_k \nabla f(x_k)$$

是一个下降方向。实际中，Wolfe 条件线搜索常用于保证这一曲率条件成立。

4.3.5.4 Broyden 更新

Broyden 型更新只要求满足割线条件

$$B_{k+1} s_k = y_k,$$

并通过秩一修正得到新矩阵。其 Hessian 近似更新形式可写成

$$B_{k+1} = B_k + \frac{(y_k - B_k s_k) s_k^T}{s_k^T s_k}.$$

对应的逆矩阵更新为

$$H_{k+1} = H_k + \frac{(s_k - H_k y_k) s_k^T H_k}{s_k^T H_k y_k}.$$

这类更新与前一节非线性方程组中的 Broyden 方法在形式上是完全同源的：都是用最小秩修正来强制满足新的割线信息。

不过，与 BFGS、DFP 相比，Broyden 更新一般不自动保持对称性和正定性，因此在纯优化问题中通常不如 BFGS 常用。

4.3.5.5 Broyden family

把 BFGS 与 DFP 作凸组合，可得到 Broyden family：

$$B_{k+1} = (1 - \varphi_k)B_{k+1}^{\text{BFGS}} + \varphi_k B_{k+1}^{\text{DFP}}, \quad \varphi_k \in [0, 1].$$

这给出了一整个连续的一参数拟牛顿更新族。不同的 φ_k 对应不同的更新策略；其中两个端点分别退化为 BFGS 与 DFP。

4.3.5.6 SR1 更新

SR1 表示 Symmetric Rank-One update，即对称秩一更新。其 Hessian 更新公式为

$$B_{k+1} = B_k + \frac{(\mathbf{y}_k - B_k \mathbf{s}_k)(\mathbf{y}_k - B_k \mathbf{s}_k)^\top}{(\mathbf{y}_k - B_k \mathbf{s}_k)^\top \mathbf{s}_k}.$$

对应的逆矩阵更新为

$$H_{k+1} = H_k + \frac{(\mathbf{s}_k - H_k \mathbf{y}_k)(\mathbf{s}_k - H_k \mathbf{y}_k)^\top}{(\mathbf{s}_k - H_k \mathbf{y}_k)^\top \mathbf{y}_k}.$$

SR1 的优点是形式简单，并且在一定程度上更容易逼近真实 Hessian 的曲率特征，特别是在 Hessian 不一定正定的情形下较有价值。但其缺点也很明显：更新分母可能非常小，且它不保证正定性，因此在“求最小值”的稳定性方面要格外小心。

4.3.5.7 几种拟牛顿法的比较

- **DFP**：历史上很重要，理论结构优美，但实践中常被 BFGS 取代；
- **BFGS**：目前最常用、最稳健的拟牛顿方法之一；
- **Broyden**：更偏向一般非线性方程求解的割线思想，在优化中不是主流；
- **Broyden family**：提供从 DFP 到 BFGS 的连续插值；
- **SR1**：能更灵活地逼近曲率，但自动保证正定性。

在现代数值优化软件中，如果问题是中等规模、可求梯度且目标函数较光滑，那么 BFGS 往往是最优先考虑的方法之一；如果变量维数特别大，则常进一步使用其有限内存版本 L-BFGS (limited-memory BFGS)。

本节小结

本节围绕优化问题的几个核心层次展开：

1. 通过梯度和 Hessian 建立局部极值判据，理解驻点、正定性与二阶 Taylor 展开的关系；
2. 从全局优化角度了解梯度法、无导数法、模拟退火、遗传算法、粒子群等典型思路；
3. 通过线性共轭梯度法与 Fletcher-Reeves 算法，理解如何利用历史方向信息加速下降；
4. 通过拟牛顿法理解：即使不显式计算真实 Hessian，也可以利用割线条件逐步构造曲率近似；
5. 掌握 DFP、BFGS、Broyden、Broyden family 与 SR1 等典型更新公式及其相互关系。

从整体上看，优化算法的发展主线可以概括为：既要利用足够多的局部信息刻画函数曲率，又要尽量降低每一步的计算代价。共轭梯度法与拟牛顿法正是在这一思想下形成的两条最重要路线。

4.3.6 BFGS 算法

BFGS 算法的全称是 Broyden-Fletcher-Goldfarb-Shanno algorithm，它是拟牛顿法 (quasi-Newton method) 中最经典、也最常用的一类算法。其核心思想是：不直接计算真实 Hessian 矩阵 $\nabla^2 f(\mathbf{x})$ ，而是构造一个对称正定矩阵 B_k 来逐步逼近它，再用该近似矩阵产生下降方向。

4.3.6.1 基本设定

考虑无约束极小化问题

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}),$$

设

$$\mathbf{g}_k = \nabla f(\mathbf{x}_k).$$

在第 k 步, BFGS 用一个对称矩阵 B_k 近似 Hessian, 并通过求解线性方程组

$$B_k \mathbf{p}_k = -\mathbf{g}_k$$

得到搜索方向 $\mathbf{p}_k$ 。若记

$$H_k = B_k^{-1},$$

则也可写成

$$\mathbf{p}_k = -H_k \mathbf{g}_k.$$

然后沿方向 $\mathbf{p}_k$ 做一维搜索 (line search), 求步长 α_k :

$$\alpha_k = \arg \min_{\alpha > 0} f(\mathbf{x}_k + \alpha \mathbf{p}_k).$$

定义步长向量

$$\mathbf{s}_k = \alpha_k \mathbf{p}_k = \mathbf{x}_{k+1} - \mathbf{x}_k,$$

以及梯度变化量

$$\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k).$$

于是拟牛顿法要求新矩阵满足割线条件 (secant equation)

$$B_{k+1} \mathbf{s}_k = \mathbf{y}_k.$$

4.3.6.2 BFGS 更新公式的推导

BFGS 的出发点是: 在保持对称性的同时, 以尽可能简单的方式修正 B_k 。设修正形式为

$$B_{k+1} = B_k + U_k + V_k,$$

并进一步设

$$B_{k+1} = B_k + \alpha \mathbf{u} \mathbf{u}^\top + \beta \mathbf{v} \mathbf{v}^\top,$$

其中取

$$\mathbf{u} = \mathbf{y}_k, \quad \mathbf{v} = B_k \mathbf{s}_k.$$

因为 B_k 为对称矩阵, 所以 $B_k^\top = B_k$ 。

把上述表达式代入割线条件:

$$B_{k+1} \mathbf{s}_k = B_k \mathbf{s}_k + \alpha \mathbf{y}_k (\mathbf{y}_k^\top \mathbf{s}_k) + \beta (B_k \mathbf{s}_k) (\mathbf{s}_k^\top B_k \mathbf{s}_k) = \mathbf{y}_k.$$

把 $B_k \mathbf{s}_k$ 看成一个向量, 比较两边系数, 可取

$$\alpha = \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k}, \quad \beta = -\frac{1}{\mathbf{s}_k^\top B_k \mathbf{s}_k}.$$

从而得到 BFGS 的 Hessian 近似更新公式

$$B_{k+1} = B_k + \frac{\mathbf{y}_k \mathbf{y}_k^\top}{\mathbf{y}_k^\top \mathbf{s}_k} - \frac{B_k \mathbf{s}_k \mathbf{s}_k^\top B_k}{\mathbf{s}_k^\top B_k \mathbf{s}_k}.$$

这一定义自动满足:

1. 对称性：若 B_k 对称，则 B_{k+1} 仍对称；
2. 割线条件： $B_{k+1}s_k = y_k$ ；
3. 在合适线搜索下，若 B_k 正定，则 B_{k+1} 仍保持正定。

4.3.6.3 逆矩阵形式

实际计算中，更常直接更新逆矩阵

$$H_k = B_k^{-1}.$$

BFGS 的逆矩阵更新公式可写成紧凑形式

$$H_{k+1} = \left(I - \rho_k s_k y_k^T \right) H_k \left(I - \rho_k y_k s_k^T \right) + \rho_k s_k s_k^T, \quad \rho_k = \frac{1}{y_k^T s_k}.$$

把它展开后得到

$$H_{k+1} = H_k + \left(1 + \frac{y_k^T H_k y_k}{s_k^T y_k} \right) \frac{s_k s_k^T}{s_k^T y_k} - \frac{H_k y_k s_k^T + s_k y_k^T H_k}{s_k^T y_k}.$$

这就是 BFGS 常用的逆矩阵更新公式。

BFGS 正定性的关键条件 BFGS 成功的一个重要原因是它能较好地保持正定性。若

$$B_k \succ 0, \quad s_k^T y_k > 0,$$

则通常有

$$B_{k+1} \succ 0, \quad H_{k+1} \succ 0.$$

其中

$$s_k^T y_k > 0$$

称为曲率条件 (curvature condition)。在实践中常通过 Wolfe 条件线搜索来保证这一点。

4.3.6.4 BFGS 的标准算法步骤

给定初值 x_0 以及初始近似矩阵 B_0 。最常见的取法是

$$B_0 = I,$$

即用单位矩阵作为初始 Hessian 近似。

BFGS 迭代过程如下：

1. 计算当前梯度

$$g_k = \nabla f(x_k).$$

2. 解线性方程组

$$B_k p_k = -g_k$$

得到搜索方向 p_k 。

3. 通过一维线搜索求步长

$$\alpha_k = \arg \min_{\alpha > 0} f(x_k + \alpha p_k).$$

4. 令

$$s_k = \alpha_k p_k, \quad x_{k+1} = x_k + s_k.$$

5. 计算梯度差

$$y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

6. 用 BFGS 公式更新 B_k 或 H_k , 进入下一步。

因此, BFGS 很适合优化无约束、光滑 (smooth) 的目标函数。

4.3.6.5 二次函数上的 BFGS 例子

考虑用 BFGS 最小化

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T Q \mathbf{x} - \mathbf{x}^T \mathbf{b} + \log(\pi),$$

其中

$$Q = \begin{bmatrix} 5 & -3 \\ -3 & 2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

常数项 $\log(\pi)$ 对极小点没有影响, 因此梯度为

$$\mathbf{g}(\mathbf{x}) = \nabla f(\mathbf{x}) = Q\mathbf{x} - \mathbf{b}.$$

取初值

$$H_0 = I_2, \quad \mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

第一步先计算初始梯度

$$\mathbf{g}^{(0)} = Q\mathbf{x}^{(0)} - \mathbf{b} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

下降方向取为

$$\mathbf{d}^{(0)} = -H_0 \mathbf{g}^{(0)} = -\begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

对二次函数沿该方向做精确线搜索。令

$$\phi(\alpha) = f(\mathbf{x}^{(0)} + \alpha \mathbf{d}^{(0)}).$$

则

$$\phi'(\alpha) = \mathbf{g}^{(0)T} \mathbf{d}^{(0)} + \alpha \mathbf{d}^{(0)T} Q \mathbf{d}^{(0)}.$$

令 $\phi'(\alpha_0) = 0$, 得

$$\alpha_0 = -\frac{\mathbf{g}^{(0)T} \mathbf{d}^{(0)}}{\mathbf{d}^{(0)T} Q \mathbf{d}^{(0)}}.$$

代入

$$\mathbf{g}^{(0)T} \mathbf{d}^{(0)} = -1, \quad \mathbf{d}^{(0)T} Q \mathbf{d}^{(0)} = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} 5 & -3 \\ -3 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 2,$$

故

$$\alpha_0 = \frac{1}{2}.$$

于是

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha_0 \mathbf{d}^{(0)} = \begin{bmatrix} 0 \\ 1/2 \end{bmatrix}.$$

从而

$$\Delta \mathbf{x}^{(0)} = \mathbf{x}^{(1)} - \mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 1/2 \end{bmatrix}.$$

再算新梯度

$$\mathbf{g}^{(1)} = Q\mathbf{x}^{(1)} - \mathbf{b} = \begin{bmatrix} 5 & -3 \\ -3 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1/2 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -3/2 \\ 0 \end{bmatrix}.$$

因此

$$\Delta \mathbf{g}^{(0)} = \mathbf{g}^{(1)} - \mathbf{g}^{(0)} = \begin{bmatrix} -3/2 \\ 1 \end{bmatrix}.$$

也就是说，在 BFGS 记号中

$$\mathbf{s}_0 = \Delta \mathbf{x}^{(0)}, \quad \mathbf{y}_0 = \Delta \mathbf{g}^{(0)}.$$

把这些量代入逆矩阵更新公式：

$$H_1 = H_0 + \left(1 + \frac{\Delta \mathbf{g}^{(0)\top} H_0 \Delta \mathbf{g}^{(0)}}{\Delta \mathbf{g}^{(0)\top} \Delta \mathbf{x}^{(0)}} \right) \frac{\Delta \mathbf{x}^{(0)} \Delta \mathbf{x}^{(0)\top}}{\Delta \mathbf{x}^{(0)\top} \Delta \mathbf{g}^{(0)}} - \frac{\Delta \mathbf{x}^{(0)} \Delta \mathbf{g}^{(0)\top} H_0 + H_0 \Delta \mathbf{g}^{(0)} \Delta \mathbf{x}^{(0)\top}}{\Delta \mathbf{x}^{(0)\top} \Delta \mathbf{g}^{(0)}}.$$

由于 $H_0 = I_2$ ，可直接算得

$$H_1 = \begin{bmatrix} 1 & 3/2 \\ 3/2 & 11/4 \end{bmatrix}.$$

第二步新的下降方向为

$$\mathbf{d}^{(1)} = -H_1 \mathbf{g}^{(1)} = - \begin{bmatrix} 1 & 3/2 \\ 3/2 & 11/4 \end{bmatrix} \begin{bmatrix} -3/2 \\ 0 \end{bmatrix} = \begin{bmatrix} 3/2 \\ 9/4 \end{bmatrix}.$$

再次作精确线搜索：

$$\alpha_1 = - \frac{\mathbf{g}^{(1)\top} \mathbf{d}^{(1)}}{\mathbf{d}^{(1)\top} Q \mathbf{d}^{(1)}}.$$

直接代入可得

$$\alpha_1 = 2.$$

于是

$$\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + \alpha_1 \mathbf{d}^{(1)} = \begin{bmatrix} 0 \\ 1/2 \end{bmatrix} + 2 \begin{bmatrix} 3/2 \\ 9/4 \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}.$$

此时梯度

$$\mathbf{g}^{(2)} = Q\mathbf{x}^{(2)} - \mathbf{b} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

因此极小点已经求得：

$$\boxed{\mathbf{x}^* = \begin{bmatrix} 3 \\ 5 \end{bmatrix}}.$$

例子的意义这个例子说明：对于二维严格凸二次函数，在精确线搜索下，BFGS 往往能在极少步数内恢复出准确的曲率信息。本题中仅经过两步就到达极小点，这与二次函数的特殊结构密切相关。

4.3.7 Broyden 优化

4.3.7.1 从非线性方程到非线性优化

在非线性方程组

$$\mathbf{f}(\mathbf{x}) = \mathbf{0}$$

的求解中，Broyden 方法用 Jacobian 矩阵 J_n 或其逆矩阵的低秩修正来近似牛顿法。其基本形式为

$$J_n = J_{n-1} + \frac{\Delta f_n - J_{n-1} \Delta x_n}{\|\Delta x_n\|_2^2} \Delta x_n^\top,$$

$$J_n^{-1} = J_{n-1}^{-1} + \frac{\Delta x_n - J_{n-1}^{-1} \Delta f_n}{\Delta x_n^\top J_{n-1}^{-1} \Delta f_n} \Delta x_n^\top J_{n-1}^{-1},$$

并配合

$$x_{n+1} = x_n - J_n^{-1} f(x_n)$$

做迭代。

对优化问题来说，若希望极小化 $f(x)$ ，则临界点满足

$$\nabla f(x) = 0.$$

因此只要把非线性方程中的

$$f(x)$$

替换成

$$\nabla f(x),$$

再把 Jacobian 替换成 Hessian，便得到 Broyden 型优化方法。

4.3.7.2 Broyden 型 Hessian 更新

设

$$s_k = \Delta x_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

若 B_k 是 Hessian 的近似，则最直接的 Broyden 型更新为

$$B_{k+1} = B_k + \frac{y_k - B_k s_k}{s_k^\top s_k} s_k^\top.$$

对应的逆矩阵更新为

$$H_{k+1} = B_{k+1}^{-1} = H_k + \frac{(s_k - H_k y_k) s_k^\top H_k}{s_k^\top H_k y_k}.$$

然后迭代格式写成

$$x_{k+1} = x_k - H_k \nabla f(x_k).$$

4.3.7.3 与 BFGS 的比较

Broyden 型更新与 BFGS 有明显相似之处：它们都满足割线条件，都是利用

$$(s_k, y_k)$$

这对信息来修正曲率矩阵。

但两者也有重要区别：

- BFGS 是秩二更新 (rank-two update)，并精心设计以保持对称正定性；
- Broyden 优化更新更接近于一般非线性方程求解中的 Broyden 思想，通常是秩一修正 (rank-one update)；
- 在纯优化问题中，BFGS 往往比简单的 Broyden 更新更稳定，也更常用。

术语提醒在线性规划中的 *simplex method* 与下面的 Nelder–Mead *simplex* 都含有“simplex”一词，但它们的含义完全不同：

- 线性规划中的 simplex 是表上迭代法；
- Nelder–Mead 中的 simplex 是几何上的 n 维单纯形。

4.3.8 线性规划与单纯形法

4.3.8.1 线性规划的标准形式

实际科学与工程问题中，经常会遇到线性规划（linear programming）问题。其典型形式是

$$\max \mathbf{c}^T \mathbf{x}, \quad \text{s.t. } \mathbf{Ax} \leq \mathbf{b}, \quad x_i \geq 0.$$

其中

$$\mathbf{x} = (x_1, \dots, x_n)^T.$$

为了应用单纯形法，需要把不等式约束变成等式约束。对于“ $\leq$ ”型约束，引入松弛变量（slack variables）：

$$\mathbf{Ax} \leq \mathbf{b} \implies \mathbf{Ax} + \mathbf{s} = \mathbf{b}, \quad \mathbf{s} \geq 0.$$

于是问题转化为标准形式

$$\mathbf{Ax} = \mathbf{b}, \quad \mathbf{x} \geq 0.$$

更一般地，若有“ $\geq$ ”约束，则需要引入剩余变量（surplus variable）；若不能直接构造初始基，还需引入人工变量（artificial variable）。本节主要讨论只含“ $\leq$ ”约束的标准情形。

把变量分成基变量（basic variables）与非基变量（nonbasic variables）后，矩阵也可写成

$$\mathbf{A} = [\mathbf{A}_B \mid \mathbf{A}_N].$$

其中 $\mathbf{A}_B$ 对应基变量列， $\mathbf{A}_N$ 对应非基变量列。

4.3.8.2 单纯形法的基本思想

单纯形法（simplex method）的核心思想是：

- 先从一个可行基解（basic feasible solution）出发；
- 每一步通过枢轴变换（pivoting）在相邻的可行基解之间移动；
- 每次迭代都使目标函数值不下降；
- 当目标行（bottom row）中已没有负的检验数时，达到最优解。

标准单纯形法可概括为以下步骤：

1. 把每个不等式约束通过加入松弛变量转为等式；
2. 构造初始单纯形表（simplex tableau）；
3. 在最后一行中找最负的元素，其所在列称为入基列（entering column）；
4. 对入基列中所有正元素，计算比值

$$\frac{b_i}{a_{ij}},$$

取最小非负比值对应的行为出基行（departing row）；

5. 入基列与出基行交点称为主元（pivot），通过初等行变换把主元化为 1，并把同列其他元素化为 0；
6. 若最后一行所有元素都非负，则得到最终表；否则返回第 3 步；
7. 最终表右下角给出目标函数最优值。

无界情形若所选入基列中所有元素都非正，则无法找到合法的出基行。这意味着目标函数可以沿该方向无限增大，故问题没有最大值。

4.3.8.3 例：用单纯形法求线性规划极大值

考虑问题

$$\max z = 4x_1 + 6x_2$$

满足

$$-x_1 + x_2 \leq 11, \quad x_1 + x_2 \leq 27, \quad 2x_1 + 5x_2 \leq 90, \quad x_1 \geq 0, x_2 \geq 0.$$

引入松弛变量 $s_1, s_2, s_3 \geq 0$, 得

$$-x_1 + x_2 + s_1 = 11,$$

$$x_1 + x_2 + s_2 = 27,$$

$$2x_1 + 5x_2 + s_3 = 90.$$

目标函数写成

$$z - 4x_1 - 6x_2 = 0.$$

于是初始单纯形表为

$$\left[\begin{array}{cccccc|c} -1 & 1 & 1 & 0 & 0 & 11 \\ 1 & 1 & 0 & 1 & 0 & 27 \\ 2 & 5 & 0 & 0 & 1 & 90 \\ -4 & -6 & 0 & 0 & 0 & 0 \end{array} \right],$$

列依次对应

$$x_1, x_2, s_1, s_2, s_3 \mid b.$$

此时基变量是

$$s_1, s_2, s_3,$$

非基变量是

$$x_1, x_2.$$

故初始基本可行解为

$$(x_1, x_2, s_1, s_2, s_3) = (0, 0, 11, 27, 90).$$

第一次 pivot 底行最负元素是 -6 , 所以选 x_2 入基。对 x_2 列做最小比值检验:

$$\frac{11}{1} = 11, \quad \frac{27}{1} = 27, \quad \frac{90}{5} = 18.$$

最小的是 11, 因此 s_1 出基, 主元是第一行第一列中的 1 (这里是 x_2 列、第一行)。

做行变换:

$$R_2 \leftarrow R_2 - R_1, \quad R_3 \leftarrow R_3 - 5R_1, \quad R_4 \leftarrow R_4 + 6R_1.$$

得到

$$\left[\begin{array}{cccccc|c} -1 & 1 & 1 & 0 & 0 & 11 \\ 2 & 0 & -1 & 1 & 0 & 16 \\ 7 & 0 & -5 & 0 & 1 & 35 \\ -10 & 0 & 6 & 0 & 0 & 66 \end{array} \right].$$

此时基变量变为

$$x_2, s_2, s_3,$$

基本可行解为

$$(x_1, x_2, s_1, s_2, s_3) = (0, 11, 0, 16, 35).$$

第二次 pivot 底行最负元素是 -10 , 故选 x_1 入基。看 x_1 列, 只有正元素能参与比值检验:

$$\frac{16}{2} = 8, \quad \frac{35}{7} = 5.$$

第一行中 x_1 系数为 -1 ，不能参与。因此最小非负比值是 5 ，对应第三行，所以 s_3 出基。

先把第三行除以 7 ，使主元变为 1 ：

$$R_3 \leftarrow \frac{1}{7}R_3.$$

即

$$R_3 = \left[1, 0, -\frac{5}{7}, 0, \frac{1}{7} \mid 5 \right].$$

然后消去其余各行中 x_1 列元素：

$$R_1 \leftarrow R_1 + R_3, \quad R_2 \leftarrow R_2 - 2R_3, \quad R_4 \leftarrow R_4 + 10R_3.$$

得到

$$\left[\begin{array}{ccccc|c} 0 & 1 & 2/7 & 0 & 1/7 & 16 \\ 0 & 0 & 3/7 & 1 & -2/7 & 6 \\ 1 & 0 & -5/7 & 0 & 1/7 & 5 \\ 0 & 0 & -8/7 & 0 & 10/7 & 116 \end{array} \right].$$

现在基变量是

$$x_2, s_2, x_1,$$

对应的基本可行解为

$$(x_1, x_2, s_1, s_2, s_3) = (5, 16, 0, 6, 0).$$

第三次 pivot 底行仍有负元素 $-8/7$ ，故选 s_1 入基。虽然 s_1 是松弛变量，但在线性规划表迭代中，任何变量都可以重新入基。

对 s_1 列作最小比值检验：

$$\frac{16}{2/7} = 56, \quad \frac{6}{3/7} = 14.$$

第三行对应系数 $-5/7 < 0$ ，不参与。故最小比值为 14 ，第二行出基，也即 s_2 出基。

先将第二行乘以 $7/3$ ，把主元化为 1 ：

$$R_2 \leftarrow \frac{7}{3}R_2,$$

得到

$$R_2 = [0, 0, 1, 7/3, -2/3 \mid 14].$$

再消去其他行中 s_1 列元素：

$$R_1 \leftarrow R_1 - \frac{2}{7}R_2, \quad R_3 \leftarrow R_3 + \frac{5}{7}R_2, \quad R_4 \leftarrow R_4 + \frac{8}{7}R_2.$$

最终得到

$$\left[\begin{array}{ccccc|c} 0 & 1 & 0 & -2/3 & 1/3 & 12 \\ 0 & 0 & 1 & 7/3 & -2/3 & 14 \\ 1 & 0 & 0 & 5/3 & -1/3 & 15 \\ 0 & 0 & 0 & 8/3 & 2/3 & 132 \end{array} \right].$$

此时底行全部非负，因此达到最优。

最终基变量为

$$x_2, s_1, x_1,$$

非基变量为

$$s_2, s_3.$$

故最优解为

$$(x_1, x_2, s_1, s_2, s_3) = (15, 12, 14, 0, 0)$$

从而

$$x_1 = 15, \quad x_2 = 12,$$

目标函数最大值为

$$z_{\max} = 4x_1 + 6x_2 = 4 \cdot 15 + 6 \cdot 12 = 132.$$

几何解释单纯形法并不是在整个可行域内部乱跳，而是在可行多面体的顶点之间移动。对于线性目标函数，若最优解存在，则一定可以在某个顶点处取得。因此“顶点到顶点”的搜索就足够了。

4.3.9 Nelder–Mead 方法（下山单纯形法）

4.3.9.1 几何上的 simplex

Nelder–Mead method 又称下山单纯形法（downhill simplex method）。这里的 simplex 指的是几何意义上的 n 维单纯形：

- 一维时是线段；
- 二维时是三角形；
- 三维时是四面体；
- 一般 n 维时由 $n+1$ 个顶点组成。

该方法与前面的线性规划单纯形法没有直接关系。它是一种无导数优化方法（gradient-free optimization），只需要函数值，不需要梯度或 Hessian。

4.3.9.2 算法基本思想

设当前单纯形的顶点为

$$\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{n+1},$$

并按函数值大小排序：

$$f(\mathbf{x}_1) \leq f(\mathbf{x}_2) \leq \dots \leq f(\mathbf{x}_{n+1}).$$

其中：

- $\mathbf{x}_1$ 是当前最好点（best point）；
- $\mathbf{x}_{n+1}$ 是当前最差点（worst point）。

接着计算除去最差点外其余 n 个点的中心

$$\mathbf{x}_o = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i.$$

然后围绕该中心对最差点做几种几何操作：反射（reflection）、膨胀（expansion）、收缩（contraction）、缩小（shrink）。

4.3.9.3 反射操作

首先做反射：

$$\mathbf{x}_r = \mathbf{x}_o + \alpha(\mathbf{x}_o - \mathbf{x}_{n+1}).$$

这里 $\alpha > 0$ 为反射参数，通常取

$$\alpha = 1.$$

几何上， $\mathbf{x}_r$ 是把最差点关于重心 $\mathbf{x}_o$ 对称地“翻”到另一侧所得到的点。

若反射点满足

$$f(\mathbf{x}_1) \leq f(\mathbf{x}_r) < f(\mathbf{x}_n),$$

则说明反射后效果有所改善但还不算特别好，这时就用 $\mathbf{x}_r$ 替换最差点 $\mathbf{x}_{n+1}$ ，形成新单纯形，然后返回排序步骤。

4.3.9.4 膨胀操作

若反射点表现得比当前最好点还好，即

$$f(\mathbf{x}_r) < f(\mathbf{x}_1),$$

则说明在这个方向上值得继续“冲出去”，于是做膨胀：

$$\mathbf{x}_e = \mathbf{x}_o + \gamma(\mathbf{x}_r - \mathbf{x}_o),$$

其中 $\gamma > 1$ 为膨胀参数，通常取

$$\gamma = 2.$$

若

$$f(\mathbf{x}_e) < f(\mathbf{x}_r),$$

则用膨胀点 $\mathbf{x}_e$ 取代最差点；否则仍用反射点 $\mathbf{x}_r$ 取代最差点。

4.3.9.5 收缩操作

若反射点并不好，说明最差点附近可能已经逼近谷底边缘或不适合继续外跳。这时做收缩：

$$\mathbf{x}_c = \mathbf{x}_o + \rho(\mathbf{x}_{n+1} - \mathbf{x}_o),$$

其中

$$0 < \rho < 1, \quad \rho = \frac{1}{2}$$

是常用选取。

若

$$f(\mathbf{x}_c) < f(\mathbf{x}_{n+1}),$$

则用收缩点 $\mathbf{x}_c$ 替换最差点。

4.3.9.6 缩小操作

如果收缩也失败，说明当前单纯形可能太大、太偏、或者整个方向选择都不理想。这时保留最好点 $\mathbf{x}_1$ ，把其余点都向它缩小：

$$\mathbf{x}_i = \mathbf{x}_1 + \sigma(\mathbf{x}_i - \mathbf{x}_1), \quad i = 2, \dots, n+1.$$

通常取

$$\sigma = \frac{1}{2}.$$

这一步相当于把整个单纯形压缩到当前最好点附近，重新开始局部搜索。

4.3.9.7 Nelder-Mead 的完整流程

综合以上步骤，Nelder-Mead 方法可写为：

1. 对顶点按函数值排序：

$$f(\mathbf{x}_1) \leq \dots \leq f(\mathbf{x}_{n+1}).$$

2. 计算除最差点外的中心

$$\mathbf{x}_o = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i.$$

3. 作反射

$$\mathbf{x}_r = \mathbf{x}_o + \alpha(\mathbf{x}_o - \mathbf{x}_{n+1}).$$

4. 若

$$f(\mathbf{x}_1) \leq f(\mathbf{x}_r) < f(\mathbf{x}_n),$$

则接受反射点。

5. 若

$$f(\mathbf{x}_r) < f(\mathbf{x}_1),$$

则作膨胀

$$\mathbf{x}_e = \mathbf{x}_o + \gamma(\mathbf{x}_r - \mathbf{x}_o),$$

并在 $\mathbf{x}_e, \mathbf{x}_r$ 中选较优者替换最差点。

6. 若反射没有改善, 则作收缩

$$\mathbf{x}_c = \mathbf{x}_o + \rho(\mathbf{x}_{n+1} - \mathbf{x}_o).$$

若收缩点优于最差点, 则接受收缩点。

7. 若收缩也失败, 则对除最好点外的所有点做缩小:

$$\mathbf{x}_i = \mathbf{x}_1 + \sigma(\mathbf{x}_i - \mathbf{x}_1), \quad i = 2, \dots, n+1.$$

常用参数取

$\alpha = 1, \quad \gamma = 2, \quad \rho = \frac{1}{2}, \quad \sigma = \frac{1}{2}.$

4.3.9.8 二维情形的几何图像

在二维情形, 当前 simplex 是一个三角形。设三个顶点按优劣排序为

$$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3,$$

其中 $\mathbf{x}_3$ 最差。去掉最差点后, 最好点与次好点的中点就是

$$\mathbf{x}_o.$$

然后:

- 反射点 $\mathbf{x}_r$ 位于 $\mathbf{x}_3$ 关于 $\mathbf{x}_o$ 的另一侧;
- 若反射很好, 则进一步向外走到膨胀点 $\mathbf{x}_e$;
- 若反射不佳, 则向内收缩到 $\mathbf{x}_c$;
- 若仍不好, 则把整个三角形向最好点 $\mathbf{x}_1$ 缩小。

因此 Nelder-Mead 可以看成: 让一个会变形的几何单纯形在目标函数图形上“往山谷底部滑动”。

方法特点 Nelder-Mead 方法的优点是:

- 不需要梯度;
- 编程实现简单;
- 对低维、小规模、函数值可计算但导数难求的问题很方便。

其缺点是:

- 在高维问题上效率往往较差;
- 理论收敛性不如梯度法和拟牛顿法完善;

- 对参数、初始单纯形形状和尺度比较敏感。

4.3.10 模拟退火法

4.3.10.1 方法背景与基本思想

模拟退火法 (Simulated Annealing, SA) 是一类通用的随机搜索算法 (randomized search algorithm)。它最早可追溯到 Metropolis 等人在 1953 年提出的 Metropolis 过程 (Metropolis process)。这一方法的核心思想,是把优化问题的求解过程与统计热力学中物体的退火过程进行类比:

- 在高温下,粒子具有较强的热运动能力,系统能够在较大的状态空间中自由跳动;
- 随着温度逐渐降低,系统越来越倾向于停留在低能量状态;
- 若降温过程足够缓慢,则系统有希望逐步接近全局最低能量态。

把这一思想移植到优化中:

- 将目标函数值类比为系统能量 (energy);
- 将变量取值或候选解类比为系统状态 (state);
- 将“寻找最优解”类比为“寻找最低能量平衡态”。

因此,模拟退火法尤其适合处理下列问题:

- 存在大量局部极值 (local extrema) 的全局优化 (global optimization) 问题;
- 目标函数不光滑 (nonsmooth);
- 函数带噪声 (noisy objective);
- 无法方便计算导数,因此需要无导数优化 (derivative-free optimization)。

与传统梯度法相比,模拟退火法最大的特点在于:它允许以一定概率接受“变差”的一步,从而有机会跳出局部极小,继续向更优区域搜索。

4.3.10.2 热力学类比

从热力学角度看,设系统处于某一状态,其“能量”为 E 。若系统从旧状态跳到新状态,能量变化为

$$\Delta E = E_{\text{new}} - E_{\text{old}}.$$

在温度 T 下,经典统计物理中常用 Boltzmann 因子

$$e^{-\Delta E/(k_B T)}$$

来衡量高能状态被接受的可能性。在数值优化中,通常把 Boltzmann 常数 k_B 并入温度参数,简化写成

$$p = \exp\left(-\frac{\Delta f}{T}\right),$$

其中 Δf 表示目标函数值的增量。

由此得到模拟退火法最重要的一条接受规则:

- 若新状态更优,即 $\Delta f \leq 0$,总是接受;
- 若新状态更差,即 $\Delta f > 0$,仍以概率

$$p = \exp\left(-\frac{\Delta f}{T}\right)$$

接受。

这意味着:

- 在高温时,较差状态也可能被较大概率接受,搜索更“活跃”;
- 在低温时,较差状态被接受的概率迅速减小,搜索更“保守”;
- 随着温度下降,算法逐渐从“全局探索”转向“局部收敛”。

4.3.11 Metropolis 过程与基本步骤

4.3.11.1 基本原理

模拟退火法通常以 Metropolis 过程为核心。设当前点为 x_0 ，当前温度为 t_k ，目标函数为 $f(x)$ ，则一次基本更新过程如下。

1. 给定初始温度 t_0 及初始点 x_0 ，计算

$$f(x_0).$$

2. 随机产生扰动 Δx ，得到新点

$$x = x_0 + \Delta x,$$

并计算新点函数值 $f(x)$ 以及差值

$$\Delta f = f(x) - f(x_0).$$

3. 若

$$\Delta f \leq 0,$$

则接受新点，把它作为下一次模拟退火的当前点。

4. 若

$$\Delta f > 0,$$

则计算接受概率

$$p(\Delta f) = \exp\left(-\frac{\Delta f}{t_k}\right).$$

同时生成一个服从区间 $[0, 1]$ 上均匀分布的伪随机数

$$r \in [0, 1].$$

若

$$p(\Delta f) \geq r,$$

则接受新点；否则拒绝新点，保留原点作为下一步的当前点。

接受劣解的意义第 4 步是模拟退火法与普通贪心下降法最本质的区别。若一个算法只接受更优点，那么它一旦进入某个局部极小附近，就很容易停在那里。而模拟退火法允许以一定概率接受更差点，因此可以“翻越势垒”，跳出局部极小，继续寻找全局更优解。

4.3.11.2 接受概率的性质

对

$$p(\Delta f) = \exp\left(-\frac{\Delta f}{t_k}\right)$$

作简单分析：

- 当 Δf 固定时，温度 t_k 越高， $p(\Delta f)$ 越大；
- 当温度固定时， Δf 越大， $p(\Delta f)$ 越小；
- 当 $t_k \rightarrow 0$ 时，若 $\Delta f > 0$ ，则

$$p(\Delta f) \rightarrow 0,$$

算法几乎不再接受变差步骤；

- 当 $t_k \rightarrow \infty$ 时，

$$p(\Delta f) \rightarrow 1,$$

算法对好坏点的区分变弱。

因此，温度参数 t_k 实际上控制了算法“允许冒险”的程度。

4.3.11.3 一次完整退火过程的结构

在真正的算法中，并不是每次更新后都立刻降温，而通常是在某一温度下重复进行若干次 Metropolis 更新，然后再降低温度。于是整个算法由两层循环组成：

- 外层循环：控制温度逐步下降；
- 内层循环：在固定温度下做多次随机扰动与接受 / 拒绝判断。

这样做的目的是：在同一个温度水平下，让系统有足够时间达到“近似热平衡”，随后再进入更低温阶段。

4.3.12 模拟退火法的伪代码结构

模拟退火的伪代码可以整理为如下形式。设当前状态记为 γ ，其代价函数为 $\text{cost}(\gamma)$ ，则：

1. 随机生成初始解 γ ；
2. 计算初始能量或代价

$$E_{\text{old}} = \text{cost}(\gamma).$$

3. 对温度做循环：

$$\text{temp} = \text{temp}_{\text{max}}, \quad \text{temp} \geq \text{temp}_{\text{min}}, \quad \text{temp} \leftarrow \text{next_temp}(\text{temp}).$$

4. 在每个温度下，重复做若干次扰动：

- (a). 调用邻域生成函数（successor function）产生一个新候选解；
- (b). 计算新能量

$$E_{\text{new}} = \text{cost}(\gamma),$$

以及

$$\Delta E = E_{\text{new}} - E_{\text{old}}.$$

- (c). 若

$$\Delta E \leq 0,$$

则无条件接受新状态，并令

$$E_{\text{old}} = E_{\text{new}}.$$

- (d). 若

$$\Delta E > 0,$$

则按概率

$$\exp\left(-\frac{\Delta E}{K \cdot \text{temp}}\right)$$

决定是否接受；若拒绝，则撤销这一步扰动（undo）；若接受，则更新

$$E_{\text{old}} = E_{\text{new}}.$$

这里的几个函数各有明确含义：

- `successor_func`：给当前解制造一个随机邻居；
- `undo_func`：若新解被拒绝，则把状态恢复到旧解；
- `next_temp(temp)`：执行降温调度（cooling schedule）。

伪代码的三层核心模拟退火法的实现可以概括为三件事：

- 如何定义目标函数；

- 如何从当前解随机产生邻居解；
- 如何设计降温方案。

其中第 2 点决定“搜索空间能否被有效遍历”，第 3 点决定“探索与收敛之间如何平衡”。

4.3.13 关键参数与调控原则

模拟退火法的性能在很大程度上取决于参数设计。通常需要重点调控以下四类关键参数。

4.3.13.1 初始温度的控制

初始温度越高，算法在开始阶段越容易接受较差解，因此：

- 优点是更容易跨越局部势垒，搜索到全局最优区域；
- 缺点是搜索太随机，所需计算时间显著增加。

若初始温度过低，则算法一开始就会过于保守，表现得更像局部搜索方法，容易较早陷入某个局部极小。

因此，初始温度应足够高，使得在起始阶段“较差解的接受率”处于较高水平，但又不能高到完全失去筛选能力。

4.3.13.2 降温方法的控制

温度下降策略称为退火计划（annealing schedule）或冷却方案（cooling schedule）。

若降温过快：

- 系统还来不及充分探索就被冻结；
- 可能错过真正的极值点；
- 算法退化为较差的局部贪心搜索。

若降温过慢：

- 搜索虽然更充分；
- 但计算代价极大；
- 收敛速度明显变慢。

常见的降温方式包括：

几何降温

$$T_{k+1} = \lambda T_k, \quad 0 < \lambda < 1.$$

这是实践中最常见的一类方案，简单有效。

线性降温

$$T_{k+1} = T_k - \delta.$$

对数降温

$$T_k = \frac{c}{\log(1+k)}.$$

它在理论分析中很重要，因为一些严格的全局收敛结论往往建立在足够缓慢的对数降温之上，但在实际中通常过慢。

4.3.13.3 每一温度下的迭代长度控制

在每个固定温度下，要进行多少次扰动，也是非常关键的设计问题。

一个自然的设计原则是：

- 在高温时，各种状态被接受的概率差别不大，许多候选状态都能被接受，因此同一温度下可以少做一些步数；

- 随着温度降低，被拒绝的状态越来越多，为了充分搜索该温度层级，可适当增加该温度下的迭代步数。一种常见做法是设置两个上限：

- 迭代步数上限 S ；
- 接受次数上限 R 。

在某一固定温度下：

- 若已经接受了 R 次新状态，则认为该温度下探索已足够，可以提前进入下一温度；
- 若尚未接受到 R 次，但已迭代到步数上限 S ，则也停止该温度层的搜索。

这种设计可以避免在某一温度层浪费过多计算量。

4.3.13.4 终止准则

模拟退火法何时停止，没有唯一标准。一种常见的基于接受概率的判据是：

给定一个较小概率 P 。在某一温度与给定迭代步数范围内，若除了当前局部最优解附近外，其他状态的接受概率几乎都小于 P ，则说明系统已经“冻结”，算法可以终止。

更常见的终止条件还包括：

- 温度降到预设下限

$$T \leq T_{\min};$$

- 连续若干温度层最优值几乎不再改善；
- 达到最大迭代次数；
- 当前解的波动范围已经足够小。

参数选择没有通用最优模板模拟退火法不像牛顿法那样具有统一而明确的局部收敛公式。它的效果高度依赖于邻域生成方式、降温速度、步数长度、初始温度等设计。因此在实际应用中，常需要通过若干次数值实验来调参。

4.3.14 模拟退火法解决旅行者问题

4.3.14.1 问题背景

旅行商问题（Traveling Salesman Problem, TSP）是组合优化中的经典问题之一。

设旅行者需要访问 N 个城市，各城市的位置为

$$(x_i, y_i), \quad i = 1, 2, \dots, N.$$

要求：

- 每个城市只访问一次；
- 最后回到起点；
- 总路程最短。

这是一个典型的离散优化（discrete optimization）与全局优化问题。随着城市数 N 增大，可能的路径数目呈阶乘级增长，穷举法很快失去可行性，因此非常适合作为模拟退火法的应用示例。

4.3.14.2 状态、目标函数与邻域

在模拟退火框架下，需要依次回答三个问题：

1. Configuration: 如何表示一个状态一个状态就是一条完整的遍历路径。例如可用一个排列

$$\pi = (\pi_1, \pi_2, \dots, \pi_N)$$

表示旅行顺序，其中 π_i 表示第 i 次访问的城市编号。

2. Objective function: 目标函数是什么最自然的目标函数就是总路径长度。若按路径顺序记相邻点为

$$(x_i, y_i), (x_{i+1}, y_{i+1}),$$

可写为

$$E = L = \sum_{i=0}^{N-1} \sqrt{(x_i - x_{i+1})^2 + (y_i - y_{i+1})^2}.$$

严格写成回路形式时，通常还应包含终点回到起点的一段，可写成

$$L = \sum_{i=1}^{N-1} \sqrt{(x_{\pi_i} - x_{\pi_{i+1}})^2 + (y_{\pi_i} - y_{\pi_{i+1}})^2} + \sqrt{(x_{\pi_N} - x_{\pi_1})^2 + (y_{\pi_N} - y_{\pi_1})^2}.$$

我们的目标就是最小化 L 。

3. Rearrangements: 如何产生新路径这一步相当于在“路径空间”中定义邻域。因此需要人为设计一种从旧路径生成新路径的方法。

常见设计包括：

- 随机交换两座城市的位置；
- 随机选取一段路径并反转顺序（2-opt 思想）；
- 将某个城市从当前位置取出并插入到另一位置；
- 对若干相邻段做更复杂的重连。

生成新路径后，比较新旧路径长度：

- 若新路径更短，则直接接受；
- 若新路径更长，则按模拟退火概率接受。

4.3.14.3 旅行商问题上的退火流程

因此，模拟退火法求解 TSP 的流程可整理为：

1. 给定初始路径 $\pi^{(0)}$ ；
2. 计算其总长度

$$L(\pi^{(0)});$$

3. 在当前温度下，对路径做一次随机重排，得到新路径 $\tilde{\pi}$ ；
4. 计算长度差

$$\Delta L = L(\tilde{\pi}) - L(\pi);$$

5. 若

$$\Delta L \leq 0,$$

则接受新路径；若

$$\Delta L > 0,$$

则以概率

$$\exp\left(-\frac{\Delta L}{T}\right)$$

接受；

6. 在该温度下重复若干次后降温；
7. 当达到终止准则后，输出当前最优路径。

4.3.14.4 图像演化的物理含义

若观察 TSP 路径随温度变化的演化图像，就会看到模拟退火法的典型行为：

- 在高温阶段，路径十分杂乱，交叉很多，长度较大；
- 随着温度下降，路径逐渐开始整理，明显较差的交叉结构被消除；

- 在低温阶段，路径已经比较稳定，只做小范围修正；
- 最终得到一条较短且结构有序的闭合回路。

这正对应了退火法的两阶段特征：

- **高温探索**：强调跳出局部极值、遍历广泛状态空间；
- **低温收敛**：强调围绕优良解做精细局部优化。

为什么高温时路径看起来“更乱”这是因为高温下即使新路径更差，也常被接受。因此算法故意允许结构变坏，以换取更强的全局搜索能力。只有这样，才有机会摆脱早期形成的坏路径结构。

4.3.15 模拟退火法的算法特征与适用场景

4.3.15.1 优点

模拟退火法具有如下显著优点：

- 能处理复杂多峰函数；
- 对局部极小不敏感，具有较强的全局搜索能力；
- 不需要梯度信息，适合不可微或噪声目标函数；
- 对连续优化、离散优化、组合优化都可应用；
- 概念简单，程序实现较直接。

4.3.15.2 缺点

它也存在明显局限：

- 参数较多，需要经验调节；
- 收敛速度通常慢于局部梯度型方法；
- 最终结果带有随机性，不同运行可能得到不同解；
- 若降温过快，容易失去全局搜索能力；
- 若降温过慢，计算代价很高。

4.3.15.3 与其他优化方法的比较

与本章前面介绍的方法相比：

- 与梯度法、拟牛顿法相比，模拟退火法不依赖导数，但局部收敛速度通常更慢；
- 与 BFGS 这类局部方法相比，模拟退火法更强调全局优化能力；
- 与 Nelder-Mead 相比，二者都可做无导数优化，但模拟退火法更依赖随机跳跃，而 Nelder-Mead 更依赖确定性的几何形变；
- 在离散与组合优化问题上，模拟退火法通常比传统连续优化方法更自然。

本节小结

模拟退火法通过模仿热力学退火过程，把优化问题看作“能量最小化”问题来处理。其核心机制是：

1. 在当前解附近随机产生新解；
2. 对更优解总是接受；
3. 对更差解按概率

$$\exp\left(-\frac{\Delta f}{T}\right)$$

接受；

4. 随温度逐步下降，算法从全局探索逐渐转入局部收敛。

这一方法最适合：

- 多局部极值的全局优化；
- 不可微、带噪、黑箱型目标函数；
- 路径、排列、组合等离散优化问题。

特别是在旅行商问题这类组合优化任务中，模拟退火法能够通过“随机重排 + 概率接受 + 逐步降温”的机制，从杂乱无章的初始路径逐渐逼近高质量的短路径解。

从更高层次看，模拟退火法提供了一种非常重要的思想：**为了获得全局最优，算法有时必须暂时允许局部变差**。这正是全局优化与局部优化在策略上的根本差别。

4.3.16 遗传算法

4.3.16.1 基本思想

遗传算法 (Genetic Algorithm, GA) 是一类模仿生物进化过程的随机优化方法。它把自然界中的遗传 (inheritance)、选择 (selection)、交叉 (crossover) 与突变 (mutation) 机制引入到数值优化中，通过“种群演化”的方式逐步逼近问题的最优解。

与前面介绍的梯度法、拟牛顿法不同，遗传算法并不直接利用导数信息，而是把候选解看成一个个“个体”(individual)，把一组候选解看成一个“种群”(population)，然后让种群在若干代迭代中不断进化。适应度较高的个体更有机会留下后代，因此整体种群会逐渐向更优区域移动。

设优化问题写成

$$\max_{\mathbf{x} \in \Omega} F(\mathbf{x}),$$

或者

$$\min_{\mathbf{x} \in \Omega} f(\mathbf{x}).$$

遗传算法并不是直接在连续变量空间上做微分意义下的迭代，而是先把变量编码，再在编码空间中进行选择、交叉和突变操作。

4.3.16.2 优化问题与生物学过程的对应

遗传算法的核心是把优化过程与生物进化过程建立对应关系：

生物学概念	优化中的对应物
染色体 (chromosome)	一个候选解的编码表示
个体 (individual)	一个具体候选解
种群 (population)	一组候选解
适应度 (fitness)	解的优劣程度
自然选择 (selection)	优解更容易保留下来
交叉 (crossover)	两个解的信息重组
突变 (mutation)	对解做小概率随机扰动

因此，遗传算法的本质可以概括为：

从一批初始候选解出发，利用“优胜劣汰 + 随机重组 + 少量变异”的演化机制，使得种群整体适应度逐代提高。

4.3.17 遗传算法的基本组成

4.3.17.1 编码 (encoding)

遗传算法的第一步是编码，即把优化变量的取值转换成某种便于操作的字符串结构。最经典的是二进制编码 (binary encoding)。

例如，若一个变量 x 的取值区间为 $[a, b]$ ，把该区间离散成

$$2^{10} = 1024$$

个可能状态，则可用一个 10 位二进制串表示一个候选值，如

$$1001111100.$$

这个二进制串就对应一个个体。

若记该二进制串对应的十进制数为 $m \in \{0, 1, \dots, 1023\}$ ，则一种常用的解码方式为

$$x = a + \frac{m}{1023}(b - a).$$

这样就把染色体和实际变量值联系起来。

编码的作用 编码的目的，是把实际问题中的变量表示成一种适合做“交叉”和“突变”的结构。二进制编码最直观，但并不是唯一选择。对于不同问题，也可以采用：

- 实数编码 (real-valued encoding)；
- 整数编码 (integer encoding)；
- 排列编码 (permutation encoding)，如旅行商问题；
- 树结构编码等。

4.3.17.2 种群 (population)

单个候选解称为个体，若干个个体组成一个种群。设第 k 代种群记为

$$P^{(k)} = \{c_1^{(k)}, c_2^{(k)}, \dots, c_N^{(k)}\},$$

其中 $c_i^{(k)}$ 表示第 k 代的第 i 个个体编码， N 为种群规模 (population size)。

遗传算法与传统局部搜索方法的一个重要区别在于：它不是只维护一个当前解，而是同时维护一批解。这使得算法天然带有“并行搜索”的性质，可以同时探索参数空间中的多个区域。

4.3.17.3 适应度函数 (fitness function)

适应度函数用于衡量一个个体的优劣，是遗传算法中最核心的量之一。若目标是求最大值，则常可直接把目标函数作为适应度函数：

$$\text{Fitness}(\mathbf{x}) = F(\mathbf{x}).$$

若目标是求最小值，则需要把代价函数 (cost function) 转化为适应度。常见方式有：

$$\text{Fitness}(\mathbf{x}) = \frac{1}{1 + f(\mathbf{x})},$$

或者

$$\text{Fitness}(\mathbf{x}) = C - f(\mathbf{x}),$$

其中 C 为足够大的常数，以保证适应度非负。

适应度越高，意味着该个体越“优秀”，在下一代中被选为父母的概率就越高。

适应度函数不等于目标函数 在求最大值问题中，两者常常可以一致；但在求最小值问题中，通常必须做变换。因此“目标函数”与“适应度函数”在概念上应区分开来。

4.3.17.4 选择 (selection)

选择操作用于从当前种群中挑选出一批个体作为父代 (parents)，用于产生下一代。

基本原则是：

适应度高的个体被选中的概率更大。

一种常见思路是：先根据适应度对个体排序，然后按某种概率规则抽取父母。例如，若把适应度从高到低排序后重新编号，再把编号归一化成概率，就可以实现“优者多选”。

更经典的选择方式包括：

轮盘赌选择 (roulette wheel selection) 设第 i 个个体的适应度为 F_i ，则选中概率取为

$$p_i = \frac{F_i}{\sum_{j=1}^N F_j}.$$

适应度越大，轮盘上占的比例越大，被选中的概率越高。

锦标赛选择 (tournament selection) 每次随机抽取若干个体，取其中适应度最高者进入父代集合。

排序选择 (ranking selection) 不直接用适应度值本身，而使用排序位置分配概率，以避免适应度差异过大导致的“垄断现象”。

4.3.17.5 交叉 (crossover)

交叉操作是遗传算法最具有代表性的步骤。它模仿生物中父母遗传信息的重组过程，把两个父代编码进行切分并重新组合，从而生成新的子代。

以二进制编码为例，设两个父代分别为

$$c^{(1)} = 0110110101, \quad c^{(2)} = 1010011010.$$

若选定某个交叉点，把两条编码从该位置剪开，再交换后半段，则可生成两个子代。

例如单点交叉 (single-point crossover) 可写成：

$$0110|110101, \quad 1010|011010$$

交叉后得到

$$0110|011010, \quad 1010|110101.$$

除了单点交叉外，还有：

- 双点交叉 (two-point crossover)；
- 均匀交叉 (uniform crossover)；
- 对排列编码专门设计的顺序交叉等。

交叉的作用，是把不同优良个体中的“好基因”进行重组，从而可能产生更优后代。

4.3.17.6 突变 (mutation)

交叉后，编码还会发生少量随机改变，这就是突变。例如把一个二进制串中的某几位随机翻转：

$$101110010 \longrightarrow 101110000.$$

这相当于把某些基因从 1 改成 0，或从 0 改成 1。

突变的概率通常较小，记为 p_m 。若突变率太低，则种群多样性不足，容易过早陷入某一局部区域；若突变率太高，则演化过程会变得过于随机，破坏已经形成的优良结构。

突变的意义 突变最重要的功能，不是“直接制造最优解”，而是：

- 维持种群多样性；
- 防止所有个体过快变得过于相似；
- 给算法提供跳出局部极值的机会。

4.3.18 遗传算法的完整流程

把上述步骤串联起来，就得到遗传算法的一般流程。

4.3.18.1 初始种群

首先随机生成初始种群

$$P^{(0)} = \{\mathbf{c}_1^{(0)}, \mathbf{c}_2^{(0)}, \dots, \mathbf{c}_N^{(0)}\}.$$

这些个体通常由随机数生成，也可以加入少量人工构造的“经验初值”。

4.3.18.2 适应度评估

对当前种群中每个个体解码，计算其目标函数值，并进一步得到适应度：

$$F_i^{(k)} = \text{Fitness}(\mathbf{c}_i^{(k)}), \quad i = 1, \dots, N.$$

4.3.18.3 选择、交叉、突变

在第 k 代中：

1. 根据适应度从种群中选择父代；
2. 对父代个体按交叉概率 p_c 做交叉操作，生成子代；
3. 对子代按突变概率 p_m 做突变操作；
4. 得到新一代种群

$$P^{(k+1)}.$$

4.3.18.4 终止判断

若满足终止条件，则输出当前最优个体；否则返回适应度评估步骤，继续迭代。

常见终止条件包括：

- 达到最大代数 $G_{\max}$ ；
- 最优适应度在若干代内几乎不再改进；
- 种群方差很小，即整体已经趋于稳定；
- 已达到预期精度。

4.3.18.5 算法伪代码

遗传算法的标准结构可以概括为：

```

初始化种群 $P^{(0)}$ ;
 $k = 0$ ;
while 未满足终止条件 do
    计算 $P^{(k)}$  中每个个体的适应度;
    按适应度执行选择;
    执行交叉, 产生后代;
    执行突变;
    形成新种群 $P^{(k+1)}$ ;
     $k \leftarrow k + 1$ ;
end while

```

最终输出

$$\mathbf{x}_{\text{best}} = \arg \max_{\mathbf{x} \in P^{(k)}} \text{Fitness}(\mathbf{x})$$

对应的最优个体。

4.3.19 遗传算法的典型行为

经过“选择-交叉-突变”的不断迭代, 通常会观察到如下现象:

- 初始几代中, 种群适应度分布较散, 整体较混乱;
- 随着代数增加, 种群中较差个体逐渐减少;
- 平均适应度不断上升;
- 最佳适应度在若干代后趋于稳定。

如果画出“每一代中所有个体的适应度散点图”和“每一代中的最佳适应度曲线”, 通常可以看到:

- 上图点云整体不断下移或上移 (取决于最小化还是最大化);
- 下图中的最优值逐代改进, 并最终趋于平稳。

若图像对应的是一个最小值问题, 则最佳适应度曲线会逐渐下降并趋于稳定。这说明种群经过进化后, 逐渐聚集到更优区域。

4.3.20 遗传算法的优点

4.3.20.1 较强的全局搜索能力

遗传算法并不是沿某一个局部方向搜索, 而是利用整个种群在解空间中并行探索。因此, 它有较强的能力寻找全局最大值或全局最小值, 而不容易像纯局部落法那样过快陷入单一局部极值。

4.3.20.2 适合处理复杂问题

对于很多复杂优化问题, 目标函数可能:

- 不光滑;
- 不可导;
- 缺乏明确解析形式;
- 是一个只能“输入参数、得到输出”的黑箱函数。

在这些场景下, 传统微分方法往往难以直接使用, 而遗传算法只要求能够评价适应度函数, 不要求解析梯度, 因此具有较好的适应性。

4.3.20.3 并行性好

在同一代中，每个个体的适应度计算彼此独立，因此遗传算法天然适合并行计算（parallel computing）。尤其在大规模数值实验中，可以把不同个体分配给不同处理器并行评估，从而显著缩短运行时间。

遗传算法最适合的问题类型遗传算法尤其适合以下类型问题：

- 多峰全局优化问题；
- 组合优化问题；
- 黑箱优化问题；
- 无法方便求导的复杂工程优化。

4.3.21 遗传算法的局限性

4.3.21.1 对具体设计非常敏感

遗传算法并不是一个“参数一设就一定好用”的统一模板。对不同问题，编码方式、交叉方式、突变方式、适应度设计、种群规模、选择策略等都可能显著影响效果。

例如：

- 某种编码方式对一个问题可能非常自然，但对另一个问题则会严重破坏变量间的结构关系；
- 某种交叉操作在二进制串中有效，但在排列问题中可能产生非法解；
- 突变概率过低或过高都会导致性能变差。

因此，遗传算法往往需要针对具体问题做“算法工程化设计”。

4.3.21.2 可能出现过早趋同

所谓过早趋同（premature convergence），是指种群在还没有找到全局最优之前，就已经变得高度相似，缺乏继续探索的能力。

这常常表现为：

- 所有个体编码越来越接近；
- 适应度改善很快停滞；
- 算法最终只找到一个局部极值，而不是真正全局最优。

造成过早趋同的典型原因有：

- 选择压力过大，优良个体迅速垄断整个种群；
- 突变率太低，无法重新引入多样性；
- 初始种群覆盖面太窄。

缓解办法包括：

- 增大种群规模；
- 提高适度的突变率；
- 使用精英保留（elitism）与多样性维持策略；
- 采用自适应交叉与突变参数。

4.3.22 几个重要实现细节

4.3.22.1 最大化与最小化的统一处理

遗传算法在教材中常以“求最大值”表述，因为适应度越高代表越优秀。但对于最小化问题，只要对目标函数做适当变换即可统一处理。

例如若要求

$$\min f(\mathbf{x}),$$

可定义适应度为

$$\text{Fitness}(\mathbf{x}) = \frac{1}{1 + f(\mathbf{x})}.$$

于是 f 越小, 适应度越大, 算法仍按“适应度最大”原则工作。

4.3.22.2 精英保留

在普通的交叉和突变过程中, 最优秀个体有时也可能被破坏。因此在实践中常使用精英保留 (elitism) 策略, 即把当前代中最优的少数个体直接复制到下一代, 以保证最优值不会退化。

4.3.22.3 参数选择

遗传算法中几个最常见的参数为:

- 种群规模 N ;
- 最大代数 $G_{\max}$;
- 交叉概率 p_c ;
- 突变概率 p_m 。

一般而言:

- N 太小, 则种群多样性不足;
- N 太大, 则每代计算代价过高;
- p_c 太低, 则后代重组不足;
- p_m 太低, 则容易过早趋同;
- p_m 太高, 则搜索过于随机。

因此这些参数也往往需要通过试验逐步调节。

本节小结

遗传算法通过模仿生物进化中的自然选择和遗传机制, 把优化问题转化为“种群进化”问题来求解。其标准流程为:

1. 对解进行编码;
2. 构造初始种群;
3. 计算适应度;
4. 执行选择、交叉、突变;
5. 形成新一代种群并不断迭代;
6. 当达到终止条件时输出最优个体。

它的优势在于:

- 不依赖导数;
- 适合复杂、黑箱、不可微问题;
- 有较强的全局搜索能力;
- 易于并行实现。

它的局限在于:

- 对编码、交叉、突变等设计比较敏感;
- 参数较多;
- 可能出现过早趋同;

- 理论分析通常不如经典凸优化方法那样精确。

从本章整体脉络看，遗传算法代表的是另一类非常重要的优化思想：**不依赖局部导数，而依赖群体演化和随机重组来寻找全局较优解**。这一路线与模拟退火法、粒子群优化等方法一起，构成了现代全局随机优化的重要方法族。

第五章 本征值问题 (Eigenvalue Problems)

内容提要

- 本征值问题与量子力学可观测量、Schrödinger 方程和量子多体对角化密切相关
- 掌握相似变换 (similarity transformation)、Jacobi 方法与 Householder 变换的基本思想
- 理解 QR 算法、位移 QR、Schur 分解以及广义本征值问题的标准化方法
- 掌握奇异值分解 (SVD)、幂法 (power method)、反幂法与 Wielandt 缩小法
- 理解 Krylov 子空间、Arnoldi 算法、Lanczos 算法及其在大规模稀疏问题中的作用
- 了解数值重整化群 (NRG) 与 DMRG 如何服务于超大规模量子多体本征值问题

5.1 本征值问题的物理背景

5.1.1 本征值、可观测量与 Hilbert 空间

在量子力学中，能够被实验测量的物理量都对应某个算符 (operator)，而测量结果正是该算符的本征值 (eigenvalue)。若算符记为 $\hat{O}$ ，波函数记为 ψ ，则本征值问题写成

$$\hat{O}\psi = \lambda\psi.$$

这就是本征值方程的最基本形式。

波函数所在的空间通常是平方可积函数构成的 Hilbert 空间 (Hilbert space of square-integrable functions)。在这个空间中，态矢量可以展开到某一组基函数上，算符则可表示为矩阵。这样一来，连续本征值问题就被转写为有限维或可截断的矩阵本征值问题。

注 量子力学中两个不对易 (do not commute) 的算符一般不能同时精确测量。这意味着在选取基底时，往往只能把某一组可同时对角化的量作为“好量子数 (good quantum numbers)”，剩下的相互作用则表现为矩阵中的非对角耦合项。

现代量子多体问题中常见的相互作用并不只是一体项 (one-body term)，还可能包含二体项 (two-body term)、三体项 (three-body term) 乃至更复杂的有效相互作用。只要这些耦合足够复杂，最后几乎都会把问题推向“大矩阵对角化”。

5.1.2 从独立粒子图像到 Fock 空间展开

研究量子多体系统时，常从独立粒子图像 (independent-particle picture) 出发。也就是说，先假设体系可以由若干单粒子轨道的占据情况近似描述。对于费米子体系，这样的参考态常写成 Slater 行列式 (Slater determinant)：

$$\Phi_0 = \frac{1}{\sqrt{N!}} \begin{vmatrix} \phi_1(x_1) & \phi_2(x_1) & \cdots & \phi_N(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \cdots & \phi_N(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x_N) & \phi_2(x_N) & \cdots & \phi_N(x_N) \end{vmatrix}.$$

如果残余相互作用 (residual interaction) 不可忽略，那么真实多体波函数就不再是单个 Slater 行列式，而必须在更大的 Fock 空间 (Fock space) 中展开：

$$|\Psi\rangle = \sum_{\alpha} c_{\alpha} |\Phi_{\alpha}\rangle.$$

这里 $\{|\Phi_{\alpha}\rangle\}$ 是不同组态 (configuration) 的集合， c_{α} 是待定展开系数。

定义 5.1

组态混合 (**configuration mixing**) 是指真实多体波函数不能由单一参考组态准确描述, 而必须由多个组态线性叠加来表示。

把上式代入多体 Schrödinger 方程

$$\hat{H}|\Psi\rangle = E|\Psi\rangle,$$

并左乘每个基矢 $\langle\Phi_\beta|$, 就得到矩阵方程

$$\sum_{\alpha} H_{\beta\alpha} c_{\alpha} = E c_{\beta}, \quad H_{\beta\alpha} = \langle\Phi_\beta|\hat{H}|\Phi_\alpha\rangle.$$

这正是标准的矩阵本征值问题

$$H\mathbf{c} = E\mathbf{c}.$$

5.1.3 为什么量子多体问题会变成巨型对角化问题

本征值问题在多体物理中之所以困难, 关键不在“方程形式”, 而在“组态空间维数”。若有 N_{orb} 个单粒子轨道、 N_{part} 个粒子, 则费米体系允许的组态数往往是组合数

$$\binom{N_{\text{orb}}}{N_{\text{part}}}.$$

它随体系规模增长极快。

例题 5.1 组合维度的爆炸 若有 20 个轨道、10 个费米子, 则组态数为

$$\binom{20}{10} = 184756.$$

若进一步增加到 40 个轨道、20 个粒子, 则

$$\binom{40}{20} \approx 1.378 \times 10^{11}.$$

这说明即使单粒子自由度只增加一倍, 多体 Hilbert 空间也会发生数量级爆炸。

因此, 多体关联 (correlation)、量子纠缠 (entanglement) 和组态混合的存在, 最终都会把问题转化成超大规模矩阵的本征值计算。

5.1.4 特征多项式视角与数值求解的现实困难

从代数学角度, 本征值可由特征方程

$$\det(A - \lambda I) = 0$$

得到。其左端是关于 λ 的 n 次多项式, 理论上求根就能得到全部本征值。

但是在数值计算中, 这条路线通常并不可取, 原因有三:

1. 显式展开 $\det(A - \lambda I)$ 本身计算量大且数值不稳定;
2. 高次多项式求根对系数误差非常敏感;
3. 对大规模矩阵, 根本没有必要先构造整个特征多项式。

因此, 数值线性代数中的主流思路是: 不直接求特征多项式, 而是利用相似变换把矩阵逐步化简, 再从简化后的形式中读出或逼近本征值。

5.2 相似变换、谱不变量与 Jacobi 方法

5.2.1 相似变换有相同的本征值

定理 5.1 (相似变换保持本征值)

设 $A, B \in \mathbb{R}^{n \times n}$, 若存在可逆矩阵 S 使得

$$B = S^{-1}AS,$$

则 A 与 B 有完全相同的本征值。

证明 考虑 B 的特征多项式:

$$\det(B - \lambda I) = \det(S^{-1}AS - \lambda I).$$

注意

$$S^{-1}AS - \lambda I = S^{-1}AS - \lambda S^{-1}IS = S^{-1}(A - \lambda I)S,$$

于是

$$\det(B - \lambda I) = \det(S^{-1}(A - \lambda I)S) = \det(S^{-1}) \det(A - \lambda I) \det(S).$$

又因为

$$\det(S^{-1}) \det(S) = 1,$$

故

$$\det(B - \lambda I) = \det(A - \lambda I).$$

所以二者特征多项式相同, 本征值完全一致。

注 数值本征值算法大量依赖这一事实。我们并不改变本征值本身, 而是通过一连串相似变换把矩阵变成更容易处理的形式, 例如三对角矩阵 (tridiagonal matrix)、上海森堡矩阵 (upper Hessenberg matrix) 或 Schur 形式 (Schur form)。

5.2.2 实对称矩阵的正交对角化

当 A 为实对称矩阵 (real symmetric matrix) 时, 问题有最理想的结构:

$$A^T = A.$$

这类矩阵满足谱定理 (spectral theorem)。

定理 5.2 (实对称矩阵的谱定理)

若 $A \in \mathbb{R}^{n \times n}$ 为实对称矩阵, 则存在实正交矩阵 S , 使得

$$S^T AS = \Lambda,$$

其中 Λ 为实对角矩阵, 其对角元正是 A 的本征值。

证明 [证明思路] 一方面, 实对称矩阵的本征值一定为实数。设 $Ax = \lambda x$, 若 $x \neq 0$, 则

$$x^T Ax = \lambda x^T x.$$

左边为实数, 右边中 $x^T x > 0$, 故 λ 必为实数。

另一方面, 不同本征值对应的本征向量彼此正交。设

$$Ax = \lambda x, \quad Ay = \mu y, \quad \lambda \neq \mu.$$

则

$$\mathbf{x}^T A \mathbf{y} = \mu \mathbf{x}^T \mathbf{y}.$$

又由于 $A^T = A$,

$$\mathbf{x}^T A \mathbf{y} = (A \mathbf{x})^T \mathbf{y} = \lambda \mathbf{x}^T \mathbf{y}.$$

两式相减得

$$(\lambda - \mu) \mathbf{x}^T \mathbf{y} = 0.$$

因为 $\lambda \neq \mu$, 故 $\mathbf{x}^T \mathbf{y} = 0$ 。

取 A 的一组标准正交本征向量

$$\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_n,$$

组成正交矩阵

$$S = [\mathbf{q}_1 \ \mathbf{q}_2 \ \cdots \ \mathbf{q}_n],$$

则

$$AS = S\Lambda,$$

左乘 S^T 得

$$S^T AS = \Lambda.$$

这一定理解释了为什么对称本征值问题比一般非对称问题“好做得多”：它不仅有实本征值，而且可以用正交矩阵完成稳定的相似对角化。

5.2.3 Jacobi 旋转的单步思想

Jacobi 方法 (Jacobi's method) 主要用于实对称矩阵。其核心思想是：反复选取一个非对角元 a_{pq} , 构造只在 (p, q) 平面内旋转的正交矩阵 P , 把它消成 0。

这种旋转矩阵也可看成 Givens 旋转 (Givens rotation) 在对称对角化中的应用。其非平凡部分是

$$\begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}.$$

若把它嵌入到 n 维单位矩阵中, 就得到

$$P = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & \cos \theta & 0 & \cdots & 0 & \sin \theta \\ 0 & 0 & \cdots & 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 1 & 0 \\ 0 & 0 & \cdots & -\sin \theta & 0 & \cdots & 0 & \cos \theta \end{bmatrix}, \text{ 满足 } P^T P = I.$$

注 对矩阵右乘 P 时, 只会改变第 p, q 两列; 左乘 P^T 时, 只会改变第 p, q 两行。这正是 Jacobi 方法每步都能“局部修改、局部消元”的根源。

对称矩阵的单步 Jacobi 变换写为

$$A^{(1)} = P^T A P.$$

由于 P 正交, 所以这仍是相似变换, 因而本征值保持不变。

为了推导角度 θ 的选择, 只需研究 A 的 (p, q) 主子块

$$\begin{bmatrix} a_{pp} & a_{pq} \\ a_{pq} & a_{qq} \end{bmatrix}.$$

对它施加二维旋转后, 新矩阵的非对角元为

$$a'_{pq} = (a_{qq} - a_{pp})cs + a_{pq}(c^2 - s^2),$$

其中

$$c = \cos \theta, \quad s = \sin \theta.$$

令 $a'_{pq} = 0$, 再用

$$\tan 2\theta = \frac{2 \tan \theta}{1 - \tan^2 \theta},$$

可得经典关系

$$\tan 2\theta = \frac{2a_{pq}}{a_{qq} - a_{pp}}.$$

为了避免直接求 θ 带来的数值不稳定, 通常设

$$\tau = \frac{a_{qq} - a_{pp}}{2a_{pq}},$$

并令

$$t = \tan \theta.$$

由上式可得

$$t^2 + 2\tau t - 1 = 0,$$

即

$$t = -\tau \pm \sqrt{1 + \tau^2}.$$

实际计算时选择绝对值较小的根, 这样可保证

$$|\theta| \leq \frac{\pi}{4},$$

从而提高数值稳定性。进一步有

$$c = \frac{1}{\sqrt{1 + t^2}}, \quad s = tc.$$

5.2.4 Jacobi 方法的完整迭代

在 Jacobi 方法中, 常选当前绝对值最大的非对角元 a_{pq} 作为消元目标。每一步用一个旋转把它变成 0, 然后继续处理下一个非对角元。于是矩阵逐步向对角阵靠近。

一种常见的收敛量是非对角元素平方和

$$\text{off}(A)^2 = \sum_{i \neq j} a_{ij}^2.$$

Jacobi 旋转每次都精确消去某个非对角元, 并保持 Frobenius 范数

$$\|A\|_F^2 = \sum_{i,j} a_{ij}^2$$

不变, 因此 $\text{off}(A)$ 会逐渐减小。对实对称矩阵而言, Jacobi 方法最终收敛到对角矩阵。

例题 5.2 二维旋转消元的直观理解 设

$$A = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix}.$$

若先针对 (1, 2) 元做旋转, 取

$$\tan \theta = \frac{1}{3},$$

则左乘与右乘后, 矩阵的 (2, 1) 与 (1, 2) 元将被大幅压小乃至精确消去。这样就把原问题中“相互耦合的两个坐标方向”旋转到了一个更接近本征方向的基底上。

5.2.5 Jacobi 方法的并行化思路

Jacobi 方法的一个重要优点是: 它天然具有并行性。

原因在于, 若两次旋转涉及的指标集合互不相交, 例如一次用 (p, q) , 另一次用 (r, s) , 并且

$$\{p, q\} \cap \{r, s\} = \emptyset,$$

那么这两次变换彼此独立, 可以并行执行。

对于偶数维矩阵, 可以把一轮旋转组织成若干组互不冲突的配对。例如 $n = 8$ 时, 每一组可安排 4 个互不重叠的平面旋转, 轮换数轮之后即可完成一整轮“全扫描 (sweep)”。这使得 Jacobi 方法在并行机器上具有实现简单、同步关系清晰的优点。

5.3 Householder 变换与矩阵约化

5.3.1 Householder 反射的定义与性质

Householder 变换 (Householder transformation) 是一类反射变换 (reflection transformation), 其标准形式为

$$P = I - 2\mathbf{w}\mathbf{w}^T, \quad \mathbf{w}^T\mathbf{w} = 1.$$

其矩阵元写成

$$P_{ij} = \delta_{ij} - 2w_i w_j.$$

定理 5.3

矩阵

$$P = I - 2\mathbf{w}\mathbf{w}^T, \quad \mathbf{w}^T\mathbf{w} = 1$$

既是对称矩阵, 又是正交矩阵。



证明 对称性显然成立, 因为

$$P^T = (I - 2\mathbf{w}\mathbf{w}^T)^T = I - 2\mathbf{w}\mathbf{w}^T = P.$$

再看正交性:

$$P^T P = P^2 = (I - 2\mathbf{w}\mathbf{w}^T)^2.$$

展开得

$$P^2 = I - 4\mathbf{w}\mathbf{w}^T + 4\mathbf{w}\mathbf{w}^T\mathbf{w}\mathbf{w}^T.$$

由于

$$\mathbf{w}^T\mathbf{w} = 1,$$

故

$$\mathbf{w}\mathbf{w}^T\mathbf{w}\mathbf{w}^T = \mathbf{w}(\mathbf{w}^T\mathbf{w})\mathbf{w}^T = \mathbf{w}\mathbf{w}^T.$$

代回即得

$$P^2 = I - 4\mathbf{w}\mathbf{w}^T + 4\mathbf{w}\mathbf{w}^T = I.$$

因此 $P^T P = I$, 故 P 正交。

5.3.2 把向量反射到坐标轴方向

Householder 变换的目的, 是把任意向量 $\mathbf{v}$ 反射到某个坐标轴方向上。最常见的是把它变成

$$P\mathbf{v} = \alpha \mathbf{e}_1,$$

其中 $\mathbf{e}_1 = (1, 0, \dots, 0)^T$ 。

设

$$\mathbf{u} = \mathbf{v} - \alpha \mathbf{e}_1, \quad \mathbf{w} = \frac{\mathbf{u}}{\|\mathbf{u}\|}.$$

则有

$$P\mathbf{v} = \mathbf{v} - 2\mathbf{w}(\mathbf{w}^T \mathbf{v}).$$

只要 $\mathbf{w}$ 与 $\mathbf{u}$ 同方向, 就能让 $P\mathbf{v}$ 落到 $\mathbf{e}_1$ 方向。

也可作如下等价推导。若

$$P\mathbf{v} = \alpha \mathbf{e},$$

则由正交变换保持范数可知

$$(P\mathbf{v})^T P\mathbf{v} = \alpha^2 = \mathbf{v}^T \mathbf{v} = \|\mathbf{v}\|^2.$$

因此

$$\alpha = \pm \|\mathbf{v}\|.$$

进一步, 由

$$\mathbf{v} - \alpha \mathbf{e} = 2\mathbf{w}(\mathbf{w}^T \mathbf{v}),$$

两边平方后可得

$$2(\mathbf{w}^T \mathbf{v})^2 = \alpha^2 - \alpha v_1.$$

于是也可把

$$\mathbf{w} = \frac{\mathbf{v} - \alpha \mathbf{e}}{2(\mathbf{w}^T \mathbf{v})}$$

视为一种等价构造公式。更常见的实现则直接写成归一化形式

$$\mathbf{w} = \frac{\mathbf{v} - \alpha \mathbf{e}}{\|\mathbf{v} - \alpha \mathbf{e}\|}.$$

 **笔记** 符号选取与数值稳定性: 若 $v_1 > 0$, 却取 $\alpha = +\|\mathbf{v}\|$, 则

$$v_1 - \alpha$$

可能是两个接近的数相减, 造成严重消去误差。因此数值计算中常取

$$\alpha = -\operatorname{sgn}(v_1)\|\mathbf{v}\|.$$

这正体现了数值计算中强调的 **Avoiding Subtraction of Like Terms** 原则。

5.3.3 Householder 相似变换如何约化矩阵

在本征值问题中, 我们不是只对一个向量做变换, 而是对矩阵做相似 Householder 变换:

$$A^{(k+1)} = P^{(k)} A^{(k)} P^{(k)}.$$

第 k 步的目标, 是把第 k 列主对角线下方的元素

$$a_{k+2,k}, a_{k+3,k}, \dots, a_{n,k}$$

全部消为 0。

做法是取子向量

$$\mathbf{v} = \begin{bmatrix} a_{k+1,k}^{(k)} \\ a_{k+2,k}^{(k)} \\ \vdots \\ a_{n,k}^{(k)} \end{bmatrix},$$

然后构造作用在该子空间中的 Householder 反射。一种常用递推形式是

$$\alpha = -\operatorname{sgn}(a_{k+1,k}^{(k)}) \left(\sum_{j=k+1}^n (a_{jk}^{(k)})^2 \right)^{1/2},$$

$$r = (\mathbf{w}^T \mathbf{v}) = \left(\frac{1}{2} \alpha^2 - \frac{1}{2} \alpha a_{k+1,k}^{(k)} \right)^{1/2},$$

$$w_1^{(k)} = \dots = w_k^{(k)} = 0, \quad w_{k+1}^{(k)} = \frac{a_{k+1,k}^{(k)} - \alpha}{2r},$$

$$w_j^{(k)} = \frac{a_{jk}^{(k)}}{2r}, \quad j = k+2, \dots, n,$$

最终

$$P^{(k)} = I - 2\mathbf{w}^{(k)}(\mathbf{w}^{(k)})^T.$$

5.3.4 从一般矩阵到上 Hessenberg, 从对称矩阵到三对角

连续做 Householder 相似变换后, 一般矩阵会被约化为上 Hessenberg 矩阵 (upper Hessenberg matrix):

$$H = \begin{bmatrix} * & * & * & \cdots & * \\ * & * & * & \cdots & * \\ 0 & * & * & \cdots & * \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & * & * \end{bmatrix}.$$

也就是说, 仅主对角线下方第一条次对角线可能非零, 其余更低位置都被消成 0。

若原矩阵 A 还是对称矩阵, 那么因为

$$A^{(k+1)} = (A^{(k+1)})^T,$$

下三角被消掉的同时, 上三角对应位置也必然被消掉, 于是最终得到更简洁的三对角矩阵 (tridiagonal matrix):

$$T = \begin{bmatrix} d_1 & e_1 & 0 & \cdots & 0 \\ e_1 & d_2 & e_2 & \cdots & 0 \\ 0 & e_2 & d_3 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & e_{n-1} \\ 0 & 0 & \cdots & e_{n-1} & d_n \end{bmatrix}.$$

注 这一步非常关键。对于稠密对称矩阵, 先用 Householder 约化到三对角形式, 再对三对角矩阵做后续 QR/QL 对角化, 是直接法求本征值的经典标准路线。

5.3.5 例：用 Householder 变换将对称矩阵化为三对角矩阵。

设

$$A = A^{(1)} = \begin{bmatrix} 4 & 1 & -2 & 2 \\ 1 & 2 & 0 & 1 \\ -2 & 0 & 3 & -2 \\ 2 & 1 & -2 & -1 \end{bmatrix}.$$

由于 A 是对称矩阵，因此可以通过正交相似变换

$$A^{(k+1)} = P^{(k)} A^{(k)} P^{(k)}$$

将其化为三对角矩阵。

第一步：消去第一列中第 3, 4 个元素。

取第一列中从第 2 行开始的子向量

$$x^{(1)} = \begin{bmatrix} a_{21}^{(1)} \\ a_{31}^{(1)} \\ a_{41}^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 2 \end{bmatrix}.$$

于是

$$\alpha_1 = -\operatorname{sgn}(a_{21}^{(1)}) \left(\sum_{j=2}^4 (a_{j1}^{(1)})^2 \right)^{1/2} = -1 \cdot \sqrt{1^2 + (-2)^2 + 2^2} = -3.$$

又有

$$r_1 = \left(\frac{1}{2} \alpha_1^2 - \frac{1}{2} a_{21}^{(1)} \alpha_1 \right)^{1/2} = \left(\frac{1}{2} (-3)^2 - \frac{1}{2} \cdot 1 \cdot (-3) \right)^{1/2} = \sqrt{6}.$$

因此 Householder 向量为

$$w^{(1)} = \begin{bmatrix} 0 \\ \frac{a_{21}^{(1)} - \alpha_1}{2r_1} \\ \frac{a_{31}^{(1)}}{2r_1} \\ \frac{a_{41}^{(1)}}{2r_1} \end{bmatrix} = \begin{bmatrix} 0 \\ \frac{1 - (-3)}{2\sqrt{6}} \\ \frac{-2}{2\sqrt{6}} \\ \frac{2}{2\sqrt{6}} \end{bmatrix} = \begin{bmatrix} 0 \\ \frac{\sqrt{6}}{3} \\ -\frac{\sqrt{6}}{6} \\ \frac{\sqrt{6}}{6} \end{bmatrix}.$$

从而

$$P^{(1)} = I - 2w^{(1)} (w^{(1)})^T.$$

注意

$$w^{(1)} = \frac{\sqrt{6}}{6} \begin{bmatrix} 0 \\ 2 \\ -1 \\ 1 \end{bmatrix},$$

所以

$$P^{(1)} = I - 2 \left(\frac{\sqrt{6}}{6} \right)^2 \begin{bmatrix} 0 \\ 2 \\ -1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 2 & -1 & 1 \end{bmatrix}.$$

即

$$P^{(1)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -\frac{1}{3} & \frac{2}{3} & -\frac{2}{3} \\ 0 & \frac{2}{3} & \frac{2}{3} & \frac{1}{3} \\ 0 & -\frac{2}{3} & \frac{1}{3} & \frac{2}{3} \end{bmatrix}.$$

作正交相似变换, 得到

$$A^{(2)} = P^{(1)} A^{(1)} P^{(1)} = \begin{bmatrix} 4 & -3 & 0 & 0 \\ -3 & \frac{10}{3} & 1 & \frac{4}{3} \\ 0 & 1 & \frac{5}{3} & -\frac{4}{3} \\ 0 & \frac{4}{3} & -\frac{4}{3} & -1 \end{bmatrix}.$$

第二步: 消去第二列中第 4 个元素。

现在只需对右下角子空间作 Householder 变换。取

$$x^{(2)} = \begin{bmatrix} a_{32}^{(2)} \\ a_{42}^{(2)} \end{bmatrix} = \begin{bmatrix} 1 \\ \frac{4}{3} \end{bmatrix}.$$

于是

$$\alpha_2 = -\operatorname{sgn}(a_{32}^{(2)}) \left[(a_{32}^{(2)})^2 + (a_{42}^{(2)})^2 \right]^{1/2} = -\sqrt{1^2 + \left(\frac{4}{3}\right)^2} = -\frac{5}{3}.$$

并且

$$r_2 = \left(\frac{1}{2} \alpha_2^2 - \frac{1}{2} a_{32}^{(2)} \alpha_2 \right)^{1/2} = \left[\frac{1}{2} \left(-\frac{5}{3} \right)^2 - \frac{1}{2} \cdot 1 \cdot \left(-\frac{5}{3} \right) \right]^{1/2} = \frac{2\sqrt{5}}{3}.$$

于是

$$w^{(2)} = \begin{bmatrix} 0 \\ 0 \\ \frac{a_{32}^{(2)} - \alpha_2}{2r_2} \\ \frac{a_{42}^{(2)}}{2r_2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \frac{1 - (-5/3)}{2 \cdot (2\sqrt{5}/3)} \\ \frac{4/3}{2 \cdot (2\sqrt{5}/3)} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \frac{2\sqrt{5}}{5} \\ \frac{\sqrt{5}}{5} \end{bmatrix}.$$

因此

$$P^{(2)} = I - 2w^{(2)} (w^{(2)})^T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{3}{5} & -\frac{4}{5} \\ 0 & 0 & -\frac{4}{5} & \frac{3}{5} \end{bmatrix}.$$

继续作正交相似变换:

$$A^{(3)} = P^{(2)} A^{(2)} P^{(2)} = \begin{bmatrix} 4 & -3 & 0 & 0 \\ -3 & \frac{10}{3} & -\frac{5}{3} & 0 \\ 0 & -\frac{5}{3} & -\frac{33}{25} & \frac{68}{75} \\ 0 & 0 & \frac{68}{75} & \frac{149}{75} \end{bmatrix}.$$

因此，经过两次 Householder 正交相似变换后，原对称矩阵被化为三对角矩阵：

$$A^{(3)} = \begin{bmatrix} 4 & -3 & 0 & 0 \\ -3 & \frac{10}{3} & -\frac{5}{3} & 0 \\ 0 & -\frac{5}{3} & \frac{33}{5} & \frac{68}{75} \\ 0 & 0 & \frac{68}{75} & \frac{149}{75} \end{bmatrix}.$$

若原矩阵 A 不对称，则同样的 Householder 相似变换一般将其化为上 Hessenberg 矩阵。

5.3.6 三对角矩阵后的后续处理

若三对角矩阵中的某个次对角元 $e_i = 0$ ，则矩阵事实上已经分裂成两个更小的独立块，可以分别对角化。这一点在数值实现中很重要，因为它允许问题自动“分块递归”。

对于得到的三对角矩阵，可以继续使用 Jacobi、QR、QL 等方法求解。其中 QR/QL 对三对角矩阵尤其高效，因为每一步都只涉及非常局部的计算。

5.4 QR 算法、位移与 Schur 分解

5.4.1 QR 算法的基本形式

QR 算法 (QR algorithm) 是现代本征值计算中最重要的方法之一。其最基本迭代是：

$$A^{(1)} = A,$$

先做 QR 分解

$$A^{(i)} = Q^{(i)} R^{(i)},$$

其中 $Q^{(i)}$ 为正交矩阵， $R^{(i)}$ 为上三角矩阵，然后定义下一步矩阵为

$$A^{(i+1)} = R^{(i)} Q^{(i)}.$$

定理 5.4

QR 迭代中， $A^{(i+1)}$ 与 $A^{(i)}$ 相似，因此与原矩阵 A 有相同本征值。

证明 因为

$$A^{(i)} = Q^{(i)} R^{(i)},$$

故

$$R^{(i)} = (Q^{(i)})^T A^{(i)}.$$

于是

$$A^{(i+1)} = R^{(i)} Q^{(i)} = (Q^{(i)})^T A^{(i)} Q^{(i)}.$$

由于 $Q^{(i)}$ 正交， $(Q^{(i)})^{-1} = (Q^{(i)})^T$ ，因此

$$A^{(i+1)} = (Q^{(i)})^{-1} A^{(i)} Q^{(i)},$$

这正是相似变换。

QR 算法的核心现象是：在适当条件下， $A^{(i)}$ 会逐渐趋近于上三角形式；若 A 为实对称矩阵，则会趋近于对角矩阵，其对角元正是本征值。

5.4.2 对 Hessenberg / 三对角矩阵做 Givens 型 QR 分解

若矩阵已经是上 Hessenberg 形式，则做一次 QR 分解只需要依次消去次对角线元素，代价远低于对一般稠密矩阵重新做完整 QR。

这里最自然的工具是 Givens 旋转。设第 k 步旋转矩阵为 P_{k+1} ，其非平凡块为

$$\begin{bmatrix} c_{k+1} & s_{k+1} \\ -s_{k+1} & c_{k+1} \end{bmatrix}, \quad c_{k+1}^2 + s_{k+1}^2 = 1.$$

它被选择成使得

$$(P_{k+1}A_k^{(1)})_{k+1,k} = 0.$$

若当前需要消去的局部向量为

$$\begin{bmatrix} x_k \\ b_{k+1} \end{bmatrix},$$

则要求

$$-s_{k+1}x_k + c_{k+1}b_{k+1} = 0.$$

因此可取

$$r = \sqrt{x_k^2 + b_{k+1}^2}, \quad c_{k+1} = \frac{x_k}{r}, \quad s_{k+1} = \frac{b_{k+1}}{r}.$$

连续做 $n-1$ 次这样的旋转后，就得到上三角矩阵 R 。这里的 Givens 旋转和前面 Jacobi 对称对角化中的旋转形式相似，但用途不同。Jacobi 是做

$$P^T A P$$

以消掉对称矩阵的非对角元，而 QR 分解是逐步左乘若干旋转矩阵以消掉次对角元。

5.4.3 例：QR 算法的一次迭代。

设

$$A = A^{(1)} = \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix}.$$

下面先对 $A^{(1)}$ 作 QR 分解。

首先用 Givens 变换 P_2 消去第一列第二个元素。取

$$c_2 = \frac{3}{\sqrt{10}} = \frac{3\sqrt{10}}{10}, \quad s_2 = \frac{1}{\sqrt{10}} = \frac{\sqrt{10}}{10},$$

于是

$$P_2 = \begin{bmatrix} \frac{3\sqrt{10}}{10} & \frac{\sqrt{10}}{10} & 0 \\ -\frac{\sqrt{10}}{10} & \frac{3\sqrt{10}}{10} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

因此

$$A_2^{(1)} = P_2 A^{(1)} = \begin{bmatrix} \frac{3\sqrt{10}}{10} & \frac{\sqrt{10}}{10} & 0 \\ -\frac{\sqrt{10}}{10} & \frac{3\sqrt{10}}{10} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{bmatrix}.$$

计算得

$$A_2^{(1)} = \begin{bmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & \frac{4\sqrt{10}}{5} & \frac{3\sqrt{10}}{10} \\ 0 & 1 & 3 \end{bmatrix}.$$

记

$$A_2^{(1)} \equiv \begin{bmatrix} z_1 & q_1 & r_1 \\ 0 & x_2 & y_2 \\ 0 & b_3^{(1)} & a_3^{(1)} \end{bmatrix}.$$

其中

$$x_2 = \frac{4\sqrt{10}}{5}, \quad b_3^{(1)} = 1.$$

为了继续消去第二列第三个元素，取

$$s_3 = \frac{b_3^{(1)}}{\sqrt{x_2^2 + (b_3^{(1)})^2}} = \frac{1}{\sqrt{\left(\frac{4\sqrt{10}}{5}\right)^2 + 1}} = 0.36761,$$

$$c_3 = \frac{x_2}{\sqrt{x_2^2 + (b_3^{(1)})^2}} = \frac{\frac{4\sqrt{10}}{5}}{\sqrt{\left(\frac{4\sqrt{10}}{5}\right)^2 + 1}} = 0.92998.$$

于是

$$P_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_3 & s_3 \\ 0 & -s_3 & c_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.92998 & 0.36761 \\ 0 & -0.36761 & 0.92998 \end{bmatrix}.$$

从而

$$R^{(1)} \equiv A_3^{(1)} = P_3 A_2^{(1)}$$

即

$$R^{(1)} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.92998 & 0.36761 \\ 0 & -0.36761 & 0.92998 \end{bmatrix} \begin{bmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & \frac{4\sqrt{10}}{5} & \frac{3\sqrt{10}}{10} \\ 0 & 1 & 3 \end{bmatrix}.$$

计算得

$$R^{(1)} = \begin{bmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & 2.7203 & 1.9851 \\ 0 & 0 & 2.4412 \end{bmatrix}.$$

由于

$$R^{(1)} = P_3 P_2 A^{(1)},$$

所以

$$A^{(1)} = P_2^T P_3^T R^{(1)}.$$

因此

$$Q^{(1)} = P_2^T P_3^T.$$

具体为

$$Q^{(1)} = \begin{bmatrix} \frac{3\sqrt{10}}{10} & -\frac{\sqrt{10}}{10} & 0 \\ \frac{\sqrt{10}}{10} & \frac{3\sqrt{10}}{10} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.92998 & -0.36761 \\ 0 & 0.36761 & 0.92998 \end{bmatrix}.$$

计算得

$$Q^{(1)} = \begin{bmatrix} 0.94868 & -0.29409 & 0.11625 \\ 0.31623 & 0.88226 & -0.34874 \\ 0 & 0.36761 & 0.92998 \end{bmatrix}.$$

于是完成了一次 QR 分解:

$$A^{(1)} = Q^{(1)} R^{(1)}.$$

QR 算法的下一次迭代矩阵由

$$A^{(2)} = R^{(1)} Q^{(1)}$$

给出, 即

$$A^{(2)} = \begin{bmatrix} \sqrt{10} & \frac{3\sqrt{10}}{5} & \frac{\sqrt{10}}{10} \\ 0 & 2.7203 & 1.9851 \\ 0 & 0 & 2.4412 \end{bmatrix} \begin{bmatrix} 0.94868 & -0.29409 & 0.11625 \\ 0.31623 & 0.88226 & -0.34874 \\ 0 & 0.36761 & 0.92998 \end{bmatrix}.$$

计算得

$$A^{(2)} = \begin{bmatrix} 3.6 & 0.86024 & 0 \\ 0.86024 & 3.12973 & 0.89740 \\ 0 & 0.89740 & 2.27027 \end{bmatrix}.$$

可以看到, $A^{(2)}$ 仍然是对称三对角矩阵。

继续迭代, 得到

$$A^{(13)} = \begin{bmatrix} 4.4139 & 0.01941 & 0 \\ 0.01941 & 3.0003 & 0.00095 \\ 0 & 0.00095 & 1.5858 \end{bmatrix}.$$

由于非对角元已经很小, 因此矩阵已经接近对角矩阵。于是原矩阵 A 的特征值近似为

$$\lambda_1 \approx 4.4139, \quad \lambda_2 \approx 3.0003, \quad \lambda_3 \approx 1.5858.$$

事实上该矩阵的精确特征值为

$$\lambda_1 = 3 + \sqrt{2}, \quad \lambda_2 = 3, \quad \lambda_3 = 3 - \sqrt{2}.$$

即

$$3 + \sqrt{2} \approx 4.4142, \quad 3, \quad 3 - \sqrt{2} \approx 1.5858.$$

5.4.4 用 Householder 反射做 QR 分解

除 Givens 旋转外, Householder 反射也可用于 QR 分解。若对矩阵 A 依次构造反射矩阵

$$P_1, P_2, \dots, P_{n-1},$$

使得每一步都把当前列主对角线下方元素消为 0, 则最终有

$$Q = P_1 P_2 \cdots P_{n-1},$$

$$R = P_{n-1} \cdots P_2 P_1 A,$$

从而得到

$$A = QR.$$

注 这里的 Householder QR 与上一节“用 Householder 做相似约化”虽然都使用反射矩阵，但目的不同：

- QR 分解中主要做左乘，目的是构造 $A = QR$ ；
- 本征值预处理时做的是双边相似变换 PAP ，目的是把矩阵约化为 Hessenberg 或三对角形式。

5.4.5 例：用 Householder 变换对矩阵作 QR 分解。

设

$$A = A^{(0)} = \begin{pmatrix} 1 & -1 & 4 \\ 1 & 4 & -2 \\ 1 & 4 & 2 \\ 1 & -1 & 0 \end{pmatrix}.$$

记 A 的第一列为

$$a_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}.$$

首先构造第一个 Householder 向量

$$v_1 = a_1 - \text{sign}(a_{11})\|a_1\|e_1.$$

由于

$$\|a_1\| = 2, \quad \text{sign}(a_{11}) = 1,$$

所以

$$v_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} - 2 \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \\ 1 \\ 1 \end{pmatrix}.$$

于是

$$H_1 = I - 2 \frac{v_1 v_1^T}{v_1^T v_1}.$$

由于

$$v_1^T v_1 = 4,$$

因此

$$H_1 = I - \frac{1}{2} \begin{pmatrix} -1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} -1 & 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{pmatrix}.$$

由

$$H_1 A^{(0)} = A^{(0)} - \frac{1}{2} v_1 v_1^T A^{(0)}$$

可得

$$H_1 A^{(0)} = \begin{pmatrix} 1 & -1 & 4 \\ 1 & 4 & -2 \\ 1 & 4 & 2 \\ 1 & -1 & 0 \end{pmatrix} - \frac{1}{2} \begin{pmatrix} -1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \begin{pmatrix} 2 & 8 & -4 \end{pmatrix}.$$

因此

$$A^{(1)} = H_1 A^{(0)} = \begin{pmatrix} 2 & 3 & 2 \\ 0 & 0 & 0 \\ 0 & 0 & 4 \\ 0 & -5 & 2 \end{pmatrix}.$$

接下来对 $A^{(1)}$ 的右下子矩阵继续作 Householder 变换。取第二列从第 2 行到第 4 行构成的子向量

$$x_2 = \begin{pmatrix} 0 \\ 0 \\ -5 \end{pmatrix}.$$

构造

$$v_2 = x_2 - 5e_1 = \begin{pmatrix} 0 \\ 0 \\ -5 \end{pmatrix} - \begin{pmatrix} 5 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} -5 \\ 0 \\ -5 \end{pmatrix}.$$

于是右下子空间中的 Householder 矩阵为

$$\tilde{H}_2 = I - 2 \frac{v_2 v_2^T}{v_2^T v_2}.$$

因为

$$v_2^T v_2 = 50,$$

所以

$$\tilde{H}_2 = I - \frac{1}{25} \begin{pmatrix} -5 \\ 0 \\ -5 \end{pmatrix} \begin{pmatrix} -5 & 0 & -5 \end{pmatrix}.$$

即

$$\tilde{H}_2 = \begin{pmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix}.$$

嵌入到四维空间中，得到

$$H_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \end{pmatrix}.$$

于是

$$A^{(2)} = H_2 A^{(1)} = \begin{pmatrix} 2 & 3 & 2 \\ 0 & 5 & -2 \\ 0 & 0 & 4 \\ 0 & 0 & 0 \end{pmatrix}.$$

因此

$$H_2 H_1 A = \begin{pmatrix} R \\ 0 \end{pmatrix},$$

其中

$$R = \begin{pmatrix} 2 & 3 & 2 \\ 0 & 5 & -2 \\ 0 & 0 & 4 \end{pmatrix}.$$

由于 Householder 矩阵满足

$$H_1^{-1} = H_1^T = H_1, \quad H_2^{-1} = H_2^T = H_2,$$

所以

$$A = (H_2 H_1)^{-1} \begin{pmatrix} R \\ 0 \end{pmatrix} = (H_2 H_1)^T \begin{pmatrix} R \\ 0 \end{pmatrix}.$$

因此

$$Q = (H_2 H_1)^T = H_1 H_2.$$

计算得

$$Q = H_1 H_2 = \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{pmatrix}.$$

于是得到矩阵 A 的 QR 分解:

$$A = Q \begin{pmatrix} R \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{pmatrix} \begin{pmatrix} 2 & 3 & 2 \\ 0 & 5 & -2 \\ 0 & 0 & 4 \\ 0 & 0 & 0 \end{pmatrix}.$$

若采用约化 QR 分解, 则取 Q 的前三列, 得到

$$A = \hat{Q} R,$$

其中

$$\hat{Q} = \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \end{pmatrix}, \quad R = \begin{pmatrix} 2 & 3 & 2 \\ 0 & 5 & -2 \\ 0 & 0 & 4 \end{pmatrix}.$$

5.4.6 位移 QR (Shifted QR)

无位移 QR 的收敛可能较慢。为加速收敛, 常引入位移 (shift) σ :

$$\begin{aligned} A^{(i)} - \sigma I &= Q^{(i)} R^{(i)}, \\ A^{(i+1)} &= R^{(i)} Q^{(i)} + \sigma I. \end{aligned}$$

这仍然是相似变换，因为

$$A^{(i+1)} = (Q^{(i)})^T (A^{(i)} - \sigma I) Q^{(i)} + \sigma I = (Q^{(i)})^T A^{(i)} Q^{(i)}.$$

对于上 Hessenberg 矩阵，设第 $j+1$ 个次对角元记为 $b_{j+1}^{(i)}$ ，则无位移时其衰减速度与

$$\left| \frac{\lambda_{j+1}}{\lambda_j} \right|$$

有关；有位移后则改进为与

$$\left| \frac{\lambda_{j+1} - \sigma}{\lambda_j - \sigma} \right|$$

有关。若把 σ 选得靠近目标本征值 λ_{j+1} ，则上式可显著变小，因此收敛会更快。

例题 5.3 为什么位移能加速 可以用一个 2×2 观察矩阵说明位移的加速作用，其局部本征值为

$$\mu_{1,2} = \frac{1}{2} \pm \frac{\sqrt{3}}{2}.$$

若取位移

$$\sigma_1 = \frac{1}{2} - \frac{\sqrt{3}}{2},$$

则与靠近该本征值的方向相关的误差项会被大幅压缩，因此 QR 迭代比无位移情形快得多。

5.4.7 Schur 分解

对一般非对称方阵，QR 算法未必收敛到对角矩阵，但会收敛到 Schur 形式。

定理 5.5 (Schur 分解)

对任意复方阵 $A \in \mathbb{C}^{n \times n}$ ，存在酉矩阵 (unitary matrix) Q 与上三角矩阵 U ，使得

$$A = QUQ^*.$$

其中 U 的对角元就是 A 的本征值。



若 A 为实矩阵，则严格地说通常得到的是实 Schur 形式 (real Schur form)，即由 1×1 与 2×2 块组成的准上三角矩阵。但在对称实矩阵这一最常见情形下，它就会进一步退化成真正的对角矩阵。

5.5 变换后的本征值问题

5.5.1 广义本征值问题的标准化

很多物理问题并不是标准形式

$$A\mathbf{x} = \lambda\mathbf{x},$$

而是广义本征值问题

$$A\mathbf{x} = \lambda B\mathbf{x}.$$

如果 A, B 都是对称矩阵，并且 B 正定 (positive definite)，则可以把它化成标准对称本征值问题。

若直接写成

$$B^{-1}A\mathbf{x} = \lambda\mathbf{x}$$

虽然形式上变成了标准本征值问题，但

$$B^{-1}A$$

一般不是对称矩阵，因此会破坏对称问题原有的良好数值性质。

正确做法是对 B 做 Cholesky 分解:

$$B = LL^T.$$

令

$$\mathbf{y} = L^T \mathbf{x}, \quad C = L^{-1} A (L^{-1})^T.$$

则原方程等价于

$$C\mathbf{y} = \lambda \mathbf{y}.$$

证明 由

$$A\mathbf{x} = \lambda B\mathbf{x} = \lambda LL^T \mathbf{x},$$

左乘 L^{-1} 得

$$L^{-1} A \mathbf{x} = \lambda L^T \mathbf{x}.$$

再代入

$$\mathbf{x} = (L^{-1})^T \mathbf{y},$$

得到

$$L^{-1} A (L^{-1})^T \mathbf{y} = \lambda \mathbf{y}.$$

也就是

$$C\mathbf{y} = \lambda \mathbf{y}.$$

同时

$$C^T = \left(L^{-1} A (L^{-1})^T \right)^T = L^{-1} A^T (L^{-1})^T = L^{-1} A (L^{-1})^T = C,$$

故 C 仍为对称矩阵。

5.5.2 Hermitian 问题化为实对称问题

另一类常见变形是: 若复矩阵可写成

$$H = A + iB,$$

其中

$$A^T = A, \quad B^T = -B,$$

则 H 是 Hermitian 矩阵。设其本征向量写成

$$\mathbf{z} = \mathbf{u} + i\mathbf{v},$$

满足

$$(A + iB)(\mathbf{u} + i\mathbf{v}) = \lambda(\mathbf{u} + i\mathbf{v}).$$

分离实部和虚部, 可得

$$A\mathbf{u} - B\mathbf{v} = \lambda\mathbf{u}, \quad B\mathbf{u} + A\mathbf{v} = \lambda\mathbf{v}.$$

于是组合成实矩阵形式

$$\begin{bmatrix} A & -B \\ B & A \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} = \lambda \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix}.$$

这样, 就把一个 $n \times n$ 的 Hermitian 本征值问题转化成了一个 $2n \times 2n$ 的实对称本征值问题。与此同时, 与特

征向量

$$\begin{bmatrix} u \\ v \end{bmatrix}$$

配对的

$$\begin{bmatrix} -v \\ u \end{bmatrix}$$

也与同一本征值相关，反映了复表示与实表示之间的等价结构。

5.6 奇异值分解 (Singular Value Decomposition)

5.6.1 SVD 的定义与基本性质

对任意矩阵

$$A \in \mathbb{R}^{m \times n},$$

奇异值分解 (Singular Value Decomposition, SVD) 指存在正交矩阵

$$U \in \mathbb{R}^{m \times m}, \quad V \in \mathbb{R}^{n \times n},$$

以及“对角”矩阵

$$\Sigma \in \mathbb{R}^{m \times n},$$

使得

$$A = U\Sigma V^T.$$

这里 Σ 的对角线上是非负数

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0,$$

称为 A 的奇异值 (singular values), $r = \text{rank}(A)$ 。

注 若 A 既对称又正定，则 A 的本征值全为正，而奇异值就是这些本征值本身。也就是说，在这种特殊情形下 singular value 与 eigenvalue 完全一致。

5.6.2 SVD 与 $A^T A$ 的关系

定理 5.6

设

$$A = U\Sigma V^T$$

是 A 的 SVD，则

$$A^T A = V\Sigma^T \Sigma V^T.$$

因此 $A^T A$ 的本征值为

$$\sigma_1^2, \sigma_2^2, \dots, \sigma_r^2, 0, \dots, 0.$$



证明 由 $U^T U = I$ ，可得

$$A^T A = (U\Sigma V^T)^T (U\Sigma V^T) = V\Sigma^T U^T U \Sigma V^T = V\Sigma^T \Sigma V^T.$$

而 $\Sigma^T \Sigma$ 是对角矩阵，其对角元正是 σ_i^2 。因此 V 的列向量是 $A^T A$ 的标准正交本征向量， σ_i^2 是对应本征值。

同理，

$$A A^T = U \Sigma \Sigma^T U^T,$$

所以 AA^T 的非零本征值也与 $A^T A$ 相同, 都是 σ_i^2 。

5.6.3 如何由特征分解构造 SVD

这一定理给出了构造 SVD 的实际路线:

1. 先解对称矩阵 $A^T A$ 的本征值问题;
2. 取其单位本征向量为 V 的列向量 $\mathbf{v}_i$;
3. 对每个非零奇异值定义

$$\mathbf{u}_i = \frac{A\mathbf{v}_i}{\sigma_i};$$

4. 再把剩余列向量补成 U 的完整正交基。

证明 [为什么 $\mathbf{u}_i$ 正交] 设 $i \neq j$, 则

$$\mathbf{u}_i^T \mathbf{u}_j = \frac{1}{\sigma_i \sigma_j} (A\mathbf{v}_i)^T (A\mathbf{v}_j) = \frac{1}{\sigma_i \sigma_j} \mathbf{v}_i^T A^T A \mathbf{v}_j.$$

由于 $\mathbf{v}_j$ 是 $A^T A$ 的本征向量,

$$A^T A \mathbf{v}_j = \sigma_j^2 \mathbf{v}_j,$$

于是

$$\mathbf{u}_i^T \mathbf{u}_j = \frac{\sigma_j}{\sigma_i} \mathbf{v}_i^T \mathbf{v}_j = 0.$$

当 $i = j$ 时,

$$\mathbf{u}_i^T \mathbf{u}_i = \frac{1}{\sigma_i^2} \mathbf{v}_i^T A^T A \mathbf{v}_i = \frac{1}{\sigma_i^2} \sigma_i^2 \mathbf{v}_i^T \mathbf{v}_i = 1.$$

故非零奇异值对应的 $\mathbf{u}_i$ 构成标准正交组。

当 $m > r$ 时, 还需要补出其余 $m-r$ 个向量。一种常见做法是任意给出若干线性无关向量, 再用 Gram-Schmidt 正交化把它们补成完整的正交基。这就是 “Add $m - k$ linearly independent vectors, then use Gram-Schmidt” 的含义。

5.6.4 例: 由特征分解构造 SVD。

设

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}.$$

先计算

$$A^T A = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 4 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

对称矩阵 $A^T A$ 的特征值为

$$\lambda_1 = 5, \quad \lambda_2 = 2, \quad \lambda_3 = 1.$$

因此 A 的奇异值为

$$\sigma_1 = \sqrt{5}, \quad \sigma_2 = \sqrt{2}, \quad \sigma_3 = 1.$$

于是

$$\Sigma = \begin{bmatrix} \sqrt{5} & 0 & 0 \\ 0 & \sqrt{2} & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

取 $A^T A$ 的一组标准正交特征向量为

$$v_1 = \begin{bmatrix} \frac{\sqrt{6}}{6} \\ \frac{\sqrt{6}}{6} \\ \frac{3}{3} \\ \frac{\sqrt{6}}{6} \\ \frac{3}{6} \end{bmatrix}, \quad v_2 = \begin{bmatrix} \frac{\sqrt{3}}{3} \\ \frac{\sqrt{3}}{3} \\ -\frac{\sqrt{3}}{3} \\ \frac{\sqrt{3}}{3} \\ \frac{3}{3} \end{bmatrix}, \quad v_3 = \begin{bmatrix} -\frac{\sqrt{2}}{2} \\ 0 \\ \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \\ 0 \end{bmatrix}.$$

即

$$V = \begin{bmatrix} \frac{\sqrt{6}}{6} & \frac{\sqrt{3}}{3} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{6}}{6} & \frac{\sqrt{3}}{3} & 0 \\ \frac{3}{3} & -\frac{\sqrt{3}}{3} & 0 \\ \frac{\sqrt{6}}{6} & \frac{\sqrt{3}}{3} & \frac{\sqrt{2}}{2} \\ \frac{3}{6} & \frac{3}{3} & \frac{\sqrt{2}}{2} \end{bmatrix}.$$

由

$$u_i = \frac{Av_i}{\sigma_i}$$

可得

$$u_1 = \frac{1}{\sqrt{5}} A \begin{bmatrix} \frac{\sqrt{6}}{6} \\ \frac{\sqrt{6}}{6} \\ \frac{3}{3} \\ \frac{\sqrt{6}}{6} \\ \frac{3}{6} \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{30}}{15} \\ \frac{15}{\sqrt{30}} \\ \frac{15}{\sqrt{30}} \\ \frac{10}{\sqrt{30}} \\ \frac{15}{\sqrt{30}} \\ \frac{10}{\sqrt{30}} \end{bmatrix}, \quad u_2 = \frac{1}{\sqrt{2}} A \begin{bmatrix} \frac{\sqrt{3}}{3} \\ \frac{\sqrt{3}}{3} \\ -\frac{\sqrt{3}}{3} \\ \frac{\sqrt{3}}{3} \\ \frac{3}{3} \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{6}}{3} \\ \frac{\sqrt{6}}{6} \\ 0 \\ -\frac{\sqrt{6}}{6} \\ 0 \end{bmatrix}, \quad u_3 = A \begin{bmatrix} -\frac{\sqrt{2}}{2} \\ 0 \\ \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \frac{\sqrt{2}}{2} \\ 0 \\ -\frac{\sqrt{2}}{2} \end{bmatrix}.$$

因为 $A \in \mathbb{R}^{5 \times 3}$, 所以 U 应为 5×5 正交矩阵。目前只得到 u_1, u_2, u_3 , 还需要补充两个单位正交向量。例如取

$$u_4 = \begin{bmatrix} 0 \\ \frac{1}{\sqrt{2}} \\ 0 \\ -\frac{1}{\sqrt{2}} \\ 0 \end{bmatrix}, \quad u_5 = \begin{bmatrix} \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} \\ -\frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} \\ -\frac{1}{\sqrt{5}} \end{bmatrix}.$$

可以验证

$$u_i^T u_j = 0 \quad (i \neq j), \quad u_i^T u_i = 1.$$

于是

$$U = \begin{bmatrix} \frac{\sqrt{30}}{15} & \frac{\sqrt{6}}{3} & 0 & 0 & \frac{1}{\sqrt{5}} \\ \frac{\sqrt{30}}{15} & -\frac{\sqrt{6}}{6} & 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{5}} \\ \frac{\sqrt{30}}{10} & 0 & \frac{\sqrt{2}}{2} & 0 & -\frac{1}{\sqrt{5}} \\ \frac{\sqrt{30}}{15} & -\frac{\sqrt{6}}{6} & 0 & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{5}} \\ \frac{\sqrt{30}}{10} & 0 & -\frac{\sqrt{2}}{2} & 0 & -\frac{1}{\sqrt{5}} \end{bmatrix}.$$

因此得到 A 的奇异值分解

$$A = U\Sigma V^T$$

即

$$A = \begin{bmatrix} \frac{\sqrt{30}}{15} & \frac{\sqrt{6}}{3} & 0 & 0 & \frac{1}{\sqrt{5}} \\ \frac{\sqrt{30}}{15} & -\frac{\sqrt{6}}{6} & 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{5}} \\ \frac{\sqrt{30}}{10} & 0 & \frac{\sqrt{2}}{2} & 0 & -\frac{1}{\sqrt{5}} \\ \frac{\sqrt{30}}{15} & -\frac{\sqrt{6}}{6} & 0 & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{5}} \\ \frac{\sqrt{30}}{10} & 0 & -\frac{\sqrt{2}}{2} & 0 & -\frac{1}{\sqrt{5}} \end{bmatrix} \begin{bmatrix} \sqrt{5} & 0 & 0 \\ 0 & \sqrt{2} & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{6}}{6} & \frac{\sqrt{6}}{3} & \frac{\sqrt{6}}{6} \\ \frac{\sqrt{3}}{3} & -\frac{\sqrt{3}}{3} & \frac{\sqrt{3}}{3} \\ -\frac{\sqrt{2}}{2} & 0 & \frac{\sqrt{2}}{2} \end{bmatrix}.$$

若只需要约化 SVD, 则取

$$\hat{U} = \begin{bmatrix} u_1 & u_2 & u_3 \end{bmatrix}, \quad \hat{\Sigma} = \begin{bmatrix} \sqrt{5} & 0 & 0 \\ 0 & \sqrt{2} & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

于是

$$A = \hat{U}\hat{\Sigma}V^T.$$

5.6.5 SVD 的几何意义与图像压缩

从几何上看, V^T 先把输入空间旋转到一组主方向上, Σ 再沿这些主方向做伸缩, 最后由 U 把结果旋转到输出空间方向。因此, SVD 把一个线性变换分解为:

旋转 $\rightarrow$ 伸缩 $\rightarrow$ 旋转.

SVD 的典型应用之一是图像压缩。若一幅灰度图像用矩阵

$$A \in \mathbb{R}^{N \times N}$$

表示, 则每个矩阵元对应一个像素亮度。对其做 SVD:

$$A = \sum_{i=1}^N \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

若只保留最大的前 k 个奇异值, 就得到秩 k 近似

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

定理 5.7 (截断 SVD 的最优性)

在所有秩不超过 k 的矩阵中, A_k 是同时关于 2-范数与 Frobenius 范数的最佳近似。

注 图像压缩的本质, 是把原本需要存储的 N^2 个像素值, 改为只存储

k 个奇异值 + kN 个左奇异向量参数 + kN 个右奇异向量参数,

即大约

$$(2N + 1)k$$

个参数。当 $k \ll N$ 时, 存储量会显著下降。

5.7 幂法、反幂法与 Wielandt 缩小法

5.7.1 幂法 (Power Method)

幂法 (power method) 是最概念化的一维迭代本征值算法。设矩阵 A 可对角化, 且本征值按绝对值排序为

$$|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|.$$

设初始向量可展开为

$$\mathbf{x}^{(0)} = c_1 \mathbf{q}_1 + c_2 \mathbf{q}_2 + \cdots + c_n \mathbf{q}_n, \quad c_1 \neq 0,$$

其中 $\mathbf{q}_i$ 是本征向量。则

$$A^k \mathbf{x}^{(0)} = c_1 \lambda_1^k \mathbf{q}_1 + c_2 \lambda_2^k \mathbf{q}_2 + \cdots + c_n \lambda_n^k \mathbf{q}_n.$$

提出最大因子 λ_1^k :

$$A^k \mathbf{x}^{(0)} = \lambda_1^k \left[c_1 \mathbf{q}_1 + c_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k \mathbf{q}_2 + \cdots + c_n \left(\frac{\lambda_n}{\lambda_1} \right)^k \mathbf{q}_n \right].$$

由于

$$\left| \frac{\lambda_j}{\lambda_1} \right| < 1, \quad j \geq 2,$$

当 $k \rightarrow \infty$ 时, 后面各项都会衰减, 于是方向上收敛到 $\mathbf{q}_1$ 。这说明幂法会找到绝对值最大的本征值对应本征向量。

实际迭代时, 为防止向量长度爆炸, 常在每步做归一化:

$$\mathbf{y}^{(k)} = A \mathbf{x}^{(k)}, \quad \mathbf{x}^{(k+1)} = \frac{\mathbf{y}^{(k)}}{\|\mathbf{y}^{(k)}\|}.$$

本征值可由 Rayleigh 商 (Rayleigh quotient) 近似:

$$\lambda^{(k)} = \frac{(\mathbf{x}^{(k)})^T A \mathbf{x}^{(k)}}{(\mathbf{x}^{(k)})^T \mathbf{x}^{(k)}}.$$

注 幂法每一步只需要保存两个向量, 因此内存代价很小。这是它在超大规模稀疏问题中仍然有重要意义的原因之一。

5.7.2 例: 用幂法求主特征值与对应特征向量。

设

$$A = \begin{bmatrix} -2 & -3 \\ 6 & 7 \end{bmatrix}, \quad \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

从初始向量 $\mathbf{x}_0$ 出发, 反复计算

$$\mathbf{x}_k = A \mathbf{x}_{k-1}, \quad k = 1, 2, \dots$$

于是

$$\begin{aligned}x_1 = Ax_0 &= \begin{bmatrix} -5 \\ 13 \end{bmatrix}, & x_2 = Ax_1 &= \begin{bmatrix} -29 \\ 61 \end{bmatrix}, & x_3 = Ax_2 &= \begin{bmatrix} -125 \\ 253 \end{bmatrix}, \\x_4 = Ax_3 &= \begin{bmatrix} -509 \\ 1021 \end{bmatrix}, & x_5 = Ax_4 &= \begin{bmatrix} -2045 \\ 4093 \end{bmatrix}, & x_6 = Ax_5 &= \begin{bmatrix} -8189 \\ 16381 \end{bmatrix}.\end{aligned}$$

由于幂法中迭代向量的增长率逐渐接近主特征值, 因此可以用相邻迭代向量对应分量之比估计主特征值。这里取第二个分量, 得到

$$\begin{aligned}\lambda_1^{(1)} &= \frac{61}{13} = 4.6923, \\ \lambda_1^{(2)} &= \frac{253}{61} = 4.14754, \\ \lambda_1^{(3)} &= \frac{1021}{253} = 4.03557, \\ \lambda_1^{(4)} &= \frac{4093}{1021} = 4.00881, \\ \lambda_1^{(5)} &= \frac{16381}{4093} = 4.00220.\end{aligned}$$

因此主特征值逐渐收敛到

$$\lambda_1 \approx 4.$$

另一方面, 将 x_6 按第二个分量归一化, 得

$$x_6 = \begin{bmatrix} -8189 \\ 16381 \end{bmatrix} \sim \begin{bmatrix} -\frac{8189}{16381} \\ 1 \end{bmatrix} \approx \begin{bmatrix} -0.49997 \\ 1 \end{bmatrix}.$$

因此主特征向量可取为

$$v_1 \approx \begin{bmatrix} -\frac{1}{2} \\ 1 \end{bmatrix}.$$

事实上, 矩阵 A 的特征多项式为

$$\det(\lambda I - A) = \begin{vmatrix} \lambda + 2 & 3 \\ -6 & \lambda - 7 \end{vmatrix} = (\lambda + 2)(\lambda - 7) + 18 = \lambda^2 - 5\lambda + 4.$$

所以

$$\lambda^2 - 5\lambda + 4 = 0,$$

即

$$\lambda_1 = 4, \quad \lambda_2 = 1.$$

当 $\lambda_1 = 4$ 时,

$$(A - 4I)v = 0,$$

即

$$\begin{bmatrix} -6 & -3 \\ 6 & 3 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = 0.$$

由

$$-6v_1 - 3v_2 = 0$$

得到

$$v_2 = -2v_1.$$

因此主特征向量可以取为

$$v_1 = \begin{bmatrix} 1 \\ -\frac{1}{2} \\ 1 \end{bmatrix},$$

这与幂法迭代所得结果一致。

说明：幂法中向量

$$x_k = A^k x_0$$

会逐渐趋近于模最大特征值对应的特征向量方向。因此只需保存当前向量和下一个向量，就可以不断更新特征值和特征向量的近似值。与此相比，QR 算法相当于同时对一整组基向量做迭代，并在每一步通过 QR 分解进行重新归一化和正交化。

5.7.3 对称矩阵情形与收敛直观

若 A 为实对称矩阵，则其本征向量组可取为标准正交基，因此幂法分析更直接，也更稳定。对称矩阵的 Rayleigh 商

$$R(x) = \frac{x^T A x}{x^T x}$$

在单位球面上的极值正好对应本征值，因此幂法可以被理解为：不断把向量推向“贡献最大的谱方向”。

在量子力学中，若通过适当平移或反演把目标变成求极端本征值，那么幂法与变分思想之间就会形成很自然的联系。这也解释了为什么在对称问题中它常表现出与基态变分逼近相似的收敛直观。

5.7.4 例：对称矩阵的幂法。

设

$$A = \begin{bmatrix} 4 & -1 & 1 \\ -1 & 3 & -2 \\ 1 & -2 & 3 \end{bmatrix}, \quad x^{(0)} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

首先有

$$Ax^{(0)} = \begin{bmatrix} 4 \\ -1 \\ 1 \end{bmatrix}.$$

由于 $x^{(0)}$ 已经归一化，因此第一次 Rayleigh 商为

$$\mu^{(1)} = \left(x^{(0)}\right)^T Ax^{(0)} = 4.$$

并且

$$\|Ax^{(0)}\|_2 = \sqrt{4^2 + (-1)^2 + 1^2} = \sqrt{18} = 3\sqrt{2}.$$

所以

$$x^{(1)} = \frac{Ax^{(0)}}{\|Ax^{(0)}\|_2} = \frac{1}{3\sqrt{2}} \begin{bmatrix} 4 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.942809 \\ -0.235702 \\ 0.235702 \end{bmatrix}.$$

继续迭代，得到如下结果：

k	$\mu^{(k)}$	$x^{(k)}$
1	4.00000	0.942809
		-0.235702
		0.235702
2	5.00000	0.816497
		-0.408248
		0.408248
3	5.66667	0.710669
		-0.497468
		0.497468
4	5.90909	0.646997
		-0.539164
		0.539164
5	5.97674	0.612836
		-0.558763
		0.558763
6	5.99415	0.595247
		-0.568190
		0.568190
7	5.99854	0.586336
		-0.572805
		0.572805

可以看到，Rayleigh 商逐渐趋近于

$$\mu^{(k)} \rightarrow 6,$$

而迭代向量逐渐趋近于

$$x^{(k)} \rightarrow \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}.$$

事实上，矩阵 A 的本征值为

$$\lambda_1 = 6, \quad \lambda_2 = 3, \quad \lambda_3 = 1.$$

对应于最大模本征值 $\lambda_1 = 6$ 的本征向量可取为

$$q_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}.$$

这与幂法迭代结果一致。

因此，在对称矩阵情形下，幂法可以理解为：

不断把向量推向模最大本征值对应的本征方向。

Rayleigh 商则给出当前方向对应的本征值近似：

$$\mu^{(k)} = \left(x^{(k-1)} \right)^T A x^{(k-1)}.$$

5.7.5 反幂法 (Inverse Power Method) 与位移幂法

若想求最小模本征值，而不是最大模本征值，可以对

$$A^{-1}$$

做幂法。因为若

$$A\mathbf{q}_i = \lambda_i \mathbf{q}_i,$$

则

$$A^{-1}\mathbf{q}_i = \frac{1}{\lambda_i} \mathbf{q}_i.$$

于是 A^{-1} 的最大模本征值对应于 A 的最小模本征值。

更一般地，可以考虑位移反幂法：

$$(A - qI)^{-1}\mathbf{q}_i = \frac{1}{\lambda_i - q} \mathbf{q}_i.$$

若位移 q 已经很靠近某个目标本征值 λ_j ，则

$$\left| \frac{1}{\lambda_j - q} \right|$$

会远大于其他特征值对应量，因此收敛非常快。这类方法通常称为 shifted power method 或 inverse power method with shift。

注 位移幂法并不显式计算逆矩阵，而是在每一步求解线性方程组

$$(A - qI)\mathbf{y}^{(k)} = \mathbf{x}^{(k)}.$$

因此它实质上是“线性求解器 + 归一化”的组合算法。

5.7.6 Wielandt 缩小法 (deflation)

已知最大本征值 λ_1 及对应本征向量 $\mathbf{v}^{(1)}$ 后，若还想继续用幂法找下一个本征值，就需要先把已知特征方向“剔除”掉。这就是缩小法 (deflation) 的思想。

Wielandt 缩小法的一种构造是：取某个指标 i ，满足

$$v_i^{(1)} \neq 0,$$

再定义行向量

$$\mathbf{x}^T = \frac{\text{第 } i \text{ 行的 } A}{\lambda_1 v_i^{(1)}}.$$

于是

$$\mathbf{x}^T \mathbf{v}^{(1)} = 1.$$

构造

$$B = A - \lambda_1 \mathbf{v}^{(1)} \mathbf{x}^T.$$

则

$$B\mathbf{v}^{(1)} = A\mathbf{v}^{(1)} - \lambda_1 \mathbf{v}^{(1)} (\mathbf{x}^T \mathbf{v}^{(1)}) = \lambda_1 \mathbf{v}^{(1)} - \lambda_1 \mathbf{v}^{(1)} = 0.$$

因此 $\mathbf{v}^{(1)}$ 成为 B 的零本征向量，而其余本征值则保留下来。

进一步，由于 $\mathbf{x}^T$ 的构造使得 B 的第 i 行全为 0，因此可删除第 i 行和第 i 列，得到一个 $(n-1) \times (n-1)$ 矩阵 B' 。可以证明， B' 的本征值正是

$$\lambda_2, \lambda_3, \dots, \lambda_n.$$

这样就能继续用幂法寻找次大本征值。

5.7.7 例：已知一个本征对后的降阶求解。

设

$$A = \begin{bmatrix} 4 & -1 & 1 \\ -1 & 3 & -2 \\ 1 & -2 & 3 \end{bmatrix}.$$

已知矩阵 A 的一个本征值和对应本征向量为

$$\lambda_1 = 6, \quad v^{(1)} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}.$$

为了将该本征值从矩阵中分离出来，取向量

$$x = \frac{1}{6} \begin{bmatrix} 4 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{2}{3} \\ -\frac{1}{6} \\ \frac{1}{6} \end{bmatrix}.$$

可以验证

$$x^T v^{(1)} = \begin{bmatrix} \frac{2}{3} & -\frac{1}{6} & \frac{1}{6} \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} = 1.$$

因此可以构造降阶矩阵

$$B = A - \lambda_1 v^{(1)} x^T.$$

先计算

$$v^{(1)} x^T = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} \begin{bmatrix} \frac{2}{3} & -\frac{1}{6} & \frac{1}{6} \end{bmatrix} = \begin{bmatrix} \frac{2}{3} & -\frac{1}{6} & \frac{1}{6} \\ -\frac{2}{3} & \frac{1}{6} & -\frac{1}{6} \\ \frac{2}{3} & -\frac{1}{6} & \frac{1}{6} \end{bmatrix}.$$

于是

$$B = \begin{bmatrix} 4 & -1 & 1 \\ -1 & 3 & -2 \\ 1 & -2 & 3 \end{bmatrix} - 6 \begin{bmatrix} \frac{2}{3} & -\frac{1}{6} & \frac{1}{6} \\ -\frac{2}{3} & \frac{1}{6} & -\frac{1}{6} \\ \frac{2}{3} & -\frac{1}{6} & \frac{1}{6} \end{bmatrix}.$$

计算得

$$B = \begin{bmatrix} 0 & 0 & 0 \\ 3 & 2 & -1 \\ -3 & -1 & 2 \end{bmatrix}.$$

由于第一行已经为零，可以转而考虑右下角的 2×2 子矩阵

$$B' = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}.$$

对 B' 求本征值，有

$$\det(\lambda I - B') = \begin{vmatrix} \lambda - 2 & 1 \\ 1 & \lambda - 2 \end{vmatrix} = (\lambda - 2)^2 - 1.$$

因此

$$(\lambda - 2)^2 - 1 = 0,$$

即

$$\lambda = 3 \quad \text{或} \quad \lambda = 1.$$

取

$$\lambda_2 = 3.$$

对应的本征向量满足

$$(B' - 3I)w' = 0,$$

即

$$\begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = 0.$$

故可取

$$w^{(2)'} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

将其嵌入到原来的三维空间中，得到

$$w^{(2)} = \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix}.$$

接下来由降阶矩阵的本征向量恢复原矩阵 A 的本征向量。公式为

$$v^{(2)} = (\lambda_2 - \lambda_1)w^{(2)} + \lambda_1 (x^T w^{(2)}) v^{(1)}.$$

先计算

$$x^T w^{(2)} = \begin{bmatrix} 2 & -\frac{1}{6} & \frac{1}{6} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} = -\frac{1}{6} - \frac{1}{6} = -\frac{1}{3}.$$

于是

$$v^{(2)} = (3 - 6) \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} + 6 \left(-\frac{1}{3} \right) \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}.$$

即

$$v^{(2)} = -3 \begin{bmatrix} 0 \\ 1 \\ -1 \end{bmatrix} - 2 \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -3 \\ 3 \end{bmatrix} + \begin{bmatrix} -2 \\ 2 \\ -2 \end{bmatrix} = \begin{bmatrix} -2 \\ -1 \\ 1 \end{bmatrix}.$$

因此，原矩阵 A 对应于 $\lambda_2 = 3$ 的一个本征向量为

$$v^{(2)} = \begin{bmatrix} -2 \\ -1 \\ 1 \end{bmatrix}.$$

最后验证：

$$Av^{(2)} = \begin{bmatrix} 4 & -1 & 1 \\ -1 & 3 & -2 \\ 1 & -2 & 3 \end{bmatrix} \begin{bmatrix} -2 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -6 \\ -3 \\ 3 \end{bmatrix} = 3 \begin{bmatrix} -2 \\ -1 \\ 1 \end{bmatrix}.$$

所以

$$Av^{(2)} = \lambda_2 v^{(2)}.$$

说明：这个过程的思想是，已知一个本征对 $(\lambda_1, v^{(1)})$ 后，通过秩一修正

$$B = A - \lambda_1 v^{(1)} x^T, \quad x^T v^{(1)} = 1,$$

把已经求出的本征值 λ_1 从问题中剥离出去。然后在降阶矩阵中继续求剩余本征值和本征向量，再恢复为原矩阵的本征向量。

5.8 Krylov 子空间、Lanczos 与 Arnoldi

5.8.1 Krylov 子空间的定义

给定矩阵 A 和初始向量 q_1 ，第 m 阶 Krylov 子空间 (Krylov subspace) 定义为

$$\mathcal{K}_m(A, q_1) = \text{span}\{q_1, Aq_1, A^2q_1, \dots, A^{m-1}q_1\}.$$

它的含义很直接：我们并不在整个高维空间里寻找本征向量，而是只在由

$$q_1, Aq_1, A^2q_1, \dots$$

张成的低维子空间里做逼近。这个想法是 Arnoldi、Lanczos 等现代大规模本征值算法的共同基础。

5.8.2 Lanczos 迭代：对称矩阵的三项递推

当 A 是实对称矩阵时，Arnoldi 正交化会大幅简化成三项递推，这就是 Lanczos 迭代 (Lanczos iteration)。设

$$Q_m = [q_1, q_2, \dots, q_m],$$

其中各列标准正交。Lanczos 的目标是构造三对角矩阵

$$T_m \in \mathbb{R}^{m \times m}$$

使得

$$Q_m^T A Q_m = T_m.$$

等价地，可写成

$$A Q_m = Q_m T_m + \beta_m q_{m+1} e_m^T.$$

由于 A 对称，新的向量 Aq_i 只需对前两个方向正交化，即

$$Aq_i = \beta_{i-1} q_{i-1} + \alpha_i q_i + \beta_i q_{i+1}.$$

两边左乘 q_j^T ，利用正交性可得

$$q_j^T A q_i = \beta_{i-1} \delta_{j,i-1} + \alpha_i \delta_{ji} + \beta_i \delta_{j,i+1}.$$

特别地，

$$\alpha_i = q_i^T A q_i.$$

因此实际迭代公式为：

$$v = Aq_i - \beta_{i-1} q_{i-1} - \alpha_i q_i,$$

$$\beta_i = \|v\|, \quad q_{i+1} = \frac{v}{\beta_i}.$$

定义 5.2 (Lanczos 迭代)

取

$$\beta_0 = 0, \quad \mathbf{q}_0 = \mathbf{0}, \quad \|\mathbf{q}_1\| = 1.$$

对 $i = 1, 2, \dots, m$:

1. 计算

$$\alpha_i = \mathbf{q}_i^T A \mathbf{q}_i;$$

2. 令

$$\mathbf{v} = A \mathbf{q}_i - \beta_{i-1} \mathbf{q}_{i-1} - \alpha_i \mathbf{q}_i;$$

3. 令

$$\beta_i = \|\mathbf{v}\|;$$

4. 若 $\beta_i \neq 0$, 则

$$\mathbf{q}_{i+1} = \frac{\mathbf{v}}{\beta_i}.$$

**5.8.3 Lanczos 为什么适合大规模稀疏问题**

Lanczos 的优点在于:

- 每一步只需要一次矩阵向量乘法 $A \mathbf{q}_i$;
- 在理想三项递推下, 只需同时保存三个向量

$$\mathbf{q}_{i-1}, \mathbf{q}_i, \mathbf{q}_{i+1};$$

- 得到的是一个很小的三对角矩阵 T_m , 其本征值称为 Ritz 值 (Ritz values), 可作为原矩阵极端本征值的高质量逼近。

当 $m \ll n$ 时, 这相当于把巨大矩阵“压缩”成一个小三对角矩阵再求解, 其思想和图像压缩中只保留主要模式有相似之处。

5.8.4 Lanczos 两步例子

考虑矩阵

$$A = \begin{bmatrix} 3 & 2 & 5 \\ 2 & 3 & 5 \\ 5 & 5 & 0 \end{bmatrix}, \quad \mathbf{q}_0 = \mathbf{0}, \quad \mathbf{q}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

先算

$$A \mathbf{q}_1 = \begin{bmatrix} 3 \\ 2 \\ 5 \end{bmatrix}, \quad \alpha_1 = \mathbf{q}_1^T A \mathbf{q}_1 = 3.$$

于是

$$\mathbf{v} = A \mathbf{q}_1 - \alpha_1 \mathbf{q}_1 = \begin{bmatrix} 0 \\ 2 \\ 5 \end{bmatrix}, \quad \beta_1 = \|\mathbf{v}\| = \sqrt{29}.$$

因此

$$\mathbf{q}_2 = \frac{1}{\sqrt{29}} \begin{bmatrix} 0 \\ 2 \\ 5 \end{bmatrix}.$$

再算

$$\alpha_2 = \mathbf{q}_2^T A \mathbf{q}_2 = \frac{112}{29}.$$

于是两步 Lanczos 得到的三对角矩阵为

$$T_2 = \begin{bmatrix} \alpha_1 & \beta_1 \\ \beta_1 & \alpha_2 \end{bmatrix} = \begin{bmatrix} 3 & \sqrt{29} \\ \sqrt{29} & \frac{112}{29} \end{bmatrix}.$$

对该 2×2 矩阵对角化，可得近似本征值

$$8.83, \quad -1.97.$$

而原矩阵的严格本征值为

$$10, \quad 1, \quad -5.$$

这个例子非常直观地展示了：即便只做很低阶的 Lanczos 截断，也已经能较快逼近最外侧的本征值。

5.8.5 Arnoldi 迭代：非对称矩阵的 Krylov 正交化

当矩阵 A 不再对称时，三项递推不再成立，就需要完整正交化，这就是 Arnoldi iteration。

Arnoldi 的递推为：

1. 给定单位初始向量 $\mathbf{q}_1$ ；
2. 令

$$\mathbf{v} = A \mathbf{q}_k;$$

3. 对所有 $j = 1, \dots, k$ 做投影消去：

$$h_{jk} = \mathbf{q}_j^T \mathbf{v}, \quad \mathbf{v} \leftarrow \mathbf{v} - h_{jk} \mathbf{q}_j;$$

4. 最后令

$$h_{k+1,k} = \|\mathbf{v}\|, \quad \mathbf{q}_{k+1} = \frac{\mathbf{v}}{h_{k+1,k}}.$$

这样得到关系

$$A \mathbf{Q}_m = \mathbf{Q}_m H_m + h_{m+1,m} \mathbf{q}_{m+1} \mathbf{e}_m^T,$$

其中 H_m 是一个上 Hessenberg 矩阵。随后只需对这个小型 H_m 做本征值求解，即可得到原矩阵的 Ritz 近似。

注 ARPACK 软件就是基于 Arnoldi/Lanczos 思想构建的大规模稀疏本征值程序库。它的优势在于：用户只需提供“矩阵乘向量”的接口，而不必显式存储完整矩阵。

若 Arnoldi 中某一步出现新向量接近零向量，说明 Krylov 子空间不再扩张，算法发生 breakdown。这在理论上意味着某个不变子空间已经被完整捕捉。

5.8.6 例：Arnoldi 迭代的数值计算。

取非对称矩阵

$$A = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 2 & 1 \end{bmatrix},$$

显然 $A \neq A^T$ 。取单位初始向量

$$\mathbf{q}_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

Arnoldi 迭代的目标是构造 Krylov 子空间

$$\mathcal{K}_m(A, q_1) = \text{span}\{q_1, Aq_1, \dots, A^{m-1}q_1\}$$

的一组标准正交基

$$Q_m = \begin{bmatrix} q_1 & q_2 & \cdots & q_m \end{bmatrix},$$

并得到关系

$$AQ_m = Q_m H_m + h_{m+1,m} q_{m+1} e_m^T,$$

其中 H_m 是上 Hessenberg 矩阵。

第一步。

先计算

$$v = Aq_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} 3 \\ 5 \\ 3 \end{bmatrix}.$$

投影到 q_1 方向:

$$h_{11} = q_1^T v = \frac{1}{3}(3 + 5 + 3) = \frac{11}{3}.$$

消去 q_1 分量:

$$v \leftarrow v - h_{11}q_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} 3 \\ 5 \\ 3 \end{bmatrix} - \frac{11}{3} \cdot \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \frac{1}{3\sqrt{3}} \begin{bmatrix} -2 \\ 4 \\ -2 \end{bmatrix}.$$

于是

$$h_{21} = \|v\|_2 = \frac{2\sqrt{2}}{3},$$

从而

$$q_2 = \frac{v}{h_{21}} = \frac{1}{\sqrt{6}} \begin{bmatrix} -1 \\ 2 \\ -1 \end{bmatrix}.$$

第二步。

计算

$$v = Aq_2 = \frac{1}{\sqrt{6}} \begin{bmatrix} 0 \\ 4 \\ 3 \end{bmatrix}.$$

先投影到 q_1 方向:

$$h_{12} = q_1^T v = \frac{7\sqrt{2}}{6}.$$

消去 q_1 分量后, 继续投影到 q_2 方向:

$$h_{22} = q_2^T v = \frac{5}{6}.$$

因此

$$v \leftarrow v - h_{12}q_1 - h_{22}q_2.$$

直接计算得

$$v = \frac{1}{2\sqrt{6}} \begin{bmatrix} -3 \\ 0 \\ 3 \end{bmatrix}.$$

于是

$$h_{32} = \|v\|_2 = \frac{\sqrt{3}}{2},$$

从而

$$q_3 = \frac{v}{h_{32}} = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}.$$

因此, 前两步 Arnoldi 迭代给出

$$Q_2 = \begin{bmatrix} \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & \frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{6}} \end{bmatrix},$$

以及

$$H_2 = \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{bmatrix} = \begin{bmatrix} \frac{11}{3} & \frac{7\sqrt{2}}{6} \\ \frac{2\sqrt{2}}{3} & \frac{5}{6} \end{bmatrix}.$$

此时 Arnoldi 关系为

$$AQ_2 = Q_2 H_2 + h_{32} q_3 e_2^T,$$

即

$$AQ_2 = Q_2 H_2 + \frac{\sqrt{3}}{2} q_3 e_2^T.$$

Ritz 值近似。

对小矩阵 H_2 求本征值:

$$\det(\theta I - H_2) = 0.$$

计算得

$$\det(\theta I - H_2) = \theta^2 - \frac{9}{2}\theta + \frac{3}{2}.$$

即

$$2\theta^2 - 9\theta + 3 = 0.$$

所以

$$\theta_{1,2} = \frac{9 \pm \sqrt{57}}{4}.$$

数值上

$$\theta_1 \approx 4.13746, \quad \theta_2 \approx 0.36254.$$

这两个数就是由二维 Krylov 子空间得到的 Ritz 值, 用来近似原矩阵 A 的部分本征值。

第三步。

继续做一步。计算

$$v = Aq_3 = \frac{1}{\sqrt{2}} \begin{bmatrix} -2 \\ 0 \\ 1 \end{bmatrix}.$$

分别计算投影系数：

$$h_{13} = q_1^T v = -\frac{\sqrt{6}}{6},$$

$$h_{23} = q_2^T v = \frac{\sqrt{3}}{6},$$

$$h_{33} = q_3^T v = \frac{3}{2}.$$

消去 q_1, q_2, q_3 的分量后得到零向量，因此

$$h_{43} = 0.$$

于是算法发生精确 breakdown。由于本例矩阵为 3×3 ，这说明三维 Krylov 子空间已经覆盖了整个空间。

因此

$$Q_3 = \begin{bmatrix} \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{6}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{3}} & \frac{2}{\sqrt{6}} & 0 \\ \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{6}} & \frac{1}{\sqrt{2}} \end{bmatrix},$$

$$H_3 = \begin{bmatrix} \frac{11}{3} & \frac{7\sqrt{2}}{6} & -\frac{\sqrt{6}}{6} \\ \frac{2\sqrt{2}}{3} & \frac{5}{6} & \frac{\sqrt{3}}{6} \\ 0 & \frac{\sqrt{3}}{2} & \frac{3}{2} \end{bmatrix}.$$

因为 $h_{43} = 0$ ，所以 Arnoldi 关系退化为精确关系

$$AQ_3 = Q_3 H_3.$$

于是

$$H_3 = Q_3^T A Q_3.$$

这说明 H_3 与 A 正交相似，因此二者具有完全相同的本征值。

事实上，

$$\det(\lambda I - A) = \lambda^3 - 6\lambda^2 + 8\lambda - 1.$$

其三个本征值约为

$$\lambda_1 \approx 4.11491, \quad \lambda_2 \approx 1.74590, \quad \lambda_3 \approx 0.13919.$$

而二维 Ritz 值中

$$\theta_1 \approx 4.13746$$

已经较好地近似了最大的本征值

$$\lambda_1 \approx 4.11491.$$

说明： Arnoldi 方法的本质是把大矩阵 A 投影到 Krylov 子空间上，得到一个小的上 Hessenberg 矩阵 H_m 。随后只需求解小矩阵 H_m 的本征值问题，即可得到原矩阵 A 的 Ritz 近似。

5.9 直接法与迭代法的比较

5.9.1 直接法：适合中等规模稠密问题

对实对称稠密矩阵 H ，若需要许多本征态，则直接法的典型流程是：

1. 用 Householder 变换把 H 约化为三对角矩阵 T ；
2. 再对 T 用 QL/QR 等算法完成对角化。

典型复杂度量级为：

- Householder 约化到三对角：约

$$\frac{2}{3}n^3$$

次运算；若还要求全部本征向量，则大约

$$\frac{4}{3}n^3;$$

- 三对角矩阵的 QL/QR 对角化：本征值部分约

$$30n^2$$

次运算；若显式回构全部本征向量，则总成本仍可达到 $O(n^3)$ 量级。

5.9.2 迭代法：适合超大规模稀疏问题

若矩阵巨大且稀疏，只需要最少几个本征值与本征向量，那么幂法、Lanczos、Arnoldi 等迭代法就更合适。可以概括为：

- **Power method**：概念最简单，但收敛一般较慢；优点是只需两个向量；
- **Lanczos**：对对称稀疏矩阵非常高效，一次只需维护三个向量；
- **Arnoldi**：适合非对称稀疏矩阵，但需要保留更多历史向量做正交化。

注 对于短程相互作用（short-range interaction）的量子多体 Hamiltonian，矩阵往往极稀疏而维数极高。此时直接法根本存不下矩阵，而 Lanczos 却仍可依赖“稀疏矩阵乘向量”工作到 $10^9 \sim 10^{10}$ 量级的基底维数。

5.10 重整化思想：NRG 与 DMRG

5.10.1 为什么精确对角化会失效

对长度为 L 的自旋链，若每个格点有 2 个局域态，则总 Hilbert 空间维数就是

$$2^L.$$

因此即使 L 稍大，精确对角化 (exact diagonalization) 也会立刻遭遇维数爆炸。这就推动了“重整化 (renormalization) + 截断 (truncation)”思想的发展。

5.10.2 Wilson 的数值重整化群 (Numerical Renormalization Group, NRG)

Wilson 数值重整化群的基本流程为：

1. 在初始子格块 A 上建立块 Hamiltonian H_A ，其 Hilbert 空间维数为 M ；
2. 将两个块拼成长度加倍的复合块 AA ，得到 Hamiltonian H_{AA} ，其维数约为 M^2 ；
3. 对 H_{AA} 做对角化（实践中常用 Lanczos），保留最低的 M 个能量本征态；
4. 把 H_{AA} 投影到这 M 维截断子空间，得到新的有效 Hamiltonian；
5. 把新块重新命名为 A ，继续重复上述过程。

注 这种方法最初在杂质问题 (impurity problem) 和 Kondo 问题中取得巨大成功。Kenneth Wilson 也因重整化群与临界现象研究而获得 1982 年诺贝尔物理学奖。

NRG 的要点是：不试图保留全部 Hilbert 空间，而只保留低能、最相关的少数态。这本质上是一种基于能量排序的截断。

5.10.3 密度矩阵重整化群 (DMRG) 的基本思想

DMRG (Density Matrix Renormalization Group) 比 NRG 更进一步。它不是简单保留最低能量本征态，而是根据约化密度矩阵 (reduced density matrix) 的本征值来决定哪些态最重要。

把系统分成两部分：

Part A and Part B.

设总波函数写成

$$|\psi\rangle = \sum_{i,j} \psi_{ij} |i\rangle_A \otimes |j\rangle_B.$$

对系数矩阵 $\Psi = (\psi_{ij})$ 做 SVD:

$$\Psi = USV^*.$$

则

$$|\psi\rangle = \sum_{\alpha=1}^r s_{\alpha} |\alpha\rangle_A \otimes |\alpha\rangle_B, \quad r = \text{rank}(\Psi),$$

这就是 Schmidt 分解 (Schmidt decomposition)。

于是 A 部分与 B 部分的约化密度矩阵分别为

$$\rho_A = \sum_{\alpha} s_{\alpha}^2 |\alpha\rangle_{AA} \langle \alpha|, \quad \rho_B = \sum_{\alpha} s_{\alpha}^2 |\alpha\rangle_{BB} \langle \alpha|.$$

因此，保留最大的若干 s_{α} ，等价于保留 ρ_A 或 ρ_B 最大本征值对应的最重要态。

结论 DMRG 的截断依据不是“哪个本征态能量低”，而是“哪个 Schmidt 权重 s_{α}^2 大”。也就是说，它是在量子纠缠与信息量意义下做最优截断。

这句话 **Truncate in basis of density matrix, not energy eigenstates** 的含义是：截断应在密度矩阵最重要的本征基中进行，而不是仅按能量本征态排序。

 **笔记** 为什么 DMRG 尤其适合低纠缠问题：若系统的纠缠熵较低 (low entanglement)，则只有少数几个 s_{α} 较大，剩余权重衰减很快。此时只保留很少的态就能逼近整个波函数，因此 DMRG 特别适用于一维局域模型、量子自旋链等低纠缠体系。

5.10.4 DMRG 中的 warm-up 与 sweeping

DMRG 的实际流程可分为两个阶段。

 **笔记 warm-up phase:**

1. 求基态 (通常用 Lanczos);
2. 构造扩展后的系统块与环境块;
3. 构造约化密度矩阵 (reduced density matrix);
4. 对约化密度矩阵对角化;
5. 把块 Hamiltonian 旋转到密度矩阵本征基中，并做截断;
6. 重复以上过程，逐步增大系统长度。

 **笔记 sweeping phase:** 在系统达到目标长度后，不再简单地一直向外扩张，而是在左到右、右到左之间反复“扫动 (sweeping)”：

- 一次 sweep 中固定环境块，优化系统块;

- 接着固定系统块，优化环境块；
- 通过多轮往返迭代，使截断基越来越适配真实基态。

注 warm-up 负责把系统“搭起来”，sweeping 则负责把基底“磨精”。后者是 DMRG 相比早期 NRG 更精确的关键原因，因为它显式保留了环境（environment）的反馈作用。

5.10.5 DMRG 的应用范围

DMRG 的典型应用包括：

- 晶格模型（lattice models）；
- 量子自旋链（spin chains）；
- 量子点（quantum dots）；
- 原子核结构问题（nuclei）；
- 量子化学（chemistry）；
- 量子信息（quantum information）。

这些问题的共同特点是：虽然总 Hilbert 空间巨大，但物理上真正重要的低能态只占其中一个极小而结构化的子空间。DMRG 正是利用这一点，把“不可能存下来的巨空间问题”压缩成“可控截断下的高精度问题”。

5.11 本章总结

本章的核心线索可以概括为：

1. 本征值问题在量子力学与量子多体理论中无处不在，最终往往表现为大规模矩阵对角化；
2. 相似变换保证本征值不变，因此可以放心地把矩阵约化成更容易处理的形式；
3. Jacobi 方法通过局部平面旋转逐步消除对称矩阵的非对角元；
4. Householder 变换用反射把一般矩阵化为 Hessenberg、把对称矩阵化为三对角；
5. QR 算法与位移 QR 是现代直接本征值算法的核心；
6. 广义本征值问题可借助 Cholesky 分解化成标准对称问题；
7. SVD 把一般矩阵分解成“正交变换 + 非负伸缩 + 正交变换”，并可用于压缩；
8. 幂法、反幂法、Wielandt 缩小法适合求少数极端本征值；
9. Krylov 子空间方法是大型稀疏本征值问题的主力。
其中对称情形最典型的是 Lanczos，非对称情形则以 Arnoldi 为代表；
10. 对更大的量子多体问题，NRG 与 DMRG 引入重整化与截断思想，使得“巨型 Hilbert 空间中的少数重要态”能够被高效逼近。

注 从算法史角度看，QR algorithm 与 Krylov subspace methods 都属于 20 世纪最具影响力的核心算法。它们共同说明：真正高水平的本征值算法，并不是去“硬解”整个矩阵，而是尽可能利用矩阵结构、谱结构与物理结构。

第六章 蒙特卡洛方法 (Monte Carlo Methods)

内容提要

- 随机数产生、伪随机数、概率分布采样与 Box-Muller 方法
- 蒙特卡洛积分、采样误差、重要性采样 (importance sampling) 与自助法 (bootstrap)
- 随机行走 (random walk)、Markov 随机过程与扩散方程
- Metropolis 算法、Metropolis-Hastings 算法与 Markov Chain Monte Carlo (MCMC)
- 二维 Ising 模型的 Monte Carlo 求解与平衡态统计
- 量子蒙特卡洛 (Quantum Monte Carlo, QMC): VMC 与 DMC

6.1 Monte Carlo 方法的基本思想

6.1.1 从统计抽样到数值计算

蒙特卡洛方法 (Monte Carlo method) 是一类用**随机抽样 (statistical sampling)** 解决确定性或随机性问题的数值方法。它的核心并不是“随便试”，而是把目标量写成某个概率分布下的平均值，再通过大量样本的统计平均来逼近真实结果。

定义 6.1

Monte Carlo 方法 (Monte Carlo method) 是指通过从给定概率分布中生成随机样本，利用样本平均、频率、随机行走或 Markov 链来估计积分、期望值、平衡分布、优化结果或复杂系统统计性质的一类方法。

它特别适合处理以下问题：

- 高维积分 (high-dimensional integration)；
- 多自由度系统模拟 (many degrees of freedom)；
- 带有不确定性或随机过程的模型；
- 很难用网格法、解析法或传统确定性算法处理的问题；
- 由某个概率分布生成的 Markov 过程。

注 Monte Carlo 的优势常常不在低维问题中体现。低维积分用 Simpson 公式、Gauss 积分等确定性方法通常更高效；但当维数升高到几十、几百甚至更多时，传统网格法会遭遇维数灾难，而 Monte Carlo 误差的主阶通常仍是 $O(N^{-1/2})$ ，不显式随维数指数增长。

6.1.2 历史背景与典型应用

Monte Carlo 方法的现代形式通常追溯到 1946 年左右。Stanislaw Ulam、John von Neumann、Nicholas Metropolis 等人在 Manhattan Project 相关研究中系统使用了这种随机抽样思想。其早期重要动机之一，是模拟中子在反应堆或核材料中的传输。

中子与原子核发生相互作用时，散射、吸收、裂变等过程受量子力学概率规律控制。人们通常不能准确预言某一个中子何时何地地发生反应，也不能确定裂变产生的新中子的方向和速度；但可以知道这些事件的概率分布。于是可以按概率抽样出大量中子的碰撞位置、速度、方向和反应类型，再用统计平均得到核物质中的中子输运性质。这一思想后来发展成辐射输运模拟中的重要工具，例如 MCNP (Monte Carlo N-Particle transport code)。

典型应用包括：

- 统计物理中的平衡态抽样；
- 分子模型与分子模拟 (Monte Carlo molecular model)；

- 多体量子系统的量子 Monte Carlo;
- 材料科学中的辐射运输;
- 粒子物理探测器设计;
- 工程、人工智能、生物学、天气预报、流体动力学等方向。

6.2 概率统计基础

6.2.1 PDF、CDF 与期望值

Monte Carlo 方法建立在概率分布的语言上。设连续随机变量 X 的概率密度函数为 $p(x)$, 则

$$p(x) \geq 0, \quad \int_{-\infty}^{\infty} p(x) dx = 1.$$

定义 6.2 (PDF 与 CDF)

概率密度函数 (**probability distribution function, PDF**) $p(x)$ 描述随机变量落在微小区间中的概率密度;
累积分布函数 (**cumulative distribution function, CDF**) 定义为

$$P(x) = \text{Prob}(X \leq x) = \int_{-\infty}^x p(x') dx'.$$

若 $f(x)$ 是随机变量的函数, 则它在概率密度 $p(x)$ 下的期望值 (expectation value) 为

$$E_p[f(x)] = \int f(x)p(x) dx.$$

对多变量情形, 若 $\mathbf{x} = (x_1, \dots, x_n)$, 概率密度为 $P(x_1, \dots, x_n)$, 则

$$\langle H \rangle_{x_1, \dots, x_n} = \int \cdots \int H(x_1, \dots, x_n) P(x_1, \dots, x_n) dx_1 \cdots dx_n.$$

若各变量相互独立 (uncorrelated / independent variables), 则联合概率密度可以分解为

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n p_i(x_i).$$

定义 6.3 (中心矩)

随机变量 x 关于均值 $\langle x \rangle$ 的 n 阶中心矩 (n -th central moment) 定义为

$$\langle (x - \langle x \rangle)^n \rangle = \int (x - \langle x \rangle)^n p(x) dx.$$

当 $n = 2$ 时, 它就是方差 (variance)。

6.2.2 中心极限定理

设 $x_1, x_2, \dots, x_m$ 是从同一分布 $p(x)$ 中独立抽取的随机变量, 均值为 μ , 方差为 σ^2 。定义样本均值

$$z = \frac{x_1 + x_2 + \cdots + x_m}{m}.$$

当 m 足够大时, z 的分布会趋近于正态分布, 而且这个结论对原始分布 $p(x)$ 的具体形状并不敏感。

定理 6.1 (中心极限定理 (Central Limit Theorem, CLT))

若 x_i 独立同分布, 且具有有限均值 μ 与有限方差 σ^2 , 则样本均值 z 的概率密度在 $m \rightarrow \infty$ 时趋近于

$$\tilde{p}(z) = \frac{1}{\sqrt{2\pi}(\sigma/\sqrt{m})} \exp\left[-\frac{(z - \mu)^2}{2(\sigma/\sqrt{m})^2}\right].$$

因此

$$\text{Var}(z) = \sigma_m^2 = \frac{\sigma^2}{m}.$$



证明 [直观推导] 样本均值是 m 个独立随机变量的线性组合。对独立变量求和时，总方差相加：

$$\text{Var}(x_1 + \cdots + x_m) = m\sigma^2.$$

因此

$$\text{Var}\left(\frac{x_1 + \cdots + x_m}{m}\right) = \frac{1}{m^2} m\sigma^2 = \frac{\sigma^2}{m}.$$

更严格地说，特征函数或矩母函数展开会说明标准化变量

$$\frac{z - \mu}{\sigma/\sqrt{m}}$$

趋近标准正态分布。

结论 Monte Carlo 估计量的标准误差通常按 $1/\sqrt{N}$ 衰减。样本数增加 100 倍，误差大约只减少 10 倍。这是 Monte Carlo 方法最基本、也最需要记住的收敛规律。

6.3 随机数与概率分布采样

6.3.1 伪随机数生成器

数值计算中的随机数通常是**伪随机数 (pseudorandom numbers)**，即由确定性算法生成、但在统计性质上尽量接近随机序列的一串数。一个合格的随机数生成器应满足：

1. 在区间 $[0, 1]$ 上近似均匀分布 (uniform distribution)；
2. 相邻或相隔若干步的随机数之间相关性尽量小；
3. 周期 (period) 尽可能长，避免序列过早重复；
4. 生成速度快，适合大规模采样。

早期常见的思路包括线性同余发生器 (linear congruential generator, LCG)

$$N_i = (aN_{i-1} + c) \bmod M,$$

或带延迟项的递推形式

$$N_i = (aN_{i-1} + cN_{i-3}) \bmod (2^{31} - 69).$$

也可以用简单非线性映射展示伪随机序列的周期性与相关性，例如

$$x_{i+1} = cx_i(1 - x_i).$$



笔记 科学计算中通常不建议自己临时发明随机数生成器。除非研究对象本身就是随机数算法，否则应优先使用经过充分测试的系统库或专业库，例如 Mersenne Twister、PCG、xoshiro 或高质量并行随机数库。

6.3.2 正态分布与 Box-Muller 方法

许多物理问题需要从高斯分布或正态分布 (normal distribution / Gaussian distribution) 中采样。设

$$p(x) = \frac{1}{b\sqrt{2\pi}} \exp\left[-\frac{(x-a)^2}{2b^2}\right],$$

则其均值与方差分别为

$$\mu = \int_{-\infty}^{\infty} xp(x) dx = a, \quad \sigma^2 = \int_{-\infty}^{\infty} (x - \mu)^2 p(x) dx = b^2.$$

证明中常用高斯积分

$$\int_{-\infty}^{\infty} e^{-ax^2} dx = \sqrt{\frac{\pi}{a}}, \quad \int_{-\infty}^{\infty} x^2 e^{-ax^2} dx = \frac{1}{2a} \sqrt{\frac{\pi}{a}}.$$

定义 6.4 (Box-Muller 方法)

若 u_1, u_2 是 $(0, 1)$ 上的独立均匀随机数, 则

$$z_1 = \sqrt{-2 \ln u_1} \cos(2\pi u_2), \quad z_2 = \sqrt{-2 \ln u_1} \sin(2\pi u_2)$$

是两个相互独立的标准正态随机变量。

若要得到均值 a 、标准差 b 的正态变量, 只需令

$$x = a + bz.$$

6.4 Monte Carlo 积分

6.4.1 空间随机投点法

最直观的 Monte Carlo 积分是**随机投点法 (hit-or-miss Monte Carlo)**。设在单位正方形 $[0, 1] \times [0, 1]$ 内有区域

$$\Omega = \{(x, y) : 0 \leq y \leq f(x)\},$$

其中 $0 \leq f(x) \leq 1$ 。产生 N 组均匀随机点

$$(x_i, y_i), \quad i = 1, 2, \dots, N.$$

若满足

$$y_i \leq f(x_i),$$

则认为该点落入区域 Ω 。设落入区域的点数为 n , 则

$$I = \int_0^1 f(x) dx \approx \frac{n}{N}.$$

注 随机投点法把积分转化成面积比例估计。它概念简单, 但通常不如直接平均函数值高效, 因为每个样本只贡献 0 或 1, 没有充分利用函数值大小。

6.4.2 求期望值法

更常用的 Monte Carlo 积分是把积分看成期望值。若在 $[0, 1]$ 区间均匀采样 x_i , 则

$$I = \int_0^1 f(x) dx \approx \bar{f} = \frac{1}{N} \sum_{i=1}^N f(x_i).$$

若采样分布为一般概率密度 $p(x)$, 则

$$\langle f \rangle = \int f(x) p(x) dx \approx \frac{1}{N} \sum_{i=1}^N f(x_i), \quad x_i \sim p(x).$$

对均匀采样, 样本方差可写为

$$\sigma_f^2 = \frac{1}{N} \sum_{i=1}^N f(x_i)^2 - \left(\frac{1}{N} \sum_{i=1}^N f(x_i) \right)^2 = \langle f^2 \rangle - \langle f \rangle^2.$$

由中心极限定理, 样本平均的方差近似为

$$\sigma_N^2 \approx \frac{\sigma_f^2}{N}, \quad \sigma_N \sim \frac{1}{\sqrt{N}}.$$

6.4.3 任意区间与多维积分

若积分区间为 $[a, b]$ ，令

$$x_i = a + (b - a)t_i, \quad t_i \sim U(0, 1),$$

则

$$I = \int_a^b f(x) dx \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i).$$

对 d 维积分，设积分区域体积为 V ，均匀抽取点 $\mathbf{r}_i$ ，则

$$I = \int_V f(\mathbf{r}) d\mathbf{r} \approx \frac{V}{N} \sum_{i=1}^N f(\mathbf{r}_i).$$

更一般地，若

$$x_i = a_i + (b_i - a_i)t_i, \quad t_i \in [0, 1],$$

则

$$\int_{a_1}^{b_1} \cdots \int_{a_d}^{b_d} f(x_1, \dots, x_d) dx_1 \cdots dx_d = \left[\prod_{i=1}^d (b_i - a_i) \right] \int_0^1 \cdots \int_0^1 f(\mathbf{x}(t)) dt.$$

误差

$$\text{Error} = \frac{\sigma}{\sqrt{n}}$$

其中 σ 是函数 $f(x)$ 的标准差。

结论 Monte Carlo 积分的误差主阶与维度没有显式指数关系，因此它很适合高维积分、复杂区域积分以及量子多体体系中的高维坐标积分。

6.4.4 量子多体体系中的积分

量子多体系统中常遇到如下能量期望值：

$$E = \frac{\int dR_1 \cdots dR_N \Psi^*(R_1, \dots, R_N) H(R_1, \dots, R_N) \Psi(R_1, \dots, R_N)}{\int dR_1 \cdots dR_N \Psi^*(R_1, \dots, R_N) \Psi(R_1, \dots, R_N)}.$$

这里 Ψ 是体系波函数， R_i 是第 i 个粒子的坐标。由于坐标维数随粒子数线性增长，而 Hilbert 空间结构又很复杂，这类积分非常适合用 Monte Carlo 抽样处理。

6.5 重要性采样与 Bootstrap

6.5.1 为什么需要改进采样

若被积函数 $f(x)$ 在 $[0, 1]$ 中只有很窄的尖峰，而在大部分区域都接近 0，则均匀采样会浪费大量样本。许多样本给出的 $f(x_i)$ 近乎没有贡献，导致方差很大。解决思路是：样本应更多落在对积分贡献大的区域。

定义 6.5

重要性采样 (**importance sampling**) 是指不再从均匀分布抽样，而是从一个更接近被积函数主要贡献区域的概率密度 $p(x)$ 抽样，并用权重修正由此带来的分布偏差。

6.5.2 重要性采样公式

考虑

$$I = \int_a^b F(y) dy.$$

若选择归一化概率密度 $p(y)$, 且在 $F(y)$ 主要非零的区域内 $p(y)$ 也较大, 则

$$I = \int_a^b p(y) \frac{F(y)}{p(y)} dy \approx \frac{1}{N} \sum_{i=1}^N \frac{F(y_i)}{p(y_i)}, \quad y_i \sim p(y).$$

当 $p(y)$ 与 $|F(y)|$ 的形状接近时, 随机变量 $F(y_i)/p(y_i)$ 的波动会显著减小, 方差也随之降低。

 **笔记** 均匀分布 $p(y) = 1/(b-a)$ 只是重要性采样的一个特殊情形。它实现简单, 但对尖峰函数、指数衰减函数或高维局域分布往往效率很低。

6.5.3 变量变换实现非均匀采样

一种常用方式是利用累积分布函数实现变量变换。令

$$x(y) = \int_a^y p(y') dy',$$

则 $x \in [0, 1]$ 是均匀变量。若能反解出

$$y = y(x),$$

就可以先生成均匀随机数 x_i , 再由 $y_i = y(x_i)$ 得到服从 $p(y)$ 的样本。

由于

$$p(y) dy = dx,$$

有

$$I = \int_a^b p(y) \frac{F(y)}{p(y)} dy = \int_0^1 \frac{F(y(x))}{p(y(x))} dx \approx \frac{1}{N} \sum_{i=1}^N \frac{F(y(x_i))}{p(y(x_i))}.$$

例题 6.1 指数分布采样 若希望从

$$p(y) = e^{-y}, \quad y \in [0, \infty)$$

抽样, 则

$$x(y) = \int_0^y e^{-y'} dy' = 1 - e^{-y}.$$

反解得

$$y(x) = -\ln(1-x).$$

于是

$$\int_0^\infty e^{-y} G(y) dy = \int_0^1 G(y(x)) dx \approx \frac{1}{N} \sum_{i=1}^N G(y(x_i)).$$

6.5.4 具体积分例子

考虑积分

$$I = \int_3^5 e^{-x} x^2 dx,$$

给定 5 个 $[0, 1]$ 上的随机数

$$0.033, \quad 0.45, \quad 0.51, \quad 0.83, \quad 0.95.$$

直接均匀采样:

令

$$x_i = 3 + (5-3)t_i.$$

对应样本点为

$$3.066, \quad 3.900, \quad 4.020, \quad 4.660, \quad 4.900.$$

直接 Monte Carlo 估计为

$$I \approx \frac{5-3}{5} \sum_{i=1}^5 e^{-x_i} x_i^2 \approx 0.5682.$$

样本方差可由

$$\sigma_f^2 = \langle f^2 \rangle - \langle f \rangle^2, \quad \sigma_I^2 = \frac{(b-a)^2 \sigma_f^2}{N}$$

估计。

指数权重的重要性采样：

将

$$y = e^{-3} - e^{-x}, \quad x(y) = -\ln(e^{-3} - y),$$

则

$$e^{-x} dx = dy,$$

积分变成

$$I = \int_0^{e^{-3}-e^{-5}} x(y)^2 dy.$$

令

$$y_i = (e^{-3} - e^{-5}) t_i,$$

则

$$I \approx \frac{e^{-3} - e^{-5}}{5} \sum_{i=1}^5 x(y_i)^2 \approx 0.6431.$$

该积分的精确值为

$$\int_3^5 e^{-x} x^2 dx = e^{-3}(3^2 + 2 \cdot 3 + 2) - e^{-5}(5^2 + 2 \cdot 5 + 2) \approx 0.5971.$$

注 重要性采样并不保证在每一组很小样本上都更接近精确值。它的优势是降低估计量的理论方差，从而在大量样本极限下提高效率。若样本数只有 5，随机波动仍可能非常明显。

6.5.5 Bootstrap 误差估计

自助法 (bootstrap / bootstrapping) 是一种从已有样本估计统计误差的方法。假设已有 Monte Carlo 样本

$$X_1, X_2, \dots, X_N,$$

并已用它们计算某个估计量 $\hat{\theta} = \hat{\theta}(X_1, \dots, X_N)$ 。Bootstrap 的做法是：

1. 从原始样本中**有放回**地抽取 N 个样本，形成一组 bootstrap 样本；
2. 对这组样本重新计算估计量 $\hat{\theta}^*$ ；
3. 重复上述过程 B 次，得到

$$\hat{\theta}_1^*, \hat{\theta}_2^*, \dots, \hat{\theta}_B^*;$$

4. 用这些重复估计量的方差估计 $\hat{\theta}$ 的统计误差。



笔记 若样本来自 Markov 链，彼此相关，则普通 bootstrap 会低估误差。此时常用分块自助法 (block bootstrap)：先把样本按连续时间块分组，再对样本块有放回抽样，从而保留一部分相关结构。

6.6 随机行走与 Markov 过程

6.6.1 一维随机行走

考虑一维随机行走 (one-dimensional random walk)。粒子每一步向右移动的概率为 R ，向左移动的概率为 L ，每步长度为

$$\Delta x = l, \quad L + R = 1.$$

若第 j 步的位移为 Δx_j ，则

$$x(n) = \sum_{j=1}^n \Delta x_j.$$

当 $L = R = 1/2$ 时，

$$\langle x(n) \rangle = 0, \quad \Delta x_j = \pm l.$$

于是

$$\langle x(n)^2 \rangle = \left\langle \left(\sum_i \Delta x_i \right) \left(\sum_j \Delta x_j \right) \right\rangle = \sum_i \langle \Delta x_i^2 \rangle \sum_{i \neq j} \langle \Delta x_i \Delta x_j \rangle.$$

不同步之间相互独立且均值为 0，因此交叉项为 0，而

$$\sum_i \langle \Delta x_i^2 \rangle = nl^2.$$

故

$$\langle x(n)^2 \rangle - \langle x(n) \rangle^2 = nl^2.$$

若 $L \neq R$ ，则

$$\langle x(n) \rangle = n(R - L)l,$$

而单步方差为

$$\text{Var}(\Delta x) = l^2 - (R - L)^2 l^2 = 4LRl^2.$$

因此

$$\langle x(n)^2 \rangle - \langle x(n) \rangle^2 = 4LRl^2 n.$$

6.6.2 Markov 随机过程

Markov 过程 (Markov process) 描述的是“下一步只依赖当前状态，而不依赖更早历史”的随机演化。微观布朗运动、随机行走以及许多统计物理抽样过程都可以写成 Markov 链。

定义 6.6

若随机过程满足

$$\text{Prob}(X_{n+1} = x_{n+1} \mid X_n = x_n, \dots, X_0 = x_0) = \text{Prob}(X_{n+1} = x_{n+1} \mid X_n = x_n),$$

则称它具有 Markov 性。离散状态空间上的 Markov 过程称为 Markov 链 (Markov chain)。

设位置离散为

$$x = il, \quad i \in \mathbb{Z},$$

时间离散为

$$t = n\epsilon.$$

对于最近邻一维随机行走，转移概率可写成

$$W_i(\epsilon) = W(il - jl, \epsilon) = \begin{cases} \frac{1}{2}, & |i - j| = 1, \\ 0, & \text{else.} \end{cases}$$

若初始概率分布为 $w_j(0)$ ，一步后有

$$w_i(t = \epsilon) = \sum_j W(j \rightarrow i) w_j(0).$$

概率归一化要求

$$\sum_i w_i(t) = 1, \quad \sum_i W(j \rightarrow i) = 1.$$

矩阵形式为

$$w_i(n\epsilon) = \sum_j (W^n(\epsilon))_{ij} w_j(0).$$

6.6.3 转移矩阵与二项式表达

令 $\hat{L}$ 与 $\hat{R}$ 分别表示向左和向右移动一步的算符，则

$$\hat{W} = \frac{1}{2}(\hat{L} + \hat{R}).$$

对 n 步随机行走，

$$\hat{W}^n(n\epsilon) = \frac{1}{2^n} \sum_{k=0}^n \binom{n}{k} \hat{R}^k \hat{L}^{n-k}.$$

也可利用

$$\hat{L}\hat{R} = \hat{R}\hat{L} = I$$

整理为

$$\hat{W}^n(n\epsilon) = \frac{1}{2^n} \sum_{k=0}^n \binom{n}{k} \hat{L}^{n-2k} = \frac{1}{2^n} \sum_{k=0}^n \binom{n}{k} \hat{R}^{2k-n}.$$

矩阵元满足

$$R_{ij}^m = \delta_{i,j+m}, \quad L_{ij}^m = \delta_{i+m,j}.$$

因此

$$W(il - jl, n\epsilon) = \begin{cases} \frac{1}{2^n} \binom{n}{\frac{1}{2}(n+i-j)}, & |i - j| \leq n \text{ 且 } n+i-j \text{ 为偶数}, \\ 0, & \text{else.} \end{cases}$$

若初始在 $x = 0$ ，即 $w_j(0) = \delta_{j0}$ ，则

$$w_i(n\epsilon) = \frac{1}{2^n} \binom{n}{\frac{1}{2}(n+i)}, \quad |i| \leq n.$$

6.6.4 从随机行走到扩散方程

由最近邻随机行走可得递推关系

$$w(x, t + \epsilon) = \frac{1}{2} w(x + l, t) + \frac{1}{2} w(x - l, t).$$

两边整理为

$$\frac{w(x, t + \epsilon) - w(x, t)}{\epsilon} = \frac{l^2}{2\epsilon} \frac{w(x + l, t) - 2w(x, t) + w(x - l, t)}{l^2}.$$

令

$$D = \frac{l^2}{2\epsilon},$$

在连续极限下得到

$$\frac{\partial w(x, t)}{\partial t} = D \frac{\partial^2 w(x, t)}{\partial x^2}.$$

这就是扩散方程 (diffusion equation), 其中 D 是扩散系数 (diffusion coefficient)。其基本解为

$$w(x, t) dx = \frac{1}{\sqrt{4\pi Dt}} \exp\left(-\frac{x^2}{4Dt}\right) dx,$$

并满足

$$\langle x^2 \rangle - \langle x \rangle^2 = 2Dt.$$

6.6.5 有限转移矩阵例子

考虑转移矩阵

$$\hat{W} = \begin{pmatrix} \frac{1}{4} & \frac{1}{8} & \frac{2}{3} \\ \frac{3}{4} & \frac{5}{8} & 0 \\ 0 & \frac{1}{4} & \frac{1}{3} \end{pmatrix}, \quad \hat{w}(t=0) = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

它的列和为 1, 因此若 w_i 表示状态 i 的概率, 则概率保持归一化。一步后

$$\hat{w}(\epsilon) = \hat{W} \hat{w}(0) = \begin{pmatrix} \frac{1}{4} \\ \frac{3}{4} \\ 0 \end{pmatrix}.$$

两步后

$$\hat{w}(2\epsilon) = \hat{W} \hat{w}(\epsilon) = \begin{pmatrix} \frac{5}{32} \\ \frac{21}{32} \\ \frac{6}{32} \end{pmatrix}.$$

平衡态满足

$$\hat{w}(t=\infty) = \hat{W} \hat{w}(t=\infty), \quad \sum_i w_i(t=\infty) = 1.$$

解得

$$\hat{w}(t=\infty) = \begin{pmatrix} \frac{4}{15} \\ \frac{8}{15} \\ \frac{3}{15} \end{pmatrix} = \begin{pmatrix} 0.2667 \\ 0.5333 \\ 0.2000 \end{pmatrix}.$$

注 这与幂法 (power method) 很相似。若把初始分布展开到 $\hat{W}$ 的本征矢量上, 则反复乘以 $\hat{W}$ 后, 最大本征值对应的本征矢量会占主导。对随机矩阵而言, 最大本征值通常为 1, 对应平衡分布。

6.7 Metropolis、Metropolis–Hastings 与 MCMC

6.7.1 Metropolis 行走的动机

若已知转移矩阵 $W(j \rightarrow i)$, Markov 链演化可直接写为

$$w_i(t+1) = \sum_j W_{ij} w_j(t).$$

平衡态满足

$$\mathbf{w}(t = \infty) = \hat{W}\mathbf{w}(t = \infty).$$

但在许多统计物理问题中,我们真正知道的不是转移矩阵,而是目标平衡分布 w_i , 例如 Boltzmann 分布。Metropolis 思想就是: **构造一个 Markov 链, 使它的平衡分布正好是我们想要的 w_i 。**

连续时间形式的平衡态方程可写为

$$\frac{dw_i(t)}{dt} = \sum_j [W(j \rightarrow i)w_j - W(i \rightarrow j)w_i].$$

平衡时

$$\frac{dw_i(t)}{dt} = 0,$$

即

$$\sum_j W(j \rightarrow i)w_j = \sum_j W(i \rightarrow j)w_i.$$

定义 6.7 (细致平衡)

若对任意 i, j 都有

$$W(j \rightarrow i)w_j = W(i \rightarrow j)w_i,$$

则称转移概率满足细致平衡 (**detailed balance**)。

细致平衡是达到正确平衡分布的充分条件, 但不是必要条件。它比总平衡条件更强, 因为它要求每一对状态之间的概率流都逐对抵消。

6.7.2 接受率与 Master 方程

通常把总转移概率拆成两部分:

$$W(j \rightarrow i) = T(j \rightarrow i)A(j \rightarrow i),$$

其中

- $T(j \rightarrow i)$ 是建议移动或试探移动概率 (proposal / transition probability);
- $A(j \rightarrow i)$ 是接受率 (acceptance rate)。

离散时间的 Master equation 可写为

$$w_i(t+1) = \sum_j [w_j(t)T_{j \rightarrow i}A_{j \rightarrow i} + w_i(t)T_{i \rightarrow j}(1 - A_{i \rightarrow j})], \quad \sum_j T_{i \rightarrow j} = 1.$$

整理得

$$w_i(t+1) - w_i(t) = \sum_j [w_j(t)T_{j \rightarrow i}A_{j \rightarrow i} - w_i(t)T_{i \rightarrow j}A_{i \rightarrow j}].$$

平衡态要求

$$\sum_j w_j T_{j \rightarrow i} A_{j \rightarrow i} = \sum_j w_i T_{i \rightarrow j} A_{i \rightarrow j}.$$

若进一步施加细致平衡, 则

$$w_j T_{j \rightarrow i} A_{j \rightarrow i} = w_i T_{i \rightarrow j} A_{i \rightarrow j},$$

也就是

$$\frac{T_{j \rightarrow i} A_{j \rightarrow i}}{T_{i \rightarrow j} A_{i \rightarrow j}} = \frac{w_i}{w_j}.$$

6.7.3 Boltzmann 分布与 Metropolis 接受率

平衡态统计中，能量为 E_i 的状态 i 的 Boltzmann 权重为

$$w_i = \frac{\exp(-\beta E_i)}{Z}, \quad \beta = \frac{1}{k_B T},$$

其中配分函数为

$$Z = \sum_j \exp(-\beta E_j).$$

任意物理量 A 的期望值为

$$\langle A \rangle = \sum_j A_j w_j = \frac{\sum_j A_j \exp(-\beta E_j)}{Z}.$$

简单均匀采样在这里通常不合适，因为状态数可能巨大甚至近似无穷，而且重要状态只集中在 Boltzmann 权重较大的区域。

若建议概率对称，

$$T(j \rightarrow i) = T(i \rightarrow j),$$

则细致平衡只要求

$$\frac{A(j \rightarrow i)}{A(i \rightarrow j)} = \frac{w_i}{w_j} = \exp[-\beta(E_i - E_j)].$$

Metropolis 选择为

$$A(j \rightarrow i) = \begin{cases} \exp[-\beta(E_i - E_j)], & E_i - E_j > 0, \\ 1, & E_i - E_j \leq 0. \end{cases}$$

也就是

$$A(j \rightarrow i) = \min\{1, \exp[-\beta(E_i - E_j)]\}.$$

若新状态能量更低，就无条件接受；若新状态能量更高，就以 Boltzmann 因子给出的概率接受。

6.7.4 Metropolis–Hastings 算法

Metropolis–Hastings algorithm 是 Metropolis 算法的推广，允许非对称建议分布。设当前状态为 $x^{(i)}$ ，从建议分布

$$f(y_t | x^{(i)})$$

产生试探点 y_t 。接受后

$$x^{(i+1)} = y_t,$$

拒绝后

$$x^{(i+1)} = x^{(i)}.$$

接受概率定义为

$$A(x \rightarrow y) = \min \left\{ \frac{w(y)f(x | y)}{w(x)f(y | x)}, 1 \right\}.$$

然后产生均匀随机数 $r \in [0, 1]$ ，若

$$A(x \rightarrow y) > r,$$

则接受新点，否则保留旧点。经过足够长的迭代后，样本分布会趋向目标分布 w 。

注 当建议分布对称，即 $f(x | y) = f(y | x)$ 时，Metropolis–Hastings 退化为普通 Metropolis 算法。它与模拟退火 (simulated annealing) 形式相似，但目标不同：MCMC 主要用于抽样给定分布，模拟退火主要用于优化。

6.7.5 MCMC 积分

Markov Chain Monte Carlo (MCMC) 的目标是构造一条 Markov 链，使其平衡分布为给定概率分布 $P(\theta | D)$ 。当链达到平衡后，可以用链上的样本估计期望值：

$$\mathbb{E}[f(x | \theta)] = \int f(x | \theta) P(\theta | D) d\theta \approx \frac{1}{N} \sum_{i=1}^N f(x | \theta_i).$$

这里 θ_i 是 Markov 链达到平衡分布后的样本。

MCMC 常用于：

- Bayesian 统计；
- 高维积分；
- 机器学习中的后验分布采样；
- 复杂物理系统中的平衡态抽样。

 **笔记** 直接采样的样本彼此独立，因此统计分析更简单；MCMC 样本通常有相关性。优点是它只需要知道如何从当前点产生下一个点，就能抽样非常复杂的分布。处理相关性时常需要 burn-in、抽样间隔、有效样本数 (effective sample size) 或 block bootstrap。

相关采样 (correlated sampling) 在比较能量差、自由能差等问题中很有用，因为两个相近系统若使用相同或相关样本，统计涨落会部分抵消。

6.8 Hamiltonian Monte Carlo

6.8.1 引入动量变量

Hamiltonian Monte Carlo (HMC) 通过引入辅助动量 p ，把原变量 q 扩展为相空间变量

$$q \longrightarrow (q, p).$$

目标联合分布写成

$$\pi(q, p) = \pi_1(q)\pi_2(p).$$

定义 Hamiltonian

$$H(q, p) = U(q) + K(p),$$

其中 $U(q)$ 是势能， $K(p)$ 是动能。若取

$$\pi(q, p) \propto e^{-H(q, p)},$$

则对 p 积分后可恢复目标的 q 分布。

6.8.2 Hamilton 方程与接受率

Hamilton 方程为

$$\begin{aligned} \frac{dq}{dt} &= \frac{\partial H}{\partial p} = \frac{\partial K(p)}{\partial p} = \frac{p}{m}, \\ \frac{dp}{dt} &= -\frac{\partial H}{\partial q} = -\frac{\partial U(q)}{\partial q}. \end{aligned}$$

数值积分这些方程，可定义一个从 (q, p) 到 (q^*, p^*) 的映射 T_ϵ 。由于数值积分会有离散误差，需要再用 Metropolis 接受率修正：

$$A((q, p) \rightarrow (q^*, p^*)) = \min \left\{ 1, \frac{\exp[-H(q^*, p^*)]}{\exp[-H(q, p)]} \right\}.$$

注 HMC 的优势是利用梯度信息沿 Hamiltonian 轨道提出远距离候选点, 因此比普通随机游走式 MCMC 更不容易在高维空间中缓慢扩散。它在贝叶斯神经网络 (Bayesian neural networks, BNN) 和高维后验采样中很有用。

6.9 统计物理中的 Monte Carlo: 二维 Ising 模型

6.9.1 二维 Ising 模型

二维 Ising 模型 (two-dimensional Ising model) 是统计物理中研究相变的经典模型。每个格点上有一个自旋变量

$$s_i = \pm 1.$$

若考虑最近邻相互作用和外磁场 B , 能量为

$$E = -J \sum_{\langle kl \rangle} s_k s_l - B \sum_k s_k.$$

无外场时常写成

$$E = -J \sum_{\text{neighbors } i, j} s_i s_j.$$

在温度 T 下, 构型 s 的概率为

$$P_s = \frac{e^{-\beta E_s}}{Z}, \quad \beta = \frac{1}{k_B T},$$

配分函数为

$$Z(\beta) = \sum_s e^{-\beta E_s}.$$

能量期望与比热可写为

$$\langle E \rangle = -\frac{\partial \ln Z}{\partial \beta}, \quad C_V = \frac{1}{kT^2} \frac{\partial^2}{\partial \beta^2} \ln Z_N.$$

若晶格大小为 $L \times L$, 自旋数为

$$N = L \times L,$$

则构型数为

$$2^N.$$

这说明即使二维格点不大, 直接枚举全部构型也会迅速变得不可行。

6.9.2 单自旋翻转的能量差

Metropolis 更新通常每次只尝试翻转一个自旋。设翻转前为构型 1, 翻转后为构型 2, 则

$$\Delta E = E_2 - E_1.$$

由于只改变一个自旋 s_l , 只有它和最近邻之间的键能会改变。若翻转前自旋为 $s_l^{(1)}$, 则

$$\Delta E = 2J s_l^{(1)} \sum_{\langle k \rangle} s_k.$$

在二维方格最近邻模型中, 一个格点有 4 个最近邻, 因此

$$\Delta E \in \{-8J, -4J, 0, 4J, 8J\}.$$

磁化强度 (magnetisation) 为

$$M = \sum_i s_i.$$

若翻转一个自旋, 则

$$M_2 = M_1 - 2s_l^{(1)} = M_1 + 2s_l^{(2)}.$$

6.9.3 二维 Ising 的 Metropolis 算法

二维 Ising 模型的标准 Metropolis 流程为:

1. 随机生成初始晶格构型, 并计算初始能量 E_b ;
2. 随机选择一个格点, 只翻转该格点的自旋, 得到试探构型;
3. 计算试探构型能量 E_t , 并令

$$\Delta E = E_t - E_b;$$

4. 若

$$\Delta E \leq 0,$$

则接受新构型;

5. 若

$$\Delta E > 0,$$

则计算

$$w = e^{-\beta \Delta E};$$

6. 生成均匀随机数 $r \in [0, 1]$ 。若

$$r \leq w,$$

则接受新构型, 否则保留旧构型;

7. 更新能量、磁化强度等观测量;
8. 重复单自旋更新, 使样本充分代表平衡态;
9. 随机遍历全部自旋一次通常称为一个 Monte Carlo cycle 或一次 measurement。最终期望值应除以 Monte Carlo cycle 的数量。

 **笔记** 高温时 $e^{-\beta \Delta E}$ 对正的 ΔE 也可能不太小, 因此系统容易接受升能量移动, 构型更无序; 低温时升能量移动几乎被拒绝, 系统更容易停留在低能有序构型。

6.9.4 相变与有限尺寸效应

二维 Ising 模型在无外场时具有二级相变 (second-order phase transition)。临界温度满足

$$\frac{k_B T_c}{J} \approx 2.269.$$

在 $T < T_c$ 时, 系统出现自发磁化 (spontaneous magnetization); 在 $T > T_c$ 时, 热涨落破坏长程序。比热 C_V 在临界温度附近出现峰值, 磁化强度在临界温度附近快速下降。

注 一维 Ising 模型在有限温度下没有相变, 而二维 Ising 模型有相变。这说明相变不仅取决于局域相互作用形式, 也强烈依赖空间维度。

6.9.5 Kosterlitz–Thouless 转变

另一类重要二维模型是平面转子或 XY 模型, 其能量可写成

$$E = -J \sum_{\text{neighbors } i, j} \mathbf{s}_i \cdot \mathbf{s}_j = -J \sum_{\text{neighbors } i, j} \cos(\theta_i - \theta_j).$$

其平衡分布为

$$p = \frac{1}{Z} \exp\left(-\frac{E}{k_B T}\right).$$

Kosterlitz–Thouless (KT) 转变是一类拓扑相变 (topological phase transition): 低温下涡旋和反涡旋形成束缚对, 高温下它们解束缚成为自由涡旋。对这类体系, 除了比热外, 还可以研究表面螺旋刚度、涡旋数目与温度的关系。

6.10 量子 Monte Carlo 方法

6.10.1 变分量子 Monte Carlo 的基本流程

量子 Monte Carlo (Quantum Monte Carlo, QMC) 常用于多体量子系统。变分量子 Monte Carlo (Variational Quantum Monte Carlo, VMC) 的基本流程为:

1. 构造含变分参数的试探波函数

$$\Psi_T(\mathbf{R}; \alpha), \quad \mathbf{R} = (R_1, \dots, R_N);$$

2. 计算 Hamiltonian 的期望值

$$\langle H \rangle = \frac{\int d\mathbf{R} \Psi_T^*(\mathbf{R}; \alpha) H(\mathbf{R}) \Psi_T(\mathbf{R}; \alpha)}{\int d\mathbf{R} \Psi_T^*(\mathbf{R}; \alpha) \Psi_T(\mathbf{R}; \alpha)};$$

3. 改变参数 α , 用优化算法最小化能量;
4. 返回第一步, 直到能量或方差收敛。

定义 6.8 (量子概率分布与局域能量)

VMC 中常使用的概率分布为

$$P(\mathbf{R}; \alpha) = \frac{|\Psi_T(\mathbf{R}; \alpha)|^2}{\int |\Psi_T(\mathbf{R}; \alpha)|^2 d\mathbf{R}}.$$

局域能量 (local energy) 定义为

$$\hat{E}_L(\mathbf{R}; \alpha) = \frac{1}{\Psi_T(\mathbf{R}; \alpha)} \hat{H} \Psi_T(\mathbf{R}; \alpha).$$

于是能量期望值可写成

$$\langle \hat{H} \rangle = \int P(\mathbf{R}; \alpha) \hat{E}_L(\mathbf{R}; \alpha) d\mathbf{R}.$$

若试探波函数正好是精确本征态, 则

$$\hat{H} \Psi_T = E \Psi_T,$$

因此局域能量在所有 $\mathbf{R}$ 上都是常数, 方差为 0。

多体 Hamiltonian 常写为

$$\hat{H} = -\frac{\hbar^2}{2m} \sum_{i=1}^N \nabla_i^2 + \sum_{i=1}^N V_{\text{onebody}}(\mathbf{r}_i) + \sum_{i<j}^N V_{\text{int}}(|\mathbf{r}_i - \mathbf{r}_j|).$$

对应的局域能量为

$$\hat{E}_L(\mathbf{R}; \alpha) = \frac{1}{\Psi_T(\mathbf{R}; \alpha)} \left(-\frac{\hbar^2}{2m} \sum_{i=1}^N \nabla_i^2 \right) \Psi_T(\mathbf{R}; \alpha) + \sum_{i=1}^N V_{\text{onebody}}(\mathbf{r}_i) + \sum_{i<j}^N V_{\text{int}}(|\mathbf{r}_i - \mathbf{r}_j|).$$

6.10.2 用 Metropolis 抽样 $|\Psi_T|^2$

VMC 中需要从

$$P(\mathbf{R}; \alpha)$$

抽样。对试探位置 $\mathbf{R}'$, 若

$$\frac{P(\mathbf{R}'; \alpha)}{P(\mathbf{R}; \alpha)} > 1,$$

则直接接受；否则生成均匀随机数 $r \in [0, 1]$ ，若

$$r \leq \frac{P(\mathbf{R}'; \alpha)}{P(\mathbf{R}; \alpha)},$$

则接受。实际流程可写成：

1. 生成试探位置

$$\mathbf{R}_p = \mathbf{R} + r \Delta \mathbf{R};$$

2. 计算概率比

$$w = \frac{P(\mathbf{R}_p)}{P(\mathbf{R})};$$

3. 用 Metropolis 判据接受或拒绝；

4. 平衡阶段后，积累局域能量与其他观测量。

能量及其平方可估计为

$$\langle \hat{E}_L \rangle \approx \frac{1}{N} \sum_{i=1}^N E_L(x_i), \quad \langle \hat{E}_L^2 \rangle \approx \frac{1}{N} \sum_{i=1}^N E_L(x_i)^2.$$

抽样空间是 $3N$ 维坐标空间：

$$\mathbf{R} = (\mathbf{r}_1, \dots, \mathbf{r}_N).$$

 **笔记** 试探波函数的选择对 QMC 至关重要。好的试探波函数不仅降低能量估计，还会显著降低局域能量方差，使 Monte Carlo 误差更小。

6.10.3 一维谐振子的 VMC 例子

考虑一维谐振子 Hamiltonian

$$H = -\frac{\hbar^2}{2m} \frac{d^2}{dx^2} + \frac{1}{2} kx^2.$$

在合适单位下可写为

$$H = -\frac{d^2}{dx^2} + x^2.$$

精确基态波函数为

$$\Psi_0(x) = \frac{1}{\pi^{1/4}} e^{-x^2/2}.$$

取试探波函数

$$\Psi_T(x) = \frac{\sqrt{\alpha}}{\pi^{1/4}} e^{-\alpha^2 x^2/2}.$$

代入局域能量定义可得

$$E_L(x) = \alpha^2 + x^2(1 - \alpha^4).$$

当 $\alpha = 1$ 时，

$$E_L(x) = 1,$$

与坐标 x 无关，说明试探波函数正好等于精确基态。

能量平均为

$$\langle E_L \rangle = \int_{-\infty}^{\infty} |\Psi_T(x)|^2 E_L(x) dx.$$

利用

$$\int_{-\infty}^{\infty} e^{-\alpha^2 x^2} dx = \sqrt{\frac{\pi}{\alpha^2}}, \quad \int_{-\infty}^{\infty} x^2 e^{-\alpha^2 x^2} dx = \frac{1}{2} \sqrt{\frac{\pi}{\alpha^3}},$$

可得

$$\langle E_L \rangle = \frac{\alpha^2}{2} + \frac{1}{2\alpha^2}.$$

方差为

$$\sigma^2 = \frac{(\alpha^4 - 1)^2}{2\alpha^4}.$$

最小化能量得到

$$\alpha = 1,$$

此时能量达到精确基态能量，方差也趋近 0。

6.10.4 VMC 计算中的误差

若进行了 M 次独立采样，能量误差满足

$$\text{err}(E) \sim \frac{\sigma}{\sqrt{M}}.$$

实践中常同时积累

$$\sum_i E_L(x_i), \quad \sum_i E_L(x_i)^2,$$

从而估计平均能量、方差与标准误差。变分计算可以用两种准则优化：

- 最小化能量；
- 最小化局域能量方差。

当试探波函数逐渐接近精确本征态时，局域能量趋于常数，方差也会降低。

6.11 Diffusion Monte Carlo

6.11.1 虚时间演化与 Green 函数

Diffusion Monte Carlo (DMC) 基于虚时间 Schrödinger 演化。设时间步长为 τ ，参考能量为 E_T ，则

$$\psi_{n+1}(\mathbf{R}) = e^{-\tau(\hat{H}-E_T)}\psi_n(\mathbf{R}) = \int d\mathbf{R}' G(\mathbf{R}, \mathbf{R}')\psi_n(\mathbf{R}').$$

短时间近似下，

$$\langle \mathbf{R} | e^{-\tau(\hat{H}-E_T)} | \mathbf{R}' \rangle = (4\pi\lambda\tau)^{-3N/2} \exp\left[-\frac{(\mathbf{R}-\mathbf{R}')^2}{4\lambda\tau}\right] \exp[-\tau(V(\mathbf{R})-E_T)] + O(\tau^2),$$

其中

$$\lambda = \frac{\hbar^2}{2m}.$$

第一项具有扩散核形式，第二项相当于分支或权重因子。DMC 因而可以理解为：用随机 walkers 在虚时间中扩散，同时按势能权复制或消灭 walkers，使其分布逐渐趋向基态波函数。

6.11.2 转移概率与 sign problem

DMC 中可构造转移概率

$$\omega_{\mathbf{R}\mathbf{R}'} = G(\mathbf{R}, \mathbf{R}'; \Delta t),$$

并用类似 Metropolis 的比值

$$q_{\mathbf{R}\mathbf{R}'} = \frac{\omega_{\mathbf{R}'\mathbf{R}}\rho(\mathbf{R}')}{\omega_{\mathbf{R}\mathbf{R}'}\rho(\mathbf{R})}$$

决定移动。这里的关键区别是：DMC 提供的是含扩散和权重的 transition probability，而不是简单的 Metropolis 随机游走。

对费米子体系，波函数有正负号结构。随机 walkers 会遇到正负贡献相互抵消的问题，这就是著名的 sign problem。实际计算中通常需要非常好的初始试探波函数，或采用固定节点近似（fixed-node approximation）等策略。

注 VMC 可看成沿一条 Markov 路径抽样试探波函数的概率分布；路径积分 Monte Carlo (PIMC) 则对多条路径进行采样，可用于计算配分函数；DMC 则利用虚时间投影逼近基态波函数。

6.12 本章总结

结论 Monte Carlo 方法的主线可以概括为：用随机 sampling 把难以直接求解的问题转化为概率平均；若直接采样效率低，就设计更合适的 sampling rule；若目标分布复杂，就构造 Markov chain，使其平衡态具有所需分布。

- Monte Carlo 积分的误差通常按 $1/\sqrt{N}$ 衰减，适合高维积分和复杂区域积分；
- 重要性采样通过改变采样分布降低方差，是提高效率的核心方法；
- Bootstrap 可从样本本身估计误差，但 Markov 链样本需要考虑相关性；
- 随机行走在连续极限下导出扩散方程，是理解扩散 Monte Carlo 的基础；
- Markov chain 的平衡态、ergodicity 和 detailed balance 是 MCMC 正确性的关键；
- Metropolis algorithm 是 20 世纪最重要的算法之一，也是统计物理抽样的核心工具；
- 二维 Ising 模型展示了如何用 Metropolis 方法研究相变、能量、比热与磁化；
- QMC 在多体量子系统中具有优势，尤其适合处理高维坐标积分和多自由度体系；
- VMC 依赖试探波函数和局域能量，DMC 依赖虚时间扩散与投影，两者共同构成量子 Monte Carlo 的基础框架。

第七章 微分方程 (Differential Equations)

内容提要

- 常微分方程初值问题与 Euler 法
- 高阶 Taylor 展开法、Runge-Kutta 方法与 predictor-corrector 方法
- 微分方程组、高阶微分方程与刚性微分方程
- 边值问题与打靶法
- Numerov 方法与球对称势 Schrödinger 方程
- Green 函数、Wronskian 与有限差分法
- Tolman-Oppenheimer-Volkoff 方程与中子星结构计算

7.1 初值问题与 Euler 法

7.1.1 一阶常微分方程初值问题

本章首先讨论常微分方程 (ordinary differential equation, ODE) 的初值问题 (initial value problem, IVP)。一元独立变量情形可写为

$$\frac{dy}{dt} = f(t, y), \quad a \leq t \leq b, \quad y(a) = \alpha.$$

其中 t 是独立变量, $y(t)$ 是待求函数, $f(t, y)$ 给出系统在状态 y 与时刻 t 下的瞬时变化率, α 是初始值。数值求解的基本思想是把连续区间 $[a, b]$ 离散为等间距网格

$$t_i = a + ih, \quad h = \frac{b-a}{N}, \quad i = 0, 1, \dots, N,$$

然后用有限个近似值

$$w_i \approx y(t_i)$$

代替连续函数。这里 h 称为步长 (step size), 它同时控制计算量和离散误差。

定义 7.1

Euler 法 (Euler's method) 是最基本的一步法 (one-step method)。从初值 $w_0 = \alpha$ 出发, 用当前点斜率 $f(t_i, w_i)$ 外推到下一网格点:

$$w_{i+1} = w_i + hf(t_i, w_i), \quad i = 0, 1, \dots, N-1.$$

Euler 法来自 Taylor 展开的一阶截断。若精确解 $y(t)$ 足够光滑, 则

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(\xi_i), \quad \xi_i \in (t_i, t_{i+1}).$$

代入 $y'(t_i) = f(t_i, y(t_i))$, 并用 w_i 近似 $y(t_i)$, 就得到 Euler 递推。被舍去的项

$$\frac{h^2}{2}y''(\xi_i)$$

是单步误差的主要来源。

 **笔记** Euler 法的局部截断误差 (local truncation error) 为 $O(h^2)$, 但每一步的误差会在 $N \sim 1/h$ 个步长上累积, 因此全局误差通常是一阶的, 即 $O(h)$ 。这说明把步长减半时, Euler 法的整体误差大约只减半。

更一般地, 设一个数值格式写成

$$w_{i+1} = w_i + h\phi(t_i, w_i),$$

则在精确解代入格式时, 局部截断误差可定义为

$$\tau_{i+1}(h) = \frac{y(t_{i+1}) - [y(t_i) + h\phi(t_i, y(t_i))]}{h} = \frac{y(t_{i+1}) - y(t_i)}{h} - \phi(t_i, y(t_i)).$$

对 Euler 法, $\phi = f$, 因而

$$\tau_{i+1}(h) = \frac{h}{2}y''(\xi_i).$$

这个公式清楚地说明: 误差来自用区间左端点的切线斜率代替整个区间上的真实平均斜率。

7.1.2 高阶 Taylor 展开法

若函数 $y(t)$ 有足够高阶连续导数, 可以保留更多 Taylor 项。对任意正整数 n , 有

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2!}y''(t_i) + \cdots + \frac{h^n}{n!}y^{(n)}(t_i) + \frac{h^{n+1}}{(n+1)!}y^{(n+1)}(\xi_i).$$

因为 $y' = f(t, y)$, 高阶导数也可沿解曲线由 f 递推得到。例如

$$y''(t) = \frac{d}{dt}f(t, y(t)) = \frac{\partial f}{\partial t}(t, y) + \frac{\partial f}{\partial y}(t, y)y'(t) = f_t(t, y) + f_y(t, y)f(t, y).$$

记沿解曲线的全导数为

$$f'(t, y(t)) = \frac{d}{dt}f(t, y(t)),$$

则 n 阶 Taylor 方法可写成

$$w_{i+1} = w_i + hT^{(n)}(t_i, w_i),$$

其中

$$T^{(n)}(t_i, w_i) = f(t_i, w_i) + \frac{h}{2!}f'(t_i, w_i) + \cdots + \frac{h^{n-1}}{n!}f^{(n-1)}(t_i, w_i).$$

Euler 法正是 $n = 1$ 的 Taylor 方法。

例题 7.1 Taylor 法求解一阶初值问题 考虑

$$y' = y - t^2 + 1, \quad 0 \leq t \leq 2, \quad y(0) = 0.5.$$

该方程的精确解为

$$y(t) = (t+1)^2 - \frac{1}{2}e^t.$$

这里

$$f(t, y) = y - t^2 + 1.$$

沿解曲线逐次求导得

$$f'(t, y(t)) = y - t^2 + 1 - 2t,$$

$$f''(t, y(t)) = \frac{d}{dt}(y - t^2 + 1 - 2t) = y' - 2t - 2 = y - t^2 - 2t - 1,$$

$$f'''(t, y(t)) = \frac{d}{dt}(y - t^2 - 2t - 1) = y' - 2t - 2 = y - t^2 - 2t - 1.$$

因此四阶 Taylor 方法中的斜率函数为

$$\begin{aligned} T^{(4)}(t_i, w_i) &= f(t_i, w_i) + \frac{h}{2}f'(t_i, w_i) + \frac{h^2}{6}f''(t_i, w_i) + \frac{h^3}{24}f'''(t_i, w_i) \\ &= \left(1 + \frac{h}{2} + \frac{h^2}{6} + \frac{h^3}{24}\right)(w_i - t_i^2) - \left(1 + \frac{h}{3} + \frac{h^2}{12}\right)ht_i + 1 + \frac{h}{2} - \frac{h^2}{6} - \frac{h^3}{24}. \end{aligned}$$

若只取二阶 Taylor 法, 并令 $h = 0.2$, $N = 10$, $t_i = 0.2i$, 则

$$w_{i+1} = w_i + h \left[\left(1 + \frac{h}{2}\right)(w_i - t_i^2 + 1) - ht_i \right] = 1.22w_i - 0.0088i^2 - 0.008i + 0.22.$$

下表给出同一问题在 $h = 0.2$ 时二阶与四阶 Taylor 法的误差对比。

t_i	$y(t_i)$	Taylor 2	$ y(t_i) - w_i $	Taylor 4	$ y(t_i) - w_i $
0.0	0.500000	0.500000	0	0.500000	0
0.2	0.829299	0.830000	0.000701	0.829300	0.000001
0.4	1.214088	1.215800	0.001712	1.214091	0.000003
0.6	1.648941	1.652076	0.003135	1.648947	0.000006
0.8	2.127230	2.132333	0.005103	2.127240	0.000010
1.0	2.640859	2.648646	0.007787	2.640874	0.000015
1.2	3.179942	3.191348	0.011407	3.179964	0.000023
1.4	3.732400	3.748645	0.016245	3.732432	0.000032
1.6	4.283484	4.306146	0.022663	4.283529	0.000045
1.8	4.815176	4.846299	0.031122	4.815238	0.000062
2.0	5.305472	5.347684	0.042212	5.305555	0.000083

高阶 Taylor 展开能显著减小截断误差，但代价是需要计算高阶全导数；当 $f(t, y)$ 很复杂或方程组维数很高时，这一点会变得不方便。

7.2 Runge–Kutta 方法

7.2.1 从 Taylor 法到二阶 Runge–Kutta 法

Runge–Kutta 方法 (Runge–Kutta methods) 的核心思想是：不显式计算高阶导数，而是在一个步长内多次取样 $f(t, y)$ ，用这些斜率的加权平均模拟 Taylor 展开中的高阶信息。

对二阶 Taylor 法，

$$T^{(2)}(t, y) = f(t, y) + \frac{h}{2} f'(t, y).$$

利用

$$f'(t, y(t)) = f_t(t, y) + f_y(t, y)f(t, y),$$

可尝试用一个移位点上的函数值

$$a_1 f(t + \alpha_1, y + \beta_1)$$

替代 $T^{(2)}$ 。对 f 作一阶 Taylor 展开：

$$a_1 f(t + \alpha_1, y + \beta_1) = a_1 f(t, y) + a_1 \alpha_1 f_t(t, y) + a_1 \beta_1 f_y(t, y) + R_1.$$

与

$$f(t, y) + \frac{h}{2} f_t(t, y) + \frac{h}{2} f_y(t, y)f(t, y)$$

逐项比较，可取

$$a_1 = 1, \quad \alpha_1 = \frac{h}{2}, \quad \beta_1 = \frac{h}{2} f(t, y).$$

于是得到中点法 (midpoint method)：

$$w_0 = \alpha, \\ w_{i+1} = w_i + hf \left(t_i + \frac{h}{2}, w_i + \frac{h}{2} f(t_i, w_i) \right).$$

它先用左端点斜率估计半步处的 y ，再用半步斜率推进一个完整步长，局部截断误差比 Euler 法低一阶。

7.2.2 Modified Euler 方法与 Heun 方法

另一种二阶显式格式是改进 Euler 法 (modified Euler method), 也被称为显式梯形法:

$$w_0 = \alpha,$$

$$w_{i+1} = w_i + \frac{h}{2} [f(t_i, w_i) + f(t_{i+1}, w_i + hf(t_i, w_i))], \quad i = 0, 1, \dots, N-1.$$

这里先用 Euler 法预测 t_{i+1} 处的值, 再把左端点斜率和预测右端点斜率取平均。它的物理含义是用区间平均斜率代替单端斜率, 因此通常比 Euler 法稳定、准确。

Heun 在 1900 年提出的三阶 Runge-Kutta 形式可写为

$$w_{i+1} = w_i + \frac{h}{4} \left[f(t_i, w_i) + 3f\left(t_i + \frac{2h}{3}, w_i + \frac{2h}{3}f\left(t_i + \frac{h}{3}, w_i + \frac{h}{3}f(t_i, w_i)\right)\right) \right].$$

它仍然是显式方法, 但通过嵌套的中间斜率获得更高阶精度。

对前面的方程 $y' = y - t^2 + 1$, 在 $h = 0.2$ 下, 中点法、改进 Euler 法与 Heun 法的误差如下:

t_i	$y(t_i)$	Midpoint	Error	Modified Euler	Error	Heun Error
0.0	0.5000000	0.5000000	0	0.5000000	0	0
0.2	0.8292986	0.8280000	0.0012986	0.8260000	0.0032986	0.0000542
0.4	1.2140877	1.2113600	0.0027277	1.2069200	0.0071677	0.0001127
0.6	1.6489406	1.6446592	0.0042814	1.6372424	0.0116982	0.0001747
0.8	2.1272295	2.1212842	0.0059453	2.1102357	0.0169938	0.0002390
1.0	2.6408591	2.6331668	0.0076923	2.6176876	0.0231715	0.0003035
1.2	3.1799415	3.1704634	0.0094781	3.1495789	0.0303627	0.0003653
1.4	3.7324000	3.7211654	0.0112346	3.6936862	0.0387138	0.0004197
1.6	4.2834838	4.2706218	0.0128620	4.2350972	0.0483866	0.0004608
1.8	4.8151763	4.8009586	0.0142177	4.7556185	0.0595577	0.0004797
2.0	5.3054720	5.2903695	0.0151025	5.2330546	0.0724173	0.0004648

Heun 法的误差明显小于二阶方法, 体现了高阶 Runge-Kutta 取样的优势。

7.2.3 四阶 Runge-Kutta 方法

四阶 Runge-Kutta 方法 (fourth-order Runge-Kutta method, RK4) 是最常用的显式初值问题算法之一。其标准形式为

$$w_0 = \alpha,$$

$$k_1 = hf(t_i, w_i),$$

$$k_2 = hf\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_1\right),$$

$$k_3 = hf\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2\right),$$

$$k_4 = hf(t_{i+1}, w_i + k_3),$$

$$w_{i+1} = w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

四个 k 分别是左端点、两个中点和右端点附近的斜率贡献。权重 $1:2:2:1$ 可以理解为 Simpson 求积思想在微分方程积分中的对应物。

对同一测试方程, 若 Euler 法取 $h = 0.025$ 、改进 Euler 法取 $h = 0.05$ 、RK4 取 $h = 0.1$, 前几个点的对比如下:

t_i	Exact	Euler $h = 0.025$	Modified Euler $h = 0.05$	RK4 $h = 0.1$
0.0	0.5000000	0.5000000	0.5000000	0.5000000
0.1	0.6574145	0.6554982	0.6573085	0.6574144
0.2	0.8292986	0.8253385	0.8290778	0.8292983
0.3	1.0150706	1.0089334	1.0147254	1.0150701
0.4	1.2140877	1.2056345	1.2136079	1.2140869
0.5	1.4256394	1.4147264	1.4250141	1.4256384

RK4 即使使用更大的步长, 也能达到很高精度, 因此在许多物理初值问题中是默认首选方法。

7.2.4 Runge-Kutta-Fehlberg 与 Verner 方法

Runge-Kutta-Fehlberg 方法 (Runge-Kutta-Fehlberg method, RKF) 把不同阶数的 Runge-Kutta 估计嵌入到同一组函数计算中, 从而可估计局部误差并自适应调整步长。一个常用的五阶估计为

$$\tilde{w}_{i+1} = w_i + \frac{16}{135}k_1 + \frac{6656}{12825}k_3 + \frac{28561}{56430}k_4 - \frac{9}{50}k_5 + \frac{2}{55}k_6,$$

其中

$$k_1 = hf(t_i, w_i),$$

$$k_2 = hf\left(t_i + \frac{h}{4}, w_i + \frac{1}{4}k_1\right),$$

$$k_3 = hf\left(t_i + \frac{3h}{8}, w_i + \frac{3}{32}k_1 + \frac{9}{32}k_2\right),$$

$$k_4 = hf\left(t_i + \frac{12h}{13}, w_i + \frac{1932}{2197}k_1 - \frac{7200}{2197}k_2 + \frac{7296}{2197}k_3\right),$$

$$k_5 = hf\left(t_i + h, w_i + \frac{439}{216}k_1 - 8k_2 + \frac{3680}{513}k_3 - \frac{845}{4104}k_4\right),$$

$$k_6 = hf\left(t_i + \frac{h}{2}, w_i - \frac{8}{27}k_1 + 2k_2 - \frac{3544}{2565}k_3 + \frac{1859}{4104}k_4 - \frac{11}{40}k_5\right).$$

该方法的局部截断误差可达到五阶。Runge-Kutta-Verner 方法进一步给出六阶公式, 常用于需要严格误差控制的高精度积分。

7.3 Predictor-Corrector 方法

7.3.1 多步法的基本思想

一步法只用当前 w_i 推进到 w_{i+1} , 多步法 (multistep methods) 则利用多个历史点上的斜率。它的优势是: 在已经拥有若干历史值之后, 每一步所需的新函数评估次数较少。

三步 Adams-Moulton 隐式方法 (Adams-Moulton three-step implicit method) 为

$$w_0 = \alpha, \quad w_1 = \alpha_1, \quad w_2 = \alpha_2,$$

$$w_{i+1} = w_i + \frac{h}{24} [9f(t_{i+1}, w_{i+1}) + 19f(t_i, w_i) - 5f(t_{i-1}, w_{i-1}) + f(t_{i-2}, w_{i-2})].$$

因为右端含 w_{i+1} , 每一步原则上都要解一个非线性方程。

三步显式 Adams-Bashforth 方法 (three-step explicit Adams-Bashforth method) 为

$$w_{i+1} = w_i + \frac{h}{12} [23f(t_i, w_i) - 16f(t_{i-1}, w_{i-1}) + 5f(t_{i-2}, w_{i-2})].$$

四步显式 Adams–Bashforth 方法为

$$w_{i+1} = w_i + \frac{h}{24} [55f(t_i, w_i) - 59f(t_{i-1}, w_{i-1}) + 37f(t_{i-2}, w_{i-2}) - 9f(t_{i-3}, w_{i-3})].$$

7.3.2 预测-校正流程

预测-校正方法 (predictor-corrector method) 把显式公式和隐式公式组合起来:

1. 先用四阶 Runge–Kutta 方法计算起始值 w_0, w_1, w_2, w_3 ;
2. 用 Adams–Bashforth 显式公式预测 w_4 ;
3. 用 Adams–Moulton 隐式公式校正 w_4 ;
4. 必要时反复校正, 直到相邻校正值之差小于给定容差。

以 w_4 为例, 预测步为

$$w_4^{(p)} = w_3 + \frac{h}{24} [55f(t_3, w_3) - 59f(t_2, w_2) + 37f(t_1, w_1) - 9f(t_0, w_0)],$$

校正步为

$$w_4^{(c)} = w_3 + \frac{h}{24} [9f(t_4, w_4^{(p)}) + 19f(t_3, w_3) - 5f(t_2, w_2) + f(t_1, w_1)].$$

若把 $w_4^{(c)}$ 再代入右端, 可继续迭代得到更高精度。时间依赖密度泛函理论 (time-dependent density functional theory, TDDFT) 中的时间演化常使用这类 predictor-corrector 思路, 因为它兼顾了精度和每步计算成本。

7.4 微分方程组与高阶微分方程

7.4.1 一阶微分方程组

许多物理系统天然包含多个相互耦合的变量。一阶微分方程组可写为

$$\begin{aligned} \frac{du_1}{dt} &= f_1(t, u_1, u_2, \dots, u_m), \\ \frac{du_2}{dt} &= f_2(t, u_1, u_2, \dots, u_m), \\ &\dots \\ \frac{du_m}{dt} &= f_m(t, u_1, u_2, \dots, u_m), \quad a \leq t \leq b, \end{aligned}$$

初值为

$$u_1(a) = \alpha_1, \quad u_2(a) = \alpha_2, \quad \dots, \quad u_m(a) = \alpha_m.$$

把 $\mathbf{u} = (u_1, \dots, u_m)^T$ 、 $\mathbf{f} = (f_1, \dots, f_m)^T$ 记为向量后, 形式上就是

$$\mathbf{u}' = \mathbf{f}(t, \mathbf{u}).$$

例题 7.2 Lorenz 方程 Lorenz 方程是非线性微分方程组中最著名的混沌模型之一:

$$\begin{aligned} \frac{dx}{dt} &= \sigma(y - x), \\ \frac{dy}{dt} &= x(r - z) - y, \\ \frac{dz}{dt} &= xy - bz. \end{aligned}$$

当

$$\sigma = 10, \quad b = \frac{8}{3}, \quad r = 28$$

时, 系统表现出混沌行为 (chaos), 即对初始条件具有敏感依赖 (sensitive dependence on initial conditions)。微小初值差异会在有限时间后产生完全不同的轨道。

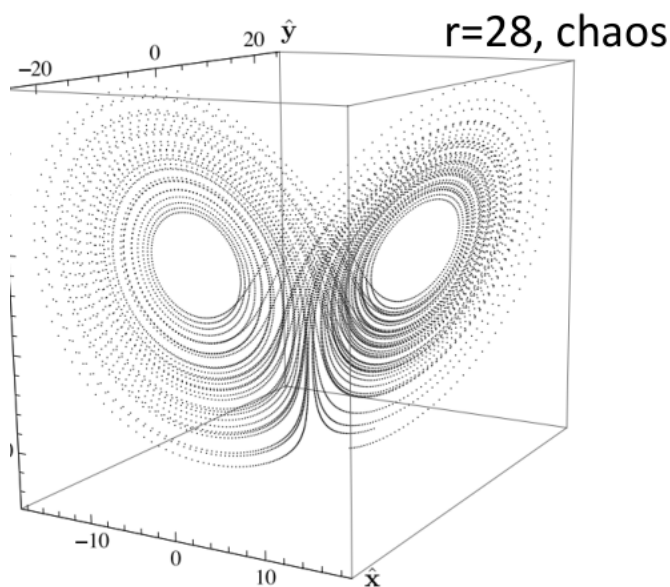


图 7.1: Lorenz 方程在 $r = 28$ 时的混沌吸引子示意图。

7.4.2 方程组的四阶 Runge–Kutta 方法

对 m 个变量的方程组，在第 j 个时间步 t_j 上记近似值为 $w_{1,j}, \dots, w_{m,j}$ 。四阶 Runge–Kutta 方法可逐分量写为

$$\begin{aligned} k_{1,i} &= hf_i(t_j, w_{1,j}, w_{2,j}, \dots, w_{m,j}), \\ k_{2,i} &= hf_i\left(t_j + \frac{h}{2}, w_{1,j} + \frac{1}{2}k_{1,1}, \dots, w_{m,j} + \frac{1}{2}k_{1,m}\right), \\ k_{3,i} &= hf_i\left(t_j + \frac{h}{2}, w_{1,j} + \frac{1}{2}k_{2,1}, \dots, w_{m,j} + \frac{1}{2}k_{2,m}\right), \\ k_{4,i} &= hf_i(t_j + h, w_{1,j} + k_{3,1}, \dots, w_{m,j} + k_{3,m}), \\ w_{i,j+1} &= w_{i,j} + \frac{1}{6}(k_{1,i} + 2k_{2,i} + 2k_{3,i} + k_{4,i}), \quad i = 1, 2, \dots, m. \end{aligned}$$

需要注意的是，某个分量的 $k_{2,i}$ 、 $k_{3,i}$ 、 $k_{4,i}$ 往往依赖所有变量的前一级 k ，因此实现时要先计算完所有分量的 k_1 ，再进入 k_2 ，依此类推。

7.4.3 高阶微分方程化为一阶方程组

高阶微分方程 (higher-order differential equation)

$$y^{(m)} = f(t, y, y', \dots, y^{(m-1)}), \quad a \leq t \leq b,$$

配合初值

$$y(a) = \alpha_1, \quad y'(a) = \alpha_2, \quad \dots, \quad y^{(m-1)}(a) = \alpha_m$$

可以化为一阶方程组。令

$$u_1 = y, \quad u_2 = y', \quad \dots, \quad u_m = y^{(m-1)},$$

则

$$\frac{du_1}{dt} = u_2, \quad \frac{du_2}{dt} = u_3, \quad \dots, \quad \frac{du_{m-1}}{dt} = u_m,$$

$$\frac{du_m}{dt} = f(t, u_1, u_2, \dots, u_m).$$

初值变为

$$u_1(a) = \alpha_1, \quad u_2(a) = \alpha_2, \quad \dots, \quad u_m(a) = \alpha_m.$$

这样就可以直接使用 Runge–Kutta 等一阶方程组算法。

例题 7.3 二阶方程的 RK4 求解 考虑

$$\begin{aligned} y'' - 2y' + 2y &= e^{2t} \sin t, & 0 \leq t \leq 1, \\ y(0) &= -0.4, & y'(0) = -0.6. \end{aligned}$$

令

$$u_1(t) = y(t), \quad u_2(t) = y'(t),$$

则原二阶方程可化为一阶方程组

$$\begin{aligned} u_1'(t) &= u_2(t), \\ u_2'(t) &= e^{2t} \sin t - 2u_1(t) + 2u_2(t), \end{aligned}$$

初值为

$$u_1(0) = -0.4, \quad u_2(0) = -0.6.$$

记

$$\begin{aligned} f_1(t, u_1, u_2) &= u_2, \\ f_2(t, u_1, u_2) &= e^{2t} \sin t - 2u_1 + 2u_2. \end{aligned}$$

取步长

$$h = 0.1, \quad t_0 = 0,$$

并记

$$w_{1,0} = u_1(0) = -0.4, \quad w_{2,0} = u_2(0) = -0.6.$$

按照方程组的四阶 Runge–Kutta 方法, 有

$$\begin{aligned} k_{1,1} &= hf_1(t_0, w_{1,0}, w_{2,0}) = hw_{2,0} = 0.1(-0.6) = -0.06, \\ k_{1,2} &= hf_2(t_0, w_{1,0}, w_{2,0}) \\ &= h[e^{2t_0} \sin t_0 - 2w_{1,0} + 2w_{2,0}] \\ &= 0.1[0 - 2(-0.4) + 2(-0.6)] \\ &= -0.04. \end{aligned}$$

第二阶段为

$$\begin{aligned} k_{2,1} &= hf_1\left(t_0 + \frac{h}{2}, w_{1,0} + \frac{1}{2}k_{1,1}, w_{2,0} + \frac{1}{2}k_{1,2}\right) \\ &= h\left(w_{2,0} + \frac{1}{2}k_{1,2}\right) \\ &= 0.1\left(-0.6 - \frac{0.04}{2}\right) \\ &= -0.062, \end{aligned}$$

$$\begin{aligned}
k_{2,2} &= hf_2 \left(t_0 + \frac{h}{2}, w_{1,0} + \frac{1}{2}k_{1,1}, w_{2,0} + \frac{1}{2}k_{1,2} \right) \\
&= h \left[e^{2(t_0+0.05)} \sin(t_0 + 0.05) - 2 \left(w_{1,0} + \frac{1}{2}k_{1,1} \right) + 2 \left(w_{2,0} + \frac{1}{2}k_{1,2} \right) \right] \\
&= 0.1 [e^{0.1} \sin 0.05 - 2(-0.4 - 0.03) + 2(-0.6 - 0.02)] \\
&= -0.03247644757.
\end{aligned}$$

第三阶段为

$$\begin{aligned}
k_{3,1} &= hf_1 \left(t_0 + \frac{h}{2}, w_{1,0} + \frac{1}{2}k_{2,1}, w_{2,0} + \frac{1}{2}k_{2,2} \right) \\
&= h \left(w_{2,0} + \frac{1}{2}k_{2,2} \right) \\
&= 0.1 \left(-0.6 + \frac{1}{2}(-0.03247644757) \right) \\
&= -0.06162382238, \\
k_{3,2} &= hf_2 \left(t_0 + \frac{h}{2}, w_{1,0} + \frac{1}{2}k_{2,1}, w_{2,0} + \frac{1}{2}k_{2,2} \right) \\
&= h \left[e^{2(t_0+0.05)} \sin(t_0 + 0.05) - 2 \left(w_{1,0} + \frac{1}{2}k_{2,1} \right) + 2 \left(w_{2,0} + \frac{1}{2}k_{2,2} \right) \right] \\
&= 0.1 \left[e^{0.1} \sin 0.05 - 2(-0.4 - 0.031) + 2 \left(-0.6 - \frac{0.03247644757}{2} \right) \right] \\
&= -0.03152409237.
\end{aligned}$$

第四阶段为

$$\begin{aligned}
k_{4,1} &= hf_1(t_0 + h, w_{1,0} + k_{3,1}, w_{2,0} + k_{3,2}) \\
&= h(w_{2,0} + k_{3,2}) \\
&= 0.1(-0.6 - 0.03152409237) \\
&= -0.06315240924, \\
k_{4,2} &= hf_2(t_0 + h, w_{1,0} + k_{3,1}, w_{2,0} + k_{3,2}) \\
&= h [e^{2(t_0+0.1)} \sin(t_0 + 0.1) - 2(w_{1,0} + k_{3,1}) + 2(w_{2,0} + k_{3,2})] \\
&= 0.1 [e^{0.2} \sin 0.1 - 2(-0.4 - 0.06162382238) + 2(-0.6 - 0.03152409237)] \\
&= -0.02178637298.
\end{aligned}$$

因此

$$\begin{aligned}
w_{1,1} &= w_{1,0} + \frac{1}{6}(k_{1,1} + 2k_{2,1} + 2k_{3,1} + k_{4,1}) \\
&= -0.4 + \frac{1}{6} [-0.06 + 2(-0.062) + 2(-0.06162382238) - 0.06315240924] \\
&= -0.4617333423, \\
w_{2,1} &= w_{2,0} + \frac{1}{6}(k_{1,2} + 2k_{2,2} + 2k_{3,2} + k_{4,2}) \\
&= -0.6 + \frac{1}{6} [-0.04 + 2(-0.03247644757) + 2(-0.03152409237) - 0.02178637298] \\
&= -0.6316312421.
\end{aligned}$$

所以由四阶 Runge-Kutta 方法得到

$$u_1(0.1) = y(0.1) \approx -0.4617333423,$$

$$u_2(0.1) = y'(0.1) \approx -0.6316312421.$$

该初值问题的精确解为

$$u_1(t) = y(t) = 0.2e^{2t}(\sin t - 2 \cos t),$$

$$u_2(t) = y'(t) = 0.2e^{2t}(4 \sin t - 3 \cos t).$$

因此

$$u_1(0.1) = y(0.1) = 0.2e^{2(0.1)}(\sin 0.1 - 2 \cos 0.1),$$

$$u_2(0.1) = y'(0.1) = 0.2e^{2(0.1)}(4 \sin 0.1 - 3 \cos 0.1).$$

这个例子说明：对于高阶微分方程，使用 Runge-Kutta 方法前，通常先引入新变量，将高阶方程化为一阶方程组，再对方程组整体应用 RK4 方法。

7.5 刚性微分方程与稳定性

7.5.1 刚性方程的现象

刚性微分方程 (stiff differential equation) 通常同时含有快衰减和慢变化尺度。即使精确解已经变得平滑，显式数值方法仍可能被快速衰减模的稳定性限制，迫使步长取得非常小。

一个典型刚性方程组为

$$\begin{aligned} u_1' &= 9u_1 + 24u_2 + 5 \cos t - \frac{1}{3} \sin t, & u_1(0) &= \frac{4}{3}, \\ u_2' &= -24u_1 - 51u_2 - 9 \cos t + \frac{1}{3} \sin t, & u_2(0) &= \frac{2}{3}. \end{aligned}$$

其精确解为

$$u_1(t) = 2e^{-3t} - e^{-39t} + \frac{1}{3} \cos t,$$

$$u_2(t) = -e^{-3t} + 2e^{-39t} - \frac{1}{3} \cos t.$$

其中 e^{-39t} 是快速衰减项， e^{-3t} 和 $\cos t$ 是相对慢的项。显式方法必须同时稳定处理快模和慢模，因而步长会受到快模控制。

更简单的一维例子是

$$y' = -30y, \quad 0 \leq t \leq 1.5, \quad y(0) = \frac{1}{3},$$

$$y(t) = \frac{1}{3}e^{-30t}.$$

若取 $h = 0.1$ ，在 $t = 1.5$ 附近有

方法	近似结果
Exact solution	9.54173×10^{-21}
Euler's method	-1.09225×10^4
Runge-Kutta method	3.95730×10^1
Predictor-corrector method	8.03840×10^5

精确解几乎已经为零，而显式数值解却剧烈发散。这不是模型本身爆炸，而是数值稳定性失败。

7.5.2 隐式梯形法与 Gear 方法

对刚性方程，隐式方法常更稳健。隐式梯形法 (implicit trapezoidal method) 为

$$w_0 = \alpha,$$

$$w_{i+1} = w_i + \frac{h}{2} [f(t_{i+1}, w_{i+1}) + f(t_i, w_i)].$$

令

$$F(w) = w - w_i - \frac{h}{2} [f(t_{i+1}, w) + f(t_i, w_i)],$$

则每一步要求解 $F(w_{i+1}) = 0$ 。可用 Newton 法迭代：

$$w_{i+1}^{(k+1)} = w_{i+1}^{(k)} - \frac{F(w_{i+1}^{(k)})}{F'(w_{i+1}^{(k)})}.$$

这里

$$F'(w) = 1 - \frac{h}{2} f_y(t_{i+1}, w).$$

对方程组则需要求解 Jacobian 线性系统。

Gear 方法 (Gear's method) 是一类用于刚性问题的后向差分公式 (backward differentiation formula, BDF)。其抽象形式可写为

$$y_{n+k} - h\beta_k f(x_{n+k}, y_{n+k}) = g_{n+k},$$

$$g_{n+k} = - \sum_{j=0}^{k-1} \alpha_j y_{n+j}.$$

二阶和三阶的典型形式分别为

$$y_{n+2} = \frac{4y_{n+1} - y_n + 2hf_{n+2}}{3},$$

$$y_{n+3} = \frac{18y_{n+2} - 9y_{n+1} + 2y_n + 6hf_{n+3}}{11}.$$

由于 $f_{n+k} = f(x_{n+k}, y_{n+k})$ 含未知量，Gear 方法也是隐式方法。对方程组 $\mathbf{y}' = \mathbf{f}(x, \mathbf{y})$ ，Newton 迭代为

$$\mathbf{y}_{n+k}^{(s+1)} = \mathbf{y}_{n+k}^{(s)} - \left[I - h\beta_k \frac{\partial \mathbf{f}}{\partial \mathbf{y}}(x_{n+k}, \mathbf{y}_{n+k}^{(s)}) \right]^{-1} \left[\mathbf{y}_{n+k}^{(s)} - h\beta_k \mathbf{f}(x_{n+k}, \mathbf{y}_{n+k}^{(s)}) - \mathbf{g}_{n+k} \right].$$

隐式方法每步更贵，但允许更大的稳定步长，因此在刚性问题中总体上常更有效。

7.5.3 显式与隐式格式的稳定性比较

考虑测试方程

$$y' = -cy, \quad c > 0.$$

显式 Euler 法给出

$$y_{n+1} = y_n + hy'_n = (1 - ch)y_n.$$

若 $|1 - ch| > 1$ ，误差会在迭代中放大，格式不稳定。稳定条件为

$$|1 - ch| < 1, \quad 0 < h < \frac{2}{c}.$$

隐式 Euler 法则写为

$$y_{n+1} = y_n + hy'_{n+1} = y_n - hc y_{n+1},$$

因此

$$y_{n+1} = \frac{y_n}{1 + ch}.$$

只要 $c > 0$ 、 $h > 0$ ，放大因子 $1/(1 + ch)$ 的绝对值总小于 1，因此稳定性明显更好。

对线性方程组

$$\mathbf{y}' = -C\mathbf{y},$$

若 C 可对角化, 记

$$TCT^{-1} = \text{diag}(\lambda_0, \lambda_1, \dots, \lambda_{N-1}),$$

并令

$$\mathbf{z} = T\mathbf{y},$$

则

$$\mathbf{z}' = -\text{diag}(\lambda_0, \lambda_1, \dots, \lambda_{N-1})\mathbf{z}.$$

原问题的各个模按

$$e^{-\lambda_i x}$$

衰减。显式 Euler 法的放大因子为

$$1 - h\lambda_i,$$

稳定条件是

$$|1 - h\lambda_i| < 1, \quad i = 0, 1, \dots, N-1.$$

若所有 $\lambda_i > 0$, 则必须满足

$$h < \frac{2}{\lambda_{\max}}.$$

刚性系统中 $\lambda_{\max}$ 很大, 显式步长因而被迫很小。

对一般非线性方程

$$y' = f(y),$$

显式差分为

$$y_{n+1} = y_n + hf(y_n),$$

隐式差分为

$$y_{n+1} = y_n + hf(y_{n+1}).$$

把 $f(y_{n+1})$ 在 y_n 附近线性化:

$$f(y_{n+1}) \approx f(y_n) + \frac{\partial f}{\partial y}(y_n)(y_{n+1} - y_n),$$

可得

$$y_{n+1} = y_n + h \left[1 - h \frac{\partial f}{\partial y}(y_n) \right]^{-1} f(y_n).$$

对方程组, 括号中的量变成矩阵

$$I - h \frac{\partial \mathbf{f}}{\partial \mathbf{y}},$$

每一步要求逆或解线性方程组。稳定性改善的代价正是线性代数计算量。

结论 显式方法每步便宜, 适合非刚性问题; 隐式方法每步昂贵, 但稳定区域通常大得多, 适合刚性方程、扩散型方程和含有快速衰减模的问题。

7.6 边值问题与打靶法

7.6.1 边值问题的基本形式

边值问题 (boundary value problem, BVP) 通常要求解

$$y'' = f(x, y, y'), \quad a \leq x \leq b,$$

并满足两端边界条件

$$y(a) = \alpha, \quad y(b) = \beta.$$

与初值问题不同, BVP 不知道一端的完整初始斜率, 而是要求积分后的结果在另一端满足条件。打靶法 (shooting method) 的思想是把未知初始斜率当作参数, 通过反复试射把 BVP 转化为 IVP。

7.6.2 线性打靶法

对线性边值问题

$$y'' = p(x)y' + q(x)y + r(x), \quad y(a) = \alpha, \quad y(b) = \beta,$$

先解两个初值问题:

$$y_1'' = p(x)y_1' + q(x)y_1 + r(x), \quad y_1(a) = \alpha, \quad y_1'(a) = 0,$$

$$y_2'' = p(x)y_2' + q(x)y_2, \quad y_2(a) = 0, \quad y_2'(a) = 1.$$

若 $y_2(b) \neq 0$, 则

$$y(x) = y_1(x) + \frac{\beta - y_1(b)}{y_2(b)} y_2(x)$$

满足两端边界条件。验证如下:

$$y(a) = y_1(a) + \frac{\beta - y_1(b)}{y_2(b)} y_2(a) = \alpha,$$

$$y(b) = y_1(b) + \frac{\beta - y_1(b)}{y_2(b)} y_2(b) = \beta.$$

同时, 由于方程是线性的,

$$y'' = y_1'' + \frac{\beta - y_1(b)}{y_2(b)} y_2'' = p(x)y' + q(x)y + r(x),$$

因此这个线性组合既满足微分方程, 也满足边界条件。实际计算中可先用四阶 Runge-Kutta 分别求 $y_1(x)$ 与 $y_2(x)$, 再按上式组合。

7.6.3 非线性打靶法

非线性情形可把初始斜率写作参数

$$y(a) = \alpha, \quad y'(a) = t.$$

对每个 t , 初值问题给出一条解 $y(x, t)$ 。边界条件要求

$$F(t) = y(b, t) - \beta = 0.$$

因此问题转化为一元非线性方程求根。

割线法 (secant method) 可写为

$$t_k = t_{k-1} - \frac{[y(b, t_{k-1}) - \beta](t_{k-1} - t_{k-2})}{y(b, t_{k-1}) - y(b, t_{k-2})}, \quad k = 2, 3, \dots$$

Newton 法为

$$t_k = t_{k-1} - \frac{y(b, t_{k-1}) - \beta}{\frac{d}{dt}y(b, t_{k-1})}.$$

Newton 法通常收敛更快，但关键困难是计算 $\frac{d}{dt}y(b, t)$ 。

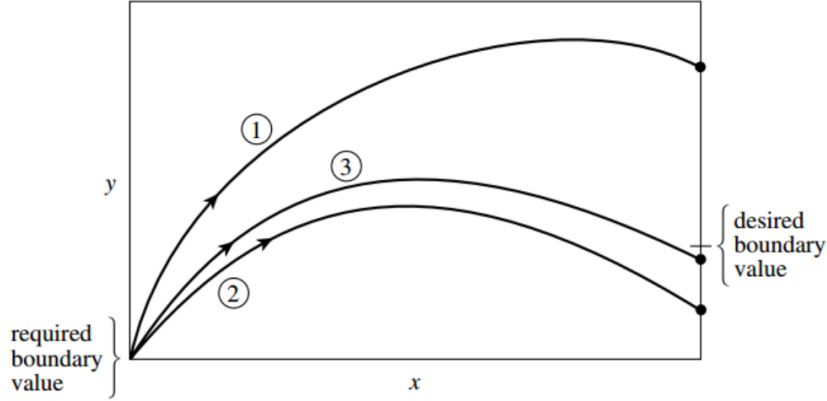


图 7.2: 打靶法示意图：从满足左端边界条件的初始点出发，调节初始斜率，使右端边界偏差逐步减小。

7.6.4 灵敏度方程与 Newton 打靶

设

$$y''(x, t) = f(x, y(x, t), y'(x, t)), \quad y(a, t) = \alpha, \quad y'(a, t) = t.$$

令

$$z(x, t) = \frac{\partial y}{\partial t}(x, t).$$

由链式法则，

$$\frac{\partial y'}{\partial t}(x, t) = \frac{\partial}{\partial x} \left(\frac{\partial y}{\partial t}(x, t) \right) = z'(x, t),$$

并且

$$\frac{\partial y''}{\partial t}(x, t) = \frac{\partial f}{\partial y}(x, y, y')z(x, t) + \frac{\partial f}{\partial y'}(x, y, y')z'(x, t).$$

因此 z 满足线性变分方程

$$z'' = f_y(x, y, y')z + f_{y'}(x, y, y')z', \quad a \leq x \leq b.$$

初值来自

$$z(a, t) = \frac{\partial y(a, t)}{\partial t} = 0, \quad z'(a, t) = \frac{\partial y'(a, t)}{\partial t} = 1.$$

于是 Newton 迭代可写为

$$t_k = t_{k-1} - \frac{y(b, t_{k-1}) - \beta}{z(b, t_{k-1})}.$$

计算时用 Runge–Kutta 同时积分 $y(x, t)$ 、 $y'(x, t)$ 、 $z(x, t)$ 、 $z'(x, t)$ ，然后更新初始斜率 t 。这样就避免了对 $y(b, t)$ 做数值差分。

7.7 Numerov 方法

7.7.1 基本推导

Numerov 方法 (Numerov's method) 是一种专门用于二阶线性微分方程的高精度算法, 特别适合形如

$$\frac{d^2x}{dt^2} = f(t)x(t)$$

或

$$\frac{d^2y}{dx^2} = -g(x)y + s(x)$$

的方程。它的局部误差可达到 $O(h^6)$, 优于许多常规四阶 Runge-Kutta 实现中的二阶方程处理方式。

对 $x(t)$ 在 0 附近作 Taylor 展开并把 $x(h)$ 与 $x(-h)$ 相加:

$$x(h) + x(-h) - 2x(0) = h^2 x^{(2)}(0) + \frac{h^4}{12} x^{(4)}(0) + \frac{h^6}{360} x^{(6)}(0) + O(h^8).$$

由方程 $x'' = f(t)x(t)$, 有

$$x^{(2)}(0) = f(0)x(0).$$

而四阶导数可用中心差分近似:

$$x^{(4)}(0) = \frac{f(h)x(h) + f(-h)x(-h) - 2f(0)x(0)}{h^2} + O(h^2).$$

引入辅助变量

$$w(t) = \left[1 - \frac{h^2}{12} f(t) \right] x(t),$$

则可整理为

$$w(h) + w(-h) - 2w(0) = h^2 f(0)x(0) + O(h^6),$$

即

$$x(t) = \left[1 - \frac{h^2}{12} f(t) \right]^{-1} w(t).$$

这就是 Numerov 格式误差高的原因: 它把二阶导数项中的主要高阶修正吸收到 $w(t)$ 中。

若已知 $x(0)$ 与 $x'(0)$, 还需要构造 $x(h)$ 作为递推起点。可由

$$2hx'(0) = \left[1 - \frac{h^2}{6} f(h) \right] x(h) - \left[1 - \frac{h^2}{6} f(-h) \right] x(-h) + O(h^5)$$

以及上面的中心关系联立, 得到 $x(h)$ 的高阶近似。实践中更常见的做法是用 Taylor 展开或 Runge-Kutta 给出前两个点, 然后进入 Numerov 递推。

7.7.2 非齐次 Numerov 公式

考虑

$$\frac{d^2y}{dx^2} + k^2(x)y = F(x), \quad a \leq x \leq b, \quad y(a) = \alpha, \quad y(b) = \beta.$$

其中 $F(x)$ 是非齐次驱动力 (inhomogeneous driving force), $k(x)$ 是实函数。中心差展开给出

$$y^{(2)}(x) = \frac{y(x+h) + y(x-h) - 2y(x)}{h^2} - \frac{h^2}{12} y^{(4)}(x) + O(h^6).$$

把算符

$$1 + \frac{h^2}{12} \frac{d^2}{dx^2}$$

作用在微分方程上, 可以把四阶导数项吸收进相邻格点的 $k^2 y$ 与 F 中, 得到

$$y_{i+1} = \frac{2\left(1 - \frac{5h^2}{12}k_i^2\right)y_i - \left(1 + \frac{h^2}{12}k_{i-1}^2\right)y_{i-1} + \frac{h^2}{12}(F_{i+1} + 10F_i + F_{i-1})}{1 + \frac{h^2}{12}k_{i+1}^2} + O(h^6).$$

若写成

$$\frac{d^2 y}{dx^2} = -g(x)y + s(x),$$

则等价递推为

$$y_{n+1} \left(1 + \frac{h^2}{12}g_{n+1}\right) = 2y_n \left(1 - \frac{5h^2}{12}g_n\right) - y_{n-1} \left(1 + \frac{h^2}{12}g_{n-1}\right) + \frac{h^2}{12}(s_{n+1} + 10s_n + s_{n-1}).$$

 **笔记** Numerov 方法本质上是三点递推, 因此需要两个起始值, 例如 y_0 与 y_1 。它对 Schrödinger 径向方程尤其有用, 因为那类方程通常正是二阶线性方程。

7.8 打靶法求解球对称势 Schrödinger 方程

7.8.1 径向方程与方势阱

对球对称势, 波函数可分离为

$$\psi(\mathbf{r}) = R(r)Y_l^m(\theta, \phi), \quad R(r) = \frac{1}{r}u(r), \quad \rho = \frac{r}{a}.$$

径向 Schrödinger 方程可写为

$$-\frac{\hbar^2}{2ma^2} \frac{d^2 u}{d\rho^2} + \left[V(\rho) + \frac{l(l+1)}{\rho^2} \frac{\hbar^2}{2ma^2} \right] u(\rho) = Eu(\rho).$$

令

$$V(r) = -V_0 f(r),$$

并定义

$$\gamma = \frac{2ma^2 V_0}{\hbar^2}, \quad \epsilon = \frac{|E|}{V_0},$$

则方程可化为

$$\frac{d^2 u}{d\rho^2} + k(\rho)u(\rho) = 0,$$

其中

$$k(\rho) = \gamma \left[f(\rho) - \frac{1}{\gamma} \frac{l(l+1)}{\rho^2} - \epsilon \right].$$

对方势阱 (square well),

$$f(r) = \begin{cases} 1, & r \leq a, \\ 0, & r > a. \end{cases}$$

于是

$$k(\rho) = \begin{cases} \gamma \left(1 - \epsilon - \frac{1}{\gamma} \frac{l(l+1)}{\rho^2} \right), & r \leq a, \\ \gamma \left(-\epsilon - \frac{1}{\gamma} \frac{l(l+1)}{\rho^2} \right), & r > a. \end{cases}$$

7.8.2 束缚态边界条件与能量匹配

在原点附近, 离心势主导, 径向方程近似为

$$\frac{d^2 u}{d\rho^2} - \frac{l(l+1)}{\rho^2} u(\rho) = 0,$$

正则解满足

$$u(\rho) \propto \rho^{l+1}, \quad \rho \rightarrow 0.$$

在远离势阱的区域, 束缚态 $E < 0$ 要求波函数指数衰减:

$$\begin{aligned} \frac{d^2 u}{d\rho^2} - \gamma \epsilon u(\rho) &= 0, \quad \gamma > 0, \\ u(\rho) &\propto e^{-\kappa \rho}, \quad \kappa = \sqrt{\gamma \epsilon}, \quad \rho \rightarrow \infty. \end{aligned}$$

等价地, 物理径向波函数满足

$$rR(r) \rightarrow 0, \quad u(r) \rightarrow 0, \quad r \rightarrow \infty.$$

数值上可从原点向外积分得到 $u^<(\rho)$, 再从足够大的外边界向内积分得到 $u^>(\rho)$ 。给定一个较大的匹配点 ρ_m , 要求对数导数连续:

$$\frac{\frac{d}{d\rho} u^<(\rho)}{u^<(\rho)} = \frac{\frac{d}{d\rho} u^>(\rho)}{u^>(\rho)} \quad \text{at } \rho = \rho_m.$$

这个条件把能量 E 变成求根变量。只有使左右积分在 ρ_m 处光滑连接的 E , 才是束缚态本征能量。

7.8.3 散射态与共振态

散射态的远区边界条件不是指数衰减, 而是含相移的振荡形式:

$$u_l(r) \sim C_l \sin\left(kr - \frac{l\pi}{2} + \delta_l\right), \quad r \rightarrow \infty.$$

其中 δ_l 是第 l 分波相移。共振态 (resonant state) 可用复能量

$$E - \frac{i\Gamma}{2}$$

描述; 按散射态求解时, 共振附近相移会突然增加约 $\pi/2$ 。常见的共振态数值方法包括复坐标标度法 (complex-scaling method)、稳定化方法 (stabilization method) 和 S -矩阵极点法 (S -matrix pole method)。

7.9 Green 函数与 Wronskian

7.9.1 一个 Dirichlet 边值问题

考虑

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u(0) = u(1) = 0.$$

先不使用 Green 函数, 直接积分:

$$u(x) = c_1 + \int_0^x u'(y) dy,$$

$$u'(y) = c_2 + \int_0^y u''(z) dz = c_2 - \int_0^y f(z) dz.$$

因此

$$u(x) = c_1 + c_2 x - \int_0^x \left(\int_0^y f(z) dz \right) dy.$$

交换积分次序:

$$\int_0^x \left(\int_0^y f(z) dz \right) dy = \int_0^x (x-y)f(y) dy,$$

从而

$$u(x) = c_1 + c_2 x - \int_0^x (x-y)f(y) dy.$$

由 $u(0) = 0$ 得 $c_1 = 0$, 由 $u(1) = 0$ 得

$$c_2 = \int_0^1 (1-y)f(y) dy.$$

于是

$$u(x) = x \int_0^1 (1-y)f(y) dy - \int_0^x (x-y)f(y) dy.$$

7.9.2 Green 函数形式

上式可写成

$$u(x) = \int_0^1 G(x, y)f(y) dy,$$

其中

$$G(x, y) = \begin{cases} y(1-x), & 0 \leq y \leq x, \\ x(1-y), & x \leq y \leq 1. \end{cases}$$

这个 $G(x, y)$ 称为 Green 函数 (Green's function)。它具有如下性质:

1. $G(x, y)$ 关于 x 连续;
2. $G(x, y) = G(y, x)$, 即对称;
3. $G(0, y) = G(1, y) = G(x, 0) = G(x, 1) = 0$, 满足齐次边界条件;
4. 当 $x \neq y$ 时, G 对 x 是分段线性函数;
5. 在 $x = y$ 处, 一阶导数存在跳跃:

$$\left. \frac{\partial G}{\partial x} \right|_{x=y^+} - \left. \frac{\partial G}{\partial x} \right|_{x=y^-} = 1$$

6. $G(x, y) \geq 0$, 且满足

$$\frac{\partial^2}{\partial x^2} G(x, y) = -\delta(x - y).$$

因此

$$\int_0^1 \delta(x - y)f(y) dy = f(x),$$

保证了

$$-u''(x) = f(x).$$

Green 函数可以理解为线性系统对点源 $\delta(x - y)$ 的响应; 任意外源 $f(y)$ 的解就是点源响应的叠加。

7.9.3 Wronskian 与一般二阶方程

对齐次二阶方程的两个解 $u_<$ 、 $u_>$, Wronskian (Wronskian) 定义为

$$W[u_<, u_>](x) = u_<(x)u_>'(x) - u_<'(x)u_>(x).$$

通常取 $u_<$ 为满足左端边界条件并从 a 向 b 积分得到的解, 取 $u_>$ 为满足右端边界条件并从 b 向 a 积分得到的解。记

$$x_< = \min(x, y), \quad x_> = \max(x, y).$$

对许多二阶自伴问题, Green 函数可由

$$G(x, y) = \frac{u_<(x_<)u_>(x_>)}{W}$$

给出, 符号取决于微分算符的约定。前面的例子中可取

$$u_<(x) = x, \quad u_>(x) = 1 - x,$$

从而得到相同的 $G(x, y)$ 。

更一般地, 对

$$\frac{d^2 y}{dx^2} + a_1(x) \frac{dy}{dx} + a_0(x)y = f(x),$$

配合边界条件

$$B_a(y) = 0, \quad B_b(y) = 0,$$

设 y_1, y_2 是对应齐次方程的两个解, 分别满足 a 、 b 两端边界条件。则可用变参数法写出

$$y(x) = \int_a^x \frac{y_1(s)y_2(x)f(s)}{W(y_1, y_2)(s)} ds + \int_x^b \frac{y_1(x)y_2(s)f(s)}{W(y_1, y_2)(s)} ds.$$

对应 Green 函数为

$$G(x, s) = \begin{cases} \frac{y_1(s)y_2(x)}{W(y_1, y_2)(s)}, & a \leq s \leq x \leq b, \\ \frac{y_1(x)y_2(s)}{W(y_1, y_2)(s)}, & a \leq x \leq s \leq b. \end{cases}$$

于是

$$y(x) = \int_a^b G(x, s)f(s) ds.$$

这表明 Green 函数只由线性算符和边界条件决定, 与具体外场 f 无关。

7.10 有限差分法求解常微分方程

7.10.1 中心差分离散

有限差分法 (finite difference method) 把微分算符直接替换为网格上的差分算符。考虑

$$f''(x) + p(x)f'(x) + q(x)f(x) = s(x),$$

并取均匀网格

$$x_n = n\Delta x.$$

中心差分公式为

$$\begin{aligned} f'(x_n) &= \frac{f(x_n + \Delta x) - f(x_n - \Delta x)}{2\Delta x} + \frac{1}{6}f'''(x_n)(\Delta x)^2 + O((\Delta x)^4), \\ f''(x_n) &= \frac{f(x_n + \Delta x) - 2f(x_n) + f(x_n - \Delta x))}{(\Delta x)^2} + \frac{1}{12}f^{(4)}(x_n)(\Delta x)^2 + O((\Delta x)^4). \end{aligned}$$

记

$$f_n = f(x_n), \quad p_n = p(x_n), \quad q_n = q(x_n), \quad s_n = s(x_n),$$

代入微分方程并忽略 $O((\Delta x)^2)$ 截断项, 得到

$$f_n \left(q_n - \frac{2}{(\Delta x)^2} \right) + f_{n+1} \left(\frac{p_n}{2\Delta x} + \frac{1}{(\Delta x)^2} \right) + f_{n-1} \left(-\frac{p_n}{2\Delta x} + \frac{1}{(\Delta x)^2} \right) = s_n.$$

每个内部点只与相邻两个点耦合, 因此线性系统是三对角的。

7.10.2 边界条件与线性方程组

若左端边界 f_0 已知, 则 $n = 1$ 的方程可写成

$$f_1 \left(q_1 - \frac{2}{(\Delta x)^2} \right) + f_2 \left(\frac{p_1}{2\Delta x} + \frac{1}{(\Delta x)^2} \right) = s_1 - f_0 \left(-\frac{p_1}{2\Delta x} + \frac{1}{(\Delta x)^2} \right).$$

类似地，右端边界项会进入最后一个方程的右端。把未知量写成

$$\mathbf{f} = (f_1, f_2, \dots, f_N)^T,$$

可得

$$D\mathbf{f} = \mathbf{s} - \mathbf{b}.$$

其中 D 是三对角矩阵：

$$D = \begin{pmatrix} q_1 - \frac{2}{(\Delta x)^2} & \frac{p_1}{2\Delta x} + \frac{1}{(\Delta x)^2} & 0 & \cdots & 0 \\ -\frac{p_2}{2\Delta x} + \frac{1}{(\Delta x)^2} & q_2 - \frac{2}{(\Delta x)^2} & \frac{p_2}{2\Delta x} + \frac{1}{(\Delta x)^2} & \cdots & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & -\frac{p_{N-1}}{2\Delta x} + \frac{1}{(\Delta x)^2} & q_{N-1} - \frac{2}{(\Delta x)^2} & \frac{p_{N-1}}{2\Delta x} + \frac{1}{(\Delta x)^2} \\ 0 & \cdots & 0 & -\frac{p_N}{2\Delta x} + \frac{1}{(\Delta x)^2} & q_N - \frac{2}{(\Delta x)^2} \end{pmatrix}.$$

边界向量 $\mathbf{b}$ 只在首尾分量非零：

$$\mathbf{b} = \begin{pmatrix} f_0 \left(-\frac{p_1}{2\Delta x} + \frac{1}{(\Delta x)^2} \right) \\ 0 \\ \vdots \\ 0 \\ f_{N+1} \left(\frac{p_N}{2\Delta x} + \frac{1}{(\Delta x)^2} \right) \end{pmatrix}.$$

因此边值问题被转化为稀疏线性方程组。若 D 是严格三对角矩阵，可以用 Thomas 算法在 $O(N)$ 时间内求解；若问题来自更高维 PDE，矩阵仍然稀疏，但需要迭代法或专门的稀疏直接法。

 **笔记** 打靶法把边值问题转成初值问题，适合低维且初值敏感性不太强的系统；有限差分法直接离散整个区间，适合线性边值问题和可推广到偏微分方程的网格问题。

7.11 Tolman–Oppenheimer–Volkoff 方程

7.11.1 中子星结构方程

Tolman–Oppenheimer–Volkoff 方程 (Tolman–Oppenheimer–Volkoff equations, TOV equations) 描述广义相对论框架下静态球对称中子星的流体静力平衡。其基本形式为

$$\begin{aligned} \frac{dP(r)}{dr} &= -\frac{G}{r^2} \left[\rho(r) + \frac{P(r)}{c^2} \right] \left[m(r) + \frac{4\pi r^3 P(r)}{c^2} \right] \left[1 - \frac{2Gm(r)}{rc^2} \right]^{-1}, \\ m(r) &= \int_0^r d^3r' \rho(r'), \quad \frac{dm}{dr} = 4\pi r^2 \rho. \end{aligned}$$

其中 $P(r)$ 是压强， $\rho(r)$ 是质量密度， $m(r)$ 是半径 r 内包围的质量， G 是引力常数， c 是光速。与 Newton 引力下的流体平衡相比，TOV 方程多出相对论压强贡献和时空曲率修正。

求解 TOV 方程还需要核物理提供状态方程 (equation of state, EOS)：

$$P = P(\rho).$$

一个简单模型是多方状态方程

$$P = K\rho^\gamma,$$

例如可取

$$\gamma = 2.75, \quad K = 1.982 \times 10^{-6} \quad [\text{cgs}].$$

给定中心密度 $\rho(0)$ ，初始条件为

$$P(0) = P(\rho(0)), \quad m(0) = 0.$$

从 $r = 0$ 向外积分，直到压强降为零：

$$P(R) = 0, \quad m(R) = M.$$

由此得到半径 R 和质量 M 。扫描不同中心密度即可得到质量-半径关系，并进一步确定中子星最大质量。

$$P(R) = 0, \quad m(R) = M,$$

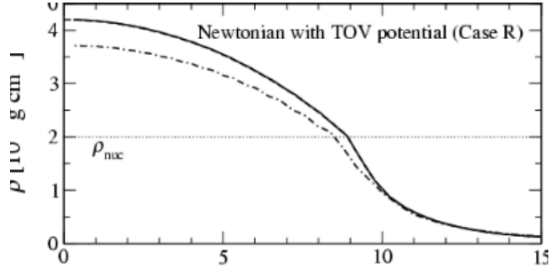


图 7.3: TOV 方程求解中的密度分布、状态方程与中子星内部结构示意图。

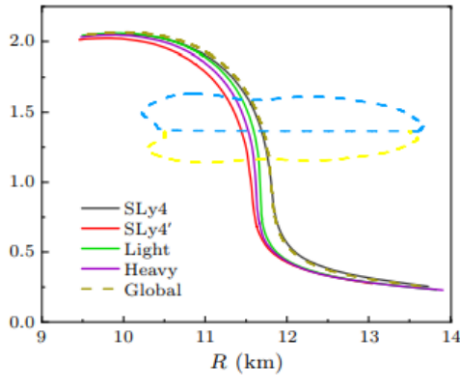


图 7.4: 不同状态方程给出的中子星质量-半径关系及与观测约束的比较。图中示例来自 Chong-Ji Jiang 等, *Chinese Physics Letters* 38(5):052101 (2021)。

7.11.2 数值积分流程

TOV 方程可看作关于 $P(r)$ 与 $m(r)$ 的一阶方程组：

$$\frac{d}{dr} \begin{pmatrix} P \\ m \end{pmatrix} = \begin{pmatrix} F_P(r, P, m, \rho) \\ 4\pi r^2 \rho \end{pmatrix}, \quad \rho = \rho(P)$$

其中 $\rho(P)$ 由 EOS 反解得到。典型 Runge-Kutta 流程为：

1. 选择中心密度 ρ_c ，令 $P_c = P(\rho_c)$ 、 $m(0) = 0$ ；
2. 从一个很小的 $r = \epsilon_r$ 开始积分，以避免 $r = 0$ 处的显式奇点；
3. 每一步由 EOS 更新 ρ ，再用 RK4 推进 P 和 m ；
4. 当 P 下降到零时停止，定义该点为星体半径 R ，此时 $M = m(R)$ ；
5. 改变 ρ_c ，重复计算，得到 $M(R)$ 曲线与最大质量。

该例说明，常微分方程求解不仅是数值分析练习，也是从微观核物理 EOS 连接到宏观天体观测的重要桥梁。

7.12 本章总结

结论 本章主线是常微分方程的初值问题和边值问题：初值问题强调时间推进、误差阶与稳定性；边值问题强调边界条件、打靶匹配、Green 函数和有限差分线性系统。

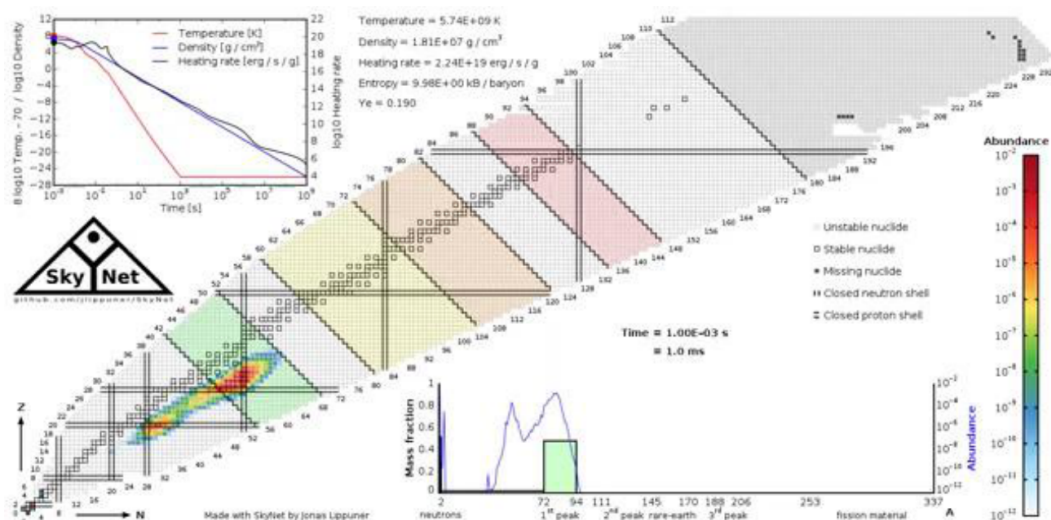


图 7.5: 核天体物理中的反应网络与丰度演化示意。类似问题常把大量核素丰度的演化写成高维微分方程组，需要稳定、高效的 ODE 算法。

- Euler 法是一阶 Taylor 方法，简单但精度低、稳定区域小；
- 高阶 Taylor 法可显著降低误差，但需要计算高阶全导数；
- Runge–Kutta 方法用多次斜率取样替代高阶导数，RK4 是物理计算中最常用的一步法之一；
- Runge–Kutta–Fehlberg 和 Runge–Kutta–Verner 方法通过嵌入公式实现局部误差估计和自适应步长；
- Predictor-corrector 方法结合显式预测和隐式校正，常用于高精度时间演化问题；
- 高阶微分方程可系统地转化为一阶方程组，方程组算法是 ODE 数值计算的统一基础；
- 刚性方程会迫使显式方法使用很小步长，隐式梯形法和 Gear 方法具有更好的稳定性；
- 打靶法通过调节未知初始斜率把边值问题转化为初值问题，线性打靶法可用两个解的线性组合严格满足边界条件；
- Numerov 方法对二阶线性方程特别高效，常用于径向 Schrödinger 方程；
- 束缚态 Schrödinger 方程可通过内外积分和对数导数匹配寻找本征能量，散射态和共振态则对应相移与复能量结构；
- Green 函数是线性响应函数，由微分算符和边界条件决定，与外源无关；
- 有限差分法把边值问题转化为稀疏线性方程组，是求解 ODE 与 PDE 的统一网格化思想；
- TOV 方程展示了 ODE 算法在中子星结构计算中的典型物理应用，EOS 决定质量-半径曲线和最大质量。

第八章 偏微分方程

内容提要

- 二维有限差分法、Poisson 方程五点格式与边界条件处理
- 高阶差分、Fourier 微分算符、FFT 及其在信号和图像处理中的应用
- 基矢展开法、谐振子基、离散变量表示 (DVR) 与 sinc DVR
- B-样条展开、轴对称方程离散、自旋轨道耦合与轴边界条件
- 小波、多小波、自适应网格、积分方程迭代与 MADNESS 框架
- 有限元方法、Coulomb 势边界、Jacobi 迭代与 Twisted 边界条件

偏微分方程 (partial differential equation, PDE) 描述未知函数关于多个自变量的变化规律, 是连续介质、场论、量子多体平均场、输运过程和电磁问题中的基本数学模型。与常微分方程相比, PDE 数值求解的核心困难在于: 未知量从一条曲线上的函数值扩展为区域或空间中的函数值, 边界条件、网格结构、离散算符、矩阵规模和稳定性都会同时影响结果。

本章按照计算物理中常见的 PDE 离散化路线展开: 二维有限差分法、Fourier 微分与 FFT、基矢展开法、离散变量表示 (discrete variable representation, DVR)、B-样条 (B-spline)、小波与多小波 (wavelet and multi-wavelet)、有限元方法 (finite element method, FEM), 最后讨论 Coulomb 势边界条件、Jacobi 迭代和 Twisted 边界条件。它们的共同目标都是把连续算符问题

$$\mathcal{L}u = f$$

转化为有限维代数问题

$$A\mathbf{u} = \mathbf{b} \quad \text{或} \quad H\mathbf{c} = E\mathbf{c},$$

其中 $\mathcal{L}$ 是微分或积分算符, A, H 是离散矩阵, $\mathbf{u}$ 表示网格值, $\mathbf{c}$ 表示展开系数。

8.1 二维有限差分法

8.1.1 二维 Poisson 方程与五点格式

定义 8.1 (二维有限差分法)

二维有限差分法 (two-dimensional finite difference method) 是在二维区域上布置离散网格, 用相邻网格点的函数值近似偏导数, 从而把偏微分方程转化为线性或非线性代数方程组的方法。它的优点是形式直观、矩阵稀疏、易于实现; 缺点是规则网格对复杂边界不够灵活, 高精度格式在边界处需要额外处理。

考虑矩形区域上的 Poisson 方程

$$u_{xx} + u_{yy} = f(x, y),$$

其中 $u(x, y)$ 是待求场, $f(x, y)$ 是给定源项。令

$$x_i = x_0 + ih_x, \quad y_j = y_0 + jh_y, \quad U_{ij} \approx u(x_i, y_j), \quad f_{ij} = f(x_i, y_j).$$

对 x 与 y 方向分别作 Taylor 展开:

$$u(x_i \pm h_x, y_j) = u_{ij} \pm h_x u_x + \frac{h_x^2}{2} u_{xx} \pm \frac{h_x^3}{6} u_{xxx} + \frac{h_x^4}{24} u_{xxxx} + O(h_x^5).$$

两式相加可消去奇数阶导数, 得到二阶中心差分

$$u_{xx}(x_i, y_j) = \frac{U_{i-1,j} - 2U_{ij} + U_{i+1,j}}{h_x^2} + O(h_x^2),$$

同理

$$u_{yy}(x_i, y_j) = \frac{U_{i,j-1} - 2U_{ij} + U_{i,j+1}}{h_y^2} + O(h_y^2).$$

代入 Poisson 方程, 得到五点差分格式

$$\frac{U_{i-1,j} - 2U_{ij} + U_{i+1,j}}{h_x^2} + \frac{U_{i,j-1} - 2U_{ij} + U_{i,j+1}}{h_y^2} = f_{ij} + T_{ij},$$

其中截断误差

$$T_{ij} = O(h_x^2 + h_y^2).$$

等价地,

$$\frac{U_{i-1,j} + U_{i+1,j}}{h_x^2} + \frac{U_{i,j-1} + U_{i,j+1}}{h_y^2} - \left(\frac{2}{h_x^2} + \frac{2}{h_y^2} \right) U_{ij} = f_{ij}.$$

当 $h_x = h_y = h$ 时,

$$U_{i-1,j} + U_{i+1,j} + U_{i,j-1} + U_{i,j+1} - 4U_{ij} = h^2 f_{ij}.$$

若问题写成更常用的

$$-\Delta u = f,$$

则五点格式变为

$$4U_{ij} - U_{i-1,j} - U_{i+1,j} - U_{i,j-1} - U_{i,j+1} = h^2 f_{ij}.$$

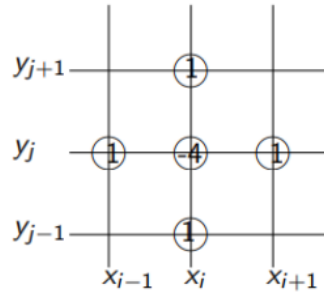


图 8.1: 二维五点差分格式。中心点 (i, j) 与左右上下四个邻点共同近似 Laplace 算符。

五点格式的物理意义是: 局域 Laplace 算符由中心点相对于邻点平均值的偏离来衡量。若 $f = 0$, 则离散调和函数满足

$$U_{ij} = \frac{1}{4} (U_{i-1,j} + U_{i+1,j} + U_{i,j-1} + U_{i,j+1}),$$

即每个内部点等于四个近邻的平均值; 非零源项 f 则表示该局域平均关系被源项破坏。

8.1.2 二维网格到一维未知量的投影

有限差分离散以后, 内部网格点的未知值需要排列成向量。设矩形区域在 x 方向有 $m-1$ 个内部点, 在 y 方向有 $n-1$ 个内部点, 可用

$$k = i + (m-1)(j-1), \quad i = 1, \dots, m-1, \quad j = 1, \dots, n-1$$

把二维指标 (i, j) 映射为一维指标 k 。例如 3×3 个内部点可排列为

$$\mathbf{x} = (U_{11}, U_{21}, U_{31}, U_{12}, U_{22}, U_{32}, U_{13}, U_{23}, U_{33})^T.$$

对方程 $\Delta u = f$ ，等距网格上的线性系统具有块三对角结构

$$A = \frac{1}{h^2} \begin{pmatrix} B & I & 0 \\ I & B & I \\ 0 & I & B \end{pmatrix}, \quad B = \begin{pmatrix} -4 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & -4 \end{pmatrix}.$$

这里 B 描述同一行内部点之间的 x 方向耦合，块外的单位矩阵 I 描述相邻行之间的 y 方向耦合。若采用 $-\Delta u = f$ 的号约定，则矩阵整体乘以 -1 ，主对角元为正，离对角近邻为负：

$$A = -\frac{1}{h^2} \begin{pmatrix} -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & -4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 \end{pmatrix}.$$

这种矩阵非常稀疏，每行最多只有五个非零元。二维 $N_x \times N_y$ 内部网格对应 $N_x N_y$ 个未知数；直接高斯消元通常代价较高，实际计算常利用稀疏矩阵存储、迭代法、共轭梯度法或多重网格法。

8.1.3 边界条件

PDE 的离散方程必须与边界条件共同构成完整问题。常见边界条件包括：

- Dirichlet 边界条件 (Dirichlet boundary condition)：直接给定边界函数值 $u|_{\partial D} = g$ ；
- Neumann 边界条件 (Neumann boundary condition)：给定法向导数 $\partial u / \partial n = g$ ；
- Mixed 边界条件或 Robin 边界条件 (mixed/Robin boundary condition)：给定函数值与法向导数的线性组合 $au + b \partial u / \partial n = g$ 。

Dirichlet 条件最直接。若边界点值已知，它们不作为未知量进入 $\mathbf{u}$ ，而是移到线性系统右端。例如对 $-\Delta u = f$ ，内部点 $(1, j)$ 与左边界点 $(0, j)$ 相邻：

$$4U_{1j} - U_{2j} - U_{1,j-1} - U_{1,j+1} = h^2 f_{1j} + U_{0j}.$$

右端的 $U_{0j} = g(x_0, y_j)$ 是已知边界贡献。

Neumann 条件可用虚点 (ghost point) 处理。设左边界 $x = x_0$ 上

$$\frac{\partial u}{\partial x}(x_0, y_j) = g(x_0, y_j).$$

中心差分给出

$$\frac{U_{1,j} - U_{-1,j}}{2h} = g(x_0, y_j), \quad U_{-1,j} = U_{1,j} - 2hg(x_0, y_j).$$

将虚点代入边界点的离散 Poisson 方程，例如对 $-\Delta u = f$ ，

$$4U_{0j} - U_{-1,j} - U_{1,j} - U_{0,j+1} - U_{0,j-1} = h^2 f_{0j},$$

得到

$$4U_{0j} - 2U_{1,j} - U_{0,j+1} - U_{0,j-1} = h^2 f_{0j} - 2hg(x_0, y_j).$$

这里 g 改变了右端项。Neumann 条件只给出导数，纯 Neumann Poisson 问题还需要满足源项相容条件，并通过固定一个参考值消除常数零模。

8.1.4 一个 Poisson 方程算例

例题 8.1 二维 Poisson 方程的差分离散 考虑

$$-\Delta u = f(x, y), \quad 0 < x < 1, \quad 0 < y < 1,$$

其中

$$f(x, y) = -2 \cos(\pi y) + \pi^2(1+x)^2 \cos(\pi y).$$

边界条件为

$$u(x, 0) = (1+x)^2, \quad u(x, 1) = -(1+x)^2,$$

$$u(0, y) = \cos(\pi y), \quad u(1, y) = 4 \cos(\pi y).$$

精确解为

$$u(x, y) = (1+x)^2 \cos(\pi y),$$

因为

$$u_{xx} = 2 \cos(\pi y), \quad u_{yy} = -\pi^2(1+x)^2 \cos(\pi y),$$

从而

$$-\Delta u = -u_{xx} - u_{yy} = -2 \cos(\pi y) + \pi^2(1+x)^2 \cos(\pi y).$$

取 $h = 1/4$, 内部点为 $x_i = i/4, y_j = j/4, i, j = 1, 2, 3$. 对第一个内部点 $(x_1, y_1) = (1/4, 1/4)$, 五点格式为

$$-U_{0,1} + 4U_{1,1} - U_{2,1} - U_{1,0} - U_{1,2} = h^2 f(1/4, 1/4).$$

将已知边界值移到右端:

$$4U_{1,1} - U_{2,1} - U_{1,2} = h^2 f(1/4, 1/4) + \cos \frac{\pi}{4} + \left(1 + \frac{1}{4}\right)^2.$$

同一行的中间点满足

$$-U_{1,1} + 4U_{2,1} - U_{3,1} - U_{2,2} = h^2 f(1/2, 1/4) + \left(1 + \frac{1}{2}\right)^2,$$

右侧点满足

$$-U_{2,1} + 4U_{3,1} - U_{3,2} = h^2 f(3/4, 1/4) + 4 \cos \frac{\pi}{4} + \left(1 + \frac{3}{4}\right)^2.$$

其余内部点按同样规则写出, 最终得到 9×9 线性方程组。由于精确解已知, 该例常用于验证误差阶: 当 $h \rightarrow h/2$ 时, 二阶差分的全局误差通常约缩小到原来的 $1/4$ 。

8.1.5 高阶差分算符与对称性

差分公式可由 Lagrange 插值或 Taylor 展开系统构造。给定若干采样点 $x_{i+r} = x_i + r\Delta x$, 要求

$$\sum_r a_r u(x_i + r\Delta x)$$

在 Taylor 展开中保留目标导数项、消去尽可能多的低阶误差项, 即可求得系数 a_r 。常用的一阶导数公式包括三阶偏置格式

$$\left(\frac{\partial u}{\partial x}\right)_i = \frac{2u_{i+1} + 3u_i - 6u_{i-1} + u_{i-2}}{6\Delta x} + O(\Delta x^3),$$

$$\left(\frac{\partial u}{\partial x}\right)_i = \frac{-u_{i+2} + 6u_{i+1} - 3u_i - 2u_{i-1}}{6\Delta x} + O(\Delta x^3),$$

以及四阶中心格式

$$\left(\frac{\partial u}{\partial x}\right)_i = \frac{-u_{i+2} + 8u_{i+1} - 8u_{i-1} + u_{i-2}}{12\Delta x} + O(\Delta x^4).$$

二阶导数的四阶中心格式为

$$\left(\frac{\partial^2 u}{\partial x^2}\right)_i = \frac{-u_{i+2} + 16u_{i+1} - 30u_i + 16u_{i-1} - u_{i-2}}{12(\Delta x)^2} + O(\Delta x^4).$$

这些公式在内部点精度较高，但在边界附近缺少对称采样点，只能改用偏置或较低阶格式。对于二维、三维问题，减小网格间距会使未知数数目按 h^{-d} 增长，内存和计算量迅速上升；提高差分阶数可在同样网格数下提高光滑解的精度，但会扩大矩阵带宽。

量子力学计算中还必须利用对称性和好量子数 (good quantum numbers)。若 Hamiltonian 与某个对称操作对易，可以把 Hilbert 空间分解为不相混合的子空间，只在每个子空间内求解。这既减少矩阵规模，也避免不同量子数态的数值混合。若体系对称性被破坏，计算量往往增加几个量级。

8.2 Fourier 微分算符与 FFT

8.2.1 均匀周期网格上的 Fourier 表示

Fourier 微分法 (Fourier differentiation) 适用于均匀网格和周期边界条件。设一维网格为

$$x_\nu = \left[-\frac{n_x - 1}{2} + \nu \right] \Delta x, \quad \nu = 1, \dots, n_x,$$

周期长度为

$$L = n_x \Delta x, \quad f(x + L) = f(x).$$

离散波数取为

$$k_n = (n - 1)\Delta k, \quad n = 1, \dots, \frac{n_x}{2},$$

$$k_n = (n - n_x - 1)\Delta k, \quad n = \frac{n_x}{2} + 1, \dots, n_x, \quad \Delta k = \frac{2\pi}{n_x \Delta x}.$$

离散 Fourier 变换 (discrete Fourier transform, DFT) 定义为

$$\tilde{f}(k_n) = \sum_{\nu=1}^{n_x} \exp(-ik_n x_\nu) f(x_\nu),$$

其逆变换为

$$f(x_\nu) = \frac{1}{n_x} \sum_{n=1}^{n_x} \exp(ik_n x_\nu) \tilde{f}(k_n).$$

直接 DFT 的计算复杂度是 $O(n_x^2)$ ，快速 Fourier 变换 (fast Fourier transform, FFT) 通过分治结构把复杂度降为 $O(n_x \log n_x)$ 。在多维问题中，二维或三维 FFT 通常按各方向的一维 FFT 依次执行。

8.2.2 Fourier 微分矩阵

平面波是微分算符的本征函数：

$$\frac{d^m}{dx^m} \exp(ik_n x) = (ik_n)^m \exp(ik_n x).$$

因此

$$f^{(m)}(x_\nu) = \frac{1}{n_x} \sum_{n=1}^{n_x} \exp(ik_n x_\nu) (ik_n)^m \tilde{f}(k_n).$$

将 $\tilde{f}$ 再写回实空间值，可得微分矩阵

$$f^{(m)}(x_\nu) = \sum_{\nu'=1}^{n_x} D_{\nu\nu'}^{(m)} f(x_{\nu'}),$$

$$D_{\nu\nu'}^{(m)} = \frac{1}{n_x} \sum_{n=1}^{n_x} \exp[ik_n(x_\nu - x_{\nu'})] (ik_n)^m.$$

一阶和二阶微分矩阵可写成正弦、余弦求和形式：

$$D_{\nu\nu'}^{(1)} = -\frac{2\Delta k}{n_x} \sum_{n=1}^{n_x/2-1} n \sin[k_n \Delta x(\nu - \nu')] - \frac{\Delta k}{n_x} \frac{n_x}{2} \sin\left[(\nu - \nu')\Delta k \frac{n_x}{2}\right],$$

$$D_{\nu\nu'}^{(2)} = -\frac{2(\Delta k)^2}{n_x} \sum_{n=1}^{n_x/2-1} n^2 \cos[k_n \Delta x(\nu - \nu')] - \frac{(\Delta k)^2}{n_x} \left(\frac{n_x}{2}\right)^2 \cos\left[(\nu - \nu')\Delta k \frac{n_x}{2}\right].$$

矩阵形式清楚展示了 Fourier 微分的非局域性：每个点的导数一般依赖所有网格点。实际计算中通常不显式构造 D ，而是执行

$$f(x) \xrightarrow{\text{FFT}} \tilde{f}(k) \xrightarrow{\times(ik)^m} (ik)^m \tilde{f}(k) \xrightarrow{\text{IFFT}} f^{(m)}(x).$$

二维函数 $\phi(x_i, y_j)$ 的偏导数可按方向分别处理：

$$\phi_x(x_i, y_j) = \sum_k D_{ik}^x \phi(x_k, y_j),$$

$$\phi_y(x_i, y_j) = \sum_k D_{jk}^y \phi(x_i, y_k).$$

这说明多维 Fourier 微分本质上是张量积结构： x 方向矩阵只作用在 x 指标上， y 方向矩阵只作用在 y 指标上。

8.2.3 FFT 的物理用途

Fourier 变换不仅用于微分，还用于滤波、图像处理、信号分析、卷积和谱分析。连续二维 Fourier 变换可写为

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) \exp[-j2\pi(ux + vy)] dx dy,$$

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) \exp[j2\pi(ux + vy)] du dv.$$

低频分量描述大尺度平滑结构，高频分量描述边缘、细节和噪声。因此可以通过截断高频实现低通滤波，通过保留高频增强边缘，或在频域中执行卷积：

$$\mathcal{F}[f * g] = \mathcal{F}[f]\mathcal{F}[g].$$

在时域信号中，阻尼振荡

$$A(t) = A \exp\left[i\left(\omega + i\frac{\Gamma}{2}\right)t\right] = A \exp\left(-\frac{\Gamma t}{2}\right) \exp(i\omega t)$$

对应频域中的有限宽度峰。宽度 Γ 与寿命成反比，是共振态和谱函数分析中的重要物理量。

8.3 基矢展开法

8.3.1 从微分方程到矩阵本征值问题

定义 8.2 (基矢展开法)

基矢展开法 (basis expansion method) 用一组已知基函数 $\{\Phi_a\}$ 展开未知波函数或场函数，把微分算符问题转化为有限维矩阵问题。若基函数与问题的对称性和边界行为相匹配，少量基函数即可给出较高精度；若基函数形状与真实解差别很大，收敛会变慢。

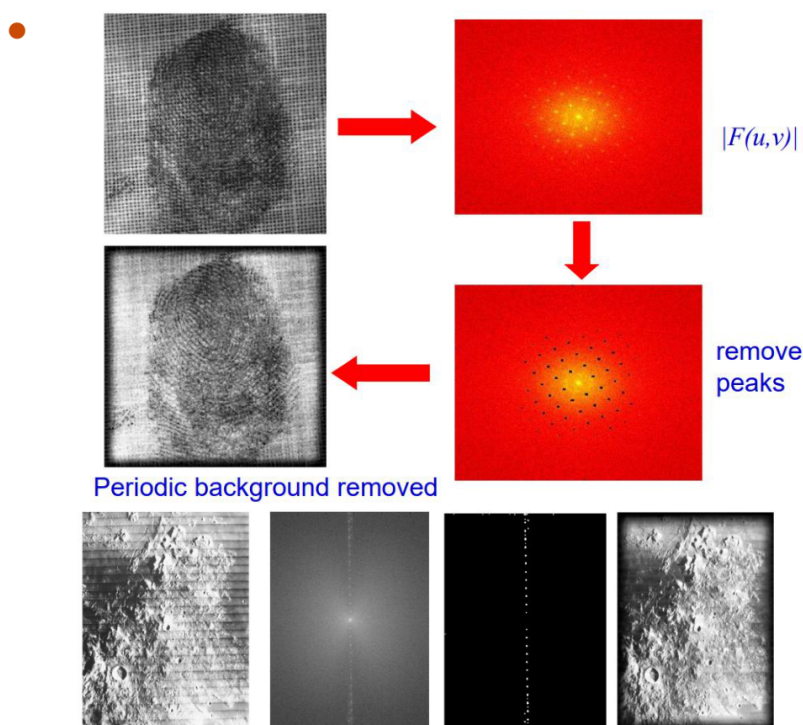


图 8.2: 二维 Fourier 变换在图像处理中的典型流程: 空间图像经 FFT 进入频域, 频域滤波后再经逆变换回到空间图像。

设量子态满足

$$H|\Psi\rangle = E|\Psi\rangle,$$

用一组正交归一基矢展开

$$|\Psi\rangle = \sum_a c_a |\Phi_a\rangle.$$

代入本征方程:

$$\sum_a c_a H|\Phi_a\rangle = E \sum_a c_a |\Phi_a\rangle.$$

左乘 $\langle\Phi_b|$ 得

$$\sum_a \langle\Phi_b|H|\Phi_a\rangle c_a = E c_b.$$

记

$$H_{ba} = \langle\Phi_b|H|\Phi_a\rangle,$$

就得到矩阵本征值问题

$$H\mathbf{c} = E\mathbf{c}.$$

在一般记号中, 若

$$\psi_i = \sum_j c_{ij} \varphi_j,$$

则

$$\sum_m H_{\varphi_m} c_{im} = E \sum_j c_{ij} \varphi_j.$$

投影到 φ_k 后

$$\sum_m H_{km} c_{im} = E c_{ik}, \quad H_{km} = \langle\varphi_k|H|\varphi_m\rangle.$$

对单粒子 Hamiltonian h_q ，常写作

$$h_{\alpha\beta}^{(q)} = \langle \Phi_\alpha | h_q | \Phi_\beta \rangle.$$

8.3.2 变形谐振子基

核结构计算中常用变形谐振子基 (deformed harmonic oscillator basis) 处理轴对称平均场。柱坐标下的基函数可分离为径向、轴向、方位角和自旋部分：

$$\Phi_\alpha(\mathbf{r}, \sigma) \sim R_{n_r}^{|m_l|}(\rho) Z_{n_z}(z) \frac{\exp(im_l\varphi)}{\sqrt{2\pi}} \chi_{m_s}(\sigma),$$

其中径向部分由 Laguerre 多项式表示，轴向部分由 Hermite 多项式表示。相应谐振子势可写为

$$V_{\text{HO}} = \frac{m}{2} (\omega_\perp^2 \rho^2 + \omega_z^2 z^2),$$

能量为

$$\epsilon_\alpha = \hbar\omega_\perp(2n_r + |m_l| + 1) + \hbar\omega_z\left(n_z + \frac{1}{2}\right).$$

常用截断量子数为

$$N = 2n_r + n_z + |m_l|.$$

若体系具有轴对称性，总角动量在对称轴上的投影 Ω 、宇称 π 等量子数可用于分块对角化。

三维 Cartesian 谐振子基则写作

$$\psi(\mathbf{r}, \sigma) = \sum_{n_x, n_y, n_z, s} A^{n_x n_y n_z s} \varphi_{n_x}(x) \varphi_{n_y}(y) \varphi_{n_z}(z) \delta_{s\sigma},$$

截断量子数为

$$N = n_x + n_y + n_z.$$

这种方法的优点是导数和矩阵元常可解析计算，边界行为由基函数自动给出；缺点是有限谐振子基对弱束缚态、连续谱、长尾密度和复杂形变的描述效率较低。若截断空间太小，会出现基截断误差；若形变或连续谱很重要，则需要更灵活的网格、DVR、B-样条或自适应基。

8.4 离散变量表示 DVR

8.4.1 DVR 的基本思想

定义 8.3 (离散变量表示)

离散变量表示 (discrete variable representation, DVR) 是一类兼具基函数展开和网格表示优点的方法。它从一组正交基或投影算符出发，构造与特定求积点对应的局域基函数，使局域势能矩阵近似对角，而动能或微分算符保留为矩阵。

设有投影算符

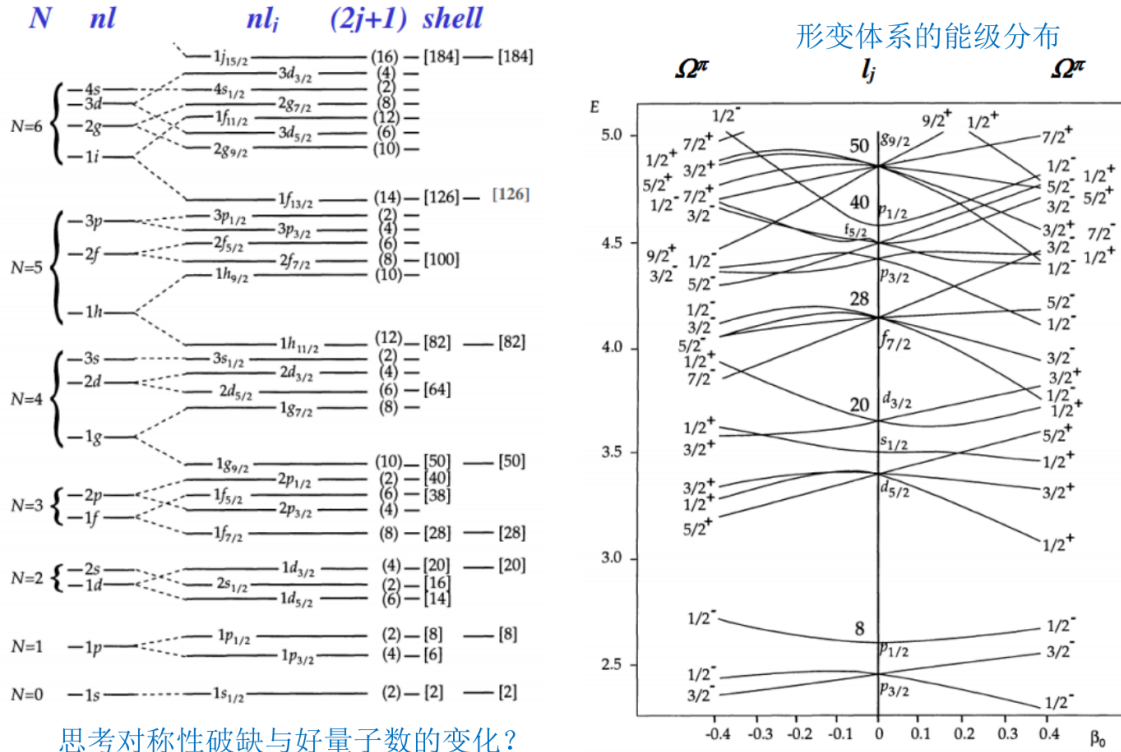
$$P = \sum_n |\phi_n\rangle \langle \phi_n|.$$

对坐标本征态 $|x_\alpha\rangle$ 投影，定义 DVR 基

$$|\Delta_\alpha\rangle = P|x_\alpha\rangle = \sum_n |\phi_n\rangle \phi_n^*(x_\alpha).$$

其重叠为

$$\langle \Delta_\alpha | \Delta_\beta \rangle = \sum_n \phi_n(x_\alpha) \phi_n^*(x_\beta) = N_\alpha \delta_{\alpha\beta}.$$



思考对称性破缺与好量子数的变化?
图 8.3: 球形壳模型能级与变形体系能级分布示意。对称性改变时, 好量子数和能级简并结构也随之改变。

选择合适格点 x_α 和权重 $w_\alpha = N_\alpha^{-1}$, 归一化的基函数满足

$$w_\beta \Delta_\alpha(x_\beta) = \delta_{\alpha\beta}.$$

于是局域势能矩阵近似为

$$\langle \phi_i | V | \phi_j \rangle \approx \sum_{k=1}^n w_k \phi_i(x_k) V(x_k) \phi_j(x_k) = V(x_i) \delta_{ij}.$$

这正是 DVR 的计算优势: 势能只需在格点上取值, 不必显式计算多维积分。

8.4.2 由正交多项式构造 DVR

常见构造从带权正交多项式出发。令

$$\phi_n(x) = W^{1/2}(x) P_n(x),$$

其中 $P_n(x)$ 是 n 阶多项式, $W(x)$ 是 Gauss 积分权重函数, 并满足

$$\langle \phi_n | \phi_m \rangle = \int W(x) P_n(x) P_m(x) dx = \delta_{nm}.$$

取 N 阶多项式的零点

$$P_N(x_\alpha) = 0, \quad \alpha = 1, \dots, N$$

作为 Gauss 积分点。DVR 函数可写为

$$\Delta_\alpha(x_\beta) = \sum_{n=0}^{N-1} \sqrt{W(x_\alpha) W(x_\beta)} P_n(x_\alpha) P_n(x_\beta) \propto \delta_{\alpha\beta},$$

归一化后

$$F_\alpha(x) = \sqrt{w_\alpha} \sum_{n=0}^{N-1} \sqrt{W(x) W(x_\alpha)} P_n(x) P_n(x_\alpha).$$

不同正交多项式给出不同 DVR，例如 Legendre DVR、Hermite DVR、Laguerre DVR；Fourier 平面波也可构造相应 DVR。

8.4.3 sinc DVR

sinc DVR 从动量截断投影开始：

$$P = \int_{-p_{\max}}^{p_{\max}} dp |p\rangle\langle p|.$$

取均匀坐标点

$$x_\alpha = \frac{\pi}{p_{\max}} \alpha, \quad \alpha \in \mathbb{Z},$$

定义

$$|\Delta_\alpha\rangle = \int_{-p_{\max}}^{p_{\max}} dp |p\rangle \frac{\exp(-ipx_\alpha)}{\sqrt{2\pi}}.$$

其正交关系为

$$\langle \Delta_\alpha | \Delta_\beta \rangle = \delta_{\alpha\beta} \frac{p_{\max}}{\pi}.$$

归一化 sinc 基函数为

$$F_\alpha(x) = \sqrt{\frac{\pi}{p_{\max}}} \Delta_\alpha(x) = \frac{1}{\sqrt{\pi p_{\max}}} \frac{\sin[p_{\max}(x - x_\alpha)]}{x - x_\alpha}.$$

任意投影空间中的函数可表示为

$$|\psi\rangle = \sum_\alpha |F_\alpha\rangle \sqrt{\frac{\pi}{p_{\max}}} \psi(x_\alpha).$$

以一维谐振子为例，

$$\left[-\frac{\hbar^2}{2m} \frac{d^2}{dx^2} + \frac{k_c^2 x^2}{2R^2} \right] \phi(x) = E \phi(x).$$

sinc DVR 中势能近似对角，

$$\langle f_k | V | f_l \rangle \approx V(x_k) \delta_{kl},$$

而动能是矩阵：

$$K_{kl} = \begin{cases} \frac{\hbar^2 \pi^2}{m N^2 a^2} \frac{(-1)^{k-l}}{\sin^2[\pi(k-l)/N]}, & k \neq l, \\ \frac{\hbar^2 \pi^2}{6 m a^2} \left(1 + \frac{2}{N^2} \right), & k = l. \end{cases}$$

Hamiltonian 矩阵为

$$H_{kl} = K_{kl} + \frac{m \omega^2 a^2 k^2}{2} \delta_{kl}.$$

DVR 方法精度高、势能处理简单，适合局域势和规则区域；其不足是动能矩阵一般较密，且高维问题仍会受到基函数数目指数增长的限制。

8.5 B-样条展开法

8.5.1 B-样条基函数

B-样条展开法 (B-spline expansion method) 用分段多项式基函数展开未知函数。给定节点序列 (knots)

$$t_1 \leq t_2 \leq \cdots \leq t_{N+M},$$

零阶 B-样条定义为

$$b_{j,0}(t) = \begin{cases} 1, & t_j < t < t_{j+1}, \\ 0, & \text{otherwise.} \end{cases}$$

高阶样条递推为

$$b_{j,n}(t) = \frac{t - t_j}{t_{j+n} - t_j} b_{j,n-1}(t) + \frac{t_{j+n+1} - t}{t_{j+n+1} - t_{j+1}} b_{j+1,n-1}(t).$$

若样条阶数为 M ，基函数数目常写为

$$N = N + M - 1.$$

B-样条具有紧支撑、分片多项式、可解析微分和可使用非均匀格点等优点。节点可在边界附近加密，以提高边界条件和快速变化区域的描述能力。

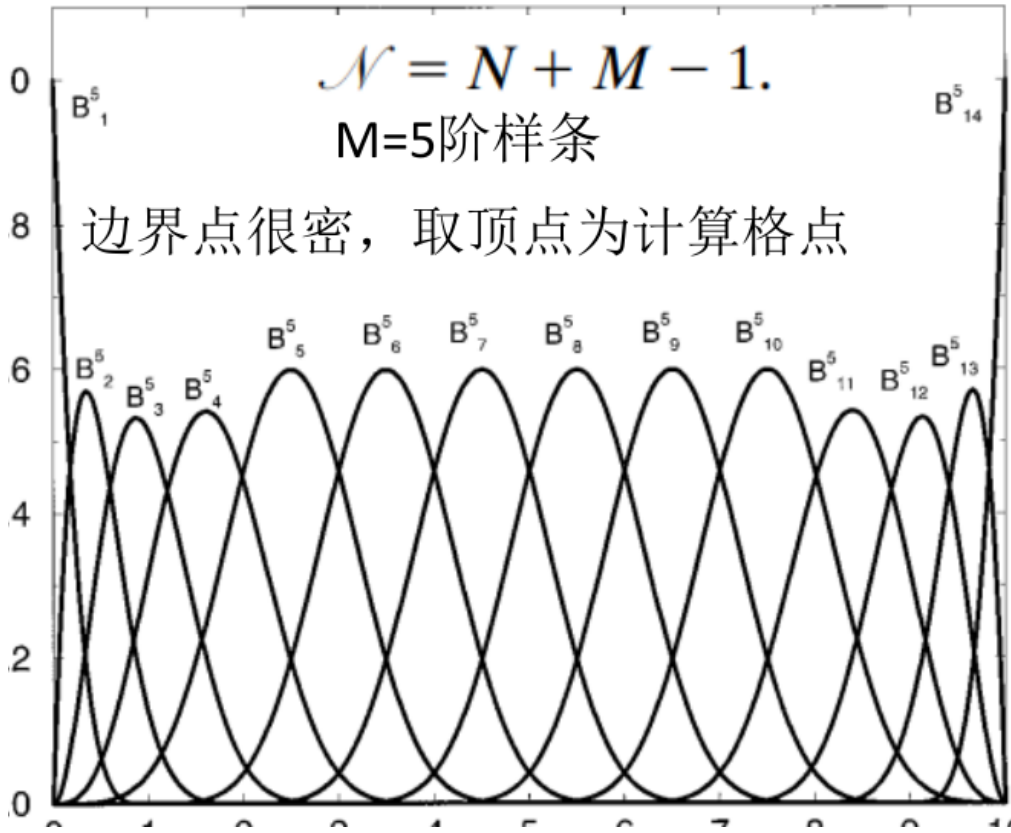


图 8.4: $M = 5$ 阶 B-样条基函数示意。每个基函数只在有限区间内非零，因此离散矩阵保持带状或稀疏结构。

B-样条满足分割统一性

$$\sum_i B_i^M(x) = 1,$$

并有积分性质

$$\int_{x_{\min}}^{x_{\max}} B_i^M(x) dx = \frac{x_{i+M} - x_i}{M}.$$

函数展开为

$$f(x) = \sum_{i=1}^N B_i^M(x) c^i.$$

在配点 x_α 上取值，

$$f_\alpha = f(x_\alpha), \quad B_{\alpha i} = B_i^M(x_\alpha),$$

得到矩阵关系

$$\mathbf{f} = B\mathbf{c}.$$

若取内部配点数为 $n = N - 2$ ，边界条件将补足决定系数所需的方程。

8.5.2 边界条件与微分算符矩阵

一般线性齐次边界条件可写为

$$\sum_{p \geq 0} K_p \frac{d^p f}{dx^p} \Big|_{\text{boundary}} = 0.$$

代入 B-样条展开：

$$\sum_i \left[\sum_{p \geq 0} K_p \frac{d^p B_i^M}{dx^p} \Big|_{\text{boundary}} \right] c^i = 0.$$

记边界行向量为

$$\beta_i = \sum_{p \geq 0} K_p \frac{d^p B_i^M}{dx^p} \Big|_{\text{boundary}},$$

则可把内部取值方程和左右边界条件合并：

$$\tilde{B} = \begin{pmatrix} B \\ \beta^L \\ \beta^R \end{pmatrix}, \quad \mathbf{c} = \tilde{B}^{-1} \begin{pmatrix} \mathbf{f} \\ 0 \\ 0 \end{pmatrix}.$$

对任意微分算符 O ，

$$(Of)_\alpha = \sum_i (OB_i^M)(x_\alpha) c^i = \sum_\beta O_\alpha^\beta f_\beta,$$

其中

$$O_\alpha^\beta = \sum_i (OB_i^M)(x_\alpha) \tilde{C}^{i\beta}, \quad \tilde{C} = \tilde{B}^{-1}.$$

这给出了 B-样条微分矩阵。由于 B-样条可解析微分，导数矩阵的构造较稳定；由于基函数紧支撑，算符矩阵通常比全局谱方法更稀疏。

8.5.3 轴对称方程中的 B-样条/DVR

轴对称问题中常出现

$$-\frac{\hbar^2}{2m} \left(\frac{\partial^2 f}{\partial r^2} + \frac{1}{r} \frac{\partial f}{\partial r} - \frac{\mu^2}{r^2} f + \frac{\partial^2 f}{\partial z^2} \right) + V(r, z) f = E f.$$

用 r 与 z 方向的一维微分矩阵离散，定义

$$R_{n_r \times n_r} = -\frac{\hbar^2}{2m} \left[D_r^2 + \text{diag} \left(\frac{1}{r_\alpha} \right) D_r - \text{diag} \left(\frac{\mu^2}{r_\alpha^2} \right) \right],$$

$$Z_{n_z \times n_z} = -\frac{\hbar^2}{2m} D_z^2.$$

二维方程离散为

$$\sum_{\alpha'} R_{\alpha\alpha'} f_{\alpha'\beta} + \sum_{\beta'} Z_{\beta\beta'} f_{\alpha\beta'} + V_{\alpha\beta} f_{\alpha\beta} = E f_{\alpha\beta}.$$

将二维指标压缩为

$$k = (\beta - 1)n_r + \alpha, \quad k' = (\beta' - 1)n_r + \alpha',$$

即可写成矩阵本征值问题

$$H\mathbf{f} = E\mathbf{f},$$

矩阵维数为 $n_r n_z \times n_r n_z$ 。离散间距 Δ 与可表示最大动量之间满足 Nyquist 判据

$$|p|_{\max} \leq \frac{\hbar\pi}{\Delta}.$$

若希望描述短波长振荡或高动量成分，必须减小格距或提高基函数分辨率。

8.5.4 自旋轨道耦合与轴对称边界

考虑带有轴对称形变谐振子势和自旋轨道耦合的单粒子 Hamiltonian:

$$\hat{H} = -\frac{\hbar^2}{2m} \left(\frac{\partial^2}{\partial r^2} + \frac{1}{r} \frac{\partial}{\partial r} + \frac{1}{r^2} \frac{\partial^2}{\partial \varphi^2} + \frac{\partial^2}{\partial z^2} \right) - V_0 + \frac{m}{2} (\omega_r^2 r^2 + \omega_z^2 z^2) - 2\hbar\omega_0 \kappa \mathbf{l} \cdot \mathbf{s}.$$

其中柱坐标为 (r, φ, z) ，体系关于 z 轴具有轴对称性。若暂不考虑自旋轨道耦合，对应的轴对称谐振子能级可写为

$$E = \hbar\omega_z \left(\nu_z + \frac{1}{2} \right) + \hbar\omega_r (2\nu_r + |\mu| + 1) - V_0,$$

其中 ν_z 是 z 方向振动量子数， ν_r 是径向量子数， μ 是轨道角动量在 z 方向的投影量子数。

形变频率常参数化为

$$\omega_r^2 = \omega_0^2 \left(1 + \frac{2}{3} \delta \right), \quad \omega_z^2 = \omega_0^2 \left(1 - \frac{4}{3} \delta \right),$$

其中

$$\hbar\omega_0 = \frac{41}{A^{1/3}} \text{ MeV}.$$

因此势能项可等价写为

$$V(r, z) = \frac{m\omega_0^2}{2} \left[r^2 \left(1 + \frac{2}{3} \delta \right) + z^2 \left(1 - \frac{4}{3} \delta \right) - \frac{2V_0}{m\omega_0^2} \right].$$

轴对称波函数的角向分解 由于 Hamiltonian 具有轴对称性，总角动量在对称轴方向的投影

$$\hat{j}_z = \hat{l}_z + \hat{s}_z$$

与 Hamiltonian 对易，因此 Ω 是好量子数。对自旋 1/2 粒子，波函数写成二分量自旋量：

$$\psi_\Omega(r, \varphi, z) = \frac{1}{\sqrt{2\pi}} \begin{pmatrix} \exp \left[i \left(\Omega - \frac{1}{2} \right) \varphi \right] \mathcal{U}(r, z) \\ \exp \left[i \left(\Omega + \frac{1}{2} \right) \varphi \right] \mathcal{L}(r, z) \end{pmatrix},$$

其中

$$\Omega = \pm \frac{1}{2}, \pm \frac{3}{2}, \pm \frac{5}{2}, \dots$$

这个角向因子的选择来自 $\hat{j}_z$ 的本征值条件。上分量对应自旋向上， $s_z = +\frac{1}{2}$ ，因此其轨道角动量投影为

$$\mu_U = \Omega - \frac{1}{2}.$$

下分量对应自旋向下， $s_z = -\frac{1}{2}$ ，因此其轨道角动量投影为

$$\mu_L = \Omega + \frac{1}{2}.$$

于是两分量具有相同的总角动量投影：

$$\mu_U + \frac{1}{2} = \Omega, \quad \mu_L - \frac{1}{2} = \Omega.$$

代入角向分解以后，三维问题化为关于 (r, z) 的二维二分量本征值问题：

$$(\hat{T} + \hat{V} + \hat{H}_{\text{SO}}) \begin{pmatrix} \mathcal{U}(r, z) \\ \mathcal{L}(r, z) \end{pmatrix} = E \begin{pmatrix} \mathcal{U}(r, z) \\ \mathcal{L}(r, z) \end{pmatrix}.$$

动能算符 角向部分已经由指数因子显式分离，因此

$$\frac{\partial^2}{\partial \varphi^2} \exp(i\mu\varphi) = -\mu^2 \exp(i\mu\varphi).$$

于是动能算符在二分量空间中变为对角矩阵：

$$\hat{T} = -\frac{\hbar^2}{2m} \begin{pmatrix} \frac{\partial^2}{\partial r^2} + \frac{1}{r} \frac{\partial}{\partial r} - \frac{\left(\Omega - \frac{1}{2}\right)^2}{r^2} + \frac{\partial^2}{\partial z^2} & 0 \\ 0 & \frac{\partial^2}{\partial r^2} + \frac{1}{r} \frac{\partial}{\partial r} - \frac{\left(\Omega + \frac{1}{2}\right)^2}{r^2} + \frac{\partial^2}{\partial z^2} \end{pmatrix}.$$

其中上分量的离心势项由 $\mu_U = \Omega - \frac{1}{2}$ 决定，下分量的离心势项由 $\mu_L = \Omega + \frac{1}{2}$ 决定。靠近 $r = 0$ 时，离心项具有奇异形式，因此必须配合正确的轴上边界条件。

形变谐振子势能算符 势能项不含自旋算符，因此在二分量空间中为对角矩阵：

$$\hat{V} = \frac{m\omega_0^2}{2} \begin{pmatrix} r^2 \left(1 + \frac{2}{3}\delta\right) + z^2 \left(1 - \frac{4}{3}\delta\right) - \frac{2V_0}{m\omega_0^2} & 0 \\ 0 & r^2 \left(1 + \frac{2}{3}\delta\right) + z^2 \left(1 - \frac{4}{3}\delta\right) - \frac{2V_0}{m\omega_0^2} \end{pmatrix}.$$

若 $\delta > 0$ ，则 ω_z^2 减小、 ω_r^2 增大，说明 z 方向束缚相对较弱，体系倾向于沿 z 方向拉长。

柱坐标下的自旋矩阵 在直角坐标中，Pauli 矩阵为

$$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

柱坐标单位矢量为

$$\mathbf{e}_r = (\cos \varphi, \sin \varphi, 0), \quad \mathbf{e}_\varphi = (-\sin \varphi, \cos \varphi, 0), \quad \mathbf{e}_z = (0, 0, 1).$$

因此

$$\begin{aligned} \sigma_r &= \boldsymbol{\sigma} \cdot \mathbf{e}_r = \sigma_x \cos \varphi + \sigma_y \sin \varphi = \begin{pmatrix} 0 & e^{-i\varphi} \\ e^{i\varphi} & 0 \end{pmatrix}, \\ \sigma_\varphi &= \boldsymbol{\sigma} \cdot \mathbf{e}_\varphi = -\sigma_x \sin \varphi + \sigma_y \cos \varphi = \begin{pmatrix} 0 & -ie^{-i\varphi} \\ ie^{i\varphi} & 0 \end{pmatrix}, \\ \sigma_z &= \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \end{aligned}$$

这些角向因子会与波函数中

$$\exp \left[i \left(\Omega \mp \frac{1}{2} \right) \varphi \right]$$

相互抵消，从而保证自旋轨道耦合后仍然不混合不同的 Ω 子空间。

自旋轨道耦合项 自旋轨道耦合项为

$$\hat{H}_{\text{SO}} = -2\hbar\omega_0\kappa \mathbf{l} \cdot \mathbf{s}.$$

由于

$$\mathbf{s} = \frac{\hbar}{2} \boldsymbol{\sigma},$$

该项会同时作用于轨道自由度和自旋自由度。经过角向分离后，它在 $(\mathcal{U}, \mathcal{L})^T$ 空间中可以写为

$$\hat{H}_{\text{SO}} = -\hbar\omega_0\kappa \begin{pmatrix} \Omega - \frac{1}{2} & -\left(\Omega + \frac{1}{2}\right) \frac{z}{r} - z \frac{\partial}{\partial r} + r \frac{\partial}{\partial z} \\ -\left(\Omega - \frac{1}{2}\right) \frac{z}{r} + z \frac{\partial}{\partial r} - r \frac{\partial}{\partial z} & -\left(\Omega + \frac{1}{2}\right) \end{pmatrix}.$$

其中对角项来自 $\hat{l}_z \hat{s}_z$ ，非对角项来自 $\hat{l}_+ \hat{s}_-$ 与 $\hat{l}_- \hat{s}_+$ 。非对角项会把上、下自旋分量相互耦合，因此 $\mathcal{U}$ 和 $\mathcal{L}$ 一般不能分别独立求解。

轴上边界条件 在 $r = 0$ 处，柱坐标存在坐标奇点。为了使完整波函数在直角坐标中单值、有限且光滑，角量子数为 μ 的分量在轴附近应具有正则行为

$$R_\mu(r) \sim r^{|\mu|}.$$

对于上分量，

$$\mu_U = \Omega - \frac{1}{2},$$

对于下分量，

$$\mu_L = \Omega + \frac{1}{2}.$$

因为二者相差 1，所以它们在 $r = 0$ 处的奇偶型边界条件相反。实际计算中常采用如下最低阶正则边界条件：

condition	upper component	lower component
$\Omega - \frac{1}{2}$ even	$\left. \frac{\partial \mathcal{U}}{\partial r} \right _{r=0} = 0$	$\mathcal{L} _{r=0} = 0$
$\Omega - \frac{1}{2}$ odd	$\mathcal{U} _{r=0} = 0$	$\left. \frac{\partial \mathcal{L}}{\partial r} \right _{r=0} = 0$

其含义是：若某一分量关于 $r = 0$ 可作偶延拓，则取 Neumann 型边界条件；若可作奇延拓，则取 Dirichlet 型边界条件。由于 μ_U 与 μ_L 的奇偶性相反，上、下分量在轴上的边界条件也相反。

B-样条展开与矩阵本征值问题 数值计算时，可在二维 (r, z) 区域上分别引入 r 方向和 z 方向的 B-样条基函数：

$$B_i^{(r)}(r), \quad B_j^{(z)}(z).$$

将两个自旋分量展开为

$$\mathcal{U}(r, z) = \sum_{i,j} C_{ij}^{(U)} B_i^{(r)}(r) B_j^{(z)}(z),$$

$$\mathcal{L}(r, z) = \sum_{i,j} C_{ij}^{(L)} B_i^{(r)}(r) B_j^{(z)}(z).$$

由于 B-样条基函数具有局域支撑，Hamiltonian 矩阵通常是带状稀疏矩阵。若使用 Galerkin 投影，则得到广义本征值问题

$$\begin{pmatrix} H_{UU} & H_{UL} \\ H_{LU} & H_{LL} \end{pmatrix} \begin{pmatrix} \mathbf{C}^{(U)} \\ \mathbf{C}^{(L)} \end{pmatrix} = E \begin{pmatrix} S & 0 \\ 0 & S \end{pmatrix} \begin{pmatrix} \mathbf{C}^{(U)} \\ \mathbf{C}^{(L)} \end{pmatrix}.$$

这里 S 是 B-样条基函数的重叠矩阵。由于柱坐标体元为

$$d^3\mathbf{r} = r dr d\varphi dz,$$

角向积分给出归一化因子后，径向与轴向内积应按

$$\langle F, G \rangle = \int_0^\infty r dr \int_{-\infty}^\infty dz F^*(r, z) G(r, z)$$

计算。因此归一化条件为

$$\int_0^\infty r dr \int_{-\infty}^\infty dz \left(|\mathcal{U}(r, z)|^2 + |\mathcal{L}(r, z)|^2 \right) = 1.$$

B-样条阶数 M 越高, 单个基函数的多项式阶数越高, 通常可以提高对波函数及其导数的逼近精度; 但同时单个基函数支撑范围变宽, 矩阵带宽增大, 计算量和存储量也会上升。因此实际计算需要在精度与计算代价之间折中。

谐振子基与坐标空间方法的精度差异 在束缚态尾部或弱束缚体系中, 普通谐振子基函数具有 Gaussian 型渐近行为, 尾部衰减往往过快, 因此可能不能很好描述密度在大距离处的指数型尾部。通过增大基函数数目或提高 B-样条阶数 M , 可以逐渐改善数值精度。

更适合描述长尾行为的做法包括:

- 使用坐标空间 B-样条或有限差分方法;
- 使用 transformed harmonic oscillator, THO, 基;
- 使用更大的计算盒与更高阶基函数;
- 在弱束缚或滴线核问题中注意波函数远距离渐近行为。

因此, 在含自旋轨道耦合的轴对称问题中, 数值求解的关键包括: 正确分离角向因子、在 $r = 0$ 施加正则边界条件、准确处理 $1/r$ 与 $1/r^2$ 奇异项, 并选择合适阶数与网格范围的 B-样条基函数。

8.6 小波与多小波分析

8.6.1 分层局域基

小波分析 (wavelet analysis) 把区域按层级逐步细分, 并用具有局域支撑的正交基描述函数。第 n 层把区间分成 2^n 个小区间。以 Legendre 多项式为例, 基本尺度函数可写作

$$\phi_i(x) = \sqrt{2i+1} P_i(2x-1), \quad x \in [0, 1],$$

其中 P_i 是 i 阶 Legendre 多项式。平移和伸缩得到

$$\begin{aligned} \phi_{il}^n(x) &= 2^{n/2} \phi_i(2^n x - l), \quad x \in 2^{-n}[l, l+1], \\ i &= 0, \dots, k-1, \quad l = 0, \dots, 2^n - 1. \end{aligned}$$

相应空间记为

$$V_n = \{\phi_{il}^n(x)\}, \quad V_0 \subset V_1 \subset V_2 \subset \dots$$

函数在第 n 层的尺度函数表示为

$$f^n(x) = \sum_{l=0}^{2^n-1} \sum_{i=0}^{k-1} s_{il}^n \phi_{il}^n(x).$$

8.6.2 多小波分解与过滤

多小波 (multi-wavelet) 引入补空间

$$W_{n-1} = V_n - V_{n-1},$$

并选取正交基 $\psi_{il}^{n-1}(x)$ 张成 W_{n-1} :

$$\psi_{il}^n(x) = 2^{n/2} \psi_i(2^n x - l).$$

于是

$$V_n = V_0 \oplus W_0 \oplus W_1 \oplus \dots \oplus W_{n-1}.$$

函数也可写成压缩形式

$$f^n(x) = \sum_{i=0}^{k-1} s_{i0}^0 \phi_{i0}^0(x) + \sum_{n'=0}^{n-1} \sum_{l=0}^{2^{n'}-1} \sum_{i=0}^{k-1} d_{il}^{n'} \psi_{il}^{n'}(x).$$

其中 $d_{il}^{n'}$ 描述相邻层级之间的差别；若某些 d 很小，说明该区域从粗网格到细网格几乎没有新信息，可截断这些系数。这样自然形成非均匀自适应网格：波函数振荡强、结构复杂的区域自动加密，平滑区域保持稀疏。

尺度函数和小波函数之间存在过滤关系

$$\begin{aligned}\phi_i(x) &= \sqrt{2} \sum_{j=0}^{k-1} \left[h_{ij}^{(0)} \phi_j(2x) + h_{ij}^{(1)} \phi_j(2x-1) \right], \\ \psi_i(x) &= \sqrt{2} \sum_{j=0}^{k-1} \left[g_{ij}^{(0)} \phi_j(2x) + g_{ij}^{(1)} \phi_j(2x-1) \right].\end{aligned}$$

对展开系数也有矩阵过滤形式

$$\begin{pmatrix} s_{il}^{n-1} \\ d_{il}^{n-1} \end{pmatrix} = \begin{pmatrix} H^{(0)} & H^{(1)} \\ G^{(0)} & G^{(1)} \end{pmatrix} \begin{pmatrix} s_{i,2l}^n \\ s_{i,2l+1}^n \end{pmatrix}.$$

这与图像压缩中的多分辨率思想相同：保留主要尺度信息和显著细节，舍弃低于阈值的细节系数。

8.6.3 微分方程到积分方程

多小波框架中，常把微分方程转化为积分方程以利用 Green 函数的平滑性质。考虑

$$(-\nabla^2 + V)\Psi = E\Psi.$$

移项得

$$(-\nabla^2 - E)\Psi = -V\Psi,$$

因此

$$\Psi = -(-\nabla^2 - E)^{-1} V\Psi = -G * (V\Psi).$$

在三维中，当 $k^2 = -E$ 时，

$$(G * f)(\mathbf{r}) = \int d^3s \frac{\exp[-k|\mathbf{r} - \mathbf{s}|]}{4\pi|\mathbf{r} - \mathbf{s}|} f(\mathbf{s}).$$

核函数可用高斯和高阶展开近似，例如

$$\frac{\exp(-kr)}{r} \approx \sum_l \sigma_l \exp(-\tau_l r^2) + O(\epsilon).$$

这样卷积可在多分辨率网格上高效执行，并通过迭代求解自洽方程。MADNESS 等程序框架把多小波数学库、并行运行时和全局矩阵库结合起来，面向化学、物理和核物理中的三维自适应计算。

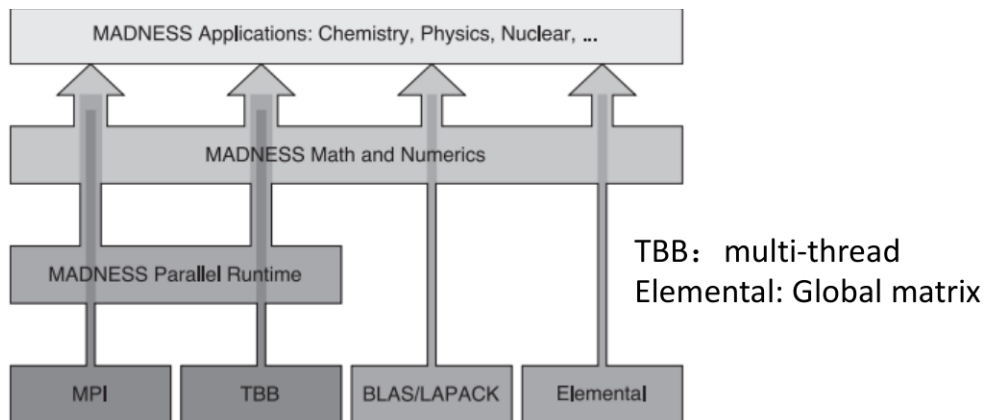


图 8.5: MADNESS 的软件层次示意：应用层建立在多小波数学与数值库之上，并通过 MPI、TBB、BLAS/LAPACK 和 Elemental 支持并行计算。

8.7 有限元方法

8.7.1 有限元思想与 Delaunay 三角剖分

定义 8.4 (有限元方法)

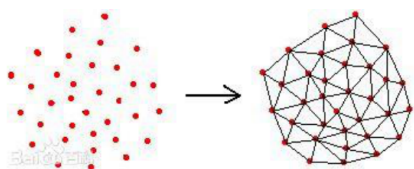
有限元方法 (finite element method, FEM) 把计算区域分割成许多简单单元, 在每个单元内用低阶或高阶多项式插值未知场, 并通过弱形式或变分原理组装全局线性方程组。它特别适合不规则边界和复杂几何。

许多工程和物理现象可由 governing equations 和 boundary conditions 描述。有限差分在规则边界上非常方便, 但处理复杂边界时通常需要特殊技巧; 有限元方法则先把区域分成小的简单单元, 再在每个单元上插值场量。二维中常用三角形单元, 三维中常用四面体单元, 这类单元统称为单纯形 (simplices)。

Delaunay triangulation 是常用三角剖分方法, 其主要性质包括:

- 最大化三角形最小角, 倾向于避免狭长三角形;
- 所有单纯形的并集等于给定点集的凸包;
- 任意点 p 的最近邻 b 通常通过剖分中的边 bp 与其相连。

Delaunay triangulation



- Delaunay triangulations maximize the minimum angle of all the angles of the triangles in the triangulation; they tend to avoid sliver triangles.
- The union of all simplices in the triangulation is the convex hull of the points.
- The closest neighbor b to any point p is on an edge bp in the Delaunay triangulation

图 8.6: Delaunay 三角剖分示意。点集被划分为质量较好的三角形单元, 便于在复杂边界上构造有限元插值。

8.7.2 Poisson 方程的弱形式

考虑 Dirichlet 边界条件下的 Poisson 方程

$$\nabla^2 \phi(\mathbf{r}) = f(\mathbf{r}).$$

有限元法常从泛函

$$J[\phi] = \int_D \{ [\nabla \phi(\mathbf{r})]^2 + f(\mathbf{r})\phi(\mathbf{r}) \} d^d r$$

出发。真实解使相应泛函在满足边界条件的函数空间中取驻值。弱形式的意义在于: 导数阶数通过分部积分降低, 对不光滑网格和复杂边界更稳定。

在三角形单元内, 可用线性插值

$$\phi(x, y) = a_i + b_i x + c_i y$$

或二次插值

$$\phi(x, y) = a_i + b_i x + c_i y + d_i xy + e_i x^2 + f_i y^2.$$

相邻三角形共享同一个顶点时, 该顶点处的场值必须一致, 以保证全局函数连续。

8.7.3 自然坐标与线性三角元

对三角形顶点 a, b, c , 自然坐标 (natural coordinates) 满足

$$\xi_a + \xi_b + \xi_c = 1, \quad \xi_i = \frac{A_i}{A}, \quad i = a, b, c,$$

其中 A_i 是点到对边形成的子三角形面积, A 是总面积。Cartesian 坐标与自然坐标关系为

$$\begin{pmatrix} 1 \\ x \\ y \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 \\ x_a & x_b & x_c \\ y_a & y_b & y_c \end{pmatrix} \begin{pmatrix} \xi_a \\ \xi_b \\ \xi_c \end{pmatrix}.$$

反变换为

$$\begin{pmatrix} \xi_a \\ \xi_b \\ \xi_c \end{pmatrix} = \frac{1}{2A} \begin{pmatrix} x_b y_c - x_c y_b & y_{bc} & x_{cb} \\ x_c y_a - x_a y_c & y_{ca} & x_{ac} \\ x_a y_b - x_b y_a & y_{ab} & x_{ba} \end{pmatrix} \begin{pmatrix} 1 \\ x \\ y \end{pmatrix},$$

其中

$$2A = \det(A) = x_{ba}y_{ca} - x_{ca}y_{ba}, \quad x_{ab} = x_b - x_a, \quad y_{ab} = y_b - y_a.$$

线性三角元的插值函数为

$$\tilde{\phi}(\mathbf{r}) = \phi_a \xi_a + \phi_b \xi_b + \phi_c \xi_c.$$

单元积分可利用

$$\int_A \xi_a^k \xi_b^l \xi_c^m d\Omega = 2A \frac{k! l! m!}{(2 + k + l + m)!}.$$

将所有顶点值组成向量 $\boldsymbol{\phi}$, 泛函离散后可写成

$$J[\boldsymbol{\phi}] = -\boldsymbol{\phi}^T K \boldsymbol{\phi} - \mathbf{r}^T \boldsymbol{\phi},$$

其中 K 是对称刚度矩阵 (stiffness matrix), $\mathbf{r}$ 是源项向量。最小化给出线性方程

$$K \boldsymbol{\phi} = \mathbf{r}.$$

8.7.4 局域刚度矩阵与组装

自然坐标梯度为

$$\nabla \xi_a = \frac{1}{2A} (y_{bc}, x_{cb}),$$

并可定义

$$\mathbf{b} = \begin{pmatrix} y_{bc} \\ y_{ca} \\ y_{ab} \end{pmatrix}, \quad \mathbf{c} = \begin{pmatrix} x_{cb} \\ x_{ac} \\ x_{ba} \end{pmatrix}.$$

则

$$\mathbf{k} = \mathbf{b} \mathbf{b}^T + \mathbf{c} \mathbf{c}^T,$$

局域刚度矩阵为

$$[k] = \frac{1}{4A} \begin{pmatrix} y_{bc}^2 + x_{cb}^2 & y_{bc}y_{ca} + x_{cb}x_{ac} & y_{bc}y_{ab} + x_{cb}x_{ba} \\ y_{bc}y_{ca} + x_{cb}x_{ac} & y_{ca}^2 + x_{ac}^2 & y_{ca}y_{ab} + x_{ac}x_{ba} \\ y_{bc}y_{ab} + x_{cb}x_{ba} & y_{ca}y_{ab} + x_{ac}x_{ba} & y_{ab}^2 + x_{ba}^2 \end{pmatrix}.$$

若源项也作线性插值

$$\tilde{f}(\mathbf{r}) = f_a \xi_a + f_b \xi_b + f_c \xi_c,$$

则局域载荷向量分量为

$$r_a = 2A \left(\frac{f_a}{12} + \frac{f_b}{24} + \frac{f_c}{24} \right),$$

$$r_b = 2A \left(\frac{f_a}{24} + \frac{f_b}{12} + \frac{f_c}{24} \right), \quad r_c = 2A \left(\frac{f_a}{24} + \frac{f_b}{24} + \frac{f_c}{12} \right).$$

最后对所有三角元求和，将局域矩阵和局域向量按共享顶点编号加到全局 K 与 $\mathbf{r}$ 中。施加 Dirichlet 边界条件后，只对内部点未知量求解。

8.8 边界条件与 Coulomb 势

8.8.1 Coulomb 势边界条件

轴对称 Coulomb 势可通过

$$\phi = \sqrt{\rho} U_c$$

把柱坐标中的方程写成

$$\frac{\partial^2 \phi(\rho, z)}{\partial \rho^2} + \frac{\phi(\rho, z)}{4\rho^2} + \frac{\partial^2 \phi(\rho, z)}{\partial z^2} = -4\pi e^2 \sqrt{\rho} \rho_p(\rho, z).$$

Coulomb 势本身为

$$U_c(\mathbf{r}) = e^2 \int \frac{\rho_p(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d^3 r'.$$

远处边界可由多极展开给出。定义电多极矩

$$Q_{lm} \equiv \int d\mathbf{r}' \rho(\mathbf{r}') (r')^l \sqrt{\frac{4\pi}{2l+1}} Y_{lm}^*(\theta', \varphi').$$

势函数写为

$$\Phi(\mathbf{r}) = \frac{1}{4\pi\epsilon} \sum_{l=0}^{\infty} \sum_{m=-l}^l \frac{Q_{lm}}{r^{l+1}} \sqrt{\frac{4\pi}{2l+1}} Y_{lm}(\theta, \varphi).$$

实际计算时，可先由多极展开给出边界值，再用二维有限差分或有限元求解内部线性方程组。

一种实用技巧是引入边界内辅助源 ρ_b ，使

$$\nabla^2 \phi_{\text{boundary}} = -4\pi \rho_b,$$

并在内部求解

$$\nabla^2 \phi = -4\pi(\rho - \rho_b).$$

这样离散微分算符包含边界信息，而投影后求解的线性方程组可以只包含内部未知量。

8.8.2 球谐函数与多极矩

球谐函数 (spherical harmonics) 定义为

$$Y_l^m(\theta, \varphi) = \sqrt{\frac{(2l+1)(l-m)!}{4\pi(l+m)!}} P_l^m(\cos \theta) \exp(im\varphi),$$

并满足正交归一关系

$$\int_{\theta=0}^{\pi} \int_{\varphi=0}^{2\pi} Y_l^m(\theta, \varphi) Y_{l'}^{m'*}(\theta, \varphi) d\Omega = \delta_{ll'} \delta_{mm'}.$$

常用 Legendre 多项式包括

$$P_2^0(x) = \frac{1}{2}(3x^2 - 1), \quad P_3^0(x) = \frac{1}{2}x(5x^2 - 3),$$

$$P_4^0(x) = \frac{1}{8}(35x^4 - 30x^2 + 3), \quad P_5^0(x) = \frac{1}{8}x(63x^4 - 70x^2 + 15).$$

低阶球谐函数包括

$$\begin{aligned} Y_0^0 &= \frac{1}{2\sqrt{\pi}}, & Y_1^0 &= \frac{1}{2}\sqrt{\frac{3}{\pi}} \frac{z}{\sqrt{x^2 + y^2 + z^2}}, \\ Y_1^1 &= -\frac{1}{2}\sqrt{\frac{3}{2\pi}} \frac{x + iy}{\sqrt{x^2 + y^2 + z^2}}, \\ Y_2^0 &= \frac{1}{4}\sqrt{\frac{5}{\pi}} \left(\frac{3z^2}{x^2 + y^2 + z^2} - 1 \right), \\ Y_2^1 &= -\frac{1}{2}\sqrt{\frac{15}{2\pi}} \frac{z(x + iy)}{x^2 + y^2 + z^2}, & Y_2^2 &= \frac{1}{4}\sqrt{\frac{15}{2\pi}} \frac{(x + iy)^2}{x^2 + y^2 + z^2}. \end{aligned}$$

这些公式用于把远场 Coulomb 势写成多极矩求和，并为数值边界提供解析参考。

8.8.3 轴对称 Coulomb 势的解析检验

轴对称密度的 Coulomb 势还可用积分表达式作为数值检验。第一种方法利用

$$\frac{1}{|\mathbf{r} - \mathbf{r}'|} = \frac{2}{\sqrt{\pi}} \int_0^\infty \frac{d\mu}{\mu^2} \exp\left[-\frac{(\mathbf{r} - \mathbf{r}')^2}{\mu^2}\right],$$

从而

$$\begin{aligned} V_d^C(\mathbf{r}) &= e^2 \frac{2}{\sqrt{\pi}} \int_0^\infty \frac{d\mu}{\mu^2} I_\mu(\mathbf{r}), \\ I_\mu(\mathbf{r}) &= \int d^3r' \exp\left[-\frac{(\mathbf{r} - \mathbf{r}')^2}{\mu^2}\right] \rho(\mathbf{r}'). \end{aligned}$$

轴对称后，

$$I_\mu(r', z') = 2\pi \int_0^\infty r dr \int_{-\infty}^\infty dz \exp\left[-\frac{r^2 + r'^2 + (z - z')^2}{\mu^2}\right] I_0\left(\frac{2rr'}{\mu^2}\right) \rho_p(r, z),$$

其中 $I_0(x)$ 是 Bessel 函数。

第二种方法利用

$$\Delta_{\mathbf{r}'} |\mathbf{r} - \mathbf{r}'| = \frac{2}{|\mathbf{r} - \mathbf{r}'|},$$

得到

$$V_d^C(\mathbf{r}) = \frac{e^2}{2} \int d^3r' |\mathbf{r} - \mathbf{r}'| \Delta_{\mathbf{r}'} \rho_p(\mathbf{r}').$$

轴对称化后

$$\begin{aligned} V_d^C(r', z') &= 2e^2 \int_0^\infty r dr \int_{-\infty}^\infty dz \sqrt{d(r, z)} E\left(\frac{4rr'}{d(r, z)}\right) \Delta \rho_p(r, z), \\ d(r, z) &= (z - z')^2 + (r + r')^2. \end{aligned}$$

这里 $E(x)$ 是第二类完全椭圆积分。这两类解析积分表达式通常不作为主要求解器，而用于检查有限差分或有限元边界处理是否可靠。

8.9 Jacobi 迭代法

Jacobi 方法 (Jacobi method) 是求解离散 Poisson 方程的基本迭代法。考虑二维区域内

$$-\nabla^2 u = f, \quad u = g \quad \text{on boundary.}$$

等距网格上，对内部点 $0 < (j, k) < N$ 有

$$\frac{1}{h^2} (4u_{j,k} - u_{j-1,k} - u_{j+1,k} - u_{j,k-1} - u_{j,k+1}) = f_{j,k},$$

边界点满足

$$u_{j,k} = g_{j,k}, \quad j = 0, N \text{ or } k = 0, N.$$

由离散方程移项可得 Jacobi 更新

$$\hat{u}_{j,k}^{\text{new}} = \frac{1}{4} [h^2 f_{j,k} + \hat{u}_{j-1,k} + \hat{u}_{j+1,k} + \hat{u}_{j,k-1} + \hat{u}_{j,k+1}].$$

每一轮更新只使用上一轮的邻点值，因此 Jacobi 方法易于并行，但收敛较慢。可加入松弛参数

$$\hat{u}_{j,k}^{\text{new}} = \hat{u}_{j,k} + \omega \left\{ \frac{1}{4} [h^2 f_{j,k} + \hat{u}_{j-1,k} + \hat{u}_{j+1,k} + \hat{u}_{j,k-1} + \hat{u}_{j,k+1}] - \hat{u}_{j,k} \right\},$$

其中

$$0 < \omega < 1$$

时为欠松弛迭代。更一般的 SOR 方法也允许 $\omega > 1$ 来加速收敛，但稳定取值依赖矩阵谱半径。Jacobi 迭代可以理解把边值问题转化为伪时间动力学初值问题：从初始猜测出发，不断扩散平滑误差直到达到稳态。

8.10 周期、吸收与 Twisted 边界条件

8.10.1 周期边界与 Fourier 微分

周期性边界条件 (periodic boundary condition, PBC) 满足

$$\psi(\mathbf{r} + \mathbf{T}_i) = \psi(\mathbf{r}),$$

其中

$$\mathbf{T}_i = L_i \mathbf{e}_i$$

是三个方向的晶格矢量。对于 PBC，Fourier 算符天然适用，因为

$$\exp[ik(x + L)] = \exp(ikx)$$

要求

$$k = \frac{2\pi n}{L}.$$

这使周期体系中的导数、动能和卷积都可以在动量空间高效计算。

有限盒子会把连续谱变为离散能级，产生 false quantization。吸收边界条件 (absorbing boundary condition, ABC) 通过在边界附近加入吸收层或复势减少反射；Twisted 边界条件 (twisted boundary condition, TBC) 则通过改变相位边界来恢复更接近连续的动量采样。

8.10.2 Twisted 边界条件

Bloch 型波函数可写作

$$\psi_{\alpha\mathbf{q}}(\mathbf{r}) = u_{\alpha\mathbf{q}}(\mathbf{r}) \exp(i\mathbf{q} \cdot \mathbf{r}),$$

其中 $\mathbf{q}$ 是波矢， $u_{\alpha\mathbf{q}}$ 具有周期性。Twisted 边界条件规定

$$\psi_{\alpha\theta}(\mathbf{r} + \mathbf{T}_i) = \exp(i\theta_i) \psi_{\alpha\theta}(\mathbf{r}),$$

其中 twist angle 为

$$\theta_i = \mathbf{q} \cdot \mathbf{T}_i, \quad i = x, y, z.$$

当

$$\mathbf{q} = 0, \quad \theta = 0$$

时退化为普通周期边界条件。

TBC 下允许的动量为

$$k_{i,n} = \frac{2\pi n + \theta_i}{L_i}, \quad n = 0, \pm 1, \pm 2, \dots, \pm n_{\max}.$$

因此通过改变 θ_i ，可以在固定盒子大小下移动动量格点。物理量可对 twist 角做平均：

$$\langle \hat{O} \rangle = \int \frac{d^3\theta}{\pi^3} \langle \Psi_\theta | \hat{O} | \Psi_\theta \rangle.$$

数值上常用 Gauss–Legendre 积分在 θ 空间取样。Fourier 微分算符也必须相应修改，因为波数由 $2\pi n/L_i$ 变为 $(2\pi n + \theta_i)/L_i$ 。

PBC、ABC 与 TBC 的适用场景不同：

- PBC 适合真正周期的晶格或无限均匀体系近似，计算简单且与 FFT 完全兼容；
- ABC 适合开放边界、散射和出射波问题，可减少边界反射；
- TBC 适合有限盒子模拟连续体系，可通过相位平均减小有限尺寸效应。

8.11 本章总结

结论 偏微分方程数值方法的核心是选择合适的表示：有限差分直接离散局域导数，Fourier 方法利用周期谱基，基矢展开把问题投影到物理基函数，DVR 兼具网格势能和基函数精度，B-样条提供紧支撑和非均匀格点，小波/多小波提供自适应多分辨率，有限元则擅长复杂几何和弱形式。

- 二维五点差分由 Taylor 展开得到，截断误差为 $O(h^2)$ ，可把 Poisson 方程转化为稀疏块三对角线性系统；Dirichlet、Neumann、Mixed 边界条件分别通过边界值、虚点导数和线性组合进入离散方程；
- 高阶差分提高光滑解精度但扩大模板，Fourier 微分在周期均匀网格上具有谱精度，FFT 将变换复杂度从 $O(N^2)$ 降为 $O(N \log N)$ ；
- 基矢展开法把 $H\psi = E\psi$ 投影为矩阵本征值问题，DVR 通过特殊格点和权重使局域势能近似对角、动能保留为矩阵；
- B-样条具有紧支撑、解析微分和非均匀格点能力；多小波用尺度空间和补空间分离粗尺度信息与细节系数，截断小系数可形成自适应非均匀网格；
- 有限元方法通过弱形式和单元组装处理复杂区域，线性三角元给出局域刚度矩阵；Coulomb 势边界可由多极展开、球谐函数或轴对称解析积分检验；
- Jacobi 迭代把离散边值问题转化为逐点松弛过程；周期、吸收与 Twisted 边界条件则分别服务于周期体系、开放边界和有限盒子连续谱修正。

第九章 动力学方程

内容提要

- 时间相关的薛定谔方程与初值演化问题
- 虚时间演化 (imaginary time evolution) 与 damping 迭代求解
- Crank–Nicolson 方法
- Split-operator method
- Krylov 子空间法
- Chebyshev Propagator
- 分子动力学
- Brown 运动

9.1 时间演化问题与量子动力学背景

9.1.1 时间相关方程作为初值问题

动力学方程的共同特征是：系统状态随时间演化，数值问题首先依赖初始值。若未知量只依赖时间 t ，普通常微分方程初值问题原则上可用 Runge–Kutta 等方法推进；但量子动力学中的时间相关薛定谔方程 (time-dependent Schrödinger equation, TDSE)

$$i\hbar \frac{d}{dt} |\Psi(t)\rangle = \hat{H} |\Psi(t)\rangle$$

具有特殊结构： $\hat{H}$ 为哈密顿算符 (Hamiltonian operator)， $|\Psi(t)\rangle$ 为态矢量， $\hbar$ 为约化 Planck 常数。若 $\hat{H}$ 厄米，则精确演化必须保持波函数范数、概率守恒和酉性。因此，TDSE 的数值求解不能只关心局部截断误差，还必须控制长期演化中的结构误差。

定义 9.1

时间演化问题 (time-evolution problem) 是指从给定初始态 $y(t_0) = y_0$ 出发，求解状态 $y(t)$ 在后续时间上的变化。其计算目标不是单个定态解，而是一条时间轨道或一族随时间变化的物理量。

在量子多体体系中，TDSE 常作为自洽动力学框架的出发点。实际理论包括：

- TD-DFT (time-dependent density functional theory)：用时间相关密度描述多体体系；
- TD-Hartree–Fock：让单粒子轨道在自洽平均场中演化；
- TD-density matrix：直接演化密度矩阵，适合处理混合态和耗散结构；
- TD-RPA (time-dependent random phase approximation)：在小振幅近似下得到线性响应理论。

量子多体动力学的一个重要特点，是集体运动 (collective motion) 与单粒子自由度 (single-particle degrees of freedom) 的相互作用。核反应中常见的非弹性散射、融合–裂变、融合–蒸发冷却、融合中的转子与振子、多核子转移、直接俘获和 knockout 等过程，都可以从这种耦合角度理解。若动力学偏离平衡态很小，可把方程在线性阶近似，得到小振幅近似 (small-amplitude approximation)，也就是线性响应理论 (linear response theory)。

9.2 扩散方程的时间离散

9.2.1 显式格式、隐式格式与中点思想

扩散方程 (diffusion equation) 是最简单的耗散型时间演化方程之一。考虑一维模型

$$\nabla^2 u(x, t) = \frac{\partial u(x, t)}{\partial t}, \quad u(x, 0) = g(x), \quad 0 < x < L,$$

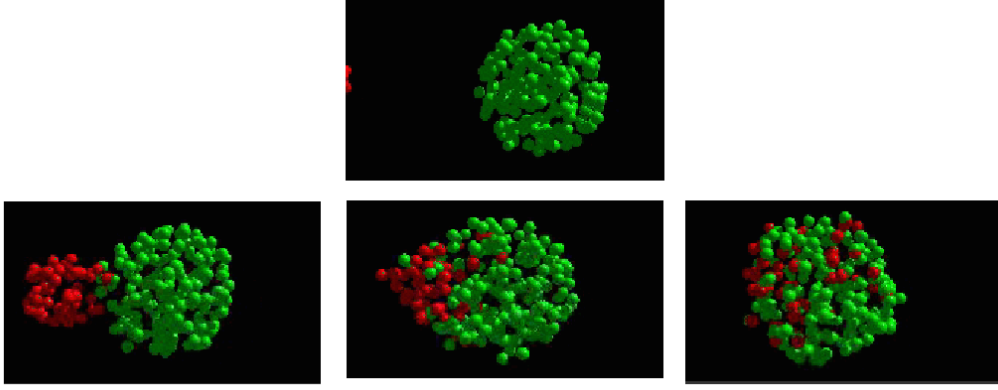


图 9.1: 量子多体反应动力学中的典型演化图像。颜色和空间分布表示不同组成部分在碰撞、融合或转移过程中的重排。

其中 $u(x, t)$ 是待求场, $g(x)$ 是初始分布。若记

$$x_i = i\Delta x, \quad t_j = j\Delta t, \quad u_{i,j} \approx u(x_i, t_j),$$

则空间二阶导数可由中心差分给出:

$$\frac{\partial^2 u}{\partial x^2}(x_i, t_j) \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2}.$$

结合最直接的向前时间差分, 得到

$$\frac{u_{i,j+1} - u_{i,j}}{\Delta t} = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2}.$$

令

$$\alpha = \frac{\Delta t}{\Delta x^2},$$

得到显式格式 (explicit scheme)

$$u_{i,j+1} = \alpha u_{i-1,j} + (1 - 2\alpha)u_{i,j} + \alpha u_{i+1,j}.$$

初始层由

$$u_{i,0} = g(x_i)$$

给出, 第一步为

$$u_{i,1} = \alpha g(x_{i-1}) + (1 - 2\alpha)g(x_i) + \alpha g(x_{i+1}).$$

该公式来自 Taylor 展开的一阶时间近似和二阶空间近似, 单步计算量小, 每个格点只依赖上一时间层的邻点。然而扩散方程具有平滑高频误差的物理性质, 显式格式若时间步过大反而会放大高频模, 因而只在

$$\alpha \leq \frac{1}{2}$$

时稳定。

向后时间差分为

$$\frac{\partial u}{\partial t}(x_i, t_j) \approx \frac{u(x_i, t_j) - u(x_i, t_j - \Delta t)}{\Delta t},$$

它把空间差分放在新时间层, 形成隐式格式 (implicit scheme)。隐式格式每步需要解线性方程组, 但对扩散方程无条件稳定。中点差分

$$\frac{\partial u}{\partial t}(x_i, t_j) \approx \frac{u(x_i, t_j + \Delta t) - u(x_i, t_j - \Delta t)}{2\Delta t}$$

提示我们可以在时间中心构造更高阶、更对称的格式, 这就是 Crank–Nicolson 方法的思想来源。

9.2.2 Crank–Nicolson 格式

Crank–Nicolson 方法 (Crank–Nicolson method, CN method) 把扩散算符在旧时间层和新时间层取平均。更一般地, 引入参数 θ :

$$\theta \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{\Delta x^2} + (1 - \theta) \frac{u_{i+1,j-1} - 2u_{i,j-1} + u_{i-1,j-1}}{\Delta x^2} = \frac{u_{i,j} - u_{i,j-1}}{\Delta t}.$$

当

$$\theta = \frac{1}{2}$$

时得到 CN 格式。代入 $\alpha = \Delta t / \Delta x^2$, 并把未知的新时间层项放到左端, 有

$$-\alpha u_{i-1,j} + (2 + 2\alpha)u_{i,j} - \alpha u_{i+1,j} = \alpha u_{i-1,j-1} + (2 - 2\alpha)u_{i,j-1} + \alpha u_{i+1,j-1}.$$

若记

$$V_j = (u_{1,j}, u_{2,j}, \dots, u_{n,j})^T,$$

并令 B 为主对角元为 2、上下副对角元为 -1 的三对角矩阵, 则

$$(2I + \alpha B)V_j = (2I - \alpha B)V_{j-1},$$

于是

$$V_j = (2I + \alpha B)^{-1}(2I - \alpha B)V_{j-1}.$$

 **笔记** CN 格式的本质是“半步显式、半步隐式”。它把正向 Euler 的低成本和反向 Euler 的稳定性结合起来, 使时间截断误差达到 $O(\Delta t^2)$, 空间仍为 $O(\Delta x^2)$ 。每步主要成本是三对角线性方程组求解, 可用 Thomas 算法在线性复杂度内完成。

扩散过程也可从随机游走理解。若扩散系数为 D , 一维无限空间中从原点出发的概率密度满足

$$w(x, t) dx = \frac{1}{\sqrt{4\pi Dt}} \exp\left(-\frac{x^2}{4Dt}\right) dx.$$

数值格式应当反映这种逐渐展宽和平滑的趋势。显式格式在 $\alpha > 1/2$ 时可能产生负概率或振荡, 正是因为它违背了扩散方程的耗散结构。

表 9.1: 扩散方程三种常用时间格式的精度与稳定性

格式	空间截断误差	时间截断误差	稳定性条件
Crank–Nicolson	$O(\Delta x^2)$	$O(\Delta t^2)$	任意 $\Delta t / \Delta x$ 稳定
Backward Euler	$O(\Delta x^2)$	$O(\Delta t)$	任意 $\Delta t / \Delta x$ 稳定
Forward Euler	$O(\Delta x^2)$	$O(\Delta t)$	$\Delta t \leq \frac{1}{2}\Delta x^2$

9.3 波动方程的时间推进

9.3.1 一维波动方程

波动方程 (wave equation) 与扩散方程不同, 它描述的是可传播的振荡过程而不是耗散平滑过程。考虑

$$\frac{\partial^2 u}{\partial x^2} = \frac{\partial^2 u}{\partial t^2},$$

配合初边值条件

$$\begin{cases} u_{xx} = u_{tt}, & x \in (0, 1), \quad t > 0, \\ u(x, 0) = g(x), & x \in (0, 1), \\ u(0, t) = u(1, t) = 0, & t > 0, \\ \left. \frac{\partial u}{\partial t} \right|_{t=0} = 0, & x \in (0, 1). \end{cases}$$

二阶中心差分给出

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u(x + \Delta x, t) - 2u(x, t) + u(x - \Delta x, t)}{\Delta x^2},$$

$$\frac{\partial^2 u}{\partial t^2} \approx \frac{u(x, t + \Delta t) - 2u(x, t) + u(x, t - \Delta t)}{\Delta t^2}.$$

于是得到 leapfrog 型更新公式

$$u_{i,j+1} = 2u_{i,j} - u_{i,j-1} + \frac{\Delta t^2}{\Delta x^2} (u_{i+1,j} - 2u_{i,j} + u_{i-1,j}).$$

该公式需要已知 t_j 和 t_{j-1} 两层数据。初始位移给出 $u_{i,0} = g(x_i)$ ，而初始速度为零意味着

$$\frac{u_{i,1} - u_{i,-1}}{2\Delta t} = 0, \quad u_{i,1} = u_{i,-1}.$$

把 $j = 0$ 代入更新式并利用上述对称性，得到第一层

$$u_{i,1} = u_{i,0} + \frac{\Delta t^2}{2\Delta x^2} (u_{i+1,0} - 2u_{i,0} + u_{i-1,0}).$$

注 扩散方程的稳定性受耗散约束控制，波动方程的稳定性则受传播速度约束控制。对 $u_{tt} = u_{xx}$ ，显式二阶格式的 CFL 条件为 $\Delta t/\Delta x \leq 1$ 。时间步过大时，数值波传播比网格允许的速度更快，会出现非物理振荡。

9.3.2 二维波动方程与自然边界处理

二维情形可写为

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = \frac{\partial^2 u}{\partial t^2},$$

配合初值

$$u(x, y, 0) = g(x, y), \quad \left. \frac{\partial u}{\partial t} \right|_{t=0} = 0,$$

以及边界上的零位移条件。令

$$t_l = l\Delta t, \quad x_i = i\Delta x, \quad y_j = j\Delta y, \quad \Delta x = \Delta y = h,$$

并记 $u_{i,j}^l \approx u(x_i, y_j, t_l)$ 。差分近似为

$$u_{xx} \approx \frac{u_{i+1,j}^l - 2u_{i,j}^l + u_{i-1,j}^l}{h^2},$$

$$u_{yy} \approx \frac{u_{i,j+1}^l - 2u_{i,j}^l + u_{i,j-1}^l}{h^2},$$

$$u_{tt} \approx \frac{u_{i,j}^{l+1} - 2u_{i,j}^l + u_{i,j}^{l-1}}{\Delta t^2}.$$

因此

$$u_{i,j}^{l+1} = 2u_{i,j}^l - u_{i,j}^{l-1} + \frac{\Delta t^2}{h^2} (u_{i+1,j}^l - 4u_{i,j}^l + u_{i-1,j}^l + u_{i,j+1}^l + u_{i,j-1}^l).$$

初始速度为零给出

$$u_{i,j}^{-1} = u_{i,j}^0 + \frac{\Delta t^2}{2h^2} (u_{i+1,j}^0 - 4u_{i,j}^0 + u_{i-1,j}^0 + u_{i,j+1}^0 + u_{i,j-1}^0),$$

等价地也可直接计算 $u_{i,j}^1$ 。二维显式波动格式的稳定性条件更严格；当波速为 1 且 $\Delta x = \Delta y = h$ 时，常用条件为

$$\frac{\Delta t}{h} \leq \frac{1}{\sqrt{2}}.$$

9.4 海啸模型：变系数波动方程

9.4.1 浅水波模型

海啸 (tsunami) 可用浅水波近似建立模型。若海水深度近似为常数，波面高度 $u(x, y, t)$ 满足

$$\lambda \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) = \frac{\partial^2 u}{\partial t^2},$$

其中 λ 与波速平方有关。一个简单初边值问题为

$$u(x, y, 0) = \sin(\pi x) \sin(2\pi y), \quad u|_{\partial\Omega} = 0, \quad \left. \frac{\partial u}{\partial t} \right|_{t=0} = 0.$$

更真实的海底地形要求深度随位置变化。若 $H(x, y)$ 是静水深度， g 是重力加速度，则浅水波速平方为

$$\lambda(x, y) = gH(x, y),$$

模型变为

$$\frac{\partial^2 u}{\partial t^2} = \nabla \cdot (\lambda(x, y) \nabla u).$$

这说明深水区域波速更大，浅水区域波长缩短、波幅可能显著增大。

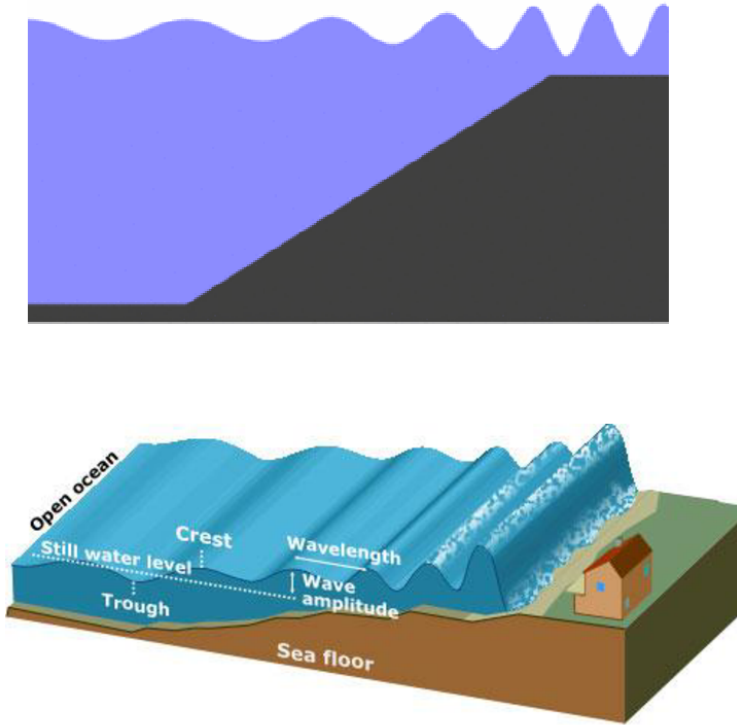


图 9.2: 海啸传播的浅水波图像。海底深度 $H(x, y)$ 改变局域波速，靠近海岸时波长缩短、波幅增大。

9.4.2 变系数项的离散

对

$$\frac{\partial}{\partial x} \left(\lambda(x, y) \frac{\partial u}{\partial x} \right)$$

应使用通量形式离散，以保证变系数介质中的守恒结构：

$$\frac{\partial}{\partial x} \left(\lambda \frac{\partial u}{\partial x} \right) \approx \frac{1}{\Delta x} \left[\lambda_{i+\frac{1}{2},j} \frac{u_{i+1,j}^l - u_{i,j}^l}{\Delta x} - \lambda_{i-\frac{1}{2},j} \frac{u_{i,j}^l - u_{i-1,j}^l}{\Delta x} \right].$$

同理

$$\frac{\partial}{\partial y} \left(\lambda \frac{\partial u}{\partial y} \right) \approx \frac{1}{\Delta y} \left[\lambda_{i,j+\frac{1}{2}} \frac{u_{i,j+1}^l - u_{i,j}^l}{\Delta y} - \lambda_{i,j-\frac{1}{2}} \frac{u_{i,j}^l - u_{i,j-1}^l}{\Delta y} \right].$$

半格点上的 $\lambda_{i+1/2,j}$ 可用相邻深度的平均值近似；若介质跳变很强，调和平均常比算术平均更能保持通量连续。

局域初始扰动常取 Gaussian 形状：

$$u(x, y, 0) = f(x, y), \quad \left. \frac{\partial u}{\partial t} \right|_{t=0} = 0,$$

$$f(x, y) = A_0 \exp \left[-\left(\frac{x - x_c}{\sigma_x} \right)^2 - \left(\frac{y - y_c}{\sigma_y} \right)^2 \right],$$

其中典型参数可取

$$A_0 = 2 \text{ m}, \quad \sigma_x = 80 \text{ km}, \quad \sigma_y = 200 \text{ km}.$$

这里 A_0 是初始海面抬升幅度， (x_c, y_c) 是震源中心， σ_x, σ_y 控制扰动的空间尺度。

9.5 时间演化算符

9.5.1 传播子与酉性

量子态的时间演化可写成传播子（propagator）形式：

$$|\Psi(t)\rangle = \hat{U}(t, t_0) |\Psi(t_0)\rangle.$$

若哈密顿量厄米，演化算符是酉算符（unitary operator）：

$$\hat{U}^\dagger(t, t_0) = \hat{U}^{-1}(t, t_0) = \hat{U}(t_0, t).$$

这保证

$$\langle \Psi(t) | \Psi(t) \rangle = \langle \Psi(t_0) | \Psi(t_0) \rangle,$$

即波函数范数和总概率守恒。

传播子满足

$$i\hbar \frac{\partial}{\partial t} \hat{U}(t, t_0) = \hat{H}(t) \hat{U}(t, t_0).$$

积分形式为

$$\hat{U}(t, t_0) = \hat{I} - \frac{i}{\hbar} \int_{t_0}^t \hat{H}(t_1) \hat{U}(t_1, t_0) dt_1.$$

若 $\hat{H}(t)$ 在不同时刻不对易，精确解需要时间排序算符（time-ordering operator）：

$$\hat{U}(t) \psi(0) = \mathcal{T} \exp \left(-\frac{i}{\hbar} \int_0^t \hat{H}(t') dt' \right) \psi(0).$$

实际计算通常把总时间分成小步，在每个时间片上近似

$$\hat{U}(t_{k+1}, t_k) \approx \exp \left[-\frac{i}{\hbar} \Delta t \hat{H}(t_k) \right].$$

9.5.2 Crank–Nicolson 作为 Cayley 传播子

对单个小时时间步，用中点哈密顿量

$$\hat{H}_{k+1/2} = \hat{H}(t_{k+1/2})$$

构造

$$|\Psi(t_{k+1})\rangle = \left[\hat{I} + \frac{i\Delta t}{2\hbar} \hat{H}_{k+1/2} \right]^{-1} \left[\hat{I} - \frac{i\Delta t}{2\hbar} \hat{H}_{k+1/2} \right] |\Psi(t_k)\rangle.$$

这是指数算符的 Cayley 近似，局部误差为 $O(\Delta t^3)$ 。若 $\hat{H}_{k+1/2}$ 厄米，则

$$\left(\left[\hat{I} + \frac{i\Delta t}{2\hbar} \hat{H} \right]^{-1} \left[\hat{I} - \frac{i\Delta t}{2\hbar} \hat{H} \right] \right)^\dagger \left(\left[\hat{I} + \frac{i\Delta t}{2\hbar} \hat{H} \right]^{-1} \left[\hat{I} - \frac{i\Delta t}{2\hbar} \hat{H} \right] \right) = \hat{I}.$$

因此 CN 传播子保持酉性。它每步需要求矩阵逆或解线性方程组，但长期稳定性好。在几何积分观点下，这类保持结构的时间推进与冯康辛算法（Feng–Kang symplectic algorithm）具有相同精神：数值格式应尽量保持原方程的几何守恒结构。

9.5.3 一维 TDSE 的 CN 离散

考虑一维外场驱动 TDSE：

$$i \frac{\partial}{\partial t} \psi(x, t) = \left[-\frac{1}{2} \frac{\partial^2}{\partial x^2} + V(x) + \mathcal{E}(t)x \right] \psi(x, t),$$

其中 $V(x)$ 是静态势能， $\mathcal{E}(t)x$ 可表示偶极近似下的时间相关外场。空间二阶导数离散为

$$\frac{\partial^2}{\partial x^2} \psi(x_j, t) = \frac{\psi(x_{j+1}, t) - 2\psi(x_j, t) + \psi(x_{j-1}, t))}{(\Delta x)^2}.$$

CN 步可写为

$$U^-(t_{k+1/2})\psi(t_{k+1}) = U^+(t_{k+1/2})\psi(t_k),$$

其中一种常用无量纲离散约定为

$$U_{j,j'}^\pm = \begin{cases} \pm \frac{i\Delta t}{2(\Delta x)^2}, & j = j' + 1, j = j' - 1, \\ 1 \mp i\Delta t \left[\frac{1}{(\Delta x)^2} + V(x_j) + \mathcal{E}(t_{k+1/2})x_j \right], & j = j', \\ 0, & \text{otherwise.} \end{cases}$$

若 Hamiltonian 中动能前因子采用 $-\frac{1}{2}\partial_x^2$ ，矩阵元的数值系数会相应带有 1/2 因子；不变的是 CN 的结构：

$$\left(I + \frac{i\Delta t}{2} H \right) \psi^{k+1} = \left(I - \frac{i\Delta t}{2} H \right) \psi^k.$$

由于 $U^\pm$ 是三对角矩阵，这一步可高效稳定地完成。

9.5.4 二阶传播格式

另一种显式时间推进来自对精确传播子的对称差分：

$$\psi(t + \Delta t) - \psi(t - \Delta t) = \left(\exp \left[-\frac{i}{\hbar} \hat{H} \Delta t \right] - \exp \left[\frac{i}{\hbar} \hat{H} \Delta t \right] \right) \psi(t).$$

利用

$$\exp(-ia) - \exp(ia) = -2i \sin a$$

并取最低阶近似 $\sin a \approx a$ ，得到

$$\psi(t + \Delta t) \approx \psi(t - \Delta t) - \frac{2i\Delta t}{\hbar} \hat{H} \psi(t).$$

令

$$a = \frac{\Delta t}{\hbar} \hat{H},$$

连续应用可写成两步矩阵形式

$$\begin{pmatrix} \psi^{n+1} \\ \psi^n \end{pmatrix} = \begin{pmatrix} 1 - 4a^2 & -2ia \\ -2ia & 1 \end{pmatrix} \begin{pmatrix} \psi^{n-1} \\ \psi^{n-2} \end{pmatrix}.$$

在 $\hat{H}$ 的本征模上，传播矩阵本征值为

$$\lambda_{1,2} = 1 - 2a^2 \pm 2a\sqrt{a^2 - 1}.$$

当 $|a| \leq 1$ 时, 根号为纯虚数且 $|\lambda_{1,2}| = 1$, 格式不放大该模。因此稳定性要求

$$\Delta t < \frac{\hbar}{E_{\max}},$$

其中 $E_{\max}$ 是离散 Hamiltonian 中需要解析的最大能量尺度。该格式每步无需解线性方程, 但稳定时间步由谱半径限制。

9.6 Split-operator method

9.6.1 算符分裂与 BCH 关系

Split-operator method (算符分裂法) 利用 Hamiltonian 的动能与势能结构:

$$\hat{H} = \hat{K} + \hat{V}.$$

其中 $\hat{K} = \hat{p}^2/(2m)$ 在动量空间对角, $\hat{V} = V(\hat{x})$ 在坐标空间对角。若 $\hat{K}$ 与 $\hat{V}$ 对易, 则

$$\exp[-\Delta\tau(\hat{K} + \hat{V})] = \exp(-\Delta\tau\hat{K}) \exp(-\Delta\tau\hat{V}).$$

实际中二者通常不对易, 简单一阶分裂只有

$$\exp[-\Delta\tau(\hat{K} + \hat{V})] = \exp(-\Delta\tau\hat{K}) \exp(-\Delta\tau\hat{V}) + O(\Delta\tau^2).$$

由 Baker–Campbell–Hausdorff 关系可构造二阶对称分裂:

$$\exp[-\Delta\tau(\hat{V} + \hat{K})] = \exp\left(-\frac{\Delta\tau}{2}\hat{V}\right) \exp(-\Delta\tau\hat{K}) \exp\left(-\frac{\Delta\tau}{2}\hat{V}\right) + O(\Delta\tau^3).$$

对实时量子演化, $\Delta\tau = i\Delta t/\hbar$, 可写为

$$\hat{U}(\Delta t) = \exp\left(-\frac{i}{\hbar}\hat{H}\Delta t\right) \approx \exp\left(-\frac{i}{2\hbar}\hat{K}\Delta t\right) \exp\left(-\frac{i}{\hbar}\hat{V}\Delta t\right) \exp\left(-\frac{i}{2\hbar}\hat{K}\Delta t\right) + O(\Delta t^3).$$

若 V 显含时间, 常用中点势能:

$$\hat{U}(t + \Delta t, t) \approx \exp\left[-\frac{i\Delta t}{2\hbar}\hat{K}\right] \exp\left[-\frac{i\Delta t}{\hbar}\hat{V}\left(t + \frac{\Delta t}{2}\right)\right] \exp\left[-\frac{i\Delta t}{2\hbar}\hat{K}\right].$$

9.6.2 FFT 实现步骤

坐标表象下 $\hat{V}$ 只需逐点相乘, 动能项则通过 Fourier 变换到动量空间实现:

$$\exp\left[-\frac{i\Delta t}{2\hbar}\hat{K}(x)\right] \psi(x) = \mathcal{F}^{-1} \exp\left[-\frac{i\Delta t}{2\hbar}\hat{K}(p)\right] \mathcal{F}\psi(x).$$

以 $V/2 - K - V/2$ 对称分裂为例, 单步流程为:

1. 在坐标空间作用半步势能指数 $\exp[-iV(x)\Delta t/(2\hbar)]$;
2. Fourier 变换到动量空间;
3. 作用动能指数 $\exp[-iK(p)\Delta t/\hbar]$;
4. 反 Fourier 变换回坐标空间;
5. 再作用半步势能指数。

该方法不需要显式构造大矩阵, 主要成本是 FFT, 适合周期边界、均匀网格和大规模波包演化。它在量子蒙特卡罗 (quantum Monte Carlo, QMC) 中的短时传播也有重要应用。缺点是: 高阶分裂需要更多指数因子; 非周期边界、强奇异势或复杂几何下, FFT 表示可能不再自然。

9.7 Krylov 子空间传播

9.7.1 Krylov 子空间与 Lanczos 递推

Krylov 子空间法 (Krylov subspace method) 把大 Hilbert 空间中的指数算符作用, 投影到由

$$u_j = \hat{H}^j \psi(0)$$

张成的小维子空间中。若只关心

$$\psi(\Delta t) = \exp\left(-\frac{i}{\hbar} \hat{H} \Delta t\right) \psi(0),$$

没有必要完整对角化 $\hat{H}$, 只需近似它在

$$\mathcal{K}_N(\hat{H}, \psi_0) = \text{span}\{\psi_0, \hat{H}\psi_0, \hat{H}^2\psi_0, \dots, \hat{H}^{N-1}\psi_0\}$$

上的作用。

对厄米 $\hat{H}$, Lanczos 方法产生正交基 q_j 。取

$$q_0 = \psi(0)$$

并归一化, 递推为

$$\hat{H}q_0 = \alpha_0 q_0 + \beta_0 q_1,$$

$$\hat{H}q_j = \beta_{j-1} q_{j-1} + \alpha_j q_j + \beta_j q_{j+1},$$

其中

$$\alpha_j = (q_j, \hat{H}q_j), \quad \beta_{j-1} = (q_{j-1}, \hat{H}q_j).$$

若 $Q_m = (q_0, q_1, \dots, q_{m-1})$, 则

$$Q_m^T A Q_m = T_m, \quad A Q = Q T,$$

其中 Q 为正交基矩阵, T_m 是小三对角矩阵。

9.7.2 小三对角 Hamiltonian 的指数

Lanczos 投影得到

$$\hat{H}_N = \begin{pmatrix} \alpha_0 & \beta_0 & 0 & \cdots & 0 \\ \beta_0 & \alpha_1 & \beta_1 & \cdots & 0 \\ 0 & \beta_1 & \alpha_2 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & \beta_{N-2} \\ 0 & 0 & \cdots & \beta_{N-2} & \alpha_{N-1} \end{pmatrix}.$$

在该子空间上近似

$$\hat{U}(\Delta t) \approx \hat{U}_N(\Delta t) = \exp\left[-\frac{i}{\hbar} \hat{H}_N \Delta t\right],$$

$$\psi(\Delta t) \approx \hat{U}_N(\Delta t) \psi(0).$$

由于 $\hat{H}_N$ 维数很小, 可以直接对角化

$$\hat{H}_N = Z^\dagger D_N Z,$$

从而

$$\hat{U}_N(\Delta t) = Z^\dagger \exp\left[-\frac{i}{\hbar} D_N \Delta t\right] Z, \quad \psi(\Delta t) = \hat{U}_N(\Delta t) q_0.$$

最后一个 Krylov 分量常可作为误差估计:

$$q_N(\Delta t) \approx \frac{(-i\Delta t/\hbar)^{N-1}}{(N-1)!} \prod_{k=0}^{N-1} \beta_k.$$

它说明: 若 Δt 不大且 β_k 不导致高阶项增大, 较小 Krylov 维数即可得到高精度。

把投影写回原空间, 可得

$$|\Psi(t + \Delta t)\rangle \approx \sum_{j=0}^L |Q_j\rangle \langle Q_j| \exp(-i\hat{H}\Delta t) |\Psi(t)\rangle,$$

进一步用 Lanczos Hamiltonian H_L 替代全空间算符:

$$|\Psi(t + \Delta t)\rangle \approx \sum_{j=0}^L |Q_j\rangle \sum_{k=0}^L [S]_{jk} \exp[-i\lambda_k \Delta t] [S^{-1}]_{k0} \|\Psi(t)\|,$$

其中

$$S^{-1} H_L S = \lambda, \quad \langle Q_j | H | Q_i \rangle \longrightarrow H_{\text{Lanczos}}.$$

这一步的好处是指数只作用在小矩阵上, 不再需要对全空间算符求指数。Lanczos 传播通常保持酉性、稳定, 并且所需 Krylov 维数较小。对非厄米矩阵或非正交投影问题, 可使用 Arnoldi 方法 (Arnoldi method) 构造上 Hessenberg 小矩阵。

9.8 多项式传播子与 Chebyshev Propagator

9.8.1 Polynomial Propagators

多项式传播子 (polynomial propagator) 的思想是把矩阵指数转化为多项式作用:

$$\exp(-i\hat{H}t) |\Psi\rangle = \sum_{n=0}^{\infty} c_n p_n(-i\hat{H}t) |\Psi\rangle.$$

这样可避免直接矩阵指数运算。若矩阵维数为 n , 一般矩阵指数的直接成本可达 $O(n^3)$, 而多项式展开只需重复矩阵-向量乘法:

$$\hat{H}^2 |\Psi\rangle = \hat{H} (\hat{H} |\Psi\rangle),$$

每次约为 $O(n^2)$, 稀疏矩阵甚至更低。

最简单的选择是 Taylor 展开:

$$\exp(-i\hat{H}t) |\Psi\rangle = \sum_{n=0}^{\infty} \frac{(-i\hat{H}t)^n}{n!} |\Psi\rangle,$$

或

$$\hat{U} = \exp\left(-\frac{i}{\hbar} \hat{H} t\right) = 1 - \frac{i\hat{H}t}{\hbar} + \frac{1}{2!} \left(\frac{i\hat{H}t}{\hbar}\right)^2 - \dots$$

Taylor 展开直观, 但实际问题中必须判断截断到多少阶、收敛是否足够快。FFT 微分算符方法中常见做法是展开到四阶左右, 以在精度和成本之间折中。

9.8.2 Chebyshev 多项式

Chebyshev propagator 利用 Chebyshev 多项式 (Chebyshev polynomials) 在区间端点附近的优良全局逼近性质。许多插值多项式在区间边界会出现 Runge 振荡, 而 Chebyshev 多项式可显著减轻这种问题。

标准 Chebyshev 多项式定义为

$$T_n(x) = \cos(n \arccos x), \quad x \in [-1, 1].$$

其递推关系为

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x), \quad n \geq 1.$$

有时也使用归一化变体

$$\tilde{T}_n(x) = \frac{1}{2^{n-1}} \cos(n \arccos x),$$

以调整最高次项系数。Chebyshev 多项式构成正交多项式族，通常比 Taylor 级数具有更快、更均匀的收敛。

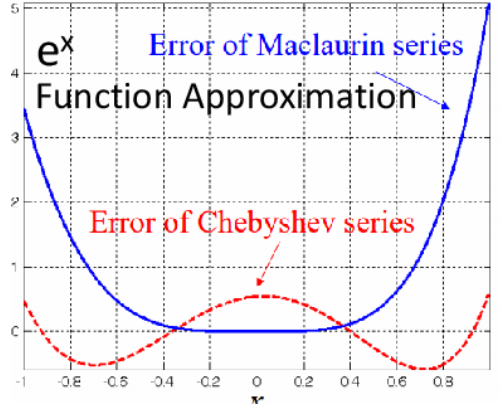


图 9.3: Maclaurin 展开与 Chebyshev 展开在区间上的误差比较。Chebyshev 展开倾向于把误差更均匀地分布在区间内。

一般多项式展开可写成能量本征基形式：

$$\exp(-i\hat{H}t)|\Psi\rangle = \sum_{n=0}^{\infty} \sum_l c_n p_n(-iE_l t) |E_l\rangle \langle E_l|\Psi\rangle.$$

这说明多项式传播子的收敛速度取决于能谱范围和多项式对 $\exp(-iEt)$ 的逼近质量。

9.8.3 Chebyshev Propagator 的归一化能谱

Chebyshev 展开要求自变量落在 $[-1, 1]$ 。若 Hamiltonian 的能谱位于

$$E_{\min} \leq E \leq E_{\max}, \quad \Delta = E_{\max} - E_{\min},$$

则定义归一化 Hamiltonian

$$\hat{H}_{\text{norm}} = 2 \frac{\hat{H} - \left(\frac{1}{2}\Delta + E_{\min}\right)}{\Delta}, \quad H_{\text{norm}} \in [-1, 1].$$

令复空间多项式

$$\phi_n(x) = T_n(-ix),$$

则传播子可写为

$$\exp(-i\hat{H} dt)|\Psi\rangle = \sum_n c_n \phi_n(-i\hat{H}_{\text{norm}})|\Psi\rangle,$$

其中展开系数为

$$c_n = (2 - \delta_{n0}) \exp\left[-i\left(\frac{\Delta}{2} + E_{\min}\right) dt\right] J_n\left(\frac{\Delta}{2} dt\right).$$

这里 $J_n(x)$ 是第一类 Bessel 函数 (Bessel functions of the first kind), δ_{n0} 是 Kronecker delta。等价的标准实 Chebyshev 写法会把 $(-i)^n$ 吸收到系数中。

递推计算为

$$\phi_n(\hat{A}) = 2\hat{A}\phi_{n-1}(\hat{A}) - \phi_{n-2}(\hat{A}), \quad \phi_1(\hat{A}) = \hat{A}, \quad \phi_0(\hat{A}) = 1.$$

实际传播时不需要保存所有多项式向量，只需滚动保存前两项。系数 c_n 在连续传播中可重复使用；随着 n 增大，

Bessel 系数很快变小，常用估计为

$$n_{\max} \approx 2\Delta dt.$$

Chebyshev propagator 常被视为数学上接近最优的多项式传播子之一，通常优于 Taylor 展开，尤其适合高精度长时间传播。其主要前提是需要知道或估计 Hamiltonian 的能谱上下界。

9.8.4 Magnus 展开

Magnus expansion (Magnus 展开) 用于处理显含时间且不同时刻不对易的线性系统：

$$\frac{dY(t)}{dt} = A(t)Y(t), \quad Y(0) = Y_0.$$

若 $A(t_1)$ 与 $A(t_2)$ 总是对易，则可直接写成

$$Y(t) = \exp\left(\int_{t_0}^t A(s) ds\right) Y_0.$$

一般情形下不能这样交换指数和积分，Magnus 展开写为

$$Y(t) = \exp(\Omega(t))Y_0, \quad \Omega(t) = \sum_{k=1}^{\infty} \Omega_k(t).$$

前三项为

$$\begin{aligned} \Omega_1(t) &= \int_0^t A(t_1) dt_1, \\ \Omega_2(t) &= \frac{1}{2} \int_0^t dt_1 \int_0^{t_1} dt_2 [A(t_1), A(t_2)], \\ \Omega_3(t) &= \frac{1}{6} \int_0^t dt_1 \int_0^{t_1} dt_2 \int_0^{t_2} dt_3 ([A(t_1), [A(t_2), A(t_3)]] + [A(t_3), [A(t_2), A(t_1)]]) . \end{aligned}$$

这些嵌套对易子补偿了时间排序造成的误差，因此在保持指数结构、提高保真度和加快收敛方面很有优势。

9.9 时间相关薛定谔方程传播方法小结

求解 TDSE 的核心等价于计算酉传播子

$$U(t) = \exp(-iHt).$$

数值方法的差别主要在于如何近似指数、如何控制酉性、以及如何降低矩阵指数成本。

- 一阶 Taylor 展开若直接使用会破坏酉性；通过 Cayley 形式修正后得到 Crank–Nicolson 方法；
- Split-step 或 split-operator 方法把指数拆成动能和势能两部分，利用 T 在 k -space 对角、 V 在 r -space 对角的结构简化计算；
- Krylov Arnoldi–Lanczos 方法相当于在全 Hilbert 空间的一个小投影子空间中计算高阶 Taylor 信息；
- Chebyshev 多项式展开比 Taylor 展开通常有更好的收敛性质，适合高精度传播。

表 9.2: TDSE 常用传播方法的特点

方法	主要思想	优点	主要限制
Crank–Nicolson	Cayley 近似	酉、稳定	需解线性方程
Split-operator	K/V 对称分裂	FFT 高效、易实现	依赖可分结构
Krylov/Lanczos	小子空间指数	高精度、维数小	需重建 Krylov 基
Chebyshev	正交多项式逼近	收敛快、精度高	需能谱范围
Magnus	指数化时间排序	适合显含时系统	高阶对易子复杂

9.10 虚时间演化与 Damped 梯度法

9.10.1 虚时间演化求基态

虚时间演化 (imaginary time evolution) 把实时变量作替换

$$t \rightarrow -i\tau.$$

设定态本征函数满足

$$\hat{H}\phi_m(x) = E_m\phi_m(x),$$

初态展开为

$$\psi(x, t) = \sum_m c_m \phi_m(x) \exp\left(-\frac{iE_m t}{\hbar}\right).$$

虚时间后

$$\psi(x, -i\tau) = \sum_m c_m \phi_m(x) \exp\left(-\frac{E_m \tau}{\hbar}\right).$$

若 E_0 是最低能量, 提出公共因子:

$$\psi(x, -i\tau) = \exp\left(-\frac{E_0 \tau}{\hbar}\right) \left[c_0 \phi_0 + c_1 \phi_1 \exp\left(-\frac{(E_1 - E_0)\tau}{\hbar}\right) + c_2 \phi_2 \exp\left(-\frac{(E_2 - E_0)\tau}{\hbar}\right) + \dots \right].$$

随 τ 增大, 所有激发态分量都比基态衰减得更快。每步归一化后, 波函数收敛到基态。若要得到更高激发态, 需要在每一步对已收敛低能态正交化, 避免再次回到基态。

 **笔记** 虚时间演化的物理意义是把薛定谔方程从振荡型传播变成扩散型投影。它不描述真实时间动力学, 而是通过指数滤波选择最低能量分量。

9.10.2 Damped 梯度法

在自洽薛定谔方程中, Hamiltonian 依赖当前波函数或密度。Damped 梯度法 (damped gradient method) 把能量最小化写成带阻尼的迭代过程。设已收敛或需排除的态为 $\{|\phi_k\rangle\}_{k=1}^A$, 投影算符为

$$\hat{\Pi} = 1 - \sum_{k=1}^A |\phi_k\rangle\langle\phi_k|.$$

一个基本更新为

$$|\phi_k^{(n+1)}\rangle = O\left\{|\phi_k^{(n)}\rangle - \delta \hat{\Pi} \hat{h} |\phi_k^{(n)}\rangle\right\},$$

其中 $\hat{h}$ 是当前自洽单粒子 Hamiltonian, δ 是步长, $O\{\dots\}$ 表示正交归一化操作。

为了对高动能分量进行 damping, 可使用带动能预条件的形式:

$$|\phi_\alpha^{(n+1)}\rangle = O\left\{|\phi_\alpha^{(n)}\rangle - \frac{1}{\hat{t} - E_0} \hat{\Pi} \hat{h} |\phi_\alpha^{(n)}\rangle\right\},$$

其中 $\hat{t}$ 是动能算符, E_0 是用于平移谱的参数。也可写成

$$|\phi_\alpha^{(n+1)}\rangle = O\left\{|\phi_\alpha^{(n)}\rangle - \hat{D} \left[\hat{h} - \langle\phi_\alpha|\hat{h}|\phi_\alpha\rangle \right] |\phi_\alpha^{(n)}\rangle\right\},$$

$$\hat{D} = x_0 \frac{1}{1 + \hat{t}/E_0}.$$

这里 $\hat{D}$ 抑制短波长、高动能误差, $\hat{\Pi}$ 去掉已经收敛的态, 能量期望值项去掉沿当前态本身的平凡归一化方向。该方法适合求解自洽基态、约束态和一组正交单粒子轨道。

9.11 分子动力学

9.11.1 经典分子动力学

经典分子动力学 (classical molecular dynamics, classical MD) 把粒子位置和速度作为基本变量。对第 i 个粒子,

$$\frac{d^2 \mathbf{r}_i(t)}{dt^2} = \frac{\mathbf{F}_i(\mathbf{R})}{m_i},$$

其中

$$\mathbf{F}_i(\mathbf{R}) = \sum_{\substack{j=1 \\ j \neq i}}^N F(|\mathbf{r}_i - \mathbf{r}_j|) \hat{\mathbf{r}}_{ij}.$$

粒子间势能可采用 Lennard-Jones 势:

$$U(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right],$$

其中 ϵ 控制势阱深度, σ 控制相互作用长度。

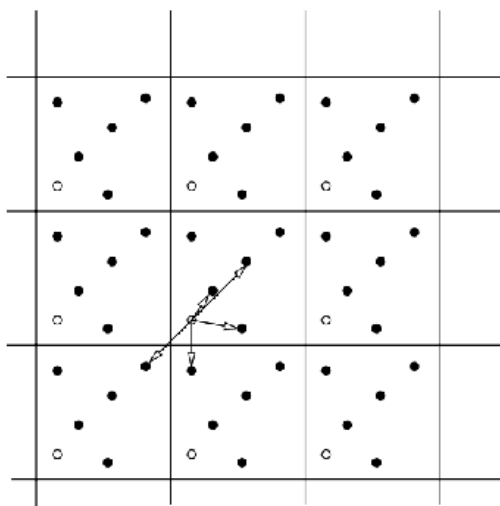


图 9.4: 经典分子动力学中周期盒子、相互作用势和粒子轨道的示意。

MD 计算的基本步骤为:

1. 给定初始位置和速度;
2. 用 Verlet 格式推进

$$\mathbf{r}(t+h) = 2\mathbf{r}(t) - \mathbf{r}(t-h) + h^2 \frac{\mathbf{F}(t)}{m},$$

速度可由中心差分估计

$$\mathbf{v}(n) = \frac{\mathbf{r}(n+1) - \mathbf{r}(n-1)}{2h};$$

3. 演化到不同系综的平衡态, 统计能量、温度、压强、关联函数等宏观性质。

若加入摩擦力, 可写成

$$\ddot{\mathbf{x}} = \mathbf{F}(\mathbf{x}) - \gamma \dot{\mathbf{x}},$$

其中 $\gamma \dot{\mathbf{x}}$ 是阻尼项。摩擦会耗散机械能, 使体系趋向某种平衡或稳态; 若再加入随机力, 就进入 Langevin 动力学。

9.11.2 量子分子动力学

量子分子动力学 (quantum molecular dynamics, QMD) 仍保留“粒子轨道”语言, 但其动力学由 Hamiltonian 主导波函数或波包参数的演化。常用近似是把单粒子波函数写成 Gaussian wave packet:

$$\phi_i(\mathbf{r}) \propto \exp \left[-\nu(\mathbf{r} - \mathbf{R}_i)^2 + \frac{i}{\hbar} \mathbf{P}_i \cdot \mathbf{r} \right],$$

其中 $\mathbf{R}_i, \mathbf{P}_i$ 是波包中心位置和动量, ν 控制宽度。若系统态记作 ψ_G , 参数集合记作 S , 则广义力可由能量泛函导出:

$$\mathbf{F}_n = -\nabla_n E(S) = -\nabla_n \left[\frac{\langle \psi_G | H(S) | \psi_G \rangle}{\langle \psi_G | \psi_G \rangle} \right].$$

这与 TD-Hartree-Fock 的区别在于: TD-Hartree-Fock 直接演化一组自洽单粒子轨道, 而 QMD 常在预设的 Gaussian 波包流形上演化有限个参数。前者更接近平均场量子动力学, 后者更接近带量子修正的分子动力学。

9.12 非平衡统计动力学

9.12.1 Brown 运动与 Langevin 方程

Brown 运动 (Brownian motion) 描述粒子在介质中受到摩擦力和随机力后的无规则运动。Langevin 方程为

$$m \frac{dv}{dt}(t) = -\gamma v(t) + F(t) + R(t),$$

其中 $-\gamma v(t)$ 是阻尼力, $F(t)$ 是外力或平均力, $R(t)$ 是随机力。随机力满足

$$\langle R(t) \rangle = 0, \quad \langle R(t)R(t+\tau) \rangle = q\delta(\tau).$$

涨落耗散定理 (fluctuation-dissipation theorem) 给出

$$q = 2k_B T \gamma,$$

其中 k_B 是 Boltzmann 常数, T 是温度。该关系说明随机涨落强度与耗散强度不是独立的: 若介质提供更强摩擦, 也必须提供相应热涨落, 才能维持正确的热平衡分布。

9.12.2 输运理论与 BUU 方程

非平衡统计动力学还可用相空间分布函数 $f(\mathbf{r}, \mathbf{p}, t)$ 描述。Boltzmann (Vlasov)-Uehling-Uhlenbeck equation, 简称 BUU 方程, 具有形式

$$\frac{d}{dt} f(\mathbf{r}, \mathbf{p}, t) \equiv \frac{\partial f}{\partial t} + \frac{\mathbf{p}}{m} \cdot \nabla_{\mathbf{r}} f(\mathbf{r}, \mathbf{p}, t) - \nabla_{\mathbf{r}} U(\mathbf{r}, t) \cdot \nabla_{\mathbf{p}} f(\mathbf{r}, \mathbf{p}, t) = \left(\frac{\partial f}{\partial t} \right)_{\text{coll}}.$$

左端是平均场中的 Vlasov 演化, 右端是碰撞项。分布函数可由量子密度矩阵的 Wigner 变换引入, 因而它连接了量子多体描述与经典相空间输运图像。

碰撞积分可写成

$$I_{\text{coll}} = \frac{4}{(2\pi)^3} \int d^3 p_2 d^3 p_3 \int d\Omega |v_{12}| \delta^3(\mathbf{p}_1 + \mathbf{p}_2 - \mathbf{p}_3 - \mathbf{p}_4) \frac{d\sigma}{d\Omega} (1 + 2 \rightarrow 3 + 4) P.$$

这里 v_{12} 是相对速度, $d\sigma/d\Omega$ 是微分散射截面, δ^3 保证动量守恒, P 表示 Pauli blocking 或其他统计因子。平均场项描述连续漂移, 碰撞项描述粒子间离散散射, 两者共同决定非平衡弛豫。

9.12.3 流体力学与 Navier-Stokes 方程

若平均自由程远小于宏观尺度, 体系可用流体力学描述; 若平均自由程很大, 则更适合用输运或统计动力学。二者的判据通常是 Knudsen number:

$$\text{Kn} = \frac{\ell_{\text{mfp}}}{L},$$

其中 ℓ_{mfp} 是平均自由程, L 是系统宏观长度。小 Kn 对应流体极限, 大 Kn 对应稀薄气体或输运极限。

Navier–Stokes equations 的动量方程可写为

$$\rho \frac{D\mathbf{u}}{Dt} = \rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla \bar{p} + \mu \nabla^2 \mathbf{u} + \frac{1}{3} \mu \nabla (\nabla \cdot \mathbf{u}) + \rho \mathbf{g}.$$

其中 ρ 是质量密度, $\mathbf{u}$ 是流速, $\bar{p}$ 是压强, μ 是黏性系数, $\mathbf{g}$ 是体力加速度。该方程与 BUU 方程的关系是: 在碰撞足够频繁、局域平衡足够快的极限下, 可从输运方程取矩得到流体方程。

9.13 本章总结

结论 动力学方程的核心是时间推进。扩散方程强调耗散稳定性, 波动方程强调 CFL 传播限制; TDSE 则要求传播子保持酉性、稳定性和长期误差控制。Crank–Nicolson、split-operator、Krylov/Lanczos、Chebyshev 和 Magnus 展开分别从不同角度近似时间演化算符。虚时间演化把量子传播转化为基态投影, 分子动力学和非平衡统计动力学则把时间推进扩展到经典粒子、量子波包、随机过程、输运理论和流体力学。

- TDSE 是描述丰富量子动力学现象的基本框架, 不要求每一步都严格求解定态方程, 而是关注随时间的传播;
- TDSE 是较特殊的单变量微分方程, 一般不宜直接套用普通 Runge–Kutta, 因为酉性、能量守恒、稳定性和长期误差积累都很关键;
- 传播子方法的主要问题是: 是否 unitary, 是否能量守恒, 是否 stable, 长期误差如何积累, 以及精度与计算量如何平衡;
- Crank–Nicolson 用矩阵求逆换取酉稳定; split-operator 利用动能和势能的对角表象; Krylov 与 Chebyshev propagator 在高精度传播中尤其有效;
- 虚时间演化和 damped 梯度法适合求基态和自洽静态解; 分子动力学、Brown 运动、BUU 方程和 Navier–Stokes 方程则提供了从微观动力学到宏观输运的连续图景。

第十章 神经网络与机器学习

内容提要

- 经典统计、Bayes 统计与误差分析
- 深度学习的应⤵、AlphaGo 与 Data Science
- Linear Regression, Gaussian Process Regression
- 物理信息机器学习、其他机器学习方法与 Bayesian Optimization
- 自助法 (bootstrapping, 也常写作 bootstrapping)
- 神经网络: 前馈网络、BP 与 BNN

10.1 机器学习、统计与物理图像

10.1.1 从统计物理到神经网络

机器学习 (machine learning) 在计算物理中的角色, 不只是提供一个新的拟合工具, 而是提供了一种从大量观测、实验数据和模拟数据中建立可预测模型的科学范式。2024 年诺贝尔物理学奖授予 John J. Hopfield 与 Geoffrey E. Hinton, 主题正是利用人工神经网络实现机器学习的基础发现与发明。这一事实也说明, 现代机器学习与统计物理、非线性动力学、优化理论和信息论之间有深层联系。

Hopfield network 是一种循环神经网络 (recurrent neural network)。它可把图像或模式存储为能量函数的局域极小值; 当输入模式被噪声污染或只有部分信息时, 网络动力学会沿能量下降方向演化, 最终落入最接近的记忆吸引盆 (basin of attraction)。典型能量函数写为

$$E = -\frac{1}{2} \sum_{i,j} w_{ij} s_i s_j - \sum_i \theta_i s_i,$$

其中 $s_i = \pm 1$ 是第 i 个神经元状态, w_{ij} 是神经元间相互作用矩阵, θ_i 是阈值或外场。若权重矩阵对称且无自连接, 异步更新会使 E 单调不增, 因此网络会向某个局域极小值收敛。其物理意义与自旋玻璃模型相似: 记忆不是存放在某个单独节点中, 而是分布式地存放在相互作用矩阵 $W = (w_{ij})$ 中。Hopfield network 可用于联想记忆、数据恢复和 travelling salesman problem 等组合优化问题。

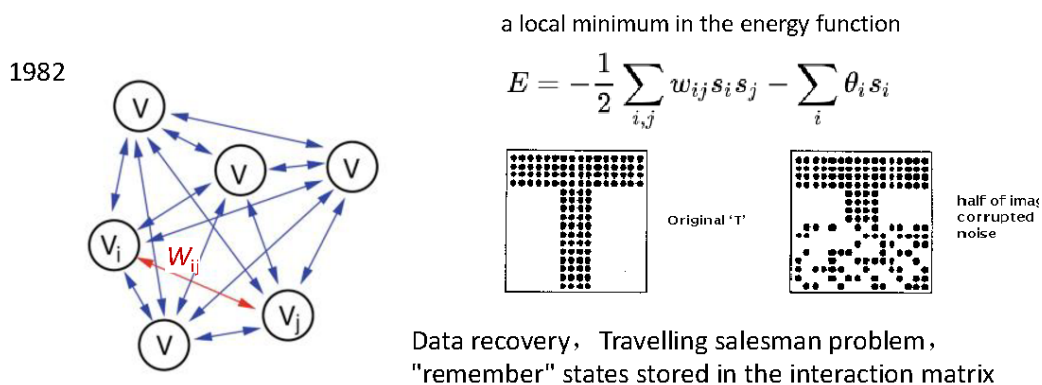


图 10.1: Hopfield network 的能量函数、联想记忆与局域极小值图像。模式恢复可理解为状态在能量景观中向吸引盆底部滚落。

Boltzmann machine 在 Hopfield network 的确定性能量下降基础上引入隐藏层 (hidden layer) 和概率分布。其能量可写成

$$E = -\left(\sum_{i<j} w_{ij} s_i s_j + \sum_i \theta_i s_i \right),$$

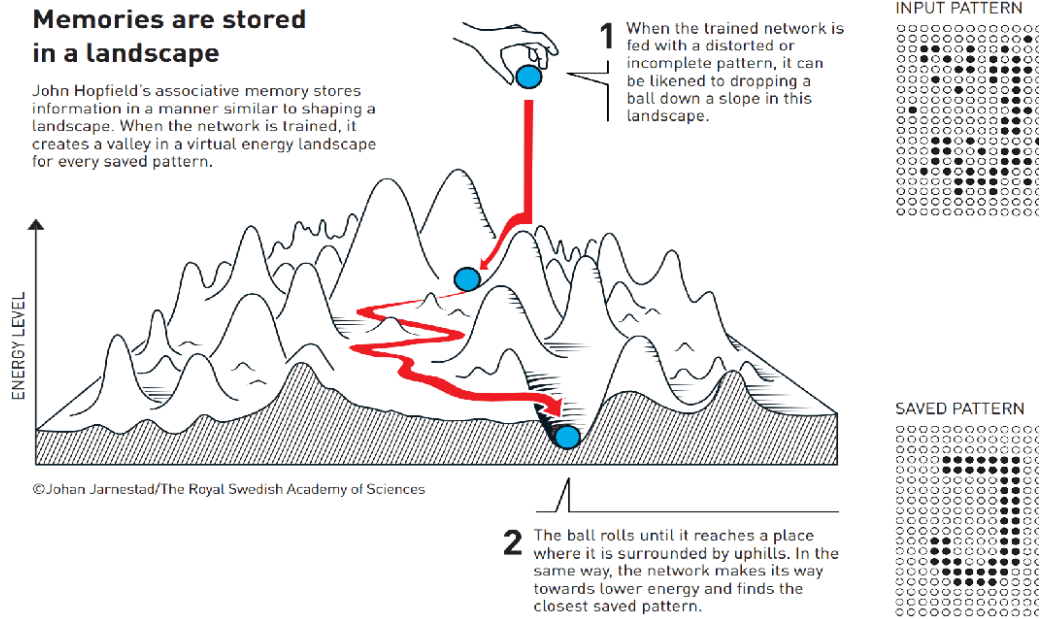


图 10.2: 记忆作为能量景观中的谷底：受噪声扰动的输入模式会沿能量下降路径收敛到最近的已存模式。

翻转第 i 个节点导致的能量变化为

$$\Delta E_i = \sum_{j>i} w_{ij}s_j + \sum_{j<i} w_{ji}s_j + \theta_i.$$

在温度 T 下，第 i 个节点取 1 的概率可写为

$$p_{i=\text{on}} = \frac{1}{1 + \exp(-\Delta E_i/k_B T)}.$$

这里 k_B 是 Boltzmann 常数。该式说明 Boltzmann machine 不是总选择能量最低方向，而是按 Boltzmann 权重进行随机采样，因此可以跳出局部极小值并学习数据分布。受限 Boltzmann 机 (restricted Boltzmann machine, RBM) 和深层 Boltzmann 机 (deep Boltzmann machine) 都是这一思想的扩展。

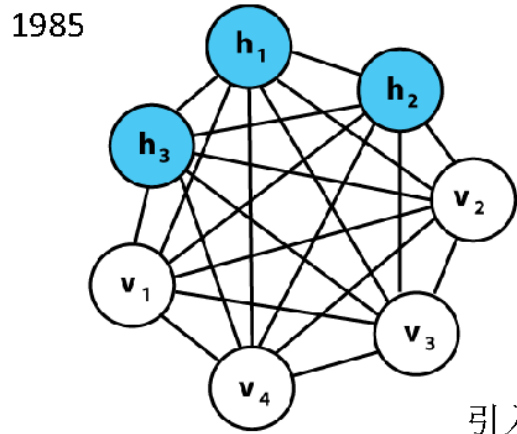


图 10.3: Boltzmann machine 的可见层、隐藏层与概率更新规则。隐藏变量使模型能够表达更复杂的数据相关性。

10.1.2 统计与神经网络问题

计算物理中的许多数据问题可以概括为：从观测中预测未知量。若已知物理方程，可以沿着

$$\text{input} \longrightarrow \text{physics equations} \longrightarrow \text{observables}$$

建立物理模型，再用实验反馈检验模型。若方程未知、过于复杂、代价太高或噪声显著，则常用

$$\text{model} + \text{random noise} \rightarrow \text{too many observations}$$

建立数据模型 (data model)。回归 (regression) 问题就是从输入到连续输出的映射学习；Bayesian inverse problem 则进一步要求从数据中同时进行推断 (inference) 与不确定性量化 (uncertainty quantification)。

这种观点覆盖了许多任务：

- forecasting 与 decision making；
- 从有限、含噪、缺失数据中学习；
- 学习复杂的个体化模型；
- data compression；
- 自动化科学建模、科学发现和实验设计。

机器学习与传统物理建模的不同，可从几个对照理解：

- 数据相关性 (data correlations) 对比物理方程 (physics equations)；
- 高维、复杂、不完美、含噪、异质数据对比理想化可控变量；
- 从数据中推断物理规律，对比从普适定律推出可观测量；
- 强表示能力 (powerful representation capability) 对比简洁普适规律 (universal laws)；
- 面向真实问题中的大数据、高自由度、过程耦合和多尺度结构。

因此，机器学习并不替代物理方程，而是在方程难以写出、难以求解或数据本身包含重要结构时提供新的科学范式。

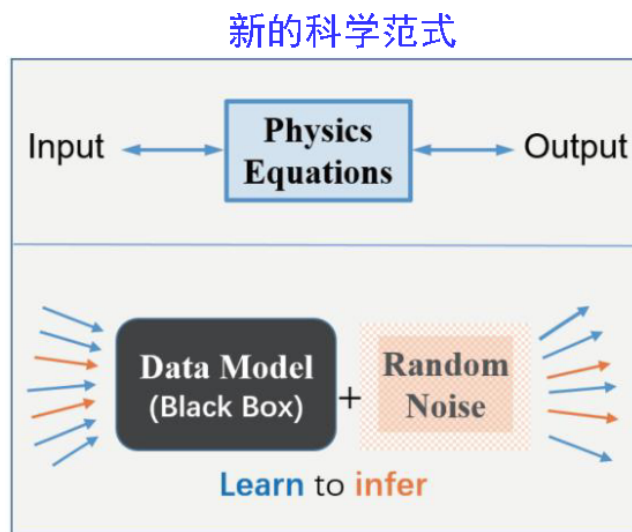


图 10.4: 物理方程范式与数据模型范式的比较。机器学习强调从数据相关性中学习映射，而物理模型强调从方程推导输出。

10.2 经典统计

10.2.1 均值、方差、标准差与相关系数

经典统计 (classical statistics) 首先关心样本对总体性质的估计。给定 N 个观测值 $x_0, \dots, x_{N-1}$ ，样本均值为

$$\bar{x} = \frac{1}{N} \sum_{j=0}^{N-1} x_j.$$

样本方差通常定义为

$$\text{Var}(x_0, \dots, x_{N-1}) = \frac{1}{N-1} \sum_{i=0}^{N-1} (x_i - \bar{x})^2,$$

其中分母 $N-1$ 是 Bessel 修正, 使其成为总体方差的无偏估计。标准差 (standard deviation) 为

$$\sigma = \sqrt{\text{Var}}.$$

若连续随机变量 X 的概率密度为 $f_X(x)$, 均值为 m_X , 则 n 阶中心矩 (central moment) 为

$$E[(X - m_X)^n] = \int_{-\infty}^{\infty} (x - m_X)^n f_X(x) dx.$$

特别地,

$$E[X^2] = \sigma_X^2 + m_X^2.$$

该式来自 $X = (X - m_X) + m_X$ 的平方展开, 用于把二阶矩与方差联系起来。

对两组数据 $\{x_i\}$ 与 $\{y_i\}$, 线性相关系数 (correlation coefficient) 定义为

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \sqrt{\sum_i (y_i - \bar{y})^2}}.$$

其取值满足 $-1 \leq r \leq 1$ 。 $r = 1$ 表示完全正线性相关, $r = 0$ 表示无线性相关, $r = -1$ 表示完全负线性相关。注意 $r = 0$ 不等价于统计独立, 只说明线性相关消失。

对二维正态分布, 指数中的二次型与相关性直接相关。例如若写成

$$p(x, y) dx dy = C \exp \left[-\frac{1}{2} (a_{00}x^2 + 2a_{01}xy + a_{11}y^2) \right] dx dy,$$

则相关系数满足

$$r = -\frac{a_{01}}{\sqrt{a_{00}a_{11}}}.$$

若二次型采用 $a_{00}x^2 - 2a_{01}xy + a_{11}y^2$ 的符号约定, 则上式符号相应改变。这个例子说明: 相关性可由协方差矩阵或其逆矩阵中的非对角项读出。

10.2.2 信息熵、相对熵与条件熵

信息熵 (information entropy) 度量概率分布的不确定性。离散分布 $p_0, \dots, p_{I-1}$ 的自然对数熵为

$$H(p) = -\sum_{i=0}^{I-1} p_i \ln p_i.$$

若用以 2 为底的对数,

$$H_2(p) = -\sum_i p_i \log_2 p_i,$$

则单位为 bit; 自然对数熵单位为 nat, 且

$$1 \text{ nat} = \frac{1}{\ln 2} \text{ bits} \approx 1.44 \text{ bits}.$$

Kullback-Leibler divergence (KL 散度) 或相对熵 (relative entropy) 定义为

$$D(p\|q) = \sum_i p_i \ln \frac{p_i}{q_i}.$$

连续情形写为

$$D_{\text{KL}}(P\|Q) = \int p(x) \log \frac{p(x)}{q(x)} dx.$$

它度量用 Q 近似真实分布 P 时的信息损失。KL 散度非负, 并且只有当 $p = q$ 几乎处处成立时等于零。最小化 $D_{\text{KL}}(P\|Q)$ 的计算物理意义是: 在给定模型族中寻找最接近目标分布的近似分布。

若 x 与 y 的联合概率为 $p_{ij} = P(x = i, y = j)$, 则

$$\sum_{i=0}^{I-1} \sum_{j=0}^{J-1} p_{ij} = 1.$$

边缘分布为 $p_i = \sum_j p_{ij}$ 、 $p_{\cdot j} = \sum_i p_{ij}$ 。熵分别为

$$H(x) = - \sum_i p_i \ln p_i, \quad H(y) = - \sum_j p_{\cdot j} \ln p_{\cdot j},$$

$$H(x, y) = - \sum_{i,j} p_{ij} \ln p_{ij}.$$

条件熵 (conditional entropy) 为

$$H(y | x) = - \sum_i p_i \sum_j \frac{p_{ij}}{p_i} \ln \frac{p_{ij}}{p_i} = - \sum_{i,j} p_{ij} \ln \frac{p_{ij}}{p_i}.$$

由联合概率 $p_{ij} = p_i P(y = j | x = i)$ 可得链式关系

$$H(x, y) = H(x) + H(y | x) = H(y) + H(x | y).$$

这说明联合不确定性可分解为一个变量的不确定性加上另一个变量在给定前者后的剩余不确定性。

10.2.3 协方差矩阵与误差传播

协方差 (covariance) 刻画两个随机变量共同涨落的方向和强度。对两组样本 x_A, x_B ,

$$\text{Cov}(x_A, x_B) = \frac{1}{N-1} \sum_{i=0}^{N-1} (x_{Ai} - \bar{x}_A)(x_{Bi} - \bar{x}_B).$$

归一化后得到关联函数 (correlation function)

$$R_{k,l} = \frac{\text{Cov}(x_k, x_l)}{\sqrt{\text{Var}(x_k) \text{Var}(x_l)}}.$$

联合方差满足

$$D(X \pm Y) = D(X) + D(Y) \pm 2 \text{Cov}(X, Y).$$

若 X, Y 正相关, 则 $X + Y$ 的方差被放大, $X - Y$ 的方差被部分抵消。

在模型参数拟合中, 常最小化

$$\chi^2(\mathbf{x}) = \sum_p \left(\frac{O_p^{(\text{th})}(\mathbf{x}) - O_p^{(\text{exp})}}{w_p} \right)^2,$$

其中 $\mathbf{x} = (x_1, \dots, x_n)$ 是模型参数, $O_p^{(\text{th})}$ 是理论预测, $O_p^{(\text{exp})}$ 是实验值, w_p 是误差权重。若由参数 $\mathbf{x}$ 推出另一个可观测量 $O(\mathbf{x})$, 一阶误差传播给出

$$\sigma_O^2 = \sum_{i,j} \text{Cov}(x_i, x_j) \left[\frac{\partial O}{\partial x_i} \frac{\partial O}{\partial x_j} \right].$$

这表明输出误差不仅取决于每个参数自己的方差, 还取决于参数之间的协方差。

在最小二乘近似下, 参数协方差矩阵可由灵敏度向量 $\mathbf{g}_i$ 估计:

$$\hat{V} \equiv \chi^2(\hat{\mathbf{x}}) \left(\sum_{i=1}^{n_d} \mathbf{g}_i \mathbf{g}_i^T \right)^{-1} \approx \text{Cov}(\hat{\mathbf{x}}).$$

其中 $\mathbf{g}_i$ 表示第 i 个观测量对参数的微分, n_d 是数据点数。矩阵逆的存在要求灵敏度向量张成足够的参数方向; 若某些参数组合对数据不敏感, 协方差会变大, 外推不确定性也会迅速放大。

若 X 是均值 m_X 、方差 σ_X^2 的随机变量, $Y = g(X)$, 则在 m_X 附近一阶展开:

$$Y = g(X) \approx g(m_X) + \left. \frac{dg}{dX} \right|_{m_X} (X - m_X).$$

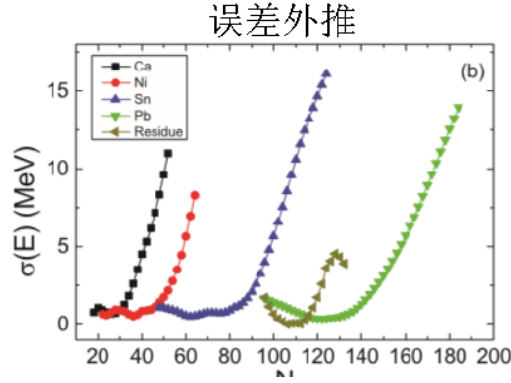


图 10.5: 外推区的误差常由参数相关性和模型灵敏度共同放大。

于是

$$m_Y = g(m_X), \quad \sigma_Y^2 = \left(\frac{dg}{dX} \Big|_{m_X} \right)^2 \sigma_X^2.$$

若 $\mathbf{X} = (X_1, \dots, X_n)$ 为随机向量,

$$g(\mathbf{X}) \approx g(\mathbf{m}_X) + \sum_{i=1}^n \frac{\partial g}{\partial X_i} \Big|_{\mathbf{X}=\mathbf{m}_X} (X_i - m_{X_i}).$$

令

$$b_i = \frac{\partial g}{\partial X_i} \Big|_{\mathbf{X}=\mathbf{m}_X},$$

则

$$m_Y = g(\mathbf{m}_X), \quad \sigma_Y^2 = \sum_{i=1}^n \sum_{j=1}^n b_i b_j \text{Cov}(X_i, X_j).$$

这是一阶误差传播公式的多变量形式，也是后续 Bayesian uncertainty quantification 的基础。

10.3 Bayes 统计

10.3.1 基本概率公式与 Bayes 定理

条件概率 (conditional probability) 定义为

$$P(A | B) = \frac{P(A, B)}{P(B)}, \quad P(B) > 0.$$

其中 $P(A, B) = P(A \cap B)$ 是事件 A 与 B 同时发生的概率。由定义可得

$$P(A \wedge B) = P(A | B)P(B) = P(B | A)P(A),$$

$$P(A \vee B) = P(A) + P(B) - P(A \wedge B).$$

若 $A_1, \dots, A_n$ 构成样本空间的一个划分，则全概率公式为

$$P(B) = \sum_{i=1}^n P(B | A_i)P(A_i).$$

对多个事件，有链式法则

$$P(AB | C) = P(A | C)P(B | AC),$$

$$P(A_4, A_3, A_2, A_1) = P(A_4 | A_3, A_2, A_1)P(A_3 | A_2, A_1)P(A_2 | A_1)P(A_1).$$

Bayes theorem 由联合概率的两个分解直接得到：

$$P(A | B) = P(A) \frac{P(B | A)}{P(B)}.$$

其中 $P(A)$ 是先验概率 (prior probability), $P(B | A)$ 是似然 (likelihood), $P(A | B)$ 是后验概率 (posterior probability), 而 $P(B)$ 是证据或归一化因子 (evidence)。商

$$\frac{P(B | A)}{P(B)}$$

可看成数据 B 对假设 A 的支持因子。对互斥完备假设 A_i , Bayes 公式写为

$$P(A_i | B) = \frac{P(B | A_i)P(A_i)}{\sum_j P(B | A_j)P(A_j)}.$$

 **笔记** 经典统计通常从重复抽样出发, 强调估计量在重复实验中的频率性质; Bayes 统计把未知参数也视为可赋予概率的对象, 强调在观测到数据后如何更新信念。Bayesian philosophy 起源于 18 世纪后期, 在 20 世纪后期因计算能力、MCMC 和复杂模型需求而重新兴盛。

10.3.2 一个医学检测例子

设某癌症在人群中的先验患病率为 $P(A) = 0.01$, 未患病概率为 $P(A^C) = 0.99$ 。检测阳性的灵敏度为 $P(+ | A) = 0.80$, 假阳性率为 $P(+ | A^C) = 0.096$ 。若某人检测阳性, 则真正患病概率为

$$\begin{aligned} P(A | +) &= \frac{P(+ | A)P(A)}{P(+ | A)P(A) + P(+ | A^C)P(A^C)} \\ &= \frac{0.01 \times 0.8}{0.01 \times 0.8 + 0.99 \times 0.096} = 0.0776. \end{aligned}$$

尽管检测灵敏度较高, 后验患病概率仍只有约 7.76%, 原因是先验患病率很低且假阳性率不可忽略。这个例子说明: 似然本身不能直接解释为后验概率, 必须与先验和归一化证据一起使用。

	Cancer	No Cancer
Prior	1%	99%
Test positive	80%	9.6%
Test negative	20%	90.4%

10.3.3 Bayesian 推断、预测与模型比较

对假设 h 和数据 D , Bayes 公式写成

$$P(h | D) = \frac{P(D | h)P(h)}{P(D)}.$$

这里 $P(h)$ 是观测数据前的 prior, $P(D | h)$ 是 hypothesis h 下产生数据 D 的 likelihood, $P(h | D)$ 是观测数据后的 posterior, $P(D)$ 是与具体假设无关的 evidence。Bayesian inference 的本质是: 观测数据把 prior 更新为 posterior; posterior 表示在给定数据条件下对假设的置信程度。

对参数模型 m 与参数 θ ,

$$P(\theta | \mathcal{D}, m) = \frac{P(\mathcal{D} | \theta, m)P(\theta | m)}{P(\mathcal{D} | m)}.$$

其中 $P(\mathcal{D} | \theta, m)$ 是 likelihood, $P(\theta | m)$ 是参数先验, $P(\theta | \mathcal{D}, m)$ 是参数后验。给定训练数据后的预测分布为

$$P(x | \mathcal{D}, m) = \int P(x | \theta, \mathcal{D}, m)P(\theta | \mathcal{D}, m) d\theta.$$

这一步把参数不确定性积分掉, 因此比只代入最优参数更能反映预测误差。

模型比较 (model comparison) 使用

$$P(m | \mathcal{D}) = \frac{P(\mathcal{D} | m)P(m)}{P(\mathcal{D})},$$

其中

$$P(\mathcal{D} | m) = \int P(\mathcal{D} | \theta, m) P(\theta | m) d\theta$$

称为模型证据 (model evidence) 或边缘似然 (marginal likelihood)。它不是只看最优参数处拟合得多好, 而是考察从先验中随机抽取参数时产生该数据的总体概率, 因此天然包含对模型复杂度的惩罚。

10.4 线性回归、高斯过程与自助法

10.4.1 线性回归与最小二乘

给定数据集

$$\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\},$$

回归函数可定义为条件期望

$$r(x) := E(Y | X = x) = \int y p(y | x) dy.$$

一元线性回归假设

$$Y = \beta_0 + \beta_1 x + \epsilon, \quad E(\epsilon | x) = 0, \quad V(\epsilon | x) = \sigma^2.$$

其中 β_0 是截距, β_1 是斜率, ϵ 是随机误差。预测值和残差为

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i, \quad \hat{\epsilon}_i = y_i - \hat{y}_i.$$

最小二乘法 (least squares) 选择参数使残差平方和最小:

$$S(\hat{\beta}_0, \hat{\beta}_1) = \sum_i \hat{\epsilon}_i^2 = \sum_i [y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)]^2.$$

令两个偏导数为零:

$$\frac{\partial S}{\partial \hat{\beta}_0} = -2 \sum_i (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0,$$

$$\frac{\partial S}{\partial \hat{\beta}_1} = -2 \sum_i x_i (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i) = 0.$$

由第一式得 $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$, 代入第二式得

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

误差方差估计为

$$\hat{\sigma}^2 = \frac{1}{n-2} \sum_i \hat{\epsilon}_i^2.$$

分母 $n-2$ 反映两个参数 β_0, β_1 已由数据估计。线性回归不仅是拟合直线, 还包含参数误差、预测区间和置信区间分析, 是机器学习中最基础的统计模型。

若进一步假设

$$\epsilon | x \sim N(0, \sigma^2),$$

则

$$Y \sim N(\beta_0 + \beta_1 X, \sigma^2).$$

给定参数 $\theta = (\beta_0, \beta_1, \sigma^2)$ 的 likelihood 为

$$L_{\mathcal{D}}(\theta) = \prod_{i=1}^n \hat{p}(x_i, y_i | \theta) = \prod_i \hat{p}(y_i | x_i) \prod_i p(x_i).$$

若 x_i 被视为固定设计点，则 $\prod_i p(x_i)$ 与参数无关，最大化 likelihood 等价于最大化

$$\prod_i \hat{p}(y_i | x_i) = \prod_i \frac{1}{\sqrt{2\pi}\hat{\sigma}} \exp \left[-\frac{(y_i - \hat{y}_i)^2}{2\hat{\sigma}^2} \right].$$

即

$$\prod_i \hat{p}(y_i | x_i) = \frac{1}{(\sqrt{2\pi})^n \hat{\sigma}^n} \exp \left[-\frac{1}{2\hat{\sigma}^2} \sum_i (y_i - \hat{y}_i)^2 \right].$$

因此在 Gaussian noise 条件下，最大似然估计 (maximum likelihood estimation, MLE) 对 β_0, β_1 的一阶条件与最小二乘完全一致。

10.4.2 多元线性回归、Bayesian linear regression 与 SVD

多元线性回归写为

$$Y = \beta_0 + \sum_{i=1}^p \beta_i X_i + \epsilon = \langle \beta, X \rangle + \epsilon,$$

其中

$$\beta = (\beta_0, \beta_1, \dots, \beta_p)^T, \quad X = (1, X_1, \dots, X_p)^T.$$

对 n 个数据点，设计矩阵 X 的维数为 $n \times (1 + p)$ ，响应向量为 Y 。最小化

$$\|Y - X\beta\|_2^2$$

的正规方程为

$$X^T X \hat{\beta} = X^T Y.$$

若 $X^T X$ 可逆，

$$\hat{\beta} = (X^T X)^{-1} X^T Y.$$

误差方差和参数方差为

$$\hat{\sigma}^2 = \frac{1}{n - p} \sum_{i=1}^n \hat{\epsilon}_i^2, \quad V(\hat{\beta}) = (X^T X)^{-1} \sigma^2.$$

若 $\epsilon \sim N(0, \sigma^2)$ ，则

$$\hat{\beta} \sim N(\beta, (X^T X)^{-1} \sigma^2).$$

Bayesian linear regression 把 β 视为随机变量并指定 prior，例如 $\beta \sim N(\beta_0, \Lambda_0^{-1})$ 。观测数据后得到 posterior，而预测时对 β 积分。与普通线性回归相比，它能自然给出参数不确定性和预测不确定性。

多项式回归 (polynomial regression) 仍然是线性回归的一种：若基函数取 $1, x, x^2, \dots$ ，未知参数仍线性进入模型。实际求解正规方程时，直接求 $(X^T X)^{-1}$ 可能病态；奇异值分解 (singular value decomposition, SVD) 可更稳定地处理近奇异或多重共线性问题。

10.4.3 Gaussian Process Regression

Gaussian process regression (GPR) 把函数本身视为随机对象。若

$$f(x) \sim GP(m(x), k(x, x')),$$

则任意有限个函数值服从多元 Gaussian distribution。这里

$$m(x) = E[f(x)],$$

$$k(x, x') = E[(f(x) - m(x))(f(x') - m(x'))]$$

分别是均值函数（mean function）与核函数（kernel function）。常用平方指数核（squared exponential kernel）为

$$k(x, x') = \sigma_f^2 \exp \left[-\frac{(x - x')^2}{2l^2} \right],$$

其中 l 是相关长度， σ_f 是函数值涨落尺度。若观测含 Gaussian noise,

$$y = f(x) + N(0, \sigma_n^2),$$

可把噪声加入核函数：

$$k(x, x') = \sigma_f^2 \exp \left[-\frac{(x - x')^2}{2l^2} \right] + \sigma_n^2 \delta(x, x').$$

设训练点协方差矩阵为 K ，预测点 x_* 与训练点的协方差为

$$K_* = [k(x_*, x_1), \dots, k(x_*, x_n)],$$

预测点自身方差为 $K_{**} = k(x_*, x_*)$ 。零均值情况下，

$$\begin{bmatrix} y \\ y_* \end{bmatrix} \sim N \left(0, \begin{bmatrix} K & K_*^T \\ K_* & K_{**} \end{bmatrix} \right).$$

由多元正态分布的条件分布公式，得到

$$y_* | y \sim N \left(K_* K^{-1} y, K_{**} - K_* K^{-1} K_*^T \right).$$

因此预测均值和预测方差为

$$\bar{y}_* = K_* K^{-1} y, \quad \text{var}(y_*) = K_{**} - K_* K^{-1} K_*^T.$$

第一项给出插值或外推曲线，第二项给出不确定性带。GPR 的重要优点是不需要显式假设 $f(x)$ 的参数化形式，只需要选择核函数。

核函数超参数

$$\theta = \{l, \sigma_f, \sigma_n\}$$

可由对数边缘似然的 maximum a posteriori 估计得到：

$$\log p(y | x, \theta) = -\frac{1}{2} y^T K^{-1} y - \frac{1}{2} \log |K| - \frac{n}{2} \log 2\pi.$$

其中第一项奖励拟合，第二项惩罚过于复杂或过于灵活的协方差结构，第三项是归一化常数。一个典型例子中可得到 $l = 1$ 、 $\sigma_f = 1.27$ 、 $\sigma_n = 0.3$ 。

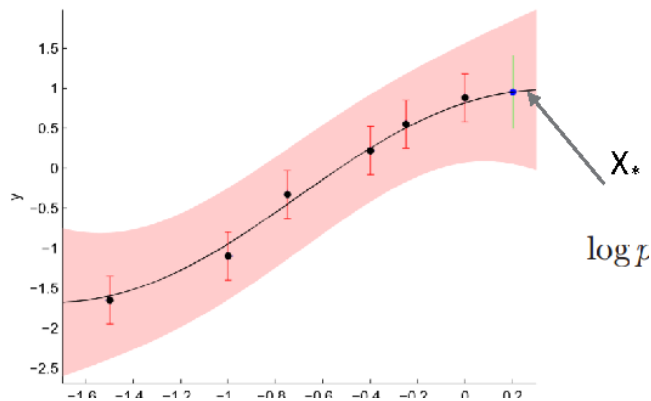


图 10.6: Gaussian process regression 的条件 Gaussian 公式、预测均值、预测方差和超参数优化。

例题 10.1 Gaussian process 的数值预测 取训练输入

$$x = [-1.50, -1.00, -0.75, -0.40, -0.25, 0.00],$$

预测点 $x_* = 0.2$ 。在 $\sigma_n = 0.3$ 的核函数下，协方差矩阵可写为

$$K = \begin{pmatrix} 1.70 & 1.42 & 1.21 & 0.87 & 0.72 & 0.51 \\ 1.42 & 1.70 & 1.56 & 1.34 & 1.21 & 0.97 \\ 1.21 & 1.56 & 1.70 & 1.51 & 1.42 & 1.21 \\ 0.87 & 1.34 & 1.51 & 1.70 & 1.59 & 1.48 \\ 0.72 & 1.21 & 1.42 & 1.59 & 1.70 & 1.56 \\ 0.51 & 0.97 & 1.21 & 1.48 & 1.56 & 1.70 \end{pmatrix}.$$

这里对角元为 $\sigma_f^2 + \sigma_n^2$ 。预测协方差为

$$K_* = [0.31, 0.68, 0.92, 1.25, 1.38, 1.54], \quad K_{**} = 1.70.$$

代入

$$\bar{y}_* = K_* K^{-1} y, \quad \text{var}(y_*) = K_{**} - K_* K^{-1} K_*^T,$$

得到

$$\bar{y}_* = 0.95, \quad \text{var}(y_*) = 0.21.$$

这说明 GPR 的输出不是一个单点，而是带有方差的预测分布。

例题 10.2 线性拟合与多项式拟合 给定四个数据点

$$(x_i, y_i) = (0.2, 0.2), (0.3, 0.35), (0.4, 0.55), (0.5, 0.63).$$

若拟合 $y = ax + b$ ，最小二乘解为

$$a = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} \simeq 1.49, \quad b = \bar{y} - a\bar{x} \simeq -0.089,$$

并且

$$\sigma^2 = \frac{1}{n-2} \sum_i (y_i - \hat{y}_i)^2 \simeq 0.89 \times 10^{-4}.$$

若拟合 $y = ax^2 + b$ ，把 $X = (a, b)^T$ 写入二次型

$$g(X) = \frac{1}{2} X^T A X - d^T X.$$

对 $n = 4$ 个数据点，

$$A = 2 \begin{pmatrix} \sum_i x_i^4 & \sum_i x_i^2 \\ \sum_i x_i^2 & 4 \end{pmatrix}, \quad d = 2 \begin{pmatrix} \sum_i x_i^2 y_i \\ \sum_i y_i \end{pmatrix}.$$

最小化 $g(X)$ 等价于解线性方程 $AX = d$ ，可用共轭梯度法：

$$\begin{aligned} r_0 &= d - AX_0, & v_1 &= r_0, \\ t_1 &= \frac{\langle r_0, r_0 \rangle}{\langle v_1, Av_1 \rangle}, & X_1 &= X_0 + t_1 v_1, \\ r_1 &= r_0 - t_1 A v_1, & s_1 &= \frac{\langle r_1, r_1 \rangle}{\langle r_0, r_0 \rangle}, & v_2 &= r_1 + s_1 v_1, \\ t_2 &= \frac{\langle r_1, r_1 \rangle}{\langle v_2, Av_2 \rangle}, & X_2 &= X_1 + t_2 v_2. \end{aligned}$$

从 $X_0 = (1, 1)^T$ 出发，可得 $X_2 = (2.06, 0.15)^T$ ，即 $a \simeq 2.06$ 、 $b \simeq 0.15$ ，误差方差约为

$$\sigma^2 = \frac{1}{n-2} \sum_i (y_i - \hat{y}_i)^2 \simeq 3.6 \times 10^{-3}.$$

在 Gaussian noise 假设下，最大似然与最小残差平方和等价，因此该例也说明了 MLE 与最小二乘的联系。

10.4.4 自助法 bootstrapping

自助法 (bootstrapping) 是一种从有限样本估计统计量不确定性的重采样方法。它不需要显式知道总体分布，而是把经验分布

$$\hat{F}_N(x) = \frac{1}{N} \sum_{i=1}^N I(x_i \leq x)$$

作为总体分布的近似。给定样本 $\mathcal{D} = \{x_1, \dots, x_N\}$ ，bootstrap 的基本步骤是：

1. 从 $\mathcal{D}$ 中有放回地抽取 N 个样本，得到一个 bootstrap sample $\mathcal{D}^{*(b)}$ ；
2. 对该样本计算统计量 $T^{*(b)} = T(\mathcal{D}^{*(b)})$ ；
3. 重复 B 次，得到 $T^{*(1)}, \dots, T^{*(B)}$ ；
4. 用这些重复值的标准差估计 T 的标准误，用分位数估计置信区间。

其标准误估计为

$$\widehat{\text{se}}_{\text{boot}}(T) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (T^{*(b)} - \bar{T}^*)^2}, \quad \bar{T}^* = \frac{1}{B} \sum_{b=1}^B T^{*(b)}.$$

Bootstrap 适合复杂统计量、解析误差传播困难和样本量中等的问题；缺点是对样本代表性敏感，且对强相关数据、极端尾部分布和严重小样本问题可能偏差较大。对时间序列或空间相关数据，需要使用 block bootstrap 等保留相关结构的变体。

10.5 AI for Science 与 Machine Learning for Physics

10.5.1 从大数据到科学信息

AI for science 的核心动机是：数字社会 and 信息技术每天产生大量数据，而科学问题常具有复杂、高代价、病态反问题 and 不确定性传播等特征。机器学习可用于 classification and regression、surrogates、inverse problems、design and optimization、control systems 等任务。一句常见判断是：AI 不会简单替代科学家，但会使用 AI 的科学家将替代不会使用 AI 的科学家。其实际含义是，AI 正在成为科学计算流程中的基础工具。

在物理学中，机器学习发展迅速。以 Phys. Rev. 系列中 neural network 相关论文为例，可见下列数量级：

期刊或时间段	数量
Phys. Rev. Lett.	225–2680
Phys. Rev. A	119–2490
Phys. Rev. B	91–5040
Phys. Rev. C	19–991
Phys. Rev. D	110–3696
Phys. Rev. E	608–2037
1980–1990	74
1990–2000	265
2000–2010	329
2010–2020	674

2000 年以后，Phys. Rev. 中 machine learning papers 的数量快速增长，2016 年 AlphaGo 成为标志性事件之一。对计算物理而言，这种增长意味着数据驱动建模、模型降阶、代理模型、误差量化和自动化发现正在成为常规工具。

- AI indeed is useful and will make a great impact

Huge data is being produced everyday(digital society& info tech); Alphago as a milestone (2016)

AI won't replace the scientist, but scientists who use AI will replace those who don't.

- From Big data to information, complex and costly problems, ill-inversed problems, uncertainty quantification

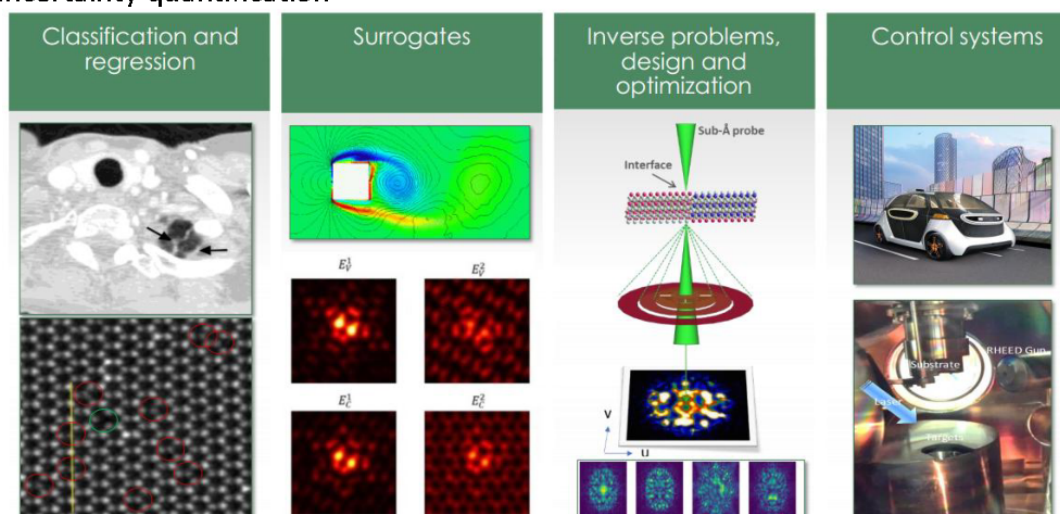


图 10.7: AI for science 的典型任务：分类与回归、替代模型、反问题与优化、控制系统。

10.6 人工神经网络

10.6.1 基本思想与学习类型

人工神经网络 (artificial neural network, ANN) 是机器学习的一类重要方法。机器学习本身是人工智能 (artificial intelligence, AI) 的一个方向，而神经网络的启发来自生物神经系统中的学习和神经元连接。神经网络的主要能力包括：

- 非线性映射 (nonlinear mapping)；
- association and storage；
- pattern recognition and classification；
- optimization。

前馈神经网络 (feed-forward neural network) 没有反馈过程，信息从输入层经隐藏层逐层传播到输出层，适合构造简单而通用的非线性映射。Back-propagation (BP) 可理解为以最小二乘误差为目标的梯度学习算法。生成对抗网络 (generative adversarial network, GAN) 通过生成器与判别器的博弈实现无监督生成；迁移学习 (transfer learning) 把已有任务上训练好的模型迁移到新任务。

从学习范式看：

- 监督学习 (supervised learning) 利用输入与期望输出的差异调整参数；
- 无监督学习 (unsupervised learning) 从数据自身结构中学习表示；
- 强化学习 (reinforcement learning) 通过奖励信号优化策略；
- 自组织学习 (self-organizing learning) 强调系统在局域规则下形成全局结构。

10.6.2 生物神经元与人工神经元

生物神经元 (biological neuron) 由细胞本体 (soma)、树突 (dendrites)、轴突丘 (hillock)、轴突 (axon) 和突触 (synapse) 组成。树突接收信息，轴突长距离传输信息，突触负责神经元之间的信号传递。人脑大约含有 86 billion neurons, 以及超过 100 trillion、某些估计达 1000 trillion 的突触连接。神经元可异步并行 firing, 网络具有 small-world 结构: 少量高度连接的 hubs 与大量低连接节点共存, 度分布部分满足 power-law。

一些生物神经元平均每秒可发放约 200 次, 神经冲动速度依类型从 0.61 m/s 到 119 m/s 不等。生物神经网络具有容错性 (fault tolerance): 信息冗余存储使局部损伤不一定导致记忆完全丢失。人脑消耗约人体总能量的 20%, 成年人脑功率约 20 W, 而一块 GPU 可能达到 250 W, 这体现了生物计算的高能效。神经可塑性 (neuroplasticity) 使神经纤维能形成新连接, 并使功能区移动或改变。

人工神经元 (artificial neuron) 把输入 $x_1, \dots, x_n$ 与权重 $w_1, \dots, w_n$ 相乘求和, 再减去阈值 θ :

$$v = w^T x - \theta.$$

输出为

$$y = f(v),$$

其中 f 是激活函数 (activation function)。常见 sigmoid 函数为

$$S(x) = \frac{1}{1 + e^{-x}},$$

双曲正切函数为

$$\tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Sigmoid 容易出现梯度饱和、收敛缓慢; tanh 以零为中心, 实际训练效果常更好。无论选择哪种激活函数, 非线性都是神经网络能够表示复杂函数的关键。

10.6.3 机器学习作为非线性优化

机器学习主要关注对数据的使用。监督学习通常分为 regression 和 classification: 前者输出连续变量, 后者输出离散类别。典型应用包括数据挖掘、计算机视觉、自然语言处理、生物特征识别、搜索引擎、医学诊断、信用卡欺诈检测、证券市场分析、DNA 序列测序、语音和手写识别、战略游戏和机器人。

训练过程可以概括为: 比较网络输出与期望值, 调整权重因子 w_{ij} 。这本质上是一个有众多参数的非线性优化问题。常用优化方法包括:

- 梯度下降法 (gradient descent);
- quasi-Newton-BFGS;
- 模拟退火法 (simulated annealing);
- 随机学习方法 (stochastic learning method)。

因此, 神经网络训练与前面章节中的优化算法直接相连。

10.6.4 BP 神经网络与误差反传

BP (backward propagation of errors) 算法的核心是链式法则。设节点 j 的输入为

$$x_j = \sum_{k \in K_j} w_{kj} y_k,$$

输出为

$$y_j = f(x_j).$$

对输出层, 目标值为 t_j , 平方误差为

$$E = \frac{1}{2} \sum_{j=1}^J (t_j - y_j)^2.$$

权重更新沿负梯度方向：

$$\Delta w_{kj} = -\alpha \frac{\partial E}{\partial w_{kj}},$$

其中 $\alpha \in (0, 1)$ 是学习率。链式法则给出

$$\frac{\partial E}{\partial w_{kj}} = \frac{\partial E}{\partial y_j} \frac{\partial y_j}{\partial x_j} \frac{\partial x_j}{\partial w_{kj}}.$$

定义误差信号

$$\delta_j := -\frac{\partial E}{\partial y_j} \frac{\partial y_j}{\partial x_j}, \quad \frac{\partial x_j}{\partial w_{kj}} = y_k.$$

若 f 为 sigmoid, 则

$$\frac{\partial y_j}{\partial x_j} = y_j(1 - y_j).$$

对输出层,

$$\frac{\partial E}{\partial y_j} = -(t_j - y_j),$$

于是

$$\frac{\partial E}{\partial w_{kj}} = -(t_j - y_j)y_j(1 - y_j)y_k.$$

因此输出层误差信号为

$$\delta_j = (t_j - y_j)y_j(1 - y_j).$$

对隐藏层节点 j , 误差通过所有上层节点 $i \in I_j$ 反传:

$$\frac{\partial E}{\partial y_j} = \sum_{i \in I_j} \frac{\partial E}{\partial y_i} \frac{\partial y_i}{\partial x_i} \frac{\partial x_i}{\partial y_j}.$$

由于

$$\frac{\partial x_i}{\partial y_j} = w_{ji},$$

并利用上层误差信号 δ_i , 得到

$$\frac{\partial E}{\partial y_j} = -\sum_{i \in I_j} \delta_i w_{ji}.$$

所以隐藏层误差信号为

$$\delta_j = \left(\sum_{i \in I_j} \delta_i w_{ji} \right) y_j(1 - y_j).$$

统一地, 权重更新可写为

$$\Delta w_{kj}(n) = \alpha \delta_j y_k + \eta \Delta w_{kj}(n-1),$$

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}.$$

其中 η 是 momentum 参数, 用于减小震荡并加速沿稳定方向的收敛。



笔记 误差反向传播学习法可应用于多种神经网络, 例如 Bayesian BP 与 CNN。标准 BP 神经网络存在局部极小、收敛慢和过度拟合 (overfitting) 等问题, 因此常结合正则化、动量、自适应学习率、批归一化或 Bayesian 方法改进。

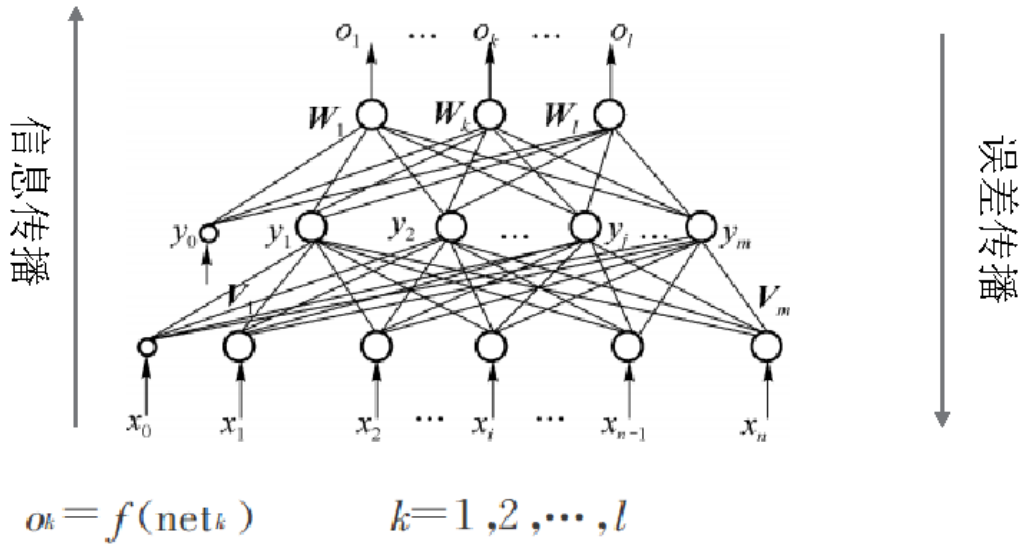


图 10.8: BP 神经网络中信息向前传播、误差向后传播。所有权重梯度都由链式法则逐层计算。

10.7 Bayesian 神经网络

10.7.1 权重作为概率分布

标准神经网络把权重看作固定参数，通过学习调整权重使误差最小。Bayesian neural network (BNN) 把权重 ω 看作随机变量，用 posterior 描述训练后的不确定性：

$$p(\omega | D) = \frac{p(D | \omega)p(\omega)}{p(D)} \propto p(D | \omega)p(\omega).$$

这里 $p(\omega)$ 是权重先验， $p(D | \omega)$ 是 likelihood， $p(\omega | D)$ 是 posterior。学习目标不再只是找到单个最优权重，而是估计权重分布。其优点是可同时刻画 parameter uncertainty、structure uncertainty 和 prediction uncertainty。

常见模型写为

$$y = f(x, \omega) + \sigma \epsilon,$$

训练数据为

$$D = \{(x_1, t_1), (x_2, t_2), \dots, (x_N, t_N)\},$$

其中 x 是输入， t 是输出。若误差近似 Gaussian，则

$$p(D | \omega) = \exp(-\chi^2/2),$$

$$\chi^2 = \sum_{n=1}^N \left(\frac{t_n - S(x_n; \omega)}{\Delta t_n} \right)^2.$$

在核物理质量残差问题中，可取

$$t_n = M^{\text{exp}}(x_n) - M^{\text{th}}(x_n).$$

一个单隐藏层网络函数为

$$S(x; \omega) = a + \sum_{j=1}^H b_j \tanh \left(c_j + \sum_{i=1}^I d_{ji} x_i \right).$$

其中 H 是 hidden neurons 数目， I 是输入维数， $\omega = \{a, b_j, c_j, d_{ji}\}$ 是需要学习的参数集合，参数总数为

$$1 + (2 + I)H.$$

若网络有 L 层、每层宽度约为 H ，输入维数为 $|x|$ 、输出维数为 $|y|$ ，全连接网络参数量随层数和宽度快速增

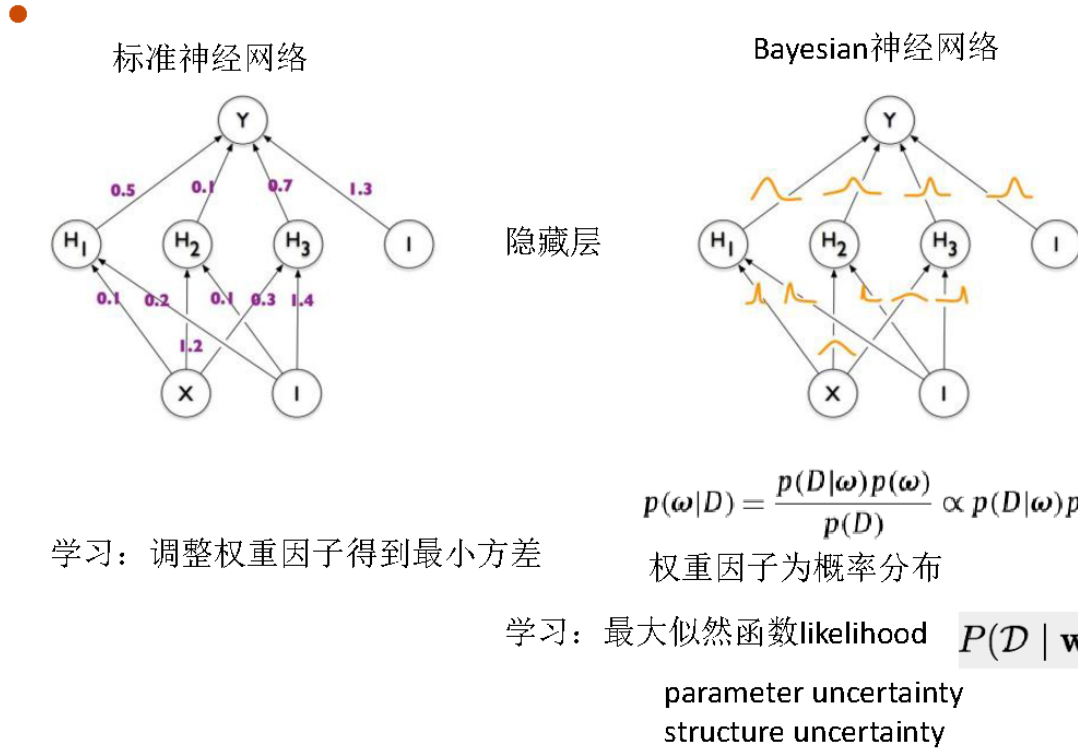


图 10.9: 标准神经网络与 Bayesian 神经网络的区别。BNN 中权重因子由概率分布描述，学习结果包含不确定性。

长，可按连接数近似估计为

$$N_{\omega} \sim (1 + |x|)H + (L - 1)H(H + 1) + (H + 1)|y|.$$

这解释了为何 BNN 完全 Bayesian 处理计算量很大。

10.7.2 Bayesian 预测与 MCMC

BNN 的预测均值为

$$\langle S \rangle = \int S(x; \omega) p(\omega | D) d\omega,$$

预测标准差为

$$\Delta S = \sqrt{\langle S^2 \rangle - \langle S \rangle^2}.$$

这两个量需要对 posterior 积分。MCMC 构造 Markov chain

$$W_{(t)} \rightarrow W_{(t+1)} \rightarrow \dots$$

使其平稳分布收敛到 $p(w | D)$ 。对测试输入 x^* 的 full Bayesian prediction 可写为

$$p(t^* | x^*, D) = \iiint p(t^* | x^*, w, \beta) p(w | D, \alpha, \beta) p(\alpha, \beta | D) dw d\alpha d\beta.$$

其中 β 常表示输出噪声精度， α 表示权重先验精度。三重积分分别考虑观测噪声、权重 posterior 和超参数 posterior，因此计算代价高，但能给出完整 uncertainty quantification。

10.7.3 共轭先验与 Bayesian learning

一个最简单的共轭先验例子是

$$y = \theta + \epsilon, \quad y | \theta \sim N(\theta, \sigma^2),$$

先验取

$$\theta \sim N(\mu, \delta^2).$$

给定观测 $y_1, \dots, y_n$, posterior 满足

$$\pi(\theta | y) \propto \exp\left[-\frac{1}{2\delta^2}(\theta - \mu)^2\right] \exp\left[-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \theta)^2\right].$$

把指数中的 θ 二次项配方, 得到

$$\begin{aligned} \theta | y &\sim N(\nu, \gamma^2), \\ \nu &= \frac{\frac{n}{\sigma^2} \bar{y} + \frac{1}{\delta^2} \mu}{\frac{n}{\sigma^2} + \frac{1}{\delta^2}}, \quad \gamma^2 = \left(\frac{n}{\sigma^2} + \frac{1}{\delta^2}\right)^{-1}. \end{aligned}$$

当 n 增大时, γ^2 变小, posterior 逐渐变窄并向数据均值集中。所谓共轭先验 (conjugate prior), 是指 prior 与 likelihood 结合后 posterior 仍属于同一分布族, 从而解析计算方便。

Bayesian learning 中常比较三种估计:

$$\begin{aligned} \theta_{\text{MLE}} &= \arg \max_{\theta} P(X | \theta) = \arg \max_{\theta} \sum_i \log P(x_i | \theta), \\ \theta_{\text{MAP}} &= \arg \max_{\theta} P(X | \theta) P(\theta) = \arg \max_{\theta} \left[\sum_i \log P(x_i | \theta) + \log P(\theta) \right]. \end{aligned}$$

若 prior 为 uniform distribution, 则 MAP 与 MLE 相同; 若 prior 非均匀, MAP 会把估计拉向 prior 支持的区域。MLE 给出点估计, Bayesian estimation 则计算 full posterior。

10.7.4 Stochastic Gradient Langevin Dynamics

Stochastic Gradient Langevin Dynamics (SGLD) 把梯度上升与 Langevin 噪声结合, 用于从 posterior 采样。给定初值 w^0 , 迭代为

$$w^{k+1} = w^k + \frac{\epsilon}{2} \nabla \log p(w^k | D) + \eta^k,$$

其中 ϵ 是 learning rate, $\eta^k \sim N(0, \epsilon)$ 是 Gaussian noise。小批量数据下可近似为

$$w^{k+1} = w^k + \frac{\epsilon}{2} \nabla \left[\log p(w^k) + \frac{N}{n} \sum_{i=1}^n \log p(y_i | x_i, w^k) \right] + \eta^k.$$

这里 N 是总数据量, n 是 mini-batch 大小。与普通随机梯度下降相比, SGLD 的噪声项使迭代不只收敛到一个最优点, 而是在 posterior 附近采样, 从而捕捉参数不确定性。若只做极大似然, 则求解

$$\frac{\partial}{\partial \theta} \log p(X | \theta) = 0.$$

10.7.5 正则化的 Bayesian 解释

神经网络中的 L^2 regularization 可由 Gaussian prior 得到。设 likelihood 写为

$$p(D | w) = \frac{\exp(-\beta E_D)}{Z_D(\beta)},$$

权重 prior 为

$$p(w) = \frac{\exp(-\alpha E_w)}{Z_w(\alpha)}.$$

对于平方误差,

$$p(D | w) = \prod_{n=1}^N p(t^n | x^n, w) = \frac{1}{Z_D(\beta)} \exp\left[-\frac{\beta}{2} \sum_{n=1}^N \{y(x^n; w) - t^n\}^2\right].$$

posterior 为

$$p(w | D) = \frac{p(D | w)p(w)}{P(D)} = \frac{1}{Z_S} \exp(-\beta E_D) \exp(-\alpha E_w) = \frac{1}{Z_S} \exp[-S(w)].$$

其中

$$S(w) = \beta E_D + \alpha E_w.$$

取对数极大化 posterior 等价于最小化

$$S(w) = \frac{\beta}{2} \sum_{n=1}^N \{y(x^n; w) - t^n\}^2 + \frac{\alpha}{2} \sum_{i=1}^W w_i^2.$$

第二项惩罚大权重, α, β 是 hyperparameters. Bayesian 观点下, 正则化不是人为技巧, 而是 prior 对模型复杂度的约束。

10.7.6 Occam's Razor 与模型证据

Occam's Razor 强调在同样解释数据时优先选择较简单的模型。在 Bayesian 模型比较中, 复杂度惩罚由 evidence 自动给出:

$$p(D) = \int p(D | w)p(w) dw.$$

若 posterior 集中在 w_{MAP} 附近, 可近似为

$$p(D) \approx p(D | w_{\text{MAP}}) \frac{\Delta w_{\text{posterior}}}{\Delta w_{\text{prior}}}.$$

取对数得

$$\ln p(D) \approx \ln p(D | w_{\text{MAP}}) + \ln \frac{\Delta w_{\text{posterior}}}{\Delta w_{\text{prior}}}.$$

若有 M 个参数,

$$\ln p(D) \approx \ln p(D | w_{\text{MAP}}) + M \ln \frac{\Delta w_{\text{posterior}}}{\Delta w_{\text{prior}}}.$$

第一项随模型复杂度增加通常会变大, 因为更复杂模型更容易拟合数据; 第二项为负, 且随参数数目 M 增加而更强。这种 balance 解释了为什么中等复杂度模型可能具有最大 evidence。

对给定观测数据 D_0 , 简单模型 M_1 可能无法拟合数据, 复杂模型 M_3 虽然能覆盖很多可能数据, 但把预测概率分散到过大空间, 对任一特定数据集的概率反而较小。中等复杂度模型 M_2 若既能解释数据又不过度分散概率, 就具有最大 evidence。选择 prior 时必须谨慎, 因为 evidence 对 prior 体积敏感。

10.7.7 物理应用中的 BNN、GPR 与置信区间

在原子核质量外推问题中, Gaussian process regression 只需少数超参数, 常能获得很好的外推表现; BNN 由于参数很多, 若数据量不足或 prior 选择不当, 可能不稳定。这个对比说明, 模型表达能力越强, 不代表实际预测一定更可靠。

裂变产额 (fission yield) 是非常关键的裂变观测量, 并决定许多裂变后观测量。主要核数据库 ENDF、JENDL、JEFF、CENDL 中, 裂变数据常只在 thermal、0.5 MeV、14 MeV 等能量点可用, 数据不完整且存在分歧。相关研究包括 Z. A. Wang, J. C. Pei, Y. Liu, Y. Qiang, Phys. Rev. Lett. 123, 122501 (2019), 以及 Z. A. Wang, J. C. Pei et al., Phys. Rev. C 106, L021304 (2022)。BNN 可用于融合评价不完整裂变产额, 并把实验误差传播到预测区间。

置信区间 (confidence interval, CI) 在频率学派和 Bayesian 语境中含义不同。设数据 $(x_1, \dots, x_n)$ 来自正态分布 $N(\mu, \sigma^2)$, μ 未知, 若 n 足够大, 则

$$0.95 = 1 - \alpha = P(-z \leq Z \leq z) = P\left(-1.96 \leq \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \leq 1.96\right),$$

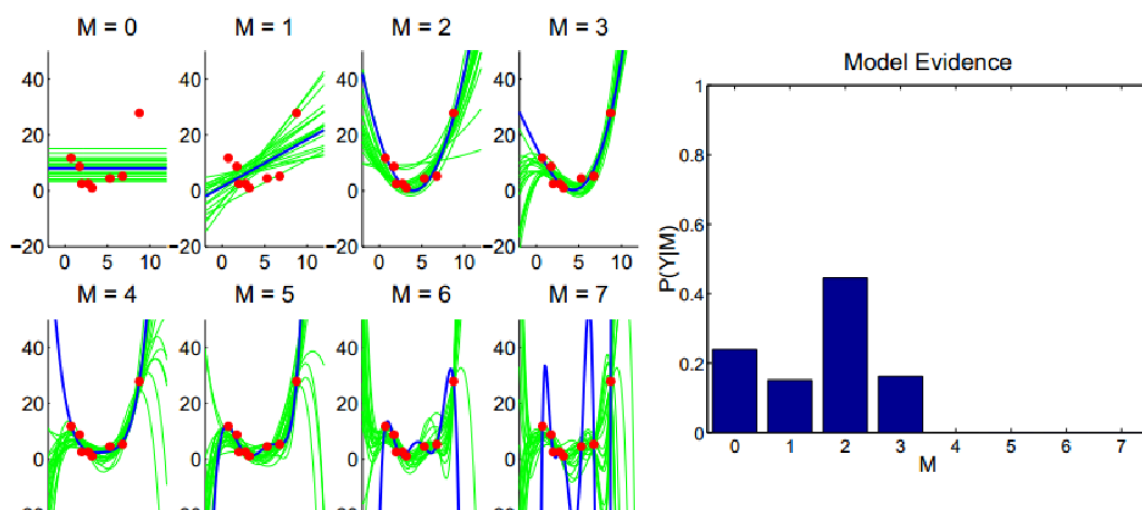


图 10.10: 模型复杂度 $M = 0, \dots, 7$ 的比较。证据最大的模型通常不是最简单或最复杂的模型，而是拟合能力与复杂度惩罚之间的平衡点。

即

$$P\left(\bar{X} - 1.96 \frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{X} + 1.96 \frac{\sigma}{\sqrt{n}}\right) = 0.95.$$

一般置信水平 $C = 1 - \alpha$ 下,

$$\left(\bar{x} - z^* \frac{\sigma}{\sqrt{n}}, \bar{x} + z^* \frac{\sigma}{\sqrt{n}}\right), \quad z^* = \Phi^{-1}\left(1 - \frac{\alpha}{2}\right) = -\Phi^{-1}\left(\frac{\alpha}{2}\right).$$

常见数值为

C	z^*
99%	2.576
98%	2.326
95%	1.96
90%	1.645

频率学派中，区间端点是样本的函数，真实参数 μ 是固定值；正确表述是：重复抽样时，有 95% 的构造区间会覆盖真实 μ 。不能说在给定区间后， μ 以 95% 概率位于其中。Bayesian credible interval 则在给定数据后由 posterior 定义：

$$\int_{\omega_l}^{\omega_h} P(\omega | D) d\omega = CI.$$

在 Bayesian 语义下，可以说给定数据后参数落在该区间的 posterior probability 为指定置信水平。

 **笔记** 过窄的 CI 会夸大模型精度，过宽的 CI 虽不夸大精度却可能浪费预测能力。W. Nazarewicz 在 Phys. Rev. C 98, 034318 (2018) 对核理论 uncertainty quantification 给出了强烈提醒：不给出误差的理论预测无法被严肃地用于精度判断。

10.8 机器学习的应用

10.8.1 数据融合、模型混合与神经量子态

机器学习在计算物理中的应用包括 simulations、optimizations、emulators、solving equations、data fusion、data correlations、inverse problems、particle identifications、model mixing、model reduction，以及 discovery with unsu-

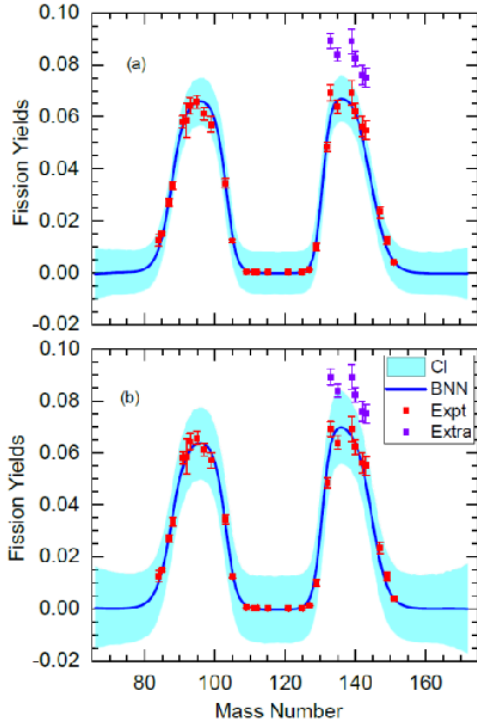


Figure 2: BNN evaluations of fission yields of $n+^{235}\text{U}$ fission by taking into account the discrepant data. (a) shows the evaluations with extra data (purple color). (b) shows the evaluations with the experimental data and 14 MeV data at left peak (pink color).

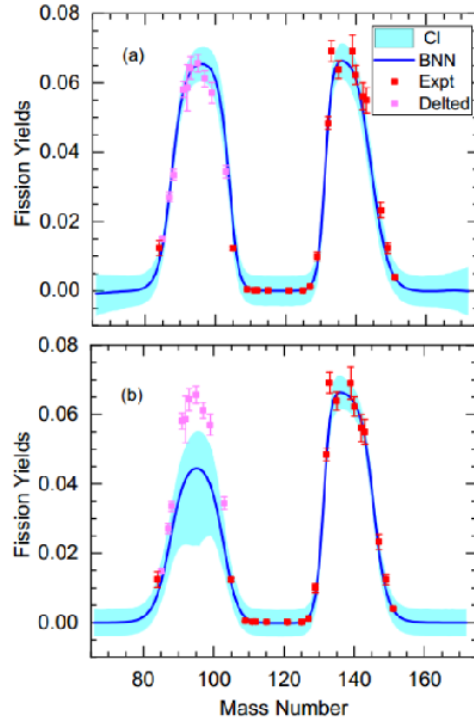


Figure 3: BNN evaluations of fission yields of $n+^{235}\text{U}$ fission of incomplete data. (a) shows the evaluations by experimental data and 14 MeV data at left peak (pink color). (b) shows the evaluations by experimental data and 14 MeV data at left peak (pink color).

图 10.11: 裂变产额评价中的 uncertainty propagation。实验误差、数据缺失和数据分歧会通过 BNN posterior 传播为预测置信区间。

pervised learning。数据融合 (data fusion) 的目标是从分离数据中提取更有用、更准确、更一致的信息，异质融合 (heterogeneous fusion) 尤其适合不同实验、不同模型和不同能区数据的联合评价。

在量子多体问题中，可用神经网络表示波函数，即 neural quantum state。若波函数记为 Ψ_{ANN} ，变分能量为

$$E = \frac{\langle \Psi_{\text{ANN}} | \hat{H} | \Psi_{\text{ANN}} \rangle}{\langle \Psi_{\text{ANN}} | \Psi_{\text{ANN}} \rangle}.$$

通过优化神经网络参数使 E 最小，可得到多体基态的变分近似。这一思想把深度学习的表示能力与量子多体变分原理结合起来。

10.8.2 Bayesian model mixing for nuclear mass

Bayesian model mixing (BMM) 将多个模型按 posterior model probability 加权。对第 k 个模型 $\mathcal{M}_k$,

$$p(\mathcal{M}_k | y) = \frac{p(y | \mathcal{M}_k) \pi(\mathcal{M}_k)}{\sum_{\ell=1}^K p(y | \mathcal{M}_\ell) \pi(\mathcal{M}_\ell)}.$$

其中 $p(y | \mathcal{M}_k)$ 是模型证据, $\pi(\mathcal{M}_k)$ 是模型 prior。对未知数据 y^* , 预测分布为

$$p(y^* | y) = \sum_{k=1}^K p(y^* | y, \mathcal{M}_k) p(\mathcal{M}_k | y).$$

这不是简单平均, 而是根据每个模型对已知数据的解释能力和 prior 权重进行 Bayesian 加权。核质量模型如 HFB-24、FRDM-2012、D1M、BCPM、SLy4、SkP、UNEDF0、UNEDF1、SV-min、SkM* 等都可纳入该框架, 用于预测粒子束缚核的数目和 posterior density。

Bayesian model mixing for nuclear mass

$$p(\mathcal{M}_k|y) = \frac{p(y|\mathcal{M}_k)\pi(\mathcal{M}_k)}{\sum_{\ell=1}^K p(y|\mathcal{M}_\ell)\pi(\mathcal{M}_\ell)}$$

Prediction:

$$p(y^*|y) = \sum_{k=1}^K p(y^*|y, \mathcal{M}_k)p(\mathcal{M}_k|y)$$

unknown data

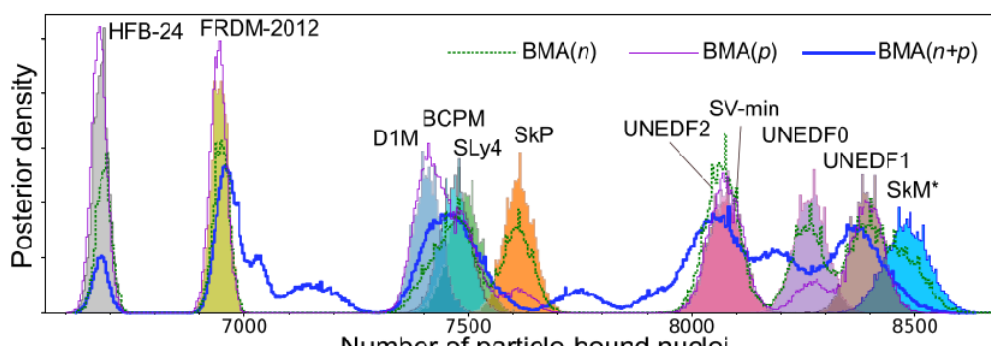


图 10.12: Bayesian model mixing for nuclear mass。不同核质量模型的证据决定其在最终预测中的权重。

10.8.3 核能、核取证与物理信息机器学习

在核能领域，机器学习可用深度学习分析中子通量数据，实现核反应堆异常检测；也可通过涨落信号进行早期异常发现。卷积神经网络（convolutional neural network, CNN）已用于相关异常识别任务，AI 也可用于核电站 human errors 检测。一个代表性工作是 F. Caliva et al., IJCNN, IEEE, 2018。

核取证（nuclear forensics）用于追踪非法放射性材料流动。机器学习方法可根据观测特征区分压水堆（pressurized water reactor, PWR）和沸水堆（boiling water reactor, BWR）来源。相关例子包括 Shengli Chen, Tianxiang Wang, Zhong Zhang, Runfeng Li, Su Yuan, Ruiyi Zhang, Cenxi Yuan, Chunyu Zhang, Jianyu Zhu, Phys. Rev. Applied 19, 034028 (2023)。

Physics informed Bayesian machine learning 的动机来自纯数据驱动机器学习的困境：缺乏物理知识、数据量少。核反应数据误差大、不完整且有分歧；核结构数据精度更高，但包含强量子效应，如壳结构、奇偶效应、形状、闯入态和轨道反转。物理信息机器学习（physics informed machine learning）可通过下列方式融入物理：

1. 输入经验物理特征量，例如壳效应、奇偶效应；
2. 在 loss 函数中加入物理约束，例如归一化和非负性；
3. 学习物理模型残差，即 model mixing；
4. 把物理模型嵌入 Bayesian prior。

其优点是综合守恒律、对称性、量子效应等物理信息。Physics-informed neural network (PINN) 已成功用于求解微分方程和流体力学问题，其核心是把微分算符嵌入 loss 函数。

在物理约束的 Bayesian 机器学习中，总损失可写为

$$\frac{\chi^2}{2} = \sum_i \frac{(Y_i^c - Y_i^e)^2}{2(\sigma_i^2 + \delta_i^2)} + \lambda \frac{\chi_1^2}{2},$$

约束损失为

$$\frac{\chi_1^2}{2} = \sum_j \frac{(t_j^e - \sum_k M_{kj} Y_k^c)^2}{2(\sigma_j^2 + \xi_j^2)}.$$

其中 Y_i^c 是约束后预测, Y_i^e 是实验或评价值, $\sigma_i, \delta_i, \xi_j$ 表示不同误差来源, M_{kj} 是把独立产额转换为累积产额的矩阵, λ 控制约束强度。迁移学习可把物理模型融入 prior; 累积产额与独立产额之间的矩阵转换关系则可作为显式约束。

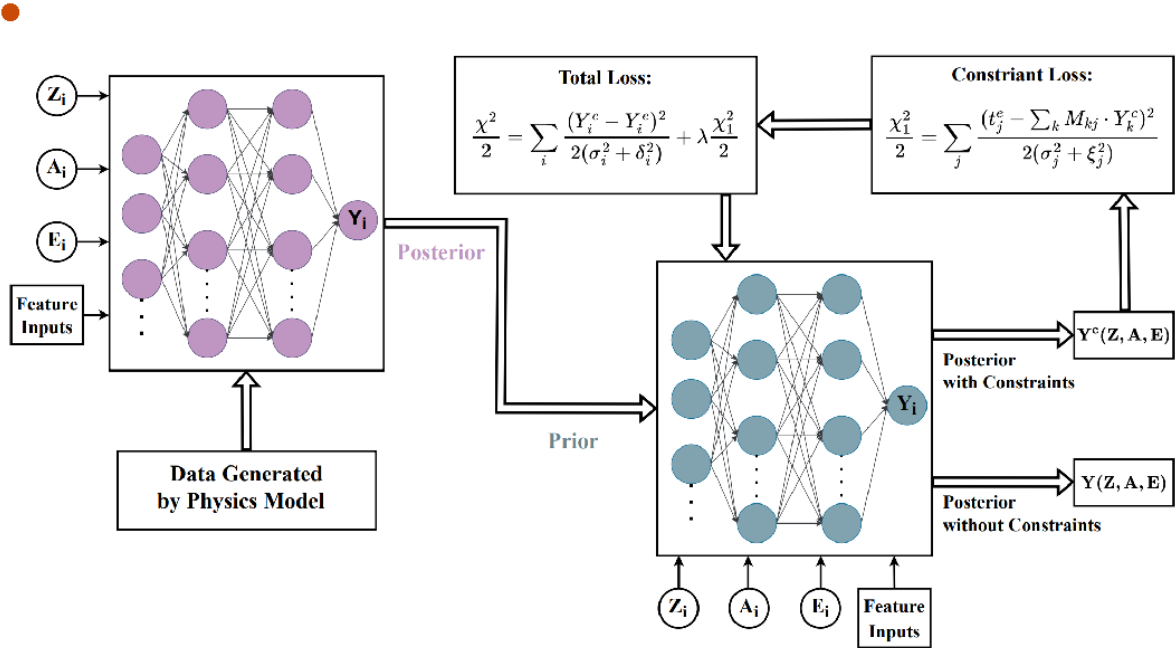


图 10.13: Physics informed Bayesian machine learning 的结构。物理模型可生成 prior, 约束 loss 可把物理关系嵌入 posterior。

10.9 其他机器学习方法

10.9.1 CNN、深度学习、RNN 与 KAN

卷积神经网络 (convolutional neural network, CNN) 主要用于图像学习和声音识别, 也常用于图像过滤处理与特征谱分析。卷积运算可写为

$$(f * g)(x) = \int_{-\infty}^{\infty} f(t)g(x-t) dt.$$

对图像数据, 例如 $32 \times 32 \times 3$ 表示宽 32、高 32、三个颜色通道。CNN 利用 local correlation 和 parameter sharing: 局部卷积核在不同位置共享权重, 因此参数数目远小于全连接网络, 并能仿照生物视觉感知提取层级特征。

深度学习 (deep learning) 使用多隐藏层网络和更多参数, 适合大数据场景。若数据量小而网络过深, 容易 overfitting。AlphaGo 的代表性意义在于把 deep neural networks 与 Monte Carlo tree search 结合, 用策略网络和价值网络辅助搜索。

循环神经网络 (recurrent neural network, RNN) 在隐藏层之间引入连接, 使网络能够记忆前面信息并用于当前输出, 适合序列相关数据。其基本思想是

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b), \quad y_t = g(W_y h_t + c),$$

其中 h_t 是隐藏状态。RNN 与 transformer 的区别在于前者按时间递推, 后者用 self-attention 直接建模序列中不同

位置之间的关系。

Kolmogorov–Arnold representation 启发了 Kolmogorov–Arnold network (KAN) 一类模型。其思想是用可学习的一维函数组合表示多变量函数，优点是可解释性和参数效率较高。与普通 MLP 在节点上放激活函数不同，KAN 常把可学习函数放在边上，因此每条连接本身具有函数形状。

10.9.2 SVM、Decision Tree、Random Forest 与 Boosting

支持向量机 (support vector machine, SVM) 主要用于 classification。线性可分时，它寻找最大间隔超平面；非线性可分时，用 kernel function 把样本映射到高维特征空间，再在该空间中做间隔最大化。其核心优化可理解为在多维特征空间中进行距离计算和分类边界选择。

决策树 (decision tree) 是一种用于分类和回归任务的非参数监督学习算法。它由根节点、分支、内部节点和叶节点组成，通过逐层划分特征空间得到预测。Random Forests 通过训练多棵树并平均或投票降低方差。LightGBM (Light Gradient Boosting Machine) 和 XGBoost (eXtreme Gradient Boosting) 属于梯度提升树方法，通过逐步拟合残差提升预测能力。

10.9.3 生成式模型、GPT 与 Transformer

ChatGPT 背后的技术属于生成式机器学习：模型不只是分类或预测已有数据，而是生成与训练数据相似且具有一定新颖性的内容。生成对抗网络应记为 GAN (generative adversarial network)，不要与 GNN (graph neural network) 混淆。GAN 包含 generator network 和 discriminator network：生成器产生样本，判别器判断样本是真实还是伪造，二者竞争改进。

Generative pre-trained transformers (GPTs) 是基于 transformer architecture 的神经网络，先在大规模未标注文本上预训练，再生成类似人类语言的新文本。Transformer 使用 self-attention，对数据中不同部分的重要性赋予不同权重。典型结构包含 encoder–decoder；与 RNN 不同，它不依赖逐步递推来传递长程信息，而是通过注意力矩阵直接建立位置之间的联系。

10.9.4 AI4S、黑箱问题与物理引导

AI for Science (AI4S) 覆盖 optimization、model reduction、model emulator、neural quantum state、event identification、data mining & evaluation、regression、classification、research agent、hypothesis generation、scientific writing & review、symbolic regression、solving PDE equation 和 physics informed learning 等任务。

深度模型常被称为 black box，其核心矛盾是 interpretability 与 accuracy。通常线性回归和决策树可解释性较强但表达能力有限；深度神经网络和 CNN/RNN 精度高但可解释性较弱。物理信息方法试图沿着 physics-informed、physics-inspired、physics-constrained、physics-guided 的方向缓解这一矛盾，包括：

- informative priors；
- transformer data；
- net structures；
- constrained loss functions；
- guide learning；
- small data problems；
- incomplete models；
- imperfect data；
- understanding deep learning。

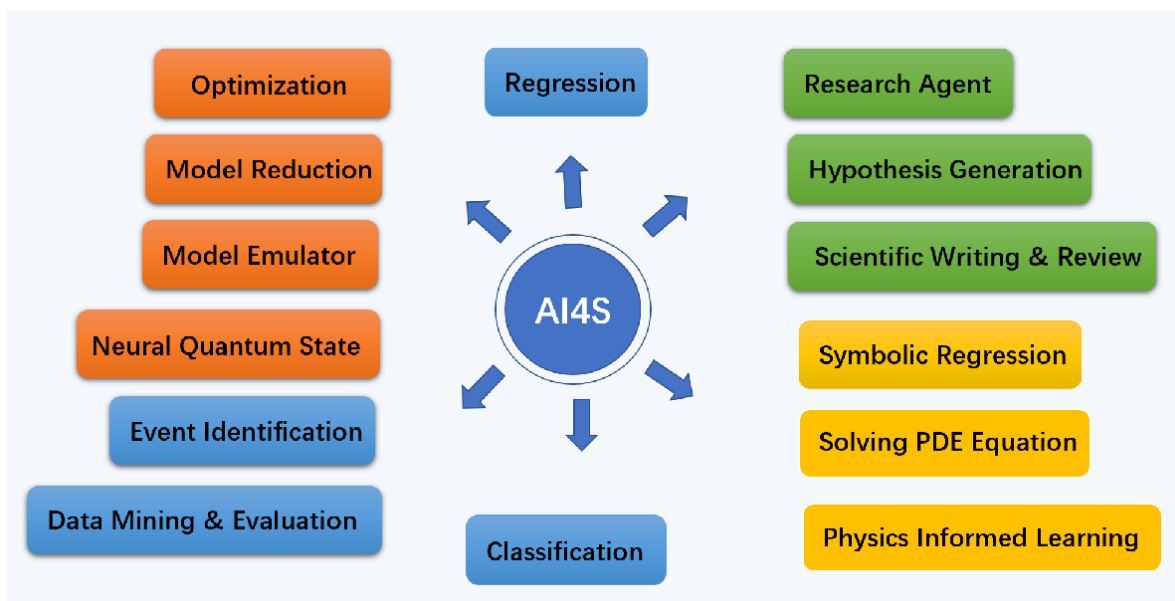


图 10.14: AI4S 的任务图谱。机器学习既可作为数值求解器，也可作为模型发现、数据评价和科学写作辅助工具。

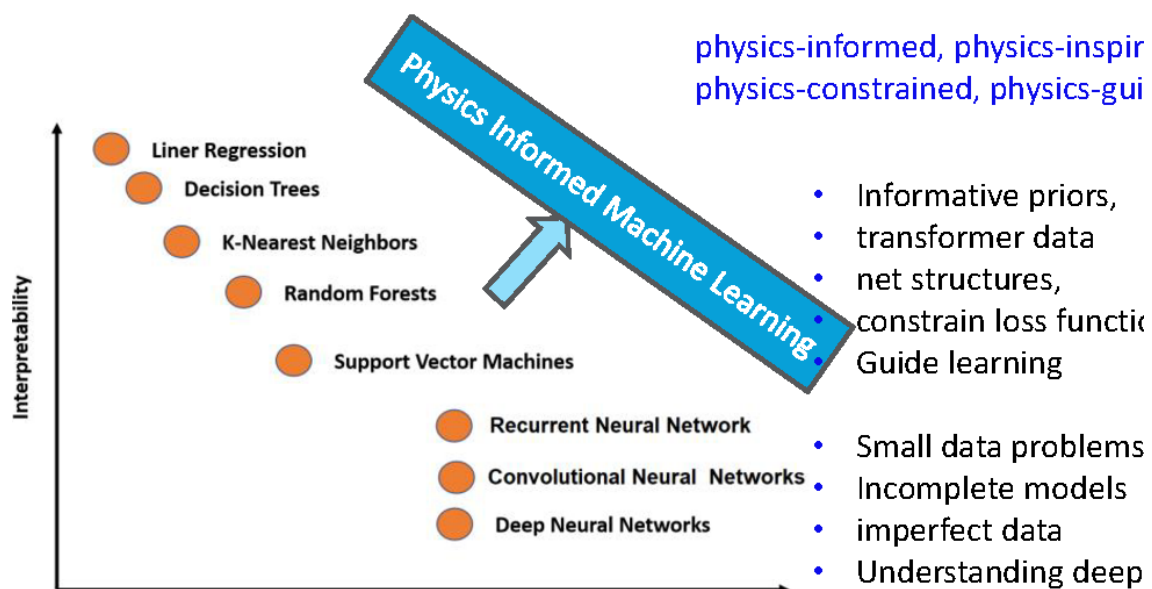


图 10.15: 可解释性与精度的权衡。物理信息机器学习希望在高精度模型中嵌入可解释的先验、结构或约束。

10.9.5 Bayesian Optimization

Bayesian optimization 用于目标函数 f 代价昂贵且显式形式不可得的优化问题。典型场景是每次实验、模拟或模型评估都很贵，因此不能大量采样。其基本流程是：

1. 对未知目标函数建立 prior probabilistic belief，通常取 Gaussian process：

$$p(f) = GP(f; \mu, K).$$

2. 用已有观测数据更新 prior。若

$$D_{1:n} = [x_1, x_2, \dots, x_n; f_1, f_2, \dots, f_n],$$

则结合 likelihood 得到 posterior。

3. 用 posterior 描述未知函数：

$$p(f | D_{1:n}) = GP(f; \mu_{f|D_{1:n}}, K_{f|D_{1:n}}).$$

根据 acquisition function 或 loss function 决定下一次评估位置。

4. 选择 x_{n+1} ，得到 f_{n+1} ，扩充数据为 $D_{1:n+1}$ ，再回到第二步。

Acquisition function 需要平衡 exploitation 与 exploration。Exploitation 在可能改进的区域采样，过多会陷入局部最小；exploration 在高不确定性区域采样，过多会浪费迭代。

10.10 本章总结

结论 机器学习章节的主线是统计建模、非线性表示和不确定性量化。经典统计给出均值、方差、协方差、熵和误差传播；Bayes 统计把 prior、likelihood、posterior 与 evidence 连接起来，为模型比较、参数估计和预测不确定性提供统一框架。线性回归和 Gaussian process regression 是从简单参数模型到非参数函数分布的过渡；神经网络提供强大的非线性映射能力；Bayesian 神经网络、正则化、Occam's Razor 和 confidence interval 则提醒我们，预测能力必须与误差分析和模型复杂度控制同时考虑。物理信息机器学习把守恒律、对称性、模型残差和物理约束融入机器学习，是 AI for Science 在计算物理中的重要方向。

- Bayesian 统计与经典统计理念不同：prior 是优点也是风险，能引入物理知识，也可能引入偏差；
- Neural network 不只是参数化函数，而是高维非线性表示、优化和统计推断的结合；
- 机器学习意味着科学范式变化，适合不能直接用物理方程描述、或方程难以求解的问题；
- Bayesian 方法的意义在于模型评估与预测、减小 overfitting、优化误差分析，并通过 Occam's Razor 鼓励适度模型和适度网络；
- 机器学习和人工神经网络的数学基础包括统计、信息论、非线性映射和参数优化，处理对象常是大数据；
- AI 在大数据时代具有硬件基础和现实需求，新技术不断出现，物理信息机器学习将有广泛应用；所谓 AI 意识在工程上可理解为由目标函数和反馈机制定义的行为目标。

第十一章 超级计算与量子计算

内容提要

- 计算机发展与物理研究之间的关系
- 量子计算基本原理、量子比特与量子门
- 超级计算机、FLOPS 标度与典型高性能计算体系结构
- 纠缠、测量与真实量子芯片误差
- MPI 与 OpenMP 并行程序设计
- 云量子计算、VQE、误差缓解与激发态算法
- GPU、协处理器、CUDA 与高性能线性代数库
- 量子傅里叶变换、相位估计与对称性恢复
- 计算物理课程核心算法脉络与未来计算挑战

本章从计算硬件的演化出发，把传统高性能计算（high performance computing, HPC）与量子计算（quantum computing）放在同一条主线中理解。计算物理的研究对象不断走向大规模、多尺度和强关联：一方面需要超级计算机提供更高的浮点运算能力和更大的并行规模；另一方面，量子多体问题的 Hilbert 空间维数随自由度指数增长，促使人们探索以量子系统模拟量子系统的新范式。

11.1 计算机发展与物理的关系

11.1.1 超级计算机的发展趋势

超级计算机（supercomputer）的发展换代非常快，国际竞争激烈。现代超级计算的主要目标之一是迈向 Exascale computing，即每秒 10^{18} 次量级的浮点运算能力。物理问题推动了这种需求：格点量子色动力学、第一性原理材料计算、核多体结构、气候模拟、天体物理流体力学和大规模机器学习训练都要求在巨大自由度上重复执行线性代数、积分、随机采样或偏微分方程求解。

一个重要变化是，单个 CPU 的主频已经不再像早期那样快速提高。性能增长更多依赖：

- 多核心 CPU；
- GPU 与协处理器（coprocessor）；
- 多节点并行；
- 高速互连网络；
- 更高效的存储、调度与通信隐藏。

因此，现代高性能计算不只是“更快的处理器”，而是 CPU、GPU、内存、网络、文件系统和软件栈共同组成的复杂体系。节点内部计算能力可以通过多核和 GPU 快速增加，但节点间通信需要移动数据，往往耗能高、延迟大，是并行效率下降的重要来源。

定义 11.1

FLOPS（floating point operations per second） 表示每秒能够完成的浮点运算次数，是衡量计算机峰值或实测浮点性能的常用指标。实际应用性能通常低于峰值性能，因为访存、通信、负载不均衡和算法结构都会造成损耗。



常用 FLOPS 标度如下：

单位	英文	每秒浮点运算次数
MFLOPS	megaFLOPS	10^6
GFLOPS	gigaFLOPS	10^9
TFLOPS	teraFLOPS	10^{12}
PFLOPS	petaFLOPS	10^{15}
EFLOPS	exaFLOPS	10^{18}
ZFLOPS	zettaFLOPS	10^{21}

其中 1 TFLOPS 也常称为“1 太拉每秒浮点运算”。在中文大数单位中，课件中列出的对应关系是：

万	10^4	亿	10^8	兆	10^{12}
京	10^{16}	垓	10^{20}	杼	10^{24}
穰	10^{28}	沟	10^{32}	涧	10^{36}
正	10^{40}	载	10^{44}	极	10^{48}
恒河沙	10^{52}	阿僧祇	10^{56}	那由他	10^{60}
不可思议	10^{64}	无量	10^{68}	大数	10^{72}

这些单位本身不是算法概念，但有助于把 EFLOPS 和 ZFLOPS 这样的计算性能量级与传统大数尺度联系起来。

11.1.2 TOP500 与性能演化

TOP500 榜单按照 Linpack benchmark 的实测性能对全球超级计算机排序。Linpack 的核心是大规模稠密线性方程组求解，因此它主要反映双精度浮点线性代数能力，而不完全代表所有科学应用的真实性能。对于稀疏矩阵、随机访存、强通信耦合或 I/O 密集型程序，实际效率可能明显不同。



图 11.1: 2020 年 11 月 TOP500 超级计算机榜单示意。榜单性能反映了当时高性能计算系统的国际竞争格局。

从性能发展图可以看出，榜单第一名、第五百名以及全榜单性能总和都随时间上升。纵轴采用对数标度，因

2025/11					
Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOF/NNSA/LLNI United States	11,340,000	1,809.00	2,821.10	29,685
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/ΓΖJ Germany	4,801,344	1,000.00	1,226.28	15,794
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure	2,073,500	561.20	846.84	

图 11.2: 2025 年 11 月 TOP500 榜单示意。超级计算持续向更高并行度、更强异构性和 Exascale 标度演化。

此图上的近似直线对应指数式性能增长。进入 Exascale 阶段以后，单纯堆叠硬件带来的能耗和通信代价越来越突出，算法必须同时考虑计算量、通信量和能耗。

11.1.3 从向量机到异构超级计算

早期超级计算机以向量处理器（vector processor）为代表。Cray-1 于 1976 年问世，采用 64 位处理器，主频约 80 MHz，峰值性能约 160 MFLOPS，内存为 1,048,576 个 64 位字，功耗约 115 kW，价格约千万美元量级。它说明早期超级计算的关键是把数组运算设计成连续向量操作，从而提高流水线效率。

到 Summit 等现代系统，单个节点已经由多核 CPU 与多块 GPU 组成。计算密集任务通常被转移到 GPU 上，CPU 负责控制、调度、数据准备和部分串行逻辑。美国 Frontier 是世界上第一批 Exascale 超级计算机之一，标志性能是每秒超过 10^{18} 次计算。其节点、加速器和互连网络共同决定了系统性能。

11.1.4 集群、互连与云计算

早期并行系统常以 cluster、grid computing 和 distributed computing 的形式出现。集群计算把多台计算节点通过网络连接起来；分布式计算更强调跨地点、跨管理域的资源协同；格点计算曾用于把不同机构的计算资源组织成统一平台。

超级计算机的互连网络与普通网络不同。三维环面互连（three-dimensional torus interconnect）曾用于 Cray XT3 等系统，其思想是把节点排列在三维网格上，每个节点与邻近节点连接，从而降低通信路径长度和延迟。InfiniBand 网络则通过高带宽、低延迟和专门的路由算法服务大规模并行任务。云计算（cloud computing）通常更强调弹性资源、虚拟化和服务化接口，而超级计算更强调低延迟互连、批处理调度和高效并行通信。

Titan Architecture 展示了典型异构节点的组成：NVIDIA Kepler GPU、AMD Opteron CPU、Cray Gemini interconnect、GPU 端 GDDR5 高带宽显存和 CPU 端 DDR3 内存共同构成计算节点。GPU 适合高吞吐并行计算，CPU 适合复杂控制与通用任务，互连网络负责节点间通信。

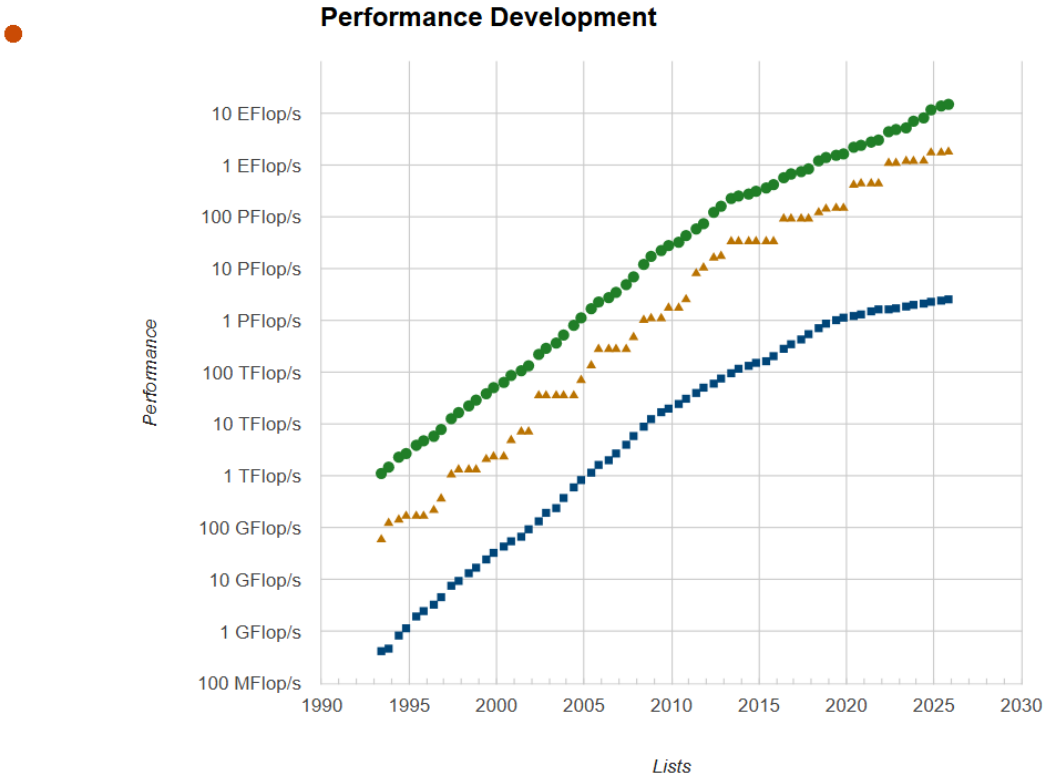
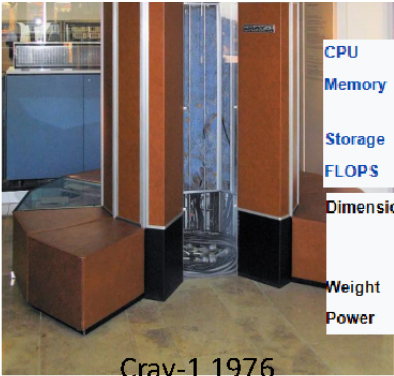


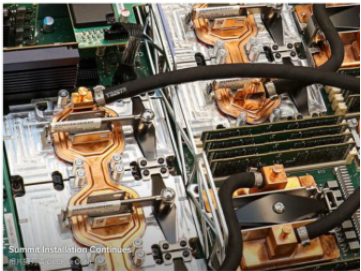
图 11.3: 超级计算机性能发展趋势。绿色、橙色和蓝色曲线分别体现榜单总性能、第一名性能和第五百名性能的长期增长。

●



CPU	64-bit processor @ 80 MHz ^[1]
Memory	8.39 Megabytes (up to 1 048 576 words) ^[1]
Storage	303 Megabytes (DD19 Unit) ^[1]
FLOPS	160 MFLOPS
Dimensions	Height : 196 cm (77 in) ^[1] Dia. (base) : 263 cm (104 in) ^[1] Dia. (columns) : 145 cm (57 in) ^[1]
Weight	5.5 tons (Cray-1A)
Power	115 kW @ 208 V 400 Hz ^[1]

Cray-1 1976
vector processor



Summit 2018

Processor: IBM POWER9™ (2/node)
GPUs: 27,648 NVIDIA Volta V100s (6/node)
Nodes: 4,608
Node Performance: 42TF
Memory/node: 512GB DDR4 + 96GB HBM2
NV Memory/node: 1600GB
Total System Memory: >10PB DDR4 + HBM + Non-volatile
Interconnect Topology: Mellanox EDR 100G InfiniBand, Non-blocking Fat Tree



图 11.4: 从 Cray-1 向量机到 Summit 异构系统。硬件主线从单机向量处理逐渐转向多节点、多核与 GPU 加速。

● Frontier

system is based on HPE Cray's new EX architecture and slingshot interconnect with optimized 3rd Gen AMD EPYC CPUs for HPC and AI, and AMD Instinct 250X accelerators.

SYSTEM SPECS	TITAN	SUMMIT	FRONTIER
Peak Performance	27 PF	200 PF	1.6 EF
Cabinets	200	256	74
Node	1 AMD Opteron CPU 1 NVIDIA K20X Kepler GPU	2 IBM POWER9™ CPUs 6 NVIDIA Volta GPUs	1 HPC and AI Optimized 3 rd Gen AMD EPYC CPU 4 Purpose Built AMD Instinct 250X GPUs
CPU/GPU Interconnect	PCI Gen2	NVLink Coherent memory across the node	AMD Infinity Fabric
System Interconnect	Gemini	2x Mellanox EDR 100G InfiniBand Non-Blocking Fat-Tree	Multiple Slingshot NICs providing 100 GB/s network bandwidth. Slingshot network which provides adaptive routing, congestion management and quality of service.
Storage	512 PB, 1.1 TB/s, Lustre filesystem	250 PB, 2.5 TB/s, GPFS™	2-4x performance and capacity of Summit's I/O subsystem. Frontier will have near node storage like Summit.

图 11.5: Frontier 超级计算系统的整体规格示意。Exascale 系统的性能来自大量异构节点、高带宽内存和高速互连。

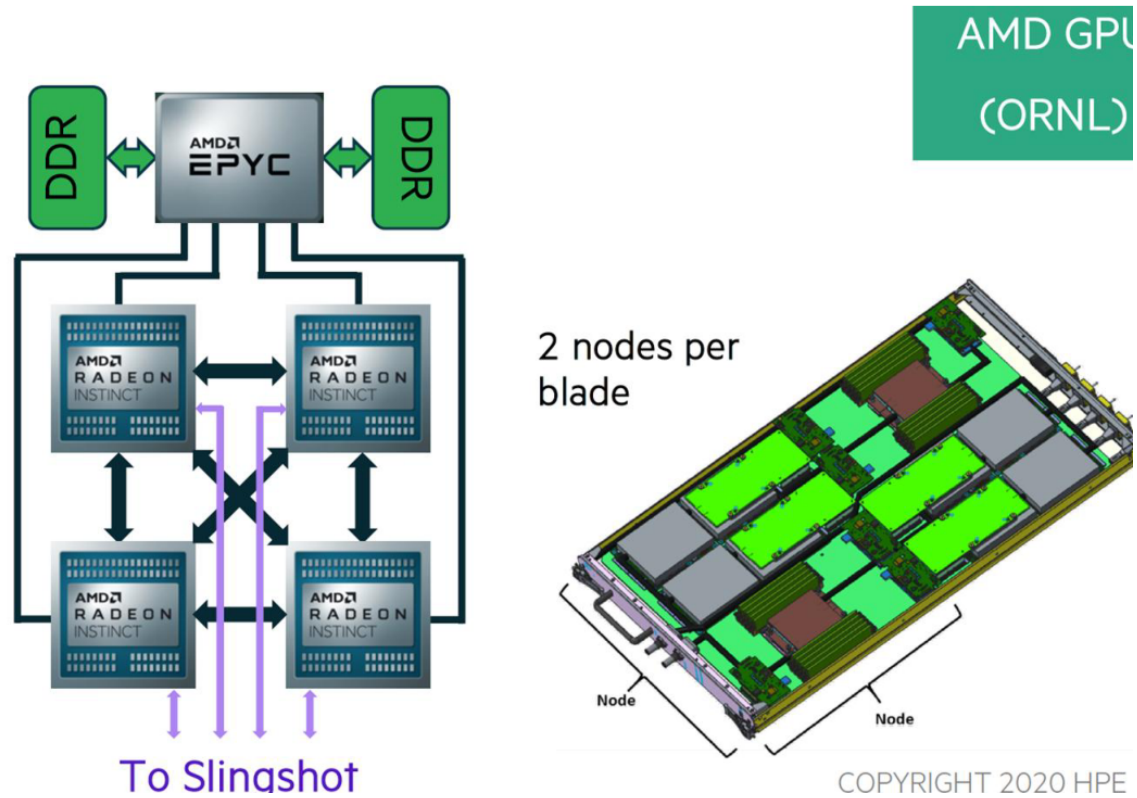


图 11.6: Frontier 节点与 AMD GPU 架构示意。节点级异构结构要求程序显式管理 CPU、GPU 与网络之间的数据移动。

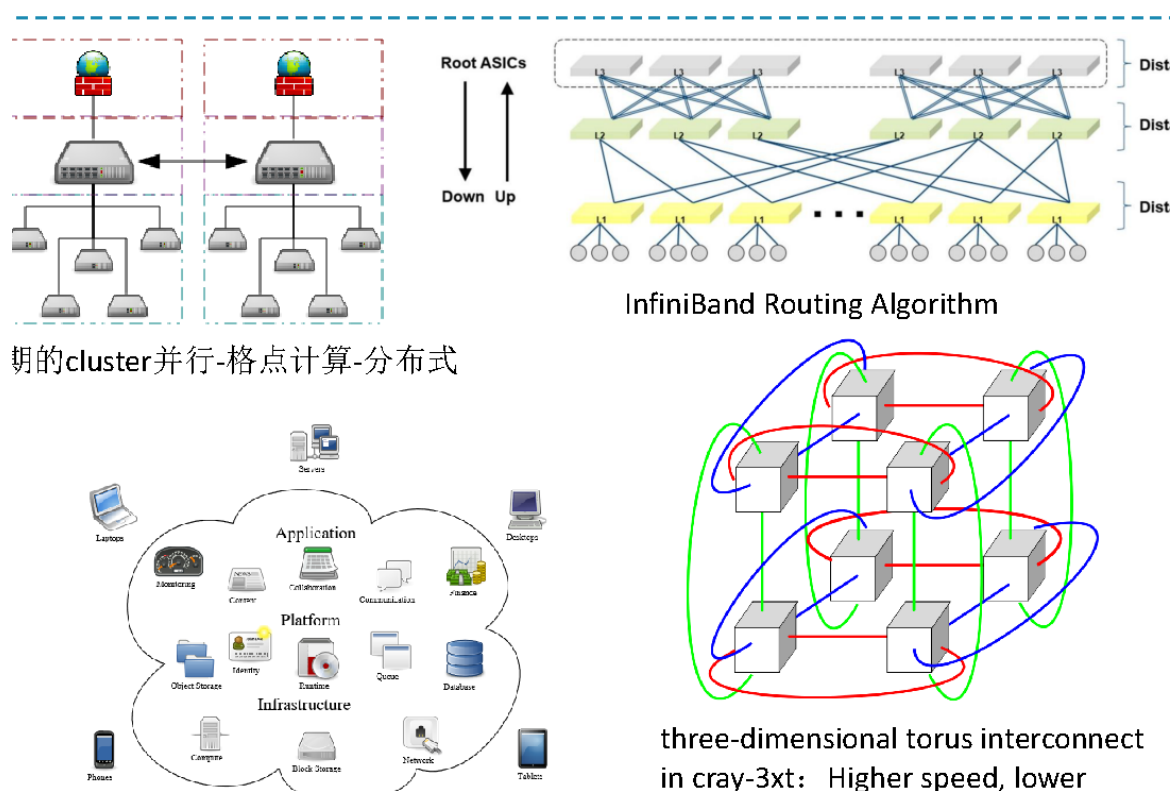
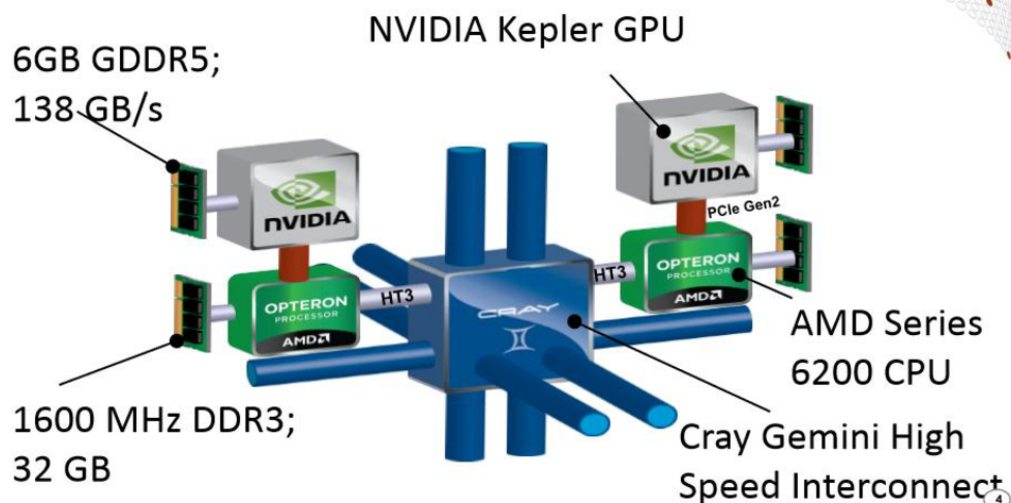


图 11.7: 集群、三维环面互连、InfiniBand 路由和云计算的对比。高性能计算的关键瓶颈常常不是峰值算力，而是节点间数据交换。

Cray XK7 Architecture



Titan Architecture

图 11.8: Titan 异构节点结构。CPU、GPU、内存和互连网络之间的数据通路决定了实际应用效率。

国内高校也建设了面向科研的超算平台，例如北京大学“未名一号”。这类平台使物理、化学、材料、生命科学和机器学习等方向能够在统一计算环境中运行大规模并行程序。

● 北大未名一号



图 11.9: 北京大学“未名一号”高性能计算系统。高校超算平台是课程算法走向科研应用的重要基础设施。

11.2 MPI 与 OpenMP 并行编程

11.2.1 MPI 的基本思想

MPI (Message Passing Interface) 是消息传递并行程序设计的标准接口。它不是一种编程语言，而是一个库：并行操作通过调用 MPI 例程完成。C 程序通常包含头文件 `mpi.h`；Fortran 程序可使用 MPI 或 MPI_F08 模块。旧式 Fortran 77 的 `mpif.h` 仍能见到，但不推荐作为新程序的首选接口。

定义 11.2

消息传递并行 (message passing parallelism) 是指多个进程拥有各自独立的地址空间，进程之间不能直接访问对方内存，必须通过显式发送和接收消息交换数据。

MPI 程序启动后通常形成一个进程集合，默认通信子为 `MPI_COMM_WORLD`。两个最基本的查询函数是：

`MPI_Comm_size`：返回通信子中的进程总数 `size`，

`MPI_Comm_rank`：返回当前进程编号 `rank`， $0 \leq rank < size$ 。

`rank` 是区分不同并行进程的核心标识。常见做法是令 `rank = 0` 的进程承担主进程 (master) 角色，其余进程承担工作进程 (worker) 角色。

一个最小 MPI 程序的逻辑结构为：

`MPI_Init` $\rightarrow$ `MPI_Comm_size` $\rightarrow$ `MPI_Comm_rank` $\rightarrow$ 并行计算与通信 $\rightarrow$ `MPI_Finalize`。

`MPI_Init` 或 `MPI_Init_thread` 初始化 MPI 运行环境，`MPI_Finalize` 结束 MPI 环境。二者之间才可以安全调用 MPI 例程。

```

program main
  use mpi
  integer :: ierr, rank, size, i, provided
  real :: A(10)

  call MPI_Init_thread(ierr)
  call MPI_Comm_size(MPI_COMM_WORLD, size, ierr)
  call MPI_Comm_rank(MPI_COMM_WORLD, rank, ierr)

  do i = 1, 10
    A(i) = i * rank
  end do

  print *, 'My rank ', rank, ' of ', size
  print *, 'Here are my values for A:'
  print *, A

  call MPI_Finalize(ierr)
end program main

```

在严格 Fortran 接口中, `MPI_Init_thread` 通常需要给出所需线程级别和返回线程级别; 若不涉及 MPI 与线程混合, 可以直接使用 `MPI_Init`。上述示例的物理意义是: 每个进程独立构造数组 $A_i = i \times \text{rank}$, 从而可以看出不同进程拥有不同的局部数据。

C 语言版本具有相同结构:

```

#include "mpi.h"
#include <stdio.h>

int main(int argc, char *argv[]) {
  int rank, size, i, provided;
  float A[10];

  MPI_Init_thread(&argc, &argv, MPI_THREAD_SINGLE, &provided);
  MPI_Comm_size(MPI_COMM_WORLD, &size);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  for (i = 0; i < 10; i++) A[i] = i * rank;

  printf("My rank %d of %d\n", rank, size);
  printf("Here are my values for A:\n");
  for (i = 0; i < 10; i++) printf("%f", A[i]);
  printf("\n");

  MPI_Finalize();
  return 0;
}

```

11.2.2 MPI 通信、阻塞与非阻塞

MPI 中最基本的点对点通信为发送和接收：

```
MPI_SEND(start,count,datatype,dest,tag,comm)
MPI_RECV(start,count,datatype,source,tag,comm,status).
```

这里 `start` 是缓冲区起始地址，`count` 是元素数目，`datatype` 是 MPI 数据类型，`dest` 和 `source` 是目标和源进程编号，`tag` 是消息标签，`comm` 是通信子，`status` 返回接收状态。

消息能否匹配取决于四个条件：源进程、目标进程、通信子和标签必须一致，数据类型和长度也必须与缓冲区设计相容。若发送数据超过接收缓冲区，程序会报错或产生未定义行为。

 **笔记** 阻塞通信 (**blocking communication**) 在通信完成前不会返回。若两个进程都先等待接收而没有发送，或者缓冲区不足导致发送无法完成，就可能出现死锁 (deadlock)。因此 MPI 程序必须设计清楚发送和接收顺序。

非阻塞通信用 `MPI_Isend` 和 `MPI_Irecv` 发起通信，随后用 `MPI_Wait` 或 `MPI_Test` 检查完成状态。其优点是能把通信与计算重叠，例如先发起接收，再做局部计算，最后等待数据到达：

发起非阻塞通信 → 执行可独立计算 → 等待通信完成 → 使用远端数据。

广播和同步屏障是最常见的集合通信：

```
MPI_Bcast : 把一个进程的数据广播给通信子内所有进程,
MPI_Barrier : 所有进程到达屏障前, 先到者必须等待.
```

`MPI_Bcast` 常用于分发参数、矩阵或初始条件；`MPI_Barrier` 可用于调试和阶段同步，但过度使用会降低并行效率。

11.2.3 Master-Worker 模式

一个典型的 master-worker 模式是：主进程向每个工作进程发送任务，工作进程完成计算后把结果传回主进程。伪代码可写为：

```
if (myrank == 0) then
  call send_to_every_worker()
  call recv_from_every_worker()
else
  call recv_from_root()
  call do_local_work()
  call send_to_root()
end if
```

这种模式适合任务天然分块、工作量相近且通信量不大的问题，例如参数扫描、蒙特卡罗抽样、不同能量点或不同频率点上的独立计算。

若使用非阻塞接收，可以先接收矩阵 ρ 和标量 bkt ，再等待完成：

```
call MPI_GET_PROCESSOR_NAME(processor_name, namelen, ierr)
call MPI_Irecv(rho, nmatrix, MPI_REAL8, 0, 21, MPI_COMM_WORLD, req1, ierr)
call MPI_Irecv(bkt, n1, MPI_REAL8, 0, 31, MPI_COMM_WORLD, req2, ierr)
call MPI_Wait(req1, status, ierr)
call MPI_Wait(req2, status, ierr)
```

主进程发送时通常循环遍历目标进程：

```
do ik = 1, nworkers
```

```

call MPI_Isend(rho, nmatrix, MPI_REAL8, ik, 21, MPI_COMM_WORLD, req1, ierr)
call MPI_Isend(bkt, n1, MPI_REAL8, ik, 31, MPI_COMM_WORLD, req2, ierr)
call MPI_Wait(req1, status, ierr)
call MPI_Wait(req2, status, ierr)
end do

```

标签 21 和 31 用来区分不同物理量。实际程序中应避免在循环中立即等待所有发送，否则可能失去非阻塞通信的重叠优势；更高效的写法是保存所有请求，再统一等待。

11.2.4 OpenMP 多线程并行

OpenMP (Open Multi-Processing) 面向共享内存并行。它通常用于单个计算节点内部的多线程计算，线程之间共享同一地址空间，不需要像 MPI 那样通过网络显式发送消息。

定义 11.3

共享内存并行 (**shared-memory parallelism**) 是指多个线程可以访问同一进程的内存空间。它适合节点内部并行循环、数组操作和局部线性代数，但必须处理数据竞争 (race condition) 和同步问题。

OpenMP 的优点是应用相对简单：在 C/C++ 中用 `#pragma omp` 指令，在 Fortran 中用 `!$omp` 指令标记并行区域。与 `pthreads` 等底层线程库相比，OpenMP 更接近“在串行程序上添加并行指令”。它尤其适合循环并行：

大循环 → 分配给多个线程 → 每个线程处理一部分迭代 → 合并结果。

如果程序没有可并行循环或共享数据竞争严重，OpenMP 的效率就会下降。

```

#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[]) {
    omp_set_num_threads(4);

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int np = omp_get_num_threads();
        printf("Hello_world_%d_of_%d\n", id, np);
    }
    return 0;
}

```

也可以在运行前设置

```
export OMP_NUM_THREADS=4
```

来指定线程数。程序中 `omp_get_thread_num()` 返回线程编号，`omp_get_num_threads()` 返回并行区域内的线程总数。

Fortran 程序中常见写法如下：

```

subroutine diagonalization(...)
    use omp_lib

    !$omp parallel private(i, j)
    !$omp do

```

```

do i = 1, n
    call local_work(i)
end do
!$omp end do

!$omp barrier

!$omp single
    call collect_or_update_global_data()
!$omp end single
!$omp end parallel
end subroutine diagonalization

```

`parallel` 创建并行区域，`do` 把循环迭代分给不同线程，`barrier` 强制同步，`single` 表示其中一个线程执行全局更新。若多个线程同时写同一变量，需要使用 `atomic`、`critical`、`reduction` 或重新设计数据布局，否则会出现竞争。

11.2.5 MPI+OpenMP 混合并行

现代节点通常包含多个 CPU 核和多块 GPU。若每个 CPU 核运行一个 MPI 进程，进程数很大、内存副本很多、节点内通信也会变重；若只运行一个 MPI 进程，则可能无法充分利用全部 CPU 核。因此常用 MPI+OpenMP 混合并行：

节点间用 MPI 通信 + 节点内用 OpenMP 多线程。

例如一个 12 核节点可以采用不同布局：

- 一个 MPI 进程负责通信，12 个 OpenMP 线程负责节点内循环，可设置 `OMP_NUM_THREADS=12`；
- 3 个 MPI 进程分别绑定到 `core-0`、`core-4`、`core-8`；每个 MPI 进程使用 4 个 OpenMP 线程，此时可设置 `OMP_NUM_THREADS=4`；
- 外层物理任务由 MPI 分发，子程序内部再用 OpenMP 加速矩阵、积分或网格循环。

在作业脚本或运行命令中，经常通过参数控制“每节点 MPI 进程数”和“每进程线程数”。典型组合包括：

- 不使用 OpenMP，每节点 16 个 MPI 进程；
- 不使用 OpenMP，每节点 8 个 MPI 进程，可用类似 `-j 1` 的参数控制资源布局；
- 使用 OpenMP，每节点 2 个 MPI 进程，每进程 8 个线程，可用类似 `-d 8`；
- 使用 OpenMP，每节点 2 个 MPI 进程，每进程 4 个线程，可用类似 `-d 4 -j 1`；
- 使用 OpenMP，每节点 1 个 MPI 进程，每进程 16 个线程，可用类似 `-d 16`；
- 使用 OpenMP，每节点 1 个 MPI 进程，每进程 8 个线程，可用类似 `-d 8 -j 1`。

具体参数名称依赖集群调度系统和运行脚本，但思想相同：在 MPI 进程数、线程数、内存占用和通信效率之间折中。

11.3 GPU 与协处理器

11.3.1 GPU 加速的基本思想

GPU (graphics processing unit) 最初用于图形渲染，后来因其大量简单计算核心而成为通用并行计算加速器。与 CPU 相比，GPU 的单个核心较简单，但核心数量多、吞吐量高，适合执行大量相同或相近的操作，例如向量加法、矩阵乘法、FFT、卷积和局部网格更新。

● MPI+OpenMP 混和并行计算

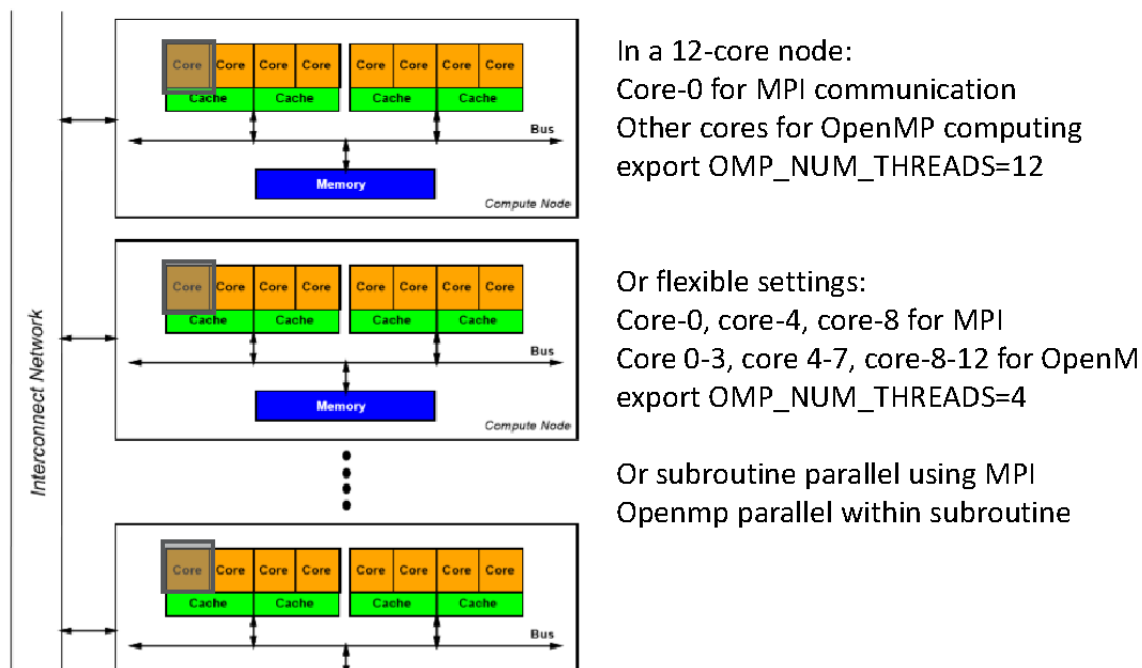


图 11.10: MPI+OpenMP 混和并行示意。MPI 负责进程级并行和节点间通信，OpenMP 负责共享内存中的线程级并行。

现代异构节点常包含：

多核 CPU + 数千个 GPU 计算核心 + 可能的协处理器。

Intel Xeon Phi 曾是典型协处理器，它位于传统 CPU 与 GPU 之间，强调大量较简单核心和向量化能力。无论是 GPU 还是协处理器，都不是自动让所有程序加速；只有当算法具有足够并行度、数据访问规律、通信与拷贝开销可控时，加速才显著。

NVIDIA GPU 常用 CUDA (Compute Unified Device Architecture) 编程。CUDA API 可以看成 C/C++ 的扩展：程序员编写在 GPU 上执行的 kernel，用大量线程并行处理数组元素。

```
#include <cuda_runtime.h>

__global__ void vectorAdd(const float *a, const float *b, float *c, int n) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    if (index < n) c[index] = a[index] + b[index];
}
```

其中 `blockIdx.x` 是线程块编号，`blockDim.x` 是每个线程块的线程数，`threadIdx.x` 是块内线程编号。因此

$$\text{index} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

给出当前线程负责的数组元素。向量加法的数学形式为

$$c_i = a_i + b_i, \quad i = 0, \dots, n-1.$$

这个例子说明 GPU kernel 的核心思想：把每个数组元素的计算映射给一个线程。

11.3.2 GPU 线性代数库与 MAGMA

直接编写 CUDA 程序需要管理显存分配、CPU-GPU 数据拷贝、kernel 启动和线程布局。对于常见线性代数任务，更实用的方式是调用成熟库。MAGMA (Matrix Algebra on GPU and Multicore Architectures) 把 LAPACK 风格的矩阵算法扩展到 GPU 与多核 CPU。

例如矩阵乘法可写成

$$C = \alpha \text{ op}(A) \text{ op}(B) + \beta C,$$

其中 α, β 为标量, $\text{op}(A)$ 可表示 A 、 A^T 或 $A^\dagger$ 。MAGMA 中类似 `magma_cgemm` 的函数用于复数单精度矩阵乘法。其计算物理意义是：本征值问题、基变换、密度矩阵演化、神经网络矩阵乘等大量任务最终都会落到高性能矩阵核上。

11.3.3 并行效率与通信瓶颈

并行效率 (parallel efficiency) 衡量使用 p 个处理单元时实际加速比与理想加速比的差距。若串行时间为 T_1 ，并行时间为 T_p ，则

$$S_p = \frac{T_1}{T_p}, \quad E_p = \frac{S_p}{p}.$$

理想情况下 $S_p = p$ 、 $E_p = 1$ 。实际中，通信、同步、负载不均衡、串行部分和内存带宽都会降低效率。

- Parallel efficiency: madness scale
communications cause latency and are not unlimited

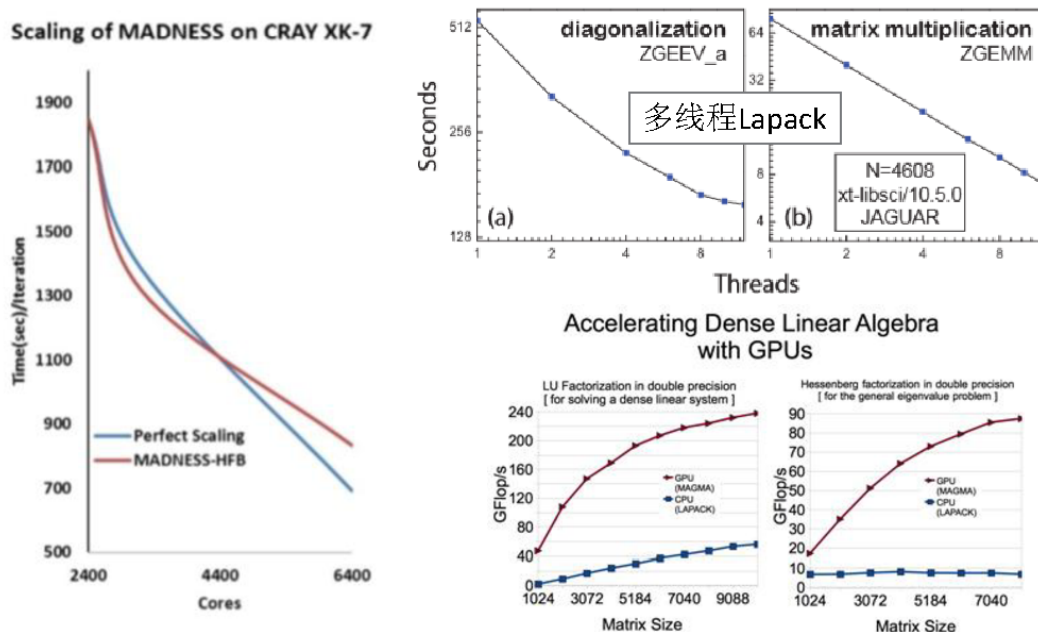


图 11.11: 并行效率与扩展性示意。MADNESS 等多分辨率框架能够展示大规模并行潜力，但通信延迟和同步开销始终限制强扩展效率。

Amdahl 定律给出一个基本限制：若程序中不可并行部分比例为 f ，则使用无限多处理器时最大加速比也只有

$$S_{\max} = \frac{1}{f}.$$

因此高性能程序设计不仅是增加线程或进程数，还要减少串行瓶颈、优化数据布局、降低通信量并提高算法的局部性。

11.4 量子计算原理

11.4.1 从经典比特到量子比特

经典比特 (classical bit) 只能处于 0 或 1 两种状态之一。量子比特 (qubit) 则可以处于基态的线性叠加：

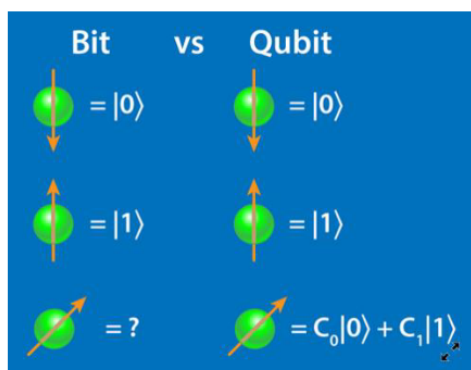
$$|\psi\rangle = c_0|0\rangle + c_1|1\rangle, \quad |c_0|^2 + |c_1|^2 = 1.$$

这里 $|0\rangle$ 与 $|1\rangle$ 是计算基, $c_0, c_1 \in \mathbb{C}$ 是概率振幅。测量时得到 0 的概率为 $|c_0|^2$, 得到 1 的概率为 $|c_1|^2$ 。因此量子计算不是一次性读出所有叠加分量, 而是通过么正演化、干涉和多次测量使目标答案具有更高概率。

若有 N 个经典比特, 系统一次只能处于 2^N 个状态中的一个; 而 N 个量子比特的一般态为

$$|\Psi\rangle = \sum_{j=0}^{2^N-1} c_j |j\rangle, \quad \sum_{j=0}^{2^N-1} |c_j|^2 = 1.$$

这说明 N 个量子比特的状态描述需要 2^N 个复振幅。量子计算的潜力来自叠加 (superposition)、纠缠 (entanglement) 和干涉 (interference), 但测量只能给出一个经典结果, 因此算法必须把有用信息编码到可观测概率分布中。

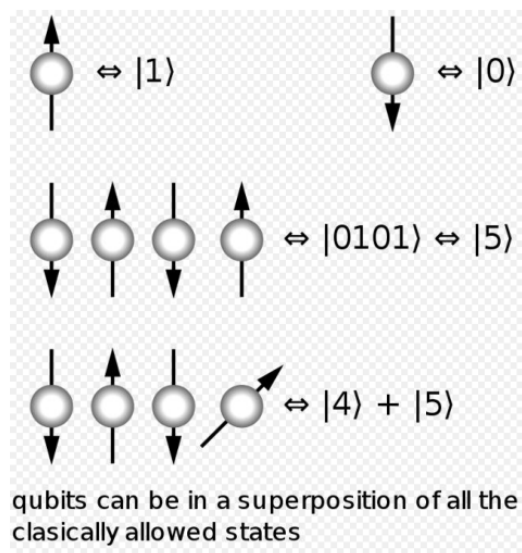


If you have N classical bits, then they will be in one state out of 2^N possible states, whereas N qubits could represent all those possible states at the same time.

Create large superposition states, entanglement, and interference

Spin up or down of atoms and others by two excited states in a superconducting circuit,

图 11.12: 经典比特与量子比特的对比。量子比特可写为 $c_0|0\rangle + c_1|1\rangle$, 多量子比特系统可表示指数多个计算基态的叠加。



量子比特的物理实现包括超导电路 (superconducting circuit)、离子阱 (ion trap) 和金刚石色心 (diamond color center) 等。超导量子比特可用电路的基态和激发态表示 $|0\rangle$ 、 $|1\rangle$, 离子阱可用离子的内部能级和集体振动模式实现量子门。

11.4.2 量子计算的优势、算法与路线图

量子计算尤其适合模拟强关联量子系统。经典计算机模拟量子多体波函数时, Hilbert 空间维数通常随粒子数指数增长; 如果直接使用量子系统表示量子系统, 在原则上可能把某些指数复杂度问题转化为多项式复杂度问题。

当前量子算法还不丰富，但已经出现若干具有代表性的方向：

- Shor 算法：利用量子傅里叶变换高效分解整数，可在多项式时间内解决大整数分解问题，威胁 RSA 加密；
- Grover 搜索：对无结构数据库搜索提供平方加速；
- 量子模拟 (quantum simulation)：直接模拟量子 Hamiltonian 的演化或能谱；
- 量子退火 (quantum annealing)：用于特定优化问题，D-Wave 公司的机器主要属于这一方向；
- VQE、QPE、量子线性方程组算法等：分别服务于变分能量估计、相位与本征值求解、线性系统求解。

D-Wave 是较早商业化量子计算公司的代表，其 D-Wave 2000Q 有 2048 个量子比特，但主要实现量子退火，而非通用门模型量子计算。通用量子计算路线仍受到量子比特数量、门错误率和退相干时间的限制。

● Roadmap of quantum computing

- Number of qubits; Error rate; Decoherence time

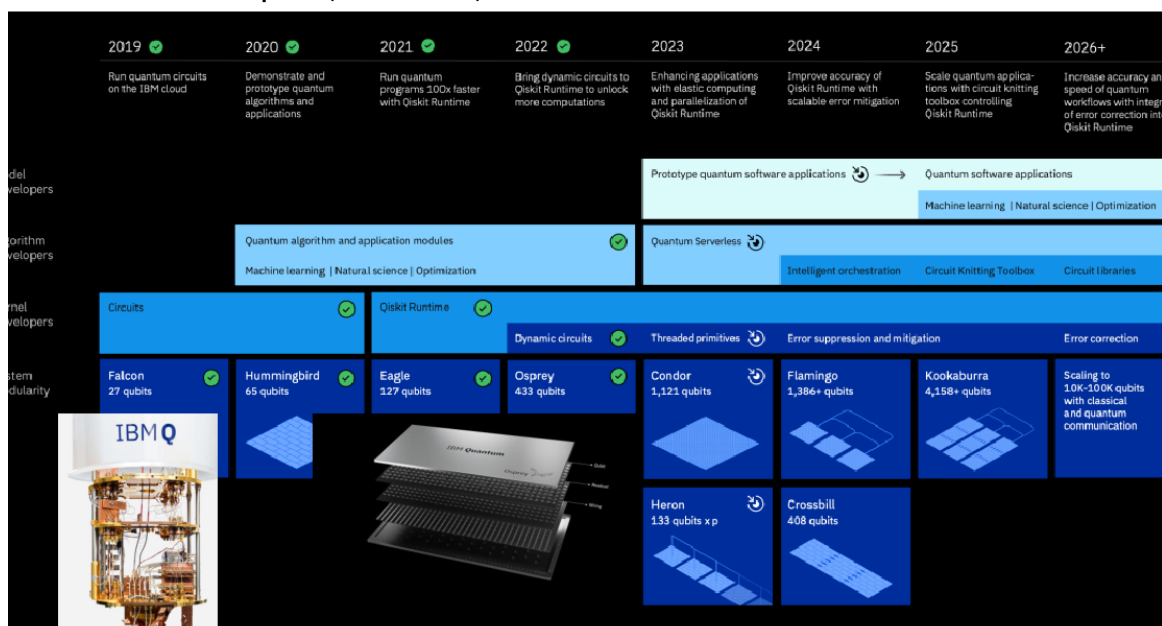


图 11.13: 量子计算路线图示意。量子比特数、错误率和退相干时间共同决定可运行算法的深度和可靠性。

云量子计算通过互联网访问 IBM、Google、Rigetti 等机构的量子芯片，使研究者可以远程提交量子线路。量子门 (quantum gate) 是对量子比特的操作，数学上由幺正矩阵表示。若初态为 $|0\rangle$ ，线路对应的总幺正为 U ，则末态为

$$|\psi\rangle = U|0\rangle.$$

这一定义要求 $U^\dagger U = I$ ，从而保持态矢量范数和总概率。

11.4.3 量子逻辑门

单量子比特计算基写为

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

两量子比特态可写为

$$|\psi\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle,$$

其中 $|a|^2 + |b|^2 + |c|^2 + |d|^2 = 1$ 。

Hadamard 门为

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

满足

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

它把计算基态变为等权叠加，是量子算法中制造相干叠加的基本操作。

Pauli 门为

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

其中 $X|0\rangle = |1\rangle$ ，相当于量子 NOT； Y 可理解为绕 y 轴的 π 旋转； Z 改变 $|1\rangle$ 分量相位。平方根 NOT 门可写为

$$\sqrt{\text{NOT}} = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix},$$

连续作用两次得到 NOT 门。

Controlled-NOT (CNOT) 是双量子比特门：当第一个量子比特为 $|1\rangle$ 时，对第二个量子比特做 NOT；当第一个量子比特为 $|0\rangle$ 时不操作。其典型作用是

$$|10\rangle \mapsto |11\rangle, \quad |11\rangle \mapsto |10\rangle.$$

常用门集合包括

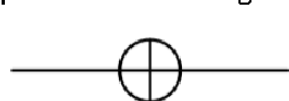
$$\{I, X, Y, Z, H, S, S^\dagger, T, T^\dagger, U_1(\lambda), U_2(\lambda, \phi), U_3(\lambda, \phi, \theta), \text{CNOT}\}.$$

● **Quantum logic gate** Basis: $|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ $|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ $a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle$

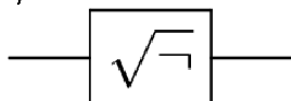
Hadamard (H) gate  $H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
 $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$

Pauli-X,Y,Z gate $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ $Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$ $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
 $X|0\rangle = |1\rangle$ Pauli Y gate is a π -rotation around Y axis

Square root of NOT gate ($\sqrt{\text{NOT}}$)

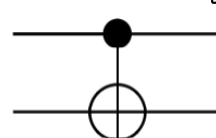


$$= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$



$$\sqrt{\text{NOT}} = \frac{1}{2} \begin{bmatrix} 1+i & 1-i \\ 1-i & 1+i \end{bmatrix}.$$

Controlled-NOT gate



performs the NOT operation on the second qubit only when the first qubit is $|1\rangle$

感兴趣还有继续参考更多逻辑门

可以在IBM提供的逻辑门编辑器模拟自己的问题

$\{I, X, Y, Z, H, S, S^\dagger, T, T^\dagger, U_1(\lambda), U_2(\lambda, \phi), U_3(\lambda, \phi, \theta), \text{CNOT}\}.$

图 11.14: 常见量子逻辑门。单量子比特门控制叠加与相位，CNOT 等受控门负责产生多量子比特关联。

$$|10\rangle \xrightarrow{\text{CNOT}} |11\rangle$$

11.4.4 CNOT、纠缠与测量

两个单量子比特态

$$|\Psi_1\rangle = \begin{pmatrix} \alpha_1 \\ \beta_1 \end{pmatrix}, \quad |\Psi_2\rangle = \begin{pmatrix} \alpha_2 \\ \beta_2 \end{pmatrix}$$

的直积为

$$|\Psi\rangle = |\Psi_1\rangle \otimes |\Psi_2\rangle = \begin{pmatrix} \alpha_1\alpha_2 \\ \alpha_1\beta_2 \\ \beta_1\alpha_2 \\ \beta_1\beta_2 \end{pmatrix}.$$

CNOT 在基 $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ 下的矩阵为

$$U_{\text{CNOT}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

它保持 $|00\rangle$ 、 $|01\rangle$ 不变，交换 $|10\rangle$ 与 $|11\rangle$ 。

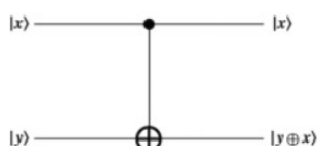
从 $|0\rangle \otimes |0\rangle$ 出发，先对第一个量子比特施加 $R_y(\theta)$ ，再施加 CNOT：

$$|0\rangle|0\rangle \xrightarrow{R_y(\theta) \otimes I} \left(\cos \frac{\theta}{2} |0\rangle + \sin \frac{\theta}{2} |1\rangle \right) |0\rangle \xrightarrow{\text{CNOT}} \cos \frac{\theta}{2} |00\rangle + \sin \frac{\theta}{2} |11\rangle.$$

当 $\theta = \pi/2$ 时得到最大纠缠 Bell 态

$$\frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

● $\Psi_1 = \begin{pmatrix} \alpha_1 \\ \beta_1 \end{pmatrix}, \Psi_2 = \begin{pmatrix} \alpha_2 \\ \beta_2 \end{pmatrix} \quad \Psi = \Psi_1 \otimes \Psi_2 = \begin{pmatrix} \alpha_1\alpha_2 \\ \alpha_1\beta_2 \\ \beta_1\alpha_2 \\ \beta_1\beta_2 \end{pmatrix}$



CNOT gate

$$U_{\text{CNOT}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

$|00\rangle, |01\rangle$
 $|10\rangle$ and $|11\rangle$

★ 量子纠缠的实现

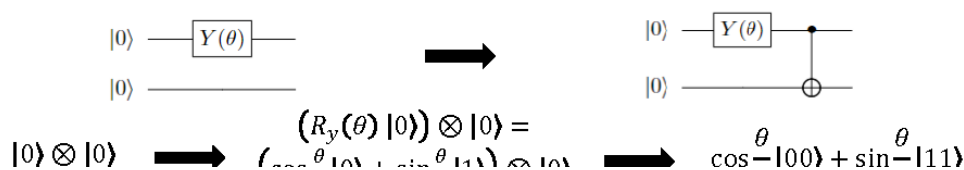


图 11.15: CNOT 与纠缠制备。受控门把单比特叠加转化为多比特量子关联。

Bell 态也可由 $|00\rangle$ 经 Hadamard 和 CNOT 制备：

$$|00\rangle \xrightarrow{H \otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

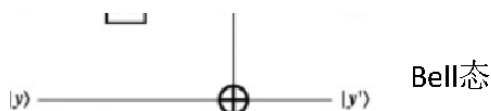
三量子比特 GHZ 态可按以下步骤得到：

$$|000\rangle \xrightarrow{H_0} \frac{|000\rangle + |100\rangle}{\sqrt{2}} \xrightarrow{\text{CNOT}_{0,1}} \frac{|000\rangle + |110\rangle}{\sqrt{2}} \xrightarrow{\text{CNOT}_{1,2}} \frac{|000\rangle + |111\rangle}{\sqrt{2}}.$$

测量在计算基

$$V = \{|000\rangle, |001\rangle, \dots, |111\rangle\}$$

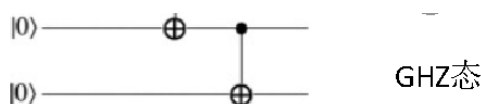
上进行时，GHZ 态只在 $|000\rangle$ 与 $|111\rangle$ 上有非零概率。



输入为 $|x\rangle = |0\rangle$ 且 $|y\rangle = |0\rangle$ ：

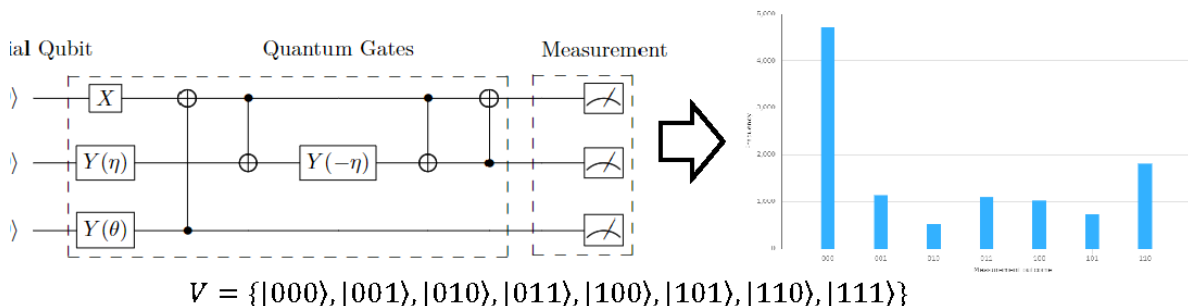
经过 H 门后，状态变为： $\frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |0\rangle = \frac{|00\rangle + |10\rangle}{\sqrt{2}}$

经过 CNOT 门后，状态变为： $\frac{|00\rangle + |11\rangle}{\sqrt{2}}$



1. 初始状态电路： $|\psi_0\rangle = |000\rangle$
2. Hadamard 门 (H)： $|\psi_1\rangle = \frac{1}{\sqrt{2}} (|000\rangle + |100\rangle)$
3. 第一个 CNOT 门： $\frac{1}{\sqrt{2}} (|000\rangle + |110\rangle)$
4. 第二个 CNOT 门： $\frac{1}{\sqrt{2}} (|000\rangle + |111\rangle)$

量子测量原理



$$V = \{|000\rangle, |001\rangle, |010\rangle, |011\rangle, |100\rangle, |101\rangle, |110\rangle, |111\rangle\}$$

图 11.16: Bell 态、GHZ 态与测量直方图。纠缠态的测量结果具有强关联，而不是各量子比特独立随机。

11.4.5 量子程序与真实量子芯片

用量子计算机求解问题的基本流程可以概括为三步：

1. 把待求问题写成幺正矩阵、Hamiltonian 或可测量算符；
2. 把这些矩阵分解为目标硬件可以实现的量子门；
3. 优化线路，尽可能减少门数、线路深度和噪声影响。

量子编程框架如 pyQuil、Qiskit 等允许用户用 Python 构造线路。例如制备 Bell 态的 pyQuil 程序本质上就是调用 H 和 CNOT 门。

量子化学示例中，可以用 Qiskit 定义 LiH 分子的基组、轨道冻结、qubit mapping、VQE 算法、COBYLA 优化器和 UCCSD 变形式，从而计算基态能量与偶极矩。这体现了云量子计算的典型工作流：

物理体系 $\rightarrow$ 二次量子化 Hamiltonian $\rightarrow$ qubit mapping $\rightarrow$ ansatz $\rightarrow$ 量子测量 $\rightarrow$ 经典优化。

真实量子芯片的限制主要来自 CNOT 门误差、单比特门误差、读出误差和退相干。以 IBM 超导芯片为例，设备会给出量子比特数、量子体积、基门集合、CNOT 误差、读出误差、 T_1 与 T_2 时间等指标。量子线路越深、

● Steps to solve problems using quantum computers

(i) formulate the problem to be solved in terms of unitary matrices

(ii) rewrite those matrices in terms of gates that can be realized on a given quantum computer

(iii) implement and try to improve the efficiency of (ii), reducing the number of gates as much as possible—given that a very small number of gates is enough to implement almost any unitary matrix

Quantum programming in Python

```
from pyquil.quil import Program
from pyquil.gates import H, CNOT
from pyquil.api import QVMConnection
# construct a Bell State program
p = Program(H(0), CNOT(0, 1))
# run the program on a QVM
qvm = QVMConnection()
```

图 11.17: 量子计算求解问题的程序化流程。问题需要先转化为门模型可执行的幺正操作，再交给模拟器或量子芯片运行。

```
aqua_chemistry_dict = {
    "driver": { "name": "PYSCHF" },
    "PYSCHF": { "atom": "", "basis": "sto3g" },
    "operator": {
        "name": "hamiltonian",
        "qubit_mapping": "parity",
        "two_qubit_reduction": True,
        "freeze_core": True,
        "orbital_reduction": [-3, -2]
    },
    "algorithm": { "name": "VQE" },
    "optimizer": { "name": "COBYLA", "maxiter": 10000 },
    "variational_form": { "name": "UCCSD" },
    "initial_state": { "name": "HartreeFock" }
}
molecule = "H .0 .0 {-0}; Li .0 .0 {0}"

pts = [x * 0.1 for x in range(6, 20)]
pts += [x * 0.25 for x in range(8, 16)]
pts += [4.0]
energies = np.empty(len(pts))
distances = np.empty(len(pts))
dipoles = np.empty(len(pts))

for i, d in enumerate(pts):
    aqua_chemistry_dict["PYSCHF"]["atom"] = molecule.format(d/2)
    solver = AquaChemistry()
    result = solver.run(aqua_chemistry_dict)
    energies[i] = result["energy"]
    dipoles[i] = result["total_dipole_moment"] / 0.393430307
    distances[i] = d

for j in range(len(distances)):
```

From IBM
Calculate LiH 基态能量
与偶极矩

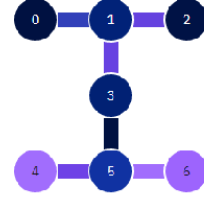
图 11.18: IBM/Qiskit 中 LiH 基态能量与偶极矩计算的代码示例。量子化学问题先映射为量子比特 Hamiltonian，再用 VQE 优化参数。

CNOT 越多，噪声累积越明显。

ibm_oslo

OpenQASM 3

7	Status:	● Online	Median CNOT Error:	8.556e-3
Qubits	Total pending jobs:	21 jobs	Median SX Error:	2.668e-4
32	Processor type ⓘ:	Falcon r5.11H	Median Readout Error:	2.080e-2
QV	Version:	1.0.21	Median T1:	184.43 us
2.6K	Basis gates:	CX, ID, RZ, SX, X	Median T2:	41.99 us
CLOPS				

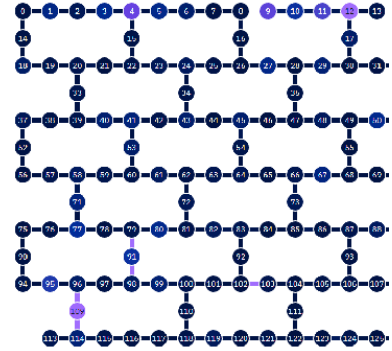


ibm_washington

OpenQASM 3

Details

127	Status:	● Online	Median CNOT Error:	1.187e-2
Qubits	Total pending jobs:	34 jobs	Median SX Error:	2.892e-4
64	Processor type ⓘ:	Eagle r1	Median Readout Error:	1.520e-2
QV	Version:	1.6.17	Median T1:	100.85 us
850	Basis gates:	CX, ID, RZ, SX, X	Median T2:	86.04 us
CLOPS				



超导量子计算机误差来源：CNOT门，Readout退相干

图 11.19: 真实 IBM 量子设备的指标示意。CNOT 门误差、读出误差和退相干是当前超导量子计算的主要误差来源。

11.5 量子多体问题的云量子计算

11.5.1 配对 Hamiltonian 的量子比特映射

核结构和凝聚态中的配对相互作用可用 seniority model 的 Hamiltonian 描述：

$$\hat{H} = -G \sum_{m, m' > 0} a_{m'}^{\dagger} a_{-m'}^{\dagger} a_{-m} a_m = -G \sum_{m, m' > 0} s_{+}^{(m')} s_{-}^{(m)}.$$

这里 G 是配对强度， $a_m^{\dagger}$ 、 a_m 是费米子产生与湮灭算符， $(m, -m)$ 表示一对时间反演轨道， $s_{+}^{(m)}$ 、 $s_{-}^{(m)}$ 是准自旋升降算符。该 Hamiltonian 描述成对粒子在不同简并轨道之间的散射。

映射到 qubit 基后，可写为

$$\hat{H} = -G \sum_{p, q > 0} s_{+}^{(p)} s_{-}^{(q)}.$$

准自旋算符用 Pauli 矩阵表示：

$$s_{+}^{(p)} s_{-}^{(p)} \longrightarrow \frac{1}{2} (I^{(p)} + \sigma_z^{(p)}),$$

$$s_{+}^{(p)} s_{-}^{(q)} \longrightarrow \frac{1}{2} (\sigma_x^{(p)} \otimes \sigma_x^{(q)} + \sigma_y^{(p)} \otimes \sigma_y^{(q)}), \quad p \neq q.$$

这说明多体 Hamiltonian 可以分解成 Pauli 字符串之和：

$$\hat{H} = \sum_{\alpha} h_{\alpha} P_{\alpha}, \quad P_{\alpha} \in \{I, X, Y, Z\}^{\otimes n}.$$

量子计算中测量能量就是逐项估计 $\langle P_{\alpha} \rangle$ ，再按系数 h_{α} 求和。

以 $j = 5/2$ 壳层为例，共有 6 条单粒子轨道，即 $\Omega = 3$ 对简并轨道。其 qubit Hamiltonian 可写为

$$\begin{aligned}\hat{H}(\Omega = 3) = & -\frac{G}{2}(3I \otimes I \otimes I + \sigma_z \otimes I \otimes I + I \otimes \sigma_z \otimes I + I \otimes I \otimes \sigma_z \\ & + \sigma_x \otimes \sigma_x \otimes I + \sigma_x \otimes I \otimes \sigma_x + I \otimes \sigma_x \otimes \sigma_x \\ & + \sigma_y \otimes \sigma_y \otimes I + \sigma_y \otimes I \otimes \sigma_y + I \otimes \sigma_y \otimes \sigma_y).\end{aligned}$$

这个例子清楚展示了从费米子多体问题到 qubit Hamiltonian 的路径：先识别物理自由度，再用 Pauli 算符表示产生、湮灭或准自旋结构。

11.5.2 VQE、UCC 与 EPA

变分量子本征求解器 (variational quantum eigensolver, VQE) 是 NISQ 阶段最常用的基态能量算法之一。它利用变分原理：

$$E(\vec{\theta}) = \langle \psi(\vec{\theta}) | \hat{H} | \psi(\vec{\theta}) \rangle \geq E_{\text{gs}},$$

其中 $|\psi(\vec{\theta})\rangle$ 是由参数化量子线路制备的试探态， $\vec{\theta}$ 是经典优化器调整的参数， E_{gs} 是精确基态能量。VQE 的基本循环是：

1. 把 $\hat{H}$ 映射到 Pauli 字符串，例如 Jordan-Wigner 或 Bravyi-Kitaev 映射；
2. 在量子芯片上制备 ansatz 态 $|\psi(\vec{\theta})\rangle$ ；
3. 测量每个 Pauli 项的期望值，得到 $E(\vec{\theta})$ ；
4. 在经典计算机上更新 $\vec{\theta}$ ，重复直到能量收敛。

VQE 所需量子线路相对浅，适合当前有噪声的中等规模量子设备。相比之下，Rodeo algorithm、QPE 和量子虚时演化等算法通常需要更深线路或更高相干性。

量子化学和核多体计算中常用 unitary coupled cluster (UCC) ansatz：

$$\begin{aligned}|\psi(\vec{\theta})\rangle &= \exp \left[i \left(\hat{T}(\vec{\theta}) - \hat{T}^\dagger(\vec{\theta}) \right) \right] |\psi_0\rangle, \\ \hat{T} &= \hat{T}_1 + \hat{T}_2 + \cdots = \sum_{a,i} \theta_i^a \hat{c}_a^\dagger \hat{c}_i + \sum_{ab,ij} \theta_{ij}^{ab} \hat{c}_a^\dagger \hat{c}_b^\dagger \hat{c}_j \hat{c}_i + \cdots.\end{aligned}$$

这里 $|\psi_0\rangle$ 通常是 Hartree-Fock 参考态， i, j 表示占据轨道， a, b 表示未占据轨道。UCC 的优点是物理意义明确，保留粒子-空穴激发结构；缺点是映射到量子线路后深度较大，对噪声敏感。

Entangled pair ansatz (EPA) 直接在 qubit 空间中构造保粒子数的纠缠块：

$$\begin{aligned}|\psi(\vec{\theta})\rangle &= \prod_i U_i(\theta_i) |\psi_0\rangle, \\ U_i(\theta_i) &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \sin \theta_i & \cos \theta_i & 0 \\ 0 & \cos \theta_i & -\sin \theta_i & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.\end{aligned}$$

这个二量子比特操作只在 $|01\rangle$ 、 $|10\rangle$ 子空间中混合态，因此可保持粒子数或 S_z 量子数。与 UCC 相比，EPA 更贴近特定配对模型，线路较浅，但通用性较弱。

11.5.3 量子测量与 Pauli 基变换

VQE 的核心成本常常不在制备态，而在测量 Hamiltonian 的期望值。若

$$\hat{H} = \sum_{\alpha} h_{\alpha} P_{\alpha},$$

则

$$\langle \hat{H} \rangle = \sum_{\alpha} h_{\alpha} \langle P_{\alpha} \rangle.$$

★ The Excitation-Preserving ansatz(EPA)

- 相较于UCC作用于物理态空间，该策略直接作用于Qubit空间。
- 类似JW变换，粒子数直接对应Qubit空间的总自旋 z 分量 S_z 。

$$|\psi(\vec{\theta})\rangle = \prod_i U_i(\theta_i) |\psi_0\rangle$$

$$U_i(\theta_i) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (8)$$

对2比特操作！

- 每个building block代表一个保持粒子数的么正变分变换。
- 在量子系统中应用这些变换，确保在操作过程中保持激发态。

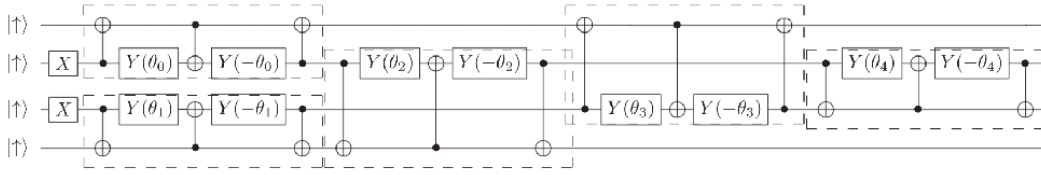


图 11.20: EPA 变分线路示意。粒子数保持的双量子比特块可在较浅线路中构造配对相关。

量子芯片通常天然在 Z 基测量；若要测量 X 或 Y ，需要先做基变换：

$$H^\dagger X H = Z, \quad S^\dagger Y S = X,$$

等价地

$$X = H Z H^\dagger, \quad Y = S H Z H S^\dagger.$$

因此测量 X 时先施加 H 再做 Z 基测量；测量 Y 时先做相位门和 Hadamard 组合，再测量。

若两个 Pauli 字符串对易，它们可以共享本征基，有机会在同一组线路中同时测量。Hamiltonian 项数很多时，按对易关系分组可以显著减少测量次数。测量误差来自有限 shots 的统计误差和硬件噪声；前者随 shots 数 N_s 约按 $1/\sqrt{N_s}$ 降低，后者需要误差缓解或纠错。

11.5.4 配对模型结果、误差缓解与激发态

三对简并轨道配对模型的基态可用量子模拟与真实量子芯片计算。理想 Qiskit 模拟给出

$$E_0 \simeq -4.013G,$$

$$|gs\rangle_{\text{sim}} = 0.588|\uparrow\downarrow\rangle + 0.554|\uparrow\uparrow\rangle + 0.590|\downarrow\uparrow\rangle.$$

精确对角化结果为

$$E_0 = -4G, \quad |gs\rangle_{\text{ext}} = 0.577(|\uparrow\downarrow\rangle + |\uparrow\uparrow\rangle + |\downarrow\uparrow\rangle).$$

真实 IBM_oslo 量子芯片给出的能量约为

$$E_0 \simeq -3.751G,$$

与精确值的偏差主要来自门误差、读出误差和退相干。该例说明 NISQ 设备已经可以演示小规模核模型，但精确科学计算仍依赖误差控制。

误差来源可分为：

- decoherence：量子态与环境耦合导致相干性损失；

- gate error: 单比特门和双比特门实现不完美;
- readout error: 真实态为 $|0\rangle$ 却读成 1, 或真实态为 $|1\rangle$ 却读成 0。

单量子比特读出误差可用校准矩阵表示:

$$S_k = \begin{pmatrix} P_{0,0}^{(k)} & P_{0,1}^{(k)} \\ P_{1,0}^{(k)} & P_{1,1}^{(k)} \end{pmatrix},$$

其中 $P_{i,j}^{(k)}$ 表示第 k 个量子比特真实为 j 时读出为 i 的概率。若 S_k 可逆, 可用 S_k^{-1} 对测量分布做读出误差修正。

零噪声外推 (zero-noise extrapolation) 通过人为放大线路噪声再外推到零噪声极限。例如利用

$$\text{CNOT} \cdot \text{CNOT} = I$$

插入成对 CNOT 门, 在不改变理想线路的情况下增加噪声强度, 然后对不同噪声强度下的能量做外推。

- 误差来源: 退相干, 量子门错误率, 读出误差.....
- 读出误差 $P(0|1), P(1|0) \rightarrow$ 误差矩阵的逆

$$S_k = \begin{pmatrix} P_{0,0}^k & P_{0,1}^k \\ P_{1,0}^k & P_{1,1}^k \end{pmatrix}$$

- 零噪声外推(zero-noise extrapolation)

$$\text{CNOT} \cdot \text{CNOT} = I$$

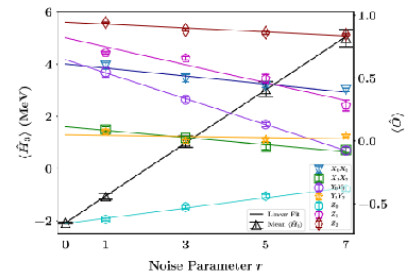


FIG. 3. Noise extrapolation of the $N=3$ qubit problem run on the QX5. The H_3 energy (left axis, black line) and individual Pauli expectation values (right axis) are given as a function of the number CNOT gate scaling factor r .

Eigenstate	$ \psi\rangle$	$E(\text{Qiskit})$	$E(\text{IBM_oslo})$	$E(\text{R-Miti})$	$E(\text{ZNE})$	$E(\text{R-Miti} + \text{ZNE})$
g.s.	$0.588 \uparrow\uparrow\downarrow\rangle + 0.554 \uparrow\downarrow\uparrow\rangle + 0.590 \downarrow\uparrow\uparrow\rangle$	-4.013	-3.751	-3.871	-3.919	-4.123
1st es	$0.309 \uparrow\uparrow\downarrow\rangle + 0.651 \uparrow\downarrow\uparrow\rangle - 0.693 \downarrow\uparrow\uparrow\rangle$	-1.024	-1.126	-1.107	-1.093	-1.064
	$0.809 \uparrow\uparrow\downarrow\rangle - 0.569 \uparrow\downarrow\uparrow\rangle - 0.146 \downarrow\uparrow\uparrow\rangle$	-1.004	-1.109	-1.086	-1.006	-0.978

E.F. Dumitrescu, et al, Phys. Rev. Lett. 120, 210501 (2018)

Chongji Jiang and Junchen Pei, Phys. Rev. C 107, 044308 (2023)

图 11.21: 读出误差修正与零噪声外推示意。误差缓解不等同于量子纠错, 但能在 NISQ 设备上改善期望值估计。

激发态可用变分量子偏转 (variational quantum deflation, VQD) 算法求解。若已知较低的 k 个本征态

$$|0\rangle, |1\rangle, \dots, |k-1\rangle,$$

则构造有效 Hamiltonian

$$\hat{H}_k = \hat{H} + \sum_{j=0}^{k-1} \lambda_j |j\rangle\langle j|,$$

并选取足够大的 λ_j , 使 $E_j + \lambda_j > E_k$ 。此时原 Hamiltonian 的第 k 个激发态会成为 $\hat{H}_k$ 的基态, 可再用 VQE 求解。等价的惩罚型目标函数为

$$E_k(\vec{\theta}) = \langle \psi(\vec{\theta}) | \hat{H} | \psi(\vec{\theta}) \rangle + \sum_{j < k} \beta_j |\langle \psi(\vec{\theta}) | \psi_j \rangle|^2.$$

VQD 与基态 VQE 使用相同量子比特数, 但需要额外的重叠测量, 线路深度和测量成本约增加。

11.5.5 有限量子体系与热激发

有限量子体系中的热激发可用于研究 $\Omega = 3$ 、 $\Omega = 4$ 等小规模配对模型。有限体系中 BCS 平均场近似可能失效, 热激发和配对关联之间的竞争可与 pseudogap 等现象联系起来。量子计算可以通过非本征态热激发构造、

★ VQD算法

➤ 构造有效哈密顿量如下：

$$\begin{aligned} E(\lambda_k) &\equiv \langle \psi(\lambda_k) | \hat{H} | \psi(\lambda_k) \rangle + \sum_{j=0}^{k-1} \beta_j |\langle \psi(\lambda_k) | \psi(\lambda_j) \rangle|^2 \\ \hat{H}(k) &\equiv \hat{H} + \sum_{j=0}^{k-1} \lambda_j |j\rangle\langle j| \end{aligned} \quad (10)$$

➤ 挑选足够大的 λ_j , $j \in \{0, 1, \dots, k-1\}$, 使得 $E_j + \lambda_j > E_k$ 。

⇒ $|k\rangle$ 成为有效哈密顿量的基态, 可以使用VQE算法得到 $|k\rangle$ 。

➤ 量子硬件资源节省, 基态相同的qubit数目和2倍的线路深度

O. Higgott, D. Wang, S. Brierley, Quantum 3, 156 (2019).

图 11.22: VQD 激发态算法示意。通过对低能态加入惩罚项, 可以把目标激发态转化为有效 Hamiltonian 的基态。

读出误差修正和 CNOT 外推等方法, 探索有限量子系统的热性质。

这一方向的意义在于: 核多体、裂变动力学、格点 QCD 等问题都具有强量子关联和巨大 Hilbert 空间。即使当前量子设备规模有限, 小模型也能帮助验证从 Hamiltonian 映射、ansatz 设计到误差缓解的完整流程。

11.5.6 氘核束缚能的云量子计算

氘核 (deuteron) 是一个质子和一个中子组成的最简单原子核, 可作为量子计算核物理的基准问题。费米子产生湮灭算符可通过 Jordan-Wigner transformation 映射到 Pauli 矩阵:

$$\begin{aligned} a_n^\dagger &\longrightarrow \frac{1}{2} \left[\prod_{j=0}^{n-1} (-Z_j) \right] (X_n - iY_n), \\ a_n &\longrightarrow \frac{1}{2} \left[\prod_{j=0}^{n-1} (-Z_j) \right] (X_n + iY_n). \end{aligned}$$

前面的 Z 字符串保证不同费米子模式之间满足正确的反对易关系。 N 条轨道下的 Hamiltonian 写为

$$H_N = \sum_{n,n'=0}^{N-1} \langle n' | (T + V)_N | n \rangle a_n^\dagger a_n,$$

其中 T 为动能, V 为相互作用势。

映射后, 小模型 Hamiltonian 可写成 Pauli 字符串。例如:

$$H_1 = 0.218291(Z_0 - I),$$

$$H_2 = 5.906709I + 0.218291Z_0 - 6.125Z_1 - 2.143304(X_0X_1 + Y_0Y_1),$$

$$H_3 = H_2 + 9.625(I - Z_2) - 3.913119(X_1X_2 + Y_1Y_2).$$

这些式子中的系数来自选定模型空间和核相互作用矩阵元。计算物理意义是: 连续核多体问题经过基截断后变为有限维 Hamiltonian, 再经过 Jordan-Wigner 映射变为量子线路可测的 Pauli 和。

● 算符转换

E. F. Dumitrescu, et al., PRL 120, 210501-2018

$$a_n^\dagger \rightarrow \frac{1}{2} \left[\prod_{j=0}^{n-1} -Z_j \right] (X_n - iY_n),$$

Jordan–Wigner transformation

$$a_n \rightarrow \frac{1}{2} \left[\prod_{j=0}^{n-1} -Z_j \right] (X_n + iY_n).$$

Pauli matrices $\leftrightarrow$ fermionic creation and annihilation operators 满足对易规则

quantum computation of the deuteron binding energy(one proton, one neutron)

$$H_N = \sum_{n,n'=0}^{N-1} \langle n' | (T + V) | n \rangle a_n^\dagger a_n. \quad N \text{ 条轨道}$$

$$H_1 = 0.218\,291(Z_0 - I) \quad E_1 = \langle \downarrow | H_1 | \downarrow \rangle$$

$$H_2 = 5.906\,709I + 0.218\,291Z_0 - 6.125Z_1 \\ - 2.143\,304(X_0X_1 + Y_0Y_1),$$

$$H_3 = H_2 + 9.625(I - Z_2) - 3.913\,119(X_1X_2 + Y_1Y_2).$$

$$\begin{aligned} a_0^\dagger a_1 + a_0^\dagger a_0 + a_1^\dagger a_0 + a_1^\dagger a_1 \\ [\sigma_a, \sigma_b] = 2i\varepsilon_{abc} \sigma_c, \\ \{\sigma_a, \sigma_b\} = 2\delta_{ab} I. \end{aligned}$$

图 11.23: 氘核束缚能计算中的 Jordan-Wigner 映射与 Pauli Hamiltonian。费米子算符被转化为量子比特上的 Pauli 字符串。

氘核 VQE 使用能够纠缠两条或三条轨道的么正算符。例如两轨道情形可取

$$U(\theta) = e^{\theta(a_0^\dagger a_1 - a_1^\dagger a_0)} = e^{i(\theta/2)(X_0Y_1 - X_1Y_0)}.$$

三轨道情形可用

$$U(\eta, \theta) = e^{\eta(a_0^\dagger a_1 - a_1^\dagger a_0) + \theta(a_0^\dagger a_2 - a_2^\dagger a_0)}$$

并近似分解为 Pauli 旋转门乘积。量子线路制备

$$|\varphi(\vec{\theta})\rangle = U(\vec{\theta})|\varphi(0)\rangle,$$

再最小化

$$\langle H \rangle = \langle \varphi(\vec{\theta}) | H | \varphi(\vec{\theta}) \rangle.$$

在量子芯片上, Hamiltonian 的每个 Pauli 项都要通过重复测量估计。量子测量是随机的, shots 越多统计误差越小; 但线路越深、门越多, 硬件噪声越强。未来可能结合量子计算、神经网络和 neuromorphic computing, 发展更适合噪声硬件的新算法。当前重要算法包括 Shor 整数分解、Grover 搜索、量子模拟退火、VQE、QFT、量子相位估计和量子线性方程组算法; 主要挑战仍是噪声。

11.6 量子傅里叶变换与量子相位估计

11.6.1 计算基与整数编码

对 n 个量子比特, 计算基态写为

$$|q_0 q_1 \cdots q_{n-1}\rangle = |q_0\rangle_0 \otimes |q_1\rangle_1 \otimes \cdots \otimes |q_{n-1}\rangle_{n-1} \equiv |j\rangle,$$

其中 $q_i \in \{0, 1\}$ 。若采用大端序编码,

$$j = q_0 2^{n-1} + q_1 2^{n-2} + \cdots + q_{n-1} 2^0.$$

● 波函数变分 variational quantum eigensolver (VQE)

unitary operators entangling two and three orbitals

波函数转动一个角

$$U(\theta) \equiv e^{\theta(a_0^\dagger a_1 - a_1^\dagger a_0)} = e^{i(\theta/2)(X_0 Y_1 - X_1 Y_0)},$$

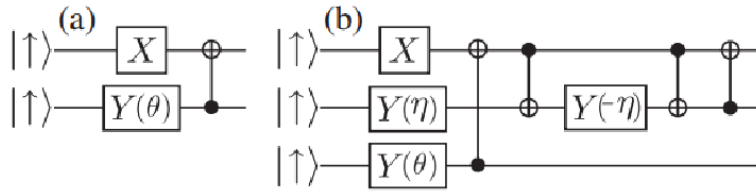
$$U(\vec{\theta}) = \exp(-i\vec{\theta} \cdot$$

$$U(\eta, \theta) \equiv e^{\eta(a_0^\dagger a_1 - a_1^\dagger a_0) + \theta(a_0^\dagger a_2 - a_2^\dagger a_0)}$$

$$R_x(\theta) \equiv e^{-i\theta X/2} = \cos(\theta/2)I - i\sin(\theta/2)$$

$$R_y(\theta) \equiv e^{-i\theta Y/2} = \cos(\theta/2)I - i\sin(\theta/2)$$

$$\approx e^{i(\eta/2)(X_0 Y_1 - X_1 Y_0)} e^{i(\theta/2)(X_0 Z_1 Y_2 - X_2 Z_1 Y_0)}.$$



$$\begin{array}{c} |A\rangle \\ |B\rangle \end{array} \xrightarrow{\text{CNOT}} \begin{array}{c} |A\rangle \\ |B \oplus A\rangle \end{array}$$

$$|\varphi(\vec{\theta}_0)\rangle = U(\vec{\theta}_0)|\varphi(0)\rangle. \quad \langle H \rangle = \langle \varphi(\vec{\theta}_0) | H | \varphi(\vec{\theta}_0) \rangle$$

相位操作

adjusts the parameters θ to minimize $\langle H \rangle$

E. F. Dumitrescu, et al., PRL 120, 210501-2018

图 11.24: 氦核 VQE 线路示意。参数化么正操作旋转波函数，并由经典优化器调节参数以最小化能量期望。

例如

$$|00 \cdots 0\rangle \equiv |0\rangle, \quad |00 \cdots 1\rangle \equiv |1\rangle, \quad |11 \cdots 1\rangle \equiv |2^n - 1\rangle.$$

这个编码把量子比特串与整数 j 对应起来，是 QFT 和 QPE 推导的基础。

11.6.2 从 DFT 到 QFT

离散傅里叶变换 (discrete Fourier transform, DFT) 把序列 f_j 变为频域序列 h_k :

$$h_k = \frac{1}{N} \sum_{j=0}^{N-1} \exp\left(i \frac{2\pi jk}{N}\right) f_j, \quad k = 0, \dots, N-1.$$

若用单位 ary 规范，更常写成

$$h_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \exp\left(i \frac{2\pi jk}{N}\right) f_j.$$

量子傅里叶变换 (quantum Fourier transform, QFT) 必须是么正变换，因此采用 $1/\sqrt{N}$ 规范。令 $N = 2^n$ ，QFT 对计算基态的定义为

$$\hat{F}|j\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} \exp\left(i \frac{2\pi jk}{2^n}\right) |k\rangle.$$

矩阵元为

$$F_{mj} = \langle m | \hat{F} | j \rangle = \frac{1}{\sqrt{2^n}} \exp\left(i \frac{2\pi jm}{2^n}\right),$$

$$(F^\dagger)_{mj} = \frac{1}{\sqrt{2^n}} \exp\left(-i \frac{2\pi jm}{2^n}\right), \quad F^\dagger F = I.$$

对任意态

$$|\varphi\rangle = \sum_{j=0}^{2^n-1} f(j)|j\rangle,$$

QFT 后系数变为

$$\tilde{f}(k) = \frac{1}{\sqrt{2^n}} \sum_{j=0}^{2^n-1} \exp\left(i\frac{2\pi jk}{2^n}\right) f(j).$$

11.6.3 QFT 的乘积形式

令整数 k 的二进制展开为

$$k = \sum_{l=0}^{n-1} k_l 2^{n-1-l}.$$

则

$$|k\rangle = |k_0 k_1 \cdots k_{n-1}\rangle.$$

将 QFT 展开为各量子比特的乘积, 可得

$$\hat{F}|j\rangle = \frac{1}{\sqrt{2^n}} \bigotimes_{l=0}^{n-1} \left(|0\rangle_l + \exp\left(i\frac{2\pi j}{2^{l+1}}\right) |1\rangle_l \right).$$

若

$$j = j_0 j_1 \cdots j_{n-1}$$

为二进制表示, 则

$$\exp\left(i\frac{2\pi j}{2^{l+1}}\right) = \exp(i2\pi 0.j_{n-l-1}j_{n-l} \cdots j_{n-1}),$$

其中 $0.j_{n-l-1} \cdots j_{n-1}$ 是二进制小数。因此

$$\begin{aligned} \hat{F}|j\rangle &= \frac{1}{\sqrt{2^n}} \left(|0\rangle_0 + e^{i2\pi 0.j_{n-1}} |1\rangle_0 \right) \left(|0\rangle_1 + e^{i2\pi 0.j_{n-2}j_{n-1}} |1\rangle_1 \right) \\ &\quad \cdots \left(|0\rangle_{n-1} + e^{i2\pi 0.j_0 j_1 \cdots j_{n-1}} |1\rangle_{n-1} \right). \end{aligned}$$

乘积形式的物理意义是: QFT 可由 Hadamard 门和一系列受控相位门实现, 而不需要显式构造 $2^n \times 2^n$ 矩阵。

11.6.4 QFT 线路实现

Hadamard 门对计算基态 $|x_k\rangle$ 的作用可写为

$$H|x_k\rangle = \frac{1}{\sqrt{2}} \left(|0\rangle + \exp\left(2\pi i \frac{x_k}{2}\right) |1\rangle \right).$$

当 $x_k = 0$ 时得到 $(|0\rangle + |1\rangle)/\sqrt{2}$, 当 $x_k = 1$ 时得到 $(|0\rangle - |1\rangle)/\sqrt{2}$ 。

相位门为

$$P(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}, \quad \theta = \frac{2\pi}{2^k}.$$

受控相位门满足

$$CP(\theta)|0x_j\rangle = |0x_j\rangle,$$

$$CP(\theta)|1x_j\rangle = |1\rangle \otimes P(\theta)|x_j\rangle = \exp\left(2\pi i \frac{x_j}{2^k}\right) |1x_j\rangle.$$

从 $|x_0 x_1 \cdots x_{n-1}\rangle$ 出发, QFT 实现步骤为:

1. 对第一个量子比特施加 H :

$$\frac{1}{\sqrt{2}} \left(|0\rangle + e^{2\pi i x_0/2} |1\rangle \right) \otimes |x_1 \cdots x_{n-1}\rangle.$$

2. 从第二个量子比特向第一个量子比特施加受控相位门，使相位变为

$$e^{2\pi i x_1/2^2 + 2\pi i x_0/2}.$$

3. 依次使用后续量子比特控制第一个量子比特，最终得到

$$\frac{1}{\sqrt{2}} \left(|0\rangle + \exp\left(2\pi i \frac{x}{2^n}\right) |1\rangle \right) \otimes |x_1 \cdots x_{n-1}\rangle,$$

其中

$$x = \sum_{l=0}^{n-1} x_l 2^{n-1-l}.$$

4. 对剩余量子比特重复同样过程，最后用 SWAP 门反转量子比特顺序。

最终形式为

$$\frac{1}{\sqrt{2^n}} \left(|0\rangle_0 + e^{2\pi i j/2^n} |1\rangle_0 \right) \left(|0\rangle_1 + e^{2\pi i j/2^{n-1}} |1\rangle_1 \right) \cdots \left(|0\rangle_{n-1} + e^{2\pi i j/2} |1\rangle_{n-1} \right),$$

这与 QFT 的乘积形式一致，只是量子比特顺序反向。QFT 的作用可以理解为把整数 j 中的信息转换为相位信息，是 Shor 算法和量子相位估计的核心模块。

➤ 量子线路如下。我们从一个 n 量子比特的输入态 $|x_0 x_1 \cdots x_{n-1}\rangle$ 开始。

➤ 1. 在第 1 个 Qubit 上应用 Hadamard 门后，量子态从输入态变换为：

$$H_1 |x_0 x_1 \cdots x_{n-1}\rangle = \frac{1}{\sqrt{2}} \left(|0\rangle + \exp\left(\frac{2\pi i}{2} x_0\right) |1\rangle \right) \otimes |x_1 \cdots x_{n-1}\rangle$$

➤ 2. 在第 1 个 Qubit 上应用受第 2 个 Qubit 控制的 $P(\theta)$ 门后，量子态变换为：

$$\frac{1}{\sqrt{2}} \left(|0\rangle + \exp\left(\frac{2\pi i}{2^2} x_1 + \frac{2\pi i}{2} x_0\right) |1\rangle \right) \otimes |x_1 \cdots x_{n-1}\rangle$$

➤ 3. 在第 1 个 Qubit 上应用最后一个由第 n 个 Qubit 控制的门后，量子态变为：

$$\frac{1}{\sqrt{2}} \left(|0\rangle + \exp\left(\frac{2\pi i}{2^n} x_{n-1} + \frac{2\pi i}{2^{n-1}} x_{n-2} + \cdots + \frac{2\pi i}{2^2} x_1 + \frac{2\pi i}{2} x_0\right) |1\rangle \right) \otimes |x_1 \cdots x_{n-1}\rangle$$

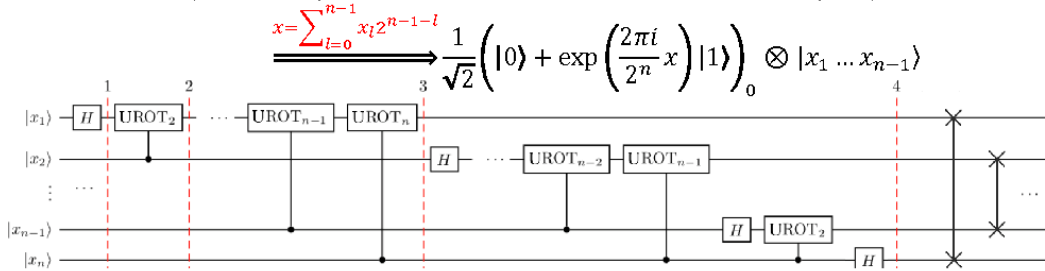


图 11.25: QFT 的线路实现步骤。Hadamard 门产生叠加，受控相位门逐位写入二进制小数相位，最后通过 SWAP 调整输出顺序。

11.6.5 量子相位估计

量子相位估计 (quantum phase estimation, QPE) 用于求么正算符本征值的相位。设

$$\hat{U}|u_a\rangle = \lambda_a|u_a\rangle, \quad |\lambda_a| = 1,$$

则可写为

$$\lambda_a = e^{i\theta_a}$$

或采用 QFT 常用规范 $\lambda_a = e^{2\pi i \phi_a}$ 。求本征值问题的关键变为估计相位 θ_a 或 ϕ_a 。

QPE 输入包含两部分：第一个寄存器有 n 个 register qubits，用于存放相位；第二个寄存器有 m 个 working qubits，制备本征态或其叠加态。初态为

$$|0\rangle^{\otimes n} \otimes |\psi\rangle.$$

第一步，对第一个寄存器施加 n 个 Hadamard 门：

$$\frac{1}{\sqrt{2^n}} \sum_{l=0}^{2^n-1} |l\rangle \otimes |\psi\rangle.$$

第二步，施加一系列受控幺正操作

$$C - U^{2^0}, C - U^{2^1}, \dots, C - U^{2^{n-1}}.$$

若 $|\psi\rangle = |u\rangle$ 且

$$U|u\rangle = e^{2\pi i \phi} |u\rangle,$$

则受控幺正作用后得到

$$\frac{1}{\sqrt{2^n}} \sum_{l=0}^{2^n-1} e^{2\pi i \phi l} |l\rangle \otimes |u\rangle.$$

这正是寄存器态上的傅里叶形式。

若相位有 n 位二进制展开

$$\phi = 0.\phi_0\phi_1 \dots \phi_{n-1},$$

则

$$2^n \phi = \phi_0\phi_1 \dots \phi_{n-1}$$

对应一个 n 位二进制整数。对第一个寄存器施加逆 QFT 后得到

$$|2^n \phi\rangle \otimes |u\rangle.$$

测量第一个寄存器即可得到相位的二进制近似。

Fudan University

1. Register Qubits + Working Qubits

2. Entangle Register and Working Qubits – Sending Physical Message to Register Qubits

3. Inverse QFT and Measurement – Get and Purify $U_k, |\psi_k\rangle$

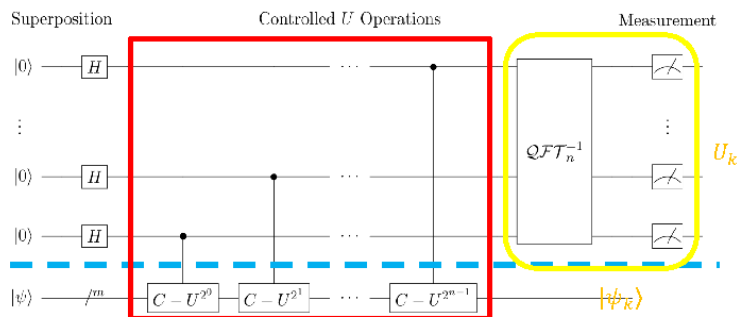


图 11.26: QPE 的三步结构：寄存器制备叠加，受控 U^{2^k} 把本征相位写入寄存器，逆 QFT 与测量读出相位。

QPE 的关键物理应用是能谱求解。若

$$U = e^{iH\Delta t},$$

且

$$H|\Psi\rangle = E|\Psi\rangle,$$

则

$$U|\Psi\rangle = e^{iE\Delta t}|\Psi\rangle = e^{i2\pi\phi}|\Psi\rangle,$$

从而

$$E = \frac{2\pi\phi}{\Delta t}.$$

因此 Hamiltonian 的本征能量可以通过么正时间演化算符的相位估计得到。

若输入 working register 是本征态叠加

$$|\psi\rangle = \sum_k \alpha_k |\psi_k\rangle,$$

则 QPE 过程给出

$$\begin{aligned} \frac{1}{\sqrt{2^n}}(|0\rangle + |1\rangle)^{\otimes n} \otimes \sum_k \alpha_k |\psi_k\rangle &\longrightarrow \sum_k \alpha_k \frac{1}{\sqrt{2^n}} \sum_{l=0}^{2^n-1} e^{i2\pi\theta_k l} |l\rangle \otimes |\psi_k\rangle \\ &\xrightarrow{\text{QFT}^{-1}} \sum_k \alpha_k |\theta_k\rangle \otimes |\psi_k\rangle. \end{aligned}$$

测量第一个寄存器会把第二个寄存器投影到相应本征态。这一思想可用于制备对称性恢复态或筛选能量本征态。

11.6.6 SBSR 与粒子数恢复

Symmetry breaking-symmetry restoration (SBSR) 把经典平均场思想与量子相位估计结合起来。先构造一个破缺对称性的简单 ansatz, 再用 QPE 或投影操作恢复守恒量。

一个量子 BCS 型 ansatz 可写为

$$|\psi(\vec{\theta})\rangle = \prod_{i=0}^{N-1} (\sin \theta_i |0_i\rangle + \cos \theta_i |1_i\rangle) = \prod_{i=0}^{N-1} R_Y^i(\pi - 2\theta_i) \bigotimes_{i=0}^{N-1} |0_i\rangle.$$

这里 R_Y 是绕 y 轴旋转门。粒子数算符对应的么正操作可写为

$$\hat{V} = \exp\left(\frac{2\pi i \hat{N}_q}{2^n}\right) = \prod_{i=0}^{N-1} \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/2^{n_q-1}} \end{pmatrix}.$$

通过对 $\hat{V}$ 做 QPE, 可以读取粒子数相位, 从而把破缺粒子数的 BCS 态投影到固定粒子数空间。其优点是 ansatz 简单、粒子数恢复清晰; 困难在于 QPE 需要受控么正和较深线路。

11.7 计算挑战与第一性原理核结构量子计算

11.7.1 量子多体计算挑战

现代计算物理面临的挑战包括:

- 第一性原理量子多体计算, 例如 $A \leq 12$ 的轻核结构;
- 格点 QCD 计算, 例如空间尺度 $L \sim 4 \text{ fm}$ 、格距约 0.1 fm 的格点;
- 用量子系统模拟另一个量子系统, 从而绕开经典 Hilbert 空间指数增长;
- 为 NISQ 或容错量子计算机发展新的量子算法。

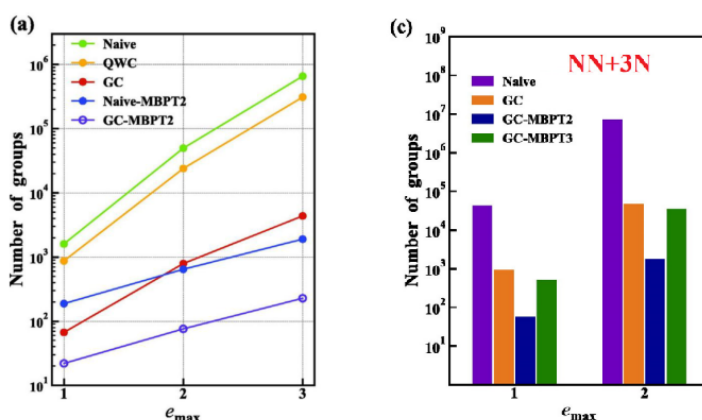
这些问题共同说明: 硬件发展会改变可求解问题的边界, 但算法设计仍然决定能否把硬件能力转化为科学结果。

11.7.2 第一性原理核结构的量子计算

第一性原理核结构计算可从 NN+3N 手征核力出发，在 no-core shell model (NCSM) 空间中描述轻核。例如 ^3H 的量子计算模拟可使用 16 个有噪声量子比特。当前变分量子线路 (variational quantum circuit, VQC) 的主要瓶颈是：Hamiltonian 中 Pauli 项很多，逐项测量非常耗时；同时有噪声量子门会降低测量精度。

一种改进策略是基于 Pauli 字符串的对易关系进行 grouped commuting (GC) 同时分组测量。若多个观测量两两对易，它们可在同一测量基下估计，从而把测量时间降低 2-3 个数量级。对核多体 Hamiltonian 来说，这种测量优化往往比单纯压缩线路同样重要。

- 变分量子计算瓶颈：算符测量极其耗时，含噪音的量子比特门
- NN+3N手征核力，NCSM无核芯壳模型模拟计算 ^3H ，16个含噪音的量子比特
- 基于算符对易关系采用GC同时分组测量，显著减少测量时间（2~3个量级）



C.J. Jiang, J.C. Pei, R.Z. Hu, S.L. Jin, H.Y. Shang, S.Q. Fan, F.R. Xu, Science Bulletin 71, 1598(2026)

图 11.27: 第一性原理核结构量子计算示意。基于对易关系的分组测量可显著降低 VQC 中 Hamiltonian 期望值估计成本。

噪声还会使模型空间受到 2^N 个组态扩散污染，并破坏粒子数、角动量投影等守恒量。对称性投影测量，尤其是粒子数恢复，可以显著改善测量准确性。这与前面的 SBSR 思想一致：先用较浅线路构造表达能力较强的试探态，再用物理守恒律约束可接受的测量结果。

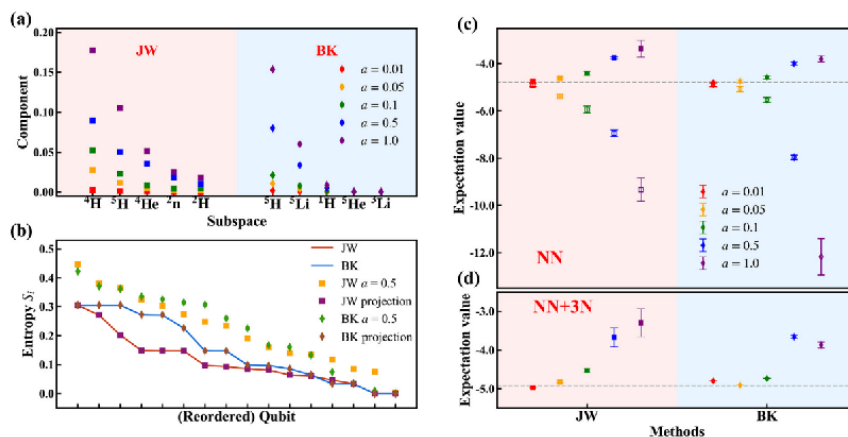
11.8 本章总结

结论 本章把超级计算与量子计算放在计算物理的同一逻辑链条中理解。传统超级计算依靠多节点、多核、GPU 和高速互连提升浮点吞吐，但通信、能耗和并行效率成为越来越突出的限制；量子计算试图利用叠加、纠缠和相位干涉处理量子多体问题，但当前仍受量子比特规模、门错误率和退相干时间限制。计算物理的前沿不是单纯等待硬件变快，而是在新硬件条件下重新设计离散化、线性代数、随机采样、变分优化和量子算法。

从整门课程的角度看，计算物理方法可以概括为以下主线：

- 线性代数：LU 分解、矩阵分解、共轭梯度法和松弛法；
- 插值与数值分析：Lagrange 插值、数值积分、Gaussian quadrature、数值微分和 cubic spline；
- 非线性与优化：Newton 法、准 Newton 法、优化方法和 simulated annealing；
- 本征值问题：Householder 变换、QR、SVD、power method 和 Krylov 方法；
- Monte Carlo：MC 积分、importance sampling、random walk、Metropolis 与 MCMC；
- 常微分方程：Euler、Runge-Kutta、稳定性、初值问题和边值问题；

- 噪音导致model space受到 2^N 个所有组态的扩散污染，守恒量遭到破坏
- 采用对称性投影测量，特别是粒子数恢复，极大改善了测量精度



C.J. Jiang, J.C. Pei, R.Z. Hu, S.L. Jin, H.Y. Shang, S.Q. Fan, F.R. Xu, *Science Bulletin* 71, 1598(2026)

图 11.28: 噪声导致的模型空间污染与对称性投影修正。物理守恒律可作为量子计算误差缓解的重要先验。

- 偏微分方程：多维有限差分、边界条件、谱方法、有限元和自适应网格；
- 动力学方程：经典动力学、量子动力学、传播子、Chebyshev 展开和分子动力学；
- 机器学习：Bayesian neural network、regression、backpropagation 和 AI for Science；
- 高性能与新型计算：parallel programming、GPU、quantum computing basics、VQE、QFT 和 QPE。

计算机与物理的关系具有双向性：硬件能力决定一个时代能够处理的模型复杂度，反过来物理问题又不断推动算法和硬件发展。传统算法多建立在离散化和数学变换上，非传统方法则包括 Monte Carlo 统计、神经网络和量子计算。实际科研不仅要求算法“能跑”，还要求 robust、reliable 和 accurate；这需要误差分析、稳定性分析、可复现实现和对硬件特性的理解。

可继续参考的经典资料包括 J. Franklin 的计算物理教材、J. M. Thijssen 的 *Computational Physics*、*Numerical Recipes in C++*、Burden 与 Faires 的 *Numerical Analysis*、Morten Hjorth-Jensen 的开放课程资料，以及相关前沿论文和技术文档。